

Learning Java

Java Developer Class

မင်းလွင်

Preview 6
Program ဆိုတာဘာလဲ 6
Java ဆိုတာဘာလဲ 7
Object Oriented Programming 7
Strong Type Programming 8
Java ဟာဘယ်လို ပေါ်ပေါက်လာခဲ့သလဲ 8
Java နဲ့ ဘာတွေရေးလို့ရလဲ 8
Java Standard Edition (Java SE) 9
Java Enterprise Edition (Java EE) 9
Java Micro Edition (Java ME) 9
Java ကိုရေးဖို့ဘာတွေလိုအပ်လဲ 9
ဒီစာအုပ်နဲ့ ဘာတွေကိုလေ့လာနိုင်မလဲ 10
References 11
Language Fundamentals 12
What is Class 12
Anatomy of a Class 13
Minimum Requirements 13
Class Declaration 14
Class Members 15
Visibility and Accessibility 35
Packages 35
Access Modifiers 40
Java Data Types 44
Primitive Data Types 44
Reference Data Types 51
Wrapper Classes of Primitives 54
Arrays 59
ဘာလို့ Array တွေကိုအသုံးပြုဖို့လိုအပ်ခဲ့တာလဲ 59
Declaring Arrays 60
Accessing Array Elements 62
Multi Dimension Arrays 64
Operators 68
Assignment Operator 69

Arithmetic Operators 71
Relational Operator 75
Logical Operators 78
Statements 82
Declaration Statements 83
Expression Statements 84
Control Flow Statements 85
Object Oriented Programming 99
About OOP 100
Encapsulation 102
Encapsulation မဖြစ်တဲ့အတွက် ဖြစ်တတ်တဲ့ပြဿနာ 102
Encapsulation ဖြစ်အောင် ဘယ်လိုရေးမလဲ 103
Encapsulation ကြောင့် ရနိုင်တဲ့ အကျိုးများ 107
Inheritance 108
Inheritance ဆိုတာဘာလဲ 108
Inheritance လုပ်လို့ရတာတွေကဘာတွေလဲ 110
Usage of Inheritance 114
Composition 115
Using Composition 116
Abstraction 117
ဘာကြောင့် Abstraction ကို အသုံးပြုရတာလဲ 117
Abstraction ဖြစ်အောင်ဘယ်လိုရေးမလဲ 119
Abstraction ကို သုံးတဲ့အတွက် ဘာဖြစ်မလဲ 124
Polymorphism 125
Object တစ်ခုကို Reference အမျိုးမျိုးကနေ Reference လုပ်နေနိုင်ခြင်း 125
Override ကြောင့် Object ပုံစံအမျိုးမျိုးရေးသားနိုင်ခြင်း 126
မှန်ကန်သော Override Rules များ 130
Override နှင့် Overload 132
Polymorphism ရဲ့ အကျိုးများ 135
Java 5 Features 139
Annotation 140
Using Annotations 141
Writing Annotation 145

Enum 152
Before Enum 152
After Enum 153
What can we do with Enum? 154
Generics 169
Before Generics 169
After Generics 173
Writing Generics 177
Using Generics Type Objects 186
How Generics Work! 192
Generics မှာ လုပ်လို့မရသော အရာများ 199
Essential Java API 205
Exceptions Handling 206
What is Exception? 207
Types of Exception 210
Handling Exceptions 211
Custom Exception 218
Benefit of Exception 220
Method Overriding 221
Using Strings 222
About String 222
Manipulating Strings 227
Composing String 234
Formatting Strings 237
Dates & Calendar 255
Creation Of Date 255
Interpretation of Calendar Values 258
Calculating Date & Time 262
Formatting Date 263
Date & Times API 265
Why Java need new Date API? 265
Types of Dates 265
Usages of Date and Times 265
Collections Framework 266
Collection Interface 266

Set Interface 268
List Interface 271
Utility Classes for Collection 275
Java IO 276
java.io.File 277
IO Streams 283
Serialization 293
NiO2 296
Java 8 New Features 297
Lambda Expression 298
Lambda Expression ဆိုတာဘာလဲ 298
ဘာ့ကြောင့် Lambda Expression ကို လိုအပ်ခဲ့ရတာလဲ 298
Lambda Expression ကို ဘယ်လို ရေးသားရသလဲ 298
Lambda Expression ကြောင့် ဘာတွေ ကောင်းသွားလဲ 298
ဘယ်လိုနေရာတွေမှာ Lambda Expression ကို အသုံးပြုသင့်သလဲ 298
Java ဟာ Functional လား OOP လား 298
Method References 299
Changes in Interfaces 300
Streams API 301
Java 9 New Features 302
JShell 302
Modularity In Java 302
Changes in Streams API 302
HTTP 2 Client 302

Preview

Programming တစ်ခုကို လေ့လာမယ်ဆိုရင် အရင်ဆုံးကြုံတွေ့ရမှာက ဘယ်ကနေ ဘယ်လို စပြီး လေ့လာသင့်တယ်ဆိုတာကို ဝေခွဲရခက်တဲ့အပိုင်းပဲဖြစ်ပါတယ်။ အထူးသဖြင့် Java ကို လေ့လာရင် ဒီပြဿနာကို ပိုပြီး ကြုံတွေ့ရမှာပါ။ ဘာလို့လဲဆိုတော့ Java နဲ့ ရေးသားလို့ရတဲ့အပိုင်းတွေ အများကြီးရှိတဲ့အတွက် စာအုပ်တွေလဲ အများကြီးရှိသလို၊ Framework တွေလဲ အများကြီးပဲ။ အဲ့ဒီတော့ ဘာပြီးရင် ဘယ်လို့လေ့လာရမယ်ဆိုတာ အမတန် အခက်တွေ့မည့်အရာတွေ ဖြစ်ပါတယ်။



ဒီစာအုပ်ဟာ Programming ဆိုတာ ဘာမှန်းကိုမသိတဲ့ သူတစ်ယောက်က Java ကို လေ့လာဖို့လိုလာပြီဆိုရင် အကူအညီပေးနိုင်တဲ့ စာအုပ်တစ်အုပ် အဖြစ် အသုံးပြုနိုင်ရန် ရည်ရွယ်ပြီး ရေးသားပါမယ်။ ဒီစာအုပ်ကို လေ့လာခြင်းအားဖြင့် Java Programming နဲ့ ပတ်သက်တဲ့ အခြေခံတွေကို လေ့လာသွားနိုင်ပါလိမ့်မယ်။

ဒီအခန်းထဲမှာတော့ အောက်ပါ အချက်တွေကို ရေးသားသွားပါမယ်။

- Program ဆိုတာဘာလဲ
- Java ဆိုတာဘာလဲ
- Java နဲ့ ဘာတွေ ရေးလို့ရလဲ
- Java ကို ရေးဖို့ဘာတွေလိုအပ်လဲ
- ဒီစာအုပ်နဲ့ ဘာတွေလေ့လာနိုင်မှာလဲ

Program ဆိုတာဘာလဲ

IT နဲ့ပတ်သက်တဲ့ အရာတွေကို လေ့လာဖူးသူတွေ အတွက်ကတော့ မထူးဆန်းပေမဲ့ ဒီလောကထဲက လူတွေ မဟုတ်ရင်တော့ မသိသေးတာဖြစ်နိုင်ပါတယ်။ Program ဆိုတာက အစီအစဉ်ပါ။ ဒါပေမဲ့ ဒီနေရာမှာ ပြောနေတာက Computer Program တစ်ခုကိုပြောတာပါ။ Computer ကို ဘာတွေလုပ်ရမယ်လို ခိုင်းစေနိုင်တဲ့ အမိန့်ပေးချက်တွေကို စုစည်းရေးသားထားတဲ့ အစီအစဉ်တစ်မျိုး ဖြစ်ပါမယ်။

အလွယ်တကူပြောရမယ်ဆိုရင် Web Browser တစ်ခုဟာလဲ Program တစ်ခုဖြစ်ပါတယ်။ Desktop ပေါ်မှာရှိတဲ့ Firefox Icon ကို Duoble Click လုပ်လိုက်ရင် Computer ကနေ Firefox Browser ကို ဖွင့်ပေးပါမယ်။ အဲ့ဒီလို ဖြစ်ရတာကလဲ ဒီလိုလုပ်လာရင် Browser ကို ဖွင့်ပေးဖို့ Computer ကို အမိန့်ပေးချက်တွေ ရေးသားထားလို့ဖြစ်တယ်။

တဖန် Search Box ထဲကနေ ရှာဖွေချင်တဲ့ Keyword ကို ရိုက်ပြီး Enter Key ကို ရိုက်လိုက်ရင် သက်ဆိုင်ရာ Search Engine ကို သုံးပြီး ရှာဖွေတွေ့ရှိချက်တွေကို Bowser ပေါ်မှာ လာပြပေးမှာ ဖြစ်ပါတယ်။ ဒါလဲ Internet ထဲမှာ ရှိတဲ့ Web Resources တွေကို ရှာဖွေပြီး Browser ပေါ်မှာ လာပြီး ပြသနိုင်အောင် အမိန့်ပေးစာတွေကို ရေးသား ထားလို့ဖြစ်ပါတယ်။

User ဆီကနေလာတဲ့ Input တွေပေါ်မှာ မူတည်ပြီး Computer ကို ဘယ်လိုလုပ်ပေးရမယ်ဆိုတဲ့ အစီအစဉ်တွေကို ရေးသားထားတာက Program ဖြစ်တယ်လို့ အလွယ်တကူမှတ်သားနိုင်မယ်ထင်ပါတယ်။

ဒါဆိုရင် Computer ကို အမိန့်ပေးနိုင်အောင် ဘယ်လိုရေးရမလဲဆိုတာ မေးစရာဖြစ်လာပါမယ်။ အဲ့ဒါဆိုရင် Computer တွေက ဘယ်လိုရေးထားတာတွေကို နားလည်သလဲ ဆိုတာကို တွေးမိမှာ ဖြစ်ပါတယ်။ Computer တွေဟာ Machine တွေဖြစ်ကြပါတယ်။

အခြေခံအားဖြင့်တော့ User နဲ့ Interact လုပ်ဖို့ Input / Output Device တွေရှိမယ်။ ပြီးရင် အတွင်းပိုင်းမှာ အလုပ်လုပ်ဖို့အတွက် CPU တို့ RAM တို့ရှိမယ်၊ အချက်အလက်တွေကို သိမ်းထားနိုင်ဖို့ HDD တွေပါရှိကြပါမယ်။

Program တစ်ခုဟာ အလုပ်လုပ်နေပြီ ဆိုရင် အဲ့ဒီ အမိန့်တွေဟာ RAM အပေါ်ကို ရောက်ရှိလာပြီး User Input တွေအပေါ်မှာ မူတည်ပြီး ဘယ်လိုလုပ်ရမယ်ဆိုတာကို CPU ကနေတဆင့် သက်ဆိုင်ရာ Device တွေကို Handle လုပ်ပြီး ခိုင်းစေပေးမှာ ဖြစ်ပါတယ်။

CPU တွေဟာ Machine Language တွေဖြစ်တဲ့ Binary Code တွေကိုသာ နားလည်တာဖြစ်ပါတယ်။ အဲ့ဒီအတွက် CPU တွေကို ခိုင်းစေမယ်ဆိုရင်တော့ Machine Language တွေနဲ့ ခိုင်းစေရမှာ ဖြစ်ပါတယ်။ အဲ့ဒီလို လူက နားလည်ရခက်ခဲတဲ့ ရေးသားထားတဲ့ Programming Language တွေကို Low Level Language တွေလို့ ခေါ်ပါတယ်။

အဲ့ဒီလို Machine Language တွေကိုရေးသားရတာ အမတန်ခက်ခဲပါတယ်။ ဒါကြောင့် နောက်ပိုင်းမှာ လူကနားလည်လွယ်တဲ့ Program တွေကို ရေးသားပြီး၊ အဲ့ဒါတွေကို ဘာသာပြန်ပေးနိုင်တဲ့ Interpreter တွေ၊ လူနားလည်လွယ်တဲ့ ရေးသားနည်းနဲ့ ရေးသားထားတဲ့ Program တွေကို Machine Language တွေအဖြစ် Compile လုပ်ပေးနိုင်တဲ့ Language တွေ ပေါ်ပေါက်လာခဲ့ပါတယ်။

အဲ့ဒီလို Programming Language တွေကိုတော့ High Level Language တွေလို့ခေါ်ကြပါတယ်။ Java ဟာလဲ လူကနားလည်လွယ်တဲ့ ရေးသားပုံတွေနဲ့ ရေးသားနိုင်တဲ့ High Level Programming Language တစ်မျိုးဖြစ်ပါတယ်။

Java ဆိုတာဘာလဲ

Java ဆိုတာ လူကနားလည်ရလွယ်တဲ့ ရေးသားပုံနဲ့ ရေးသားထားတဲ့ High Level Programming Language အမျိုးအစားတစ်ခုဖြစ်ပါတယ်။ နောက်ပြီး Object Oriented Programming (OOP) ကို အခြေခံထားတဲ့ Programming Language တစ်ခုလဲ ဖြစ်ပါတယ်။ ပြီးတော့ Strong Type Programming Language လို့လဲ ပြောလေ့ရှိပါတယ်။ ပြီးတော့ Java Programming ဆိုတာ ဘယ်လို ပေါ်ပေါက်လာခဲ့ရသလဲ ဆိုတာကို ဖော်ပြသွားပါမယ်။

Object Oriented Programming

Object Oriented Programming ကို OOP လို့လဲခေါ်ကြပြီး Program ထဲမှာပါတဲ့ Programming Unit တွေကို Object တွေလို့မြင်ပြီး ရေးသားရတဲ့ Programming ရေးသားပုံ ရေးသားနည်း Concept တစ်ခုဖြစ်ပါတယ်။

ပုံမှန်အားဖြင့်တော Program တစ်ခုမှာ Data တွေနဲ့ Function တွေပါဝင်တယ်လို့ မှတ်နိုင်မှာ ဖြစ်တယ်။ ဒါပေမဲ့ OOP မှာတော့ Data တွေ Function တွေလို့ မမြင်ပဲ Object တွေလို့ သတ်မှတ်ပြီး အသုံးပြုပါတယ်။ ပြီးတော့ အဲ့ဒီ Object တွေအချင်းချင်း Communicate လုပ်ခြင်းအားဖြင့် မိမိလိုအပ်တဲ့ Application ကို ဖြစ်အောင် ရေးသားသွားတာပဲ ဖြစ်ပါတယ်။

အပြင်လောကမှာရှိတဲ့ Object တွေကို Model လုပ်ပြီးရေးသာရတဲ့အတွက် နားလည်ရလွယ်ကူစေပြီး တစ်ခါရေးပြီးသား Object တွေကို တစ်ခြား နေရာတွေမှာလဲ ပြန်ပြီး အသုံးပြုနိုင်မှာ ဖြစ်တဲ့အတွက် Reusable ကိုလဲ တိုးတက်စေနိုင်ပါတယ်။ ဒဲ့ကြောင့် လက်ရှိ Programming Language အတော်များများဟာလဲ OOP ကို အခြေခံထားကြတာများပါတယ်။ Java Programming ဟာလဲ OOP ကို အခြေခံထားတဲ့ Programming Language တစ်ခုဖြစ်ပါတယ်။

နောက်ပိုင်းအခန်းတွေမှာ OOP နဲ့ ပတ်သက်ပြီး အသေးစိတ် ဖော်ပြသွားပါမယ်။

Strong Type Programming

Programming အမျိုးအစားတွေကို အသုံးပြုနေတဲ့ Data တွေရဲ့ Type အပေါ်မူတည်ပြီး ခွဲခြားလေ့လဲရှိကြပါတယ်။ Java Programming ဟာ Data တွေရဲ့ Type ကို တိတိကျကျ သတ်မှတ်ပေးရန်လိုအပ်ပါတယ်။ အသုံးပြုတော့မယ်ဆိုရင် ဘယ်လို Type မျိုးလဲ ဆိုတာကို ဖေါ်ပြရေးသားရမှာ ဖြစ်ပါတယ်။ ထိုကဲ့သို့ Type ကို အတိအကျသတ်မှတ်ဖေါ်ပြရန်လိုအပ်သော Programming အမျိုးအစားမျိုးကို Strong Type Programming လို့ခေါ်ပါတယ်။

တစ်ချို့သော Language တွေက Type ကို အထွေအထူးသတ်မှတ်ရေးသားစရာမလိုအပ်ပါဘူး။ အလုပ်လုပ်နေစဉ်အခါမှ Runtime ထဲမှာ Type ကို သတ်မှတ်ပေးသွားနိုင်အောင်ပြင်ဆင်ထားတဲ့ အတွက်ဖြစ်ပါတယ်။ အဲ့ဒီလို Language မျိုးကို Dynamic Type Programming လို့ခေါ်လေ့ရှိပါတယ်။ PHP လို javascript လို Language တွေဟာ Dynamic Type Programming တွေ ဖြစ်ကြပါတယ်။

Strong Type နဲ့ Dynamic Type ဘယ်ဟာ ပိုကောင်းလဲလို့ အငြင်းပွားစရာရှိပါတယ်။ ဘယ်အရာမဆို ကောင်းတာတွေလဲရှိသလို မကောင်းတဲ့အချက်တွေလဲ ရှိနေမြဲပါပဲ။ Dynamic Type တွေဟာ Type တွေကို သတ်မှတ်စရာမလိုတဲ့အတွက် ရေးသာရလွယ်ကူတတ်တဲ့ အချက်ရှိပါတယ်။ ဒါပေမဲ့ Type မှားပြီး သုံးမိရင် Runtime ကျမှ သိရလေ့ရှိတဲ့အတွက် Release လုပ်ပြီးမှ မှားမှန်းသိနိုင်တဲ့ အားနည်းချက်ရှိပါတယ်။

Strong Type တွေဟာ ရေးသားရခက်ခဲတယ်ဆိုပေမဲ့ Compile Time မှာကထဲက Type မှားနေတာကို သိနိုင်တဲ့အတွက် Runtime အချိန်ကျမှ Type မှားတယ်ဆိုတာ ဖြစ်စရာမရှိပါဘူး။ အဲ့ဒီလို Type ကို အသေအချာ သတ်မှတ်ပြီး အသုံးပြုရတဲ့အတွက် အလုပ်လုပ်နေချိန်မှာ Memory ကို ဘယ်လောက် သုံးရမယ်ဆိုတာကို အတိအကျတွက်ချက်ပေးနိုင်တဲ့အတွက် Performance မှာ ပိုကောင်းမွန်စေနိုင်ပါတယ်။

Java ဟာဘယ်လို ပေါ်ပေါက်လာခဲ့သလဲ

Java Programming Language ကိုကော Java Platform ကိုပါ 1990 ကနေ Sun Microsystem ကုမ္ပဏီမှ တီထွင်ခဲ့တာဖြစ်ပါတယ်။ Low Power Electric Device တွေမှာ အသုံးပြုမည့် C++ Library ကို တည်ထွင်ရန် ရည်ရွယ်ပြီး စတင်ခဲ့ကြတာဖြစ်ပါတယ်။

ဒါပေမဲ့ အဲ့ဒီလိုရေးသားရင်း C++ ဟာ Electric Power ကို အမတန်လိုအပ်တဲ့အတွက် OOP ကို အခြေခံတဲ့ နောက်ထပ် Language အသစ်တစ်ခု ရေးသားခဲ့ကြခြင်းဖြစ်ပါတယ်။ ဒါ့ကြောင့် Java ရဲ့ ရေးသားပုံအတော်များများဟာ Cညည နဲ့ ဆင်တူနေကြတာဖြစ်ပါတယ်။ Java ကို စတင်ရေးသားစဉ်က OAK လို့ခေါ်ဆိုခဲ့ကြပေမဲ့ OAK ဆိုတဲ့ Product ဟာ အခြားတနေရာမှာ အသုံးပြုနေတာရှိတဲ့အတွက် Java လို့ပြောင်းပြီး ခေါ်ဆိုခဲ့ကြတာဖြစ်ပါတယ်။

1991 ခုနှစ်ကထဲက OAK ဆိုတဲ့အမည်နဲ့ စတင်ခဲ့တဲ့ Java ဟာ 1996 ခုနှစ်မှာ တရားဝင် Release လုပ်လာနိုင်ခဲ့ပါတယ်။ လက်ရှိ 2017 ခု 2 လပိုင်းအချိန်မှာတော့ Java Version ဟာ 1.8 ဖြစ်ပြီး ဒီနှစ်ထဲမှာပဲ 1.9 ကို Release လုပ်မယ်လို့ ကြော်ငြာထားပါတယ်။

Java နဲ့ ဘာတွေရေးလို့ရလဲ

Java ကို စတင်လေ့လာသူတိုင်း ဒီအချက်ကို သိချင်နေကြမှာ ဖြစ်တယ်။ Java ကို မိမိအသုံးပြုလိုတဲ့ ပတ်ဝန်းကျင်အလိုက် Edition တွေကို ၃ မျိုးခွဲခြားထားပါတယ်။

Java Standard Edition (Java SE)

Java SE လို့လဲ ခေါ်လေ့ရှိကြပြီး၊ Java Programming ရဲ့ အခြေခံ Edition တစ်ခုဖြစ်ပါတယ်။ Java Programming ကို လေ့လာမယ်ဆိုရင် မဖြစ်မနေ Java SE ကို လေ့လာရမှာ ဖြစ်ပါတယ်။ အဲ့ဒီ အထဲမှာ Java Programming ကို ဘယ်လို ရေးသားရမယ်ဆိုတဲ့အချက်တွေ Desktop Application တစ်ခုကို ရေးသားမယ်ဆိုရင် လိုအပ်တဲ့ အခြေခံ API တွေကို စုစည်းပေးထားပါတယ်။

Java SE ကိုလေ့လာပြီးရင် Desktop Application တစ်ခုကို ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။ ဒီစာအုပ်ထဲမှာတော့ Java SE 8 ကို အခြေခံပြီး ရေးသားထားပါတယ်။

Java Enterprise Edition (Java EE)

Java EE လို့လဲ ခေါ်လေ့ရှိပါတယ်။ Enterprise Application တစ်ခုကို ရေးသားရန်အတွက်လိုအပ်တဲ့ Service များကို Server အပေါ်မှာ စုစည်းပေးထားပြီး အလွယ်တကူ အသုံးပြုနိုင်ရန် ပြင်ဆင်ပေးထားတဲ့ စုပေါင်းပတ်ဝန်းကျင်လဲ ဖြစ်ပါတယ်။

Enterprise Application တစ်ခုရဲ့ အခြေခံလိုအပ်ချက်တွေ ဖြစ်ကြတဲ့ Security, Transaction, Concurrency, Messaging, Remote Access, Web Services အစရှိတဲ့ Service များကို Server အပေါ်မှာ ထည့်သွင်းပေးထားပြီး Application များထဲမှနေ အလွယ်တကူအသုံးပြုနိုင်အောင် ပြင်ဆင်ထားပါတယ်။

Java EE ကို လေ့လာပြီး Web Application တွေ Cooperation တွေမှာ သုံးမည့် Enterprise Application တွေကို ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။ Java EE ကို လေ့လာမယ်ဆိုရင် အရင်ဆုံး Java SE ကို လေ့လာထားမှ ဖြစ်ပါမယ်။

Java Micro Edition (Java ME)

Java ME လို့လဲခေါ်ဆိုလေ့ရှိပြီး Embedded System တွေရဲ့ Application တွေကိုရေးသားနိုင်ဖို့ပြင်ထားတဲ့ Edition လဲ ဖြစ်ပါတယ်။ လျှပ်စစ်မီးဖိုတို့၊ TV - Video Player တို့၊ Car Computer Box တို့လို Embedded System တို့မှာ ရေးသားဖို့အတွက် ပြင်ဆင်ထားတဲ့ Edition လဲဖြစ်ပါတယ်။ Java ME ကို ရေးသားမယ်ဆိုရင်လဲ Java SE ကို သိရှိပြီးမှသာ ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

Android မပေါ်ခင် ခေတ်စားခဲ့တဲ့ Java Phone ရဲ့ Application တွေကို ရေးသားမယ်ဆိုရင်လဲ Java ME ကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။

တဖန် Android ဟာလဲ Android Platform ပေါ်မှာ အလုပ်လုပ်တာဆိုပေမဲ့ ရေးသားထားတဲ့ Language နေရာမှာ Java ကို အသုံးပြုထားတာ ဖြစ်တဲ့အတွက် Android Application တွေကို ရေးသားမယ်ဆိုရင်လဲ Java SE ကို ပိုင်ပိုင်နိုင်နိုင်သိရှိထားဖို့လိုအပ်ပါတယ်။

ဒါကြောင့် Java SE ကို လေ့လာထားရင် Desktop Application မျိုးကို ရေးသားနိုင်မှာဖြစ်ပြီး Web Application တွေ Enterprise Application တွေ၊ Embedded Application တွေနဲ့ Android Application တို့ကို ဆက်လက်လေ့လာနိုင်စေရန် အထောက်အကူပြုမှာ ဖြစ်ပါတယ်။

Java ကိုရေးဖို့ဘာတွေလိုအပ်လဲ

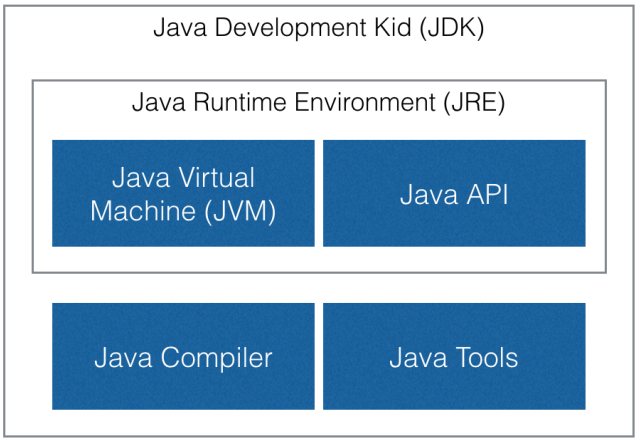
Java ကိုရေးဖို့ဘာတွေလိုအပ်သလဲ ဆိုတဲ့အချက်ကို မသွားသေးခင် Java Program ကို ဘယ်လိုရေးပြီး ဘယ်လို အလုပ်လုပ်စေတာလဲ ဆိုတာကို အရင်လေ့လာသွားပါမယ်။

အရင်ဆုံး Java Program တစ်ခုကိုရေးသားမယ်ဆိုင် Text File အနေနဲ့ပဲ ရေးသားရမှာ ဖြစ်ပါတယ်။ အဲ့ဒီ File ရဲ့ Extension ကိုတော့ java ဆိုပြီး သိမ်းပေးရပါမယ်။ ပြင်ပမှာရှိတဲ့ အရာတွေကို နမူနာပြုပြီး Java Program တွေကို ရေးသားလေ့ရှိပါတယ်။ အောက်ပါအတိုင်း Person.java ဆိုပြီး ရေးသား သိမ်းဆည်းနိုင်မှာ ဖြစ်ပါတယ်။ အဲ့ဒီ File ကိုတော့ Java Source Code File လို့ခေါ်ပါတယ်။

ရေးသားထားတဲ့ Source File တွေကို Java Compiler ကို အသုံးပြုပြီး Compile လုပ်ရပါမယ်။ Compile လုပ်ပြီးတဲ့အခါမှာ class file တွေကိုရရှိပါမယ်။ class file တွေဟာ java byte code တွေနဲ့ရေးသားထားတာဖြစ်ပြီး လူကတော့ ဖတ်ပြီး နားလည်မှာ မဟုတ်တော့ပါဘူး။

ရရှိလာတဲ့ Class File တွေကို Java Virtual Machine (JVM) တွေပေါ်မှာ အလုပ်လုပ်စေမှာ ဖြစ်ပါတယ်။ Java Class File တွေဟာ OS ပေါ်မှာ တိုက်ရိုက် အလုပ်လုပ်တာမဟုတ်ပဲ၊ JVM တွေပေါ်မှာ အလုပ်လုပ်ပြီး JVM က တဆင့် OS ကို ခိုင်းစေမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် JVM သာရှိမယ်ဆိုရင် Java Class File တွေဟာ ဘယ် OS ပေါ်မှာ မဆိုအလုပ်လုပ်မှာ ဖြစ်ပါတယ်။

ဒီအချက်ကြောင့် Java ဟာ “Write Once Run Anywhere” ဆိုပြီး Programming လောကမှာ အောင်မြင်လာခဲ့ရခြင်းလဲဖြစ်ပါတယ်။



ဆက်လက်ပြီး Java ကို ရေးသားရန်လိုအပ်တဲ့ Java Platform တွေကို လေ့လာကြည့်ကြရအောင်။ Java Program တွေကို အလုပ်လုပ်စေဖို့ JVM ဆိုတာ လိုအပ်ပါတယ်။

အဲ့ဒါအပြင် Program ရေးသားရာမှာ အလွယ်တကူ အသုံးပြုနိုင်ဖို့ Java Platform ထဲမှာ Standard API တွေကို ပြင်ဆင်ထားပါတယ်။

ဒါ့ကြောင့် Java Program တစ်ခုကို အလုပ်လုပ်စေဖို့ဆိုရင် JVM နဲ့ Standard Java API တွေလိုအပ်ပါတယ်။ အဲ့ဒါတွေကို စုစည်းပြီး Java Runtime Emvironment (JRE) လို့ခေါ်ပါတယ်။

JRE ရှိရင် ရေးထားတဲ့ Java Program တွေကို အလုပ်လုပ်စေနိုင်မှာ မှန်သော်လည်း ရေးသားထားသော Program တွေကို Compile လုပ်ဖို့ကတော့ Java Compiler ရှိမှ ဖြစ်မှာပါ။ JRE ရယ်၊ Java Compiler ရယ် အခြား အသုံးဝင်တဲ့ Tools တွေကို စုစည်းပြီး Java Development Kit (JDK) လို့ခေါ်ဆိုပါတယ်။

JRE ကော JDK ပါ Oracle ရဲ့ တရားဝင် Web Site ကနေ Download လုပ်လို့ရနိုင်ပါတယ်။

ဒီစာအုပ်နဲ့ ဘာတွေကိုလေ့လာနိုင်မလဲ

Easy Java ဟာ Java ကို စတင်လေ့လာသူတွေကို အလွယ်တကူလေ့လာနိုင်အောင် ရည်ရွယ်ပြီး ရေးသားထားတဲ့ စာအုပ်တစ်အုပ်ဖြစ်ပါတယ်။ ပါဝင်မည့် အကြောင်းအရာတွေကတော့ အောက်ပါ အတိုင်း ဖြစ်ပါတယ်။

Language Fundamentals

အဲ့ဒီအတွက် Java ကို ရေးသားရာမှာ မရှိမဖြစ်လိုအပ်တဲ့ Language ရေးသားပုံစည်းမျဉ်းတွေကို စုစည်း ဖေါ်ပြသွားပါမယ်။ Class ဆိုတာဘာလဲ။ Object ဆိုတာဘာလဲ။ ပြီးတော့ Class တစ်ခုကို ဘယ်လိုရေးရသလဲ။ ပြီး

တော့ Java Data Type တွေ Java Statement တွေကို ဘယ်လို ရေးသားရမယ်ဆိုတာကို လေ့လာနိုင်မှာ ဖြစ်ပါတယ်။

Object Oriented Programming

Java Programming Language ဟာ Object Oriented Programming (OOP) ကို အခြေခံထားတဲ့ Language တစ်ခုဖြစ်ပါတယ်။ အဲ့ဒီအတွက် Language Fundamentals ကို သိရှိပြီးတဲ့ နောက်မှာ Java မှာ အသုံးပြုထားတဲ့ OOP ရဲ့ Feature တွေဖြစ်ကြတဲ့ Inheritance, Encapsulation, Abstraction, Composition နဲ့ Polymorphism တို့အကြောင်းကို လေ့လာကြပါမယ်။ အဲ့ဒါအပြင် Object တွေကို ရေးသားတဲ့ နေရာမှာ သတိထားသင့်တဲ့ Object Design Principles တွေကိုလဲ့ လေ့လာသွားနိုင်ပါမယ်။

Essential Java API

Language Fundamentals နဲ့ OOP တို့ကို လေ့လာပြီးတဲ့အခါမှာ Java Language နဲ့ Application တွေကို ရေးသားရာမှာ အသင့် အသုံးပြုနိုင်တဲ့ Java API တွေကို လေ့လာသွားကြပါမယ်။ ဒီနေရာမှာတော့ Exception Handling, Numbers, Strings, Dates, Collections Framework, File I/O, NiO2 နဲ့ Streams API တို့ကို လေ့လာသွားကြပါမယ်။

Environment Preparation

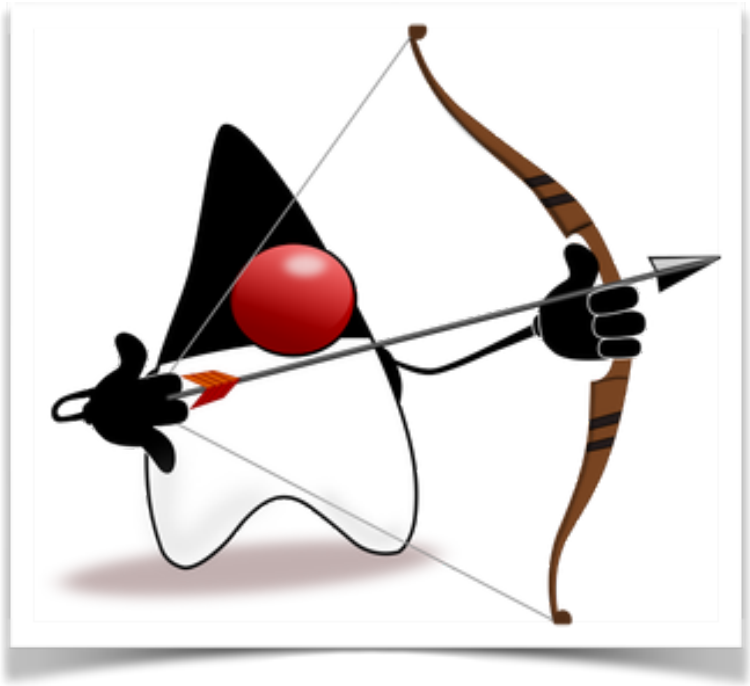
Java Programming ရေးသားဖို့အတွက် လိုအပ်တဲ့ ပတ်ဝန်းကျင် ပြင်ဆင်မှုတွေကို ဖေါ်ပြသွားပါမယ်။ ကျွန်တော်တို့ ဒီစာအုပ်မှာတော့ Eclipse IDE ကို အသုံးပြုပြီး ပတ်ဝန်းကျင်ကို ဘယ်လိုပြင်ဆင်မယ်။ ပြီးတော့ IDE ရဲ့ အသုံးဝင်တော့ Short Cut တွေရဲ့ အသုံးပြုပုံတွေ၊ Template တွေကို ကိုယ်တိုင်ရေးသား အသုံးပြုပုံတွေကို ဖေါ်ပြသွားပါမယ်။

References

ဒီစာအုပ်လို ရေးသားရာမှာ အသုံးပြုခဲ့တဲ့ References စာအုပ်တွေကတော့ အောက်ပါအတိုင်းဖြစ်ပါတယ်။

Language Fundamentals

ပတ်ဝန်းကျင် ပြင်ဆင်မှုအကြောင်းကို လေ့လာပြီးတဲ့နောက်မှာ ဆက်လက်ပြီး Java Programming ကို ဘယ်လို ရေးသားရတယ်ဆိုတဲ့ အပိုင်းကို လေ့လာသွားကြပါမယ်။ Java နဲ့ Programming ကို ရေးသားတဲ့ နေရာမှာ ရေးသားနိုင်တဲ့ Class တွေ၊ Method တွေ Variable တွေ ပြီးတော့ Enum တွေ Generics တွေရှိပါတယ်။ တဖန် Programming Logic ကို ရေးသားဖို့အတွက် လိုအပ်တဲ့ Operators တွေ ပြီးတော့ Statement လဲ ရှိပါတယ်။ ဒါ တွေကို ဒီအခန်းမှာ လေ့လာသွားနိုင်ပါတယ်။



Java Programming မှာ ဘာတွေ ရေးလို့ရ တယ်၊ ပြီးတော့ မမှားအောင် ရေးဖို့ ဘယ်လို ရေးရမယ်ဆိုတာကို ဒီနေရာမှာ သိရှိသွားစေ ချင်ပါတယ်။ ပါဝင်မည့် အကြောင်းအရာတွေ ကတော့ အောက်ပါအတိုင်း ဖြစ်ပါတယ်။

- What is A Class
- Anatomy of a Class
- Visibility and Accessibility
- Java Data Types
- Using Arrays
- Operators
- Statements

What is Class

Class ဆိုတာ Programming Unit တစ်ခုဖြစ်ပါတယ်။ လိုအပ်တဲ့ Function တွေကို ရေးသားလိုတဲ့အခါမှာလဲ Class ကို ရေးပြီး အဲ့ဒီ Class ထဲမှာ Function တွေကိုရေးသားရပါမယ်။ Java Programming မှာ Function တွေကို Method လို့ခေါ်လေ့ရှိပါတယ်။

တဖန် Class တွေဟာ Object တစ်ခုမှာပါတဲ့ State တွေနဲ့ Behavior တွေကို သတ်မှတ်ပေးနိုင်ပါတယ်။ Object ထဲမှာ ပါသင့်တဲ့ State တွေကို Variable တွေနဲ့ သတ်မှတ်ပြီး Behavior တွေကိုတော့ Method တွေနဲ့ ရေးသား ဖော်ပြရမှာ ဖြစ်ပါတယ်။

ဒါ့ကြောင့် Class တစ်ခုကို ရေးရတဲ့ ရည်ရွယ်ချက်ဟာ ၂ ခုရှိမှာ ဖြစ်ပါတယ်။ ပထမရည်ရွယ်ချက်ကတော့ Utility Function တွေကို ရေးသားလိုတဲ့အခါမှာလဲ Class ကိုရေးသားပြီးမှ Function တွေကို Static Method တွေ အနေနဲ့ ရေးသားရပါမယ်။ နောက်ရည်ရွယ်ချက်တစ်ခုကတော့ Object တွေကို တည်ဆောက်တဲ့အခါမှာ လိုအပ်တဲ့ State တွေနဲ့ Behavior တွေကို သတ်မှတ်ပေးဖို့ Class တွေကို ရေးရမှာဖြစ်ပါတယ်။

Anatomy of a Class

Java Programming ကို စပြီးရေးသားတော့မယ်ဆိုရင် Class ကနေ စပြီး ရေးသားရမှာ ဖြစ်ပါတယ်။ ကျွန်တော်တို့ ဒီနေရာမှာ Java Class တစ်ခုကို ဘယ်လို ရေးသားရမယ်ဆိုတာကို လေ့လာသွားမှာ ဖြစ်ပါတယ်။ အရင်ဆုံး Class တစ်ခုကို မမှားအောင် ဘယ်လိုရေးသားရမလဲ ဆိုတာကို လေ့လာသွားပြီး၊ Class ထဲမှာ ဘာတွေ ရေးလို့ရတယ်၊ ပြီးတော့ ဘယ်လို အသုံးပြုရတယ်ဆိုတောကို လေ့လာသွားပါမယ်။

Minimum Requirements

Class တစ်ခုကို မမှားအောင်ရေးတယ်ဆိုတာက Compile လုပ်လိုက်တဲ့အခါမှာ Error မဖြစ်အောင် ရေးနိုင်တာကို ဆိုလိုတာ ဖြစ်ပါတယ်။

```
class HelloJava {}
```

Class တစ်ခုကို ရေးသားမယ်ဆိုရင် ဖေါ်ပြပါ အတိုင်း ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။ အနဲဆုံးအောက်ပါ အချက်အလက်တွေကို ရေးသားထားရင် Compileလုပ်တဲ့ အခါမှာ Error တက်မှာ မဟုတ်ပါဘူး။

class keyword

class keyword ဟာ Class တစ်ခုဖြစ်ကြောင်းကို သတ်မှတ်ပေးနေတဲ့ keyword ဖြစ်ပါတယ်။ Class တစ်ခုကို ရေးသားရာမှာ မဖြစ်မနေပါဝင်ရမည့် အရာတစ်ခုဖြစ်ပါတယ်။ Java ဘာသာရပ်ဟာ Case Sensitive ဖြစ်တဲ့ အတွက် Lower Case တွေဖြစ်တဲ့ class လို့သာရေးသားရမှာ ဖြစ်ပါတယ်။ Class လို့ရေးသားမယ်ဆိုရင် keyword အနေနဲ့ အသိအမှတ်ပြုမှာ မဟုတ်ပါဘူး။ ပြီးတော့ CLASS လို့လဲ ရေးသားလို့မရပါဘူး။

Identifier

HelloJava ကတော့ Class Name ဖြစ်ပြီး Identifier လို့ခေါ်ပါတယ်။ Class ကို ဘယ်လိုခေါ်ဝေါ်မလဲ ဆိုတာကို သတ်မှတ်ပေးတာပါ။ အခြခံအားဖြင့်တော့ Class Name တွေကို Noun တွေကို အသုံးပြုပြီး ရေးသားလေ့ရှိပါတယ်။ Upper Case နဲ့ စပြီးရေးသားပေးရပါမယ်။ Noun တွေကို ၂ ခုတွဲပြီး ရေးသားမယ်ဆိုရင်တော့ (Hello နဲ့ Java) ဒုတိယ Noun ရဲ့ အစစာလုံးကို Upper Case နဲ့ ရေးသားသင့်ပါတယ်။ Camel Case ရေးသားပုံလို့ ခေါ်လေ့ရှိပါတယ်။

Class Name ကို Lower Case ကို သုံးလို့မရဘူးလား။ Camel Case နဲ့ မရေးလို့မရဘူးလားဆိုတော့ မရေးရင်လဲ ရပါတယ်။ Compiler Error တက်မှာ မဟုတ်ပါဘူး။ ဒါပေမဲ့ စနစ်ကျတဲ့ ရေးသားပုံတော့ မဟုတ်ပါဘူး။ Team Development လုပ်တဲ့ အခါတွေမှာ ဆိုရင် သတ်မှတ်ထားတဲ့ စည်းကမ်းတွေကို လိုက်နာပြီးရေးသားတာဟာ ကောင်းတဲ့ အကျင့်ဖြစ်ပါတယ်။

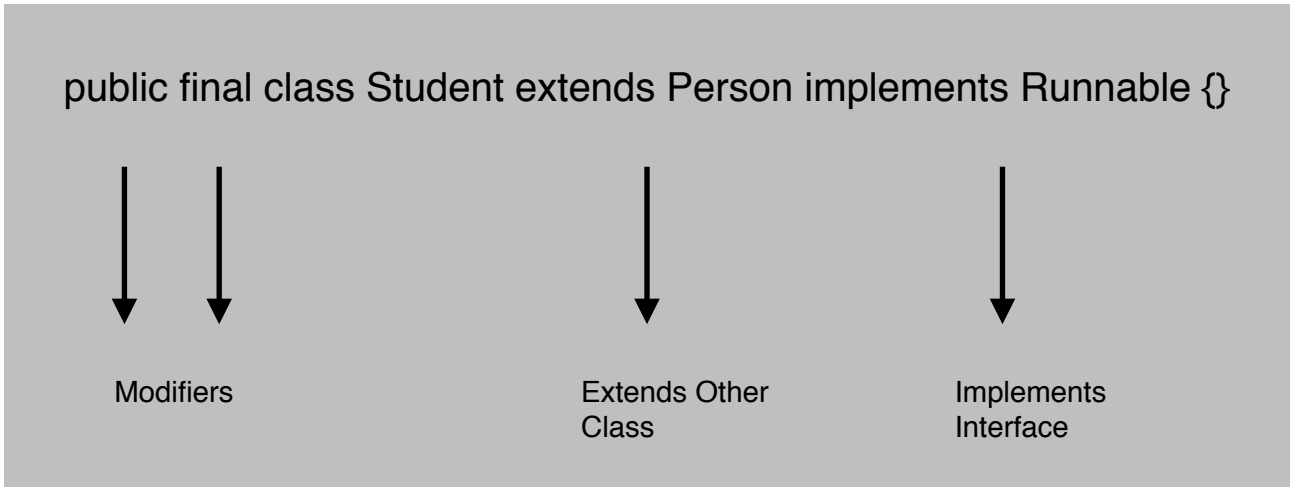
Class Name Identifier တွေကို ရေးသားတဲ့ နေရာမှာ Java ရဲ့ Keyword တွေကို အသုံးပြုလို့မရပါဘူး။ Unicode Charactor တွေ၊ Currency Code တွေ၊ Number တွေနဲ့ Under Score ကို အသုံးပြုပြီး ရေးသားနိုင်ပါတယ်။ ဒါပေမဲ့ Number တွေနဲ့ စပြီး ရေးသားလို့မရပါဘူး။

Class Body

အတွန့်ကွင်း အဖွင့်နဲ့ အပိတ်ကတော့ Class Body ဖြစ်ပါတယ်။ class တစ်ခုကို ရေးသားတဲ့ အခါမှာ Class Body ကို မဖြစ်မနေရေးသားရပါမယ်။ Class Body အတွင်းမှာတော့ Class ထဲမှာ ပါစေချင်တဲ့ အရာတွေကို ရေးသားရမှာ ဖြစ်ပါတယ်။

Class Declaration

အထက်ဖော်ပြပါအတိုင်း keyword, Class name Identifier နဲ့ Class Body တို့သာပါမယ်ဆိုရင် Java Compiler ကနေ မှန်ကန်တဲ့ Class တစ်ခုအဖြစ် အသိအမှတ်ပြုမှာ ဖြစ်ပါတယ်။ ဒီဖော်ပြပုံကတော့ Class တစ်ခုအတွက် အနဲဆုံးလိုအပ်ချက်တွေ ဖြစ်ကြပါတယ်။ ဒါပေမဲ့ Class တစ်ခုမှာ ရေးသားမယ်ဆိုရင် အခြားရေးသားနိုင်တာတွေရှိပါသေးတယ်၊ ဆက်လက်ပြီး လေ့လာသွားကြပါမယ်။



Java Class တစ်ခုမှာ ရေးလို့ရသမျှကို ရေးသားထားတဲ့ နမူနာ ပုံစံဖြစ်ပါတယ်။ ရှေ့မှာ လေ့လာလာခဲ့သလိုဆိုရင် မဖြစ်မနေရေးသားရန်လိုအပ်သော ပုံစံမှာ class Student {} သာဖြစ်ပါတယ်။ ကျန်တာတွေကတော့ လိုအပ်တဲ့အခါမှာ ထည့်ရေးသားရသော အရာများဖြစ်ကြပါတယ်။

Modifiers

အထက်ပါပုံတွင်ပါဝင်သော public နဲ့ final တို့ဟာ Modifier တွေဖြစ်ကြပါတယ်။ Modifier ဆိုတာဟာ Default အရည်အသွေးတွေကို ပြုပြင်လိုတဲ့အခါ (Modify) အသုံးပြုနိုင်သော အရာများဖြစ်ပါတယ်။ Modifier တွေကို Access Modifier နဲ့ Non Access Modifier ဆိုပြီး ၂ မျိုးခွဲခြားနိုင်ပါတယ်။ နမူနာထဲတွင်ပါဝင်သော public ဆိုတာဟာ Access Modifier တစ်ခုဖြစ်ပြီး Student Class ကို မည်သည့် နေရာမှမဆို အသုံးပြုနိုင်ကြောင်း သတ်မှတ်ပေးနေပါတယ်။

- Access Modifier နဲ့ ပတ်သက်ပြီး Modifier အခန်းနှင့် OOP ရဲ့ Encapsulation အခန်းတို့မှာ အသေးစိတ်ဖော်ပြသွားပါမယ်။

final ဆိုတာကလဲ Non Access Modifier တစ်ခုဖြစ်ပါတယ်။ final လို့ရေးသားလိုက်တာနဲ့ Student Class ဟာ final class ဖြစ်သွားပြီး Student Class ကို အခြား Class တွေကလာပြီး Extends လုပ်လို့ရတော့မှာ မဟုတ်ပါဘူး။

- Non Access Modifier တွေကိုလဲ Modifier အခန်းမှာ အသေးစိတ်ဖော်ပြသွားပါမယ်

Extends Other Class

Class တစ်ခုဟာ လိုအပ်ရင် အခြား Class တစ်ခုကို Extends ပြုလုပ်ပိုင်ခွင့်ရှိပါတယ်။ ဒီလို Extends လုပ်လိုက်ခြင်းအားဖြင့် Class တွေကြားမှာ IS A Relationship ကို ဖြစ်ပေါ်စေပြီး Super Class ထဲမှာရှိတဲ့ private ကလွဲတဲ့ Method တွေ Variable တွေကို အသုံးပြုလို့ရသွားပါမယ်။

အထက်ပါပုံမှာ Student extends Person လို့ရေးသားထားတဲ့အတွက် Student ဟာ Person Class ကို Extends လုပ်ထားတယ်လို့ ဖော်ပြထားပါတယ်။ ဒါ့ကြောင့် Student is A Person ဖြစ်သွားပါမယ်။

- OOP အခန်းမှာ အသေးစိတ်ဖော်ပြသွားပါမယ်။

Implements Interface

Class တစ်ခုဟာ Interface တွေကို Implement လုပ်နိုင်ပြီး implements keyword နောက်မှာ Implement လုပ်လိုတဲ့ Interface တွေကို ကော်မာ ခံပြီးရေးသားပေးရမှာ ဖြစ်ပါတယ်။ Interface Implementation တွေကို အမြဲတမ်း Class Extension တွေရဲ့နောက်မှာ ရေးသားရမှာ ဖြစ်ပါတယ်။ Class တစ်ခုဟာ Class တစ်ခုထဲကိုသာ Exends လုပ်ပိုင်ခွင့်ရှိပေမဲ့ Interface တွေကိုတော့ တစ်ခုထက်မက Implements လုပ်ပိုင်ခွင့်ရှိပါတယ်။

အထက်ဖော်ပြပါပုံမှာ implements Runnable လို့ရေးသားထားတာဟာ Student Class ဟာ Runnable Interface ကို Implement လုပ်ထားတယ်ဆိုတာကို ဖော်ပြနေတာ ဖြစ်ပါတယ်။

- Types of Class နှင့် OOP အခန်းမှာ အသေးစိတ်ဖော်ပြသွားပါမယ်။

Class Members

Class Body အတွင်းမှာ ရေးသားလို့ရတဲ့အရာတွေကတော့ Class Members တွေဖြစ်ပါတယ်။ ဒါဖြင့် Class Body ထဲမှာ ဘာတွေရေးလို့ရလဲ။

1. Variables
2. Methods
3. Constructors
4. Nested Classes and Interfaces
5. Nested Enums

Nested Member (No.4 , No.5)တွေနဲ့ပတ်သက်ပြီး Types of Classes မှာလဲ ဖော်ပြသွားပါမယ်။ ဒါ့ကြောင့် ဒီနေရာမှာတော့ Variables, Methods နဲ့ Constructors တွေအကြောင်းကိုပဲ အဓိကထားပြီး ဖော်ပြသွားပါမယ်။

Variables

Variable ဟာဘာလဲလို့ပြောရမယ်ဆိုရင် ကျွန်တော်တို့ အလယ်တန်းလောက်မှာ သင်ခဲ့ဘူးတဲ့ မသိကိန်း တစ်ခုနဲ့ တူပါတယ်။ မသိတဲ့ ကိန်းတစ်ခုကို x လို့ပေးပြီး x နေရာကို 10 လို့ပေးလိုက်ရင် x ကို 10 လို့ထားပြီး တွက်ခဲ့ကြတာကို မှတ်မိပါဦးမယ်။ Java ရဲ့ variable ဟာလဲ ဒီသဘောပါပဲ။ ဒါပေမဲ့ Java မှာက Variable ကို အသုံးပြုချင်ရင် သုံးပါမယ်ဆိုပြီး ဖော်ပြပေးရပါတယ်။ အဲ့ဒါကို Variable Declaration လို့ ခေါ်ပါတယ်။ Variable တစ်ခုကို Declare လုပ်တဲ့ နေရာမှာ Variable Type ရယ် Variable Name ကို ဖော်ပြပေးဖို့လိုအပ်ပါတယ်။

နောက်ပြီး Java Class တွေထဲမှာ Variable တွေကို ကြိုက်တဲ့ နေရာမှာ Declare လုပ်ပြီး အသုံးပြုနိုင်ပါတယ်။ ကျွန်တော်တို့ Variable တွေကို လေ့လာတဲ့နေရာမှာ အောက်ပါအချက်တွေကို သိထားဖို့လိုအပ်ပါတယ်။

- Variable ကို ဘယ်လို ရေးရမလဲ
- Variable အမျိုးအစားတွေဟာ ဘယ်လိုရှိလဲ
- Variable တွေကို ဘယ်လိုသုံးရမလဲ

Variable ကို ဘယ်လိုရေးရမလဲ

Variable တွေကို Class ထဲက ကြိုက်တဲ့နေရာမှာ မဆို ရေးသားနိုင်ပါတယ်။ ရေးသားထားတဲ့ နေရာအပေါ်မူတည်ပြီး အမျိုးအစားတူမှာ မဟုတ်ပါဘူး။ အဲ့ဒီအကြောင်းနဲ့ ပတ်သက်ပြီး Variable Types တွေအကြောင်းပြောတဲ့အခါမှပဲ အသေးစိတ် ပြောပြပါမယ်။

ဒီနေရာမှာတော့ Variable တစ်ခုကို ဘယ်လိုရေးရင်မှန်မလဲ ဆိုတာကို ကျွန်တော်တို့ လေ့လာသွားကြရအောင်။

```
[Type] [Variable Name];
```

Variable တွေကို Program ထဲမှာ အသုံးပြုတော့မယ်ဆိုရင် အရင်ဆုံး Declare လုပ်ထားဖို့လိုအပ်ပါတယ်။ Variable တွေကို Declare လုပ်တဲ့နေရာမှာ အထက်ပါအတိုင်း Type နဲ့ Variable Name ကို ရေးလိုက်ရင် ရပါပြီ။ တနည်းပြောရမယ်ဆိုရင် Java ဘာသာရပ်မှာ Variable တွေကို Declare လုပ်မယ်ဆိုရင် အဲ့ဒီ Variable ရဲ့ Type ကို မသတ်မှတ်ပဲ ရေးလို့မရပါဘူး။ ဒါ့ကြောင့် Java Programming Language ကို Strong Type Language လို့လဲ ခေါ်လေ့ရှိပါတယ်။

int တို့ char တို့လို Primitive Data Type တွေကိုလဲ Type အဖြစ် အသုံးပြုနိုင်သလို String တို့လို Java API က ပြင်ဆင်ထားတဲ့ Class တွေကိုလဲ Type အဖြစ်အသုံးပြုနိုင်ပါတယ်။ ဒါ့အပြင် User က ရေးသားထားတဲ့ Class တွေကိုလဲ Type အဖြစ် အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

```
int count;          // primitive data type
String name;        // Predefined Type
Account account;    // User Defined Class
```

Variable အမျိုးအစားတွေဟာ ဘယ်လိုရှိလဲ

Variable တွေကို ဘယ်လိုရေးရမယ်ဆိုတာကို လေ့လာပြီးတဲ့ နောက်မှာ Variable အမျိုးအစားတွေ ဘယ်လိုရှိလဲ ဆိုတာကို လေ့လာသွားကြရအောင်။ Variable တွေကို ရေးသားလိုက်တဲ့နေရာအပေါ်မှာ မူတည်ပြီး အမျိုးအစားလေးခွဲခြားနိုင်ပါတယ်။ အဲ့ဒါအပြင် Non Access Modifier တွေကို ရေးသားခြင်းဖြင့်လဲ အမျိုးအစားတွေကို သတ်မှတ်လို့ရပါတယ်။

အရင်ဆုံးရေးသားထားတဲ့နေရာအပေါ်မှာ မူတည်ပြီး အမျိုးအစားခွဲခြားပုံကနေ စပြီး လေ့လာသွားကြရအောင်။ မိမိရေးသားထားတဲ့ နေရာအပေါ်မှာ မူတည်ပြီး Variable တွေကို အသုံးပြုနိုင်တဲ့ Scope တွေဟာ ကွဲပြားကြပါတယ်။

ရေးသားထားတဲ့ နေရာအပေါ်မူတည်ပြီး Variable တွေကို Global Variable နဲ့ Local Variable ဆိုပြီး ခွဲခြားနိုင်ပါတယ်။

Global Variables

Class Body အောက်တည့်တည့်မှာ ရေးထားတဲ့ Variable တွေဟာ Global Variable တွေဖြစ်ကြပါတယ်။ အဲ့ဒီ Variable တွေကို Class အတွင်းကြိုက်တဲ့နေရာမှာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ဒီနေရာမှာ အဲ့ဒီ Variable တွေကို အသုံးပြုနိုင်တဲ့ အချိန် အတိုင်းအတာဆိုတာကိုလဲ သိဖို့လိုအပ်ပါတယ်။ ဒါကိုတော့ နောက်မှာ ဆက်ပြီး ရှင်းပြသွားပါမယ်။

```
class Student {

    String name;

    public void showName() {
        System.out.println(name);
    }
}
```

အထက်ပါအတိုင်း Student Class ရဲ့ Class Body အောက်မှာ String name လို့ရေးထားတာဟာ String Type Variable ဖြစ်တဲ့ name ဆိုတဲ့ Variable တစ်ခုကို ရေးသားလိုက်တာပါပဲ။ Class Body အောက်မှာ ရေးသားထားတာ ဖြစ်တဲ့ အတွက် ဒါဟာ Global Variable ဖြစ်ပါတယ်။

ဒါ့ကြောင့် name variable ကို showName method ထဲကနေလဲ အသုံးပြုနေနိုင်တာ ဖြစ်ပါတယ်။ Global Variable တွေကိုဆိုရင် Class တစ်ခုလုံးကနေ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Global Variable တွေမှာ Value ကို သတ်မှတ်မထားဘူးဆိုရင် Compiler ကနေ Default တန်ဖိုးကို သတ်မှတ်ပေးမှာ ဖြစ်ပါတယ်။ Number တွေရဲ့ Default တန်ဖိုးဟာ 0 ဖြစ်ပြီး၊ boolean တွေရဲ့ Default တန်ဖိုးဟာ false ဖြစ်ပါတယ်။ Object တွေရဲ့ Default တန်ဖိုးကတော့ null ဖြစ်ပါတယ်။

Global Variable တွေကိုထပ်ပြီး ဘယ်နေရာမှာ သုံးချင်တာလဲ ဆိုတဲ့ ရည်ရွယ်ချက်နဲ့ ထပ်ပြီး ခွဲခြားနိုင်ပါတယ်။ What is Class ဆိုတာကိုရှင်းပြရင်းနဲ့ Class တစ်ခုကို ရေးသားရခြင်း ရည်ရွယ်ချက် ၂ ခုရှိတယ်လို့ပြောခဲ့ပါတယ်။ အဲ့ဒါတွေကတော့ Class ကနေ တဆင့် Object ကိုတည်ဆောက်ဖို့ရည်ရွယ်ချက်နဲ့ Utility Function တွေကို ရေးသားဖို့ရည်ရွယ်ချက်ပါ။

အဲ့ဒီ ရည်ရွယ်ချက်တွေအပေါ်မူတည်ပြီး Variable တွေကိုလဲ Object တွေမှာသုံးချင်တဲ့ Varable တွေနဲ့ Class ထဲမှာ ထားပြီး Function တွေထဲမှာ အသုံးပြုချင်တဲ့ Variable တွေဆိုပြီး ခွဲခြားနိုင်ပါတယ်။

Instance Variable

Instance Variable တွေဆိုတာကတော့ Object တွေမှာ အသုံးပြုချင်တဲ့ Variable တွေဖြစ်ပါတယ်။ Class တစ်ခုကို ရေးသားရတဲ့ အဓိက ရည်ရွယ်ချက်ဟာ Object ဆောက်ဖို့ဆိုတာ ဖြစ်တဲ့ အတွက် Default အတိုင်းဆိုရင် Global Variable တွေဟာ Instance Variable တွေ ဖြစ်ကြပါတယ်။

အထက်ပါ နမူနာထဲမှာဆိုရင် String name; လို့ Class Body အောက်မှာ ရေးသားထားတဲ့ အတွက် name ဟာ String type Variable ဖြစ်တယ်။ ပြီးတော့ Global Variable လဲဖြစ်ပြီး Instance Variable တစ်ခုဖြစ်တယ်လို့ သိနိုင်မှာ ဖြစ်ပါတယ်။

Global Variable ဖြစ်ပြီး value ကို ဘာမှ မရေးထားတဲ့ အတွက် Default တန်ဖိုးဟာ null ဖြစ်နေမယ်ဆိုတာကို သိနေနိုင်ပါတယ်။ ပြီးတော့ Instance Variable တစ်ခုဖြစ်တယ်ဆိုတာကိုလဲ သိနိုင်ပါတယ်။ ရှေ့မှာလဲပြောခဲ့ပြီး ဖြစ်ပါတယ်။ Instance Variable တွေဟာ Class ထဲမှာ ထားပြီးသုံးတဲ့ Variable တွေမဟုတ်ပဲ အဲ့ဒီ Class ကနေ Object ကို တည်ဆောက်တဲ့အခါမှာ အဲ့ဒီ Object တွေမှာ ပိုင်ဆိုင်သင့်တဲ့ State တွေကို သိမ်းပေးထားဖို့အတွက် သုံးပြုပါတယ်။

ဒါ့ကြောင့် Instance Variable တွေဟာ Object တစ်ခုချင်းဆီနဲ့ သက်ဆိုင်တဲ့ Variable တွေ ဖြစ်ကြပါတယ်။ Class တစ်ခုကနေ Object ဘယ်နှစ်ခုဆောက်နိုင်လဲဆိုရင် အကြိမ်ကြိမ်တည်ဆောက်နိုင်ပါတယ်။ ပြီးတော့ အဲ့ဒီ Object တစ်ခုချင်းစီဟာလဲ သီးခြားစီရပ်တည်နေတာဖြစ်ပါတယ်။

```
class StudentTest {

    public static void main(String [] args) {
        Student s1 = new Student();
        s1.name = “Aung Aung”;

        Student s2 = new Student();
        s2.name = “Thidar”;
    }

}
```

Student s1 = new Student() လို့ရေးထားတာဟာ အရင်ဆုံး Stack Memory ပေါ်မှာ s1 ဆိုတဲ့ အမည်နဲ့ Student Type Variable တစ်ခုကို သွားဆောက်ပါတယ်။ ပြီးမှ new Student() လို့ရေးတဲ့ ကုဒ်ကို Evaluate လုပ်ပြီး Heap Memory ပေါ်မှာ Student Object ကို သွားဆောက်ပါတယ်။ ဒီလို Object ဆောက်တဲ့အခါမှာ Student Class ထဲမှာရှိတဲ့ instance variable တွေကို Copy ကူးသွားပြီး ယူသွားပါတယ်။ ဒါ့ကြောင့် Student Object ထဲမှာ name ဆိုတဲ့ Instance Variable လဲ ပါသွားပါတယ်။

ပြီးမှ s1 နေရာကို တည်ဆောက်ထားတဲ့ Object ရဲ့ Memory Address ကို အစားထိုးပါတယ်။ ဒီလိုပဲ s2 Variable ကိုလဲ တည်ဆောက်သွားတာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် s1 ရဲ့ name နဲ့ s2 ရဲ့ name တွေဟာ သီးခြားစီ ဖြစ်နေမှာ ဖြစ်ပါတယ်။ s1.name = “Aung Aung” လို့ ရေးလိုက်တယ်ဆိုရင် s1 ရဲ့ name သာ Aung Aung ဖြစ်သွားမှာ ဖြစ်ပြီး s2 ရဲ့ name နဲ့ သက်ဆိုင်ခြင်း ရှိမှာ မဟုတ်ပါဘူး။ အလားတူ s2.name = “Thidar” လို့ရေးလိုက်ရင်လဲ s2 ရဲ့ name သာ တန်ဖိုးပြောင်းသွားတာ ဖြစ်ပြီး s1 ရဲ့ name နဲ့ ပတ်သက်မှုရှိမှာ မဟုတ်ပါဘူး။

Instance Variable တွေဟာ Object တွေအတွက် ရေးထားတဲ့ Variable တွေ ဖြစ်တဲ့ အတွက် Instance Variable တွေကို Object Variable တွေလို့လဲ ခေါ်လေ့ရှိပါတယ်။ ဒါ့ကြောင့် Instance Variable တွေကို Access လုပ်ချင်တဲ့ အခါမှာ Owner ဖြစ်တဲ့ Object က တဆင့် objectName.variableName လို့ ရေးသားရတာ ဖြစ်ပါတယ်။

Static Variable

Java ဘာသာရပ်မှာ Class တွေကို ရေးသားတာဟာ Object ဆောက်ဖို့ ကြိုးပဲ မဟုတ်ပါဘူး။ Java မှာ Class မပါပဲ ကုဒ်တွေကို ရေးသားလို့ မရနိုင်ပါဘူး။ ဒါ့ကြောင့် Utility Function တွေကို ရေးသားမယ်ဆိုရင်တောင်မှ Class ထဲမှာပဲ ရေးသားရမှာ ဖြစ်ပါတယ်။ အဲ့ဒီလို Class ထဲမှာပဲထားပြီး အသုံးပြုချင်တဲ့ Method တွေကိုတော့ Static Method လို့ခေါ်ပါတယ်။ တဖန် အဲ့ဒီ Method တွေအကြားမှာ ဘုံအနေနဲ့ ထားပြီး သိမ်ထားချင်တဲ့ တန်ဖိုးတွေအတွက် Variable တွေလိုအပ်လာပါတယ်။ Object မှာ သုံးဖို့ မဟုတ်ပဲ Class ထဲမှာပဲ ထားပြီး အသုံးပြုချင်တဲ့ Variable တွေကိုတော့ Static Variable တွေလို့ ခေါ်ကြပါတယ်။

ကဲ ဒီနေရာကစပြီး Static Variable တွေကို ဘယ်လိုရေးရမယ်၊ ပြီးတော့ Static Variable တွေရဲ့ ထူးခြားချက်တွေနဲ့ Static Variable တွေကို ဘယ်လို အသုံးပြုရမယ် ဆိုတာကို လေ့လာသွားကြရအောင်။

Static Variable တွေကိုဘယ်လိုရေးကြမလဲ

Global Variable တွေကို ဒီအတိုင်း ရေးထားရင် Instance Variable တွေဖြစ်ပါတယ်။ တနည်း ပြောရမယ်ဆိုရင် Global Variable တွေဟာ Default အတိုင်းဆိုလို့ကတော့ Object တွေမှာ သုံးမည့် Instance Variable တွေ ဖြစ်နေမှာပါပဲ။ Static Variable အဖြစ် အသုံးပြုလိုတဲ့အခါမှာ static ဆိုတဲ့ Non Access Modifier ကို Variable Type ရဲ့ ရှေ့မှာ ရေးသားပြီး Static Variable ဖြစ်ကြောင်း ဖေါ်ပြနိုင်မှာ ဖြစ်ပါတယ်။

```
class Student {

    static int count;
    String name;

    void show() {
        System.out.println("Count : " + count);
        System.out.println("Name  : " + name);
    }

}
```

အထက်ပါ Student Class ထဲမှာ Global Variable ၂ ခုရှိပါတယ်။ count နဲ့ name ပါ။ count ဟာ int type ဖြစ်ပြီး static လို့ရေးထားတဲ့ အတွက် static variable ဖြစ်ပါတယ်။ ပြီးတော့ name ကတော့ Type အပြင် ဘာမှ မရေးထားတဲ့ အတွက် Instance Variable ဖြစ်ပါတယ်။

Static Variable တွေရဲ့ ထူးခြားချက်တွေ

Instance Variable တွေဟာ Object တွေ အတွက် ဖြစ်တဲ့ အတွက် Class ကနေ Object ဆောက်တဲ့ အခါမှာ ကော်ပီကူးသွားပြီး Object ထဲမှာ State တွေကို သိမ်းထားဖို့ အတွက် အသုံးပြုပါတယ်။ ဒါပေမဲ့ Static Variable တွေကတော့ Utility Function တွေအကြားမှာ State တွေကို Share လုပ်ပြီး အသုံးပြုဖို့ အတွက် ဖြစ်ပါတယ်။

ဒါ့ကြောင့် Object ဆောက်တဲ့ အခါမှာ Static Variable တွေကို Copy ကူးပြီး ယူမသွားပါဘူး။ Class ထဲမှာပဲ ချန်ထားခဲ့ပါတယ်။ Utility Function တွေအကြားမှာ အသုံးပြုဖို့ ထားတဲ့ Variable တွေဖြစ်တဲ့ အတွက် Static Variable တွေဟာ Class ထဲမှာပဲ ရှိနေမှာ ဖြစ်ပါတယ်။ ပြီးတော့ Class တွေဟာ JVM ပေါ်မှာ တစ်ခုထဲသာရှိတဲ့

```
class Student {

     static int count;
     String name;

     void show() {
          System.out.println(“Count : “ + count);
          System.out.println(“Name  : “ + name);
     }

     void countUp() {
          count ++;
     }

     public static void main(String [] args) {
          Student s1 = new Student();
          s1.countUp();
          s1.name = “Aung Aung”;
          s1.show();

          Student s2 = new Student();
          s2.countUp();
          s2.name = “Thidar”;
          s2.show();
     }

}
```

အတွက် Static Variable တွေဟာလဲ JVM မှာ တစ်ခုထဲသာရှိတဲ့ Variable တွေအနေနဲ့ ရပ်တည်မှာ ဖြစ်ပါတယ်။ ဒီနေရာမှာ Static Variable တွေရဲ့ ထူးခြားချက်ကို သိအောင် အောက်ပါ Code တွေနဲ့ စမ်းကြည့်ကြရအောင်။ အထက်ပါ ကုဒ်ထဲမှာ countUp ဆိုတဲ့ Method ထဲမှာ count ++ ဆိုပြီး static variable ကို တစ်တိုးထားပါတယ်။

ပြီးတော့ main method ထဲကို ကြည့်ကြရအောင်။ အရင်ဆုံး. s1 ဆိုတဲ့ Object တစ်ခုကို ဆောက်တယ်။ s1 ရဲ့ countUp method ကို invoke လုပ်ပါတယ်။ ဒါဆိုရင် ဘာဖြစ်သွားလဲ ဆိုတာကို ကြည့်ကြရအောင်။ count ဆိုတဲ့ variable ကို တစ်တိုးပါမယ်။ ဒါ့ကြောင့် count ရဲ့ တန်ဖိုးဟာ 1 ဖြစ်သွားမယ်။

ပြီးတော့ ဆက်ပြီး s1.show() ကို ခေါ်ပါတယ်။ ဒီနေရာမှာ s1 ရဲ့ count နဲ့ name ကို Console ပေါ်မှာ ပြပေးပါမယ်။ သတိထားဖို့လိုတာက count ဟာ Static Variable ဖြစ်ပြီးတော့ name ဟာ Instance Variable ဖြစ်နေတာကိုပါပဲ။ ဆက်ပြီး s1 ဆိုတဲ့ နာမည်နဲ့ Object တစ်ခုကို ဆောက်ပြန်ပါတယ်။ ပြီးတော့ countUp() method ကို ခေါ်ပြန်တယ်။ ပြီးမှ s2 ရဲ့ name ကို Thidar လို့ အစားထိုးပြီး s2 ရဲ့ show() Method ကို ခေါ်ပြန်တယ်။

S1 ရဲ့ show() Method ကို ခေါ်တုန်းက အဖြေဘယ်လိုပြပြီး s2 ရဲ့ show() Method ခေါ်တော့ အဖြေဘယ်လို ထွက်မယ်ဆိုတာကို သတိထားမိစေချင်ပါတယ်။

```
Count : 1
Name  : Aung Aung
Count : 2
Name  : Thidar
```

အထက်ပါ အတိုင်း အဖြေထွက်ပါမယ်။ s1.show() ကို ခေါ်ခဲ့တုန်းက count ရဲ့ တန်ဖိုးဟာ 1 ဖြစ်ပါတယ်။ show() method ကို မခေါ်ခင် countUp() Method ကို ခေါ်ခဲ့တယ်။ ဒါ့ကြောင့် count ဟာ 1 ဖြစ်သွားပါတယ်။

ဒုတိယ အကြိမ် s2.countUp() ကို ခေါ်ပြန်တယ်။ count ကို တစ်ပြန်တိုးပါတယ်။ တကယ်လို့သာ count ဟာ Object တွေဆောက်တိုင်း Copy ကူးသွားရင် အဲ့ဒီနေရာမှာလဲ count ကို s2 က ပိုင်မှာ ဖြစ်တဲ့ အတွက် တန်ဖိုးဟာ ၁ ပဲဖြစ်မှာပါ။ ဒါပေမဲ့ count ဟာ Static Variable ဖြစ်တယ်။ ဒါ့ကြောင့် s2 ဟာ JVM တစစ်ခုလုံးမှာ တစ်ခုထဲသာ ရှိပါတယ်။ s1 က သိတဲ့ count ကော s2 ကသိတဲ့ count ပါ တစ်နေရာထဲမှာ ရှိတဲ့ count ကို မြင်နေတာ ဖြစ်တယ်။ ဒါ့ကြောင့် s1 ရဲ့ count ကို တစ်တိုးလိုက်တော့ count ရဲ့ တန်ဖိုးဟာ 1 ဖြစ်လာပြီး၊ နောက်တစ်ကြိမ် s2 ကနေ count ကို တစ်ပြန်တိုးတော့ တန်ဖိုးဟာ 2 ဖြစ်သွားတာ ဖြစ်ပါတယ်။

ဒီနေရာမှာ Static Variable တွေဟာ JVM ပေါ်မှာ တစ်ခုထဲသာ ရှိမှာ ဖြစ်တဲ့ အတွက် ဘယ် Object ကပဲ သုံးသုံး တစ်ခုထဲကို သုံးနေတယ်ဆိုတာကို သိစေချင်ပါတယ်။

Static Variable တွေကို ဘယ်လိုနေရာမျိုးတွေမှာ အသုံးပြုကြမလဲ

ဒါကတော့ Static Variable တွေရဲ့ ထူးခြားချက်ကို ကြည့်ရင် သိနိုင်ပါလိမ့်မယ်။ Static Variable တွေကို Utility Function တွေအကြားမှာ State တွေကို ဘုံအနေနဲ့ အသုံးပြုချင်တဲ့ အခါမျိုးမှာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ နောက်တစ်ခုက မိမိရဲ့ Application တစ်ခုလုံးမှာ Share လုပ်ပြီး သုံးချင်တဲ့ Data မျိုးတွေမှာလဲ Static Variable တွေကို အသုံးပြုနိုင်ပါတယ်။

ဒါ့ကြောင် Desktop Application တွေမှာ Login User ရဲ့ Information တွေကို Application တစ်ခုလုံးမှာ ဘုံအနေနဲ့ Share လုပ်ပြီး အသုံးပြုလိုတဲ့ အခါမျိုးမှာ Static Variable တွေကို အသုံးပြုကြတာကို တွေ့ရပါမယ်။ ပြီးတော့ Application တစ်ခုလုံးမှာ Object တစ်ခုထဲသာ ရှိစေလိုတဲ့ Singleton Design Patterns မျိုးမှာလဲ Static Variable တွေကို အသုံးပြုတာကို တွေ့ရပါတယ်။

OOP ကို သင်ကြားပြီးတဲ့ အခါမှာ Design Patterns တွေအကြောင်းကို Class Design တွေနဲ့ လက်တွေ့ ပြဿနာတွေကို ဘယ်လို ဖြေရှင်းတယ်ဆိုတာကို ဖေါ်ပြသွားပါမယ်။ အဲ့ဒီ အခန်းရောက်ရင် Singleton Pattern အကြောင်းကို အကြောင်းကို ဆက်ပြီး ဖေါ်ပြသွားပါမယ်။

Local Variables

Code Block တစ်ခုခုအတွင်းမှာ စာပြီး Declare လုပ်ထားတဲ့ Variable တွေဟာ Local Variable တွေ ဖြစ်ကြပါတယ်။ Code Block လို့ပြောတဲ့နေရာမှာ အတွန့်ကွင့် { ဖြစ့်စပြီး အတွန့်ကွင်း အပိတ်နဲ့ } ဆုံးတဲ့ Code တွေကို ဆိုလိုတာပါ။ Instance Block တွေ၊ Static Block တွေ၊ Method Body တွေ၊ if / else statement Block တွေ၊ For နဲ့ While Block တွေဟာ Code Block တွေဖြစ်ကြပါတယ်။

တဖန် Method Arguments တွေနဲ့ for loop ထဲမှာ ရေးထားတဲ့ Variable တွေကလဲ Local Variable တွေ ဖြစ်ကြပါတယ်။ Local Variable တွေဟာ သူတို့ကို စပြီးရေးထားတဲ့ နေရာကနေ စပြီး သူတို့ကို ရေးထားတဲ့ Block ဆုံးတဲ့အထိ အသုံးပြုလို့ရပါတယ်။ Local Variable တွေဟာ Global Variable တွေနဲ့ မတူပဲ Default တန်ဖိုးကို Compiler က ထည့်မပေးနိုင်ပါဘူး။ ဒါ့ကြောင့် Local Variable တွေကို အသုံးပြုတော့မယ်ဆိုရင် တန်ဖိုး တစ်ခုခုကို Assign လုပ်ပြီးမှာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ တကယ်လို့ တန်ဖိုးကို အစားမထိုးပဲ အဲ့ဒီ Local Variable တစ်ခုကို Access လုပ်ရင် Compile Time မှာ Error တက်မှာ ဖြစ်ပါတယ်။

Local Variables in Blocks

```
class LocalVariablesInBlocks {

    {
        // Instance Block
        int data = 10;
    }

    static {
        // static block
        int data = 11;
    }
}
```

Class Body အတွင်းမှာ { နဲ့စပြီး } နဲ့ ဆုံးထားတာဟာ Instance Block တွေဖြစ်ပါတယ်။ Instance Block တွေဟာ Class ကနေ Object ကိုထည်ဆောက်တဲ့အခါမှာ Constructor ကို အလုပ်မလုပ်ခင် တစ်ကြိမ် အလုပ်လုပ်စေပါတယ်။ အထက်ပါအတိုင်း Instance Block ထဲမှာ int data = 10 လို့ရေးထားတဲ့ အတွက် data ဟာ Local Variable တစ်ခုဖြစ်ပြီး } နေရာအထိ အဲဒီ data ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

တဖန် static { } လို့ရေးထားတာကတော့ static block ဖြစ်ပါတယ်။ JVM ပေါ်ကို အဲ့ဒီ Class ကို Load လုပ်ပြီးတာနဲ့ static block ထဲမှာရှိတဲ့ ကုဒ်တွေကို အလုပ်လုပ်စေမှာ ဖြစ်ပါတယ်။

Local Variable in Methods

Method တစ်ခုအတွင်းမှာ ရေးသားထားတဲ့ Variable တွေဟာလဲ Local Variable တွေ ဖြစ်ကြပါတယ်။ အဲ့ဒီ Variable တွေဟာလဲ စပြီး Declare လုပ်တဲ့ နေရာကနေစပြီး Method Body ဆုံးတဲ့ အထိသာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ Method အထဲမှာ Local Variable တွေကို ၂ နေရာမှာ တွေ့နိုင်ပါတယ်။

ပထမဆုံးတွေ့နိုင်တာကတော့ Method Body.အတွင်းမှာ Declare လုပ်ထားတဲ့ Variable တွေပါ။ နောက်တစ်ခုကတော့ Method Argument တွေမှာ ရေးသားထားတဲ့ Variable တွေပဲ ဖြစ်ကြပါတယ်။

```
class LocalVariablesInMethod {

    void someMethod() {
        // declare in method
        int data = 10;
    }

    void takeArgument(int data) {
        // can use data in method body
    }
}
```

အထက်ပါ နမူနာထဲမှာ someMethod() ထဲမှာ int data = 10 ဆိုပြီးရေးထားပါတယ်။ Type ကို တွဲရေးပြီဆိုတာနဲ့ ဒါဟာ Variable တစ်ခုကို လာပြီး Declare လုပ်တယ်ဆိုတာကို သဘောပါက်ဖို့လိုပါတယ်။ Method Body ထဲမှာ စပြီး Declare လုပ်ထားတဲ့ အတွက် ဒါဟာ Local Variable ဖြစ်ပါတယ်။ အဲ့ဒီ Variable ကို Declare လုပ်တဲ့ နေရာရဲ့ နောက်ကနေစပြီး Method ဆုံးတဲ့ အထိ အသုံးပြုနိုင်ပါတယ်။

နောက် Method တစ်ခုကတော့ takeArgument(int data) လို့ရေးထားပါတယ်။ Method Argument အနေနဲ့ int data ကို ရေးသားထားပါတယ်။ ဒီနေရာမှာလဲ Type နဲ့ အတူရေးထားတာ ဖြစ်တဲ့ အတွက် Local Variable ဖြစ်ပါတယ်။ အဲ့ဒီ Variable ကို takeArgument method အတွင်းမှာပဲ အသုံးပြုလို့ ရမှာ ဖြစ်ပါတယ်။ ဒီနေရာမှာ someMethod ထဲက int data နဲ့ takeArgument ထဲက int data တွေဟာ Type နဲ့ Name ဟာ တူနေပေမဲ့ အသုံးပြုနိုင်တဲ့ နေရာ (Scope) ဟာ မတူပါဘူး။ ဒါကြောင့် သီးခြား Variable တွေ ဖြစ်ကြပါတယ်။

Local Variable in Statements

```
for(int i=0; i < 10; i++) {
    String str = “Count is “;
    System.out.println(str + i);
}
```

Decision Statement တွေနဲ့ Looping Statement တွေထဲမှာ ရေးသားထားတဲ့ Variable တွေဟာလဲ Local Variable တွေ ဖြစ်ကြပါတယ်။ အထက်ပါ နမူနာထဲမှာ Local Variable ၂ ခု ပါဝင်ပါတယ်။ int i = 0 လို့ ရေးထားတဲ့ အတွက် ဒါဟာလဲ Local Variable ဖြစ်ပါတယ်။ ပြီးတော့ String str လို့ ရေးထားတာလဲ Local Variable ဖြစ်ပါတယ်။

Local Variable တွေဟာ သူတို့ကို စရေးထားတဲ့ နေရာကနေစပြီး သူတို့ရှိနေတဲ့ Block ဆုံးတဲ့ အထိ အသုံးပြုနိုင်မှာဖြစ်ပါတယ်။ ဒါ့ကြောင့် for(int i = 0; i < 10; i ++) ဆိုပြီး i ကို ပြန်ပြီး အသုံးပြုနေနိုင်တာ ဖြစ်ပါတယ်။

Variable Hidings

ကျွန်တော်တို့ Variable တွေကို ရေးသားတဲ့ နေရာမှာ တစ်ခုသတိထားဖို့လိုတာက Scope တစ်ခုအတွင်းမှာ နာမည်တူ Variable တွေကို ရေးသားလို့ မရနိုင်ပါဘူး။ Scope မတူရင်တော့ နာမည်တူ Variable တွေကိုရေးသားနိုင်ပါတယ်။

Local Variable တွေအချင်းချင်း Scope မတူတာက ပြဿနာမရှိပေမဲ့ Global Scope ထဲက Variable နဲ့ Local Scope ထဲက Variable တွေမှာ နာမည်တူခဲ့ရင် Local Scope ထဲမှာတော့ Local Scope ထဲက Variable ကိုပဲ အသုံးပြုသွားမှာ ဖြစ်ပါတယ်။ ဒီလို Local Scope ထဲက Variable က Global Scope ထဲက Variable ကို သွားပြီး ဖုန်းကွယ်သလို ဖြစ်သွားတာကို Variable Hiding လို့ခေါ်ပါတယ်။

Global Variable တွေမှာ Instance Variable နဲ့ Static Variable ဆိုပြီး ရှိပါတယ်။ အဲ့ဒီအတွက် Variable Hiding ဖြစ်နိုင်တဲ့ ပုံစံတွေကို ကြည့်ကြည့်ရအောင်။

```
public class VariableHiding {

     String name;
     static int count;

     public void setValues(String name, int count) {
          name = name;
          count = count;
     }
}
```

အထက်ပါနမူနာကို ကြည့်ရင် Global Scope ထဲမှာ Instance Variable name ဆိုတာရှိပြီး Static Variable count ဆိုတာရှိပါတယ်။ ပြီးတော့ setValues method ထဲမှာလဲ Argument အနေနဲ့ name နဲ့ count ကို ယူထားပါတယ်။ ဒီလို ရေးလိုက်တာနဲ့ Global Scope ထဲက Variable တွေနဲ့ Local Scope ထဲက Variable တွေဟာ နာမည်တူသွားပါပြီ။

ဒါ့ကြောင့် အဲ့ဒီ Scope ထဲမှာ name လို့ Access လုပ်ရင် Local Scope ထဲက Variable တွေကိုပဲ အသုံးပြုတော့မှာ ဖြစ်ပါတယ်။ အဲ့ဒီလိုပဲ count လို့ရေးလိုက်ရင်လဲ Local Scope ထဲက Variable ကိုပဲ အသုံးပြုတော့မှာပါ။ ဒါ့ကြောင့် ဒီနေရာမှာ name = name ဆိုရင် Local Variable name ကို Local Variable name နဲ့ အစားနေတော့မှာ ဖြစ်ပါတယ်။

ဒါပေမဲ့ ဒီ Method က Argument အနေနဲ့ ယူထားတဲ့ တန်ဖိုးတွေနဲ့ Global Scope ထဲက Variable တွေကို သတ်မှတ်ပေးနေတာ ဖြစ်ပါတယ်။ အဲ့ဒီလိုနေရာမျိုးမှာ Global Scope ထဲက Variable ကို အသုံးပြုလိုပါက Owner.variable လို့ ရေးပေးရမှာ ဖြစ်ပါတယ်။

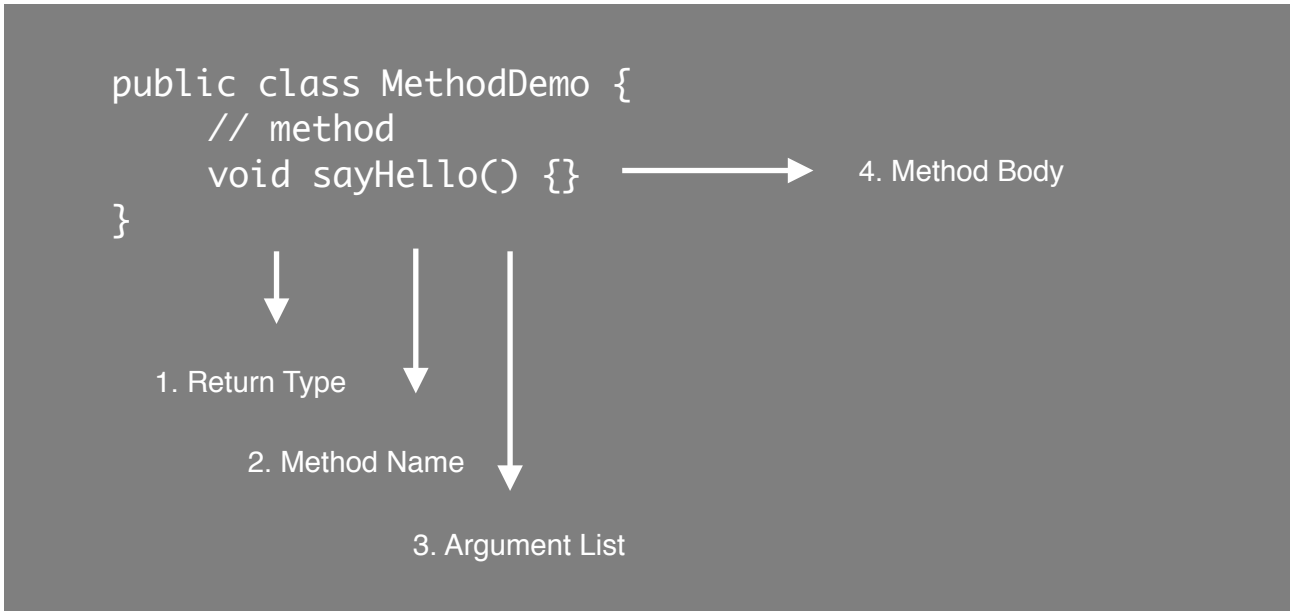
Global Variable name ဟာ Instance Variable ဖြစ်တဲ့အတွက် သူ့ရဲ့ Owner ဖြစ်တဲ့ this ကို ရှေ့မှာ ရေးပေးရမှာ ဖြစ်ပြီး this.name = name လို့ရေးပေးရမှာ ဖြစ်ပါတယ်။ တဖန် count ဟာ Static Variable ဖြစ်ပြီး Owner ဟာ Class ဖြစ်တဲ့ အတွက် VariableHiding.count = count လို့ ရေးပေးရမှာ ဖြစ်ပါတယ်။

Variable Hiding မဖြစ်တဲ့ နေရာတွေမှာ ဆိုရင်တော့ Class တစ်ခုထဲက Variable တွေကို Owner ထည့်ပြီး မရေးလဲ ရနေမှာ ဖြစ်ပြီး၊ Variable Hiding ဖြစ်မှသာ Owner ကို ထည့်ပြီး ရေးပေးရမှာ ဖြစ်ပါတယ်။

Methods

Java ဘာသာရပ်မှာ Utility Function တွေ၊ Object တွေရဲ့ Behavior တွေကို ဖေါ်ပြဖို့အတွက် Method တွေကို အသုံးပြုပါတယ်။ Variable တွေတုန်းကလိုပဲ Method တွေမှာလဲ Class တွေမှာ သုံးမှာလား Object တွေမှာ သုံးမှာလားဆိုပြီး Static Method နဲ့ Instance Method တွေဆိုပြီး ခွဲခြားထားပါတယ်။

အရင်ဆုံး Method တစ်ခုကို မှန်အောင် ဘယ်လိုရေးရမယ်ဆိုတဲ့ အကြောင်းကနေ စတင် လေ့လာသွားကြရအောင်။



Method တွေကို ရေးသားတဲ့နေရာမှာ Class Body အထဲမှာရေးသားရပါတယ်။ Java ဘာသာရပ်မှာ Class တွေရဲ့ အပြင်မှာ Method တွေကို ရေးသားလို့ မရပါဘူး။ ဒါ့အပြင် အခြားသော Code Block တွေထဲမှာလဲ Method တွေကို ရေးသားလို့ မရနိုင်ပါဘူး။

ရေးသားမယ်ဆိုရင် [1. Return Type] [2. Method Name] [3. Argument List] [4. Method Body] ဆိုပြီး ရေးသားရမှာ ဖြစ်ပါတယ်။ အနဲဆုံး အထက်ပါအချက်တွေပါတယ်ဆိုရင် Method အနေနဲ့ Compiler ကနေ အသိအမှတ်ပြုမှာ ဖြစ်ပါတယ်။ ဒီနေရာမှာ Method Name ကတော့ Method ရဲ့ နာမည်ဖြစ်ပြီး Method Body ကတော့ လုပ်ဆောင်ချက်တွေကို ရေးသားရမှာ ဖြစ်တဲ့ အတွက် အထွေအထူး မဖေါ်ပြတော့ပဲ Return Type နဲ့ Argument List အကြောင်းကိုပဲ ဖော်ပြသွားပါမယ်။

Return Type

Method တွေကို ရေးသားတဲ့ အခါမှာ အဲ့ဒီ Method ကို Invoke လုပ်တဲ့အခါမှာ ဘာကို ပြန်ပေးရမလဲ ဆိုတဲ့ Return Type ကို သတ်မှတ်ပေးဖို့လိုအပ်ပါတယ်။ တကယ်လို့ Method တစ်ခုကို Invoke လုပ်ပြီးတဲ့ အခါမှာ int type ကို ပြန်ပြီး လိုချင်တယ်ဆိုရင် Return Type နေရာမှာ int လို့ရေးရပြီး String ကို လိုချင်တယ်ဆိုရင် String လို့ရေးရပါမယ်။

ပြီးတော့ Method ထဲက နောက်ဆုံး Statement ရဲ့ နောက်မှာ return ဆိုပြီး သတ်မှတ်ထားတဲ့ Type ရဲ့ တန်ဖိုးကို Return လုပ်ပေးရမှာ ဖြစ်ပါတယ်။ အဲ့ဒီ Statement ကို Return Statement လို့ ခေါ်ပါတယ်။ Method ထဲက Return Statement ရဲ့ နောက်မှာ ဘာကုဒ်မှ ရေးသားလို့ မရနိုင်ပါဘူး။

တကယ်လို့ မိမိရေးသားထားတဲ့ Method ဟာ Return ပြန်စရာမလိုဘူးဆိုရင် Return Type နေရာကို void လို့ ရေးသားပေးရပါမယ်။ ဒါဆိုရင် အဲ့ဒီ Method ထဲမှာ ဘာတန်ဖိုးမှ Return လုပ်စရာမလိုတော့ပါဘူး။

ဆက်လက်ပြီး ဘယ်လို Method တွေက Return လုပ်ဖို့လိုပြီး ဘယ်လို Method တွေက Return လုပ်စရာမလိုဘူးလဲ ဆိုတာကို ဆက်လက်ပြီး စဉ်းစားသွားကြရအောင်။ Utility Function ပဲပြောပြော Object ရဲ့ Behavior လိုပဲပြောပြော Method တွေဆိုတာ လုပ်ဆောင်မှုတွေကို ဆောင်ရွက်ပေးနိုင်တဲ့ အစိတ်အပိုင်းတွေ ဖြစ်ပါတယ်။ ကျွန်တော်တို့ Student Object ကို အသုံးပြုနိုင်ဖို့ Student Class ကို ရေးသားနေတယ် ဆိုကြပါစို့။ အဲ့ဒီ Class ထဲမှာ Instance Variable အနေနဲ့ String Type name ဆိုတဲ့ Variable ရှိပါတယ်။

ဒီနေရာမှာ Student Object ကနေ name ကို ပြန်ပေးနိုင်တဲ့ Behavior နဲ့ Student Object ကို မိတ်ဆက်စာကို Console ပေါ်မှာ Print လုပ်နိုင်တဲ့ Behavior တွေကို ထည့်ပေးဖို့လိုလာတဲ့ အခါကို စဉ်းစားကြည့်ပါ။ ကျွန်တော်တို့ Method တွေကို ရေးဖို့ လိုအပ်လာပါပြီ။

```
public class Student {

     private String name;

     public String getName() {
          return name;
     }

     public void printGreeting() {
          System.out.println(“Hello, I am “ + name);
     }
}
```

အထက်ပါ အတိုင်း Student Object ရဲ့ name ကို ပြန်ပေးစရာလိုတဲ့ getName() Method မှာတော့ Return Type နေရာမှာ name ရဲ့ Type ဖြစ်တဲ့ String ကို ရေးသားထားရပါတယ်။ နောက်ပြီး Console ပေါ်မှာ Greeting ကို Print လုပ်ရုံသာလိုတဲ့ printGreeting() Method မှာတော့ Return ပြန်စရာမလိုတဲ့ အတွက် void လို့ ရေးသားပေးဖို့လိုအပ်ပါတယ်။ တစ်ခြား Language တွေမှာလို Return ပြန်စရာမလိုတဲ့ အတွက် ဘာမှ မရေးပဲထားလို့ မရပါဘူး။ မပြန်ဘူးဆိုရင်လဲ void ဆိုပြီး ရေးပေးရမှာ ဖြစ်ပါတယ်။

ဒါကြောင့် Method တွေကို ရေးသားတဲ့ အခါမှာ အဲ့ဒီ Method ကို သုံးပြီးတဲ့ အခါမှာ တစ်ခုခုကို ပြန်ရချင်တာလား ဆိုတာကို အရင်ဆုံး စဉ်းစားရပါမယ်။ ပြန်လိုချင်တယ်ဆိုရင်လဲ ဘယ် Type မျိုးကို လိုချင်တာလဲဆိုတာကို ဆုံးဖြတ်ရပါမယ်။ ပြီးမှ လိုချင်တဲ့ Type ကို Return Type အနေနဲ့ သတ်မှတ်ပေးရပါမယ်။ Return ပြန်မလိုချင်တော့ဘူးဆိုရင်တော့ void ကို Return Type အနေနဲ့ ရေးသားပေးရမှာပါ။

Return Statement ကို ရေးသားတဲ့နေရာမှာလဲ သတ်မှတ်ထားတဲ့ Return Type နဲ့ ထပ်တူဒါမှမဟုတ် Sub Type တွေဆိုရင် Return Value အနေနဲ့ အသုံးပြုနိုင်ပါတယ်။ ဆိုကြပါစို့ Student Class ဟာ Person Class ရဲ့ Sub Class ဆိုရင် Person Type ကို Return ပြန်လိုတဲ့ Method ထဲမှာ Student Type ကို Return Value အနေနဲ့ အသုံးပြုနိုင်ပါတယ်။ အဲ့ဒီလို Sub Type ကို Return ပြန်နိုင်တာက Covariant Return လို့ခေါ်ပြီး Java SE 5 ကနေ စတင် အသုံးပြုလာနိုင်တာ ဖြစ်ပါတယ်။

ဒါပေမဲ့ လုံးဝ Type မတူတဲ့ Value ကိုတော့ Return ပြန်လို့မရပါဘူး။

Argument List

Method တစ်ခုရဲ့ Argument ဆိုတာဟာ အဲ့ဒီ Method ထဲကနေ အသုံးပြုဖို့ အတွက်လိုအပ်တဲ့ ဒေတာတွေကို ရယူနိုင်ဖို့ အတွက် ရေးသားရတဲ့ Variable တွေ ဖြစ်ကြပါတယ်။ ဆိုကြပါစို့၊ ကျွန်တော်တို့ int ၂ ခုကို ပေါင်းပြီး ရလဒ်ကို ပြန်ပေးနိုင်တဲ့ Method တစ်ခုကို ရေးနေတဲ့ အခါမှာ int variable ၂ ခုကို Method ထဲကနေ မသိနိုင်ရင် ရေးလို့ ရမှာ မဟုတ်ပါဘူး။ အဲ့ဒီ int နှစ်ခုကို Method Argument အနေနဲ့ ရေးသားပြီး Method ထဲကနေ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

ဆက်လက်ပြီးတော့ Method Argument တွေကို ဘယ်လို ရေးရတယ်ဆိုတာကို လေ့လာသွားကြရအောင်။ method name ရဲ့ နောက်မှာ () လို့ရေးထားတဲ့ နေရာရှိပါတယ်။ အဲ့ဒီ “(“ နဲ့ “)” ကြားမှာ Method အတွင်းမှာ အသုံးပြုလိုတဲ့ Argument တွေကို Variable အနေနဲ့ Declare လုပ်ပေးရပါမယ်။ Method Argument တွေကိုလဲ မလိုအပ်ပါက မရေးပဲနေနိုင်သလို၊ လိုအပ်ပါက တစ်ခုထက်မက Comma ခံပြီး ရေးသားပေးရမှာ ဖြစ်ပါတယ်။

```
public class ArgumentDemo {

     void noArgument() { // codes }

     double makeDouble(double d) {
          return d * 2;
     }

     double add(double a, double b) {
          return a + b;
     }
}
```

အထက်ပါနမူနာထဲမှာ အတိုင်း Argument တွေကို လိုအပ်သလို ရေးသား အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Method Signature and Method Overloading

Method တစ်ခုမှာ အနဲဆုံးရေးသားဖို့ လိုအပ်တဲ့ အစိတ်အပိုင်းတွေကို လေ့လာပြီးတဲ့ နောက်မှာ ဆက်လက်ပြီး Java Compiler နဲ့ Java Runtime မှာ Method တစ်ခုကို ဘယ်လို သိနေလဲ ဆိုတာကို လေ့လာသွားကြပါမယ်။ Variable မှာတုန်းက Scope တစ်ခုအတွင်းမှာ Name တူတဲ့ Variable တွေကို ရေးသားလို့ မရနိုင်ဘူးဆိုတာကို လေ့လာခဲ့ကြရမှာပါ။ ဘာလို့လဲ ဆိုတော့ Java Compiler နဲ့ Java Runtime မှာ Variable ရဲ့ Name ကို ကြည့်ပြီး Identifined လုပ်နေတဲ့ အတွက် ဖြစ်ပါတယ်။

ဒါဖြင့်ရင် Method တွေကိုကော ဘယ်လို Identified လုပ်နေတာလဲ ဆိုတာကို ဆက်ပြီး လေ့လာသွားကြရအောင်။ Class တစ်ခုထဲမှာ နာမည်တူတဲ့ Method တွေ ရေးလို့ရပါတယ်။ ဒါ့ကြောင့် Java Compiler နဲ့ Runtime က Method တစ်ခုကို ကို နာမည် တစ်ခုထဲနဲ့ သတ်မှတ်နေတာ မဟုတ်ပါဘူး ဆိုတာကို သိနိုင်ပါတယ်။

Compiler နဲ့ Runtime ကနေ Method ကို Identified လုပ်နေတဲ့ အစိတ် အပိုင်းတွေကို Method Signature လို့ ခေါ်ပါတယ်။ Method Signature မှာ Method Name နဲ့ Argument List တို့ ပါဝင်ကြပါတယ်။ Java ဘာသာရပ်မှာက Method Name တူပေမဲ့ Method Signature မတူရင် Method တစ်ခုစီလို့ သတ်မှတ်ပါတယ်။

```
public class OverloadDemo {

     public void takeNote(Memo memo) {
          // codes to write note in memo
     }

     public void takeNote(Tablet tablet) {
          // code to write note in tablet
     }
}
```

အထက်ပါ နမူနာထဲမှာ Method နှစ်ခုလုံး နာမည်တူပေမဲ့ Argument တွေမတူပါဘူး ဒါ့ကြောင့် Java Compiler က Method တစ်ခုစီလို့ မြင်ပါတယ်။ ဒါကတော့ Method Argument Type မတူတဲ့အတွက် Method Signature မတူဘူးလို့ သတ်မှတ်သွားတာပါ။

```
public class OverloadDemo {

     public void takeNote(Memo memo) {
          // codes to write note in memo
     }

     public void takeNote(Memo memo, Pen pen) {
          // code to write note in tablet
     }
}
```

အထက်ပါအတိုင်း Argument List အရေအတွက် မတူရင်လဲ Method Signature မတူဘူးလို့ သတ်မှတ်ပါတယ်။ ဥ်ကဲ့သို့ Method Name တူပြီး Argument List မတူတဲ့ Method တွေကို ရေးသားခြင်းကို Method Overloading လို့ခေါ်ဆိုပါတယ်။

JRE ထဲမှာပါတဲ့ Java Standard API တွေမှာလဲ Method Overloading ကို အများအပြား အသုံးပြုရေးသားထားတာကို တွေ့ရပါတယ်။

Type of Methods

Method တွေကို အသုံးပြုလိုတဲ့ ရည်ရွယ်ချက်အပေါ်မူတည်ပြီး အမျိုးအစားခွဲခြားနိုင်ပါတယ်။ Global Variable တွေမှာတုန်းကလိုပဲ Object တွေရဲ့ Behavior အနေနဲ့ အသုံးပြုလိုတဲ့ Method တွေကိုတော့ Instance Method လို့ ခေါ်ပြီး၊ Class ထဲမှာပဲထားပြီး Utility Function တွေအနေနဲ့ အသုံးပြုလိုတဲ့ Method တွေကိုတော့ Static Method တွေလို့ခေါ်ပါတယ်။

Instance Method

Instance Method တွေဟာ Object တွေရဲ့ Behavior အနေနဲ့ အသုံးပြုလိုတဲ့ အခါမှာ ရေးသားရတဲ့ Method တွေ ဖြစ်ပါတယ်။ Class တစ်ခုကို ရေးသားရတဲ့ အဓိက ရည်ရွယ်ချက်ဟာ Object ဆောက်ဖို့ ဖြစ်တဲ့ အတွက် Variable တွေတုန်းကလိုပဲ Class တစ်ခုအတွင်းမှာ Default အတိုင်းရေးသားထားတဲ့ Method တွေဟာ Instance Method တွေ ဖြစ်ကြပါတယ်။

Instance Method တွေဟာ Object တွေနဲ့ သက်ဆိုင်တဲ့ Method တွေ ဖြစ်တဲ့ အတွက် အသုံးပြုလိုတဲ့ အခါမှာ Object တွေကတဆင့် အသုံးပြုရမှာ ဖြစ်ပါတယ်။ ထို့အပြင် Object တွေရဲ့ ကိုယ်ပိုင် State တွေဖြစ်တဲ့ Instance Variable တွေကိုလဲ Instance Method ထဲကနေ Access လုပ်ပြီး အသုံးပြုနိုင်ပါတယ်။ တဖန် Class တစ်ခု အတွင်းမှာရှိတဲ့ Instance Method အချင်းချင်းလဲ အသုံးပြုနိုင်ပါတယ်။

```
public class InstanceMethodDemo {

      private String format = “Hello %s%n”;

      public void showMessage(String name) {
            System.out.format(format, name);
      }

      public void showMessages(String [] names) {
            for(String name : names) {
                  showMessage(name);
            }
      }

      public static void main(String [] args) {
            InstanceMethodDemo demo = new InstanceMethodDemo();
            demo.showMessage(“Java”);

            String [] array = {“Java”, “Programming”};
            demo.showMessages(array);
      }

}
```

အထက်ပါ InstanceMethodDemo Class ထဲမှာ ရေးသားထားတဲ့ showMessage() Method နဲ့ showMessages() Method တွေဟာ static ဆိုတဲ့ Non Access Modifier မပါတဲ့ အတွက် Instance Method တွေ ဖြစ်ကြပါတယ်။ တဖန် main method မှာတော့ static ဆိုတဲ့ Non Access Modifier ပါတဲ့ အတွက် Static Method ဖြစ်ပါတယ်။ Static Method နဲ့ ပတ်သက်ပြီး နောက် အခန်းမှာ အသေးစိတ် ဖေါ်ပြသွားပါမယ်။

ဒီနေရာမှာတော့ Instance Method တွေကို ဘယ်လို ရေးရမယ်ဆိုတာနဲ့ Instance တွေကို ဘယ်လို ခေါ်သုံးရမယ် (Invoke) ဆိုတာကိုပဲ လေ့လာသွားကြပါမယ်။ Instance Method တွေဟာ Object တွေနဲ့ ဆိုင်တဲ့ Method တွေ ဖြစ်တဲ့ အတွက် အခြား Class ထဲကခေါ်တဲ့အခါ၊ ဒါမှမဟုတ် Main Method လို Static Method ထဲကနေ ခေါ်ချင်တဲ့ အခါတွေမှာ Object ဆောက်ပြီး Object ကနေပဲ ခေါ်လို့ရမှာ ဖြစ်ပါတယ်။

ဒါ့ကြောင့် အထက်ပါ နမူနာထဲမှာ Instance Method တွေကို Main Method ထဲကနေ Invoke လုပ်ချင်တဲ့ အတွက် demo ဆိုတဲ့ Object ကို အရင်တည်ဆောက်ပြီးကာမှ demo.showMessage("Java") ဆိုပြီး Invoke လုပ်နေတာဖြစ်ပါတယ်။ demo.showMessages(array) လို့ ရေးထားတာလဲ Instance Method ဖြစ်တဲ့ အတွက် Object ကနေ Invoke လုပ်နေရတာဖြစ်ပါတယ်။

ဒါပေမဲ့ Class တစ်ခုထဲမှာရှိတဲ့ Instance Method အချင်းချင်း Invoke လုပ်လိုတဲ့အခါမှာတော့ Object တစ်ခုထဲမှာ သုံးမည့် Method အချင်းချင်း ဖြစ်တဲ့အတွက် Object ဆောက်စရာမလိုအပ်ပဲ အသုံးပြုလို့ရနေတာဖြစ်တယ်။ showMessages() Method ထဲမှာ showMessage(name) လို့ခေါ်တဲ့အခါမှာ Object ဆောက်စရာမလိုတာကို တွေ့ရပါမယ်။

ဆက်လက်ပြီး Instance Method ထဲကနေဘာတွေကို Access လုပ်လို့ရလဲဆိုတာကို လေ့လာသွားကြရအောင်။

```
public class AccessDemo {
      private static String FORMAT = "Hello %s";
      private String name;

      public void setName(String name) {
            this.name = name;
      }

      public static String changeUpper(String str) {
            return str.toUpperCase();
      }

      public void greet() {
            String message = String.format(FORMAT, name);
            String upperMessage = changeUpper(message);
            System.out.println(upperMessage);
      }
}
```

အထက်ပါနမူနာထဲမှာ ကြည့်ရင်သိနိုင်သလို greet() method ဟာ Instance Method တစ်ခုဖြစ်ပါတယ်။ အဲ့ဒီ Method ထဲမှာ FORMAT ဆိုတဲ့ Static Variable ကိုလဲ အသုံးပြုနိုင်ပါတယ်။ တဖန် name ဆိုတဲ့ Instance Variable ကိုလဲ အသုံးပြုနိုင်သလို changeUpper() ဆိုတဲ့ Static Method ကိုလဲ အသုံးပြုနိုင်ပါတယ်။

ဒါ့အပြင် ရှေ့မှာဖေါ်ပြခဲ့သလို Instance Method ထဲကနေ အခြားသော Instance Method တွေကိုလဲ အသုံးပြုနိုင်ပါတယ်။

Static Method

Class ထဲမှာ ထားပြီး Utility Function တွေအနေနဲ့ အသုံးပြုချင်တဲ့ Method တွေကို Static Method အနေနဲ့ ရေးသားရမှာ ဖြစ်ပါတယ်။ Method တွေကို Default အတိုင်းရေးသားထားမယ်ဆိုရင် Instance Method တွေ ဖြစ်ကုန်ပါတယ်။ Static Method အဖြစ် အသုံးပြုလိုပါက static ဆိုတဲ့ Non Access Modifier ကို Return Type ရဲ့ ရှေ့မှာ ရေးသားပေးရမှာ ဖြစ်ပါတယ်။

ဆက်လက်ပြီး Static Method တွေကို ဘယ်လိုရေးမလဲ၊ Static Method တွေကို ဘယ်လို Invoke လုပ်မလဲ ဆိုတာနဲ့ Static Method တွေက အသုံးပြုနိုင်တာတွေက ဘာတွေလဲ ဆိုတာကို လေ့လာသွားကြပါမယ်။

```
public class StaticMethodDemo {
      private static String FORMAT = “Hello %s%n”;

      public static void show(String message) {
            System.out.printf(FORMAT, message);
      }

      public static void show(String [] messages) {
            for(String m : messages) {
                  show(m);
            }
      }
}
```

အထက်ပါ ကုဒ်နမူနာထဲမှာ ရေးသားထားတဲ့ show() Method နှစ်ခုလုံးမှာ static ဆိုတဲ့ Modifier ကို ရေးသားထားတဲ့ အတွက် Method နှစ်ခုလုံး Static Method တွေ ဖြစ်ကြပါတယ်။ အပေါ်က show() Method ထဲမှာတော့ FORMAT ဆိုတဲ့ Static Variable ကို အသုံးပြုထားပါတယ်။ Static Method တွေဟာ Class တစ်ခုထဲမှာ ရှိတဲ့ Static Member တွေကို အသုံးပြုနိုင်ပါတယ်။

ဒုတိယ show() Method ထဲမှာတော့ for loop ထဲမှာ Static Method နောက်တစ်ခုကို ခေါ်သုံးနေပါတယ်။ Class တစ်ခုထဲမှာရှိတဲ့ Member အချင်းချင်းဆိုပါက Owner ကို ရေးသားစရာမလိုပဲ တိုက်ရိုက် ရေးသား အသုံးပြုနိုင်ပါတယ်။ ဆက်လက်ပြီး အခြား Class တစ်ခုကနေ Static ကို အသုံးပြုပုံကို လေ့လာကြရအောင်။

```
public class StaticMethodTest {

      public static void main(String [] args) {
            String [] array = {“Static Method”, “Java”};
            StaticMethodDemo.show(array);
      }
}
```

StaticMethodTest Class ထဲကနေ StaticMethodDemo Class ရဲ့ Static Method ကို အသုံးပြုမယ်ဆိုရင်တော့ Owner ဖြစ်တဲ့ Class ကနေ တဆင့် ခေါ်ယူအသုံးပြုရမှာ ဖြစ်ပါတယ်။

တစ်ခုသတိထားဖို့လိုအပ်တာက Static Method တွေဟာ Class တစ်ခုထဲမှာရှိတဲ့ အခြား Static Method တွေနဲ့ Static Variable တွေကို အသုံးပြုနိုင်ပါတယ်။ ဒါပေမဲ့ အခြား Instance Method တွေ Instance Variable တွေကိုတော့ အသုံးပြုလို့ မရပါဘူး။

Constructor

Constructor ဆိုတာ Class ကနေ Object ကို တည်ဆောက်တဲ့အခါမှာ အသုံးပြုတဲ့ Special Method တစ်မျိုးဖြစ်ပါတယ်။ Constructor တွေကို Object တစ်ခုကို တည်ဆောက်တဲ့ နေရာမှာ တစ်ကြိမ်သာ အသုံးပြုပြီး Object ဖြစ်သွားပြီးရင်တော့ Constructor တွေကို အသုံးမပြုတော့ပါဘူး။ Constructor တွေကို Instance Variable တွေကို Initialize လုပ်တဲ့အခါမှာ အသုံးပြုဖို့ ပြင်ဆင်ထားတာ ဖြစ်ပါတယ်။

Constructor ရဲ့ နာမည်ဟာ Class နာမည်နဲ့ ထပ်တူတူညီရပါမယ်။ ပြီးတော့ Constructor မှာ Return Type ရေးလို့မရပါဘူး။ Class တစ်ခုအတွက် Constructor ကို ရေးသားတဲ့ အခါမှာ Argument List မတူတဲ့ Construcotr တွေကို Overload လုပ်ပြီး ရေးသားနိုင်ပါတယ်။

Default Constructor

အောက်ဖေါ်ပြပါကုဒ်ထဲမှာလို new Student() ဆိုပြီးရေးသားလိုက်တာဟာ Student Class ရဲ့ Constructor ကို လှမ်းခေါ်ပြီး Object ကို တည်ဆောက်လိုက်တာပဲ ဖြစ်ပါတယ်။

```
public class Student {
	private String name;

	public static void main(String [] args) {
		Student student = new Student();
	}
}
```

ဒါပေမဲ့ သေသေခြာခြာကြည့်မယ်ဆိုရင် Student Class ထဲမှာ Student() ဆိုတဲ့ Constructor ကို ရေးမထားပါဘူး။ ဒါပေမဲ့ new Student() ဆိုပြီး ရေးလို့ရနေပါတယ်။ ဒါဟာ Student Class ကို Compile လုပ်တဲ့အခါမှာ Java Compiler ကနေ Constructor မပါတဲ့ Class တွေဆိုရင် Default Constructor တစ်ခုကို ထည့်ပေးနေလို့ပဲ ဖြစ်ပါတယ်။

```
	public Student() {
		super();
	}
```

Default Constructor တွေဟာ public ဖြစ်ပြီး Argument မပါပါဘူး။ ပြီးတော့ အတွင်းပိုင်းမှာ super() ဆိုတဲ့ ကုဒ်တစ်ကြောင်းကို ရေးသားထားမှာ ဖြစ်ပါတယ်။ တကယ်လို့သာ မိမိတို့ရေးသားထားတဲ့ Class အတွင်းမှာ Constructor တစ်ခုခုကို ရေးသားထားပြီဆိုရင်တော့ Compiler ကနေ Default Constructor ကို ထည့်ပေးတော့မှာ မဟုတ်ပါဘူး။

Constructor ကိုဘာလို့ရေးရတာလဲ

Constructor တွေဟာ Class ကနေ Object ကို တည်ဆောက်တဲ့အခါမှာ မဖြစ်မနေခေါ်ဆိုရမည့် Special Method တွေဖြစ်ပြီးတော့ Instance Varable တွေနဲ့ Instance Method တွေကို အသုံးပြုနိုင်ပါတယ်။ ဒါ့ကြောင့် Object တွေရဲ့ Initial State တွေကို သတ်မှတ်လိုတဲ့အခါမှာ ကိုယ်တိုင် Constructor ကို ရေးသားဖို့စဉ်းစားရပါတော့မယ်။

```
public class Student {
      private String name;

      public Student() {
            this.name = “Aung Aung”;
      }
}
```

ဆောက်သမျှ Object တိုင်းရဲ့ name ဟာ Aung Aung ဖြစ်သွားမယ်လို့ ပြောစရာရှိပေမဲ့၊ အထက်ပါ နမူနာထဲမှာလို Student ရဲ့ name ကို Initialize လုပ်လိုတဲ့အခါမျိုးမှာ Conatructor ကို အသုံးပြုနိုင်ပါတယ်။ Object မဆောက်မီ Instance Variable တွေကို Initialize လုပ်ဖို့အတွက် Instance Block တွေကိုလဲ အသုံးပြုနိုင်ပါတယ်။

```
public class Student {
      private String name;

      {
            this.name = “Aung Aung”;
      }
}
```

Instance Block တွေဟာလဲ Object တွေကို စတင်တည်ဆောက်စဉ် Constructor တွေကို အလုပ်မလုပ်ခင် အလုပ်လုပ်နိုင်ပါတယ်။ အဲ့ဒီအတွက် Instance Variable တွေကို Initialize လုပ်တာတွေကို ရေးသားနိုင်ပါတယ်။ ဒါဖြင့် Constructor နဲ့ Instance တွေကို ဘယ်လို ရွေးချယ် အသုံးပြုရမလဲ။

```
public class Student {
      private String name;

      public Student(String name) {
            this.name = name;
      }
}
```

Constructor တွေက Initialize လုပ်လိုတဲ့ တန်ဖိုးတွေကို Constructor Argument တွေကလက်ခံရယူနိုင်ပေမဲ့ Instance Block တွေကတော့ Argument တွေကို လက်ခံလို့ မရနိုင်ပါဘူး။

Constructor Overloading

Class တစ်ခုအတွင်းမှာ Argument List မတူတဲ့ Constructor တွေကို ရေးသားခြင်းကို Constructor Overloading လို့ခေါ်ပါတယ်။ Constructor ဆိုတာက Class ကနေ Object ကို တည်ဆောက်တဲ့ အခါမှာ အသုံးပြုပြီး Instance Variable တွေကို Initialize လုပ်လို့တဲ့ အခါမှာ အသုံးပြုလေ့ရှိပါတယ်။

Object တွေကို တန်ဖိုးအမျိုးမျိုးပေးပြီး Object ဆောက်လိုတဲ့ အခါမှာ Constructor Overloading ကို အသုံးပြုနိုင်ပါတယ်။ Student Object မှာ name နဲ့ age ကို instance variale အနေနဲ့ ပိုင်ဆိုင်တယ် ဆိုကြပါစို့။ Student Object ကို ဘာ Argument မှ မပေးပဲတည်ဆောက်လိုတဲ့ အခါလဲရှိမယ်။ ပြီးတော့ name ကြီးပဲ ပေးပြီးဆောက်ချင်တဲ့အခါလဲ ရှိမယ်၊ age ကြီးပဲ ပေးပြီးဆောက်ချင်တဲ့ အခါလဲရှိမယ်။ name ကော age ပါပေးပြီး ဆောက်ချင်တဲ့ အခါလဲရှိပါတယ်။ ဒီလို အခါမျိုးမှာ Constructor Overloading ကို အသုံးပြုပြီး Constructor တွေကို ပြင်ထားနိုင်ပါတယ်။

```
public class Student {
      private String name;
      private int age;

      public Student() {}

      public Student(String name) {
            this.name = name;
      }

      public Student(int age) {
            this.age = age;
      }

      public Student(String name, int age) {
            this.name = name;
            this.age = age;
      }
}
```

တကယ်လို့ ဘာ Argument မှ မပေးပဲ Object ဆောက်ချင်ရင် Student s = new Student() လို့ ရေးသားနိုင်ပါတယ်။ ပြီးတော့ Name ကို ပေးပြီး Object ကို တည်ဆောက်လိုလျှင် Student s = new Student(“Aung Aung”) လို့ ရေးသားနိုင်ပါတယ်။

Constructor တွေကို ပြင်ဆင်ထားသလောက် Object တွေကို ပုံစံအမျိုးမျိုးနဲ့ တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။

Visibility and Accessibility

Java ဘာသာရပ်မှာ Class တွေအချင်းချင်း တွေ့မြင်နိုင်မှု (Visibility) နဲ့ အခြား Class တစ်ခုကနေ အခြား Class တစ်ခုရဲ့ Member တွေကို သုံးနိုင်သလား မသုံးနိုင်ဘူးလား (Accessibility) ကို ခွဲခြားသတ်မှတ်နိုင်အောင် ပြင်ဆင်ထားပါတယ်။

Class တွေအချင်းချင်းရဲ့ Visibility နဲ့ Accessibility ကို Package တွေနဲ့ Access Modifier တို့ကို အသုံးပြုပြီး သတ်မှတ် ရေးသားနိုင်ပါတယ်။

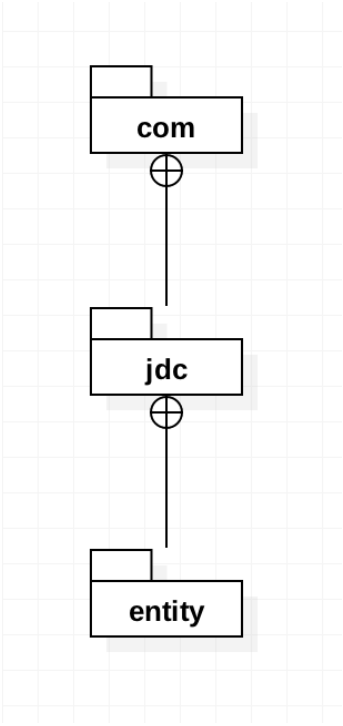
Packages

Package ဆိုတာက Java Programming ဘာသာရပ်မှာ အသုံးပြုနေတဲ့ Name Space နဲ့ Visibility ကို သတ်မှတ်ပေးနိုင်တဲ့ နည်းလမ်းတစ်မျိုးဖြစ်ပါတယ်။ အလွယ်ပြောရမယ်ဆိုရင် Packaege ဆိုတာဟာ Computer ထဲက Folder တစ်ခုနဲ့ ဆင်တူပါတယ်။ Folder တစ်ခုထဲမှာ နာမည်တူ File နှစ်ခုကို တည်ဆောက်လို့ မရနိုင်သလို Java မှာလဲ တစ်နေရာထဲမှာ Class နှစ်ခုကို တည်ဆောက်လို့ မရနိုင်ပါဘူး။

တကယ်လို့ Package သာမရှိခဲ့ရင် Java Class တွေကို နာမည်ပေးတဲ့ နေရာမှာ အတော်လေးခက်ခဲသွားမှာ ဖြစ်ပါတယ်။ ဒီလိုနေရာမျိုးမှာ Package တွေကို အသုံးပြုပြီး Class တွေကို ခွဲခြားသတ်မှတ်ပေးနိုင်ပါတယ်။ Folder မတူရင်တော့ နာမည်တူ File တွေကို ရေးသာလို့ရသလိုပဲ Package မတူရင်တော့ နာမည်တူ Java Class တွေကို ရေးသားလို့ရလာနိုင်ပါတယ်။

ဒါ့အပြင် Java Package တွေဟာ Class တွေ အချင်းချင်းရဲ့ Visibility ကိုလဲ သတ်မှတ်ပေးနိုင်ပါတယ်။

Writing a Class in a Package



Class တစ်ခုကို Package ထဲမှာ ရေးသားလိုပါက Package Statement ကို ရေးသား သတ်မှတ်နိုင်ပါတယ်။
Package Statement တွေကို ရေသားတဲ့ နေရာမှာ Source File တွေရဲ့ ထိပ်ဆုံးမှာ ရေးသားရမှာ ဖြစ်ပါတယ်။ Package Statement ရဲ့ ရှေ့မှာ Source Code မဟုတ်တဲ့ Comment တွေကို ရေးသာမယ်ဆိုရင်တော့ ရေးသားနိုင်ပါတယ်။ ဒါပေမဲ့ Source Code တွေရဲ့ ထိပ်ဆုံးမှာတော့ Package Statement ကို ရေးသားရမှာ ဖြစ်ပါတယ်။ တဖန် Source File တစ်ခုမှာလဲ Package Statement တစ်ခုကို ရေးသားရမှာ ဖြစ်ပါတယ်။

Package Statement တွေကို ရေးသားတဲ့ နေရာမှာ Reverse Domain Name ကို အသုံးပြုလေ့ရှိပြီး ရည်ရွယ်ချက်တူတဲ့ Class တွေကို Package အလိုက် ထားရှိလေ့ရှိပါတယ်။ Folder တွေထဲမှာ Sub Folder တွေကို ထားရှိနိုင်သလို Package တွေအောက်မှာလဲ Sub Package တွေကို ထားရှိနိုင်ပါတယ်။

com.jdc.entity လို့ရေးသားထားရင် com ဆိုတဲ့ package အောက်မှာ jdc package ရှိပြီး ၎င်းရဲ့ အောက်မှာ entity package ရှိတယ်လို့ မှတ်ယူနိုင်ပါတယ်။ Package တစ်ခုနဲ့ တစ်ခုအကြားမှာ [.] ကို ခံပြီး ရေးသားရမှာ ဖြစ်ပါတယ်။

```
// JDC Online Registration System
package com.jdc.entity;

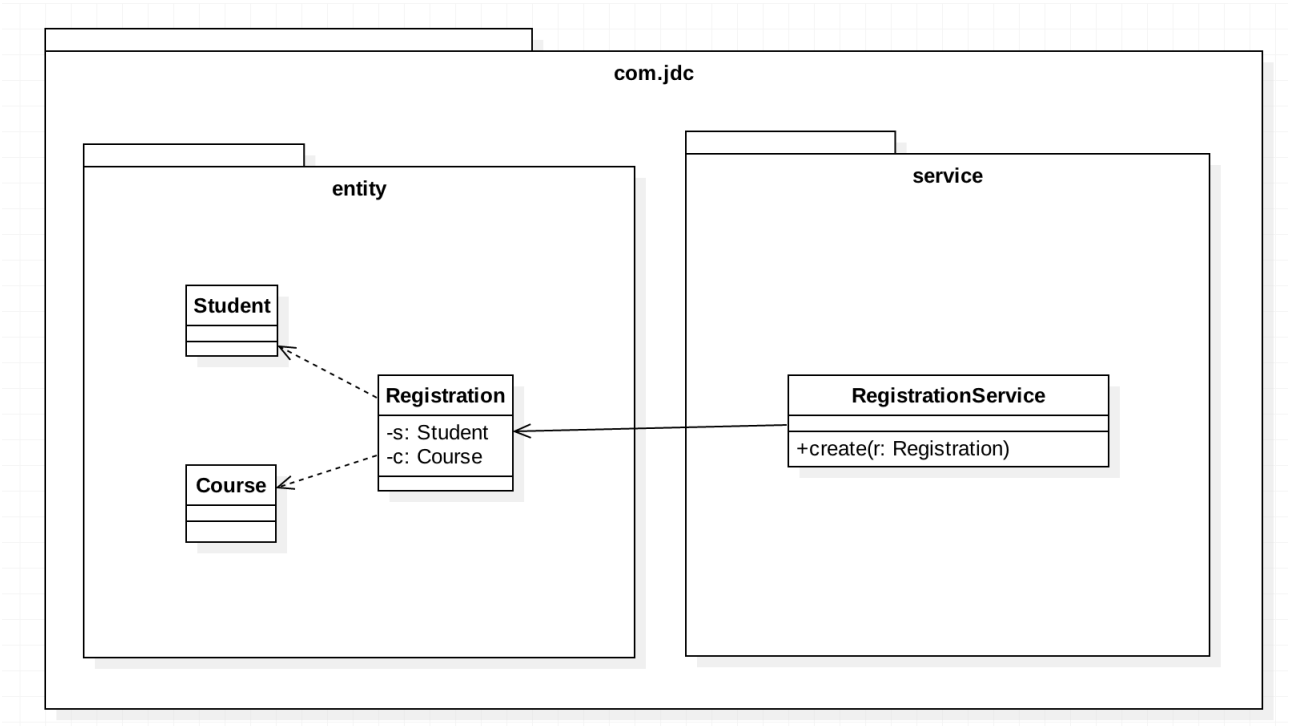
public class Student {

}
```

အထက်ပါအတိုင်း ရေးသားထားပါက com.jdc.entity Package အောက်မှာ Student Class ကို တည်ဆောက်ပေးသွားမှာ ဖြစ်ပါတယ်။

Visibility of Classes

Program တွေကို ရေးသားတဲ့ နေရာမှာ Class တွေကို တစ်ခုထက်မက အသုံးပြုပြီး ရေးသားလေ့ရှိပါတယ်။ Object တစ်ခုရဲ့ Member အနေနဲ့ အခြား Object တွေကို အသုံးပြုနေနိုင်ပါတယ်။ တနည်းဆိုရမယ် ဆိုရင် Class တစ်ခုကို တစ်ခြား Class တွေထဲမှာ Variable တွေရဲ့ Type အနေနဲ့အသုံးပြုနေနိုင်ပါတယ်။ ဒီလို အသုံးပြုတဲ့ နေရာမှာ အသုံးပြုနေတဲ့ Class ကို လက်ရှိ အသုံးပြုနေတဲ့ Class ကနေ မြင်နေမှသာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။



အထက်ပါ Diagram ထဲမှာ com.jdc Package အောက်မှာ entity နဲ့ service Package တို့ရှိပါတယ်။ တဖန် entity Package ထဲမှာ Student, Course နဲ့ Registration Class တို့ရှိကြပြီး service Package ထဲမှာ RegistrationService Class တို့ရှိကြပါတယ်။

Package တွေဟာ Class တွေရဲ့ Visibility ကို သတ်မှတ်ပေးနိုင်တာ ဖြစ်တဲ့ အတွက် Package မတူတော့တာနဲ့ Class တွေရဲ့ Visibility ကလဲ ကွဲပြားသွားမှာ ဖြစ်ပါတယ်။ Package တစ်ခုအတွင်းမှာရှိမယ်ဆိုရင် Class တွေဟာ အချင်းချင်းမြင်နရမှာ ဖြစ်တဲ့အတွက် Registration ထဲကနေ Student နဲ့ Course တို့ကို မြင်နေရမှာ ဖြစ်ပါတယ်။

```
package com.jdc.entity;

public class Registration {
	private Student student;
	private Course course;
}
```

Registration, Student နဲ့ Course တွေဟာ Package တစ်ခုအတွင်းမှာရှိတဲ့ အတွက် Student နဲ့ Course တို့ကို Registration ထဲမှာ ဒီအတိုင်းပဲ ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

```
package com.jdc.service;

public class RegistrationService {

	public void create(Registration r) {
		// codes
	}
}
```

RegistrationService ဟာ Registration Class ကို Method Argument အနေနဲ့ အသုံးပြုလိုပါတယ်။ ဒါပေမဲ့ RegistrationService ဟာ com.jdc.service Package အောက်မှာရှိပြီး Registration ဟာ com.jdc.entity Package အောက်မှာရှိတဲ့ အတွက် မြင်မနေနိုင်ပါဘူး။ တစ်ခြား Package အောက်က Class တွေကို အသုံးပြုလိုပါက Class တွေကို Qualified Name နဲ့ ရေးသား အသုံးပြုနိုင်ပါတယ်။

```
package com.jdc.service;

public class RegistrationService {

	public void create(com.jdc.entity.Registration r) {
		// codes
	}
}
```

Qualified Name ကို ရေးသားတဲ့ နေရာမှာ Package Name နဲ့ Class Name တို့ကို တွဲပြီး ရေးသားရမှာ ဖြစ်ပါတယ်။ Qualified Name ကိုရေးတဲ့နေရာမှာ Type ကို ရေးတဲ့ နေရာတိုင်း Package Name တွေပါ ရေးနေရမှာ ဖြစ်တဲ့ အတွက် ကုဒ်တွေများလာလေလေ Package Name တွေများလာပြီး ကုဒ်တွေကို ဖတ်ရတာ လွယ်ကူမှာ မဟုတ်တော့ပါဘူး။

တစ်ခြား Package ထဲမှာ ရှိတဲ့ Class တွေကို Packate တစ်ခုထဲမှာ ရှိသကဲ့သို့ အသုံးပြုလိုပါက Import Statement တွေကို ရေးသား အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Import Statements

Import Statement တွေကို Package Statement ရဲ့ အောက် Class Declaration ရဲ့ အပေါ်မှာ ရေးသားရမှာ ဖြစ်ပါတယ်။ Import Statement တွေကို Class ရဲ့ အောက်မှာတို့ Class ထဲမှာတို့ ရေးလို့ မရပါဘူး။ Import လုပ်လိုတဲ့ Class ကို Qualified နဲ့ ရေးသားရမှာ ဖြစ်ပြီး၊ Import လုပ်ပြီးတာနဲ့ Package Name ရေးစရာမလိုတော့ပဲ Class Name နဲ့ Type တွေကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ Import Statement တွေကို ရေးသားတဲ့ နေရာမှာ import [Qualified Name] လို့ ရေးသားရမှာ ဖြစ်ပါတယ်။

```
package com.jdc.service;

import com.jdc.entity.Registration;

public class RegistrationService {

	public void create(Registration r) {
		// codes
	}
}
```

အထက်ပါအတိုင်း Registration Class ကို Import လုပ်ထားပါက Package Name ကို မရေးပဲ Registration Class ကို Package တစ်ခုထဲမှာရှိတဲ့ အတိုင်း ရေးသားလို့ရမှာ ဖြစ်တယ်။ အခြားနေရာမှာ Type အဖြစ်သုံးမယ်ဆိုရင်လဲ ရေးစရာလိုတော့ပါဘူး။တဖန် Package တစ်ခုထဲက Class တွေကို Import လုပ်ချင်တယ်ဆိုရင် Wild Card (*) ကို အသုံးပြုပြီးလဲ Import လုပ်နိုင်မှာ ဖြစ်ပါတယ်။

```
package com.jdc.service;

import com.jdc.entity.*;

public class StudentService {

	public void create(Student student) {
		// codes
	}

	public List<Registration> findRegistration(Student s) {
		// codes and return statements
	}
}
```

အထက်ပါအတိုင်း com.jdc.entity.* လို့ Wild Card ကို အသုံးပြုပြီး Import လုပ်ထားမယ်ဆိုရင် com.jdc.entity Package အောက်က Class တွေအားလုံးကို Import လုပ်နိုင်မှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ တစ်ခုသတိထားဖို့ လိုတာက Wild Card Import ဟာ Current Package အောက်မှာရှိတဲ့ Class တွေကိုသာ Import လုပ်နိုင်တာဖြစ်ပြီး Sub Package အောက်က Class တွေကိုတော့ Import လုပ်နိုင်မှာ မဟုတ်ပါဘူး။

Import လုပ်စရာမလိုသော Class များ

ကျွန်တော်တို့ ရှေ့မှာ Type အနေနဲ့ String တို့ System.out.println တို့ ကို ရေးသားခဲ့ကြပါတယ်။ ဒီနေရာမှာ မေးစရာရှိလာပါတယ်။ အခဲ့ဒီ String တို့ System တို့ကကော တစ်ခြား Package ထဲမှာ မရှိလို့လား လို့။ ဟုတ်ပါတယ် String တို့ System တို့ဟာ java.lang Package အောက်က Class တွေဖြစ်ပါတယ်။

Java Language ကို ရေးသားတဲ့နေရာမှာ အသုံးများတဲ့ Class တွေ Interface တွေ Enum တွေကို java.lang Package အောက်မှာ ရေးသားထားကြပါတယ်။ ပြီးတော့ java.lang Package အောက်က Class တွေကို Import လုပ်စရာမလိုပဲ ရေးသားနိုင်အောင် ခြွင်းချက် အနေနဲ့ ထားပေးထားပါတယ်။ ဒါ့ကြောင့် String တို့ System တို့ကို ရေးသားတဲ့ နေရာမှာ Import လုပ်စရာမလိုပဲ ရေးသားနိုင်တာ ဖြစ်ပါတယ်။

Primitive Type ရဲ့ Wrapper Class တွေ၊ String နဲ့ StringBuffer, StringBuilder Class တွေ၊ အသုံးများတဲ့ Exception Class တွေကို java.lang Package အောက်မှာထားရှိတဲ့ အတွက် အခဲ့ဒီ Class တွေကို အသုံးပြုမယ် ဆိုရင် Import လုပ်စရာမလိုအပ်ပါဘူး။

Static Import

Java SE 5 အရောက်မှာ Static Variable တွေနဲ့ Static Method တွေကိုလဲ Static Import Statement ကို အသုံးပြုပြီး Import လုပ်လို့ရလာပါတယ်။ import static [QualifiedName of Class].[member] ဆိုပြီး ရေးသားရမှာ ဖြစ်ပါတယ်။ Static Import Statement တွေမှာလဲ Wild Card ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

```
import static java.lang.Integer.parseInt;
import static java.lang.Integer.MAX_VALUE;
import static java.lang.Double.*;

public class StaticImportDemo {

      public static void main(String [] args) {

            int maxInt = MAX_VALUE;
            int psInt = parseInt("100");
            double maxDouble = MAX_VALUE;

            System.out.println(MAX_VALUE);
      }
}
```

Static Member တွေကို Import လုပ်ထားခြင်းအားဖြင့် တစ်ခြား Class ထဲမှာရှိတဲ့ Static Variable တွေနဲ့ Static Method တွေကို မိမိ Class ထဲက Member တွေလို Owner ဖြစ်တဲ့ Class Name ကို ရေးစရာမလိုပဲ ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။ အထက်ပါ နမူနာထဲမှာလို Integer.MAX_VALUE နဲ့ Wild Card နဲ့ Import လုပ်ထားတဲ့ Double.MAX_VALUE တို့ရှိခဲ့မယ်ဆိုရင် အတိအကျ Import လုပ်ထားတဲ့ Integer.MAX_VALUE ကို အသုံးပြုသွားမှာ ပါတယ်။ double maxDouble = MAX_VALUE လို့ double variable မှာ အစားထိုးမယ်ဆိုရင်တော့ Wild Card ထဲက Double.MAX_VALUE ကို အသုံးပြုမှာ ဖြစ်ပါတယ်။

Access Modifiers

Class တွေနဲ့ Class Members တွေရဲ့ Access လုပ်နိုင်ခွင့်ကို သတ်မှတ်ပေးနေတာကတော့ Access Modifier တွေပဲ ဖြစ်ကြပါတယ်။ Class တွေနဲ့ Member တွေမှာ Access Modifier တွေကို မရေးသားထားရင် အဲဒီ Class တည်ရှိနေတဲ့ Package ထဲမှာသာ အသုံးပြုလို့ရမှာ ဖြစ်ပါတယ်။

တကယ်လို့ တစ်ခြား Package တွေကပါ အသုံးပြုစေချင်တဲ့အခါ ဒါမှမဟုတ် Package တစ်ခုထဲမှာ ရှိတဲ့ Class တွေ အချင်းချင်း Access လုပ်ပိုင်ခွင့်ကို သတ်မှတ်လိုတဲ့ အခါမျိုးမှာ Access Modifier တွေကို သုံးပြီး သတ်မှတ်ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

Types of Access Modifier

Modifiers	Descriptions
default	Access Modifier အား ရေးသားထားခြင်းမရှိပါက Package အတွင်းရှိ Class များမှ ဆက်သွယ်နိုင်မည်
public	မည်သည့်နေရာမှ မဆို ဆက်သွယ် အသုံးပြုနိုင်မည်
protected	ထို Class အား Inheritance လုပ်ထားသော Class များမှ ဆက်သွယ်နိုင်မည်
private	ထို Class အတွင်းမှ တခြား ဆက်သွယ်နိုင်မည်မဟုတ်ပေ

Class တွေနဲ့ Class Member တွေဖြစ်ကြတဲ့ Variable တွေ၊ Method တွေ၊ Constructor တွေနဲ့ အခြား Nested Member တွေမှာ Access Modifier ကို ရေးသားပြီး အသုံးပြုပိုင်ခွင့်ကို သတ်မှတ်ပေးနိုင်ပါတယ်။ ဘာကြောင့် Access လုပ်ပိုင်ခွင့်ကို လိုအပ်ရတာလဲ ဘယ်လိုရေးသားသင့်သလဲနဲ့ ပတ်သက်ပြီးတော့ OOP ရဲ့ Encapsulation အကြောင်းပြောတဲ့ အခါမှာလဲ ထပ်မံဖော်ပြသွားမှာ ဖြစ်ပါတယ်။

Default Access

Class တွေနဲ့ Class Members တွေမှာ Access Modifier တွေကို ရေးသားထားခြင်း မရှိပါက Default Access ကို ရရှိမှာ ဖြစ်ပါတယ်။ Package တစ်ခုအတွင်းမှာ ရှိတဲ့ အခြား Class တွေကနေ ဆက်သွယ် အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ အခြား Package တွေရဲ့ Class တွေကနေတော့ မြင်ရမှာ မဟုတ်တဲ့ အတွက် အသုံးပြုခွင့် ရှိမှာ မဟုတ်ပါဘူး။

```
package com.jdc.common;

class StringConverter {

	String getUnicode(String str) {
		// codes and return statement
	}
}
```

အထက်ပါအတိုင်း StringConverter Class မှာ Access Modifier ကို ရေးမထားပါက Default Access ဖြစ်နေပြီး com.jdc.common Package အတွင်းမှာရှိတဲ့ အခြား Class တွေကသာ အသုံးပြုနိုင်မှာ ရှိပါတယ်။ ဒီလို

Default Access တွေကို Package တစ်ခုထဲက အခြား Class တွေကသာ အသုံးပြုစေလိုတဲ့ အခါမှာ ရေးသားလေ့ ရှိပါတယ်။

Public Access

Class တွေနဲ့ Class Members တွေမှာ Public Access Modifier ကို အသုံးပြုပြီးရေးသားထားရင်တော့ အဲ့ဒါ Class တွေ Member တွေဟာ Public Access ကို ရရှိမှာ ဖြစ်ပါတယ်။ Public Access ဆိုတာကတော့ မည်သည့် Class က မဆို Access လုပ်ပိုင်ခွင့်ရရှိတာကို ဆိုလိုတာပါ။

```
package com.jdc.common;

public class StringConverter {

	public String getUnicode(String str) {
		// codes and return statement
	}
}
```

အထက်ပါအတိုင်း StringConverter Class နဲ့ getUnicode() Method မှာ public Access Modifier ကို public Access Modifier ကို ရေးသားထားတဲ့ အတွက် အခြား Package တွေကနေလဲ Access လုပ်ပြီး အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

တကယ်လို့ getUnicode() Method ကို public လုပ်ထားပြီး StringConverter Class ကို Default Access အတိုင်းထားမယ်ဆိုရင်တော့ Class ကို မမြင်ရတဲ့ အတွက် Method ကို အသုံးပြုနိုင်မှာ မဟုတ်ပါဘူး။ Class ထဲက Member တွေကို အသုံးပြုနိုင်ဖို့အတွက် Class ကိုလဲ အသုံးပြုနိုင်မှရမှာ ဖြစ်ပါတယ်။

```
package com.jdc.repository;

import com.jdc.common.StringConverter;

public class MessageRepository {

	private StringConverter converter;

	public void create(Message obj) {
		String zawgyi = obj.getMessage();
		String unicode = converter.getUnicode(zawgyi);
		obj.setMessage(unicode);

		// Other Codes
	}
}
```

အထက်ပါအတိုင်း StringConverter နဲ့ getUnicode method တို့ကို တစ်ခြား Package ထဲမှာရှိတဲ့ MessageRepository ထဲကနေ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Protected Access

Class တစ်ခုရဲ့ Member တွေကို Package တစ်ခုထဲမှာ ရှိတဲ့ Class တွေနဲ့ Sub Classs တွေကသာ အသုံးပြုစေချင်တယ်ဆိုရင် Proteced Access Modifier ကို အသုံးပြုနိုင်ပါတယ်။

```
package com.jdc;

public class Shape {

	protected double width;
	protected double height;

	public Shape(double w, double h) {
		width = w;
		height = h;
	}

	protected double getArea() {
		return width * height;
	}
}
```

အထက်ပါအတိုင်း Shape Class ထဲမှာ width နဲ့ height ကို protected Access Modifier နဲ့ ရေးသားထားတဲ့အတွက် တစ်ခြား Package ထဲမှာရှိတဲ့ Sub Class တွေကပါ width နဲ့ height ကို အသုံးပြုလာနိုင်မှာ ဖြစ်ပါတယ်။

```
package com.jdc;

public class ShapeTest {

	public static void main(String[] args) {
		Shape shape = new Shape(100, 50);
		System.out.println(shape.width);
		System.out.println(shape.height);
		System.out.println(shape.getArea());
	}
}
```

အထက်ပါနမူနာထဲမှာလို ShapeTest နဲ့ Shape တို့ဟာ com.jdc ဆိုတဲ့ Package ထဲမှာ အတူရှိတဲ့ အတွက် shape object ကနေ Protected Member တွေ ဖြစ်ကြတဲ့ shape.width နဲ့ shape.height တို့ကို အသုံးပြုလို့ရနေတာ ဖြစ်ပါတယ်။

Protected Method တွေဆိုရင်လဲ အလားတူပါပဲ။ Package တစ်ခုထဲမှာဆိုရင် Access လုပ်လို့ရနေမှာပါ။ ဒါပေမဲ့ Package မတူတော့ဘူးဆိုတာနဲ့ ဒီအတိုင်းဆိုရင် Protected Member တွေကို Access လုပ်လို့ရမှာ မဟုတ်တော့ပါဘူး။ Protected Member တွေကို Access လုပ်လိုလျင် Shape Class ကို Extends လုပ်ထားမှသာရရှိမှာ ဖြစ်ပါတယ်။

```
package com.jdc.other;

import com.jdc.Shape;

public class OtherShape extends Shape {

	public OtherShape(double width, double height) {
		super(width, height);
	}

	public static void main(String[] args) {
		OtherShape other = new OtherShape(100, 50);
		System.out.println(other.height);
		System.out.println(other.width);
		System.out.println(other.getArea());
	}
}
```

OtherShape Class ဟာ Shape Class နဲ့ Package မတူပါဘူး။ ဒါပေမဲ့ OtherShape ဟာ Shape Class ကို Extends လုပ်ထားပါတယ်။ ဒါ့ကြောင့် OtherShape Object ဖြစ်တဲ့ other ကနေ height ကိုကော with ကိုပါ Access လုပ်နိုင်ပြီး getArea() Method ကိုလဲ အသုံးပြုနိုင်တာ ဖြစ်ပါတယ်။

Protected Modifier တွေဟာ Package အတွင်းမှာလဲ အသုံးပြုနိုင်သလို Package အပြင်မှာရှိတဲ့ Sub Class တွေကနေတဆင့်လဲ အသုံးပြုနိုင်အောင် ဆောင်ရွက်ပေးနိုင်ပါတယ်။ ဒါ့ကြောင့် Protected Access Modifier တွေကို Package ++ Access Modifier လို့လဲ ခေါ်ဆိုနိုင်တာ ဖြစ်ပါတယ်။

Private Access

လက်ရှိ Class တစ်ခုထဲကသာ အသုံးပြုနိုင်တဲ့ Member တွေကို ရေးသားလိုတဲ့ အခါမျိုးမှာ Private Access Modifier ကို အသုံးပြုရေးသားနိုင်ပါတယ်။ Class Member တွေမှာ private လို့ရေးလိုက်တာနဲ့ အဲ့ဒီ Member တွေဟာ လက်ရှိ Class ထဲကသာ အသုံးပြုနိုင်တော့မှာ ဖြစ်ပါတယ်။

```
package com.jdc;

public class Shape {

	private double width;
	private double height;

	public Shape(double w, double h) {
		width = w;
		height = h;
	}
}
```

Java Data Types

Java Data Type ဆိုသည်မှာ Variable အနေနဲ့ အသုံးပြုနိုင်တဲ့ Data Type အမျိုးအစားများ ဖြစ်ကြပါတယ်။ Java Data Type များကို အခြေခံအားဖြင့် နှစ်မျိုးခွဲခြားနိုင်ပါတယ်။ အဲ့ဒါတွေကတော့ Primitive Data Type နဲ့ Reference Data Type တို့ ဖြစ်ကြပါတယ်။

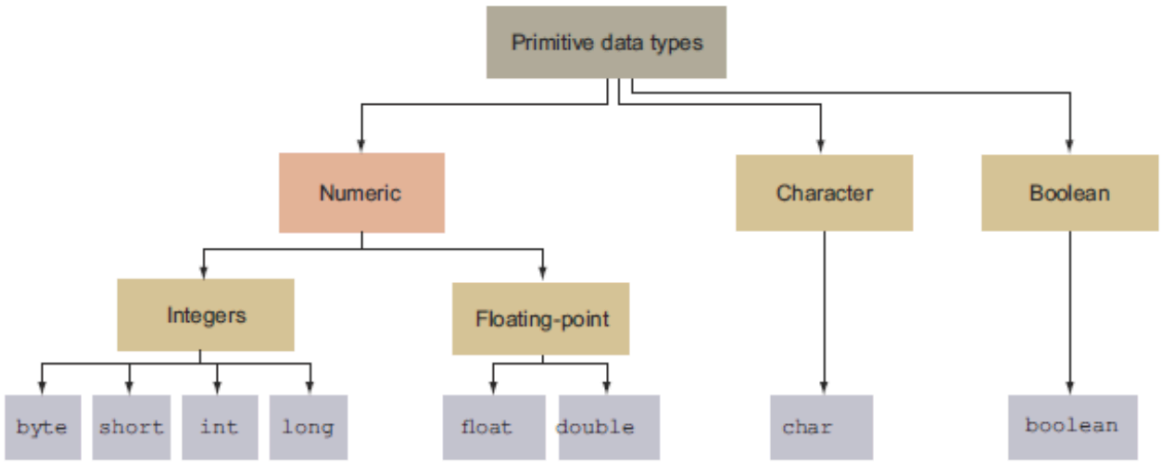
အသုံးပြုနေတဲ့ Data Type အပေါ်မူတည်ပြီးတော့ Varaible တွေရဲ့ အလုပ်လုပ်ပုံက တူမှာ မဟုတ်ပါဘူး။ ဆက်လက်ပြီး Java Data Type အမျိုးအစားတွေနဲ့ အလုပ်လုပ်ပုံတွေကို လေ့လာသွားကြရအောင်။

Primitive Data Types

Primitive Data Type တွေကတော့ Java Data Type တွေထဲမှာ အခြေခံအကျဆုံး Data Type တွေဖြစ်ကြပါတယ်။ Program တွေဆိုတာက OS ကနေ တဆင့် CPU ကို ဟိုဟာ လုပ်ပါ၊ ဒီဟာ လုပ်ပါဆိုင်း စေနေတာပဲဖြစ်ပါတယ်။ အဲ့ဒီအတွက် OS တွေကနေ Programming Language တွေမှာ အသုံးပြုဖို့ဆိုပြီး Data Type တွေကို ပြင်ပေးထားပါတယ်။

Java မှာပြောတဲ့ Primitive Data Type တွေဆိုတာက OS ကနေ ပြင်ပေးထားတဲ့ Data Type တွေ ဖြစ်ပါတယ်။ ဒီလို ကနဦး Data Type တွေကို Primitive Data Type တွေလို့ ခေါ်တာ ဖြစ်ပါတယ်။

Primitive Data အမျိုးအစားများ



အခြေခံအားဖြင့် Primitive Data အမျိုးအစား ၃ မျိုးရှိပါတယ်။ True နဲ့ False ကိုပဲ ဖေါ်ပြနိုင်တဲ့ Boolean ၁ အမျိုးအစား၊ စာလုံးတွေကို ကိုယ်စားပြုတဲ့ Character အမျိုးအစားတွေ နဲ့ Number တွေပဲ ဖြစ်ပါတယ်။ Number တွေကိုမှထပ်ပြီး အပြည့်ကိန်း Integer တွေနဲ့ ဒသမကိန်း Floating Number တွေဆိုပြီး ခွဲခြားနိုင်ပါတယ်။

Integer တွေအထဲမှာမှ အသုံးပြုနေတဲ့ Memory အလိုက် byte, short, int, long ဆိုပြီးခွဲခြားပြီး၊ ဒသမကိန်းတွေကိုလဲ float နဲ့ double ဆိုပြီး ခွဲခြားပါတယ်။

ဒါ့ကြောင့် Java မှာ အသုံးပြုနေတဲ့ Primitive Data Type အားလုံးပေါင် ၈ မျိုးရှိတယ်လို့ ပြောနိုင်မှာ ဖြစ်ပါတယ်။

Type	Description	Type	Description
byte	8bit, + or - Integer	float	32bit, + or - fp value
short	16bit, + or - Integer	double	64bit, + or - fp value
int	32bit, + or - Integer	char	16bit, Unicode Character
long	64bit, + or - Integer	boolean	true or false

Number တွေထဲမှာ Integer အနေနဲ့ byte, short, int, long တို့ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ကွာခြားတာကတော့ Memory ပေါ်မှာ အသုံးပြုနိုင်တဲ့ Size ပါပဲ။ အထက်ပါဇယားမှာ ဖေါ်ပြထားတဲ့အတိုင်း byte တွေဆိုရင် 8 bit + or - Integer တွေကို အသုံးပြုနိုင်မှာ ဖြစ်ပြီး long type ဆိုရင် + or - 64 bit Integer တွေကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Floating Number တွေဆိုရင်တော့ float နဲ့ double type တွေကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ float ကတော့ 32 bit + or - Floating Number တွေကို အသုံးပြုနိုင်မှာ ဖြစ်ပြီး double ဆိုရင်တော့ 64 bit + or - Floating Number တွေကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Character တွေကို ကိုယ်စားပြုတဲ့ char type တွေမှာတော့ 16 bit Unicode Character တွေကို အသုံးပြုနိုင်ပါတယ်။ တဖန် True နဲ့ False သာပိုင်တဲ့ boolean type တွေမှာတော့ true နဲ့ false ကိုသာ အသုံးပြုနိုင်ပါမယ်။

Literals

Java Source File တွေထဲမှာ Source Code တွေ၊ Comment တွေနဲ့ Variable တွေရဲ့ Value တွေကို ရေးသားနိုင်ပါတယ်။ Variable တွေကို Initialize လုပ်တဲ့အခါ ဒါမှမဟုတ် Variable တစ်ခုရဲ့ တန်ဖိုးလို အစားထိုးတဲ့ အခါမျိုးမှာ Source Code တွေထဲမှာ Value တွေကို တိုက်ရိုက်ရေးသားလေ့ရှိပါတယ်။ Source Code တွေထဲမှာ တိုက်ရိုက်ရေးသားထားတဲ့ Value တွေကို Literals တွေလို့ခေါ်ပါတယ်။ Primitive Type တွေမှာ ကိုယ်ပိုင် Literals တွေကို ပိုင်ဆိုင်ကြပါတယ်။

```
package com.jdc;

public class LiteralDemo {

	public static void main(String [] args) {
		boolean b = true;
		char c    = 'a';
		byte d    = 10;
		short e   = 100;
		int f     = 1000;
		long l    = 10000;
		float m   = 10.01;
		double n  = 100.01918;
	}
}
```

Boolean Literals

Boolean Type Variable တွေမှာ အသုံးပြုနိုင်တဲ့ literals တွေကတော့ true နဲ့ false သာရှိပါတယ်။

```
package com.jdc;

public class LiteralDemo {

	public static void main(String [] args) {
		boolean yes = true;
		boolean no = false;
	}
}
```

Character Literals

Character တွေဆိုရင်တော့ Single Quotation[‘] နဲ့ အစာဆုံးကို ပိတ်ပြီး ရေးသားပေးရပါမယ်။ အထက်ဖော်ပြပါ အတိုင်း ‘a’ ဆိုပြီး Character တစ်လုံးကို ရေးသားပေးရမှာ ဖြစ်ပါတယ်။ Character တွေကို ရေသားတဲ့ နေရာမှာ Unicode Code Point ကို အသုံးပြုပြီးလဲ ရေးသားနိုင်ပါတယ်။ ‘\u1018’ ဆိုပြီး ရေးသားနိုင်ပါတယ်။

```
package com.jdc;

public class LiteralDemo {

	public static void main(String [] args) {
		char a = ‘a’;
		char ba = ‘\u1018’;
	}
}
```

Numbers Literals

Integer တွေဖြစ်ကြတဲ့ အပြည့်ကိန်းတွေမှာ ဆိုရင် Decimal, Hexadecimal, Octal နဲ့ Binary ပုံစံနဲ့ တန်ဖိုးတွေကို ရေးသားလို့ရနိုင်ပါတယ်။ Decimal ဆိုတာက ဆယ်ဆတိုးပုံစံနဲ့ ရေးသားနည်းပါ။ ကျွန်တော်တို့ အပြင်မှာ အသုံးပြုနေတဲ့ ပုံစံဖြစ်ပြီး အသုံးများတဲ့ ပုံစံဖြစ်ပါတယ်။

Binary ပုံစံကတော့ 0 တွေ 1 တွေနဲ့ Integer ကို ဖော်ပြတဲ့ ပုံစံဖြစ်ပါတယ်။ 1 ဟာ 1 ဖြစ်ပြီး 10 ဆိုတာက 2 ဖြစ်ပါမယ်။ တဖန် 11 ဆိုတာကတော့ 3 ကို ဖော်ပြတာ ဖြစ်ပါမယ်။ Literal တွေကို ရေးသားတဲ့ နေရာမှာ ရှေ့မှာ Prefix အနေနဲ့ 0B ဒါမှမဟုတ် 0b ကို ရေးသားပေးရပါမယ်။

```
package com.jdc;

public class LiteralDemo {

	public static void main(String [] args) {

		int binary = 0b11;		// Binary form
		int octal = 011;		// Octal form
		int decimal = 10;		// Decimal Form
		int hexadecimal = 0x1A;	// Hexadecimal form
		float f = 10.10;
		double d = 1091.0191;
	}
}
```

Octal ဆိုတာကတော့ 8 ဆတိုးပုံစံနဲ့ ဖေါ်ပြတာဖြစ်ပါတယ်။ 0 ကနေ 7 အထိရှိပြီး 10 ဆိုတာ 8 ဖြစ်ပြီး 11 ဆိုရင် 9 ဖြစ်ပါမယ်။ Octal ပုံစံနဲ့ ရေးမယ်ဆိုရင်တော့ ရှေ့မှာ Prefix အနေနဲ့ 0 ကို ရေးပေးရမှာ ဖြစ်ပါတယ်။

Hexadecimal ဆိုတာကတော့ 16 ဆတိုးကိန်းစနစ်နဲ့ ဖေါ်ပြတာဖြစ်ပါတယ်။ Hexadecimal ပုံစံမှာ ကိန်းတစ်လုံးဟာ. ၁၆ လုံးအထိဖေါ်ပြပေးနိုင်ပါတယ်။ ကိန်းတစ်နေရာအတွက် 0 မှ 9 အထိရယ် A မှ F အထိဖေါ်ပြပေးနိုင်ပါတယ်။ 0 မှ 9 ကျော်သွားပြီဆိုတာနဲ့ A ကနေ F အထိ ဖေါ်ပြပေးပါတယ်။ 10 ဆိုရင် A လို့ဖေါ်ပြပြီး 11 ဆိုရင် B လို့ ဖေါ်ပြပါမယ်။ Java မှာ Hexadecimal ပုံစံနဲ့ရေးမယ်ဆိုရင် ရှေ့မှာ Prefix အနေနဲ့ 0x ဒါမှ မဟုတ် 0X ကို ရေးပေးရပါမယ်။

Floating Number တွေအနေနဲ့ float နဲ့ double တို့ပဲရှိပါတယ်။ အဲ့ဒါတွေကိုတော့ ဒသမကိန်းရေးသလိုပဲ ရေးသားနိုင်ပါတယ်။

Suffix of Numbers

Integer Number တွေအတွက် byte, short, int, long ဆိုပြီးရှိပါတယ်။ ပြီးတော့ Floating Number တွေအတွက် float နဲ့ double ရှိပါတယ်။ Literal နဲ့ ဒီအတိုင်းရေးမယ်ဆိုရင် အဲ့ဒီ Type တွေကို ခွဲပြီး ဖေါ်မပြနိုင်ပဲ Default Type ကို အသုံးပြုသွားပါမယ်။

Integer Literal တွေနဲ့ ရေးထားမယ်ဆိုရင် Default အတိုင်းဆိုပါက int ကို အသုံးပြုသွားပါမယ်။ ပြီးတော့ Floating Number တွေဆိုရင်လဲ default အတိုင်းဆိုပါက double type ကို အသုံးပြုသွားမှာ ဖြစ်ပါတယ်။ Variable တစ်ခုမှာ Assign လုပ်တဲ့ နေရာမှာ မသိသာပေမဲ့ Method တွေကို Invoke လုပ်တဲ့ အခါမှာ Parameter တွေနေရာမှာ Literals တွေကိုရေးသားတဲ့ နေရာမှာ သတိပြုသင့်ပါတယ်။

အောက်ပါ ကုဒ်နမူနာထဲမှာလို take() method ကို Argument Type မတူအောင် Method ၄ မျိုးရေးသားထားပါတယ်။ Argument Type တွေကတော့ အသီးသီး byte, short, int, long တို့ဖြစ်ကြပါတယ်။ Method Name တူပေမဲ့ Argument Type မတူတဲ့ အတွက် သူတို့တွေဟာ Method တစ်ခုစီလို့ Java Compiler ကကော Java Runtime ကပါ မှတ်ယူနေကြပါတယ်။

```
public class TypeDemo {

    public static void main(String[] args) {
        take(10);
    }

    static void take(byte b) {
        System.out.println("Byte Value");
    }

    static void take(short s) {
        System.out.println("Short Value");
    }

    static void take(int i) {
        System.out.println("Int Value");
    }

    static void take(long l) {
        System.out.println("Long Value");
    }
}
```

တကယ်လို့ Main Method ထဲကလို take(10) လို့ရေးလိုက်ရင် ဘယ် Method ကို အလုပ်လုပ်သွားမှာလဲ။ ဒီနေရာမှာတော့ Integer Literal တွေရဲ့ Default Type ဟာ int ဖြစ်တဲ့ အတွက် int type ကို Argument အနေနဲ့ ယူတဲ့ Method ကို အလုပ်လုပ်စေမှာ ဖြစ်ပါတယ်။

တကယ်လို့ long type အနေနဲ့ အသုံးပြုချင်တယ်ဆိုရင်တော့ Literals ရဲ့ နောက်မှာ l ဒါမှမဟုတ် L ကို Suffix အနေနဲ့ ရေးသားပေးရပါမယ်။ take(10L) လို့ရေးလိုက်ရင် long Type ဖြစ်တယ်ဆိုတာကို သတ်မှတ်ပေးနိုင်တဲ့ အတွက် take(long l) Method ကို အလုပ်လုပ်စေမှာ ဖြစ်ပါတယ်။

အလားတူပဲ suffix ကို အသုံးပြုနိုင်တာက float နဲ့ double ဖြစ်ပါတယ်။ float type အတွက် suffix အနေနဲ့ f ဒါမှမဟုတ် F ကို အသုံးပြုနိုင်ပြီး double type အတွက် d နဲ့ D ကို အသုံးပြုနိုင်ပါတယ်။

ဒါဖြင့်ရင် byte, short, int မှာတော့ suffix မရှိကြပါဘူး။ ဒါဖြင့် byte ဒါမှမဟုတ် short ကို သုံးတဲ့ Method ကို Invoke လုပ်ချင်ရင် ဘယ်လိုလုပ်မလဲ။ int ကို အသုံးလိုတဲ့ Type ကို ပြောင်းဖို့လိုအပ်ပါတယ်။ ဒါကို Type Casting လို့ခေါ်ပါတယ်။

Primitive Casting

Size သေးတဲ့ Type ကနေ Size ကြီးတဲ့ Type ကို ပြောင်းချင်ရင် Java Compiler က အလိုအလျောက်ပြောင်းပေးနိုင်တယ်။ အဲ့ဒါကို Implesit Casting လို့ခေါ်ပါတယ်။ ဥပမာအားဖြင့် byte variable တစ်ခုကို int variable ဆီကို အစားထိုးတယ် ဆိုကြပါစို့။ အဲ့ဒါဆိုရင် ဘာမှအထွေအထူးပြောင်းစရာမလိုပဲ ရေးသားနိုင်ပါတယ်။ ဒါပေမဲ့ int variable ကနေ byte Variable ကို Assign လုပ်မယ်ဆိုရင်တော့ (byte) ဆိုပြီး cast လုပ်ပေးဖို့လိုအပ်ပါတယ်။

```
public class Casting {

    public static void main(String[] args) {

        byte b = 10;
        int i = b;

        b = (byte) i;

        float f = 10.1;
        double d = f;

        f = (float)d;
    }
}
```

အလားတူပဲ float variable ကို double variable နေရာမှာ Assign လုပ်မယ်ဆိုရင် double က float ထက်ပိုပြီး ကြီးတဲ့ အတွက် အထွေအထူးပြောင်းပြီးရေးစရာမလိုအပ်ပါဘူး။ ဒါပေမဲ့ float variable နေရာကို double variable ကို Assign လုပ်မယ်ဆိုရင်တော့ (float) ဆိုပြီး Type Cast လုပ်ပေးမှရပါမယ်။

```
public class TypeDemo {

    public static void main(String[] args) {
        take((byte)10);
    }

    static void take(byte b) {
        System.out.println("Byte Value");
    }

    static void take(int i) {
        System.out.println("Int Value");
    }
}
```

အထက်ပါ နမူနာထဲမှာ take(10) လို့ရေးရင် take(int i) ကို ခေါ်သွားပါမယ်။ ဒါ့ကြောင့် take(byte b) method ကို Invoke လုပ်စေချင်ရင်တော့ take((byte)10) ဆိုပြီး Type Cast လုပ်ပြီးမှ take() Method ကို Invoke လုပ်ပေးဖို့ လိုအပ်ပါတယ်။

Underscore In Number Literals

ကျွန်တော်တို့ ကိန်းဂဏန်းတွေကို အရှည်ကြီး ရေးသွားမယ်ဆိုရင် အမတန်ကို ဖတ်ရခက်ပါတယ်။ ခု၊ ဆယ် ရာ ထောင် သောင်း အစရှိသဖြင့် မနဲ့ ဖတ်ယူရတာကို ကြုံဘူးကြမှာပါ။ ဒီနေရာမှာ သုံးလုံးတစ်နေရာ Comma (,) ကို ခံပြီး ရေးထားရင် ဖတ်ရတာလွယ်ကူစေပါတယ်။ 100,000,000 ဆိုတာမျိုးပေါ့။

Java SE 7 ကနေစပြီး Java မှာလဲ ကိန်းတွေကြားမှာ ဖတ်ရလွယ်အောင် ပြင်ဆင်လာခဲ့တာရှိပါတယ်။ Comma ကတော့ Java မှာ အသုံးပြုနေတဲ့ စာလုံးဖြစ်တဲ့ အတွက် အသုံးပြုလို့မရပါဘူး။ အဲ့ဒီအစား Under Score (_) ကို အသုံးပြုနိုင်ပါတယ်။ ဒါပေမဲ့ Under Score ကို သုံးလို့မရတဲ့ နေရာတွေရှိပါတယ်။

- ✓ ရှေ့ဆုံး၊ နောက်ဆုံးတွေမှာ ရေးလို့ မရပါဘူး
- ✓ Prefix တွေရဲ့ နောက်တွေ၊ Suffix တွေရဲ့ ရှေ့တွေမှာ ရေးလို့မရပါဘူး
- ✓ ဒသမ စာလုံးရဲ့ ရှေ့နောက်တွေမှာ ရေးလို့မရပါဘူး

ရေးလို့မရတဲ့ ပုံစံတွေဖြစ်ပါတယ်။

```
        public static void main(String[] args) {

            int data1 = _10_000_000;     // အရှေ့ဆုံးမှာ ရေးမရပါ
            int data2 = 10_110_110_;     // အနောက်ဆုံးမှာ ရေးမရပါ
            int data3 = 0b_11011;        // Prefix နောက်မှာ ရေးမရပါ
            long data4 = 10000_L;        // Suffix ရှေ့မှာ ရေးမရပါ
            float data5 = 10191._101;    // ဒသမကိန်း နောက်မှာ ရေးမရပါ
        }
```

ရေးလို့ရတဲ့ ပုံစံဖြစ်ပါတယ်။

```
        public static void main(String[] args) {

            int data1 = 10_000_000;
            int data2 = 10_110_110;
            int data3 = 0b11_011;
            long data4 = 10_000L;
            float data5 = 10_191.101;
        }
```

Reference Data Types

Primitive Data Type တွေအကြောင်းကို လေ့လာပြီးတဲ့ နောက်မှာ ဆက်လက်ပြီး လေ့လာသွားကြမှာကတော့ Reference Data Type ဖြစ်ပါတယ်။ Object တွေမှန်သမျှဟာ Reference Data Type တွေ ဖြစ်ကြပါတယ်။ Java မှာရှိတဲ့ Type အနေနဲ့ အသုံးပြုလို့ရတဲ့ Class တွေ၊ Interface တွေ၊ Enum တွေ အားလုံးဟာ Reference Data Type တွေဖြစ်ကြတယ်။

ကျွန်တော်တို့ နမူနာထဲမှာ အသုံးပြုနေကြတဲ့ String တို့လို Java SE API ထဲမှာပါတဲ့ Class တွေကော၊ ကျွန်တော်တို့ ကိုယ်တိုင်ရေးသားထားတဲ့ Class တွေအားလုံးဟာလဲ Reference Data Type တွေဖြစ်ကြပါတယ်။ ဒါ့ကြောင့် Java မှာ Primitive Data Type က ၈ မျိုးသာရှိပေမဲ့ Reference Data Type ကတော့ Class တွေရှိသလောက်ရှိနေမှာ ဖြစ်ပါတယ်။

ကဲဆက်လက်ပြီး ဘာလို့ Reference Data Type လို့ခေါ်တာလဲ ဆိုတာကို လေ့လာသွားကြရအောင်။

```
public class ReferenceDemo {
    public static void main(String[] args) {
        int a = 77;
        Person p = new Person();
        p.name = “Aung Aung”;
    }
}

class Person {
    Stirng name;
}
```

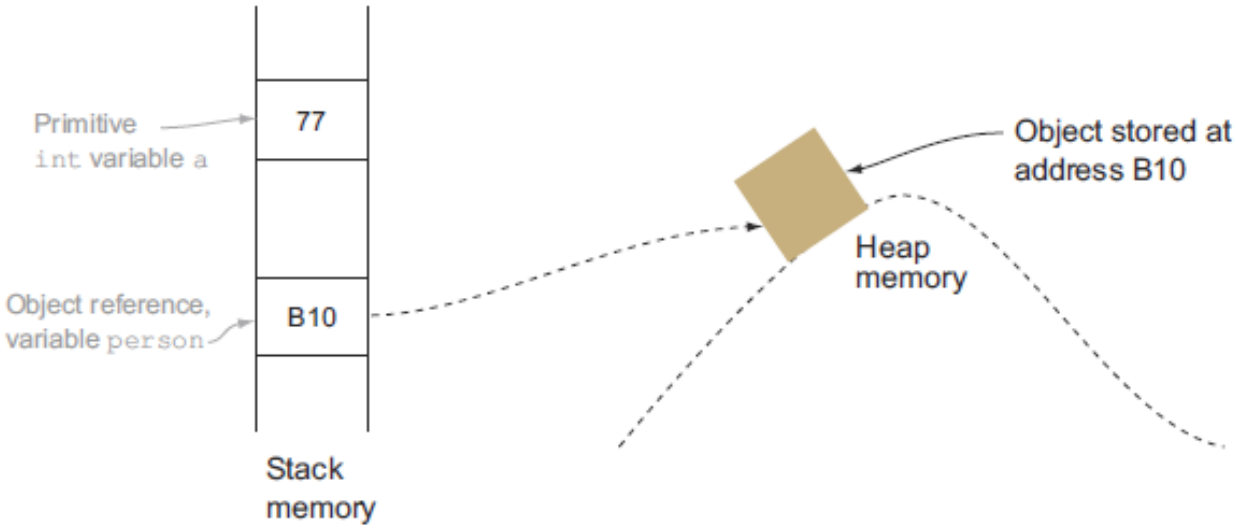
ကျွန်တော်တို့ ရှေ့မှာလဲ ရေးဘူးပါတယ်။ Person p = new Person() ဆိုတာ new Person() ဆိုပြီး Object ဆောက်ပြီး Person type p Variable နေရာမှာ လာအစားထိုးတယ်လို့ပဲ သိလာကြမှာ ဖြစ်ပါတယ်။ တကယ်တော့ Person Type p variable ကို Stack Memory ပေါ်မှာ သွားဆောက်ပါတယ်။

(Stack Memory ဆိုတာက JVM ပေါ်မှာ အသုံးပြုတဲ့ Memory Aera တစ်မျိုးဖြစ်ပါတယ်။ JMV ပေါ်မှာ Stack နဲ့ Heap ဆိုပြီး Memory Area နှစ်ခုကို အသုံးပြုကြပါတယ်။ Stack Memory ဆိုတာက CPU နဲ့ အနီးဆုံးနေရာမှာ ရှိပြီး CPU ကနေ တိုက်ရိုက် ဆက်သွယ် အသုံးပြုနိုင်တဲ့ Memory Area ဖြစ်ပြီး Life Time ကလဲ အတော်လေးကို တိုတောင်းပါတယ်။ ကျွန်တော်တို့ ရေးထားတဲ့ ကုဒ်တွေကို အလုပ်လုပ်နေစဉ် အဲ့ဒီ Code တွေဟာ Stack ပေါ်မှာ ရှိနေပါမယ်။ ပြီးသွားရင်တော့ ပျောက်ကုန်မှာ ဖြစ်ပါတယ်။ Heap ဆိုတာကတော့ Stack ထက်ပိုပြီး ကျယ်ပြန့်ပြီး CPU ကနေ တိုက်ရိုက် ဆက်သွယ်ပြီး အသုံးပြုလေ့မရှိပါဘူး။ အကြာကြီးလဲ ဒေတာတွေကို သိမ်းထားပြီး အသုံးပြုနိုင်ပါတယ်)

နောက်ကပါတဲ့ new Person() ဆိုတာကတော့ Heap Memory ပေါ်မှာ Person Object ကို သွားဆောက်တာပဲ ဖြစ်ပါတယ်။ Object ဆောက်တဲ့နေရာမှာ Instance Variable တွေရှိရင် Class ထဲကနေ Copy ကူးသွားပြီး Object ထဲမှာ သိမ်းထားပါမယ်။ ဒါကြောင့် name Variable ဟာလဲ Object ထဲကိုပါသွားပါမယ်။ ပြီးတော့မှ ဆောက်ခဲ့တဲ့ Memory Address ကို Person p နေရာမှာ လာပြီး အစားထိုးပါတယ်။

p.name ဆိုပြီး Variable p ရဲ့ name ကို အသုံးပြုပြီဆိုရင် p နေရာမှာ ရှိတဲ့ Memory Address ကနေ Object ကို Reference လုပ်ပြီး name နေရာကို အသုံးပြုမှာ ဖြစ်ပါတယ်။ ဒီလို Variable တွေကနေ Object ရှိတဲ့နေရာကို Reference လုပ်ပြီး သုံးနေရတဲ့ အတွက် Object တွေကို Reference Type Variable တွေလို့ ခေါ်ဆိုရခြင်း ဖြစ်ပါတယ်။

```
int a = 77;
Person person = new Person();
```



တကယ်တော့ Primitive တွေတုန်းက Variable နေရာကို Assign လုပ်တာနဲ့ ပတ်သက်ပြီး ဒီနေရာမှာ ပြောချင်တဲ့ အတွက် ရှေ့မှာ မပြောခဲ့ပါဘူး။ Primitive Type Variable တွေကတော့ Reference Type Variable တွေနဲ့ မတူပါဘူး။

int a = 77; လို့ရေးထားရင် Stack Memory ပေါ်မှာ a အတွက် 32 Bit စာ နေရာယူပါတယ်။ ပြီးရင် 77 ကို Value အနေနဲ့ အစားထိုးမှာ ဖြစ်ပါတယ်။ မှတ်မယ်ဆိုရင် Primitive Type Variable တွေက Variable တွေနေရာမှာ တကယ့် တန်ဖိုးကို လာအစားထိုးပြီး Reference Data Type တွေကတော့ ဆောက်ထားတဲ့ Object ကို Reference လုပ်နိုင်တဲ့ Memory Address ကို အစားထိုးတာပဲ ဖြစ်ပါတယ်။

```
public class ReferenceDemo {
    public static void main(String[] args) {
        int a = 77;
        Person p = new Person();
        p.name = "Aung Aung";

        int b = a;
        Person q = p;

        a = 100;
        p.name = "Thidar";
    }
}
```

Variable တွေကို အခြား Variable တွေမှာ အစားထိုးကြည့်ရအောင်။ int b = a; လို့ရေးလိုက်တာနဲ့ a ရဲ့ တန်ဖိုးဖြစ်တဲ့ 77 ဟာ b ရဲ့ တန်ဖိုးဖြစ်သွားပါမယ်။ ဒါ့ကြောင့် အောက်မှာ a = 100 လို့ a ရဲ့ တန်ဖိုးကို 100 လို့ အစားထိုးရင်လဲ b ရဲ့ တန်ဖိုးဟာ 77 ပဲ ဖြစ်နေမှာပါ။

ဒါပေမဲ့ Reference Type တွေကတော့ ဒီလိုမဟုတ်ပါဘူး။ Person q = p; လို့ရေးလိုက်တာဟာ q ရဲ့ တန်ဖိုးကို p ကနေ Reference လုပ်နေတဲ့ Object ရဲ့ Memory Address ကို လာပြီး အစားထိုးတာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့်

Reference Variable p ကော q ပါ တစ်ခုထဲသော Person Object ကို လှမ်းပြီး Reference လုပ်နေကြမှာ ဖြစ်ပါတယ်။ ဒါကြောင့် p.name = “Thidar” လို့ပြောင်းလိုက်တာနဲ့ q.name ဟာလဲ “Thidar” ဖြစ်သွားမှာ ဖြစ်ပါတယ်။ Reference Variable တွေကို အသုံးပြုတဲ့ နေရာမှာ တစ်ခြား Variable တွေကနေ သုံးနေတဲ့ Object ကို Reference လုပ်နေနိုင်တယ်ဆိုတာကို သတိထားဖို့လိုအပ်ပါတယ်။

ဆက်လက်ပြီး Method Argument တွေမှာ သုံးတဲ့ Variable တွေဆိုတာ ဘယ်လိုဟာမျိုးလဲ ဆိုတာကို လေ့လာသွားကြရအောင်။ Primitive Type ကနေ စပြီး လေ့လာသွားကြရအောင်။

```
public class ReferenceDemo {
    public static void main(String[] args) {

        int a = 77;
        change(a);
        System.out.println(a);
    }

    static void change(int a) {
        a = 100;
    }
}
```

အထက်မှာ ဖော်ပြထားသလို change() method က int type Variable တစ်ခုကို Argument အနေနဲ့ ယူပါတယ်။ ပြီးတော့မှ Method ထဲမှာ a = 100 ဆိုပြီး a ရဲ့ တန်ဖိုးကို ပြောင်းလိုက်ပါတယ်။ Main Method ထဲမှာတော့ change(a) ဆိုပြီး Invoke လုပ်ပါတယ်။ System.out.println(a) လို့ရေးလိုက်ရင် a ရဲ့ တန်ဖိုးဟာ ဘာဖြစ်သွားမလဲ။ Method ထဲမှာ a = 100 လို့ ပြောင်းထားတဲ့ အတွက် a ရဲ့ တန်ဖိုးဟာ 100 လို့ပြောင်းသွားတယ်လို့ ထင်ပါမယ်။ ဒါပေမဲ့ အဖြေဟာ 77 ပဲဖြစ်နေမှာ ဖြစ်ပါတယ်။

ဘာလို့လဲ ဆိုတော့ change(a) လို့ ရေးလိုက်တာဟာ Variable a ကို change() Method ကို ပေးလိုက်တာမဟုတ်ပါဘူး။ Variable a ရဲ့ value ဖြစ်တဲ့ 77 ကို change() Method ရဲ့ Argument ဖြစ်တဲ့ a နေရာကို Copy ကူးပြီး ပေးလိုက်တာ ဖြစ်ပါတယ်။ Value ကိုပဲ ပေးလိုက်တာဖြစ်ပါတယ်။ ဒါကြောင့် change() method ထဲက a ဟာ Main Method ထဲက Variable a နဲ့ သက်ဆိုင်မှုမရှိပါဘူး။ change() Method ထဲက a ရဲ့ တန်ဖိုးကို ပြောင်းလိုက်တာလဲ change() method ထဲက a သာ ပြောင်းလဲသွားတာ ဖြစ်ပြီး၊ Main Method ထဲက a ရဲ့ တန်ဖိုးဟာ နဂို အတိုင်း 77 ပဲဖြစ်နေတာ ဖြစ်ပါတယ်။

```
public class ReferenceDemo {
    public static void main(String[] args) {
        Person p = new Person();
        p.name = "Aung Aung";
        change(p);
        System.out.println(p.name);
    }

    static void change(Person p) {
        p.name = "Thidar";
    }
}
```

Reference Variable တွေမှာကော ဘယ်လိုလဲ ဆိုတာကို ဆက်ပြီးလေ့လာသွားကြရအောင်။ စောစောက အတိုင်းပါပဲ။ Person p ကို change(p) ဆိုပြီး change() Method ကို Invoke လုပ်နေပါတယ်။ ဒီနေရာမှာလဲ သတိပြုဖို့ လိုတာက change(Person p) လို့ရေးထားတာနဲ့ Main Method ထဲမှာ Person p လို့ရေးထားတာက နာမည်သာ တူတယ်၊ Scope ချင်းမတူတဲ့ အတွက် Local Variable တစ်ခုစီလို့ သိထားစေချင်ပါတယ်။

Main Method ထဲမှာ change(p) လို့ရေးလိုက်တာဟာ Main Method ထဲမှာရှိတဲ့ p variable ရဲ့ Value ကို change() method ရဲ့ local variable p မှာ Copy ကူးပေးလိုက်တာပါပဲ။ Method တွေကို Invoke လုပ်တဲ့ နေရာမှာ Variable တွေကို Parameter အနေနဲ့ ပေးလိုက်တယ်ဆိုတာ Variable ကို ပေးလိုက်တာမဟုတ်ပဲ Variable ရဲ့ Value ကိုပဲ ပေးလိုက်တာ ဖြစ်ပါတယ်။

Reference Type Variable တွေရဲ့ Value ဟာ Reference လုပ်နေတဲ့ Object တွေရဲ့ Memory Address သာ ဖြစ်တဲ့ အတွက် change() Method ထဲမှာ ရှိနေတဲ့ p ဟာလဲ Main Method ထဲက p က Reference လုပ်နေတဲ့ Object ကိုပဲ Reference လုပ်နေပါတယ်။ change() method ထဲမှာ p.name ကို ပြောင်းလိုက်တဲ့ အတွက် Main Method ထဲမှာရှိတဲ့ p.name ဟာလဲ ပြောင်းသွားမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် System.out.println(p.name) လို့ ရေးလိုက်တဲ့ အခါမှာ အဖြေဟာ “Thidar” လို့ ဖေါ်ပြသွားမှာ ဖြစ်ပါတယ်။

Wrapper Classes of Primitives

ရှေ့ခန်းမှာ Primitive Data Type တွေအကြောင်းကိုလေ့လာခဲ့တဲ့ နေရာမှာ Primitive Data Type တွေမှာ Method တွေမပါဘူးဆိုတာကို သတိပြုမိပါမယ်။ တကယ်လို့ Type တစ်ခုကနေတစ်ခုကို ပြောင်းချင်တဲ့အခါ၊ ဒါမှမဟုတ် String ကနေ တစ်ခြား Type တစ်ခုခုကို ပြောင်းချင်တဲ့အခါ၊ Primitive Type ကနေ Stiring ကိုပြောင်းချင်တဲ့ အခါမျိုးတွေမှာ အခက်အခဲ တွေ့ရတတ်ပါတယ်။ အဲ့ဒီလိုလုပ်ဆောင်မှုတွေကို အလွယ်တကူဆောင်ရွက်နိုင်အောင် Java မှာ Primitive Type တွေအတွက် Wrapper Class တွေကို ပြင်ပေးထားပါတယ်။

Primitive	Wrapper	Argument Types of Constructor
boolean	Boolean	boolean or String
char	Character	char
byte	Byte	byte or String
short	Short	short or String
int	Integer	int or String
long	Long	long or String
float	Float	float, double or String
double	Double	double or String

အထက်ပါအတိုင်း Primitive Data Type အလိုက် Wrapper Class တွေကို ပြင်ပေးထားပြီး Primitive Type မှ ဒါမှမဟုတ် String တစ်ခုကနေ Object အဖြစ်ပြောင်းနိုင်အောင် Constructor တွေကို ပြင်ပေးထားပါတယ်။

```
Integer a = new Integer(10);
Integer b = new Integer(“20”);
Float c = new Float(14.10F);
Float d = new Float(“10.8181”);
```

အထက်ပါအတိုင်း သက်ဆိုရင်ရာ Primitive အလိုက် Wrapper Object တွေကို တည်ဆောက်နိုင်ပါတယ်။

Auto Boxing - Unboxing

Java Version 5 အရောက်မှာ Auto Boxing - Unboxing ဆိုတဲ့ Feature တစ်ခုကို အသစ်ဖြည့်စွက်လာပါတယ်။ ဒါကတော့ Primitive Type တွေနဲ့ Wrapper Object တွေကြားမှာ Object ဆောက်စရာမလိုပဲ Assign လုပ်လာနိုင်သလို၊ Object တွေမှာလဲ Method ကို ခေါ်စရာမလိုပဲ Primitive အဖြစ်ပြောင်းလာနိုင်ပါတယ်။

```
int a = 10;
Integer b = new Integer(a);

int c = b.intValue();
```

Java SE 5 မတိုင်ခင်ကဆိုရင် primitive type a ကို Integer အဖြစ်ပြောင်းချင်ရင် new Integer(a) ဆိုပြီး Constructor ကိုအသုံးပြုပြီး Object အဖြစ်ပြောင်းမှရပါတယ်။ ဒီလိုမျိုး Primitive Type ကနေ သူ့ရဲ့ Wrapper Object အဖြစ်ပြောင်းတာကို Boxing လို့ခေါ်ပါတယ်။ Object ဆိုတဲ့ သေတ္တာထဲထည့်တယ်ဆိုတဲ့ သဘောပေါ့ဗျာ။

တဖန် Wrapper Object ကနေ primitive type ကို ပြန်လိုချင်ရင်လဲ b.intValue() ဆိုပြီး Method ကို အသုံးပြုပြီး Type ကို ပြန်ပြောင်းပေးဖို့လိုပါတယ်။ အဲ့ဒါကိုတော့ Unboxing လို့ခေါ်ပါတယ်။ Object ဆိုတဲ့ သေတ္တာထဲကနေ ထုတ်ပြီး Primitive ကို ပြန်ပြောင်းပေးတယ်ဆိုတဲ့ သဘောပေါ့။

ဒါပေမဲ့ Java SE 5 ရောက်တဲ့ အခါမှာ ဒါတွေဟာ မလိုအပ်တော့ပါဘူး။ Boxing တွေ Unboxing တွေကို JVM က အလိုအလျောက်လုပ်ဆောင်ပေးလာနိုင်ပါတယ်။

```
int a = 10;
Integer b = a;

int c = b;
```

အထက်ပါအတိုင်း Wrapper Object b နေရာကို Primitive Variable တစ်ခုဖြစ်တဲ့ a ကို တိုက်ရိုက် အစားထိုးနိုင်သလို၊ primitive type variable တစ်ခုဖြစ်တဲ့ c နေရာကိုလဲ int ရဲ့ Wrapper Object တစ်ခုဖြစ်တဲ့ b နဲ့ အစားထိုးလာနိုင်ပါတယ်။

Wrapper Class တွေနဲ့ ဘာလုပ်လို့ရလဲ

Primitive Wrapper တွေနဲ့ ဘာတွေလုပ်လို့ရလဲ ဆိုတာဟာ ကျွန်တော်တို့ သိထားသင့်တဲ့ အချက် ဖြစ်ပါတယ်။ Primitive Type Value တွေကို ပြန်ပြီး ထုတ်ယူလို့ရတယ်။ ဒါပေမဲ့ Auto Boxing Unboxing ကြောင့် ဒါဟာ သိပ်ပြီး အရေးမပါတော့ပါဘူး။ Variable နှစ်ခုကို Compare လုပ်လို့ရမယ်။ တန်ဖိုးတူမတူ စစ်လို့ရမယ်။ ပြီးရင် အခြား Type တွေကို ပြောင်းလို့ရမယ်။

Wrapper Class တိုင်းမှာ compare() နဲ့ equals() Method တွေပါလာပါတယ်။ ဒါပေမဲ့ ဒါဟာလဲ Primitive Type တွေနဲ့ Operators တွေကို တွဲပြီးရေးမယ်ဆိုရင် ဖြစ်နေတာဖြစ်တဲ့ အတွက် အရမ်းကို လိုအပ်နေတာတော့ မဟုတ်ပါဘူး။ ဒါပေမဲ့ တစ်ခြား Primitive Type တွေ အဖြစ်ပြောင်းလို့ရတဲ့ အချက်ကတော့ သုံးချင်စရာ အချက်တစ်ခုဖြစ်ပါတယ်။

Compare with other Variable

Number အမျိုးအစား Wrapper Type တွေမှာ compare() ဆိုတဲ့ Static Method တစ်ခုနဲ့ compareTo() ဆိုတဲ့ Instance Method တွေပါဝင်ပြီး Variable နှစ်ခုကို နှုင်းယှဉ်ရာမှာ အသုံးပြုနိုင်ပါတယ်။

```
public static void main(String[] args) {
	int a = 10;
	int b = 13;

	int result1 = Integer.compare(a, b);
	System.out.println(result1);

	Integer c = a;
	Integer d = b;

	int result2 = c.compareTo(d);
	System.out.println(result2);
}
```

Integer.compare(int a, int b) Method ကတော့ Static Method ဖြစ်ပြီး int variable နှစ်ခုကို နှုင်းယှဉ်ရာမှာ အသုံးပြုနိုင်အောင် ပြင်ဆင်ထားတဲ့ Method တစ်ခုရှိပါတယ်။ အခြားသော Number Type Class တွေမှာလဲ compare method ကို ပြင်ပေးထားပါတယ်။ Double.compare() Method ကို အသုံးပြုမယ်ဆိုရင် double variable နှစ်ခုကို Compare လုပ်နိုင်မှာ ဖြစ်ပါတယ်။ compare() Method ရဲ့ Return Type ကတော့ int ဖြစ်ပြီး variable နှစ်ခုတန်ဖိုးတူနေရင် 0 ကို Return ပြန်မှာ ဖြစ်ပြီး ပထမ Variable ကကြီးနေရင် 1 ကို Return ပြန်ပြီး၊ ပထမ Variable က ငယ်နေရင် -1 ကို Return ပြန်မှာ ဖြစ်ပါတယ်။

အထက်ပါ နမူနာထဲမှာ Integer.compare(a, b) လို့ရေးထားပြီး a ဟာ b ထက်ငယ်တဲ့ အတွက် result1 ဟာ -1 ဖြစ်ပါမယ်။ တဖန် Wrapper Object တွေဖြစ်ကြတဲ့ c နဲ့ d တွေမှာ compareTo() ဆိုတဲ့ Method ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ c ဟာ d ထက်ငယ်တဲ့ အတွက် c.compareTo(d) ရဲ့ ရလဒ်ဖြစ်တဲ့ result2 ဟာလဲ -1 ပဲဖြစ်ပါမယ်။

```
static void compare(int a, int b) {
	int result = a - b;

	String sign = result == 0 ? "=" : result > 0 ? ">" : "<";

	System.out.printf("%d %s %d%n", a, sign, b);
}
```

Compare လုပ်တဲ့ နေရာမှာ အထက်ပါအတိုင်း a - b ဆိုပြီး ရလဒ်နဲ့လဲ ကြည့်လို့ရပါတယ်။ result ဟာ 0 ဖြစ်ရင် a နဲ့ b ဟာ တူနေပြီး၊ 0 ထက်ကြီးရင် a ဟာ b ထက်ကြီးနေပါမယ်။ တဖန် result က 0 ထက်ငယ်နေရင်တော့ a ဟာ b ထက်ငယ်တယ်ဆိုပြီးလဲ သိနိုင်ပါတယ်။

တူမတူစစ်ချင်တဲ့အခါမှာ

Variable တန်ဖိုးနှစ်ခု တူမတူ စစ်ချင်တဲ့ အခါမှာ Primitive တွေမှာ Operators တွေကို အသုံးပြုပြီး စစ်လို့ရပါတယ်။ ဒါပေမဲ့ Wrapper Class တွေမှာ java.lang.Object ကနေပါလာတဲ့ equals() method ကို အသုံးပြုပြီး စစ်ဆေးနိုင်မှာ ဖြစ်ပါတယ်။

```
    public static void main(String[] args) {
          double a = 10.101;
          double b = 10.101;

          System.out.println(a == b);

          Double c = a;

          System.out.println(c.equals(b));
    }
```

အထက်ပါအတိုင်း a နဲ့ b ဟာ Primitive Type တွေဖြစ်ကြတဲ့ အတွက် a == b ဆိုပြီး တူမတူ စစ်ဆေးနိုင်ပါတယ်။ တဖန် c ဟာ Wrapper Type ဖြစ်တဲ့ အတွက် Object ကနေ ပါလာတဲ့ equals() method ကို အသုံးပြုပြီး စစ်ဆေးနိုင်ပါတယ်။ ဒီနေရာမှာ c.equals(b) လို့ရေးထားတဲ့ နေရာမှာ b ဟာ primitive type ဖြစ်ပေမဲ့ Auto Boxing - Unboxing ကြောင့် Wrapper Object အဖြစ်အလိုလိုပြောင်းပြီး စစ်ဆေးနိုင်တာ ဖြစ်ပါတယ်။

Type ပြောင်းချင်တဲ့အခါ

Primitive Type တွေမှာ Integer တွေအတွက် Type လေးမျိုးရှိပြီ” Floating Number တွေအတွက် Type နှစ်မျိုးရှိပါတယ်။ Primitive Type တွေမှာ ဆိုရင် Type တွေကို ပြောင်းချင်ရင် Type Casting ကို အသုံးပြုလို့ရပါတယ်။ ဒါပေမဲ့ Wrapper Object တွေကို အသုံးပြုမယ်ဆိုရင် Type တစ်မျိုးကနေ တစ်မျိုးကို အလွယ်တကူ ပြောင်းလို့ရပါတယ်။ Number Type Wrapper Class တွေဟာ Number ဆိုတဲ့ Abstract Class ကို Extends လုပ်ထားကြပြီး Number Class ထဲမှာ Type ပြောင်းဖို့ အောက်ပါ Method တွေကို ပြင်ဆင်ထားပါတယ်။

Modifier and Type	Method and Description
`byte`	`byteValue()` Returns the value of the specified number as a `byte`, which may involve rounding or truncation.
`abstract double`	`doubleValue()` Returns the value of the specified number as a `double`, which may involve rounding.
`abstract float`	`floatValue()` Returns the value of the specified number as a `float`, which may involve rounding.
`abstract int`	`intValue()` Returns the value of the specified number as an `int`, which may involve rounding or truncation.

`abstract long`	`longValue()` Returns the value of the specified number as a `long`, which may involve rounding or truncation.
`short`	`shortValue()` Returns the value of the specified number as a `short`, which may involve rounding or truncation.

အထက်ဖော်ပြပါ Method တွေကို Number အမျိုးအစား Wrapper Object တွေအားလုံးမှာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

```
public static void main(String[] args) {
      Double d = 10.11;
      float floatData   = d.floatValue();
      byte byteValue    = d.byteValue();
      short shortValue  = d.shortValue();
      int intData       = d.intValue();
      long longData     = d.longValue();
}
```

အထက်ပါအတိုင်း Primitive Wrapper Object တွေကို အသုံးပြုမယ်ဆိုရင် အခြား Type တွေကိုလဲ အသင့်ပြင်ထားတဲ့ Method တွေကို အသုံးပြုပြီး အလွယ်တကူပြောင်းလဲနိုင်မှာ ဖြစ်ပါတယ်။ အခြားသော Number Type Wrapper Class တွေမှာလဲ Type ပြောင်းတဲ့ Method တွေကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Arrays

ဒီအခန်းမှာတော့ Java မှာ အသုံးပြုနေတဲ့ Array ဆိုတာ ဘယ်လိုဟာမျိုးလဲ။ ပြီးတော့ Array တွေကို ဘာလို့ လိုအပ်ကြတာလဲ၊ Array Object တွေကို ဘယ်လို တည်ဆောက်ကြမလဲ၊ Array ထဲမှာပါတဲ့ Element တွေကို ဘယ်လို Access လုပ်မလဲဆိုတာနဲ့ Multi Dimension Array တွေအကြောင်းကို လေ့လာသွားကြပါမယ်။

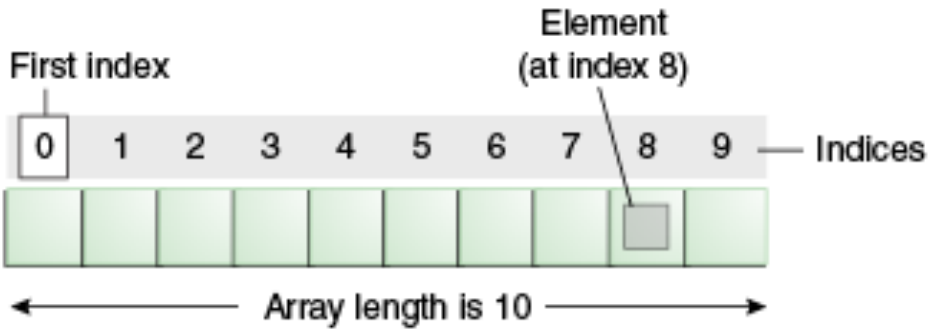
Array ဆိုတာက အမျိုးအစားတူတဲ့ Data တွေကို အစီအစဉ်တကျ သိမ်းပေးထားနိုင်တဲ့ Object တစ်မျိုးဖြစ်ပါတယ်။ ဒါပေမဲ့ Array တွေက Object တွေဖြစ်ပေမဲ့ တစ်ခြား Object တွေနဲ့ ရေးသားပုံ မတူညီကြပါဘူး။ တစ်ခြား Object တွေဆိုရင် Type တွေကို Class Name နဲ့ ရေးသားဖေါ်ပြပြီး new Operator ကို အသုံးပြုပြီး Constructor တွေကနေ တဆင့် Object တွေကို တည်ဆောက်နိုင်ပါတယ်။

ဒါပေမဲ့ Array တွေက Type ရေးသားပုံကော၊ Object တည်ဆောက်ပုံတွေပါ တစ်ခြား Object တွေနဲ့ မတူကြပါဘူး။ ဒီ အခန်းမှာတော့ Array တွေအကြောင်းကို လေ့လာသွားကြပါမယ်။

ဘာလို့ Array တွေကိုအသုံးပြုဖို့လိုအပ်ခဲ့တာလဲ

တကယ်လို့ ကျွန်တော်တို့ Twitter ပေါ်မှာရေးသားခဲ့တဲ့ Twit တွေကို ဖေါ်ပြနိုင်တဲ့ Application တစ်ခုရေးနေတယ်လို့ တွေးကြည့်ကြရအောင်။ ပထမဆုံး Twit ကိုဖေါ်ပြဖို့ String firstTweet ဆိုတဲ့ Variable နဲ့ ဖေါ်ပြမယ်၊ ဒုတိယ Twit အတွက် secondTwit ဆိုတဲ့ Variable နဲ့ ဖေါ်ပြကြမယ်။ ဒီလိုနဲ့ တတိယ Twit နောက်ဆုံး Twit တစ်ခုအတွက် Variable တစ်ခု ရေးနေရရင် Variable တွေဆုံးမှာ မဟုတ်တော့ပါဘူး။

ဒီလိုနေရာမျိုးမှာ String အမျိုးအစား Data တွေကို အစီစဉ်အလိုက် သိမ်းပေးထားနိုင်တဲ့ Object တွေလိုလာပါမယ်။ ဒီလို လိုအပ်ချက်တွေအတွက် အသုံးပြုနိုင်တာကတော့ Array Object တွေဖြစ်ကြပါတယ်။



Array တွေဟာ Type အမျိုးအစားတူတဲ့ Data တွေကို သိမ်းပေးထားနိုင်တဲ့ ခွက်လေးတွေကို တန်းစီပြီးထားတဲ့ ရထားတွဲ တစ်ခုနဲ့ တူပါတယ်။ Array တစ်ခုကို အသုံးပြုတော့မယ်ဆိုရင် ဘာ Type ကို သုံးမယ်ဆိုတာနဲ့ ဘယ်နှစ်ခု သိမ်းထားနိုင်တယ်ဆိုတာကို ကြိုပြီး သတ်မှတ်ပေးထားဖို့လိုအပ်ပါတယ်။

အစကထဲက int အမျိုးအစား Data ၁၀ ခု အသုံးပြုမယ်လို့ သတ်မှတ်ထားတယ်ဆိုရင် အဲ့ဒီ Array ထဲကို int ကလွဲပြီး သိမ်းထားလို့မရနိုင်ပါဘူး။ ပြီးတော့ Data ကိုလဲ 10 ခုထက်ပိုပြီး သိမ်းမပေးထားနိုင်ပါဘူး။ ပြီးတော့ Array ထဲမှာ သိမ်းထားတဲ့ Data တွေကိုလဲ အခန်း နံပတ်နဲ့ သီးခြား သတ်မှတ်ပြီး သိမ်းပေးထားနိုင်ပါတယ်။ ပြီးတော့ ဘယ်နံပါတ်က Data လိုချင်လဲ ဆိုတာနဲ့ ဘယ်နံပါတ်မှာ ဘယ် Data ကို သိမ်းမယ်ဆိုတာကို သတ်မှတ် အသုံးပြုနိုင်ပါတယ်။

Array မှာ အသုံးပြုနေတဲ့ အခန်း နံပါတ်တွေဟာ သုည(၀)ကနေစပါတယ်။ ဒါ့ကြောင့် Data 10 ခု သိမ်းထားနိုင်တဲ့ Array ဆိုရင် နံပါတ် 0 ကနေ 9 အထိအခန်းတွေကို ပိုင်ဆိုင်မှာ ဖြစ်ပါတယ်။

Declaring Arrays

Array တွေကို အသုံးပြုတော့မယ်ဆိုရင် အနဲဆုံး ဘယ်လိုအမျိုးအစား Array ကို အသုံးပြုမှာလဲ ဆိုတာကို သိဖို့ လိုအပ်ပါတယ်။ Array အနေနဲ့ Primitive Type တွေကိုလဲ အသုံးပြုနိုင်သလို Reference Type တွေကိုလဲ အသုံးပြုနိုင်ပါတယ်။ int type တွေကို အသုံးပြုတဲ့ Array ဆိုရင် int array လို့ခေါ်ဆိုလေ့ရှိသလို String Type တွေကို အသုံးပြုတဲ့ Array တွေကို String Array လို့လဲ ခေါ်လေ့ရှိပါတယ်။ Custom Object ဖြစ်တဲ့ User Class တွေကို အသုံးပြုနိုင်တဲ့ Array တွေဆိုရင်လဲ User Array လို့ခေါ်လေ့ရှိပါတယ်။

အခြေခံအားဖြင့် Array Type Variable တွေကို Declare လုပ်ပုံ နှစ်မျိုးရှိနိုင်ပါတယ်။ ပထမတစ်နည်းကတော့ ပါဝင်မည့် Element တွေကိုကြိုသိတဲ့ အခါရေးသားတဲ့ နည်းနဲ့ နောက်တစ်နည်းကတော့ အရင်ဆုံး Array ကို ကြိုဆောက်ထားပြီး နောက်မှ Program ထဲကနေ Element တွေကို Set လုပ်လိုတဲ့ အခါမျိုးမှာ အသုံးပြုနိုင်တဲ့ နည်းတို့ပဲ ဖြစ်ပါတယ်။

If you know Array Elements

Array ထဲမှာပါမည့် Element တွေကို ကြိုပြီးသိထားမယ်ဆိုရင် အောက်ပါ အတိုင်း Array ကို Declare လုပ်နိုင်ပါတယ်။

```
int [] array = {1,2,3,4,5};
```

Array Variable ကို Declare လုပ်မယ်ဆိုရင် Array မှာ အသုံးပြုမည့် Type ရယ် Array Sing ဖြစ်တဲ့ [] ရယ်၊ Variable Name ကို ရေးသားပေးရပါမယ်။ အထက်ပါအတိုင်း int [] array လို့ရေးသားထားရင် int ဆိုတာ Array ကို အသုံးပြုမည့် Type ဖြစ်ပြီး array ဆိုတာက Variable Name ဖြစ်ပါတယ်။

Array ကို ရေးတဲ့ နေရာမှာ int [] array လို့လဲ ရေးသားနိုင်သလို int array [] လို့လဲ ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။ အထက်ပါအတိုင်း Array ကို Declare လုပ်မယ်ဆိုရင် {1,2,3,4,5} နဲ့တိုက်ရိုက် အစားထိုး ရေးသားနိုင်သော်လဲ Array ကို Declare လုပ်ပြီးတဲ့ အခါကျမှ Assign လုပ်တဲ့ Statement တွေမှာဆိုရင်တော့ ရေးသားလို့ရမှာ မဟုတ်ပါဘူး။

```
int [] array;
array = {1,2,3,4,5};
```

အထက်ပါအတိုင်း Assignment Statement တွေကို ရေးသားလို့ မရနိုင်ပါဘူး။ ရေးသားမယ်ဆိုရင် အောက်ပါအတိုင်း ရေးသားရမှာ ဖြစ်ပါတယ်။

```
int [] array;
array = new int[] {1,2,3,4,5};
```

တကယ်တော့ new int[] {1,2,3,4,5} လို့ရေးတာက Array Object ကို Instantiate လုပ်တဲ့ ရေးသားပုံအမှန် ဖြစ်ပါတယ်။ Java မှာက Declaration Statement တွေမှာ အလွယ်ရေးသားနိုင်အောင် {1,2,3,4,5} ကို Short Cut အနေနဲ့ ပြင်ပေးထားတာဖြစ်ပြီး Declaration Statement တွေမှာပဲ ရေးသားလို့ရမှာ ဖြစ်ပါတယ်။

နောက်မှ Program ထဲကနေ Element တွေကို Set လုပ်လိုတဲ့ အခါ

တစ်ခါတစ်လေကျရင် Array ထဲမှာ ဘာတွေရှိနေမယ်ဆိုတာ ကြိုပြီးသိမနေပဲ User Input တွေကနေ ရယူလာတဲ့ Data တွေကို Array ထဲမှာ သိမ်းထားလိုတဲ့ အခါမျိုးတွေ ရှိတတ်ပါတယ်။ အဲ့ဒီလိုအခါမျိုးတွေမှာ Array Object ကို ကြိုဆောက်ထားဖို့လိုအပ်ပါတယ်။

တစ်ခု သတိထားရမှာက Array Object ကို တည်ဆောက်လိုတဲ့အခါမျိုးမှာ Element ဘယ်နှစ်လုံးကို အသုံးပြုမယ် ဆိုတာကို ကြိုပြီး သတ်မှတ်ထားဖို့လိုအပ်ပါတယ်။

```
int [] array = new int [5];
```

String 10 ခုကို အသုံးပြုချင်တဲ့ Array မျိုးဆိုရင် အောက်ပါ အတိုင်းရေးသားရပါမယ်။

```
String [] array = new String [10];
```

Array မှာ Java Language မှာပါတဲ့ Data Type တွေသာမက မိမိကိုယ်တိုင်ရေးသားထားတဲ့ Custom Object တွေကိုလဲ Type အနေနဲ့ အသုံးပြုနိုင်ပါတယ်။ Student Type တွေကို Array မှာ အသုံးပြုချင်တယ်ဆိုရင်လဲ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

```
public class ArrayDemo {

    public static void main(String[] args) {

        int array [] = new int[3];

        Student [] students = new Student[3];

    }
}

class Student {
    String name;
    int age;
}
```

အထက်နမူနာထဲမှာလို new int[10] လို့ရေးလိုက်တာနဲ့ int ရဲ့ Default တန်ဖိုးနဲ့ {0, 0, 0} ကို တည်ဆောက်သွားမှာ ဖြစ်ပြီး new Student[3] လို့ရေးလိုက်တာနဲ့ {null, null, null} ဆိုပြီး Array Object ကိုတည်ဆောက်ပေးမှာ ဖြစ်ပါတယ်။ students Array ထဲမှာ Student Object ၃ ခုကို သိမ်းပေးထားနိုင်မှာ ဖြစ်ပါတယ်။

Accessing Array Elements

Array Object တွေဟာ Element တွေကို Index Number တွေနဲ့ သတ်မှတ်ပြီး သိမ်းပေးထားနိုင်မှာ ဖြစ်တယ်။ Array တွေကို အသုံးပြုမယ်ဆိုရင် Array ထဲကို Element တွေ ထည့်ပြီး သိမ်းထားတတ်ဖို့လိုအပ်သလို Array ထဲမှာ သိမ်းထားတဲ့ Element တွေကိုလဲ ပြန်ပြီး ခေါ်ယူ အသုံးပြုတတ်ဖို့လိုအပ်ပါတယ်။

Array ထဲမှာ Element တွေကို သိမ်းထားမယ်ဆိုရင်လဲ Index Number တွေနဲ့ သိမ်းထားဖို့လိုအပ်သလို ပြန်ပြီး ခေါ်ယူမယ်ဆိုရင်လဲ Index Number နဲ့ ပြန်ပြီးခေါ်ယူ အသုံးပြုရမှာ ဖြစ်ပါတယ်။

```
public class ArrayDemo {

    public static void main(String[] args) {

        Student [] students = new Student[3];

        students[0] = new Student("Aung Aung", 20);
        students[1] = new Student("Maung Maung", 21);
        students[2] = new Student("Thidar", 25);
    }
}

class Student {
    String name;
    int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Array ကို Declare လုပ်စဉ် Type မျိုးဆိုပါက Array ထဲမှာ လာပြီး သိမ်းထားနိုင်မှာ ဖြစ်ပါတယ်။ Student [] students လို့ Declare လုပ်ထားတဲ့ အတွက် students Array ထဲမှာ Student Object တွေနဲ့ Student Class ရဲ့ Sub Class Object တွေဆိုရင် လာပြီး သိမ်းထားလို့ရပါမယ်။

တဖန် Array ကို Object ဆောက်ထားသလောက် အရေအတွက် သိမ်းပေးထားနိုင်မှာ ဖြစ်တယ်။ new Student [3] လို့ရေးထားတဲ့ အတွက် Array မှာ အခန်း ၃ ခန်းပိုင်ပြီး Student Object ၃ ခုသိမ်းပေးထားနိုင်မှာ ဖြစ်ပါတယ်။ Array ရဲ့ Index Number ဟာ 0 ကနေ စတာဖြစ်တဲ့ အတွက် student[0] ဆိုပြီး ရေးမယ်ဆိုရင် ရှေ့ဆုံးအခန်းမှာ Student Object တစ်ခုကို လာသိမ်းမှာ ဖြစ်ပါတယ်။

တကယ်လို့ ရှိတဲ့ အခန်းထက်ကျော်ပြီး Array ကို Access လုပ်မိရင် ArrayIndexOutOfBoundsException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။ students[-1] တို့ students[3] တို့ကို Access လုပ်ရင် အထက်ပါ Exception ကို ဖြစ်ပေါ်စေမှာပဲ ဖြစ်ပါတယ်။

```
public class ArrayDemo {

    public static void main(String[] args) {

        Student [] students = new Student[3];

        students[0] = new Student("Aung Aung", 20);
        students[1] = new Student("Maung Maung", 21);
        students[2] = new Student("Thidar", 25);

        Student firstStudent = students[0];
        System.out.println(firstStudent.name);

        Student secondStudent = students[1];
        System.out.println(secondStudent.name);

        Student lastStudent = students[2];
        System.out.println(lastStudent.name);
    }
}

class Student {
    String name;
    int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

အထက်ပါအတိုင်း Array ထဲက Element တွေကို ပြန်ယူချင်တယ်ဆိုရင်လဲ သိမ်းထားတဲ့ index number နဲ့ပဲ ပြန်ပြီး ခေါ်ယူရမှာ ဖြစ်ပါတယ်။ students[0] ဆိုရင် Array ထဲက ပထမဆုံး Student ကို ရရှိနိုင်ပြီး students[1] ဆိုရင် ဒုတိယ Student Object ကိုရရှိမှာ ဖြစ်ပါတယ်။ students[2] ဆိုရင်တော့ နောက်ဆုံး Student ကိုရရှိနိုင်မှာ ဖြစ်ပါတယ်။

ဒီနေရာမှာလဲ မရှိတဲ့ index နဲ့ Access လုပ်မိမယ်ဆိုရင် ArrayIndexOutOfBoundsException ကို ဖြစ်ပွားစေမှာ ဖြစ်ပါတယ်။

Multi Dimension Arrays

Array ဆိုတာက Java Data တွေကို တစ်ခုထက်မက အစီအစဉ်လိုက် သိမ်းပေးထားနိုင်တဲ့ Container Object တစ်မျိုးဖြစ်ပါတယ်။ Multi Dimension Array ဆိုတာက Array Object တွေကို သိမ်းပေးထားနိုင်တဲ့ Array Object တစ်မျိုးသာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် Multi Dimension Array ထဲမှာ အခြားသော Array Object တွေကို သိမ်းပေးထားနိုင်မှာ ဖြစ်ပါတယ်။

Multi Dimension Array ကို ဘယ်လို Declare လုပ်ရလဲ

Multi Dimension Array အမျိုးအစား Variable တစ်ခုကို ရေးသား အသုံးပြုလိုတဲ့ အခါမှာ Variable ကို Declare လုပ်တတ်ဖို့လိုပါတယ်။ Multi Dimension Array ဆိုတာ Array တွေကို သိမ်းပေးထားနိုင်တဲ့ Array ဖြစ်တဲ့ အတွက် အသုံးမပြုခင် ဘယ်လို အမျိုးအစား Array ကို အသုံးပြုမှာလဲ ဆိုတာကို သိထားဖို့လိုပါတယ်။

ဆိုကြပါစို့၊ int array တွေကို သိမ်းပေးထားနိုင်တဲ့ Multi Dimension Array မျိုး၊ ဒါမှမဟုတ် String Array တွေကို သိမ်းပေးထားနိုင်တဲ့ Array မျိုးဆိုပြီး သိရှိနေဖို့လိုအပ်ပါတယ်။

```
int [][] array;          // Array of int array
String [][] strArray;    // Array of String Array
```

သာမန် Array တွေကို ရေးခဲ့တုန်းကလို ဘယ်လိုအမျိုးအစား Array ဆိုတာကို သိဖို့လိုအပ်ပြီး၊ အဲ့ဒီ Type ကို ရှေ့ဆုံးမှာ ရေးပေးရပါမယ်။ ပြီးရင် Array Sign ဖြစ်တဲ့ [] ကို အသုံးပြုလိုသလောက် ရေးသားပေးရပါမယ်။ Two Dimension ဆိုရင် [][] လို့ရေးပြီး Three Dimension ဆိုရင်တော့ [][][] လို့ ရေးသားပေးရပါမယ်။ ပြီးကာမှ Identifier ဖြစ်တဲ့ Variable Name ကို ရေးသားရမှာပါ။

ဒါပေမဲ့ Multi Dimension Array ကို Declare လုပ်တဲ့ နေရာမှာ အောက်ပါအတိုင်း ရေးသားရင်လဲ Compile Error ဖြစ်မှာ မဟုတ်ပါဘူး။

```
int [][] array;
int array [][];
int [] array [];
```

Identifier ဖြစ်တဲ့ Array Name ကို Array Sign တွေရဲ့ ရှေ့မှာကော၊ နောက်မှာကော၊ ကြားမှာပါ ရေးသာဖေါ်ပြနိုင်မှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ Appication တစ်ခုထဲမှာ ဟိုလိုရေးလိုက် ဒီလိုရေးလိုက်ဟာ ကောင်းတဲ့ အကျင့် တစ်ခု မဟုတ်ပါဘူး။ Application တစ်ခုကို ရေးသားတဲ့ နေရာမှာ မိမိတစ်ယောက်ထဲ ရေးသားနေတာမဟုတ်ပဲ အခြားသူ တွေပါ ပါဝင်ရေးသားနေနိုင်တယ်ဆိုတာကို သတိထားသင့်ပါတယ်။

ဒါ့ကြောင့် ဖတ်ရလွယ်ပြီး နားလည်ရလွယ်ကူအောင် အမြဲသတိထားပြီး ရေးသားသင့်ပါတယ်။

ဘယ်လို **Instantiate** လုပ်ကြမလဲ

Instantiate လုပ်တယ်ဆိုတာက Object ဆောက်တာကို ပြောတာပါ။ ဒီနေရာမှာ Multi Dimension Array တစ်ခု ကို ဘယ်လို Object ဆောက်ကြမလဲဆိုတာကို လေ့လာသွားကြရအောင်။ Array တွေကို လေ့လာခဲ့တုန်းကလိုပါပဲ၊ Object တည်ဆောက်လိုတဲ့ အခါမှာ ပါဝင်မဲ့ Element တွေကို ကြိုပြီး သိထားနိုင်သလို၊ မသိထားနိုင်တဲ့ အခါမျိုးလဲ ရှိပါလိမ့်မယ်။ အရင်ဆုံး ပါဝင်မည့် Element တွေကို ကြိုသိထားရင် ဘယ်လို Instantiate လုပ်မယ်ဆိုတာကို လေ့လာကြည့်ကြရအောင်။

```
int [][] array = {{1,2}, {3,4}};
```

အထက်ပါအတိုင်း ရေးသားထားတာဟာ int array တွေကိုသိမ်းထားနိုင်တဲ့ array Object တစ်ခုကို အရင်ဆုံး Declare လုပ်ပါတယ်။ ပြီးမှ {1,2} ပါတဲ့ int Array နဲ့ {3,4} ပါတဲ့ int Array ကို အသုံးပြုပြီး int Array တွေကို သိမ်းထားနိုင်တဲ့ Array Object တစ်ခုကို တည်ဆောက်သွားတာဖြစ်ပါတယ်။

```
int [][] array = {{1,2}, {3,4,5}};
```

အထက်ပါကုဒ်ကို ကြည့်ကြည့်ရအောင်။ Multi Dimension Array နေရာကို Dimension မတူတဲ့ Array တွေနဲ့ လာပြီး အစားထိုးနေပါတယ်။ ဒါပေမဲ့ ဒီလိုရေးထားရင်လဲ မှားမှာမဟုတ်ပါဘူး။ Java မှာ အသုံးပြုနေတဲ့ Array ဟာ C တို့ C++ တို့မှာ အသုံးပြုနေတဲ့ Array နဲ့ မတူပဲ Object တစ်ခုသာ ဖြစ်တဲ့ အတွက် Int Array ဖြစ်မယ်ဆိုရင် အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ Dimension တွေဆိုတာထက် Int အမျိုးအစား Array တွေကို သိမ်းပေးထားနိုင်တဲ့ Array တွေလို့ မြင်တာကောင်းပါမယ်။

Array ထဲမှာ ပါမည့် Element တွေကို ကြိုပြီး မသိထားတဲ့ အခါမျိုးမှာ ဆိုရင်လဲ အောက်ပါအတိုင်း Array Object မျိုးကို တည်ဆောက်နိုင်ပါမယ်။

```
int [][] array1 = new int[2][3];
int [][] array2 = new int[2][];
```

အထက်ပါအတိုင်း new int[2][3] ဆိုပြီး Initialize လုပ်မယ်ဆိုရင် {{0,0,0}, {0,0,0}} ဆိုပြီး Array Object ကို တည်ဆောက်သွားမှာ ဖြစ်ပါတယ်။ တဖန် new int[2][] လို့ ရေးမယ်ဆိုရင်လဲ ဖြစ်ပါတယ်။ ဒီအတိုင်း ရေးသားထား မယ်ဆိုရင်တော့ {null, null} ဆိုပြီး Array Object ကို တည်ဆောက်သွားပါမယ်။

Int [2][] လို့ရေးထားတာက int [] ၂ ခုပါတဲ့ Array Object လို့ ဖေါ်ပြနေတာဖြစ်ပါတယ်။ int [2][] လို့ရေးထားတဲ့ အတွက် အထဲမှာပါတဲ့ Array ကို Initialize မလုပ်နိုင်ပါဘူး။ Array ဆိုတာလဲ Object ဖြစ်တဲ့ အတွက် Object ရဲ့ Default တန်ဖိုးဖြစ်တဲ့ null တွေနဲ့ Object ကို တည်ဆောက်သွားတာ ဖြစ်ပါတယ်။

Multi Dimension Array ကို ဘယ်လိုဆောက်ရမလဲဆိုတာကို လေ့လာပြီးတဲ့ နောက်မှာ Array ထဲက Element တွေကို ဘယ်လို Access လုပ်ကြမယ်ဆိုတာကို ဆက်လက် လေ့လာသွားကြရအောင်။

Multi Dimension Array Member တွေကို ဘယ်လို Access လုပ်မှာလဲ

Member တွေကို Access လုပ်တယ်လို့ ပြောတဲ့ နေရာမှာ Member တွေကို Asign လုပ်တာလဲပါသလို့၊ Member တွေရဲ့ တန်ဖိုးကို ရယူတာလဲ ပါပါတယ်။ သာမန် Array တွေတုန်းလိုပဲ Multi Dimension Array တွေရဲ့ Member တွေကို Access လုပ်တဲ့ နေရာမှာ index Number ကို အသုံးပြုပြီး Access လုပ်ရမှာ ဖြစ်ပါတယ်။

```
jshell> int [][] array = {{1,2},{3,4}}
array ==> int[2][] { int[2] { 1, 2 }, int[2] { 3, 4 } }

jshell> int a = array[0][0]
a ==> 1

jshell> int b = array[0][1]
b ==> 2

jshell> int c = array[0][2]
|  java.lang.ArrayIndexOutOfBoundsException thrown: 2
|        at (#5:1)
```

အထက်ပါအတိုင်း Multi Dimension Array တစ်ခုကို jshell ကို အသုံးပြုပြီး တည်ဆောက်ကြည့်ပါမယ်။ အထက်နမူနာပါ int [][] array ဟာ int [] တွေကို သိမ်းပေးထားနိုင်တဲ့ Array တစ်မျိုးသားဖြစ်ပါတယ်။ array ထဲမှာ int [] တွေ ဖြစ်ကြတဲ့ {1,2} နဲ့ {3,4} ကို သိမ်းပေးထားပါတယ်။

ဒါ့ကြောင့် array [0][0] လို့ပြောရင် ရှေ့ဆုံး Array ထဲက ပထမဆုံးနေရာမှာ ရှိတဲ့ 1 ကို ရရှိမှာ ဖြစ်ပါတယ်။ တဖန် array[0][1] လို့ဖေါ်ပြရင် ရှေ့ဆုံး Array ထဲက ဒုတိယနေရာမှာရှိတဲ့ 2 ကို ရရှိမှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ array[0][2] လို့ရေးလိုက်တဲ့အခါမှာ ရှေ့ဆုံး Array ရဲ့ တတိယနေရာဟာ မရှိတဲ့ အတွက် ArrayIndexOutOfBoundsException ကို ဖြစ်ပေါ်စေတာ ဖြစ်ပါတယ်။ သာမန် Array တွေတုန်းကလိုပဲ မရှိတဲ့ Index Number ကို Access လုပ်မိရင် ArrayIndexOutOfBoundsException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

```
jshell> array[0][0] = 10
$6 ==> 10

jshell> a = array[0][0]
a ==> 10

jshell> array[0][2] = 15
|  java.lang.ArrayIndexOutOfBoundsException thrown: 2
|        at (#9:1)
```

Multi Dimension Array ထဲမှာရှိတဲ့ တန်ဖိုးကို Assign လုပ်လိုတဲ့ အခါမျိုးမှာလဲ Index Number တွေကိုပဲ အသုံးပြုရမှာ ဖြစ်ပါတယ်။ array[0][0] = 10 လို့ရေးပြီး ပြန်ကြည့်တဲ့အခါမှာ တန်ဖိုးဟာ 10 ကို ပြောင်းသွားတာကို တွေ့ရပါမယ်။ မရှိတဲ့ Index ကို အသုံးပြုမိရင်လဲ ArrayIndexOutOfBoundsException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

Multi Dimension Array ထဲက တန်ဖိုးတွေလို့ပြောတဲ့နေရာမှာ အောက်ဆုံး Element အထိ Access လုပ်နိုင်သလို Array တွေကိုလဲ ပြန်ပြီး Access လုပ်နိုင်ပါတယ်။

```
jshell> int [] array2 = array[1]
array2 ==> int[2] { 3, 4 }

jshell> array[0] = new int[]{30,40}
$11 ==> int[2] { 30, 40 }

jshell> array
array ==> int[2][] { int[2] { 30, 40 }, int[2] { 3, 4 } }
```

အထက်ပါအတိုင်း Multi Dimension Array ကို First Level Index ကို အသုံးပြုပြီး Access လုပ်မယ်ဆိုရင် အဲ့ဒီ Array ထဲမှာ သိမ်းပေးထားတဲ့ Array ကို ပြန်လည်ရရှိနိုင်မှာ ဖြစ်ပါတယ်။ array[1] ဆိုပြီး Access လုပ်ရင် ဒုတိယနေရာမှာ ရှိတဲ့ Array Object ကို ပြန်လည်ရရှိမှာ ဖြစ်ပါတယ်။ အဲ့ဒီအတိုင်းပဲ array[0] ရဲ့နေရာကို အခြား int Array တစ်ခုနဲ့ အစားထိုးနိုင်မှာ ဖြစ်ပါတယ်။

ဒါပေမဲ့ Type မတူရင်တော့ Asign လုပ်တာကော Reference လုပ်တာကော လုပ်လို့ရမှာ မဟုတ်ပါဘူး။ array variable ရဲ့ type ကို int [][] လို့ Declare လုပ်ထားတဲ့ အတွက် First Level Element ဟာ int [] ပဲ ဖြစ်နေမှာ ဖြစ်ပါတယ်။ ပြီးတော့ Second Level Element ရဲ့ type ဟာလဲ int ပဲဖြစ်ရပါမယ်။

```
jshell> int data = array[0]
|  Error:
|  incompatible types: int[] cannot be converted to int
|  int data = array[0];
|             ^------^

jshell> array[1] = 10
|  Error:
|  incompatible types: int cannot be converted to int[]
|  array[1] = 10
|             ^^
```

အထက်ပါအတိုင်း int data နေရာမှာ array[0] နဲ့ အစားထိုးလို့မရပါဘူး။ အစားထိုးချင်တဲ့ data ရဲ့ type ဟာ int ဖြစ်ပြီး array[0] လို့ပြောရင် ရရှိနိုင်မည့် type ဟာ int [] ဖြစ်နေတဲ့ အတွက်ဖြစ်ပါတယ်။ တဖန် array [1] နေရာကိုလဲ 10 ဆိုတဲ့ တန်ဖိုးနဲ့ အစားထိုးလို့ မရနိုင်ပါဘူး။ ဘာလို့လဲ ဆိုတော့ array[1] ဟာ int [] ဖြစ်ပြီး 10 ဟာ int ဖြစ်တဲ့ အတွက် ဖြစ်ပါတယ်။

Java Programming Language မှာက Variable တစ်ခုကို Declare လုပ်ခဲ့တဲ့ Type နဲ့ မတူတဲ့ တန်ဖိုးတွေကို အစားထိုးလို့ မရနိုင်ပါဘူး။

Operators

ကျွန်တော်တို့ ရှေ့ခန်းတွေမှာ Class တွေ၊ Variable တွေ၊ Method တွေနဲ့ Java မှာ အသုံးပြုနိုင်တဲ့ Data Type တွေအကြောင်းလေ့လာခဲ့ကြပါတယ်။ ဒီတစ်ခေါက်တော့ Variable တွေနဲ့ တွဲပြီး ရေးသားနိုင်တဲ့ Operator တွေနဲ့ Expression တွေ အကြောင်းကို လေ့လာကြပါမယ်။

Operator ဆိုတာကတော့ Variable တွေကို ဘာလုပ်မှာလဲဆိုတာကို ဖော်ပြပေးနိုင်တဲ့ အရာတွေဖြစ်ပြီးတော့ Expression ဆိုတာကတော့ Variable တွေနဲ့ Operator တွေကို အသုံးပြုပြီး ရေးသားရမည့် ဖော်ပြချက်တွေ ဖြစ်ကြပါတယ်။ JVM က Expression တွေကို နားလည်ပြီး OS ကနေ တဆင့် Computer ကို လုပ်ဆောင်မှုတွေကို ညွှန်ကြားချက်တွေ ပေးနိုင်မှာ ဖြစ်ပါတယ်။

ဆိုကြပါစို့ဗျာ။ Variable a နဲ့ Variable b တို့ကို ပေါင်းစေချင်တယ်ဆိုရင် ပေါင်းပါဆိုတာကို Operator နဲ့ ဖော်ပြပေးရမှာ ဖြစ်တယ်။ တဖန် Variable တစ်ခုဟာ Type တစ်ခုရဲ့ Object ဖြစ်ရဲ့လားဆိုတာကို စစ်ချင်တယ်၊ Variable တွေကို တစ်တိုးတာတို့ တစ်နှုတ်တာတို့ လုပ်ချင်တယ်၊ နောက်ပြီး Variable နှစ်ခုရဲ့ တန်ဖိုးဟာ တူမတူ စစ်ဆေးချင်တယ်၊ Variable တွေကို အသုံးပြုပြီး Logic တွေကို စစ်ဆေးချင်တယ်၊ Variable နှစ်ခုဘယ်သူကြီးလဲ ငယ်လဲဆိုတာကို သိချင်တယ်၊ ဒီလိုအခါမျိုးတွေမှာ Operator တွေကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။

Java ဘာသာရပ်မှာ အသုံးပြုနိုင်တဲ့ Operators တွေကတော့ အောက်ပါ အတိုင်းဖြစ်ကြပါတယ်။

Operator Type	Operators	Description
Assignment	=, +=, -=, *=, /=	Variable များအား အစားထိုးရာတွင် အသုံးပြု
Arithmetic	+, -, *, /, %, ++, - -	Primitive Variable များအား အပေါင်း၊ အနုတ်၊ အမြှောက်၊ အစား၊ အကြွင်းရှာ ရာတွင် အသုံးပြု
Relational	<, <=, >, >=, !=, ==	Primitive Variable များအား နှိုင်းယှဉ်ရာတွင် အသုံးပြု
Logical	!, &, &&, |, ||, ^	Primitive Variables များ: NOT, AND, OR, XOR Logic များအား အသုံးပြုနိုင်

Bitwise Operator ဆိုတာရှိပါသေးတယ်။ ဒါပေမဲ့ Binary တန်ဖိုးတွေကိုတွက်ချက်တဲ့ နေရာမှာ အသုံးပြုတာပါ။ ဒါပေမဲ့ Business Application တွေကို ရေးသားတဲ့ နေရာမှာ သိပ်ပြီး မသုံးတဲ့ အတွက် ဒီစာအုပ်ထဲမှာတော့ ထည့်မရေးတော့ပါဘူး။

Assignment Operator

Assignment Operator တွေကတော့ Variable နေရာမှာ အစားထိုးပေးနိုင်တဲ့ Operator များဖြစ်ကြပါတယ်။ သာမန် "=" တစ်ခုကို အသုံးပြုပြီး ရေးသားနိုင်သလို အခြားသော Arithmetic Operator တွေနဲ့လဲ တွဲဖက်ပြီး ရေးသားနိုင်ပါတယ်။

Assignment in Primitive Type Variables

= Assignment Operator တွေကိုတော့ Primitive Variable တွေကိုရော Reference Variable တွေမှာပါ အစားထိုးတဲ့ နေရာမှာ အသုံးပြုနိုင်ပါတယ်။ ဒါပေမဲ့ Variable Type အပေါ်မူတည်ပြီး လုပ်ဆောင်ပုံချင်း မတူညီကြပါဘူး။

```
jshell> int a
a ==> 0

jshell> a = 10
a ==> 10

jshell> int b = a
b ==> 10

jshell> a = 15
a ==> 15

jshell> b
b ==> 10
```

အထက်ပါအတိုင်း JShell နဲ့ ရေးသားထားတဲ့ Code Snippet တွေကို လေ့လာကြည့်ကြရအောင်။ အရင်ဆုံး int a ဆိုပြီး Variable တစ်ခုကို Declare လုပ်ပါတယ်။ Primitive Type Variable တစ်ခုပါ။

ထို့နောက် a = 10 လို့ရေးထားပါတယ်။ a variable ရဲ့ တန်ဖိုးကို 10 နဲ့ အစားထိုးတာဖြစ်ပါတယ်။ a ရဲ့ တန်ဖိုးဟာ 10 ဖြစ်သွားပါမယ်။ တဖန် int b = a လို့ ရေးထားတာကို တွေ့ရပါမယ်။ ဒီစာကြောင်းဟာ int type variable b ရဲ့တာဖိုးကို a ရဲ့ တန်ဖိုးနဲ့ အစားထိုးနေတာ ဖြစ်ပါတယ်။ ဒီနေရာမှာ a ရဲ့ တန်ဖိုးဟာ 10 ဖြစ်တဲ့ အတွက် b ရဲ့ တန်ဖိုးဟာလဲ 10 ဖြစ်သွားပါမယ်။

တဖန် a = 15 ဆိုပြီး အစားထိုးပြန်တယ်။ ဒီနေရာမှာ a ရဲ့ တန်ဖိုးသာ ပြောင်းသွားမှာ ဖြစ်ပါတယ်။ b နေရာကို a က အစားထိုးထားလို့ဆိုပြီး b ရဲ့ တန်ဖိုးဟာ လိုက်မပြောင်းသွားပါဘူး။ b ရဲ့ တန်ဖိုးဟာ နဂိုမူလအတိုင်း 10 ပဲ ဖြစ်နေမှာ ဖြစ်ပါတယ်။ Primitive Type Variable တွေမှာ Assignment တွေကို လုပ်တဲ့ အခါမှာ Actual Value ကို အစားထိုးကြတာ ဖြစ်တဲ့ အတွက် ဖြစ်ပါတယ်။

ဆက်လက်ပြီး Reference Type Variable တွေကို Assign လုပ်တာကို လေ့လာကြည့်ကြရအောင်။

Assignment in Reference Type Variables

Assignment Operator ဖြစ်တဲ့ = ကို Reference Type Variable တွေကို အစားထိုးတဲ့ နေရာမှာလဲ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ Reference Type တွေရဲ့ အစားထိုးပုံက Primitive Type တွေနဲ့ မတူပါဘူး။ Primitive Type Variable တွေမှာ Actual Value ကို အစားထိုးတာမှန်ပေမဲ့ Reference Type Variable တွေမှာတော့ Variable ကနေ Reference လုပ်မဲ့ Object ရဲ့ Memory Address ကိုသာ အစားလာထိုးမှာ ဖြစ်ပါတယ်။

Variable မှာလာအစားထိုးထားတဲ့ Memory Address ကနေ တဆင့် Object တွေကို Reference လုပ်ပြီး အသုံးပြုနေတာ ဖြစ်တဲ့ အတွက် Reference Variable တွေလို့ ခေါ်ခဲ့ကြခြင်းလဲ ဖြစ်ပါတယ်။
အထက်ပါနမူနာမှာ အရင်ဆုံး String Type Variable တစ်ခုပါတဲ့ Person Class ကို တည်ဆောက်ပါတယ်။ ပြီးတော့မှ Person Type Variable ဖြစ်တဲ့ thidar ဆိုတဲ့ နေရာမှာ new Person() ဆိုပြီး Person Object ကို တည်ဆောက်ပြီးတော့ Assign လုပ်နေပါတယ်။ ဒီနေရာမှာ Person Type ဟာ Reference Type ဖြစ်တဲ့ အတွက် new Person() လို့ Object ကို Heap Memory ပေါ်မှာတည်ဆောက်ပြီး အဲ့ဒီ Object ရဲ့ Memory Address ကို thidar နေရာမှာ အစားထိုးနေတာ ဖြစ်ပါတယ်။

```
jshell> class Person {
   ...> String name;
   ...> }
|  created class Person

jshell> Person thidar = new Person();
thidar ==> Person@6fd02e5

jshell> thidar.name = "Thidar"
$3 ==> "Thidar"

jshell> Person nilar = thidar;
nilar ==> Person@6fd02e5

jshell> nilar.name = "Nilar"
$5 ==> "Nilar"

jshell> thidar.name
$6 ==> "Nilar"

jshell> nilar.name
$7 ==> "Nilar"
```

တဖန် Person nilar = thidar လို့ရေးထားတာကလဲ Assignment Operator ကို အသုံးပြုပြီး nilar နေရာကို thidar ရဲ့ တန်ဖိုးနဲ့ အစားထိုးနေပြန်တယ်။ ဒီနေရာမှာ thidar မှာရှိနေတဲ့ တန်ဖိုးဟာ Person Object ကို Reference လုပ်နိုင်တဲ့ Memory Address ကိုပဲ nilar နေရာမှာ အစားထိုးသွားပြန်တယ်။ Object တစ်ခုထဲကို Reference Variable နှစ်ခုကနေ Reference လုပ်နေတယ်ဆိုတာကို သိရှိထားသင့်ပါတယ်။ ဒါ့ကြောင့်မို့လို့ nilar.name = “Nilar” လို့အစားထိုးလိုက်တဲ့ အခါမှာ thidar.name ဟာလဲ “Nilar” လို့ပြောင်းသွားတာ ဖြစ်ပါတယ်။

Primitive Type Variable တွေကို Assign လုပ်တဲ့အခါမှာ Actual Value ကို အစားထိုးပြီး Reference Type Variable တွေကို Assign လုပ်တဲ့အခါမှာ Heap Memory ပေါ်မှာရှိတဲ့ Object ကို Reference လုပ်နိုင်တဲ့ Memory Address ကိုသာ အစားထိုးသွားတယ်ဆိုတာကို သိရှိထားဖို့လိုအပ်ပါတယ်။

Assignment Operators ဖြစ်တဲ့ = နဲ့ Arithmetic Operators တွေနဲ့ တွဲဖက်ပြီးလဲ အသုံးပြုနိုင်ပါတယ်။ ဒီအကြောင်းကိုတော့ Arithmetic Operators တွေအခန်းမှာ ဖေါ်ပြသွားပါမယ်။

Arithmetic Operators

Number အမျိုးအစား Primitive Type Variable တွေကို အသုံးပြုပြီး သခ်ၤာ ပိုင်းဆိုင်ရာ တွက်ချက်မှုတွေ ပြုလုပ်လိုတဲ့ အခါမျိုးမှာ Arithmetic Operators တွေကို အသုံးပြုနိုင်ပါတယ်။ အပေါင်း၊ အနှတ်၊ အမြောက်၊ အစားနဲ့ အကြွင်းရှာတဲ့ တွက်ချက်မှုတွေကို ပြုလုပ်လိုတဲ့ အခါမျိုးမှာ Arithmetic Operators တွေကို အသုံးပြုနိုင်ပါတယ်။

Arithmetic Operators တွေနဲ့ အသုံးပြုနိုင်တဲ့ Data Type တွေကတော့ Number အမျိုးအစား Primitive Data Type တွေနဲ့ သူတို့ရဲ့ Wrapper Class တွေမှာ အသုံးပြုနိုင်ပါတယ်။ ပြီးတော့ Number အနေနဲ့ ဖေါ်ပြနိုင်တဲ့ Primitive Data Type ဖြစ်တဲ့ char type တွေမှာလဲ အသုံးပြုနိုင်ပါတယ်။ ချွင်းချက်အနေနဲ့ String Type Variable တွေမှာလဲ + Operator ကို အသုံးပြုနိုင်ပါတယ်။

```
jshell> int a = 10
a ==> 10

jshell> int b = 20
b ==> 20

jshell> a + b
$3 ==> 30
```

အထက်ပါ နမူနာထဲမှာ အရင်ဆုံး a နေရာမှာ 10 ကို အစားထိုးပါတယ်။ ပြီးတော့ b နေရာမှာ 20 ကို အစားထိုးပါတယ်။ a + b လို့ ရေးလိုက်တဲ့ အခါမှာ 30 ရပါတယ်။ Number အမျိုးအစား Variable တွေရဲ့ ပေါင်းခြင်းကို + Operator ကို အသုံးပြုနိုင်ပါတယ်။

```
jshell> char a = 'a' + 'b'
a ==> 'Ã'

jshell> Character c = a
c ==> 'Ã'

jshell> char d = a + c
|  Error:
|  incompatible types: possible lossy conversion from int to char
|  char d = a + c;
|           ^---^
```

အထက်ပါ နမူနာကုဒ်ထဲမှာလို Number တန်ဖိုးအနေနဲ့ ပြောင်းပြီး အသုံးပြုနိုင်တဲ့ char type တွေမှာလဲ +

```
jshell> String hello = "Hello"
hello ==> "Hello"

jshell> String java = "Java"
java ==> "Java"

jshell> hello + " " + java
$15 ==> "Hello Java"
```

Operators တွေကို အသုံးပြုနိုင်ပါတယ်။ ဒါပေမဲ့ တစ်ခု သတိပြုဖို့ လိုအပ်တာက Wrapper Class ဖြစ်တဲ့ Character Class တွေမှာတော့ Arithmetic Operator တွေကို အသုံးပြုလို့ ရမှာ မဟုတ်ပါဘူး။
အထက်ပါ နမူနာကတော့ String Type Variable တွေကြားမှာ + Operator ကို အသုံးပြုတာပါ။ ချွင်းချက်အနေနဲ့ Object တွေမှာ String တစ်ခုထဲ + Operator ကို အသုံးပြုနိုင်တာ ဖြစ်ပါတယ်။ + Operator ကို String Type Variable တွေနဲ့ String Litrals တွေနဲ့ ပူးတွဲအသုံးပြုနိုင်ပါတယ်။ String Value တွေကို ဆက်ပေးတဲ့ အနေနဲ့ အသုံးပြုနိုင်ပါတယ်။

String Variable hello ရဲ့ တန်ဖိုးဟာ “Hello” ဖြစ်ပြီး java ရဲ့ တန်ဖိုးဟာ “Java” ဖြစ်တဲ့ အတွက် hello + “ “ + java ဆိုတာဟာ “Hello Java” ဖြစ်သွားတာကို တွေ့ရပါမယ်။

ဆက်လက်ပြီး String Type Variable နဲ့ Primitive Type Variable တွေကြားမှာ + Operator ကို တွဲပြီး အသုံးပြုတဲ့အခါမျိုးမှာ ဘယ်လို လုပ်ဆောင်သွားတယ်ဆိုတာကို လေ့လာကြည့်ကြရအောင်။

```
jshell> 10 + "1"
$18 ==> "101"
```

10 + “1” ရဲ့ အဖြေဟာ “101” ဖြစ်ပါတယ်။ 10 ဟာ Primitive type အမျိုးအစားဖြစ်တဲ့ int ဖြစ်ပြီး “1” ဟာ String ဖြစ်ပါတယ်။ Primitive Type တစ်ခုနဲ့ String တစ်ခုကို + ကို အသုံးပြုပြီး ရေးသားတဲ့ အခါမှာ Primitive ရဲ့တန်ဖိုးကို String တစ်ခုအနေနဲ့ယူပြီး တွက်ချက်သွားပါမယ်။ ဒါ့ကြောင့် “10” + “1” လို့ယူပြီး အဖြေဟာ “101” ဖြစ်သွားပါမယ်။

```
jshell> 10 + 1 + "1"
$19 ==> "111"
```

အထက်ပါနမူနာထဲမှာ 10 + 1 + “1” လို့ Expression တစ်ခုကို ရေးထားပါတယ်။ 10 နဲ့ 1 တို့ဟာ primitive type ဖြစ်ပြီး “1” ဟာ String ဖြစ်ပါတယ်။ အပါမှာပါခဲ့တဲ့ နမူနာထဲမှာဆိုရင် String တစ်ခုနဲ့ပေါင်းရင် အားလုံးဟာ String လိုပြောင်းပြီးပေါင်းကြတယ်ဆိုတာကို တွေ့ခဲ့ရမှာပါ။ ဒါပေမဲ့ Expression တစ်ခုမှာ Arithmetic Operator တွေ တစ်ခုထက်မကပါဝင်ကြတဲ့ အခါမှာ Operator Precident လို့ခေါ်တဲ့ ဦးစားပေးစဉ်းစားပုံနဲ့ ဘယ် Operator ကို အရင် Eveluate လုပ်မယ်ဆိုတာကို စီးမျဉ်းသတ်မှတ်ထားပါတယ်။

Operator တွေတူနေရင်တော့ ဘယ်ဘက်မှာရှိတဲ့ Operator ကို အရင် Evaluate လုပ်ပါတယ်။ ဒါ့ကြောင့် 10 နဲ့ 1 ကို အရင်ပေါင်းပြီး ရလာတဲ့ ရလဒ်ဖြစ်တဲ့ 11 ကို “1” နဲ့ ပေါင်းပြီး “111” လို့ အဖြေထွက်တာ ဖြစ်ပါတယ်။

```
jshell> "1" + 10 + 1
$20 ==> "1101"
```

တကယ်လို့ “1” + 10 + 1 လို့ရေးခဲ့ရင် အရင်ဆုံး “1” + 10 ကို Evaluate လုပ်ပါမယ်။ ဒါ့ကြောင့် အဖြေဟာ “101” ဖြစ်ပြီး “101” + 1 ကို Evaluate လုပ်ပြီး ရလဒ်ကတော့ “1101” ဖြစ်သွားမှာ ဖြစ်ပါတယ်။

Operator Precedents

Variable တွေနဲ့ Operators တွေကိုတွဲပြီး ရေးသားထားတာတွေကို Expression လို့ခေါ်ဆိုလေ့ရှိပါတယ်။ Expression တစ်ခုမှာ Operator တစ်ခုအထက်မက ပါဝင်လေ့ရှိပါတယ်။ Arathematic Operator တွေကို အသုံးပြုတဲ့နေရာမှာ Operator တစ်ခုဆိုပါက ပြဿနာမရှိပါဘူး။

```
jshell> 10 + 1
$21 ==> 11

jshell> 10 - 5
$22 ==> 5

jshell> 10 * 3
$23 ==> 30

jshell> 10 / 3
$24 ==> 3

jshell> 10 % 3
$25 ==> 1
```

Operator ရဲ့ ဘယ်ဘက်နဲ့ ညာဘက်မှာရှိတဲ့ တန်ဖိုးတွေကို Operator အတိုင်း တွက်ထုတ်ပြီး အဖြေကိုပြရုံပါပဲ။ အထက်ပါအတိုင်း အပေါင်း၊ အနှုတ်၊ အမြှောက်၊ အစားနဲ့ အကြွင်းရှာတာကို တွက်ချက်ပြီး ဖေါ်ပြရုံပါပဲ။ ဒါပေမဲ့ Expression တစ်ခုအတွင်းမှာ Operator တစ်ခုအထက်မကပါဝင်လာပြီဆိုရင်တော့ Java Programming မှာ တော့ Operator Precidents အတိုင်း တွက်ချက် လုပ်ဆောင်သွားမှာ ဖြစ်ပါတယ်။

Multiple Operator Expression တွေနဲ့ ပတ်သက်ပြီး တူညီတဲ့ Operator တွေကြီးဆိုရင်တော့ ဘယ်ဘက်ကနေ ညာဘက်ကို Evaluate လုပ်သွားပါမယ်။ ဒါပေမဲ့ ပါဝင်တဲ့ Operator တွေမတူရင်တော့ Operator Precident အပေါ်မှာ မူတည်ပြီး Operator Precident မြင့်မားတဲ့ အရာတွေကို အရင်ဆုံး Evaluate လုပ်သွားပါမယ်။

```
jshell> 5 + 1 * 10
$26 ==> 15
```

အထက်ပါ နမူနာကို ကြည့်မယ်ဆိုရင် 5 + 1 * 10 လို့ရေးထားပါတယ်။ ဘယ်ဘက်ကနေ ညာဘက်ကို Evaluate လုပ်သွားတယ်ဆိုရင် အဖြေက 5 + 1 ကို အရင် Evaluate လုပ်ပြီး ရလာတဲ့ ရလဒ်ဖြစ်တဲ့ 6 * 10 ဆိုပြီး အဖြေက 60 ဖြစ်သွားသင့်ပါတယ်။

ဒါပေမဲ့ 1 * 10 ကို အရင် Evaluate လုပ်ပြီး ရလာတဲ့ ရလဒ်ဖြစ်တဲ့ 10 နဲ့ 5 ကိုပေါင်းသွားပါတယ်။ ဒါ့ကြောင့် အဖြေဟာ 15 ဖြစ်သွားပါတယ်။ Java မှာက အပေါင်းအနှုတ် Operator တွေထက် အမြှောက် အစား Operator တွေက ပိုပြီး Precident မြင့်မားတဲ့ အတွက် Precident မြင့်မားတဲ့ Expression ကနေ စပြီး Evaluate လုပ်သွားတာ ဖြစ်ပါတယ်။

ဆက်လက်ပြီး အစားနဲ့ အမြှောက် ပြီးတော့ အပေါင်းနဲ့ အနှုတ်ကကော Precedent ဘယ်သူကြီးလဲ၊ ပြီးတော့ အဲ့ဒီ Operator တွေကို Expression ထဲမှာ ရောရေးရင်ဘယ်လို ဖြစ်မလဲဆိုတာကို လေ့လာကြည့်ကြရအောင်။

```
jshell> 5 / 2 * 3
$28 ==> 6

jshell> 3 * 5 / 2
$29 ==> 7
```

အစားနဲ့ အမြောက်ဟာ Operator Precedent အရ အတူတူပဲ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် Left to Right Evaluate လုပ်သွားတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။ အပေါ်ဆုံး အကြောင်းမှာ 5 / 2 * 3 လို့ရေးထားပါတယ်။ အရင်ဆုံး 5 ကို 2 နဲ့ စားပြီး ရလာတဲ့ ရလဒ်ဖြစ်တဲ့ 2 ကို 3 နဲ့ မြှောက်ပါတယ်။ ဒါ့ကြောင့် အဖြေဟာ 6 ဖြစ်သွားပါတယ်။

နောက်တစ်ကြောင်းမှာတော့ 3 * 5 / 2 လို့ရေးထားပါတယ်။ 3 * 5 ရဲ့ ရလဒ်ဟာ 15 ဖြစ်ပြီး 15 / 2 ကို Evaluate ဆက်လုပ်ပါတယ်။ ဒါ့ကြောင့် အဖြေဟာ 7 ဖြစ်သွားပါတယ်။ အပေါင်းအနှုတ်ဟာလဲ Operator Precedent တူတဲ့ အတွက် ဘယ်ဘက်ကနေ ညာဘက်ကို Evaluate လုပ်သွားကြမှာ ဖြစ်ပါတယ်။

Assignment Operator နဲ့ တွဲပြီး အသုံးပြုခြင်း

Arithmetic Operator တွေကို Assignment Operator တွေနဲ့လဲ တွဲဖက်ပြီး ရေးသား အသုံးပြုနိုင်ပါတယ်။ Variable နှစ်ခုအကြားမှာ += တို့လို -= တို့လို Operator တွေကို ပူးတွဲပြီး ရေးသားနိုင်ပါတယ်။ ဒီလိုရေးသားတဲ့ နေရာမှာ ဘယ်ဘက်မှာ အစားထိုးချင်တဲ့ Variable ကိုရေးပြီး Operator တွေကို ရေးပြီးမှ Oponent ကို ရေးသားရမှာ ဖြစ်ပါတယ်။

```
jshell> a += 5
$31 ==> 15

jshell> a -= 1
$32 ==> 14

jshell> a *= 2
$33 ==> 28

jshell> a /= 1
$34 ==> 28

jshell> a %= 3
$35 ==> 1
```

အထက်ပါနမူနာမှာလို a += 5 လို့ ရေးထားရင် a ရဲ့ တန်ဖိုးနဲ့ 5 ကို ပေါင်းပြီးမှ ရတဲ့ တန်ဖိုးနဲ့ a ကို အစားထိုးမှာ ဖြစ်ပါတယ်။ တစ်ခြား Arathmetic Operator တွေကိုလဲ အလားတူရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

```
jshell> String str = "Hello "
str ==> "Hello "

jshell> str += "Java"
$37 ==> "Hello Java"
```

String Type Variable တွေမှာလဲပေါင်းပြီး အစားထိုးတဲ့ Operator မျိုးကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ str ရဲ့ တန်ဖိုးဟာ “Hello “ ဖြစ်တဲ့ အတွက် “Java” နဲ့ ပေါင်းပြီး အစားထိုးလိုက်တဲ့ အခါ str ရဲ့ တန်ဖိုးဟာ “Hello Java” ဖြစ်သွားမှာ ဖြစ်ပါတယ်။

Relational Operator

Variable နှစ်ခုရဲ့ တန်ဖိုးတွေကို နှိုင်းယှဉ်တဲ့နေရာမျိုးမှာ Relational Operator တွေကို အသုံးပြုနိုင်ပါတယ်။ နှိုင်းယှဉ်တယ်လို့ ပြောတဲ့နေရာမှာ Variable နှစ်ခုဟာ တူသလား မတူဘူးလား၊ ပြီးတော့ ဘယ်သူက ကြီးသလဲ ငယ်သလဲ အစရှိသဖြင့် နှိုင်းယှဉ်တာကို ဆိုလိုတာ ဖြစ်ပါတယ်။

အရင်ဆုံး တူမတူ နှိုင်းယှဉ်တဲ့ အပိုင်းကနေစပြီး လေ့လာသွားကြရအောင်။ ပုံမှန်အားဖြင့် a = b လို့ရေးရင် a နဲ့ b ဟာတူတယ်လို့ယူဆကြမှာ ဖြစ်ပေမဲ့ Java မှာတော့ a = b ဆိုတာက a နေရာကို b နဲ့ အစားထိုးတာလို့ ဆိုလိုပါတယ်။ a နဲ့ b နဲ့ တူတယ်ဆိုတာကို a == b နဲ့ ဖေါ်ပြလေ့ရှိပါတယ်။ Relational Operator တွေကို Variable နှစ်ခု အကြားမှာ ရေးသားရမှာ ဖြစ်ပြီး Expression ကို Evaluate လုပ်လိုက်တဲ့ အခါမျိုးမှာ boolean တန်ဖိုးတစ်ခုကို ရလဒ်အနေနဲ့ ရရှိမှာ ဖြစ်ပါတယ်။ a == b လို့ရေးထားရင် a ဟာ b နဲ့ တူရင် true ကို ရရှိမှာ ဖြစ်ပြီး မတူရင်တော့ false ကို ရရှိမှာ ဖြစ်ပါတယ်။

```
jshell> int a = 4, b = 5
a ==> 4
b ==> 5

jshell> a == b
$41 ==> false

jshell> a != b
$42 ==> true
```

အထက်ပါ နမူနာထဲမှာ a ကို 4 နဲ့ ပြီးတော့ b ကို 5 နဲ့ အစားထိုးထားပါတယ်။ အဲ့ဒါကြောင့် a == b လို့ ရေးသားထားတဲ့ Expression မှာ a နဲ့ b ဟာ မတူတဲ့ အတွက် false လို့ အဖြေထွက်တာ ဖြစ်ပါတယ်။ တဖန် a != b လို့ ရေးထားတာဟာ a နဲ့ b ဟာ မတူပါဘူးလို့ ရေးသားထားတာဖြစ်ပြီး တကယ်လဲ မတူတဲ့ အတွက် true လို့ အဖြေရတာ ဖြစ်ပါတယ်။

တူလားမတူဘူးလားဆိုတာကို နှိုင်းယှဉ်တဲ့ Operator ကို String Type Variable တွေမှာ အသုံးပြုနိုင်ပါတယ်။ String မှာအသုံးပြုတဲ့ == ရဲ့ ထူးခြားပုံကို လေ့လာသွားကြရအောင်။

Equality in Strings

Java မှာ အသုံးပြုနေတဲ့ String ဟာ အမတန်ထူးဆန်းတဲ့ Type အမျိုးအစားဖြစ်ပါတယ်။ String ဟာ Java Standard API ထဲမှာ ပါဝင်တဲ့ Class တစ်မျိုးဖြစ်ပြီး Data Type အနေနဲ့ အသုံးပြုနိုင်ပါတယ်။ String Data Type တွေဟာ Character Array တွေကို ကိုယ်စားပြုနိုင်တဲ့ Object တစ်မျိုးဖြစ်ပါတယ်။

Object ဖြစ်တဲ့ အတွက် String ဟာ Reference Data Type ဖြစ်ပါတယ်။ ဒါပေမဲ့ ထူးထူးခြားခြား String တစ်မျိုးထဲ Source Code တွေထဲမှာ Literals တွေကို အသုံးပြုပြီး တန်ဖိုးကို တိုက်ရိုက် အစားထိုးရေးသားနိုင်တာ ဖြစ်ပါတယ်။ String str = “Hello Java” လို့ရေးထားခဲ့ရင် “Hello Java” ဆိုတာက String ရဲ့ တန်ဖိုးကို Literal အနေနဲ့ ရေးသားထားတာ ဖြစ်ပါတယ်။

တဖန် Constructor ကို အသုံးပြုပြီး String str2 = new String(“Hello Java”) လို့ ရေးသားနိုင်ပါတယ်။ ဒါပေမဲ့ str နဲ့ str2 တို့ရဲ့ သဘာဝတွေဟာ မတူညီကြပါဘူး။

```
jshell> String str = "Hello Java"
str ==> "Hello Java"

jshell> String str1 = "Hello Java"
str1 ==> "Hello Java"

jshell> str == str1
$45 ==> true
```

အထက်ပါအတိုင်း str နဲ့ str1 တို့ကို Literal တန်ဖိုးဖြစ်တဲ့ “Hello Java” နဲ့ အစားထိုးရေးသားခဲ့ပါတယ်။ ပြီးတော့ str == str1 လို့ ရေးလိုက်တဲ့အခါမှာ true လို့ အဖြေရပါတယ်။

```
jshell> String str2 = new String("Hello Java")
str2 ==> "Hello Java"

jshell> str == str2
$47 ==> false
```

အထက်ပါ ကုဒ်ထဲမှာတော့ String str2 = new String(“Hello Java”) ဆိုပြီးရေးသားထားပြန်တယ်။ ပြီးတော့ str == str2 လို့ ရေးသားထားတဲ့ အခါမှာ မတူဘူးဆိုပြီး false လို့ အဖြေထွက်ပါတယ်။ ဒီနေရာမှာ သိသာတာက Literal နဲ့ ရေးသားထားတဲ့ Variable ရဲ့ တန်ဖိုးနဲ့ Constructor ကို အသုံးပြုပြီး တည်ဆောက်ထားတဲ့ တန်ဖိုးချင်း မတူဘူးလို့ ပြောနေတာလို့ သိထားသင့်ပါတယ်။

String Type Variable တွေကို Literal တွေနဲ့လဲ Assign လုပ်နိုင်သလို၊ Constructor မှတဆင့်လဲ Object ဆောက်ပြီး Assign လုပ်နိုင်ပါတယ်။ Literal နဲ့ ရေးသားထားတဲ့ String တွေကို Heap Memory ပေါ်မှာရှိတဲ့ String Pools ဆိုတဲ့ နေရာမှာ သိမ်းထားလေ့ရှိပါတယ်။ တကယ်လို့ Source Code ထဲမှာ “Hello Java” ဆိုပြီး Literal နဲ့ ရေးသားထားတာကို တွေ့ရင် အရင်ဆုံး String Pools ထဲမှာရှိလားလို့ကြည့်ပါတယ်။ ရှိတယ်ဆိုရင် အဲ့ဒီ Reference ကို အသုံးပြုပါတယ်။ တကယ်လို့ မရှိဘူးဆိုရင်တော့ String Pools ထဲမှာ String Object ကို ဆောက်ပြီးမှ အဲ့ဒီ Reference ကို အသုံးပြုမှာ ဖြစ်ပါတယ်။

ဤနည်းအားဖြင့် တူညီတဲ့ String Object တွေ အများကြီး ရှိမနေအောင် Memory ကို Optimize လုပ်ခဲ့ကြတာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် str == str1 ဟာ true လို့ အဖြေထွက်တာ ဖြစ်ပါတယ်။

ဒါပေမဲ့ String str2 = new String("Hello Java") လို့ ရေးလိုက်တာဟာ new Operator ကို အသုံးပြုထားတဲ့ အတွက် String Object အသစ်တစ်ခုကို Heap Memory ပေါ်မှာ တည်ဆောက်သွားမှာ ဖြစ်ပါတယ်။ String Pool ထဲက Reference ကို အသုံးပြုမှာ မဟုတ်ပါဘူး။ ဒါ့ကြောင့် str == str2 လို့ရေးလိုက်တဲ့အခါမှာ တန်ဖိုးတွေက တူရဲ့သားနဲ့ false ဆိုပြီး အဖြေထွက်တာ ဖြစ်ပါတယ်။

ဒါပေမဲ့ String တွေမှာတော့ Memory Address တူတာ မတူတာထက် သူတို့ရဲ့ တန်ဖိုးတူရင် တူတယ်လို့ အဖြေထွက်ချင်ကြမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် String တွေကို တူတယ်မတူဘူးဆိုတာကို စစ်ဆေးချင်ရင် equals method ကို အသုံးပြုပြီး စစ်ဆေးသင့်ပါတယ်။

```
jshell> String str2 = new String("Hello Java")
str2 ==> "Hello Java"

jshell> str == str2
$47 ==> false

jshell> str.equals(str2)
$48 ==> true
```

အခြားသော Relational Operators

Primitive တန်ဖိုးနှစ်ခု ဒါမှမဟုတ် Primitive Variable နှစ်ခုဟာ မတူပါဘူးဆိုတာကို စစ်ဆေးဖို့ != Operator ကို အသုံးပြုနိုင်ပါတယ်။

```
jshell> 10 != 11
$49 ==> true

jshell> 10 != 10
$50 ==> false
```

10 != 11 လို့ရေးထားရင် မတူတာမှန်တဲ့ အတွက် အဖြေဟာ true ဖြစ်မှာ ဖြစ်ပြီး 10 != 10 လို့ရေးထားရင် မတူဘူးဆိုတာ မှားနေတဲ့ အတွက် အဖြေဟာ false ဖြစ်ပါတယ်။

```
jshell> 10 > 10
$52 ==> false

jshell> 10 >= 10
$53 ==> true
```

အထက်ပါနမူနာမှာတော့ 10 > 10 လို့ရေးထားတာဟာ 10 ဟာ 10 ထက်ကြီးပါတယ်လို့ စစ်ဆေးနေတာပဲ ဖြစ်ပြီး 10 ဟာ 10 ထက်မကြီးတဲ့ အတွက် အဖြေဟာ false ထွက်ပါတယ်။ နောက်တစ်ကြောင်းမှာတော့ 10 >= 10 လို့ Expression ကို ရေးသားထားတဲ့ နေရာမှာ 10 ဟာ 10 ထက်ကြီးတယ်ဒါမှမဟုတ် တူတယ်လို့ ဖေါ်ပြထားတာ ဖြစ်ပါတယ်။ ဒီနေရာမှာတော့ 10 နဲ့ 10 ဟာတူညီတဲ့ အတွက် အဖြေဟာ true ဖြစ်သွားတာ ဖြစ်ပါတယ်။

အလားတူ Less Than နဲ့ Less Than or Equal To (<, <=) Operator တွေကိုလဲ အသုံးပြုနိုင်မှာဖြစ်ပါတယ်။ ဒီလို Relation Operator တွေနဲ့ Variable တွေကို တွဲဖက်ရေးသားထားတာတွေကို Relational Expression လို့ ခေါ်ဆိုလေ့ရှိပါတယ်။ သေခြာတာတစ်ခုကတော့ Relational Expression တွေရဲ့ ရလဒ်တွေဟာ Boolean တန်ဖိုး ဖြစ်တဲ့ true သို့မဟုတ် false ပဲ ဖြစ်နေမှာပဲ ဖြစ်ပါတယ်။

Logical Operators

Boolean တန်ဖိုးတွေ၊ Boolean Expression တွေနဲ့ တွဲဖက်ပြီး Logic တွေကို ရေးသားလိုတဲ့ အခါမျိုးမှာ အသုံးပြုနိုင်တဲ့ Operator တွေ ဖြစ်ကြပါတယ်။ အသုံးပြုနိုင်တဲ့ Operator တွေကတော့ AND (&), OR (|), Short Cut AND (&&), Short Cut OR (||), NOT (!) နဲ့ XOR (^) တို့ပဲ ဖြစ်ကြပါတယ်။

Not (!) Operator

Boolean တန်ဖိုးတွေ၊ Boolean Type Variable တွေနဲ့ Boolean Expression တွေရဲ့ ရှေ့မှာ ရေးသား အသုံးပြုနိုင်ပါတယ်။ လက်ရှိ Boolean ရဲ့ တန်ဖိုးကို ပြောင်းပြန်ပြုလုပ်ပေးနိုင်ပါတယ်။ Not True ဆိုရင် False ဖြစ်သွားမှာ ဖြစ်ပြီး Not False ဆိုရင်တော့ True ဖြစ်သွားမှာပါ။

```
jshell> !true
$64 ==> false

jshell> !false
$65 ==> true
```

AND (&) Operator

And Operator တွေကို Boolean တန်ဖိုးနှစ်ခု ဒါမှမဟုတ် Boolean Expression နှစ်ခုအကြားမှာ အသုံးပြုနိုင်ပြီး တန်ဖိုးနှစ်ခုလုံးမှန်တယ်ဆိုပါမှ true ကို ပြန်ပေးပါမယ်။ ဒါမှမဟုတ်ရင်တော့ false တန်ဖိုးကို ရရှိမှာ ဖြစ်ပါတယ်။

```
jshell> true & true
$58 ==> true

jshell> true & false
$59 ==> false

jshell> false & true
$60 ==> false

jshell> false & false
$61 ==> false
```

OR (|) Operator

And Operator တွေကို Boolean တန်ဖိုးနှစ်ခု ဒါမှမဟုတ် Boolean Expression နှစ်ခုအကြားမှာ တစ်ခုခုမှန်နေရင် True ကို ပြန်ပေးမှာ ဖြစ်ပြီး နှစ်ခုစလုံးမှားမှသာ False ကို ပြန်ပေးမှာ ဖြစ်ပါတယ်။

```
jshell> true | true
$66 ==> true

jshell> true | false
$67 ==> true

jshell> false | true
$68 ==> true

jshell> false | false
$69 ==> false
```

Short Cut Logical Operators

AND နဲ့ OR Operator တွေကို ကြည့်မယ်ဆိုရင် Boolean Expression နှစ်ခုအကြားမှာ ရေးပြီး အသုံးပြုရပါတယ်။ AND ကို ကြည့်မယ်ဆိုရင်လဲ Expression နှစ်ခုလုံး မှန်တယ်ဆိုမှသာ true ကို ရရှိမှာ ဖြစ်ပြီး OR ဆိုရင်လဲ နှစ်ခုလုံးမှားမှသာ false ကို ရရှိမှာ ဖြစ်ပါတယ်။ Logical Operator တွေမှာ ဘယ်ဘက်က Expression ကို အရင်လုပ်ပြီးမှ ညာဘက်က Expression ကို Evaluate လုပ်နေပါတယ်။ ဒါ့ကြောင့် AND Operator ရဲ့ ဘယ်ဘက်က Expression ရဲ့ တန်ဖိုးဟာ false ဖြစ်ခဲ့ပြီဆိုရင် ဘယ်လိုအကြောင်းကြောင့်မှ true ဖြစ်မလာနိုင်တော့ပါဘူး။ ဒီလိုပဲ OR Operator ရဲ့ ဘယ်ဘက်က Experssion ရဲ့ တန်ဖိုးက true ဖြစ်ခဲ့ပြီဆိုရင်လဲ ဘယ်လို အကြောင်းကြောင့်မှ false ဖြစ်မလာနိုင်တော့ပါဘူး။

ဒီလိုအနေအထားမျိုးမှာ သာမန် AND (&) နဲ့ OR (|) Operator တွေကို အသုံးပြုခဲ့ရင် ဘယ်ဘက်က Expression ကို Evaluate လုပ်ပြီး true ဖြစ်စရာ အကြောင်းမရှိခဲ့ရင်တောင်မှ၊ ဒါမှမဟုတ် false ဖြစ်စရာအကြောင်းမရှိခဲ့ရင်တောင်မှ၊ ညာဘက်က Expression ကို ဆက်ပြီး Evaluate လုပ်နေမှာ ဖြစ်ပါတယ်။ အဲ့ဒါတွေကို စမ်းသပ်ဖို့ Boolean တန်ဖိုးတွေကို ပြန်ပေးနိုင်တဲ့ Method လေးတွေကို ပြင်ထားကြည့်ကြရအောင်။

```
jshell> boolean getTrue() {
   ...> System.out.println("Call True");
   ...> return true;
   ...> }
|  created method getTrue()

jshell> boolean getFalse() {
   ...> System.out.println("Call False");
   ...> return false;
   ...> }
|  created method getFalse()
```

အထက်ပါ နမူနာမှာပါတဲ့ getTrue() method ကတော့ true ကို Return ပြန်ပေးနိုင်တဲ့ Method ဖြစ်ပြီး getFalse() method ကတော့ false ကို Return ပြန်ပေးနိုင်တဲ့ method ဖြစ်ပါတယ်။ Method တွေထဲမှာတော့ ဘယ် Method ကို Evluate လုပ်သွားတယ်ဆိုတာကို သိအောင် System.out.print တွေကို ရေးသားထားပါတယ်။

```
jshell> getFalse() & getTrue()
Call False
Call True
$3 ==> false
```

အထက်ပါ နမူနာထဲမှာဆိုရင် getFalse() ကို Evaluate လုပ်ပြီးတဲ့အခါမှာ false ကို ရမှာ ဖြစ်တဲ့ အတွက် AND Expression ရဲ့ ရလဒ်ဟာ ဘယ်တော့မှ true ဖြစ်လာစရာ အကြောင်းမရှိတော့ပါဘူး။ ဒါပေမဲ့ ညာဘက်က getTrue() Expression ကို Evaluate လုပ်သွားတဲ့ အတွက် Call False လို့ဖေါ်ပြပြီးတဲ့ အခါမှာ Call True ကို ဖေါ်ပြသွားတာ ဖြစ်ပါတယ်။

```
jshell> getFalse() && getTrue()
Call False
$5 ==> false
```

အထက်ပါနမူနာကတော့ AND (&) အစား Short Cut AND (&&) ကို အသုံးပြုသွားတာပါ။ ဘယ်ဘက်က Expression ဖြစ်တဲ့ getFalse() ကို Evaluate လုပ်ပြီးတဲ့အခါမှာ ရလဒ်ဟာ false ဖြစ်နေပြီဆိုရင် ဘယ်နည်းနဲ့မှ true ဖြစ်စရာမရှိတော့တဲ့ အတွက် ညာဘက်က Expression ဖြစ်တဲ့ getTrue() ကို Evaluate မလုပ်တော့ပဲ တန်းပြီး အဖြေကို false လို့ထုတ်ပေးခဲ့ပါတယ်။

```
jshell> getTrue() || getFalse()
Call True
$6 ==> true
```

အလားတူပဲ Short Cut OR (||) Operator ဟာလဲ ဘယ်ဘက်က Expression ကို Evaluate လုပ်ပြီး အဖြေဟာ True ဖြစ်ခဲ့ရင် ဘယ်နည်းနဲ့မှ False ဖြစ်စရာအကြောင်းမရှိတော့တဲ့ အတွက် ညာဘက်က Expression ဖြစ်တဲ့ getTrue() method ကို Invoke မလုပ်တော့ပဲ အဖြေကို တန်းပြီး true လို့ ထုတ်ပေးခဲ့တာဖြစ်ပါတယ်။

Short Cut တွေဟာ ဘယ်ဘက်က Expression ရဲ့ တန်ဖိုးကို ကြည့်ပြီး အဖြေကိုမရသေးဘူးဆိုရင်တော့ ညာဘက်က Expression ကို ဆက်ပြီး Evaluate လုပ်သွားမှာ ဖြစ်ပါတယ်။

```
jshell> getFalse() || getTrue()
Call False
Call True
$7 ==> true
```

အထက်ပါနမူနာမှာဆိုရင် getFalse() ကိုခေါ်တဲ့အခါမှာ false ဖြစ်နေတဲ့ အတွက် နောက်တစ်ခုက false ဖြစ်ရင် false ဖြစ်ပြီး true ဖြစ်ရင် true ဖြစ်နေနိုင်ပါသေးတယ်။ ဒါ့ကြောင့် ညာဘက်က Expression ဖြစ်တဲ့ getTrue() Method ကို ဆက်ပြီး Evaluate လုပ်သွားပြီးမှာ အဖြေကို ပြန်ပြီးဖေါ်ပြပေးသွားပါတယ်။

AND ပဲ သုံးသုံး Short Cut AND ပဲသုံးသုံး၊ OR ပဲ သုံးသုံး Short Cut OR ပဲ သုံးသုံး နောက်ဆုံးရလဒ်ကတော့ အတူတူပဲ ဖြစ်ပါမယ်။ ဒါပေမဲ့ Short Cut တွေက ဘယ်ဘက်က တန်ဖိုးကိုကြည့်ပြီး အဖြေကို ရသွားရင် ညာဘက်က Expression တွေကို Evaluate လုပ်စရာမလိုတဲ့ အတွက် Performance မှာပိုပြီး ကောင်းမွန်စေနိုင်ပါတယ်။

XOR (^) Operator

Exclusive OR ရဲ့ အတိုကောက် အခေါ်အဝေါ်ဖြစ်ပြီး တစ်ဘက်ဘက်မှာ true ပါမှ true ဖြစ်မှာ ဖြစ်ပြီး နှစ်ဘက်လုံး true ကြီးပဲ ဒါမှမဟုတ် false ကြီးပဲ ဖြစ်နေရင်တော့ false ကိုပဲရရှိမှာ ဖြစ်ပါတယ်။

```
jshell> true ^ true
$14 ==> false

jshell> false ^ true
$15 ==> true

jshell> true ^ false
$16 ==> true

jshell> false ^ false
$17 ==> false
```

Notes

Logical Operator တွေကို Boolean Expression တွေကြားမှာ အသုံးပြုတာတွေကိုပဲ ဖေါ်ပြခဲ့ပါတယ်။ ဒါပေမဲ့ Logical Operator တွေကို အခြားသော Primitive Type တွေကြားမှာလဲ အသုံးပြုနိုင်ပါတယ်။ အဲ့ဒီလို အသုံးပြုတဲ့ အခါမှာ Binary Data အဖြစ်ပြောင်းပြီးတွက်ချက်သွားမှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ Business Application ရဲ့ Logic တွေကို ရေးသားကြတဲ့အခါမျိုးမှာ Boolean Expression တွေကို အသုံးပြုတာကများတဲ့အတွက် ဒီစာအုပ်ထဲမှာတော့ Boolean Expression နဲ့ တွဲဖက်အသုံးပြုတဲ့ နမူနာများကို ဖေါ်ပြသွားတာ ဖြစ်ပါတယ်။

Statements

Java Compilar ကနေ Compile လုပ်နိုင်ပြီး JVM ကနေ လုပ်ဆောင်မှုတွေကို လုပ်ဆောင်ပေးနိုင်တဲ့ အဓိပ္ပါယ်ပြည့်စုံတဲ့ စာကြောင်းတွေကို Statement လို့ခေါ်ပါတယ်။ အောက်က နမူနာအတိုင်း Class တစ်ခုကို ရေးသားထားပါမယ်။

```
public class Hello {
      String str = "Hello"
}
```

အထက်ပါအတိုင်းရေးထားတဲ့ Class ကို Compile လုပ်လိုက်တဲ့အခါမှာ Error တက်ပါလိမ့်မယ်။ String Type Variable ဖြစ်တဲ့ str ကို Assignment Operator ဖြစ်တဲ့ = ကို အသုံးပြုပြီး “Hello” နဲ့ အစားထိုးနေတာမှန်ပါတယ်။ ဒါပေမဲ့ အဲ့ဒီ စာကြောင်းဟာ ပြီးပြည့်စုံတဲ့ ဝါကျတစ်ကြောင်း မဟုတ်သေးပါဘူး။ ဒါ့ကြောင့် Compile Time မှာ Error ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။ ကဲ ဒီအခန်းမှာတော့ ပြီးပြည့်စုံတဲ့ ဝါကျ ဒါမှမဟုတ် Statement တွေကို ဘယ်လို ရေးရမယ်၊ ပြီးတော့ Statement အမျိုးအစားတွေ ဘယ်လိုရှိတယ်ဆိုတာကို လေ့လာသွားကြရအောင်။

Statement တွေကို အမျိုးအစားခွဲခြားမယ်ဆိုရင် အောက်ပါအတိုင်းခွဲခြားနိုင်မှာ ဖြစ်ပါတယ်။

✓ Declaration Statements
Variable တွေကို အသုံးပြုမယ်ဆိုပြီး Declare လုပ်တဲ့အခါမှာ ရေးသားနိုင်တဲ့ ဝါကျအမျိုးအစားမျိုးဖြစ်ပါတယ်

✓ Expression Statements
Variable တွေနဲ့ Operator တွေကို တွဲဖက်ရေးသားထားတာကို Expression လို့ခေါ်တယ်ဆိုတာ ရှေ့ခန်းမှာ လေ့လာခဲ့ပြီးပါပြီ။ အချို့သော Expression တွေကို နောက်မှာ ; ကိုရေးလိုက်ရုံနဲ့ Statement အနေနဲ့ အသုံးပြုနိုင်ပါတယ်။

✓ Control Flows Statements
Code Block တွေနဲ့ Method တွေထဲမှာ ရေးသားနိုင်တဲ့ Statement တွေဖြစ်ကြပြီး၊ Program ရဲ့ Flow တွေကို Control လုပ်ပေးနိုင်တဲ့ Statement တွေဖြစ်ကြပါတယ်။ အဓိကအားဖြင့် Decision Statements တွေနဲ့ Looping Statement တွေဆိုပြီး ရှိကြပါတယ်။ ဒီလိုအနေအထားမျိုးဆိုရင် ဒီလိုလုပ်မယ်ဒါမှမဟုတ်ရင်ဒီလိုလုပ်မယ်ဆိုတဲ့ ရေးသားပုံတွေကို Decision Statement တွေနဲ့ ရေးသားဖော်ပြနိုင်ပြီး။ အခြေအနေအပေါ်မူတည်ပြီး အကြိမ်ကြိမ် လုပ်ဆောင်မှုတွေကို ပြုလုပ်စေလိုတဲ့ အခါမျိုးတွေမှာ Looping Statement တွေကို အသုံးပြုနိုင်ပါတယ်။

Declaration Statements

Java Program တွေထဲမှာ အသုံးပြုမည့် Variable တွေကို Declare လုပ်တဲ့အခါမှာ အသုံးပြုနိုင်တဲ့ Statement တွေ ဖြစ်ပါတယ်။ Variable တွေကို Declare လုပ်တဲ့အခါမှာ Type နဲ့ Name တို့ကို ရေးသားရန်လိုအပ်ပြီး အနောက်မှာ ; ကို ရေးသားလိုက်ရင်ရပါပြီ။ အခြေခံအားဖြင့် Variable တွေကို Declare လုပ်တဲ့ နေရာမှာ ပုံစံနှစ်မျိုးရှိနိုင်ပါတယ်။ အဲ့ဒါတွေကတော့ Variable တွေကို ရိုးရိုး Declare လုပ်တဲ့ပုံစံနဲ့ Initial တန်ဖိုးတွေကို Assign လုပ်ပြီး Declare လုပ်တဲ့ ပုံစံဆိုပြီး နှစ်မျိုးရှိနိုင်ပါတယ်။

```
public class Hello {

      String str1;

      public Hello() {
            str1 = "Hello Java";
      }
}
```

အထက်ပါပုံစံကတော့ သာမန် ရိုးရိုး Declare လုပ်တဲ့ ပုံစံပါ။ String str1; ဆိုတာက str1 Variable ကို Declare လုပ်နေတဲ့ Declaration Statement ဖြစ်ပါတယ်။ ပြီးတော့မှ Constructor ထဲမှာ str1 = “Hello Java”; ဆိုပြီး Assignment လုပ်နေပါတယ်။

```
public class Hello {

      public Hello() {
            String str1;
      }
}
```

အထက်ပါနမူနာကတော့ String str1; ကို Constructor ထဲမှာ ရေးသားထားတဲ့ အတွက် Local Variable တစ်ခုကို Declare လုပ်နေတာပဲ ဖြစ်ပါတယ်။ Local Variable တွေကို အသုံးပြုတဲ့ နေရာမှာ တစ်ခု သတိထားသင့်တာကတော့ Initialize မလုပ်ပဲ အသုံးပြုလို့ မရတဲ့ အချက်ဖြစ်ပါတယ်။ Local Variable တွေဟာ Default တန်ဖိုးပါမလာတဲ့ အတွက် တန်ဖိုးကို Assign မလုပ်ခင် အသုံးပြုလို့ မရပါဘူး။

```
public class Hello {

      public Hello() {
            String str1 = “Hello Java”;
      }
}
```

အထက်ပါနမူနာကတော့ တန်ဖိုးကို အစားထိုးပြီး Declare လုပ်တဲ့ ပုံစံဖြစ်ပါတယ်။

Expression Statements

Expression တွေရဲ့ နောက်မှာ ; ရေးလိုက်တာနဲ့ Statement ဖြစ်သွားတဲ့ Statement မျိုးကို Expression Statement လို့ခေါ်ကြပါတယ်။ များသောအားဖြင့် Assignment Expression တွေ၊ Increment, Decrement Expressionတွေ၊ Method Invocation Expression တွေနဲ့ Object Creation Expression တွေကို Statement အနေနဲ့ အသုံးပြုနိုင်ပါတယ်။

```
public class Hello {

	String str1;

	public Hello() {
		str1 = "Hello Java";
	}
}
```

အထက်ပါနမူနာမှာ str1 = “Hello Java” လို့ရေးသားထားတာဟာ str variable နေရာကို “Hello Java” နဲ့ အစားထိုးနေတဲ့ Assignment Expression ဖြစ်ပြီး နောက်ကနေ ; ကိုရေးသားလိုက်တာနဲ့ Assign Statement တစ်ကြောင်း ဖြစ်လာပါတယ်။

```
public class Hello {

	int count;

	public void countUp() {
		count ++;
	}

	public static void main(String [] args) {
		Hello h;
		h = new Hello();
		h.countUp();
	}
}
```

အထက်ပါနမူနာထဲမှာ countUp() method ထဲမှာ count ++; လို့ရေးသားထားတာဟာ count ကို တစ်တိုးပေးတဲ့ increment statement ဖြစ်ပါတယ်။ ပြီးတော့ main method ထဲမှာ ရေးသားထားတဲ့ h = new Hello(); ဆိုတာက Object Creation Statement ဖြစ်ပြီး h.countUp(); ဆိုတာကတော့ Method Invocation Statement ဖြစ်ပါတယ်။

ဖေါ်ပြပါအတိုင်း Expression တွေရဲ့ အနောက်မှာ ; ကို ရေးလိုက်ရုံနဲ့ Statement ဖြစ်သွားတဲ့ Statement တွေဟာ Expression Statement တွေ ဖြစ်ကြပါတယ်။

Control Flow Statements

Program တစ်ခုရဲ့ အသွားအလာတွေကို Flow လို့ခေါ်ပါတယ်။ အဲ့ဒီ Flow တွေကို Control လုပ်ပေးနိုင်တဲ့ Statement တွေကို Control Flow Statement တွေလို့ခေါ်ပါတယ်။ Java Program ဆိုတာဘာလဲ ဆိုတာကို ပြန်ပြီး တွေးကြည့်ကြရအောင်။ Java Programming ဘာသာရပ်နဲ့ ရေးသားထားတဲ့ အစီအစဉ် အစုအဝေးတစ်ခုသာ ဖြစ်ပါတယ်။

ပြီးခဲ့တဲ့ အခန်းတွေတုန်းက Variable တွေ၊ Java Data Type တွေ၊ Method တွေ၊ Class တွေ နဲ့ Operator တွေ အကြောင်းကို လေ့လာခဲ့ကြပါတယ်။ ဒါပေမဲ့ Method တွေထဲမှာ ကုဒ်တွေကို ဘယ်လို ရေးရမယ်ဆိုတာကိုတော့ မသိရသေးပါဘူး။ ဘယ်လိုလုပ်ရမယ်ဆိုတာကို ရေးသားနိုင်တာတွေကတော့ Control Flow Statements တွေပဲ ဖြစ်ကြပါတယ်။

Controls Flow Statement အနေနဲ့ Decision Statement နဲ့ Looping Statement ဆိုပြီးအမျိုးအစားခွဲခြားထားပါတယ်။ အခြေအနေတစ်ခုအပေါ်မူတည်ပြီး Program အသွားအလာတွေကို ခွဲခြားပေးနိုင်တဲ့ Statement တွေက Decision Statemebt တွေဖြစ်ကြပြီး အကြိမ်ကြိမ် လုပ်ဆောင်မှုတွေကို ရေးသားနိုင်အောင်ပြင်ပေးထားတဲ့ Statement တွေကတော့ Looping Statement တွေပဲ ဖြစ်ကြပါတယ်။

Decision Statement

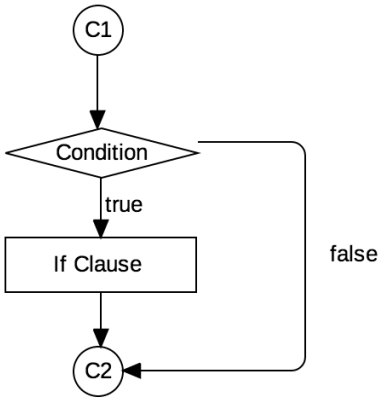
ဆိုကြပါစို့၊ ကိန်းပြည့်တစ်ခုကို ကြည့်ပြီး စုံကိန်းလား မကိန်းလားဆိုတာကို ခွဲခြားချင်တယ်။ ဒါကို ဘယ်နေရာမှာသုံးရမလဲ။ သုံးလို့ရတဲ့ နေရာတွေ အများကြီးရှိပါတယ်။ View ပေါ်မှာ Table တစ်ခုရဲ့ Row တွေကို စုံမပေါ်မှာခွဲပြီး အရောင်မတူအောင်ဖေါ်ပြချင်ရင် ဒီလိုမျိုး ကိန်းတစ်ခုကို ကြည့်ပြီး စုံလားမလားဆိုတာကို ခွဲခြားရတော့မှာပါပဲ။

စုံဆိုရင် အဖြူရောင်ကိုဖေါ်ပြပြီး မဆိုရင် မီးခိုးရောင်နဲ့ ဖေါ်ပြချင်တယ်ဆိုရင် အရင်ဆုံး Row ရဲ့ Index Number ဟာ စုံလားမလားဆိုတာကို ခွဲခြားဖို့လိုပါတယ်။ ဒီလိုအနေအထားမျိုးဆိုရင် ကျွန်တော်တို့ဟာ Decision Statement တွေကို အသုံးပြုတော့မှာ ဖြစ်ပါတယ်။

Java Programming Language မှာ အသုံးပြုနိုင်တဲ့ Decision Statement တွေကတော့ အောက်ပါ အတိုင်း ဖြစ်ကြပါတယ်။

If Statement

အနေအထားတစ်ခုအပေါ်မူတည်ပြီး မှန်ကန်တဲ့အခါမှ လုပ်ဆောင်မှုတွေကို လုပ်ဆောင်စေလိုတဲ့အခါမျိုးမှာ If Statement ကို ရေးသား အသုံးပြုနိုင်ပါတယ်။




```
Program Flow C1

if(Condition) {
    If Clause
}

Program Flow C2
```

If Statement ကို ရေးသားမယ်ဆိုရင် အရင်ဆုံး Condition တစ်ခုကို if (Condition) ကို ရေးသားရမှာ ဖြစ်ပါတယ်။ ပြီးမှ Condition သာမှန်ခဲ့ရင် အလုပ်လုပ်စေချင်တဲ့ ကုဒ်တွေကို { If Clause } ထဲမှာ ရေးသားရမှာ ဖြစ်ပါတယ်။ Condition နေရာမှာ Boolean တန်ဖိုးတစ်ခုကို ပြန်ပေးနိုင်တဲ့ ရေးသားပုံ (Boolean Expression) တစ်ခုခု ကို ရေးသားရမှာ ဖြစ်ပါတယ်။

Boolean Expression ဆိုတာ ရေးသားထားတဲ့ Expression ကို Evaluate လုပ်ပြီးတဲ့အခါမှာ နောက်ဆုံး Boolean တန်ဖိုးတစ်ခု ထွက်လာနိုင်တဲ့ Expression မျိုးဖြစ်ပါတယ်။ အောက်ပါ Expression တွေက Boolean တန်ဖိုးကို ပြန်ပေးနိုင်တဲ့ Expression တွေပဲ ဖြစ်ပါတယ်။

- Boolean Type Literals (true or false)
- Boolean Type Variables
- Boolean Type ကို Return ပြန်စေတဲ့ Method Invocations
- Relational Expressions
- Logical Expressions

ဒီနေရာမှာ Literals တွေရေးတာတော့ ရကောင်းရပါမယ်။ ဒါပေမဲ့ လုံးဝ အဓိပ္ပါယ်မရှိပါဘူး။ if(true) လို့ရေးထားရင် အမြဲတမ်း မှန်နေမှာ ဖြစ်ပြီး if(false) လို့ရေးထားရင် အမြဲတမ်းမှားနေမှာ ဖြစ်တဲ့ အတွက် if Statement က အလုပ်လုပ်မှာ မဟုတ်ပါဘူး။ ကြိုသိပြီးသား ကိစ္စတစ်ခု ဖြစ်တဲ့ အတွက် ဆုံးဖြတ်စရာတောင်မလိုအပ်ပါဘူး။ ကျွန်တော်တို့ JShell ကို အသုံးပြုပြီး စမ်းသပ်ကြည့်ကြရအောင်။

```
jshell> int a = 10
a ==> 10

jshell> if (a == 10) {
   ...> System.out.println("A is " + 10);
   ...> }
A is 10

jshell> if (a > 10) {
   ...> System.out.println("A is greater than " + 10);
   ...> }
```

အထက်ပါနမူနာထဲမှာ a ရဲ့ တန်ဖိုးကို 10 နဲ့ အစားထိုးထားပါတယ်။ ပြီးတော့ အောက်မှာ a == 10 ဆိုတာက Condition အပိုင်ဖြစ်တဲ့ Boolean Expression ပါ။ a က 10 ဖြစ်ရင် true ကိုပြန်ပြီး မဖြစ်ရင် false ကို ပြန်ပေးမဲ့ Expression တစ်ခုဖြစ်ပါတယ်။

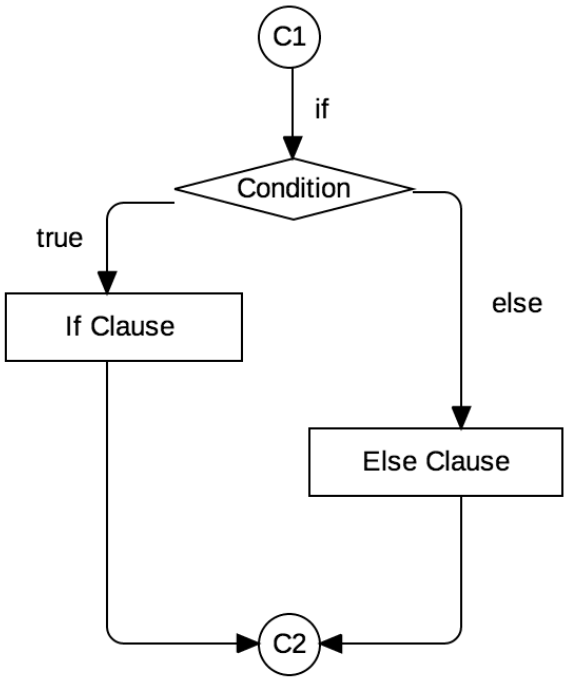
If Clause ထဲမှာတော့ Condition ကို စစ်ပြီးမှန်ရင် အလုပ်လုပ်မည့် ကုဒ်တွေကို ရေးထားပါတယ်။ အထက်ပါ Condition က မှန်တဲ့ အတွက် ရေးသားထားတဲ့ အတိုင်း A is 10 ဆိုပြီး အလုပ်လုပ်သွားတာကို တွေ့မရပါမယ်။

ဆက်လက်ပြီး မမှန်တဲ့ Expression တစ်ခုကို အောက်မှာရေးသားထားပါတယ်။ a > 10 လို့ ရေးသားထားပါတယ်။ a ဟာ 10 ဖြစ်တဲ့ အတွက် a ဟာ 10 ထက်မကြီးပါဘူး။ ဒါ့ကြောင့် a > 10 Expression ကို Evaluate လုပ်ရင် false ပဲရမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် If Clause ထဲမှာ ရေးသားထားတဲ့ Code တွေကို အလုပ်လုပ်စေမှာ မဟုတ်တော့ပါဘူး။

ဒါ့ကြောင့် IF Statement တွေကို Condition မှာရေးသားတားတဲ့ အတိုင်းမှန်ရင် အလုပ်လုပ်စေလိုတဲ့အခါမျိုးမှာ ရေးသား အသုံးပြုသင့်ပါတယ်။

IF / ELSE Statement

IF / ELSE Statement တွေကို ရေးသားထားတဲ့ Expression အပေါ်မှာမူတည်ပြီး မှန်ရင်တစ်မျိုး အလုပ်လုပ်စေလိုပြီး မှားရင် တစ်မျိုးလုပ်စေလိုတဲ့ အခါမျိုးမှာ အသုံးပြုနိုင်ပါတယ်။




```
Program Flow C1

if(Condition) {

    If Clause

} else {

    Else Clause
}

Program Flow C2
```

If Else Statement တွေကလဲ IF Statement နဲ့ အတော်လေးဆင်တူပါတယ်။ မတူတာကတော့ တကယ်လို့ Condition ကို စစ်ဆေးတဲ့နေရာမှာ မဟုတ်ခဲ့ရင် ဘာလုပ်မလဲဆိုတာကို ရေးသားထားနိုင်တာပါပဲ။ အထက်ပါအတိုင်း If Clause နဲ့ Else Clause တိုကို ရေးသားထားနိုင်ပြီး တကယ်လို့ Condition ကမှန်ခဲ့ရင် If Statement က အလုပ်လုပ်မှာ ဖြစ်ပြီး မှားခဲ့ရင် Else Clause ထဲက လုပ်ဆောင်ချက်တွေကို လုပ်ဆောင်စေမှာ ဖြစ်ပါတယ်။

```
jshell> int a = 10
a ==> 10

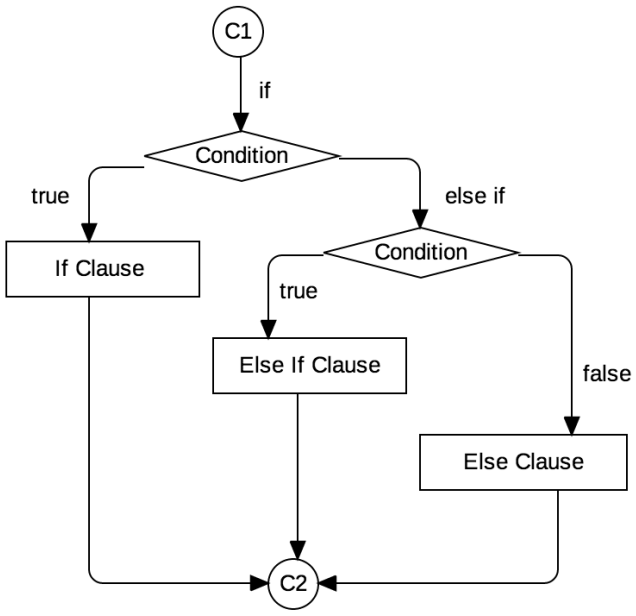
jshell> if (a > 10) {
   ...> System.out.println("A is greater than " + 10);
   ...> } else {
   ...> System.out.println("A is not greater than " + 10);
   ...> }
A is not greater than 10
```

အထက်ပါနမူနာမှာ ဖေါ်ပြထားတဲ့ အတိုင်း a ကို 10 နဲ့ အစားထိုးပါတယ်။ ပြီးတော့ Condition မှာ a > 10 လို စစ်ထားပါတယ်။ ဒါပေမဲ့ a ဟာ 10 ဖြစ်တဲ့ အတွက် a ဟာ 10 ထက်မကြီးတဲ့ အတွက် IF Clause ကို အလုပ်မလုပ်တော့ပဲ Else Clause ကိုပဲအလုပ်လုပ်သွားမှာ ဖြစ်ပါတယ်။

ဒါ့ကြောင့် Else Clause ထဲမှာ ရေးသားထားတဲ့ ကုဒ်တွေကို အလုပ်လုပ်သွားမှာ ဖြစ်တဲ့ အတွက် A is not greater than 10 ဆိုပြီး ဖေါ်ပြသွားတာကို တွေ့ရပါမယ်။

IF / ELSE IF / ELSE Statement

ဒီ Statement ကတော့ Condition အမျိုးအမျိုးအပေါ်မူတည်ပြီး မိမိရဲ့ Program ရဲ့ Flow ကိုပြောင်းလဲလိုတဲ့ အခါမျိုးမှာ ရေးသားအသုံးပြုနိုင်ပါတယ်။



Program Flow C1

```
if(Condition) {
    If Clause
} else if (Condition) {
    Else If Clasue
} else {
    Else Clause
}
```

Program Flow C2

ပထမ Conditon ကို စစ် ကိုက်ရင်တစ်မျိုးလုပ်၊ မကိုက်ရင် နောက် Condition တစ်မျိုးကို စစ် ကိုက်ရင်တစ်မျိုးလုပ် မကိုက်ရင် နောက်ထပ် Condition အမျိုးမျိုးကို ထပ်စစ်ဆေးနိုင်ပါတယ်။ နောက်ဆုံးမှ မစစ်ချင်ရင်လဲ Else Clause ကို ရေးသားထားလို့ရပါတယ်။

```
jshell> /list showGrade

   8 : void showGrade(int mark) {
           if(mark < 40) {
               System.out.println("Fails");
           } else if (mark >= 40 && mark < 80) {
               System.out.println("Pass");
           } else if (mark >= 80 && mark < 100) {
               System.out.println("Excellent");
           } else if (mark == 100) {
               System.out.println("Perfect");
           } else {
               System.out.println("Impossible");
           }
       }
```

အထက်ပါအတိုင်း showGrade Method ကို ရေးသားထားပါတယ်။ Argument အနေနဲ့ int mark ကို ရယူနေပါတယ်။ အမှတ်အပေါ်မူတည်ပြီး အဆင့်ကို ခွဲခြားပေးနိုင်တဲ့ Method တစ်ခုပါ။

ပထမဆုံး if ရဲ့ Decision အနေနဲ့ mark < 40 လို့ရေးထားပါတယ်။ mark သာ 40 ထက်ငယ်ရင် if block ထဲက ကုဒ်တွေကို အလုပ်လုပ်စေမှာ ဖြစ်ပြီ “Fails” လို့ ဖေါ်ပြသွားမှာ ဖြစ်ပါတယ်။ အဲ့ဒီ Condition မမှန်တော့မှ အခြား Condition တွေကို ပြန်ပြီး စစ်ဆေးမှာ ဖြစ်ပါတယ်။

```
jshell> showGrade(39)
Fails
```

ဒါ့ကြောင့် အထက်ပါ အတိုင်း showGrade(39) လို့ ရေးတဲ့အခါမှာ 39 ဟာ 40 ထက်ငယ်နေတဲ့ အတွက် if Condition နဲ့ ကိုက်ညီပြီး Fails လို့ဖေါ်ပြသွားတာဖြစ်ပါတယ်။

```
jshell> showGrade(40)
Pass

jshell> showGrade(79)
Pass
```

ပြီးတော့ ဒုတိယ Condition ဖြစ်တဲ့ else if ထဲမှာ mark >= 40 && mark < 80 လို့ရေးထားပါတယ်။ ဒါကြောင့် ပထမ Condition နဲ့ မကိုက်တာတွေအားလုံးဟာ ဒုတိယ Condition ထဲကို ရောက်လာကြပါတယ်။ အထက်ပါ နမူနာထဲမှာတော့ showGrade Method ကို Parameter အနေနဲ့ 40 နဲ့ 79 ကို အသုံးပြုထားကြပါတယ်။ 40 ကော 79 ပါ 40 ဒါမှမဟုတ် 40 ထက်ကြီးနေပြီး 80 ထက်လဲ ငယ်နေကြပါတယ်။ ဒါ့ကြောင့် Pass ဆိုပြီး ဖေါ်ပြနိုင်ကြတာ ဖြစ်ပါတယ်။

```
jshell> showGrade(80)
Excellent

jshell> showGrade(99)
Excellent

jshell> showGrade(100)
Perfect

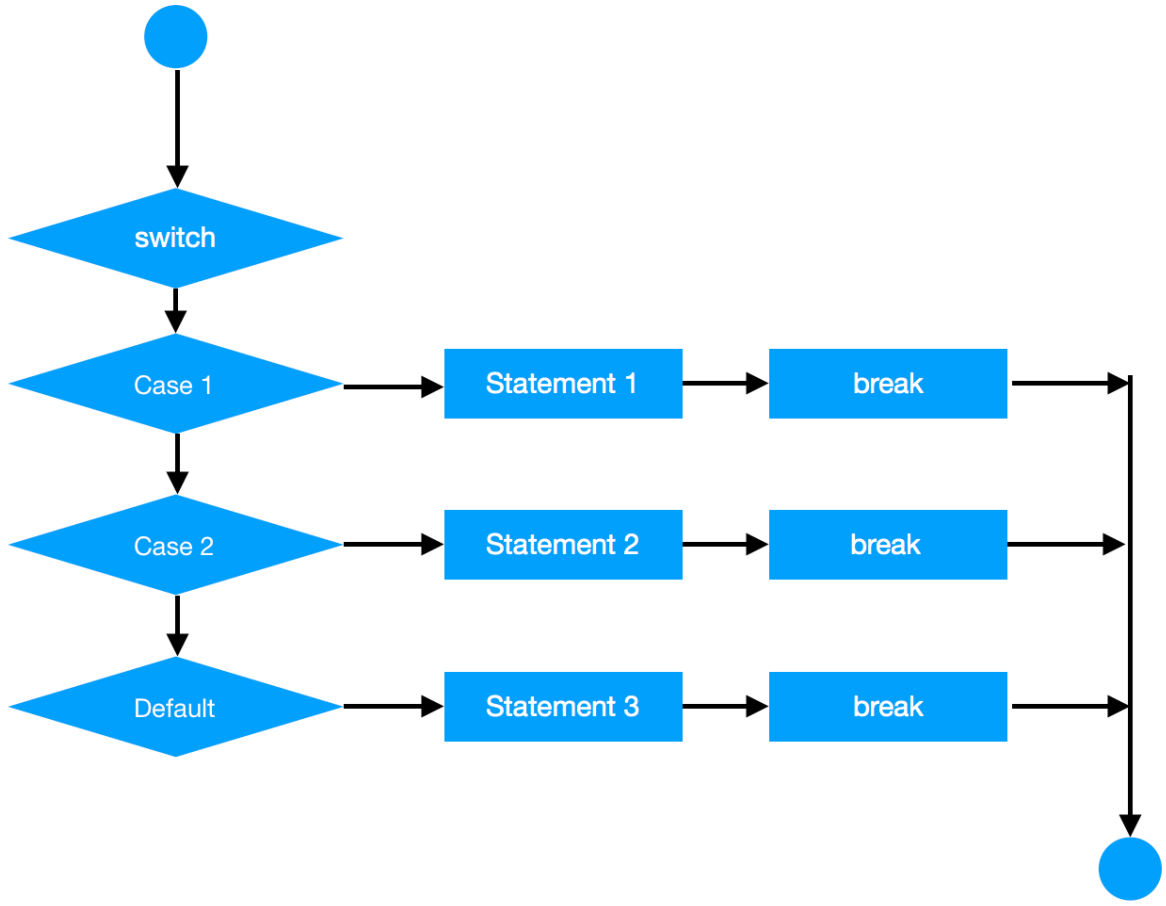
jshell> showGrade(101)
Impossible
```

အထက်ပါနမူနာထဲမှာဖေါ်ပြထားတဲ့ အတိုင်း 80 ဒါမှမဟုတ် 99 နဲ့ showGrade Method ကို ခေါ်တဲ့ အခါမှာ Excellent ဆိုတာကို ဖေါ်ပြနေတာကို တွေ့ရပါမယ်။ ဒါဟာ တတိယ Condition ထဲမှာ ရေးသားထားတဲ့ mark >= 80 && mark < 100 နဲ့ ကိုက်ညီသွားလို့ ဖြစ်ပါတယ်။

နောက်ပြီး showGrade(100) လို့ Method ကို Invoke လုပ်လိုက်တဲ့ အခါမှာလဲ Excellent လို့ ဖေါ်ပြနိုင်တာကို တွေ့ရပါတယ်။ ဒါကလဲ စတုတ္ထ Condition ဖြစ်တဲ့ mark == 100 နဲ့ ကိုက်ညီတဲ့ အတွက် ဖေါ်ပြပေးသွားတာ ဖြစ်ပြီး နောက်ဆုံး 101 နဲ့ Method ကို Invoke လုပ်တဲ့အခါ အထက်ဖေါ်ပြပါ Condition တစ်ခုနဲ့မှ မကိုက်ညီပဲ Else Clause ထဲက လုပ်ဆောင်ချက်တွေကိုလုပ်ပေးသွားတဲ့ အတွက် Impossible ဆိုတာကို ဖေါ်ပြပေးသွားတာ ဖြစ်ပါတယ်။ ဒီ Statement မျိုးကို Condition အမျိုးမျိုးအပေါ် လုပ်ဆောင်မှုတွေခွဲခြားပြီး လုပ်ဆောင် စေလိုတဲ့အခါမှာ အသုံးပြုနိုင်ပါမယ်။

Switch Statement

Switch Statement တွေဟာ if - else if - else statement တွေနဲ့ တူညီသလိုရှိပါတယ်။ Decision အများအပေါ်မှာ မူတည်ပြီး လုပ်ဆောင်မှုတွေကို ခွဲခြားလုပ်ဆောင် စေနိုင်မှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ if - else if - else statement တွေမှာတော့ Decision တွေကို လိုအပ်သလို ရေးသားနိုင်ပေမဲ့၊ switch statement တွေမှာတော့ decision တွေကို ကြိုတင်သတ်မှတ်ထားတဲ့ တန်ဖိုးအပေါ်မှာ မူတည်ပြီး လုပ်ဆောင်မှုတွေကို ခွဲခြားလုပ်ဆောင် စေမှာ ဖြစ်ပါတယ်။



Switch Statement တွေဟာ တန်ဖိုးတစ်ခုအပေါ်မှာ မူတည်ပြီး လုပ်ဆောင်မှုမတွေကို ခွဲခြားပြီး လုပ်ဆောင်ပေးနိုင်ပါတယ်။ Switch မှာ အသုံးပြုနိုင်တဲ့တန်ဖိုးတွေကတော့ Primitive Data Type ထဲကတော့ char, byte, short နဲ့ int တို့ကို အသုံးပြုနိုင်ပါတယ်။ Java SE 5 အနောက်မှာ ပါဝင်လာခဲ့တဲ့ enum type တွေကိုလဲ Switch Statement ရဲ့ တန်ဖိုးအနေနဲ့ အသုံးပြုနိုင်ပါတယ်။ Java SE 7 ကနေစပြီးတော့လဲ String Type Variable တွေကိုလဲ Switch Statement ရဲ့ Value အနေနဲ့ အသုံးပြုလာနိုင်ပါတယ်။

Switch Statement တွေထဲမှာ Case တွေကို တစ်ခုထက်မက ရေးသားလို့ရနိုင်ပါတယ်။ Case တွေမှာ Switch လုပ်မည့် Value ရဲ့ Type အလိုက် Literals တွေ ဒါမှမဟုတ် Constant တွေကို ရေးသားနိုင်ပါတယ်။ တကယ်လို့ Value ဟာ Switch လုပ်မည့် Value နဲ့ ကိုက်ညီမယ်ဆိုရင် Case တွေထဲကို ဝင်ရောက် အလုပ်လုပ်စေမှာ ဖြစ်ပါတယ်။

Case တွေရဲ့ Statement တွေကို လုပ်ဆောင်ရင်း break; ကို တွေ့ရင်တော့ Switch Statement တွေထဲကနေ ထွက်ခါသွားမှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ beak ကို မတွေ့ခဲ့ရင်တော့ Statement တွေအားလုံးကို ပြီးတဲ့ အထိ လုပ်ဆောင် သွားမှာ ဖြစ်ပါတယ်။

နမူနာအနေနဲ့ int Number အလိုက် ဘာသာရပ်တွေကို ဖေါ်ပြတဲ့ Statement တစ်ခုကို ရေးသားကြည့်ပါမယ်။

```
        void show(int i) {

              switch(i) {
              case 1:
                  System.out.println("Java SE");
                  break;
              case 2:
                  System.out.println("Java EE");
                  break;
              case 3:
                  System.out.println("Spring");
                  break;
              default:
                  System.out.println("Everything");
                  break;
              }
          }
```

အထက်ပါအတိုင်း JShell နဲ့ show ဆိုတဲ့ Method တစ်ခုကို ရေးသားထားပါတယ်။ အဲ့ဒီ Method မှာ int အမျိုးအစား Variable i ကို Parameter အနေနဲ့ ရယူထားပါတယ်။ ပြီးတော့မှ i အပေါ်မူတည်ပြီး Switch လုပ်နေပါတယ်။ { နဲ့ စပြီး } နဲ့ ဆုံးတဲ့ နေရာကတော့ Switch Block ဖြစ်ပြီး အဲ့ဒီအထဲမှာတော့ case တွေကို တစ်ခုထက်မက ရေးသားနိုင်ပါတယ်။

အထက်ပါ နမူနာထဲမှာဆိုရင် i ရဲ့ တန်ဖိုးဟာ ပထမဆုံး case မှာဖေါ်ပြထားတဲ့ တန်ဖိုးနဲ့ ကိုက်ညီရင် အဲ့ဒီ case ထဲမှာ ရေးသားထားတဲ့ Statement တွေကို အလုပ်လုပ်စေမှာ ဖြစ်ပါတယ်။ အဲ့ဒီနေရာမှာ break ကို တွေ့တာနဲ့ switch statement ထဲက ထွက်သွားမှာ ဖြစ်ပြီး တကယ်လို့ break သာ မတွေ့ပါက Switch Statement ဆုံးတဲ့အထိ အထဲက Statement တွေကို လုပ်ဆောင်သွားမှာ ဖြစ်ပါတယ်။

Switch Block ထဲမှာ case တွေအပြင် default ကိုလဲ တစ်ခု ရေးသားနိုင်ပါတယ်။ i ရဲ့ တန်ဖိုးဟာ ဘယ် case ရဲ့ တန်ဖိုးနဲ့မှ မတူခဲ့ရင် default ထဲက Statement ကိုလုပ်ဆောင်သွားမှာ ဖြစ်ပါတယ်။

```
jshell> show(2)
Java EE

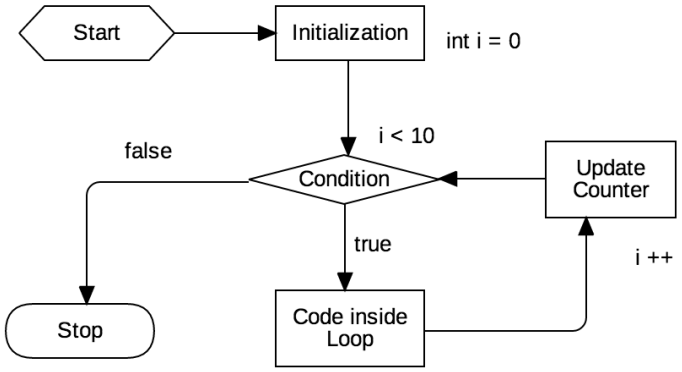
jshell> show(4)
Everything
```

အထက်ပါ ကုဒ်တွေထဲမှာလို show method ကို 2 နဲ့ ပေးခေါ်ကြည့်တဲ့ အခါမှာ case 2 ထဲက Statement ကို လုပ်ဆောင် သွားတဲ့ အတွက် Java EE လို့ ဖေါ်ပြနေပြီး 4 နဲ့ ပေးခေါ်ကြည့်တဲ့အခါ default ထဲက Statement ကို လုပ်ဆောင်သွားစေတဲ့ အတွက် Everything လို့ဖေါ်ပြပေးသွားတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။

Looping Statements

လုပ်ဆောင်မှုတွေကို အကြိမ်ကြိမ် လုပ်ဆောင်စလိုတာတို့၊ Array ဒါမှမဟုတ် Collection ထဲက Object တွေကို အသုံးပြုပြီး တစ်ခုပြီးတစ်ခု လုပ်ဆောင်မှုတွေကို ပြုလုပ်စေလိုတဲ့ အခါမျိုးတွေမှာ Looping Statement တွေကို အသုံးပြုနိုင်ပါတယ်။ Java မှာ Looping Statement အနေနဲ့ for statement, while statement နဲ့ do - while statement ဆိုပြီး ရေးသားနိုင်ကြပါတယ်။

For statement




```
for (int i = 0; i < 10; i ++) {

    // code inside Loop

}
```

Java Programming မှာ For Statement ကို Counter အပေါ်မှာ မူတည်ပြီး Loop ပတ်တဲ့ ပုံစံနဲ့ Collection ဒါမှမဟုတ် Array တစ်ခုထဲမှာ ရှိတဲ့ Element တွေကို Iterate လုပ်တဲ့ ပုံစံဆိုပြီး နှစ်မျိုးရေးသားနိုင်ပါတယ်။ အထက်ပါ ပုံစံကတော့ Counter အပေါ်မှာမူတည်ပြီး Loop ပတ်တဲ့ ပုံစံမျိုး ဖြစ်ပါတယ်။

Counter ကို အသုံးပြုပြီး ရေးသားတဲ့ For Statement တွေဟာ Java Programming ကို စတင် Release လုပ်တဲ့ အချိန်ကထဲက ပါဝင်ခဲ့တဲ့ ရေးသားပုံဖြစ်ပြီး Index Number တွေကို အသုံးပြုလိုတဲ့ အခါမျိုးမှာ အသုံးပြုသင့်တဲ့ ရေးသားပုံဖြစ်ပါတယ်။

For Statement မှာ အစိတ် အပိုင်း ၄ ခုပါဝင်ပါတယ်။ အဲ့ဒါတွေကတော့ Initialization, Condition, Update Counter နဲ့ For Block တို့ ဖြစ်ကြပါတယ်။ Initialization မှာတော့ For Loop မှာ အသုံးပြုမည့် Counter ကို Initialize လုပ်တဲ့ Expression ဖြစ်ပါတယ်။ int i = 0 လို့ရေးသားထားတဲ့ အပိုင်း ဖြစ်ပါတယ်။ အဲ့ဒီနေရာမှာ i ဟာ Counter ဖြစ်ပြီး ကနဦးတန်ဖိုးကို 0 လို့ သတ်မှတ်နေပါတယ်။

ဒုတိယ Expression ကတော့ Condition ဖြစ်ပြီး Loop အတွင်းဝင်မယ် မဝင်ဘူးဆိုတာကို ဆုံးဖြတ်ပေးနိုင်တဲ့ Expression ဖြစ်ပါတယ်။ i < 10 လို့ ရေးသားထားတဲ့ အစိတ်အပိုင်းဖြစ်ပြီး i ဟာ 10 ထက်ငယ်နေရင် Loop အတွင်း ဝင်ရောက်မှာ ဖြစ်ပြီး for block { } အတွင်းမှာ ရေးသားထားတဲ့ Code တွေကို လုပ်ဆောင် စေမှာ ဖြစ်ပါတယ်။ တကယ်လို့ i ရဲ့ တန်ဖိုးဟာ 10 ထက် မငယ်တော့ဘူးဆိုရင်တော့ for loop ကို ပြီးဆုံးစေမှာ ဖြစ်ပါတယ်။

နောက် Expression တစ်ခုကတော့ Counter ကို Update လုပ်ပေးမည့် Expression ဖြစ်ပါတယ်။ Loop ထဲကို ဝင်ရောက်ပြီး For Block ထဲက ကုဒ်တွေကို လုပ်ဆောင်ပြီးတိုင်း Counter ကို Update လုပ်မှာ ဖြစ်ပါတယ်။ အထက်ပါ နမူနာထဲမှာတော့ i ++ ကို ရေးသားထားတဲ့ အတွက် တစ်ကြိမ် Loop ပတ်ပြီးတိုင်း Counter ဖြစ်တဲ့ i ရဲ့ တန်ဖိုးက တစ်တိုးသွားမှာ ဖြစ်ပါတယ်။

အထက်ပါ နမူနာ အတိုင်းဆိုရင် ၁၀ ကြိမ် Loop ပတ်ပြီးတဲ့ အခါမှာ Counter ရဲ့ တန်ဖိုးဟာ 10 ဖြစ်လာပြီး i < 10 ဆိုတဲ့ Boolean Expression ရဲ့ ရလဒ်ဟာ false ဖြစ်လာတဲ့အခါမှာ For Loop ကို ပြီးဆုံးစေမှာ ဖြစ်ပါတယ်။

```
jshell> String [] array = {"Hello", "Java"}
array ==> String[2] { "Hello", "Java" }

jshell> for(int i = 0; i < array.length; i ++) {
   ...> System.out.println(array[i]);
   ...> }
Hello
Java
```

အထက်ပါ နမူနာထဲမှာ array ထဲမှာ “Hello” နဲ့ “Java” တို့ကို ရေးသားထားပါတယ်။ ပြီးတော့ for statement ရဲ့ Counter ကို 0 ကနေ စစေပြီး Condition အနေနဲ့ i < array.length လို့ ရေးသားထားတဲ့ အတွက် i ဟာ array ရဲ့ Length ဖြစ်တဲ့ 2 ထက် ငယ်နေသမျှ Loop အတွင်းရှိ ကုဒ်တွေကို အလုပ်လုပ်စေမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် နှစ်ကြိမ် Loop ပတ်သွားတာကို တွေ့ရပါလိမ့် မယ်။

ဒီလိုမျိုး Array ထဲမှာရှိတဲ့ Element တွေကို Iterate လုပ်ပြီး အသုံးပြုချင်ရင် နောက်တစ်မျိုး ရေးသားနိုင်ပါသေးတယ်။ အဲ့ဒါကတော့ Java 5 မှာ စတင်ပါဝင်လာခဲ့တဲ့ Enhance For Statement ဖြစ်ပါတယ်။

Enhance For Statement တွေဟာ Array ဒါမှမဟုတ် Collection တွေရဲ့ Element တွေကို တစ်ခုချင်း Iterate လုပ်ပြီး အသုံးပြုလိုတဲ့ အခါမျိုးမှာ အသုံးဝင်ပါတယ်။ for (Type obj : list) ဆိုပြီး ရေးသားထားတဲ့ နေရာမှာ Type ဟာ list ထဲမှာရှိတဲ့ Element ရဲ့ Type ကို ရေးသားရမှာ ဖြစ်ပါတယ်။ list Object ထဲမှာရှိတဲ့ Element တွေက obj အနေနဲ့ Itelrate လုပ်ပြီး for block ထဲမှာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ List ထဲမှာရှိတဲ့ Element တွေ အားလုံး Iterate လုပ်ပြီးတဲ့ အခါမှာ for loop ကို ပြီးဆုံးစေမှာ ဖြစ်ပါတယ်။

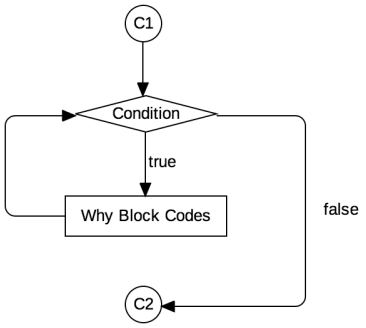
```
jshell> for(String str : array) {
   ...> System.out.println(str);
   ...> }
Hello
Java
```

အထက်ပါနမူနာထဲမှာ for (String str : array) လို့ရေးသားထားတဲ့ အတွက် array ထဲမှာရှိတဲ့ String တွေကို Iterate လုပ်သွားမှာ ဖြစ်ပါတယ်။ array = {“Hello”, “Java”} လို့ရေးသားထားတဲ့ အတွက် array ထဲမှာ String နှစ်ခုရှိပြီး အဖြေမှာလဲ Hello နဲ့ Java ကို ထုတ်ပြသွားနိုင်တာကို တွေ့ရပါမယ်။

Enhance For Statement တွေဟာ Array ဒါမှမဟုတ် Collection ထဲက Element တွေကို တစ်ခုချင်း Iterate လုပ်ပြီး အသုံးပြုနိုင်တဲ့ အတွက် For Each Statement လို့ခေါ်ဆိုလေ့ရှိပါတယ်။

While Statement

Condition တစ်ခုဟာ မှန်နေသ၍ အကြိမ်ကြိမ် အလုပ်လုပ်နေစေလိုတဲ့ အခါမျိုးမှာ while statement ကို ရေးသားအသုံးပြုနိုင်ပါတယ်။




```
while (Condition) {

        While Block Codes

 }
```

While Statement ကို ရေးသားတဲ့ နေရာမှာ While Block ထဲကို ဝင်ပြီး အလုပ်လုပ်မလား မလုပ်ဘူးလားဆိုတာကို စစ်ဆေးနိုင်တဲ့ Condition နဲ့ အဲ့ဒီ Condition မှန်နေခဲ့ရင် အလုပ်လုပ်စေချင်တဲ့ ကုဒ်တွေကို ရေးထားလို့ရတဲ့ While Block တို့ ကို ရေးသားထားဖို့လိုအပ်ပါတယ်။

Program Flow အရဆိုရင် အရင်ဆုံး Condition နဲ့ စစ်တယ်၊ ကိုက်ရင် While Block ထဲက ကုဒ်တွေကို အလုပ်လုပ်စေပါတယ်။ မကိုက်ရင် While Loop ထဲက နေ ထွက်ပါမယ်။ While Block ထဲက ကုဒ်တွေကို လုပ်ဆောင်ပြီးတိုင်း Condition ကို ပြန်ပြီး စစ်ပါတယ်။ Condition နဲ့ ကိုက်ညီနေတိုင်း While Block ထဲက ကုဒ်တွေကို အလုပ်လကုပ်စေမှာ ဖြစ်ပါတယ်။ Condition နေရာကို Boolean Expression နဲ့ ရေးသားနိုင်ပြီး အဲ့ ဒီနေရာမှာ true ကို သာရေးထားမယ်ဆိုရင် While Loop ထဲကနေ ထွက်နိုင်တော့မှာ မဟုတ်ပါဘူး။

```
jshell> int count = 0;
count ==> 0

jshell> int limit = 5;
limit ==> 5

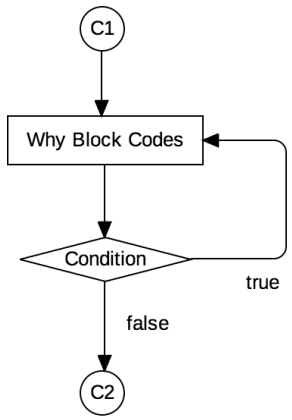
jshell> while(count < limit) {
   ...> System.out.print(count);
   ...> count++;
   ...> }
01234
```

အထက်ပါနမူနာထဲမှာတော့ count ကို 0 လို့ပေးထားပြီး limit ကို 5 လို့ပေးထားပါတယ်။ While Loop ထဲကို ဝင်မလားဆိုတာကို စစ်တဲ့ Condition နေရာမှာ count < limit လို့ ရေးသားထားပါတယ်။ count ဟာ limit ထက်ငယ်နေတဲ့ အတွက် While Block ထဲက ကုဒ်တွေကို အလုပ်လုပ်စေမှာ ဖြစ်ပါတယ်။ While Block ထဲမှာတော့ count ကို Print လုပ်ပြီး count ကို တစ်တိုးနေစေပါတယ်။ ဒါကြောင့် Loop တစ်ခါ ပတ်ပြီးတိုင်း Count ဟာ တစ်တိုးသွားပါမယ်။ ၅ ကြိမ် Loop ပတ်ပြီးတဲ့အခါမှာတော့ count ဟာ 5 ဖြစ်လာပြီး Condition စစ်တဲ့အခါမှာ မမှန်တော့ပဲ Loop ထဲကနေ ထွက်သွားမှာ ဖြစ်ပါတယ်။

While Statement ကို ရေးသားတဲ့အခါမှာ Condition ကို တစ်ချိန်ချိန်မှာ ထွက်အောင် ရေးသားထားဖို့လိုပါတယ်။ မဟုတ်ရင် Endless Loop ဖြစ်သွားစေနိုင်တယ်ဆိုတာကို သတိထားဖို့လိုအပ်ပါလိမ့်မယ်။

Do While Statement

Do While Statement ဟာလဲ Condition တစ်ခုအပေါ်မူတည်ပြီး အကြိမ်ကြိမ်လုပ်ဆောင် စေနိုင်တဲ့ Statement တစ်မျိုးဖြစ်ပါတယ်။




```
do {

        Do Block Codes

} while (Condition);
```

While Statement နဲ့ ဆင်တူသလိုရှိပေမဲ့ မတူညီတာကတော့ Do Block ထဲက ကုဒ်တွေကို အရင် အလုပ်လုပ်စေပြီးမှ While ထဲက Condition ကို စစ်ဆေးနေတာပဲ ဖြစ်ပါတယ်။ အဲ့ဒီအတွက် Do Block ထဲက Code တွေကို အနဲဆုံး တစ်ကြိမ်တော့ အလုပ်လုပ်စေမှာ ဖြစ်ပါတယ်။ Do Block ထဲက ကုဒ်တွေကို အလုပ်လုပ်ပြီးတဲ့ အခါမှာ While Condition နဲ့ စစ်ဆေးပြီး မှန်တယ်ဆိုရင် နောက်တစ်ကြိမ် Do Block ကို အလုပ်လုပ်စေပြီး မှားတယ်ဆိုရင်တော့ Do While Loop ထဲကနေ ထွက်သွားမှာ ဖြစ်ပါတယ်။

```
jshell> int count = 3;
count ==> 3

jshell> while(count < 3) {
   ...> System.out.println(count);
   ...> count ++;
   ...> }

jshell>

jshell> do {
   ...> System.out.println(count);
   ...> count ++;
   ...> } while (count < 3);
3
```

အထက်ပါနမူနာထဲမှာ while ရဲ့ Condition နဲ့ do while ရဲ့ Condition တို့ဟာ အတူတူပါပဲ။ count ဟာ 3 ဖြစ်တဲ့အတွက် နှစ်ခုလုံး Condition နဲ့ မကိုက်ညီပါဘူး။ While Statement ရဲ့ Loop ထဲကို မဝင်ပေမဲ့ Do While Statement ထဲကိုတော့ တစ်ကြိမ် အလုပ်လုပ်သွားတာကို တွေ့ရပါမယ်။ Do While Statement ဟာ Do ကို တစ်ကြိမ် အလုပ်လုပ်ခိုင်းပြီးမှ Condition ကို စစ်ဆေးနေတဲ့ အတွက် ဖြစ်ပါတယ်။

Branching Statement

ပရိုဂရမ် အသွားအလာတွေကို ပြောင်းပစ်အောင် ရေးသားလိုတဲ့ အခါမျိုးတွေမှာ Branching Statement တွေကို အသုံးပြုနိုင်ပါတယ်။ ဆိုကြပါဆို့၊ Method တစ်ခုကို ရေးသားနေတဲ့ နေရာမှာ ရုတ်တရက် အဲ့ဒီ Method ကိုပြီးဆုံးစေလိုတဲ့အခါမျိုး ရှိတတ်ကြပါမယ်။ တဖန် Looping တွေထဲကနေ အနေအထားတစ်မျိုးကြောင့် Looping ကို ဆက်ပြီး မလုပ်စေလိုတဲ့ အခါမျိုးရှိပါမယ်။ ပြီးတော့ လက်ရှိ Looping ထဲက Statement တွေကို ဆက်ပြီး မလုပ်စေလိုတော့ပဲ နောက်တစ်ခေါက် Loop ပတ်စေလိုတဲ့ အခါမျိုး ရှိတတ်ကြပါမယ်။ အဲ့ဒီလို အခါမျိုးမှာ Branching Statement တွေကို အသုံးပြုနိုင်ပါတယ်။

Branching Statement အနေနဲ့ အသုံးပြုနိုင်တာကတော့ Return, Break နဲ့ Continue တို့ပဲ ဖြစ်ပါတယ်။ Branching Statement တွေက Label တွေနဲ့တွဲပြီး အသုံးပြုနိုင်ပြီး မိမိလိုအပ်သလို Programming Flow ကို တစ်မျိုးပြောင်းပြီး လုပ်ဆောင်စေနိုင်ပါတယ်။

Return Statement

Return Statement တွေကို Return Type ပါတဲ့ Method တွေမှာ ရေးသားခဲ့ဘူးပါတယ်။ Method တွေရဲ့ နောက်ဆုံးနေရာမှာ Method Signature မှာပါတဲ့ Return Type အလိုက် တန်ဖိုးတစ်ခုခုကို Return ပြန်ပေးခဲ့ကြမှာပါ။ တနည်းပြောရမယ်ဆိုရင် အဲ့ဒါဟာလဲ Return Statement တစ်ခုဖြစ်ပါတယ်။

```
boolean isEven(int data) {
    return data % 2 == 0;
}
```

အထက်ပါ Method ဟာ int တန်ဖိုးတစ်ခုကို စုံကိန်းတစ်ခု ဟုတ်မဟုတ် စစ်ပေးနိုင်တဲ့ Method တစ်ခု ဖြစ်ပါတယ်။ အဲ့ဒီ Method မှာ boolean type ကို Return Type အနေနဲ့ သတ်မှတ်ထားတဲ့ အတွက် အဲ့ဒီ Method ရဲ့ နောက်ဆုံးတစ်နေရာမှာ boolean value ကို Return ပြန်ပေးရပါမယ်။ ဒီနေရာမှာ return data % 2 == 0 လို့ Boolean Expression တစ်ခုကို ရေးသားထားခဲ့ပါတယ်။ ဒါ့ကြောင့် data ကို 2 နဲ့ စားပြီး အကြွင်းဟာ 0 ဖြစ်ရင် true ကို return ပြန်မှာ ဖြစ်ပြီး မဟုတ်ရင်တော့ false ကို return ပြန်မှာ ဖြစ်ပါတယ်။

ဒီနေရာမှာ သတိပြုစေလိုတာကတော့ return ပြန်ပြီးရင် အဲ့ဒီ Method ထဲက ဘာကုဒ်ကိုမှ ရေးလို့ မရတော့ဘူးဆိုတာကိုပါပဲ။ ဒီနေရာမှာ ရေးသားထားတဲ့ return data % 2 == 0; ဆိုတာက return statement ဖြစ်ပါတယ်။

Return ပြန်စရာမလိုတဲ့ void method တွေမှာလဲ return statement ကို ရေးသားနိုင်ပါတယ်။ void method တွေမှာ return; ဆိုပြီး ရေးသား လိုက်တာနဲ့ Method ထဲမှာရှိတဲ့ return statement ရဲ့ နောက်က ကုဒ်တွေကို ဆက်မလုပ်တော့ပဲ အဲ့ဒီ Method ကို ပြီးဆုံးစေမှာ ဖြစ်ပါတယ်။

```
void sell(String item) {
     if(isNoItem()) {
          return;
     }
     System.out.println(item + " has to sell");
}
```

အထက်ပါ sell method ထဲမှာ if statement တစ်ခုကို ရေးသားထားပါတယ်။ if statement ထဲမှာတော့ isNoItem() ဆိုတဲ့ Method ကို Invoke လုပ်ထားပါတယ်။ အဲ့ဒီ isNoItem() ဆိုတဲ့ Method ကို if statement ထဲမှာ ရေးသားထားတဲ့ အတွက် isNoItem() method ရဲ့ Return Type ဟာ boolean တန်ဖိုးတစ်ခု ဖြစ်ရပါမယ်။ တကယ်လို့ ရောင်းစရာ မရှိတော့ရင် true ကို Return ပြန်ပြီး ရောင်းစရာရှိသေးရင်တော့ false လို့ Return ပြန်ပါမယ်။

ရောင်းစရာမရှိတော့ရင် if block ထဲကို ဝင်လာမှာဖြစ်ပြီး retrun; statement ကို အလုပ်လုပ်စေမှာ ဖြစ်ပါတယ်။ return statement ကို method ထဲမှာပဲရေးသားလို့ရပြီး၊ အဲ့ဒီ Statement ကို တွေ့တာနဲ့ လက်ရှိ Method ကို အလုပ်လုပ်နေတာကို ပြီးဆုံးစေမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် ရောင်းစရာမရှိတော့ရင် အဲ့ဒီနေရာမှာပဲ Method လုပ်ဆောင်မှုကို ပြီးဆုံးစေပြီး နောက်က ကုဒ်တွေကို ဆက်ပြီး အလုပ်လုပ်မှာ မဟုတ်တော့ပါဘူး။

Return Statement ကို ရေးသားတဲ့ နေရာမှာ return statement ဟာ လက်ရှိရှိနေတဲ့ Block ရဲ့ နောက်ဆုံး ကုဒ် ဖြစ်ဖို့လိုအပ်ပါတယ်။ အထက်ပါနမူနာထဲမှာ ဆိုရင် return ကို if block ထဲမှာ ရေးသားထားတဲ့ အတွက် အဲ့ဒီ Block ထဲမှာဆိုရင် return ဟာ နောက်ဆုံး Statement ဖြစ်နေဖို့လိုပါတယ်။

တကယ်လို့ block တစ်ခုထဲမှာ return statement ရဲ့ နောက်မှာ ရေးသားထားတဲ့ ကုဒ်တွေရှိနေမယ်ဆိုရင် Compile Time မှာတင် Exception ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။ အထက်ပါနမူနာထဲမှာလို System.out.print လို့ရေးထားတဲ့ ကုဒ်တွေဟာ return အောက်မှာ ရှိနေပေမဲ့ Block မတူပါဘူး။ return ဟာ if block ထဲမှာရှိပြီး System.out.print ဟာ method block ထဲမှာ ရှိပါတယ်။ ဒီလိုမျိုး Block မတူရင်တော့ ရေးသားလို့ရပါတယ်။ သတိထားရမှာက Block တစ်ခုထဲမှာ return statement ရဲ့ နောက်မှာ ဘာကုဒ်မှ ရေးလို့ မရဘူးဆိုတဲ့ အချက်ပဲ ဖြစ်ပါတယ်။ Java Programming ကို စတင်လေ့လာတဲ့ နေရာမှာ ကုဒ် Block တွေနဲ့ သူတို့ရဲ့ Scope တွေကို ကောင်းကောင်း နားလည်ထားဖို့လိုအပ်ပါတယ်။

Break Statement

Break Statement တွေကို Switch Statement တွေ၊ Looping Statement တွေဖြစ်ကြတဲ့ For, While, Do While Statement တွေနဲ့ တွဲဖက်ရေးသားနိုင်ပါတယ်။ အကြောင်းတစ်မျိုးမျိုးကြောင့် break statement ကို ရောက်ပြီဆိုတာနဲ့ လက်ရှိ Statement ထဲကနေ ထွက်ပေးနိုင်ပါတယ်။ Switch Statement နဲ့ break ကို တွဲသုံးတာကိုတော့ Switch Statement မှာ လေ့လာခဲ့ပြီး ဖြစ်တဲ့ အတွက် ဒီနေရာမှာတော့ Looping Statement နဲ့ ပတ်သက်တာတွေကိုပဲ ဖေါ်ပြသွားမှာ ဖြစ်ပါတယ်။

```
for (int i = 0; i < 10; i ++) {
     if(i == 5) {
          break;
     }
     System.out.println("Lopping count is " + (i + 1));
}
```

အထက်ပါနမူနာကုဒ်တွေမှာ For Statement တစ်ခုကို ရေးသားထားပါတယ်။ Counter ဖြစ်တဲ့ i ဟာ 0 ကနေ စပြီး 10 ထက်ငယ်နေသမျှတော့ ဒီ Loop ထဲမှာ အလုပ်လုပ်နေမှာပါ။ ဒါပေမဲ့ for block ထဲမှာ if Statement တစ်ခုကို ရေးသားထားပြီး i == 5 လို့ ရေးသားထားခဲ့ပါတယ်။

ဒါ့ကြောင့် Counter ဖြစ်တဲ့ i ဟာ 5 ဖြစ်သွားရင် break statement ကို အလုပ်လုပ်စေမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် 5 ကြိမ် Loop ပတ်ပြီးတဲ့ အခါမှာ i ရဲ့ တန်ဖိုးဟာ 5 ဖြစ်သွားပြီး if block ကို အလုပ်လုပ်စေမှာ ဖြစ်ပါတယ်။ if block ထဲမှာ break ကို ရေးထားတဲ့ အတွက် for loop ကို ပြီး ဆုံးစေမှာ ဖြစ်ပါတယ်။

Break Statement ဟာလဲ block ရဲ့ နောက်ဆုံး Statement ဖြစ်တဲ့ အတွက် break ရဲ့ နောက်မှာ ဘာကုဒ်တွေမှ ရေးသားလို့ ရမှာ မဟုတ်ပါဘူး။ break; လို့ရေးထားတာနဲ့ Current Looping Block ကို ပြီးဆုံးစေမှာ ဖြစ်ပါတယ်။ တဖန် break statement ကို label တွေနဲ့လဲ တွဲပြီး အသုံးပြုနိုင်ပါတယ်။

ဒီလို ရေးသားပုံက Nested Loop တွေမှာ အသုံးပြုနိုင်ပါတယ်။ Nested Loop ဆိုတာက Looping Statement တွေမှာ ထပ်ပြီး Looping Statement တွေကို ရေးသားတဲ့ ပုံစံပါ။

```
String [] months = {"JAN", "FEB", "MAR", "APR", "MAY",
        "JUN", "JUL", "AUG", "SEP", "OCT", "NOV", "DEC"};
int [] days = {1,2,3,4,5,6,7,8,9,10};

month:
for(String m : months) {

    day:
    for(int i : days) {

        if(i == 5) {
            continue month;
        }

        if((m.equals("MAR") || m.equals("MAY")) && i == 3) {
            break day;
        }

        System.out.printf("%s : %d%n", m, i);
    }
}
```

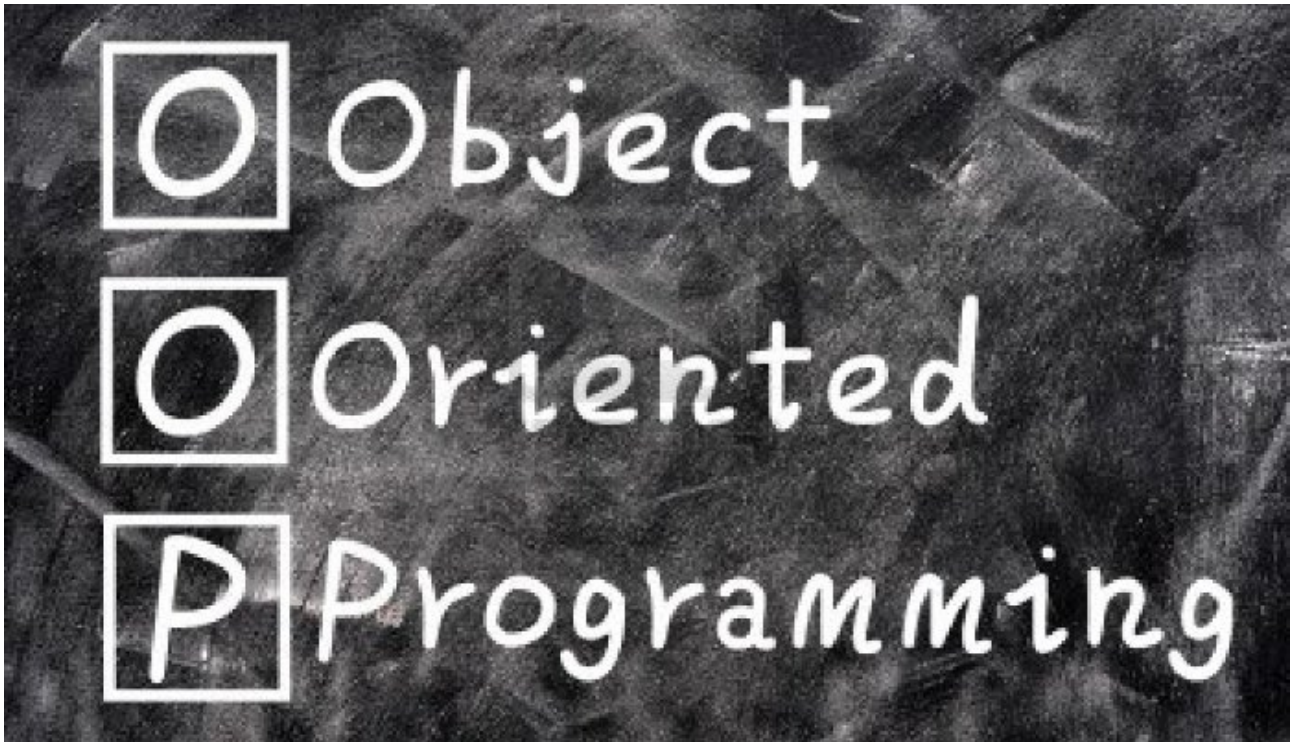
Java ဘာသာရပ်မှာ Nested Loop ကို ရေးသားတဲ့ နေရာမှာ Condition အပေါ်မူတည်ပြီး အတွင်းပိုင်း Loop ထဲကနေ Label တွေကို အသုံးပြုကာ အပြင်ဘက်မှာရှိတဲ့ Loop တွေကိုလဲ Control လုပ်နိုင်မှာ ဖြစ်ပါတယ်။

အထက်ပါနမူနာထဲမှာ ဖေါ်ပြထားတဲ့အတိုင်း အပြင်ဘက် For Loop ရဲ့ရှေ့မှာ month: လို့ရေးသားထားပြီး month ဟာ အပြင်ဘက်က Loop ရဲ့ Label ဖြစ်ပါတယ်။ တဖန် day: ကိုလဲ အတွင်းပိုင်း Loop မှာ ရေးသားထားပြီး day ဟာ အတွင်းပိုင်း Loop ရဲ့ Label ဖြစ်ပါတယ်။

Condition အပေါ်မူတည်ပြီး continue တို့ break တို့နဲ့ Label တွေကို အသုံးပြုပြီး နှစ်သက်ရာ Loop ကို Control လုပ်နိုင်မှာ ဖြစ်ပါတယ်။ အထက်ပါနမူနာထဲမှာ i ဟာ 5 နှင့်တူရင် continue month လို့ရေးသားထားတဲ့ အတွက် i ဟာ 5 ဖြစ်တိုင်း အပြင်က Loop ကို Continue လုပ်စေမှာ ဖြစ်ပါတယ်။ ဒုတိယ Condition မှာတော့ break day လို့ရေးသားထားတဲ့အတွက် အတွင်းပိုင်း Loop ကို break လုပ်မှာ ဖြစ်ပါတယ်။

တကယ်လို့ Label ဖြစ်တဲ့ day ကို မရေးသားထားခဲ့ရင်လဲ Current Loop ကို Break လုပ်မှာ ဖြစ်တဲ့ အတွက် အတွင်းပိုင်း Loop ကိုပဲ Break လုပ်သွားမှာ ဖြစ်ပါတယ်။

Object Oriented Programming



Object Oriented Programming ဆိုတာက Application ထဲမှာပါဝင်တဲ့ Programming Unit တွေကို Object အနေနဲ့မြင်ပြီး၊ မိမိလိုအပ်တဲ့ Application Logic တွေကို Object အချင်းချင်း Communicate လုပ်ခြင်အားဖြင့် ဖြစ်မြောက်အောင် ဆောင်ရွက်စေသော Programming ရေးသားပုံရေးသားနည်း တစ်မျိုးဖြစ်ပါတယ်။

၁၉၈၀ခုနှောက်ပိုင်းမှာ ထွက်ပေါ်လာတဲ့ Programming Language အတော်များများဟာ OOP ကို အခြေခံထားတဲ့ Programming Language တွေဖြစ်ကြပါတယ်။ အဲ့ဒါကလဲ OOP ရဲ့ ထူးခြားချက်နဲ့ ခေတ်ရေစီးကြောင်းနဲ့ ပတ်သက်မယ်လို့ထင်ပါတယ်။

OOP ဟာ ပြင်ပမှာရှိတဲ့ Object တွေကို Model ယူပြီး ရေးသားထားတဲ့ Programming Paradigm တစ်ခုဖြစ်တဲ့အတွက် နားလည်ရလွယ်ကူစေပြီး၊ တစ်ခါရေးသားထားတဲ့ Class တွေကို အခြား System တွေမှာလဲ ပြန်လည်အသုံးချနိုင်တဲ့အတွက် Reusable ကိုလဲ ပိုပြီး တိုးတက်စေနိုင်ပါတယ်။

OOP မတိုင်မီက ဆိုရင် Programming တွေကို Function တွေ Data တွေနဲ့ ဖွဲ့စည်းထားပါတယ်။ ဒါ့ကြောင့် Reusable သိပ်မဖြစ်ပါဘူး။ Application တစ်ခုကိုရေးသားတော့မယ်ဆိုရင် အစအဆုံးပြန်ရေးနေရတာမျိုးပါတယ်။ ပြီးတော့လဲ လေ့လာရခက်ခဲတဲ့အတွက် ရေးသားနိုင်တဲ့ Developer အရေအတွက်ဟာ နဲပါးခဲ့ပါတယ်။ ၁၉၈၀ လောက်ကစပြီး Industry တွေနဲ့ Business Coorporation တွေမှာ System တွေကို အများအပြား အသုံးပြုလာကြပါတယ်။

အဲ့ဒီလို အနေအထားအရ လေ့လာရလွယ်ကူပြီး ပြန်လည် အသုံးချနိုင်တဲ့ Programming Paradigm ဟာ ခေတ်ရဲ့ Requirement တစ်ခုဖြစ်လာပါတယ်။ OOP ကို စတင်အောင်မြင်စွာ အသုံးပြုလာနိုင်တာကတော့ Cညည ဘာသာရပ်ဖြစ်ပါတယ်။ အဲ့ဒီနောက်မှာမှ Java ဟာလဲ OOP ကို အခြေခံတဲ့ Language တစ်ခုအနေနဲ့ ထွက်ပေါ်လာရခြင်း ဖြစ်ပါတယ်။

About OOP

OOP ဆိုတာက Java မဟုတ်ပါဘူး။ Java က OOP ကို အခြေခံပြီး ရေးသားထားတဲ့ Programming Language တစ်ခုဖြစ်ပါတယ်။ ကျွန်တော်တို့ ဒီနေရာမှာတော့ Java ဘာသာရပ်ကနေ အခြေခံထားတဲ့ OOP ရဲ့ Concept တွေကို စုစည်းဖော်ပြသွားပါမယ်။ OOP Concept တွေကို ဖော်ပြတဲ့ နေရာမှာ Class, Object, Method, Constructor, Abstract Class နဲ့ Interface တို့ကို အသုံးပြုပြီး ဖော်ပြသွားမှာ ဖြစ်ပါတယ်။

တဖန် OOP ရဲ့ Concept တွေဟာလဲ တစ်ခုနဲ့ တစ်ခုဆက်နွယ်နေတာကများပါတယ်။ ဒါ့ကြောင့် Concept တွေ အကြောင်းကို တစ်ခုချင်းစီ ဖော်ပြတဲ့နေရာမှာ ဟိုနေရာမှာလဲပါ ဒီနေရာမှာလဲပါ ဖြစ်နေတဲ့ အကြောင်းအရာလေးတွေပါဝင်သွားမှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ Concept တစ်ခုချင်းစီကို သေသေချာချာလေ့လာပြီး သဘောတရားကို နားလည်သွားအောင် လေ့လာသွားစေလိုပါတယ်။ ပါဝင်မည့် အကြောင်းအရာတွေကတော့ အောက်ပါအတိုင်းဖြစ်ပါတယ်။

Encapsulation

OOP ရဲ့ အဓိက Concept တစ်ခုဖြစ်ပြီး Object တစ်ခုရဲ့ အတွင်းပိုင်းဖွဲ့စည်းတည်ဆောက်ပုံတွေကို ပြင်ပကနေ မမြင်အောင် ဖုန်းကွယ်ပေးထားနိုင်သော သဘောတရားဖြစ်ပါတယ်။ Java ဘာသာရပ်မှာ Access Modifier တွေနဲ့ Interface တို့ကို အသုံးပြုပြီး Encapsulation ကို ရရှိအောင် ရေးသားနိုင်ပါတယ်။

Encapsulation အကြောင်းကိုလေ့လာတဲ့အခါမှ Encapsulation ဖြစ်အောင်ဘယ်လိုရေးမယ်၊ ပြီးတော့ Encapsulation ကြောင့် ဘယ်လို အကျိုးတွေရလာနိုင်တယ်ဆိုတာကို ဆက်လက်လေ့လာသွားမှာ ဖြစ်ပါတယ်။

Inheritance

မျိုးရိုးဆက်ခံပိုင်ခွင့်လို့ အဓိပ္ပါယ်ဖြစ်ပြီး Class တွေဟာ အခြားသော Class တွေကနေတဆင့် မျိုးရိုးဆက်ခံပိုင်ခွင့်ရှိပါတယ်။ မျိုးရိုးဆက်ခံခြင်းကြောင့် Class တွေ အကြားမှာ IS A Relationship ကို ဖြစ်ပေါ်စေပြီး Super Class ကနေ ခွင့်ပြုထားတဲ့ Instance Member တွေကို Sub Class တွေကနေ အလိုလို အသုံးပြုလာနိုင်မှာ ဖြစ်ပါတယ်။

Java ဘာသာရပ်မှာ Inheritance ကို အသုံးပြုခြင်းအားဖြင့် Super Class ထဲမှာရှိတဲ့ Code တွေကို Sub Class ထဲကနေ ပြန်လည် အသုံးချနိုင်မှာ ဖြစ်သလို Super Class နဲ့ Sub Class တွေအားလုံးကို အုပ်စုတူတဲ့ Class တွေ အနေနဲ့လဲ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

ဒီနေရာမှာတော့ Inheritance ဆိုတာဘာလဲ၊ ဘယ်အရာတွေကို Inheritance လုပ်ပိုင်ခွင့်ရှိလဲ ပြီးတော့ Inheritance ကို ဘယ်လိုနေရာမှာ အသုံးပြုသင့်သလဲ ဆိုတဲ့ အချက်တွေကို လေ့လာသွားမှာ ဖြစ်ပါတယ်။

Composition

OOP ဟာ Code တွေကို ပြုပြင်ပြောင်းလဲရလွယ်ကူစေရန်၊ ပြန်လည် အသုံးချနိုင်ရန်ရည်ရွယ်ထားတဲ့ Programming Concept တစ်ခုဖြစ်ပါတယ်။ Java မှာက တစ်ခါရေးထားတဲ့ ကုဒ်တွေကို ပြန်လည် အသုံးချနိုင်အောင် ရေးသားနည်း ၂ မျိုးရှိပါတယ်။

ပထမတစ်နည်းကတော့ Inheritance ပါ။ Inheritance လုပ်ခြင်းအားဖြင့် Super Class ထဲက ကုဒ်တွေကို အမွေရလိုက်သလို Sub Class ထဲကနေ ပြန်လည် အသုံးချနိုင်မှာ ဖြစ်ပါတယ်။ IS A Relationship လို့လဲ ခေါ်လေ့ရှိပါတယ်။

ဒုတိယနည်းကတော့ အခြား Object တွေကို မိမိရဲ့ Class ထဲမှာ Reference လုပ်ထားပြီး အသုံးလိုတဲ့အခါမှာ အဲ့ဒီ Object တွေကို ပြန်လည် အသုံးချတဲ့ နည်းဖြစ်ပါတယ်။ ဒါကတော့ Has A Relationship ကို အသုံးပြုပြီး ကုဒ်တွေကို ပြန်လည် အသုံးချနိုင်မှာ ဖြစ်ပါတယ်။ Composition ဆိုတာကတော့ HAS A Relationship ကို အသုံးပြုပြီး ရေးသားတဲ့ နည်းကို ခေါ်ပါတယ်။ ဒီအခန်းမှာတော့ IS A Relationship ရဲ့ ဖြစ်လေ့ရှိတဲ့ ပြဿနာတွေကို ဖေါ်ပြပြီး Composition ကို ဘယ်လို အသုံးပြုသင့်တယ်ဆိုတာကို ဖေါ်ပြသွားမှာ ဖြစ်ပါတယ်။အ

Abstraction

Abstraction ဆိုတာ OOP ရဲ့ အခြေခံ Concept တစ်ခုဖြစ်ပါတယ်။ ဘုံသဘောဆောင်သော Type များကို ရေးသား အသုံးပြုနိုင်ခြင်းလို့ အဓိပါယ်ရပါတယ်။ ဒီအခန်းမှာတော့ Abstract Class ကို အသုံးပြုပုံ၊ Interface များရဲ့ အသုံးပြုပုံတို့ကိုလေ့လာသွားပါမယ်။

Java ဘာသာရပ်မှာ Abstract Class တွေကို အသုံးပြုပြီးသော်၎င်း၊ Interface တွေကို အသုံးပြုပြီးသော်၎င်း Abstraction ဖြစ်အောင် ရေးသားနိုင်ပါတယ်။

ဒီနေရာမှာတော့ Abstraction ကြောင့် Object တွေရဲ့ အကြားမှာရှိတဲ့ Coupling ကို လျော့ချနိုင်တယ်ဆိုတဲ့ အချက်ကို လေ့လာသွားပါမယ်။

Polymorphism

OOP ရဲ့ အဓိက Concept တစ်ခုဖြစ်ပါတယ်။ Polymorphism ဆိုတာဟာ Many Form (ပုံစံအမျိုးမျိုးရှိနေနိုင်ခြင်း) လို့ အဓိပါယ်ရပါတယ်။ Class တွေကို IS A Relationship ဖြစ်အောင် ရေးသားခြင်းအားဖြင့် Object တစ်ခုကို Reference Type အမျိုးမျိုးကနေ Reference လုပ်နေနိုင်မယ်၊ ပြီးတော့ Super Class ထဲမှာပါတဲ့ General ကျတဲ့ Behavior ကို Override လုပ်ခြင်းအားဖြင့် မတူကွဲပြားတဲ့ Behavior တွေကို Sub Class တွေရဲ့ Object တွေမှာ ရရှိနိုင်ပါတယ်။

Java ဘာသာရပ်မှာ Class တွေ အချင်းချင်း Inheritance လုပ်ခိုင်းခြင်းအားဖြင့် သော်၎င်း၊ Abstract Class တွေကို အသုံးပြုပြီးသော်၎င်း၊ Interface တွေကို အသုံးပြုပြီးသော်၎င်း Polymophic ဖြစ်အောင် ရေးသားနိုင်ပါတယ်။

ဘယ်လိုနေရာမျိုးတွေမှာ Polymophic ဖြစ်တယ်ဆိုတာတွေ၊ Override လုပ်ခြင်းတွေနဲ့ Polymorphic Code တွေကို ဘယ်လိုနေရာမျိုးတွေမှာ အသုံးပြုသင့်တယ်ဆိုတဲ့ အချက်တွေကို လေ့လာနိုင်မှာ ဖြစ်ပါတယ်။

Encapsulation

Encapsulation ဆိုတာဟာ OOP ရဲ့ အရေးပါတဲ့ Feature တစ်ခုဖြစ်ပါတယ်။ Object တွေမှာ State တွေနဲ့ Behavior တွေကို ပိုင်ဆိုင်ကြပါတယ်။ State ဆိုတာကတော့ အဲ့ဒီ Object ရဲ့ အရည် အသွေး ပိုင်ဆိုင်မှုဖြစ်ပါတယ်။ Encapsulation ဆိုတာက Object တစ်ခုရဲ့ ပိုင်ဆိုင်မှုတွေကို ပြင်ပက အလွယ်တကူ ထင်သလိုပြုပြင်ပြောင်းလဲ လို့မရနိုင်အောင် ပြုလုပ်ထားနိုင်ခြင်း ဖြစ်ပါတယ်။

ဆက်လက်ပြီး အောက်ပါ အကြောင်းအရာတွေကို လေ့လာသွားကြပါမယ်။

- Encapsulation မဖြစ်တဲ့ Java Program တွေမှာ ဖြစ်တတ်တဲ့ပြဿနာ
- Java Program တွေမှာ Encapsulation ဖြစ်အောင် ဘယ်လိုရေးမလဲ
- Encapsulation ကြောင့် ရနိုင်တဲ့ အကျိုးတွေ

Encapsulation မဖြစ်တဲ့အတွက် ဖြစ်တတ်တဲ့ပြဿနာ

ဆိုကြပါစို့ Person ဆိုတဲ့ Class မှာ age ဆိုတဲ့ instance variable တစ်ခုပါပါမယ်။ တကယ်လို့ ပြင်ပကနေ ပြင်ချင်တိုင်းပြင်နေလို့ရနေရင် age နေရာမှာ အနုတ်တန်ဖိုးတွေ ၁၅၀ ကျော်တာတွေ လာပြီး သတ်မှတ်လို့ရသွားပါမယ်။ ဒီလိုဆိုရင် ကျွန်တော်တို့ရဲ့ Object ဟာ မှန်ကန်တဲ့ တန်ဖိုးကို ထိမ်းပေးထားနိုင်မှာ မဟုတ်တော့ပါဘူး။

အဲ့ဒါအပြင် Method တွေကိုလဲ အပြင်ကို ပေးသုံးစေချင်တဲ့ Method တွေလဲ ရှိသလို အပြင်ကနေ မသုံးစေချင်တဲ့ Method တွေလဲ ရှိပါတယ်။ အဲ့ဒီလို အခါမျိုးမှာ Encapsulation ကို မှန်မှန်ကန်ကန် သတ်မှတ်ထားနိုင်ဖို့လိုအပ်ပါတယ်။

```
class Person {
      String name;
      int age;

      void greet() {
            System.out.println("My name is " + name);
            System.out.println("I am " + age + " years old");
      }
}

public class PersonTest {

      public static void main(String [] args) {
            Person p = new Person();
            p.name = "Thidar";
            p.age = -10;

            p.greet();
      }
}
```

အထက်ပါ နမူနာထဲကလို age ကို int အနေနဲ့ သတ်မှတ်ထားပြီး Access Modifier ကို သတ်မှတ်ထားခြင်းမရှိတဲ့ အတွက် p.age ဆိုတဲ့ Variable မှာ int ဖြစ်ရင် ကြိုက်တဲ့ တန်ဖိုးကို အစားထိုးချင်တိုင်း အစားထိုးလို့ရနေပါတယ်။ ဒါ့ကြောင့် နမူနာထဲကလို -10 ဆိုပြီး အစားထိုးနေနိုင်ခြင်း ဖြစ်ပါတယ်။ ဒါဆိုရင် p ဆိုတဲ့ Object ဟာ အသက် အနုတ် ၁၀ ရှိတဲ့ Person ဆိုတဲ့ Object ကို တည်ဆောက်လို့ရသွားပါမယ်။

အဲ့ဒါအပြင် အတွင်းပိုင်းမှာရှိတဲ့ State တွေကို ပြင်ပကနေ တိုက်ရိုက် Access လုပ်နေနိုင်ခြင်းဟာ တစ်ခြား ပြဿနာ တစ်ခုကိုလဲ ဖြစ်ပေါ်စေနိုင်ပါတယ်။ ကိုယ်မသိတဲ့ နေရာတွေမှာ ကိုယ့်ရဲ့ State တွေကို အသုံးပြုနေနိုင်မှာ ဖြစ်ပါတယ်။ Person Class ထဲကနေ age တို့ name တို့ကို ဘယ်နေရာမှာ သုံးနေတယ်ဆိုတာကို ခန့်မှန်းလို့ မရ နိုင်တော့ပါဘူး။ အဲ့ဒီအတွက် လိုအပ်လာလို့ အဲ့ဒီ State တွေကို ပြုပြင်ချင်တယ်ဆိုရင်တောင် တစ်ခြားဘယ်နေရာ တွေမှာ အကျိုးသက်ရောက်မှုရှိနေမယ်ဆိုတာကို မခန့်မှန်းနိုင်တော့ပါဘူး။ အဲ့ဒီအတွက် ပြုပြင်ပြောင်းလဲရခက်ခဲတဲ့ Class တွေကို ရေးမိနေမှာ ဖြစ်ပါတယ်။

ဒါဖြစ်ရင် Java Program တွေမှာ Encapsulation ဖြစ်အောင် ဘယ်လိုရေးရမယ်ဆိုတာကို ဆက်ပြီး လေ့လာသွား ကြရအောင်။

Encapsulation ဖြစ်အောင် ဘယ်လိုရေးမလဲ

Java Program တွေထဲမှာ Access Modifier တွေကို အသုံးပြုပြီး သော်ငင်း၊ Interface ကို အသုံးပြုပြီး သော်ငင်း Encapsulation ဖြစ်အောင် ရေးသားနိုင်ပါတယ်။

Using Access Modifier

Access Modifier ဆိုတာက Class, Interface, Enum, Variable နဲ့ Method တွေရဲ့ အသုံးပြုပိုင်ခွင့်ကို သတ်မှတ်ပေးနိုင်တဲ့ Modifier တစ်မျိုး ဖြစ်ပါတယ်။ Access Modifier တွေကို Object တွေရဲ့ Member တွေမှာ သင့်တော်သလို သတ်မှတ် ရေးသားခြင်းဖြင့်လဲ Encapsulation ကို ရရှိအောင် ရေးသားနိုင်ပါတယ်။

အသုံးပြုနိုင်တဲ့ Access Modifier တွေကတော့ အောက်ပါ အတိုင်းဖြစ်ပါတယ်။

Access Modifier	Description
public	မည်သည့်နေရာကမဆို ဆက်သွယ် အသုံးပြုနိုင်ပါတယ်
protected	လက်ရှိ Class နဲ့ Sub Class တွေကနေတဆင့် အသုံးပြုနိုင်ပါတယ်
private	လက်ရှိ Class ကသာ အသုံးပြုနိုင်ပါတယ်။ တစ်ခြားနေရာတွေကနေတော့ အသုံးပြုနိုင်မှာ မဟုတ်ပါဘူး။
(default)	ဘာမှ မရေးထားရင်ရရှိမည့် Access Modifier ဖြစ်ပါတယ်။ လက်ရှိ Class နဲ့ Package အတူ အတွင်းမှာရှိသော Class တွေမှနေ ဆက်သွယ် အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်

Class အတွင်းမှာရှိတဲ့ Instance Variable နဲ့ Instance Method တွေရှေ့မှာ သင့်တော်ရာ Access Modifier ကို ရေးသားခြင်းအားဖြင့် Encapsulation ဖြစ်အောင် ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

ရှေ့မှာ ဖော်ပြခဲ့သလို Person Class ရဲ့ Instance Variable ဖြစ်တဲ့ name နဲ့ age တို့ကို private Access Modifier ကို ရေးသားခြင်း အားဖြင့် ပြင်ပမှတိုက်ရိုက် ဆက်သွယ် အသုံးမပြုနိုင်အောင် ဆောင်ရွက်နိုင်မှာ ဖြစ်ပါ တယ်။

```
class Person {
      private String name;
      private int age;

      public void greet() {
            System.out.println("My name is " + name);
            System.out.println("I am " + age + " years old");
      }
}
```

အထက်ပါ အတိုင်း Instance Variable တွေကို private လို့သတ်မှတ်လိုက်ခြင်းအားဖြင့် ပြင်ပကနေ တိုက်ရိုက် ဆက်သွယ်ခွင့် ရှိမှာ မဟုတ်တော့ပါဘူး။ ဒါပေမဲ့ ဒီအတိုင်း ဆိုပြန်ရင်လဲ Person Object ရဲ့ Instace Variable တွေ ရဲ့ တန်ဖိုးကို သတ်မှတ်လို့ရမှာ မဟုတ်တော့တဲ့အတွက် အသုံးပြုနိုင်မှာ မဟုတ်တော့ပါဘူး။

ဒါပေမဲ့ Instance Variable တွေရဲ့ တန်ဖိုးတွေကို သတ်မှတ်ဖို့အတွက် Method ဒါမှ မဟုတ် Constructor တွေကို

```
public Person(String name, int age) {
      this.name = name;
      if(age <= 0 || age > 130) {
            throw new RuntimeException(
                  “Age must be between 1 and 130");
      }
      this.age = age;
}
```

အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Constructor ရဲ့ Argument ကနေ Instance Variable တွေကို လက်ခံယူပြီး သတ်မှတ်မယ်ဆိုရင် ရနိုင်ပါတယ်။ Constructor ထဲမှာ if statement ကို အသုံးပြုပြီး လိုအပ်သလို စစ်ဆေးပြီးမှ Instance Variable တွေကို

```
public void setAge(int age) {
      if(age <= 0 || age > 130) {
            throw new RuntimeException(
                  “Age must be between 1 and 130");
      }
}
```

အစားထိုးသတ်မှတ်ပေးနိုင်မှာ ဖြစ်ပါတယ်။

setter method တွေကို သုံးပြီးလဲ ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။ Method ထဲမှာလဲ စစ်ဆေးပြီးမှ တန်ဖိုးကို သတ်မှတ်ပေးနိုင်ပါတယ်။ setter method တွေဟာ Object ရဲ့ state တွေကို ပြောင်းလဲနိုင်တဲ့ အတွက် Mutator လို့လဲ ခေါ်ဆိုလေ့ရှိပါတယ်။

```
class Person {
      private String name;
      private int age;

      public void setAge(int age) {
            if(age <= 0 || age > 130) {
                  throw new RuntimeException(
                        "Age must be between 1 and 130");
            }

            this.age = age;
      }

      public void setName(String name) {
            this.name = name;
      }

      public int getAge() {
            return this.age;
      }

      public String getName() {
            return this.name;
      }

      public void greet() {
            System.out.println("My name is " + name);
            System.out.println("I am " + age + " years old");
      }
}
```

တဖန် private member တွေကို ပြင်ပကို ပြန်ပြီးဖော်ပြပေးဖို့လိုတဲ့အခါမှာ getAge, getName အစရှိတဲ့ method တွေကို getter method တွေကို အသုံးပြုနိုင်ပါတယ်။ getter method တွေကို Accessor Method တွေလို့လဲ ခေါ်ဆိုလေ့ရှိပါတယ်။

ဤနည်းအားဖြင့် မိမိရေးသားနေတဲ့ Class တစ်ခုကို Encapsulation ဖြစ်အောင် ရေးသားလိုပါက Instance Variable တွေနဲ့ အတွင်းပိုင်းမှာသာ အသုံးပြုလိုတဲ့ method တွေကို private Access Modifier ကို အသုံးပြုပြီး ပြင်ပကနေ အသုံးမပြုနိုင်အောင် ဖုန်းကွယ်ထားဖို့လိုအပ်ပါတယ်။

တဖန် တန်ဖိုးများကို သတ်မှတ်ရန်လိုအပ်ပါက setter method နဲ့ constructor တို့ကို အသုံးပြုရေးသားနိုင်ပြီး၊ တန်ဖိုးများကို ပြန်လည် ဖော်ပြလိုပါက getter method တို့ကို အသုံးပြုရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

Using Interface

Interface ဆိုတာက Public နဲ့ Abstract Method တွေသာ ရေးသားလို့ရတဲ့ ရာနှုန်းပြည့် Public Abstract Entity အမျိုးအစားတစ်မျိုးပါ။ Abstract ဆိုတာက Method Signature ကိုသာ ရေးသားဖေါ်ပြထားပြီး Body မပါတဲ့ Method အမျိုးအစားပါ။ Method Body မပါဘူးဆိုတော့ ဘယ်လိုလုပ်မယ်ဆိုတာကို မဖေါ်ပြနိုင်ပဲ ဘာတွေလုပ်လို့ရတယ်ဆိုတာကိုပဲ ဖေါ်ပြပေးနိုင်ပါတယ်။

```
public interface Movable {
      void move();
}
```

Interface ကို Implement လုပ်တဲ့ Class တွေကျတော့မှ Interface မှာပါတဲ့ Abstract Method တွေအားလုံးကို ဖြည့်စွက်ရေးသားပေးရမှာပါ။ ဒါ့ကြောင့် Interface ဟာ Interface ကို လာပြီး Implement လုပ်တဲ့ Class တွေအားလုံးအတွက် Interface မှာပါတဲ့ Method တွေကို ဖြည့်ရေးပေးရမယ် ဆိုတဲ့ Contract ကို ဖြစ်ပေါ်စေပါတယ်။

```
public class Car implements Movable {
      public void move() {
            System.out.println("Car is Moving!");
      }

      public void stop() {
            System.out.println("Car is Stoping");
      }
}
```

အထက်ပါနမူနာထဲမှာလို Car Class ဟာ Movable Interface ကို Implement လုပ်လိုက်တဲ့ အတွက် Car Class ထဲမှာ Movable ထဲမှာပါတဲ့ move method ကို အပြည့်အစုံ ရေးသားပေးရတော့မှာ ဖြစ်ပါတယ်။ ဒီလို ရေးသားလိုက်တာနဲ့ Car Class ကို Movable အနေနဲ့ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

```
public static void main(String [] args) {

      Movable move = new Car();
      move.move();

}
```

အထက်ပါ နမူနာထဲမှာ Movable move = new Car(); ဆိုပြီး Interface Type Variable နေရာကို Car Object ကို ဆောက်ပြီး အစားထိုးနေပါတယ်။ Interface Type Variable ကနေ အဲ့ဒီ Interface ကို Implement လုပ်ထားတဲ့ Class အမျိုးအစား Object တွေကို Reference လုပ်နိုင်ပါတယ်။ Movable Interface ကို implement လုပ်ခြင်းအားဖြင့် Car ကို Movable အနေနဲ့ အသုံးပြုသွားနိုင်ပါတယ်။ Movable Interface မှာ move ဆိုတဲ့ method တစ်ခုပါပြီး Car Class မှာတော့ move နဲ့ stop public method တွေပါတယ်ဆိုတာကို သတိပြုစေချင်ပါတယ်။

move variable ဟာ Movable Type ဖြစ်ပေမဲ့ နောက်ကွယ်မှာတော့ Car Object ကို Reference လုပ်နေတာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ Compiler က အဲ့ဒီလို အနေအထားမျိုးမှာ move object ဟာ Movable Type လို့ပဲ သိနေပါ တယ်။ ဒါ့ကြောင့် move object ကနေ တဆင့် Movable မှာပါတဲ့ move() method ကို သာ အသုံးပြုနိုင်မှာ ဖြစ်ပါ တယ်။ Runtime မှာ အလုပ်လုပ်တဲ့ အခါကျတော့ Car Object ရဲ့ move() method က အလုပ်လုပ်သွားမှာပါ။

Compile Time မှာတော့ Reference Variable ရဲ့ Type ကို သာ သိရှိပြီး Runtime ကျရင်တော့ Actual Object ကို အသုံးပြုနေတာဖြစ်ပါတယ်။

ဒီနေရာမှာ ပြောချင်တာကတော့ Interface ကို Reference Type အနေနဲ့ အသုံးပြုခြင်းအားဖြင့် Interface ကနေ ခွင့်ပြုထားတဲ့ public method တွေကိုသာ အသုံးပြုသူက သိရှိစေနိုင်ပြီး နောက်ကွယ်က Implementation တွေ ကို အသုံးပြုသူဘက်ကနေ မမြင်အောင် ဖုန်းကွယ်ပေးထားနိုင်ပါတယ်။ ဒါ့ကြောင့် Interface တွေကို ခံပြီး အသုံးပြု ခြင်းအားဖြင့်လဲ Encapsulation ကို မြင့်မားအောင် ဆောင်ရွက်ပေးနိုင်ပါတယ်။

(Interface နဲ့ပတ်သက်ပြီး Polymorphism နဲ့ Abstraction တို့မှာ ဆက်လက်ဖော်ပြပါဦးမယ်။)

Encapsulation ကြောင့် ရနိုင်တဲ့ အကျိုးများ

တကယ်တော့ Encapsulation ကို အပြင်လောကမှာ တွေ့ဖူးနေကျ ဖြစ်ပါတယ်။ ကျွန်တော်တို့ အသုံးပြုနေတဲ့ Smart Phone တစ်လုံးကို ကြည့်ကြရအောင်။ Power On တယ်၊ ဖုန်းခေါ်တယ်၊ Facebook ကြည့်တယ်၊ Post တင်တယ် Comment ရေးတယ်။ ဒါတွေဆိုတာဟာ ခွင့်ပြုထားတာတွေကို အသုံးပြုလို့ရနေတာပါ။ ဖုန်းခေါ်လိုက်ရင် နောက်ကွယ်မှာ ဘယ် Circut ထဲကနေ ဘယ်လို လုပ်သွားတယ်။ ပြီးတော့ ဘယ် ပရိုဂရမ်တွေက ဘယ်လို အလုပ်လုပ်သွားတယ် ဆိုတာကို သိလဲ မသိနိုင်သလို သိစရာလဲ မလိုအပ်ပါဘူး။ အဲ့ဒီအတွက် Software တွေ Update လုပ်တယ်၊ ပြီးတော့ Feature အသစ်တွေ တိုးတယ် ဒါတွေကို အသုံးပြုသူတွေ သိစရာမလိုပဲ အတွင်းပိုင်း မှာပဲ ပြုပြင်လို့ရပါတယ်။

- Object တွေရဲ့ State တွေကို မှန်ကန်အောင် ထိမ်းပေးထားနိုင်ခြင်း
  Object တွေရဲ့ State တွေ ဖြစ်ကြတဲ့ Instance Variable တွေရဲ့ တန်ဖိုးကို Setter Method တွေကနေတဆင့် သတ်မှတ်စေသောကြောင့် မှန်ကန်မှုကို စစ်ဆေးမှုတို့ကို ပြုလုပ်နိုင်မှာ ဖြစ်တဲ့ အတွက် Object တွေရဲ့ State တွေကို မှန်ကန်အောင် ထိမ်းပေးထားနိုင်မှာ ဖြစ်ပါတယ်

- Flexibility ကို မြင့်မားအောင် ဆောင်ရွက်ပေးနိုင်ခြင်း
  Object တွေကို အသုံးပြုနေတဲ့ နေရာမှာ public method တွေ Interface Method တွေကနေ တဆင့် အသုံးပြုရမှာ ဖြစ်တဲ့ အတွက် အသုံးပြုနေတဲ့ နေရာမှာ အတွင်းပိုင်း အသေးစိတ် ကိစ္စတွေကို သိစရာမလိုအပ် တော့ပါဘူး။ အဲ့ဒီ Object ကို ပြင်ဆင်လိုတဲ့အခါမှာလဲ public method တွေရဲ့ Signature ကိုသာမှန်ကန်အောင် ပြုလုပ်ထားမယ်ဆိုပါက အသုံးပြုနေတဲ့ဘက်ကို ထိခိုက်မှုမရှိအောင် ဆောင်ရွက်ပေးနိုင်မှာ ဖြစ်တဲ့ အတွက် Flexibility ကိုမြင့်မားအောင် ဆောင်ရွက်ပေးနိုင်မှာ ဖြစ်ပါတယ်

- Maintainability ကို မြင့်မားအောင် ဆောင်ရွက်ပေးနိုင်ခြင်း
  ပြုပြင်စရာလိုအပ်လာရင်လဲ အတွင်းပိုင်းကို ပြုပြင်လိုက်ရုံသာဖြစ်ပြီး ပြင်ပကို အကျိုးသက်ရောက်မှုနဲ့စေမှာ ဖြစ် တဲ့အတွက် Maintainability ကို တိုးတက်စေမှာ ဖြစ်ပါတယ်။

Inheritance

Inheritance ဆိုတာမြန်မာလို တိုက်ရိုက်ဘာသာပြန်မယ်ဆိုရင် မျိုးရိုးဆက်ခံခြင်းလို့ ပြန်ဆိုနိုင်မှာ ဖြစ်ပြီး OOP ရဲ့ အဓိက Feature တစ်ခုလဲ ဖြစ်ပါတယ်။ Inheritance နဲ့ ပတ်သက်ပြီး အောက်ပါ အကြောင်းအရာများကို လေ့လာသွားပါမယ်

- Inheritance ဆိုတာဘာလဲ
- Inheritance လုပ်လို့ရတာတွေက ဘာတွေလဲ
- ဘယ်လိုနေရာတွေမှာ Inheritance ကို အသုံးပြုသင့်ပြီး ဘယ်လိုနေရာတွေမှာ မသုံးသင့်တာလဲ

Inheritance ဆိုတာဘာလဲ

Class တစ်ခုဟာ Class တစ်ခုကို Extends လုပ်ပိုင်ခွင့်ရှိပြီး Inheritance လုပ်တယ်လို့လဲ ခေါ်လေ့ရှိပါတယ်။ ဒါ့ကြောင့် Inheritance လုပ်တယ်လို့ပြောရင် Code ထဲမှာ extends လို့ရေးတယ်လို့ နားလည်နိုင်ပါတယ်။ ဆိုကြပါစို့၊ class Dog extends Animal လို့ရေးထားရင် Dog ဟာ Animal ကို Inheritance လုပ်ထားတယ်လို့ သိနိုင်ပါတယ်။

```
public class Animal {
      public void eat() {
            System.out.println("Eating");
      }
}

class Dog extends Animal {
}
```

အထက်ပါ အတိုင်း Dog ဟာ Animal ကို Inheritance လုပ်လိုက်တဲ့ အတွက် အောက်ပါ အချက် ၂ ခုကို ဖြစ်ပေါ်စေပါတယ်။

- Dog နဲ့ Animal ကြားမှာ IS A Relationship ကို ဖြစ်ပေါ်စေပါတယ်။
- Animal မှာရေးထားတဲ့ Instance Members တွေထဲက မျိုးရိုးဆက်ခံပိုင်ခွင့်ရှိတာတွေကို Dog ဆီမှာ ရရှိနိုင်ပါတယ်။

```
public static void main(String [] args) {

      Animal papi = new Dog();

      papi.eat();

}
```

IS A Relationship ကနေ အရင်ဆုံး လေ့လာကြည့်ကြရအောင်။ Dog extends Animal လို့ရေးလိုက်တာနဲ့ Dog IS A Animal လို့ပြောနိုင်ပြီး Dog Object တွေကို Animal Type Reference Variable တွေကနေလဲ Reference လုပ်လာနိုင်ပါတယ်။

အရင်ဆုံးခေါ်လေ့ရှိတဲ့ keyword လေးတွေကို ပြောပြထားပါမယ်။ IS A Relationship ကို ဖြစ်ပေါ်စေတဲ့ နေရာမှာ Inheritance အလုပ်ခံရတဲ့ Class ဒီနေရာမှာတော့ Animal Class ကို Parent Class ဒါမှ မဟုတ် Super Class လို့ခေါ်လေ့ရှိပြီး၊ Inheritance လုပ်တဲ့ Class ဒီနေရာမှာတော့ Dog Class ကို Child Class ဒါမှမဟုတ် Sub Class လို့ခေါ်လေ့ရှိပါတယ်။

Java အကြောင်းကိုလေ့လာတဲ့နေရာမှာ Reference Type နဲ့ Object Type တွေဟာ မတူဘူးဆိုတာကိုလဲ ကွဲကွဲပြားပြားသိဖို့လိုအပ်ပါတယ်။ Reference Type ဆိုတာ Reference Variable တွေမှာ Declare လုပ်ထားတဲ့ Type ကို ဆိုလိုတာဖြစ်ပြီး Object Type ဆိုတာက new Operator ကို အသုံးပြုပြီး Object တည်ဆောက်ထားတဲ့ Object ရဲ့ Type ကို ဆိုလိုခြင်းဖြစ်ပါတယ်။

ရှေ့နမူနာထဲမှာလို Animal papi = new Dog() လို့ရေးထားတယ်ဆိုကြပါစို့။ အဲ့ဒီ Code Line ဟာ Animal Type papi ဆိုတဲ့ reference variable ကို JVM ရဲ့ Stack Memory ထဲမှာ သွားတည်ဆောက်ပါတယ်။ ပြီးတော့မှ new Dog ဆိုပြီး Heap Memory မှာ Dog Object ကို တည်ဆောက်ပါတယ်။ ပြီးတော့မှ Dog Object ရဲ့ Memory Address ကို papi ရဲ့ တန်ဖိုးအဖြစ် အစားထိုးပါတယ်။

ဒါ့ကြောင့် Reference Variable တွေဟာ Actual Value ကို ပိုင်ဆိုင်ခြင်းမရှိပဲ၊ Object တစ်ခုကို Reference လုပ်နိုင်တဲ့ Memory Address ကိုသာ ပိုင်ဆိုင်နေတာဖြစ်ပါတယ်။ ဒီနမူနာထဲမှာ ဆိုရင် Animal ဆိုတာက Reference Type ဖြစ်ပြီးတော့ Dog ကတော့ Object Type ဖြစ်ပါတယ်။

IS A Relationship ကို ဖြစ်ပေါ်စေတဲ့အတွက် Class တစ်ခုကနေ သူနဲ့ သူရဲ့ Sub Class တွေရဲ့ Object တွေကို Reference လုပ်လာနိုင်ပါတယ်။

```
public static void main(String [] args) {

      Dog papi = new Dog();

      papi.eat();

}
```

ဆက်ပြီး မျိုးရိုးဆက်ခံခွင့်ဆိုတာကို ဆက်ကြည့်ကြရအောင်။ Animal Class ထဲမှာတော့ eat() ဆိုတဲ့ Method ပါဝင်ပြီး Dog Class ထဲမှာတော့ eat() ဆိုတဲ့ method ကို ရေးမထားတာကို သတိထားမိမှာပါ။
Dog papi ရဲ့ Reference Type ဟာ Dog ဖြစ်ပြီး Dog Class ထဲမှာ eat() method ကို ရေးမထားပါဘူး။ ဒါပေမဲ့ papi reference ကနေ eat() ဆိုတဲ့ method ကို Invoke လုပ်နေပါတယ်။ Dog Class ထဲမှာ eat behavior မပါပေမဲ့ အသုံးပြုလို့ရနေပါတယ်။ အဲ့ဒါဟာ Dog ဟာ Animal ကို Inheritance လုပ်ထားတာဖြစ်တဲ့အတွက် Animal ရဲ့ eat behavior ကို Dog ကနေ မျိုးရိုးဆက်ခံနိုင်တဲ့ အတွက် ဖြစ်ပါတယ်။

Dog Object ဟာ Animal ရဲ့ eat() method ကို Inheritance လုပ်နိုင်တဲ့ အတွက် papi.eat() ဆိုပြီး အသုံးပြုနိုင်တာ ဖြစ်ပါတယ်။

Inheritance လုပ်လို့ရတာတွေကဘာတွေလဲ

Super Class ကနေ Sub Class အတွက် ဘာတွေကို Inheritance လုပ်ပိုင်ခွင့်ပေးတာလဲ။ အတိုဆုံးပြောရရင်တော့ Instance Member တွေကို Inheritance လုပ်ပိုင်ခွင့်ပေးနိုင်ပါတယ်။ အသေအချာ စဉ်းစားရင်တော့ Inheritance ဆိုတဲ့ နေရာမှာ ၂ မျိုးရှိနိုင်ပါတယ်။ အဲ့ဒါတွေကတော့ သားသမီးဖြစ်ရုံနဲ့ ရရှိလာနိုင်တဲ့ အခွင့်အရေးတွေနဲ့ မဖြစ်မနေ လုပ်ဆောင်ပေးရမည့် တာဝန်တို့ဆိုပြီး ခွဲနိုင်ပါတယ်။

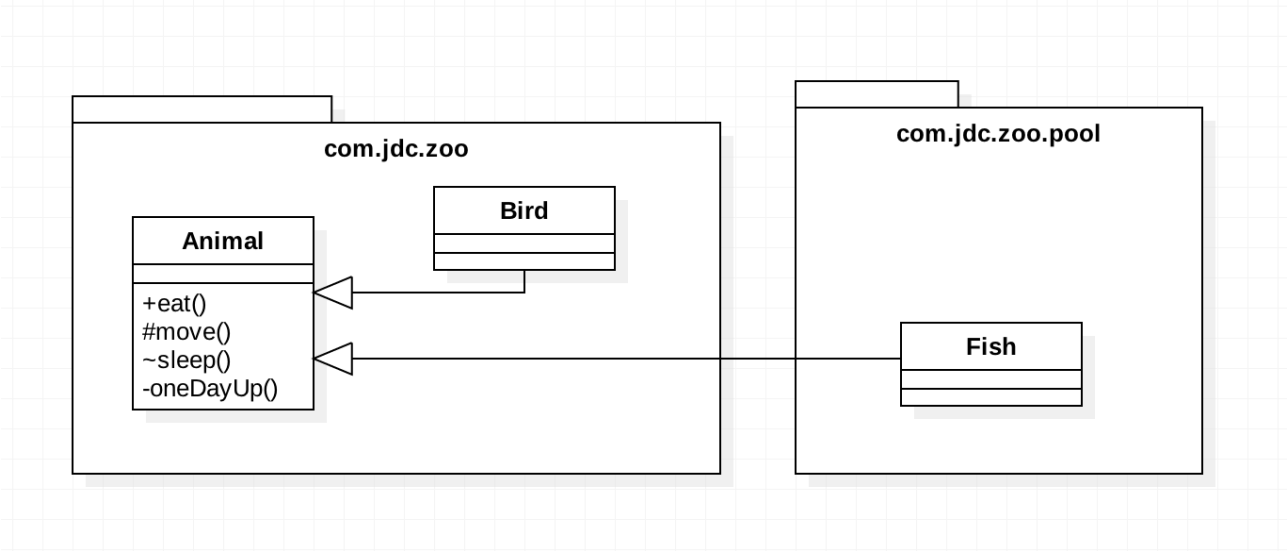
မျိုးရိုးဆက်ခံခြင်း

ရရှိနိုင်တဲ့ အမွေတွေကတော့ Super Class ထဲက မြင်နေနိုင်တဲ့ ပိုင်ဆိုင်မှုတွေ၊ Instance Variable တွေနဲ့ Instance Method တွေ ဖြစ်ပါတယ်။ ဒီနေရာမှာ မြင်နေနိုင်တဲ့လို့ တမင်ပြောတာပါ။ အမွေရတယ်ဆိုတာက သိနေနိုင်တဲ့ အရာကိုပဲရမှာပါ။ မသိအောင် ဖွက်ထားတာ၊ ပြီးတော့ မသိနိုင်တဲ့ အရာတွေကိုတော့ ရမှာ မဟုတ်ပါဘူး။

ဖွက်ထားတဲ့ အရာတွေဆိုတာက private access modifier ကို သုံးပြီး ရေးထားတဲ့ instance method တွေနဲ့ instance variable တွေကိုတော့ Sup Class တွေက Inheritance လုပ်လို့မရပါဘူး။ ပြီးတော့ Package မတူတဲ့ နေရာမှာရှိပြီး Default Access ဖြစ်နေရင်လဲ Package မတူရင် မမြင်နိုင်တဲ့ အတွက် Inheritance လုပ်မည့် Member တွေထဲမှာ မပါပါဘူး။

Access Modifier	Same Package	Different Package
public	Can Inherit	Can Inherit
protected	Can Inherit	Can Inherit
default	Can Inherit	Can not Inherit
private	Can not Inherit	Can not Inherit

Access Modifier အရပြောရမယ်ဆိုရင် public နဲ့ protected တွေကိုဆိုရင်တော့ ဘယ်နေရာကမဆို မြင်နေနိုင်တဲ့အတွက် Inheritance လုပ်ပိုင်ခွင့်ရှိပါတယ်။ private Access Modifier ဆိုရင်တော့ ဘယ် Class ကမှာ Inheritance လုပ်လို့ မရပါဘူး။ Default Access ဆိုရင်တော့ Package တူနေရင် Inheritance လုပ်ပိုင်ခွင့်ရှိပြီး Package မတူရင်တော့ Inheritance လုပ်ပိုင်ခွင့် မရှိပါဘူး။



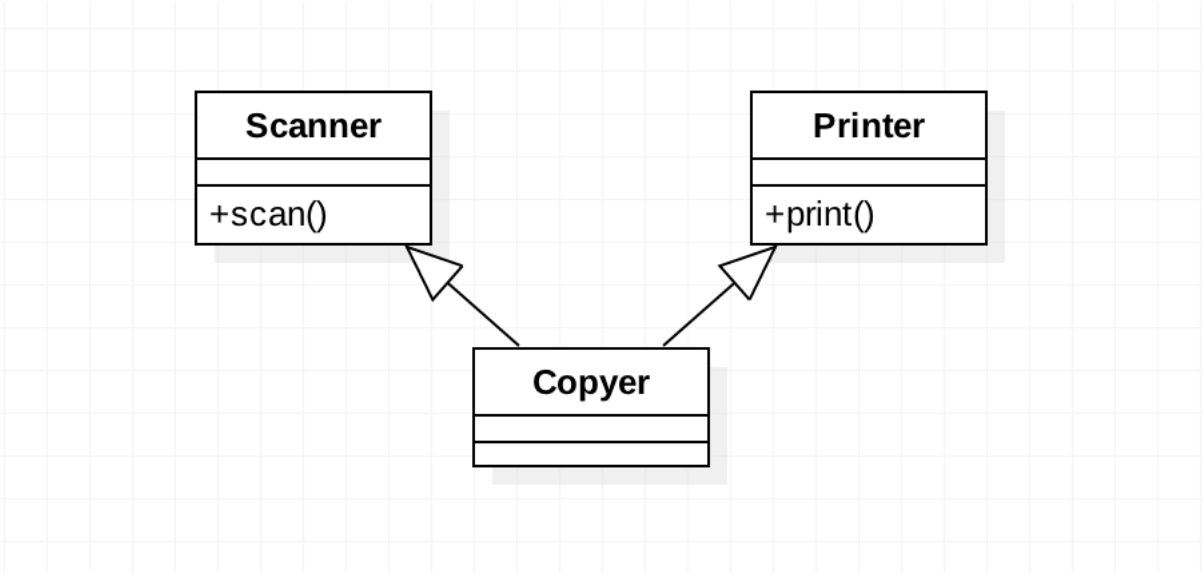
အထက်ပါ Class Diagram ထဲမှာတော့ Animal Class နဲ့ Bird Class တို့ဟာ com.jdc.zoo Package ထဲမှာ အတူတူရှိကြပါတယ်။ com.jdc.zoo.pool Package ထဲမှာတော့ Fish Class ရှိပါတယ်။ ပြီးတော့ Bird Class ကော Fish Class ပါ Animal Class ကို Extend လုပ်ထားပါတယ်။ မတူတာကတော့ Bird နဲ့ Animal Class ဟာ Package အတူတူထဲမှာရှိပြီး Fish Class ကတော့ တစ်ခြား Package တစ်ခုထဲမှာ ရှိပါတယ်။

Animal Class ထဲမှာတော့ eat() ဆိုတဲ့ public method ရယ် move() ဆိုတဲ့ protected method ရယ် sleep() ဆိုတဲ့ Default Method ရယ် oneDayUp() ဆိုတဲ့ private method တို့ကို ပိုင်ပါတယ်။ Animal Class ကို Inheritance လုပ်လိုက်ခြင်းအားဖြင့် Bird နဲ့ Fish တို့မှာ ဘာတွေကို မျိုးရိုးဆက်ခံပိုင်ခွင့် ရှိသွားမလဲ။

eat() နဲ့ move() method တို့ကတော့ Public နဲ့ Protected ဖြစ်ကြတဲ့ အတွက် Animal ရဲ့ Sub Class တွေ အားလုံး မျိုးရိုးဆက်ခံခွင့်ကို ရရှိကြမှာပါ။ sleep() method ကတော့ Default Access ဖြစ်တဲ့ အတွက် Package ချင်းတူတဲ့ Bird မှာတော့ မျိုးရိုးဆက်ခံပိုင်ခွင့်ရှိပြီး Package မတူတဲ့ Fish မှာတော့ မျိုးရိုးဆက်ခံပိုင်ခွင့် ရှိမှာ မဟုတ်ပါဘူး။ oneDayUp() method ကတော့ private ဖြစ်တဲ့ အတွက် ဘယ် Sub Class ကမှ မျိုးရိုးဆက်ခံပိုင်ခွင့် ရှိမှာ မဟုတ်ပါဘူး။ Instance Variable တွေလဲ အလားတူပဲ အလုပ်လုပ်မှာ ဖြစ်ပါတယ်။

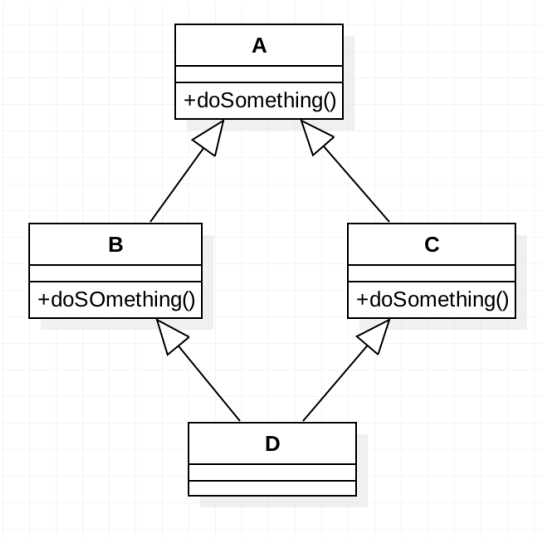
SINGLE INHERITANCE

ကျွန်တော်တို့ ဘာတွေကို Inheritance လုပ်လို့ရတယ်ဆိုတာကို သိပြီးတဲ့နောက်မှာ ဘယ်လောက် အထိ Inheritance လုပ်လို့ရနိုင်လဲဆိုတာကို လေ့လာသွားကြရအောင်။



ဆိုကြပါစို့။ ကျွန်တော်တို့ရဲ့ System ထဲမှာ scan လုပ်နိုင်တဲ့ Scanner Class တစ်ခုရယ် print လုပ်နိုင်တဲ့ Printer တစ်ခုရှိပါတယ်။ Copyer က scan လဲလုပ်ချင်တယ်၊ ပြီးတော့ print လဲ လုပ်ချင်လာရင် Scanner နဲ့ Printer ကို Extends လုပ်လိုက်ရင်ရမယ်လို့ ထင်ပါလိမ့်မယ်။ Class တစ်ခုထက်မက ကို extends လုပ်နိုင်တာကို Multiple Ihneritance လို့ခေါ်ပါတယ်။ ဒါပေမဲ့ Java ဘာသာရပ်မှာက အဲ့ဒီလို လုပ်လို့ မရပါဘူး။

Java ဟာ တစ်ခြား Programming Language တွေလို Multiple Inheritance ကို ခွင့်မပြုထားပါဘူး။ ဆိုလိုတာက Class တစ်ခုဟာ Class တစ်ခုကိုပဲ Extends လုပ်ပိုင်ခွင့်ရှိပါတယ်။ ဒီလို ဖြစ်ရခြင်းဟာလဲ အကြောင်းပြချက်ခိုင်ခိုင်မာမာရှိပါတယ်။



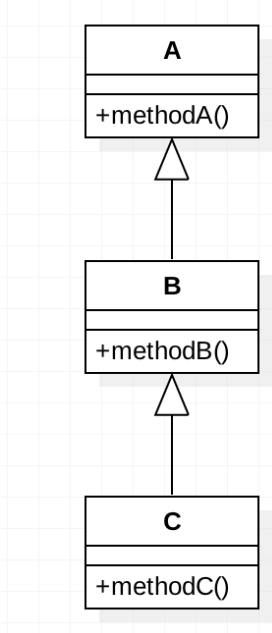
ဆိုကြပါစို့။ Class A ထဲမှာ doSomething() method ရှိပြီး Class B နဲ့ Class C ကနေ Extends လုပ်ထားပါတယ်။ ပြီးတဲ့ Class B နဲ့ C ထဲမှာလဲ doSomething() method ကို Override လုပ်ထားပါတယ်။

အဲ့ဒီနောက်မှာ Class D ကနေ Class B နဲ့ C ကို Extend လုပ်လိုက်ပြီဆိုရင် Class D ထဲမှာ B ထဲက doSomething() ကို Inheritance လုပ်မလား C ရဲ့ doSomething() ကို inheritance လုပ်မလား ဝေခွဲလို့ မရနိုင်ပါဘူး။ ဒီလိုပြဿနာမျိုးကို **Deadly Diamond of Death** လို့ ခေါ်လေ့ရှိပါတယ်။

ဒါ့ကြောင့် Java Programming မှာ Multiple Inheritance ကို လက်မခံပဲ Class တစ်ခုဟာ Class တစ်ခုထဲကိုသာ Inheritance လုပ်ရမယ်လို့ သတ်မှတ်ခဲ့ခြင်း ဖြစ်ပါတယ်။

Java ဘာသာရပ်မှာရှိတဲ့ Class တွေအားလုံးဟာ java.lang.Object ရဲ့ Sub Class တွေဖြစ်ကြပြီး Class တစ်ခုထဲကိုသာ Extend လုပ်ပိုင်ခွင့် ရှိပါတယ်။ Java ဘာသာရပ်မှာရှိတဲ့ Object တိုင်းမှာ java.lang.Object ထဲမှာပါတဲ့ method တွေကို ရေးထားထား မရေးထားထား သုံးလို့ရနေတာဖြစ်ပါတယ်။

IF YOU WANT TO INHERITANCE FROM MULTIPLE CLASSES




```
class A {
    public void methodA() {
        // some logic
    }
}

class B extends A {
    public void methodB() {
        // some logic
    }
}

class C extends B {
    public void methodC() {
        // some logic
    }
}
```

ရှေ့မှာ ဖော်ပြခဲ့သလို Java ဘာသာရပ်မှာ Class တွေဟာ Multiple Inheritance ကို လုပ်လို့မရပါဘူး။ အဲ့ဒီတော့ Class အမျိုးမျိုးကို Inheritance လုပ်လိုတဲ့ အခါမှာ ဘယ်လို လုပ်ကြမလဲ။ အဲ့ဒီအတွက် နည်းလမ်းတစ်ခုကို ဖွင့်ပေးထားပါတယ်။ အဲ့ဒါကတော့ အဆင့်ဆင့် မျိုးရိုးဆက်ခံစေခြင်းပဲ ဖြစ်ပါတယ်။

အထက်မှာ ဖော်ပြထားသလို Class A ကို Class B က Inheritance လုပ်ပြီး Class B ကို Class C ကနေ Inheritance လုပ်လိုက်မယ်ဆိုရင် Class C ထဲမှာ Class B က Instance Member တွေကော Class A ထဲက Instance Member တွေကို အသုံးပြုသွားနိုင်မှာ ဖြစ်ပါတယ်။

တဖန် Class C ဟာ Class B ရဲ့ Reference ကနေ Reference လုပ်လို့ရမှာ ဖြစ်သလို Class A ရဲ့ Reference ကနေလဲ Reference လုပ်လို့ရသွားပါမယ်။ နောက်ပြီး Class C နဲ့ Class A ရဲ့ အကြားမှာလဲ IS A Relationship

ကို ဖြစ်ပေါ်စေပါမယ်။ Class B ဟာ Class A ဖြစ်ပြီး Class C ဟာ Class B ဖြစ်တဲ့ အတွက် Class C ဟာလဲ Class A ဖြစ်သွားပါမယ်။

လုပ်ဆောင်ရမည့် တာဝန်

Inheritance လုပ်လိုက်ရုံနဲ့ Parent မှာရှိတာတွေကို မျိုးရိုးဆက်ခံပိုင်ခွင့်ရှိသွားပါတယ်။ အဲ့ဒါနဲ့ အတူ မဖြစ်မနေ လုပ်ဆောင်ရမည် အရာတွေလဲရှိပါသေးတယ်။ အဲ့ဒါတွေကတော့ Super Class မှာ Argument ကို ယူတဲ့ Constructor တစ်ခုခုရေးထားမယ်ဆိုရင် Sub Class တွေရဲ့ Constructor ထဲကနေ အဲ့ဒီ Constructor ကို ခေါ်ပေးရမှာ ဖြစ်ပါတယ်။

ကျွန်တော်တို့ Language Fundamental မှာတုန်းက Default Constructor အကြောင်း လေ့လာခဲ့သလို Java Compiler တွေက Constructor တွေကိုမရေးထားတဲ့ Class တွေကို တွေ့ရင် အလိုအလျောက် Argument မပါတဲ့ Constructor တစ်ခုကို ဖြည့်စွက်ပေးပါတယ်။ အဲ့ဒါကို Default Constructor လို့ခေါ်ပါတယ်။

အဲ့ဒါအပြင် Compiler ကနေ အကူအညီပေးတာရှိပါသေးတယ်။ Constructor တွေရဲ့ ပထမဆုံး ကုဒ်စာကြောင်းနေရာမှာ super() ဆိုပြီးလဲ ထည့်ပေးပါသေးတယ်။ ဒါကလဲ Argument မပါတဲ့ super() တွေကိုသာ ဖြည့်စွက်ပေးတာဖြစ်ပါတယ်။

Java ဘာသာရပ်မှာ Super Class ထဲမှာ Default Constructor မရှိပဲ Argument ကိုယူထားတဲ့ Constructor တစ်ခုခုရေးထားမယ်ဆိုရင် Sub Class တွေရဲ့ Constructor တွေကနေ အဲ့ဒီ Constructor ရဲ့ Signature အတိုင်း ပြန်ပြီး ရေးသားပေးရမှာ ဖြစ်ပါတယ်။

```
class A {

      public A(String data) {
            // initialization process
      }
}

class B extends A {

      public B (String data) {
            super(data);
      }
}
```

အထက်ပါ နမူနာထဲမှာလို့ Class A ကို Class B က Extends လုပ်ထားတယ်။ ပြီးတော့ Class A ထဲမှာ String တစ်ခုကို ယူထားတဲ့ Constructor ပါနေတယ်။ ဒါဆိုရင် Class B ရဲ့ Constructor ထဲမှာ super ကို String တစ်ခုကို ပေးပြီး ခေါ်ပေးရပါမယ်။ ဒီနေရာမှာ Sub Class ရဲ့ Constructor ကို Super Class အတိုင်း ရေးရမယ်လို့ မဆိုလိုပါဘူး။ Hard Coding ပဲဖြစ်ဖြစ် super(data) ကို ခေါ်ပေးဖို့သာ အဓိက ဖြစ်ပါတယ်။ ဘာလို့လဲဆိုတော့ Super Class ကို Initialize မလုပ်ပဲ Sub Class တွေရဲ့ Object တွေကို မဆောက်နိုင်တဲ့ အတွက် ဖြစ်ပါတယ်။

Usage of Inheritance

Inheritance အကြောင်းကိုလေ့လာခဲ့တဲ့နေရာမှာ Super Class ထဲမှာရေးသားထားတဲ့ Instance Member တွေကို Sub Class တွေမှာ ပြန်ပြီး အသုံးပြုလို့ရနိုင်တယ်ဆိုတာကို တွေ့ရပါလိမ့်မယ်။ ဒါဟာ Code တွေကို Reuse လုပ်လို့ရတဲ့ အတွက် ကောင်းတဲ့ အချက်တစ်ခုဖြစ်တာတော့ မှန်ပါတယ်။ ဒါပေမဲ့ Inheritance လုပ်ခြင်းဟာ IS A Relationship ကို ဖြစ်ပေါ်စေတဲ့ အတွက် Object တွေအကြားမှာရှိတဲ့ Coupling ကို ပိုပြီး မြင့်မားစေပါတယ်။

Object တွေဟာ Coupling ဖြစ်လေလေ၊ Dependency မြင့်မားလေလေ ဖြစ်လာပါမယ်။ Dependency မြင့်မားတဲ့ ကုဒ်တွေဟာ တစ်ကိုယ်ထဲဆောင်ရွက်နိုင်မှုကို ကျဆင်းစေပါမယ်။ Class တစ်ခုကို အသုံးပြုချင်ရင် သူရဲ့ Dependency တွေပါမှ သုံးလို့ရလာပါလိမ့်မယ်။ ပြီးတော့လဲ တစ်နေနေရာမှာ ပြုပြင်လိုက်ရင်လဲ Side Effect တွေကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် Code တွေကို Reuse လုပ်ချင်တဲ့ ရည်ရွက်ချက်တစ်ခုထဲနဲ့တော့ Inheritance ကို အသုံးပြုတာကို ရှောင်သင့်ပါတယ်။ ကဲ ဒါဖြင့်ရင် ဘယ်လိုနေရာတွေမှာ Inheritance ကို အသုံးပြုသင့်သလဲ။

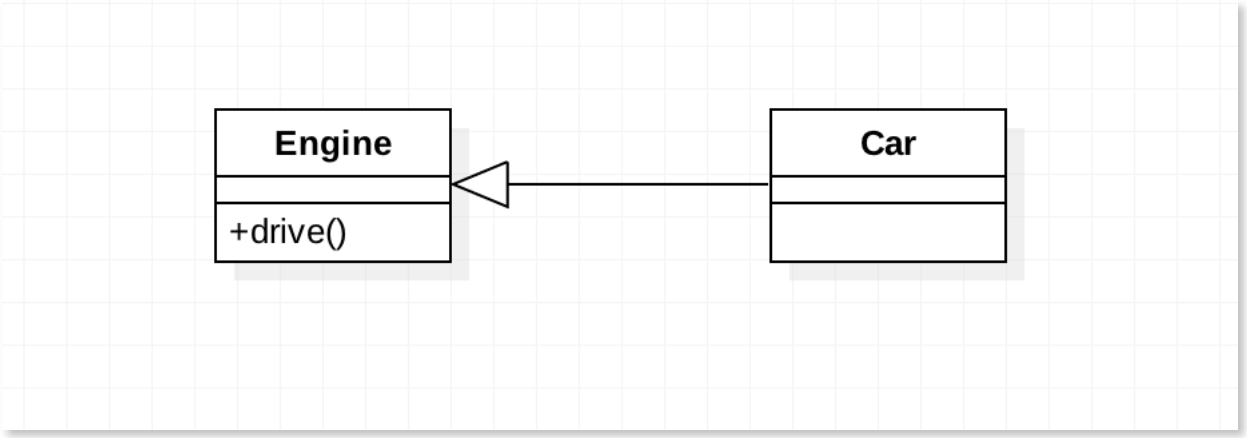
တူညီတဲ့ Behavior တွေရှိတဲ့ Object တွေကို အုပ်စုဖွဲ့ပြီး အသုံးပြုလိုချင်ရင်တော့ Inheritance ကို အသုံးပြုသင့်ပါတယ်။ ဒီအချက်နဲ့ ပတ်သက်ပြီးတော့ နောက်အခန်းတွေဖြစ်ကြတဲ့ Polymorphism, Abstraction နဲ့ Composition တွေမှာလဲ ဆက်ပြီး လေ့လာသွားကြပါဦးမယ်။

Composition

မိမိရေးသားထားတဲ့ Code တွေကို ပြန်လည် အသုံးပြုနိုင်အောင် Inheritance ကို အသုံးပြုပြီးရေးသားနိုင်တယ်ဆိုတာကို လေ့လာခဲ့ပါတယ်။ အဲ့ဒါအပြင် Composition ကိုလဲ အသုံးပြုပြီး ရေးသားနိုင်ပါတယ်။ Inheritance ကော Composition ပါ Object တွေအကြားမှာ Relationship တွေကို ဖြစ်ပေါ်စေပါတယ်။

Inheritance ကို အသုံးပြုခြင်းအားဖြင့် IS A Relationship ကို ဖြစ်ပေါ်စေပြီး Composition ကို အသုံးပြုခြင်းအားဖြင့် HAS A Relationship ကို ဖြစ်ပေါ်စေပါတယ်။ ရှေ့မှာ ဖော်ပြခဲ့သလိုပဲ Inheritance ကြောင့် ကုဒ်တွေကို ပြန်ပြီး အသုံးချနိုင်မယ်ဆိုတာက မှန်ပေမည့် Class အကြားမှာ ပတ်သက်မှုကို ပိုပြီး ခိုင်မာစေပါတယ်။ အဲ့ဒီအတွက် Class တစ်ခုကို အသုံးပြုလိုရင် သူ့ရဲ့ Super Class တွေမရှိပဲ အသုံးပြုလို့ရမှာ မဟုတ်တော့ဘူး။ Super Class ရဲ့ ကုဒ်တွေကိုပြုပြင်လိုက်ရင် သူ့ရဲ့ Sub Class တွေမှာလဲ အကျိုးသက်ရောက်မှုတွေကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ် ဒါ့ကြောင့် အဲ့ဒီ Class ရဲ့ Portability ကို ကျဆင်းစေနိုင်ပါတယ်။ ဒါ့ကြောင့် အဲ့ဒီကုဒ်တွေဟာ ပြုပြင်ပြောင်းလဲရခက်ခဲပြီး ပြန်လည် အသုံးချနိုင်မှုကို ကျဆင်းစေမှာ ဖြစ်ပါတယ်။

အောက်ပါ နမူနာကို ကြည့်ကြရအောင်။



Engine ဆိုတဲ့ Class ထဲမှာ drive() ဆိုတဲ့ Method ကို ပိုင်ပါတယ်။ ပြီးတော့ ကျွန်တော်တို့ Drive Feature ကို ပိုင်တဲ့ Car ဆိုတဲ့ Class ကို ရေးသားဖို့လိုအပ်လာကြပြီဆိုကြပါစို့။

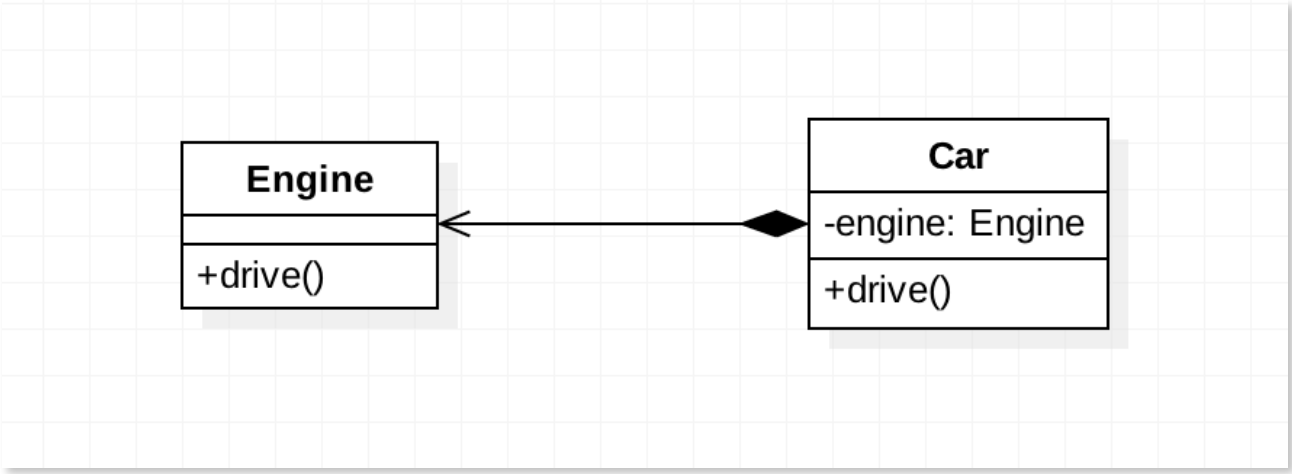
Engine Class ထဲက drive() method ကို အသုံးပြုလိုတဲ့အတွက် Car ကနေ Engine ကို Inheritance လုပ်ပြီး ရေးလိုက်တယ်လို့ဆိုကြပါစို့။ Car ဟာ Engine ကို Extends လုပ်လိုက်တဲ့ အတွက် Engine ရဲ့ drive() ကို Car က နေလဲ အသုံးပြုသွားနိုင်ပါမယ်။ အိုကေ ကျွန်တော်တို့လိုချင်တဲ့ Code Reuse ကို ရရှိလိုက်ပါမယ်။ ဒါပေမဲ့ ဒါဟာ သင့်တော်သလားဆိုတာကို စဉ်းစားကြည့်ပါမယ်။

Car ကနေ Engine ကို Inheritance လုပ်လိုက်ခြင်းအားဖြင့် Car နဲ့ Engine ကြားမှာ IS A Relationship ကို ဖြစ်ပေါ်စေပါမယ်။ ဆိုလိုတာက Car ဟာ Engine ဖြစ်သွားပါမယ်။ ဒါဆိုရင် ကားကို Engine သုံးသလို သွားသုံးနေပါလိမ့်မယ်။ မော်တော်ဘုတ်ရဲ့ အင်ဂျင်နေရာမှာ ကားကို သွားပြီး အသုံးပြုလို့ရသွားပါမယ်။ ဒါဟာ သင့်တော်တဲ့ အသုံးပြုပုံမဟုတ်ပါဘူး။

ဒါ့ကြောင့် Code တွေကို ပြန်ပြီး အသုံးပြုလိုတဲ့ ရည်ရွယ်ချက်တစ်ခုထဲနဲ့ Inheritance ကို အသုံးမပြုသင့်ပါဘူး။

Using Composition

ကုဒ်တွေကိုပြန်လည် အသုံးချလိုတဲ့အခါ နောက်တနည်း အနေနဲ့ အသုံးပြုနိုင်တာကတော့ Composition ဖြစ်ပါတယ်။ Composition ဆိုတာကတော့ အသုံးပြုလိုတဲ့ Behavior ကို ပိုင်တဲ့ Object ကို မိမိရဲ့ ရဲ့ Member အနေနဲ့ အသုံးပြုပြီး လိုအပ်တဲ့နေရာမှာ အဲ့ဒီ Object ဆီက Behavior ကို ယူပြီး အသုံးပြုခြင်းပါပဲ။



အထက်ပါအတိုင်း Car ထဲမှာ Instance Variable အနေနဲ့ Engine Object ကို ထားပြီး၊ Car ရဲ့ drive() method ကို ရေးသားတဲ့အခါမှာ engine.drive() လို့ ရေးသားလိုက်ရုံပါပဲ။

```
class Engine {
	public void drive() {
		System.out.println("Driving");
	}
}

class Car {
	private Engine engine;

	public Car(Engine engine) {
		this.engine = engine;
	}

	public void drive() {
		this.engine.drive();
	}
}
```

Composition ကို အသုံးပြုခြင်းအားဖြင့် တစ်ခါရေးထားတဲ့ Engine Class ကို Car မှာလဲ အသုံးပြုနိုင်မှာ ဖြစ်သလို MotorBoat မှာလဲ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် ကုဒ်တွေကို ပြန်လည် အသုံးပြုလိုတဲ့ အခါမျိုးမှာ Inheritance အစား Composition ကို အသုံးပြုသင့်ပါတယ်။

Abstraction

Abstraction ကို မြန်မာလိုဘာသာပြန်ရတာ အမတန်ခက်ခဲပါတယ်။ “ဘုံသဘောဆောင်အောင်ပြုလုပ်ခြင်း“ ဒါမှမဟုတ် “ဘုံအဖြစ်အသုံးပြုနိုင်သော State နှင့် Behavior တို့ကိုစုထုတ်ပြီး အမျိုးအစားအုပ်စုဖွဲ့ခြင်း” လို့ဘာသာပြန်မိမှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ နားလည်မလားဆိုတော့ နားလည်မှာ မဟုတ်ပါဘူး။

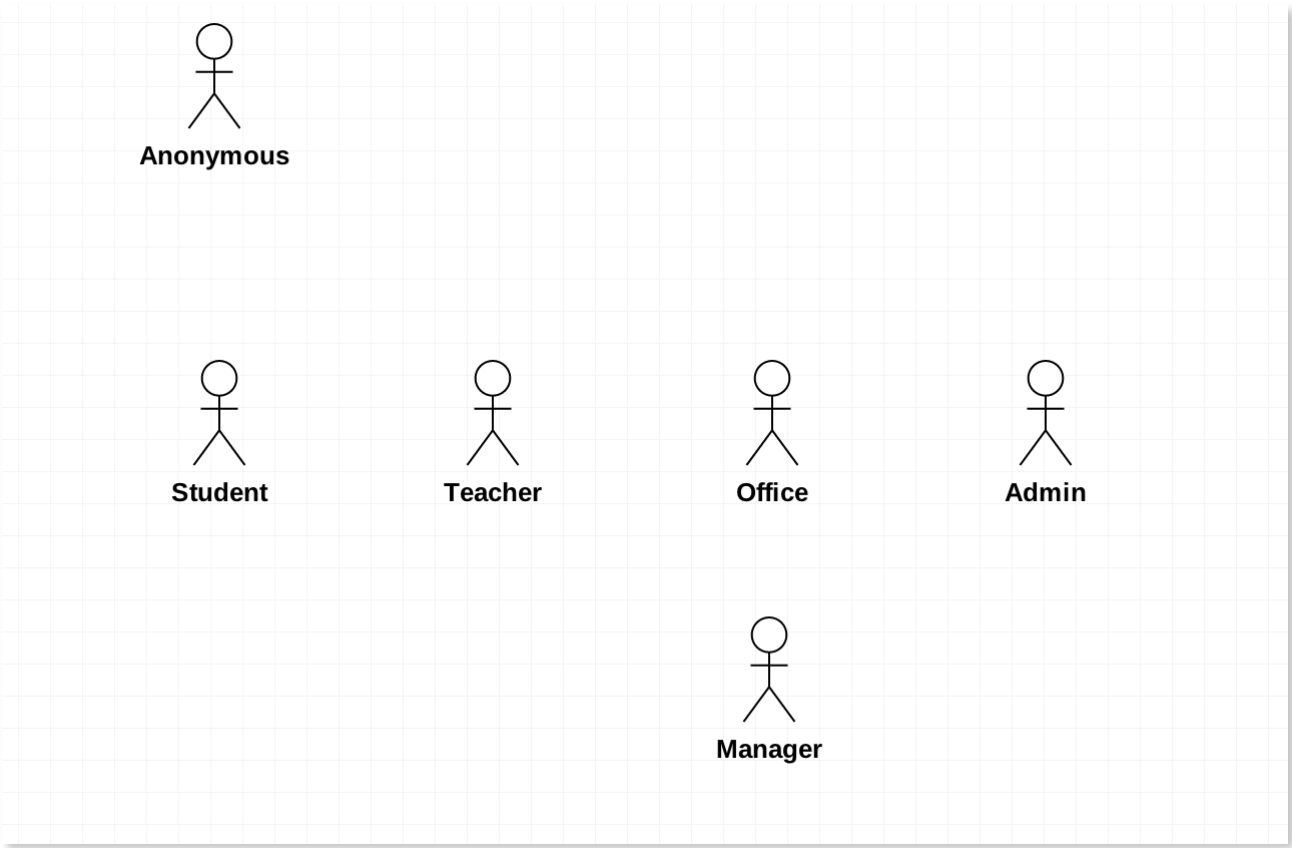
Abstraction ကို အသုံးပြုတဲ့ နေရာတွေကိုကြည့်ရအောင်။ Class is an abstraction of Objects and Abstract class is abstraction of Class လို့ပြောလေ့ရှိပါတယ်။ တဖန် Interface is 100% Abstract Entity လို့လဲပြောလေ့ရှိပါတယ်။ ပြီးတော့ Abstract Method ဆိုတာ Method Body မပါပဲ Method Signature သာပါတဲ့ Method လို့သတ်မှတ်ထားပြန်တယ်။

အဲ့ဒီတော့ Abstraction ဆိုတာ တကယ်မရှိပဲ သဘောတရားအရရှိသင့်တယ်လို့ သတ်မှတ်ပေးထားနိုင်ခြင်းလို့ သတ်မှတ်နိုင်ပါလိမ့်မယ်။ ဒီအခန်းထဲမှာတော့ Abstraction နဲ့ပတ်သက်တဲ့ အောက်ပါ အကြောင်းအရာတွေကို လေ့လာသွားပါမယ်။

- ဘာကြောင့် Abstraction ကို အသုံးပြုရတာလဲ
- Abstraction ဖြစ်အောင်ဘယ်လိုရေးမလဲ
- Abstraction ကို သုံးတဲ့အတွက် ဘာဖြစ်မလဲ

ဘာကြောင့် Abstraction ကို အသုံးပြုရတာလဲ

ဆိုကြပါစို့ ကျွန်တော်တို့ Student Management System တစ်ခုကို ရေးနေကြတယ်။ ကျွန်တော်တို့ Application ကို အသုံးပြုမည့်သူတွေကတော့ အောက်ပါအတိုင်းဖြစ်ကြပါတယ်။



Anonymous User အနေနဲ့ကတော့ Login လုပ်စရာမလိုပဲ Public Information တွေကို အသုံးပြုနိုင်ပါလိမ့်မယ်။ တဖန် Application ဆီကို မေးခွန်းတွေကို မေးမယ်ဆိုရင်လဲ မေးနိုင်ပါလိမ့်မယ်။

Student တွေအနေနဲ့ကတော့ သူတက်ခဲ့တဲ့ အတန်းနဲ့ ပတ်သက်တဲ့ Information တွေ၊ သူ့ရဲ့ Billing Status တွေ ပြီးတော့ သူ့ရဲ့ Personal Information တွေကို အသုံးပြုနိုင်မယ်။ ပြီးတော့ Public Information တွေကိုလဲ အသုံးပြုနိုင်ပါလိမ့်မယ်။

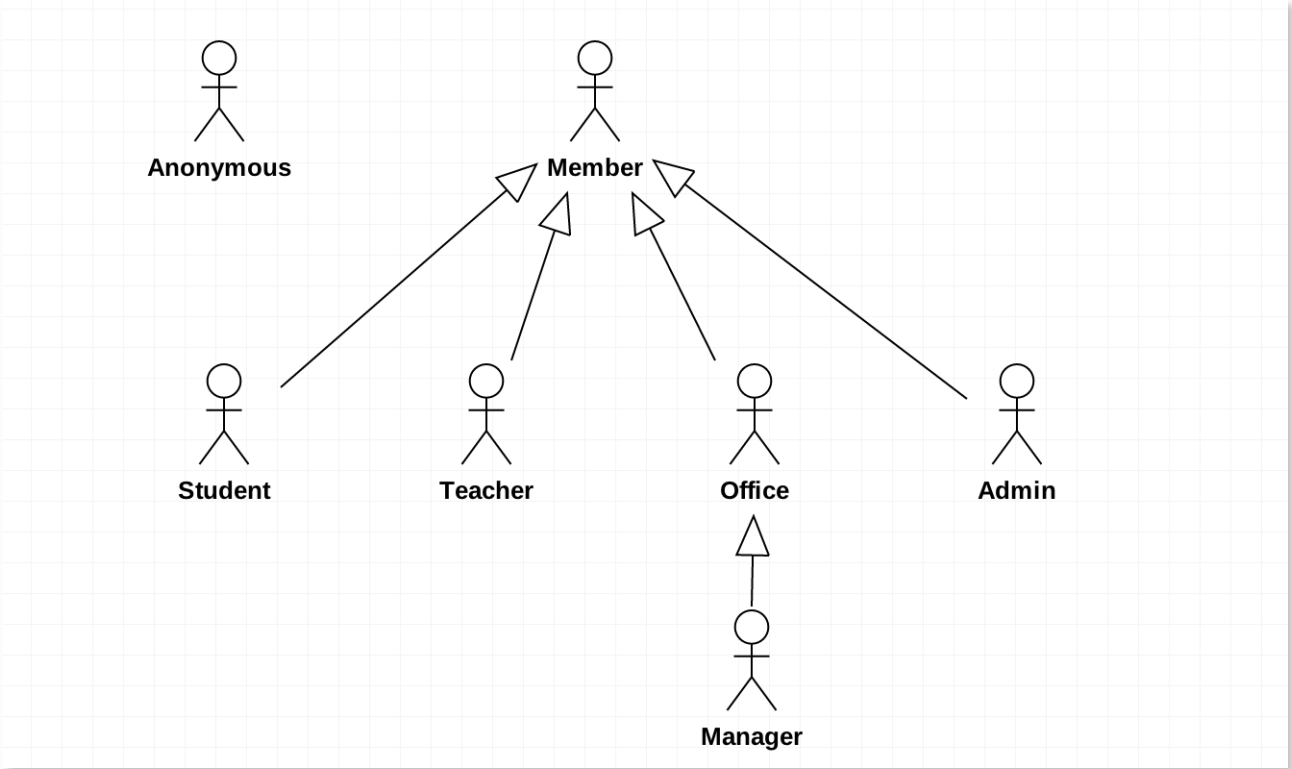
Teacher အနေနဲ့ကတော့ Course အသစ်တွေကို Create လုပ်နိုင်မယ်။ ကျောင်းသားတွေရဲ့ Attendance တွေ၊ အတန်း အချိန်စာရင်းတွေ၊ ပြီးတော့ ကျောင်းသားတွေရဲ့ တိုးတက်မှု မှတ်တမ်းတွေကို အသုံးပြုနိုင်ပါတယ်။

Office ကတော့ ကျောင်းသားတွေရဲ့ Registration တွေ၊ ပြီးတော့ Billing Status တွေ၊ အတန်း အသစ်ဖွင့်တာတွေ၊ Schedule Management လုပ်တာတွေကို လုပ်ကိုင်နိုင်ပါတယ်။ Manager ကတော့ Office က လုပ်နိုင်တာတွေကို လုပ်နိုင်သလို ငွေစာရင်းတွေ၊ Budget တွေကိုလဲ Management လုပ်နိုင်ပါလိမ့်မယ်။

ဒီလိုနေရာမျိုးမှာ Office နဲ့ Manager တွေဟာ Behavior တွေဆင်တူကြတယ်။ ပြီးတော့ Manager ဆိုတာကလဲ Office အမျိုးအစားတစ်ခုပဲလို့ယူဆလို့ရပါတယ်။ ဒါ့ကြောင့် Office ကို Extends လုပ်ပြီး Manager ကို တည်ဆောက်လိုက်လို့ရပါတယ်။

Admin ကတော့ Site Admin ဖြစ်ပါတယ်။ သူကတော့ Application အတွက်လိုအပ်တဲ့ Configuration တွေကို ပြလုပ်နိုင်ရမယ်။ ပြီးတော့ Master Data တွေကို Manage လုပ်နိုင်မယ်။ Log တွေကိုလဲ Manage လုပ်နိုင်ရပါမယ်။

ဒီနေရာမှာ ကြည့်မယ်ဆိုရင် Anonymous User ကလွဲရင် ကျန်တဲ့ User တွေမှာ တူညီတဲ့ Behavior တွေရှိကြပါတယ်။ Login လုပ်တာတို့၊ Password ပြောင်းတာတို့ ပြီးတော့ Profile ကို Update လုပ်တာတို့လို Member တစ်ယောက်အနေနဲ့ လုပ်စေချင်တဲ့ Behavior တွေရှိကြပါတယ်။ ဒီနေရာမှာ Member ဆိုတဲ့ Keyword ထွက်လာပါတယ်။ ဒါဆိုရင် Member Behavior ကိုပိုင်တဲ့ Member Class ကို ဆောက်ပြီးတော့ Member Behavior ကို လိုချင်တဲ့ Class တွေကို Extends လုပ်ခိုင်းလိုက်ရုံပါပဲ။



ဒါပေမဲ့ ဒီနေရာမှာ Member Class နဲ့ ပတ်သက်ပြီး စဉ်းစားကြည့်ကြရအောင်။ Member Class ကနေတော့ Object ကို မဆောက်ချင်ဘူး။ အခြား Object တွေရဲ့ တူညီတဲ့ Behavior တွေကို စုပြီး အုပ်စုတစ်ခုအနေနဲ့ သတ်မှတ်ချင်တာပဲ ဖြစ်တယ်။ အဲ့ဒီလို Object ဆက်ချင်တဲ့ ရည်ရွယ်ချက်နဲ့ မဟုတ်ပဲ အခြား Object တွေကို အုပ်စုဖွဲ့ပြီး အသုံးပြုလိုတဲ့ အခါမျိုးမှာ Abstraction ကို အသုံးပြုနိုင်ပါတယ်။

Abstraction ဖြစ်အောင်ဘယ်လိုရေးမလဲ

Java ဘာသာရပ်မှာ Abstraction ဖြစ်အောင် နည်း ၂မျိုးနဲ့ ရေးသားနိုင်ပါတယ်။ ပထမတစ်နည်းကတော့ Abstract Class တွေကို အသုံးပြုပြီး ရေးသားတဲ့နည်း ဖြစ်ပြီးတော့ နောက်တစ်နည်းကတော့ Interface တွေကို အသုံးပြုပြီး ရေးသားတဲ့ နည်းဖြစ်ပါတယ်။

Abstract Class တွေကို အသုံးပြုခြင်း

Class ဆိုတာ Object ဆောက်ဖို့ ရေးသားရတယ်လို့ လေ့လာခဲ့ကြမှာပါ။ ဒါပေမဲ့ Abstract Class ဆိုတာက Object ဆောက်ဖို့မဟုတ်ပါဘူး။ အမျိုးအစားတူ Class အုပ်စုကို ဖွဲ့စည်းနိုင်ဖို့ရည်ရွယ်ချက်နဲ့ ရေးသားရတာ ဖြစ်ပါတယ်။

အရင်ဆုံး Abstract ဆိုတာဘယ်လိုဟာမျိုးလဲ၊ ဘာတွေလုပ်နိုင်လဲဆိုတာကို လေ့လာကြည့်ကြရအောင်။

- ✓ Class Declaration မှာ Abstract Keyword ကို အသုံးပြုပြီး ရေးသားရမယ်
- ✓ Abstract Class တွေကို final အဖြစ်သတ်မှတ်လို့မရပါဘူး
- ✓ Instance Variable တွေကို ရေးသားပိုင်ခွင့်ရှိတယ်
- ✓ Instance Method တွေကိုကော Abstract Method တွေကိုပါရေးသား အသုံးပြုပိုင်ခွင့်ရှိတယ်
- ✓ Abstract Method တွေကို ရေးသားချင်ရေးသားနိုင်သလို မရေးသားပဲလဲ နေနိုင်တယ်
- ✓ Constructor တွေကို ရေးသားအသုံးပြုနိုင်တယ်
- ✓ Abstract Class ထဲမှာ Constructor ကို ရေးသားထားမယ်ဆိုရင် Sub Class တွေကရဲ့ Constructor ထဲက နေ super(Arguments) ကို ပြန်ပြီး ခေါ်ပေးရမှာ ဖြစ်ပါတယ်
- ✓ တိုက်ရိုက် Object ကို မဆောက်နိုင်ဘူး
- ✓ Sub Class တွေရဲ့ Object တွေကို Reference လုပ်နိုင်တယ်

```
public abstract class Animal {

	private String name;

	public Animal(String name) {
		this.name = name;
	}

	public void great() {
		System.out.println("Hello my name is " + name);
	}

	public abstract void breath();
}
```

အထက်ပါ နမူနာထဲမှာ Animal ဆိုတဲ့ Class ကို ရေးသားထားပါတယ်။ Class Declaration ထဲမှာ abstract keyword ကို ရေးသားထားတဲ့ အတွက် Abstract Class တစ်ခုဖြစ်ပါတယ်။

Animal Class ထဲမှာ Method Signature သာပါဝင်ပြီး Method Body မပါတဲ့ breath() ဆိုတဲ့ Method ကို ရေးသားထားပါတယ်။ Abstract Method ဖြစ်ပါတယ်။ ဆိုလိုတာက Animal ဆိုတဲ့ အမျိုးအစားဟာ breath() ဆိုတာကို လုပ်ဆောင်နိုင်ရပါမယ်။ ဒါပေမဲ့ ဘယ်လိုလုပ်ရမယ်ဆိုတာကိုတော့ Animal ထဲမှာ ဖော်ပြထားခြင်းမရှိပါဘူး။

Animal ကို Extends လုပ်ထားတဲ့ Sub Class တွေအားလုံးကနေ breath() method ကို Override လုပ်ပြီး ရေးသားပေးရမှာ ဖြစ်ပါတယ်။ ခွေးတွေ ကြောင်တွေက အဆုတ်နဲ့ အသက်ရှူချင်ရှူမယ်၊ ငါးတွေက ပါးဟတ်နဲ့ အသက်ရှူမယ်။ သက်ဆိုင်ရာအမျိုးအစားမှာ Behavior တွေကို သတ်မှတ်ရေးသားပေးရပါမယ်။

```
public class Dog extends Animal {

	public Dog(String name) {
		super(name);
	}

	@Override
	public void breath() {
		System.out.println("Breath by Lungs!");
	}
}
```

ဒီနေရာမှာ Concrete Class ဖြစ်တဲ့ Dog က Abstract Class ဖြစ်တဲ့ Animal ကို Extends လုပ်ထားပါတယ်။ Dog တွေကို Animal အနေနဲ့ အသုံးပြုလို့ရသွားမှာ ဖြစ်ပါတယ်။ အဲ့ဒီအတွက် Dog တွေမှာ Animal မှာ လုပ်နိုင်တယ်လို့သတ်မှတ်ထားတဲ့ breath() Behavior ကို လုပ်နိုင်ဖို့လိုအပ်ပါတယ်။

ဒါကြောင့် Dog ထဲမှာ Abstract Method ဖြစ်တဲ့ breath() ကို Override လုပ်ပြီး ရေးသားဖို့လိုအပ်လာပါတယ်။ တကယ်လို့ Dog ကို Abstract Class အနေနဲ့ ထားမယ်ဆိုရင် Dog ဟာ Abstract Class ဖြစ်သွားတဲ့ အတွက် breath() method ကို Override မလုပ်ရင်လဲရသွားပါမယ်။ ဘာလို့လဲဆိုတော့ Abstract Class တွေမှာ Abstract Method တွေကို ရေးသားပိုင်ခွင့်ရှိတဲ့ အတွက်ဖြစ်ပါတယ်။

ဒီလိုပဲ Concrete Class ဖြစ်နေရင်တော့ Super Class ထဲမှာပါတဲ့ Abstract Method တွေအားလုံးကို Override လုပ်ပြီး ရေးပေးဖို့လိုအပ်ပါတယ်။

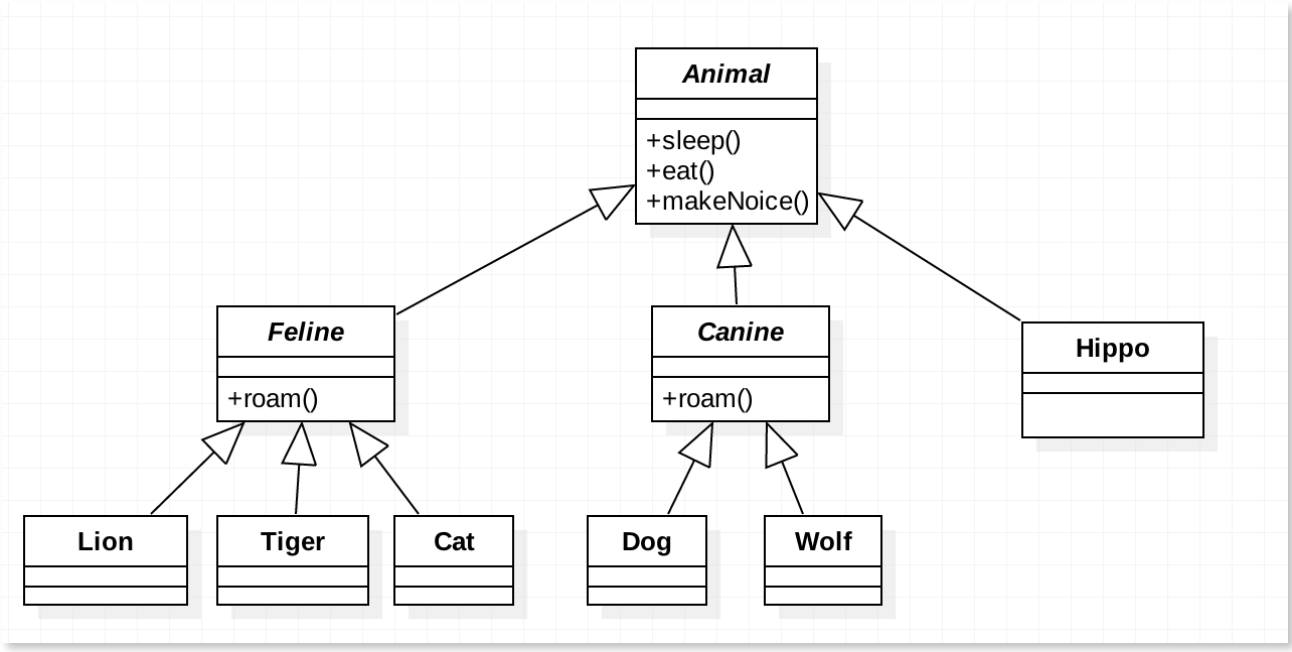
Interface တွေကို အသုံးပြုခြင်း

Abstraction ဖြစ်အောင် Abstract Class တွေကို အသုံးပြုပြီးရေးသားနိုင်သလို Interface ကို အသုံးပြုပြီးလဲ ရေးသားနိုင်ပါတယ်။

- ✓ Interface ဆိုတာက 100% Abstract Entity တစ်မျိုးဖြစ်ပါတယ်
- ✓ Interface Method တွေဟာ Default Abstract ဖြစ်ပြီး Default Public ဖြစ်ပါတယ်
- ✓ Interface တွေမှာ Constant တွေ၊ Abstract Method တွေ Default Method တွေနဲ့ Static Method တွေကို ရေးသားနိုင်ပါတယ်
- ✓ Interface ကို Reference Type အနေနဲ့ အသုံးပြုနိုင်ပါတယ်
- ✓ Interface ကို အသုံးပြုခြင်းအားဖြင့် Interface ကို အသုံးပြုနေတဲ့ ဘက်မှာကော၊ Interface ကို Implemen လုပ်ထားတဲ့ Class ဘက်မှာပါ Contract တစ်ခုကို ဖြစ်ပေါ်စေပြီး၊ Interface ကို အသုံးပြုတဲ့ Class ဘက်ကလဲ Interface ထဲမှာရေးသားထားတဲ့ Method တွေကိုပဲ သုံးပိုင်ခွင့်ရှိပါမယ်၊ Interface ကို Implement လုပ်တဲ့ Class အတွက်ကတော့ Interface မှာ ရေးသားထားတဲ့ Abstract Method တွေအားလုံးကို Override လုပ်ပြီး ရေးသားပေးရပါမယ်
- ✓ Interface တစ်ခုဟာ Interface တစ်ခုထက်မကကို Extends လုပ်နိုင်ပါတယ်
- ✓ Class တစ်ခုဟာလဲ Interface တစ်ခုထက်မကကို Implements လုပ်နိုင်ပါတယ်
- ✓ Interface ကနေတော့ Class ကို ဘာမှ လုပ်လို့မရပါဘူး

ဒီနေရာမှာ Abstract Class တွေနဲ့လဲ Abstraction ဖြစ်အောင်ရေးလို့ရသားနဲ့ ဘာလို့ Interface ကို အသုံးပြုဖို့လိုလာရတာလဲ ဆိုပြီး သံသယဖြစ်ဖွယ်ရှိပါတယ်။

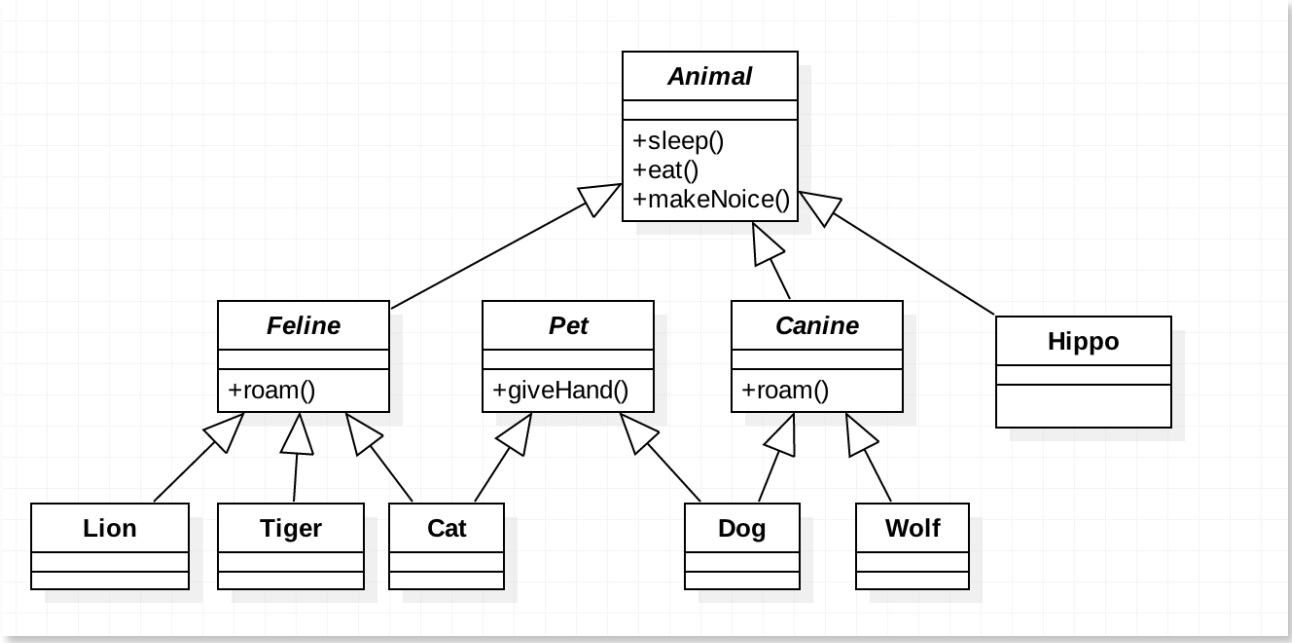
အောက်ပါ Class Diagram ကို ကြည့်ကြရအောင်။ Lion, Tiger နဲ့ Cat တို့ရဲ့ တူညီတဲ့ Behavior ကိုစုပြီး Feline ဆိုတဲ့ Abstract Class ကိုတည်ဆောက်ထားတယ်။ ပြီးတော့ Dog နဲ့ Wolf တို့ရဲ့ အုပ်စုဖြစ်တဲ့ Canine ကို Abstract Class နဲ့တည်ဆောက်ထားပါတယ်။ ထို့နောက် တိရိစ္ဆာန်အားလုံးကို ကိုယ်စားပြုဖို့အတွက် Animal ဆိုတဲ့ Abstract Class တစ်ခုကိုတည်ဆောက်ထားပါတယ်။



ဒီလို Abstraction ကို အသုံးပြုထားတဲ့ အတွက် Lion ကော Hippo ကိုပါ Animal အနေနဲ့ အသုံးပြုနိုင်မှာ ဖြစ်သလို Dog တွေ Wolf တွေကို Canine အနေနဲ့ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ Abstraction ဖြစ်အောင် ရေးသားထားခြင်းအားဖြင့် သက်ဆိုင်ရာ Abstraction Level ကနေ အုပ်စုဖွဲ့ပြီး အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

ဆိုကြပါစို့။ အိမ်မွေးတိရိစ္ဆာန်တွေဖြစ်တဲ့ Cat နဲ့ Dog မှာ giveHand ဆိုတဲ့ Behavior ကို အသုံးပြုလိုလာပါတယ်။ ဘယ်လိုလုပ်ကြမလဲ။ Cat နဲ့ Dog မှာ giveHand ဆိုတဲ့ Method ကို ထည့်လိုက်ရင်ရပါမယ်။ ဒါပေမဲ့ Cat နဲ့ Dog ကို အိမ်မွေးတိရိစ္ဆာန်အနေနဲ့ စုပြီး သုံးလို့ ရမှာ မဟုတ်ပါဘူး။ Cat နဲ့ Dog ဟာ အုပ်စုမတူပါဘူး။

ဒါဖြင့်ရင် အားလုံးရဲ့ Abstraction ဖြစ်တဲ့ Animal ဆိုတဲ့ Animal မှာ giveHand method ကို ထည့်လိုက်ရင်ကော လို့ မေးစရာရှိပါမယ်။ Lion, Tiger, Wolf နဲ့ Hippo တွေပါ လက်တွေ့လာပေးနေကြမယ်။



နောက်တစ်ခုက Pet ဆိုတဲ့ Abstract Class တစ်ခုကိုဆောက်၊ ပြီးရင် Cat နဲ့ Dog ကို Extends လုပ်ခိုင်းလိုက်ရင်ကော လို့မေးနိုင်ပါလိမ့်မယ်။ ဒါဆိုရင် Pet Behavior တွေကို Cat နဲ့ Dog ကပဲရတော့မယ်။ Lion တွေ Tiger တွေကို လက်ပေးခိုင်းလို့မရတော့ဘူးလို့ ဆိုနိုင်ပါလိမ့်မယ်။

ဒီလိုဆိုရင် Cat က Filne နဲ့ Pet Class တို့ကို Extends လုပ်ထားပြီး Dog ကလဲ Cainie နဲ့ Pet တို့ကို Extends လုပ်ထားသလိုဖြစ်သွားပါမယ်။ ဒါပေမဲ့ Java မှာက Multiple Inheritance ကို ခွင့်မပြုပါဘူး။ ဒါကလဲ Java ရဲ့ နာမည်ကြီး ပြဿနာတစ်ခုဖြစ်တဲ့ Deadly Dead of Diamond လို့ခေါ်တဲ့ ပြဿနာကြောင့်ဖြစ်ပါတယ်။ Deadly Dead of Diamond နဲ့ပတ်သက်ပြီး Inheritance အခန်းမှာ ဖေါ်ပြခဲ့ပြီး ဖြစ်ပါတယ်။

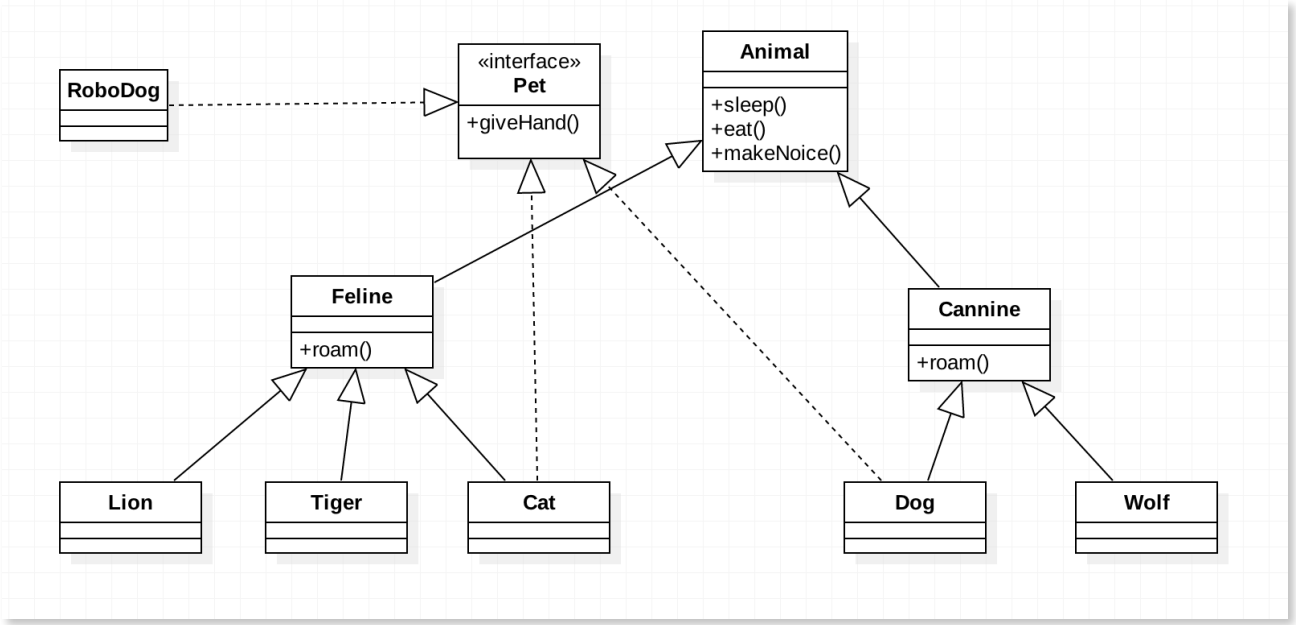
ဒါကြောင့် Abstract Class နဲ့ ရရှိနိုင်တဲ့ Abstraction ဟာ Inheritance Hierarchy တစ်ခုအတွင်းမှာရှိနေမှ ရရှိနိုင်တယ်ဆိုတဲ့ ကန့်သတ်ချက်ကို ဖြစ်ပေါ်စေပါတယ်။

RAISING OF INTERFACE

Java ဟာ OOP ကို အခြေခံထားပါတယ်။ ဒါပေမဲ့ Class တစ်ခုဟာ Class တစ်ခုဆီကသာ မျိုးရိုးဆက်ခံပိုင်ခွင့်ရှိတယ်ဆိုတဲ့ Single Inheritance ကို အသုံးပြုထားပါတယ်။ ဒါကြောင့် Java Class တစ်ခုဟာ Class တစ်ခုကိုသာ Extends လုပ်လို့ရပါတယ်။

အထက်ပါနမူနာလိုမျိုး Inheritance Hierarchy မတူတဲ့ Class နှစ်ခုအကြားမှာ Abstraction ကို အသုံးပြုလိုတဲ့ အခါမျိုးမှာ Interface ကို အသုံးပြုပြီး ရေးသားကြရမှာ ဖြစ်ပါတယ်။ Class တစ်ခုဟာ Class တစ်ခုကိုသာ

Extend လုပ်နိုင်ပေမဲ့ Interface ကိုတော့ တစ်ခုထက်မက Implement လုပ်နိုင်ပါတယ်။ ပြီးတော့ ဘယ်လို Inheritance Hierarchy ကမဆို Interface ကို Implement လုပ်နိုင်ပါတယ်။ Interface ကို Type အနေနဲ့ အသုံးပြုနိုင်တဲ့ အတွက် Interface ကို အသုံးပြုမယ်ဆိုရင် Inheritance Hierachy မတူတဲ့ Object တွေအကြား မှာ Abstraction ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။



အထက်ပါ အတိုင်း Cat ကော Dog ကိုပါ Pet Interface ကို Implement လုပ်ခိုင်းခြင်းအားဖြင့် Cat ကိုကော Dog ကိုပါ Pet အနေနဲ့ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ထို့အပြင် Robot ဖြစ်တဲ့ RoboDog ကိုလဲ Pet ကို Implement လုပ်ပါက RoboDog ကိုပါ Pet အနေနဲ့ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

ဒါ့ကြောင့် Interface ကို အသုံးပြုမယ်ဆိုရင် Inheritance Hierachy မတူညီတဲ့ Object တွေကြားမှာ Abstraction ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Abstraction ကို သုံးတဲ့အတွက် ဘာဖြစ်မလဲ

Abstraction ကြောင့် Object တွေရဲ့ Dependency တွေကို လျော့ကျစေနိုင်ပါတယ်။ Dependency ဆိုတာက Object တစ်ခုနဲ့ တစ်ခုအကြားမှာရှိတဲ့ မှီခိုမှုကို ဆိုလိုခြင်းဖြစ်ပါတယ်။ Dependency များလေလေ Object တွေကြားမှာရှိတဲ့ Coupling များလေလေဖြစ်ပါမယ်။ Coupling ဆိုတာက Object တွေအကြားမှာရှိတဲ့ တွယ်ကပ်နေမှုကို ဆိုလိုတာပါ။

Object A နဲ့ Object B အကြားမှာ Coupling များနေတယ်ဆိုရင် Object A နဲ့ B တို့ကို ခွဲပြီး သုံးလို့ရမှာ မဟုတ်တော့ပါဘူး။ Object A ကို သုံးချင်ရင် Object B ရှိမှသုံးလို့ရမည့် အနေအထားဖြစ်လာပါမယ်။ အဲ့ဒီလို Class တွေဟာ ပြုပြင်ပြောင်းလဲရခက်ခဲပြီး ပြန်လည် အသုံးချရလဲ ခက်ခဲသွားပါမယ်။

```
public class House {

      private Dog dog;
}
```

အထက်ပါ House Class ထဲမှာ Dog Instance Variable ကို အသုံးပြုထားရင် အဲ့ဒီ House Class ထဲမှာ Dog ကလွဲရင် ကျန်တဲ့ Object တွေကို သုံးလို့ရမှာ မဟုတ်တော့ပါဘူး။

```
public class House {

      private Pet pet;
}
```

အဲ့ဒီအစား ပိုပြီး Abstraction ကျတဲ့ Pet Interface Object ကို အသုံးပြုထားရင် Pet Interface ကို Implement လုပ်ထားတဲ့ Object တွေအားလုံးကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

အထက်ပါအတိုင်း Class တစ်ခုရဲ့ Dependency ဖြစ်တဲ့ Instance Variable တွေနေရာမှာ Abstraction ကို အသုံးပြုခြင်းအားဖြင့် Coupling ကို လျော့ချနိုင်ပြီး ပြုပြင်ပြောင်းလဲရလွယ်ကူပြီး ပြန်လည် အသုံးချနိုင်သော ကုဒ်တွေကို ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

Method Argument တွေနေရာမှာလဲ Specific ဖြစ်တဲ့ Type တွေအစား ပိုပြီး Abstraction ကျတဲ့ Abstract Class တွေ Interface တွေကို အသုံးပြုတာဟာ ပြုပြင်ပြောင်းလဲရလွယ်ကူပြီး ပြန်လည် အသုံးချနိုင်သော ကုဒ်တွေကို ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

Polymorphism

Polymorphism ဆိုတာက Poly နဲ့ Morphic ဆိုတဲ့ Greek စကားလုံးများကနေ ဆင်းသက်လာတာဖြစ်ပါတယ်။ Poly ဆိုတာက အမျိုးမျိုးရှိနိုင်ခြင်းကိုဆိုလိုပြီး Morphic ဆိုတာကတော့ အသွင်၊ ပုံစံလို့ အဓိပါယ်ရပါတယ်။ ဒါ့ကြောင့် Polymorpishm ဆိုတာကတော့ ပုံစံအမျိုးမျိုးရှိနိုင်ခြင်းလို့ အဓိပါယ်ရပါတယ်။

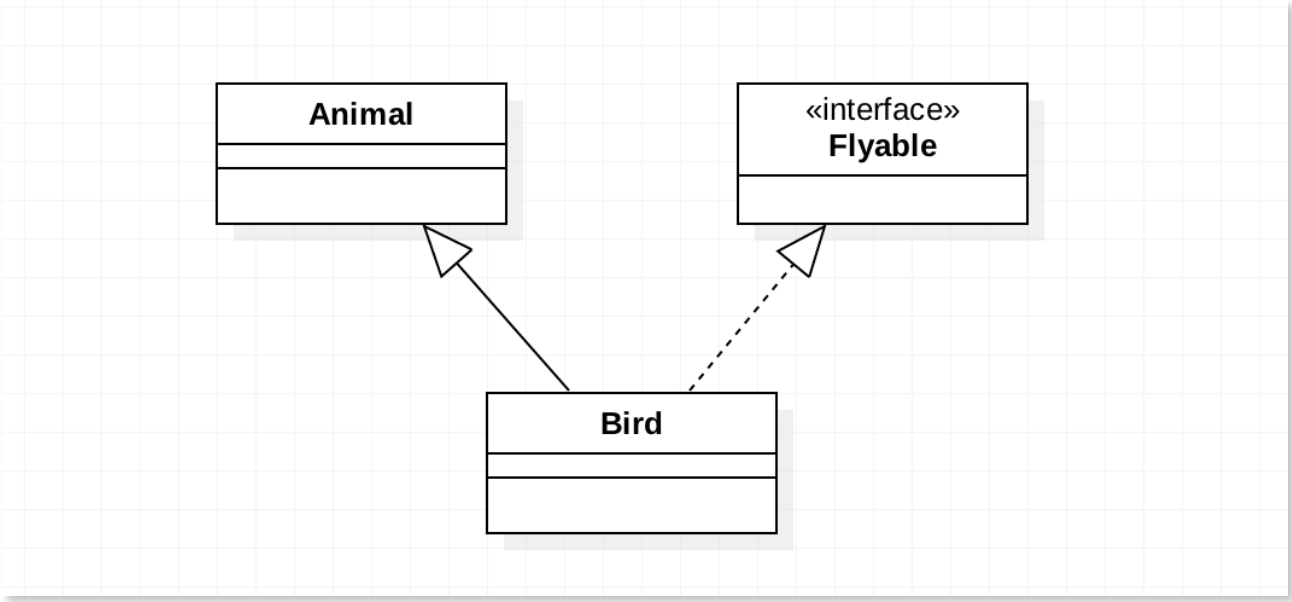
အခဲ့ဒီ ပုံစံအမျိုးမျိုးရှိနေနိုင်ခြင်းနဲ့ ပတ်သက်ပြီး ရှင်းပြရမယ်ဆိုရင် အခြေခံပုံစံ ၂ မျိုးနဲ့ ရှင်းပြနိုင်ပါလိမ့်မယ်။ ပထမ ပုံစံက Object တစ်ခုကို Reference အမျိုးမျိုးကနေ Reference လုပ်နေနိုင်ခြင်း ဖြစ်ပြီး နောက်တစ်နည်းကတော့ Override လုပ်ထားခြင်းကြောင့် Object တစ်ခုရဲ့ လုပ်ဆောင်ပုံဟာ အမျိုးမျိုးရှိနေနိုင်ခြင်းတို့ ဖြစ်ကြပါတယ်။

ကျွန်တော်တို့ ဒီနေရာမှာ အောက်ပါ အကြောင်းအရာတွေကိုလေ့လာသွားပါမယ်။

- Object တစ်ခုကို Reference အမျိုးမျိုးကနေ Reference လုပ်နေနိုင်ခြင်း
- Override လုပ်ထားခြင်းကြောင့် ပုံစံအမျိုးမျိုးသော Object တွေကို ရေးသားနိုင်ခြင်း
- မှန်ကန်သော Override Rules များ
- Override နှင့် Overload
- Polymorphism ရဲ့ အကျိုးများ

Object တစ်ခုကို Reference အမျိုးမျိုးကနေ Reference လုပ်နေနိုင်ခြင်း

Java မှာ ရေးသားသမျှ Class တွေအားလုံးဟာ java.lang.Object ရဲ့ Sub Class တွေ ဖြစ်ကြတယ်။ ပြီးတော့ အခြား Class ကိုလဲ Extends လုပ်နိုင်ပါတယ်။ တဖန် Interface ကိုလဲ Implement လုပ်နေနိုင်ပါတယ်။



အထက်မှာ ဖေါ်ပြထားသလို Bird Class ဟာ Animal Class ကို Extends လုပ်ထားသလို Flyable Interface ကိုလဲ Implement လုပ်ထားနိုင်ပါတယ်။ အခဲ့ဒီအတွက် Bird Object ကို Bird အနေနဲ့ သော်၎င်း Animal အနေနဲ့ သော်၎င်း Flyable အနေနဲ့ သော်၎င်း အသုံးပြုနိုင်ပါတယ်။ တဖန် Java Class တွေ အားလုံးဟာ java.lang.Object ရဲ့ Sub Class တွေ ဖြစ်ကြတဲ့ အတွက် Object အနေနဲ့လဲ အသုံးပြုနိုင်ပါတယ်။

အောက်ပါ နမူနာကိုကြည့်မယ်ဆိုရင် Bird Object ကို Bird bird = new Bird() ဆိုပြီး တည်ဆောက်ထားပါတယ်။ ပြီးတော့ Animal reference နေရာမှာလဲ Bird Object ကို အစားထိုးနိုင်ပါတယ်။ Animal နေရာကနေ Bird Object ကို Reference လုပ်ပြီး Animal အနေနဲ့ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ တဖန် Flyable အနေနဲ့ကော Object အနေနဲ့ ပါ Bird Object ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ထိုကဲ့သို့ Object တစ်ခုကို ပုံစံအမျိုးမျိုးနဲ့ အသုံးပြုနိုင်ခြင်းကို Polymorphism လို့ခေါ်ပါတယ်။

```
	public static void main(String [] args) {

		Bird bird = new Bird();
		Animal animal = bird;
		Flyable flyable = bird;
		Object object = bird;
	}
```

Override ကြောင့် Object ပုံစံအမျိုးမျိုးရေးသားနိုင်ခြင်း

Inheritance ကိုလေ့လာခဲ့တဲ့နေရာမှာ Super Class ထဲက Instance Member တွေကို Inheritance လုပ်လို့ရ မယ်ဆိုတာကို သိခဲ့ပြီးပါပြီ။ အဲ့ဒီအရာတွေဟာ IS A Relationship ရှိတာနဲ့ အလိုလိုကို ရရှိနေမှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ Super Class ထဲကနေရလာတဲ့ Behavior ကို မသုံးချင်ဘူးဆိုရင်ကော ဘယ်လိုလုပ်ရမလဲ။

Method Override

ဆိုကြပါစို့ Fish ဆိုတဲ့ Class မှာ swim ဆိုတဲ့ Behavior ပါလာတယ်။ ရွှေငါးတစ်ကောင်နဲ့ ရိုးရိုးငါးတစ်ကောင်တို့ ဟာ ရေကူးတာချင်း မတူနိုင်ဘူး။ အဲ့ဒီတော့ ရေကူးခြင်း ဆိုတဲ့ Behavior ကို Sub Class တွေက ပြင်ဖို့လိုအပ်လာ

```
	class Fish {
		public void swim() {
			System.out.println("Fish is Swimming");
		}
	}

	class GoldenFish extends Fish {
		public void swim() {
			System.out.println("Golden Fish is Swimming");
		}
	}

	public class Test {
		public static void main(String [] args) {
			Fish fish = new GoldenFish();
			fish.swim();
		}
	}
```

ပြီ။ အဲ့ဒါကို Java ထဲမှာ ဘယ်လိုရေးရလဲ ဆိုတော့ Super Class ထဲမှာ ပါတဲ့ Method နဲ့ Signature ချင်းတူတဲ့ Method ကို Sub Class ထဲမှာ ရေးထားလို့ရပါတယ်။ အဲ့ဒီလို ရေးသားခြင်းကို Override လုပ်တယ်လို့ခေါ်ပါတယ်။

နမူနာထဲမှာဆိုရင် Fish fish = new GoldenFish() လို့ရေးထားပါတယ်။ ရှေ့မှာ ဖေါ်ပြခဲ့သလိုပဲ Reference Type ဟာ Fish ဖြစ်ပြီး Object Type ဟာ GoldenFish ဖြစ်နေမှာ ဖြစ်ပါတယ်။

Reference Variable တစ်ခုကနေ Invoke လုပ်လို့ရတဲ့ Method တွေကို Compile Time မှာ Reference Type ကို ကြည့်ပြီး ဆုံးဖြတ်ပါတယ်။ တကယ်တန်း အလုပ်လုပ်တဲ့အခါကျတော့ Runtime ရောက်တော့မှ Object Type ကို ကြည့်ပြီး ဆုံးဖြတ်တာဖြစ်ပါတယ်။

ဒါ့ကြောင့် Reference Type ဟာ Fish ဖြစ်ပေမဲ့ တကယ်အလုပ်လုပ်သွားတာကတော့ Override လုပ်ထားတဲ့ GoldenFish ရဲ့ swim() method ပဲ ဖြစ်ပါတယ်။ Super Class ထဲက swim() method ကို အသုံးမပြုပဲ Sub Class ထဲမှာ ပြုပြင်ထားတဲ့ Method ကိုပဲ အသုံးပြုသွားတယ်ဆိုတာကို သိနိုင်ပါတယ်။

Override ဆိုတာက Inheritance လုပ်ထားတဲ့ Instance Method တွေကို Sub Class ထဲကနေ အသုံးမပြုလိုတဲ့ အခါမှာ ထပ်ရေးထားတဲ့ Method တွေကို ဆိုလိုခြင်းဖြစ်ပါတယ်။ ဒါ့ကြောင့် Static Method တွေ Variables တွေမှာ Override ဆိုတာ မရှိပါဘူး။ Inheritance မဖြစ်တဲ့ နေရာတွေမှာလဲ Override ဆိုတာ မရှိနိုင်ပါဘူး။

Many Forms

Method Override ကြောင့် Super Class ထဲမှာ သတ်မှတ်ထားတဲ့ Behavior နဲ့ မတူတဲ့ Sub Class တွေကို ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။ တနည်းဆိုရမယ်ဆိုရင် General ကျတဲ့ Behavior တွေကို ပိုပြီး Specialize ကျတဲ့ Behavior တွေကို ရေးသားအသုံးပြုလိုတဲ့ အခါမျိုးမှာ Method Override ကို အသုံးပြုနိုင်ပါတယ်။

```
class Bird {
      public void fly() {
            System.out.println("Bird is flying!");
      }
}

class Eagle extends Bird {
      public void fly() {
            System.out.println("Eagle is flying!");
      }
}

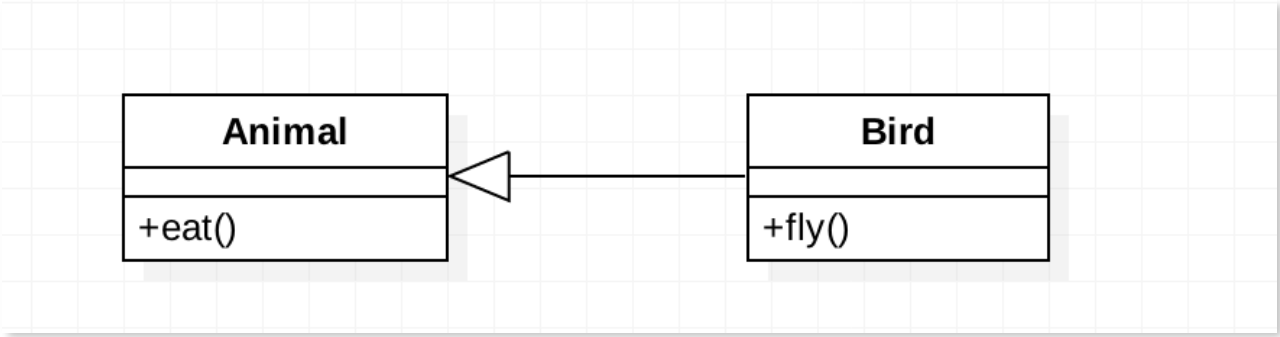
class Craw extends Bird {
      public void fly() {
            System.out.println("Craw is flying!");
      }
}
```

အထက်ပါ နမူနာထဲမှာ Bird Class ဟာ fly behavior ကို ပိုင်ဆိုင်ပါတယ်။ ပြီးတော့ Eagle နဲ့ Craw Class တွေဟာ Bird ကို Extends လုပ်ထားပြီး fly() method ကို Override လုပ်ထားပါတယ်။ ဒါ့ကြောင့် သူတို့ဟာ Bird တွေဖြစ်တဲ့အတွက် fly ဆိုတဲ့ Behavior ကို ပိုင်ဆိုင်ကြပါတယ်။ ဒါပေမဲ့ fly() method ကို Override လုပ်ထားခြင်းအား

ဖြင့် Bird နဲ့ မတူတဲ့ ပုံသန်းမှုမျိုးကို ပြုလုပ်နိုင်မှာ ဖြစ်ပါတယ်။ အမျိုးအစားတူသော်လဲ အပြုအမူမတူတဲ့ Object တွေကို တည်ဆောက်နိုင်ခြင်းကိုလဲ Polymorphism လို့ခေါ်ပါတယ်။

Type Casting

Many Form နဲ့ ပတ်သက်ပြီး Declare လုပ်ထားတဲ့ Reference Type နဲ့ လုပ်လို့ရတဲ့ Behavior တွေနဲ့ ပတ်သက်ပြီး လေ့လာကြည့် ကြရအောင်။



အထက်ပါ Class Diagram ထဲမှာ Animal ဆိုတဲ့ Class ရှိပြီး eat() ဆိုတဲ့ Method တစ်ခုကို ပိုင်ပါတယ်။ ပြီးတော့ Bird က Animal ကို Extends လုပ်ထားပြီး fly() ဆိုတဲ့ Method တစ်ခုကိုလဲ ရေးထားပါတယ်။ ဒါဆိုရင် Bird IS A Animal ဖြစ်ပြီး Bird ထဲမှာ Animal ကနေ Inheritance လုပ်လို့ရလာတဲ့ eat() method ကိုကော fly() method ကိုပါ ပိုင်ဆိုင်နေပါတယ်။

Java မှာက Object တစ်ခုကနေ Invoke လုပ်လို့ရတဲ့ Method တွေကို Reference Type အပေါ်မှာ မူတည်ပြီး ဆုံးဖြတ်ပါတယ်။ Bird Object ကို Bird Reference ကနေလဲ Reference လုပ်နေနိုင်သလို Animal Reference ကနေလဲ Reference လုပ်နေနိုင်ပါတယ်။

```
    public static void main(String [] args) {

        Bird bird1 = new Bird();
        Animal bird2 = bird1;

        bird1.eat();
        bird1.fly();

        bird2.eat();
    }
```

အထက်ပါ ကုဒ်ထဲမှာဆိုရင် new Bird() ဆိုတာကို တစ်ကြိမ်သာ ရေးသားထားတဲ့ အတွက် Object ကတော့ တစ်ခုပဲ ရှိပါတယ်။ အဲ့ဒီ Object ကို Bird bird1 Reference ကနေ Reference လုပ်ထားပါတယ်။ ပြီးတော့ Animal bird2 = bird1 လို့ရေးထားပြန်တဲ့ အတွက် bird1 ကနေ Reference လုပ်နေတဲ့ Object ကိုပဲ Animal Type Reference ဖြစ်တဲ့ bird2 ကနေလဲ Reference လုပ်နေပြန်တယ်။

တကယ်ရှိတာကတော့ Bird Object တစ်ခုထဲပါ။ Bird Type Reference ကကော Animal Type Reference ကပါ Reference ကပါ Reference လုပ်နေပါတယ်။ တကယ်တော့ Object ရဲ့ Type ဟာ Bird ဖြစ်တဲ့ အတွက် eat() method ကော fly() method ကိုပါ Invoke လုပ်လို့ရသင့်ပါတယ်။

ဒါပေမဲ့ Java မှာက Invoke လုပ်လို့ရတဲ့ Method တွေကို Reference Type ကို ကြည့်ပြီး ဆုံးဖြတ်တာ ဖြစ်တဲ့အတွက် bird1 ကဆိုရင် eat() ကော fly() ကိုပါ အသုံးပြုနိုင်မှာ ဖြစ်ပြီး Animal Type ဖြစ်တဲ့ bird2 ကနေဆိုရင် eat() method ကိုသာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Java မှာက Variable တစ်ခုကို Declare လုပ်လိုက်တဲ့ အခါမှာ Type ကို သတ်မှတ်ပေးရန်လိုအပ်ပြီး အဲ့ဒီ Type ဟာလဲ Declare လုပ်ပြီးတဲ့ နောက်မှာ ပြောင်းလို့ မရတော့ပါဘူး။

```
public static void main(String [] args) {

      Animal animal = new Bird();

      // if we need Bird behavior
      Bird bird = (Bird)animal;

      bird.fly();
}
```

အထက်နမူနာထဲမှာလို animal ဟာ Animal Type အဖြစ် Declare လုပ်လိုက်တဲ့ အတွက် နောက်ကွယ်မှာ Bird Object ကို Reference လုပ်နေပေမဲ့ Animal အနေနဲ့ပဲအလုပ်လုပ်ပါလိမ့်မယ်။ Bird အဖြစ်ပြောင်းလို့မရတော့ပါဘူး။ ဒါပေမဲ့ Bird Behavior ကို အသုံးပြုချင်တဲ့အခါမျိုးဆိုရင် Bird Type bird variable ကို အသစ်ထပ်ပြီး Declare လုပ်ရပါမယ်။

အဲ့ဒီနေရာကို animal variable ကို လာပြီး အစားထိုးရပါတော့မယ်။ ဒီနေရာမှာ animal ဟာ Bird Object ကို Reference လုပ်နေတာမှန်ပေမဲ့ Referenc Type ဟာ Animal ဖြစ်ပါတယ်။ Bird ဟာ Animal ကို Extends လုပ်ထားတဲ့ အတွက် Bird တွေဟာ Animal ဖြစ်ပေမဲ့ Animal တိုင်းဟာ Bird မဖြစ်နိုင်ပါဘူး။ ဒါ့ကြောင့် animal Variable ကို Bird နေရာမှာ ဒီအတိုင်း အစားထိုးလို့ မရနိုင်ပါဘူး။

(Bird)animal ဆိုပြီး Animal Type ကို Bird အဖြစ်ပုံစံပြောင်းပေးဖို့လိုအပ်ပါတယ်။ ဒါဆိုရင် Compiler ကနေ Bird အဖြစ်လက်ခံသွားပါမယ်။ ဒီလို ရေးသားခြင်းကို Type Casting လို့ခေါ်ပါတယ်။ Type Casting ကို ရေးသားရာမှာ Inheritance Relationship ရှိတဲ့ Class တွေ အချင်းချင်းမှသာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ဘာမှ မဆိုင်တဲ့ Class တွေဆိုရင် Cast လုပ်လို့ရမှာမဟုတ်ပါဘူး။

```
public static void main(String [] args) {

      Animal animal = new Animal();

      // if we need Bird behavior
      Bird bird = (Bird)animal;

      bird.fly();
}
```

အထက်ပါအတိုင်း Animal Object ကို Bird အဖြစ် Cast လုပ်လို့ရလားဆိုတော့ Compiler ကတော့ လက်ခံသွားပါတယ်။ Compiler Error ဖြစ်မှာ မဟုတ်ပါဘူး။ ဒါပေမဲ့ Runtime ရောက်တဲ့ အခါကျမှ Reference လုပ်နေတဲ့ Object က Bird မဟုတ်ပဲ Animal ဖြစ်နေတဲ့ အတွက် ClassCastException ကို ဖြစ်ပေါ်စေနိုင်ပါတယ်။

ဒါ့ကြောင့် Class Casting တို့ကိုပြုလုပ်တော့မယ်ဆိုရင် if statement ကို အသုံးပြုပြီး animal ဟာ Bird ရဲ့ Instance ဟုတ်မဟုတ်စစ်ဆေးပြီးမှ Casting ကို ပြုလုပ်သင့်ပါတယ်။

မှန်ကန်သော Override Rules များ

Method တွေကို Override လုပ်ခြင်းအားဖြင့် Polymorphic ဖြစ်အောင် ရေးသားနိုင်တယ်ဆိုတာကို လေ့လာပြီးတဲ့ နောက်မှာ Method တွေကို Override လုပ်တဲ့နေရာမှာ ဘယ်လိုရေးရင်မှန်တယ် ဘယ်လိုရေးရင်မှားတယ်ဆိုတာကို ဆက်လက်ပြီး လေ့လာသွားကြရအောင်။

Override ဆိုတာ Inheritance မရှိပဲ မဖြစ်နိုင်ပါဘူး။

Super Class ထဲမှာရေးသားထားတဲ့ Method တွေကို Sub Class ထဲကနေ မြင်မနေရင် Override ဆိုတာရှိလာမှာ မဟုတ်ပါဘူး။ ဆိုလိုတာက Private Method တွေ ပြီးတော့ Package မတူတဲ့နေရာကနေ Inheritance လုပ်ထားရင် Default Access Method တွေဆိုရင် Sub Class ကနေ မြင်မှာမဟုတ်ဘူး။ မမြင်တဲ့ အရာတွေကို မျိုးရိုးဆက်ခံပိုင်ခွင့် မရှိသလို Override လဲ လုပ်လို့ရမှာ မဟုတ်ပါဘူး။

Super Class ထဲမှာ ရေးထားတဲ့ Method နဲ့ Sub Class ထဲမှာ ရေးထားတဲ့ Method Signature တွေဟာ ထပ်တူညီနေရပါမယ်။

Method Signature ဆိုတာက Method Name နဲ့ Argument List ကို ပေါင်းပြီးခေါ်ဆိုကြတာပါ။ Override လုပ်မယ်ဆိုရင် Super Class ထဲမှာရေးသားထားတဲ့ Method နဲ့ Sub Class ထဲက Method တွေဟာ Signature ချင်း တူနေမှရပါမယ်။ JVM ကနေ Method တွေကို Invoke လုပ်တဲ့အခါမှာ Method Signature ကို ကြည့်ပြီး Invoke လုပ်ရတာဖြစ်ပါတယ်။ Method Signature မတူတာနဲ့ နောက်ထပ် Method တစ်ခုလို့ မှတ်ယူပါတယ်။ ဒါ့ကြောင့် Method Name တော့တူပြီး Argument မတူရင် Override ဖြစ်မှာ မဟုတ်ပါဘူး။ Overload လုပ်တာပဲ ဖြစ်မှာပါ။ Method Overload အကြောင်းကိုတော့ နောက်မှဆက်ပြောပါမယ်။

Access Modifier တွေဟာ Super Class ထက် ကျဉ်းမြောင်းနေလို့မရပါဘူး

Method တစ်ခုရဲ့ Access Modifier ဆိုတာက Method ကို ဘယ်ကနေ သုံးခွင့်ရှိလဲဆိုတာကို သတ်မှတ်ပေးနိုင်တဲ့ အရာတွေပါ။ Super Class ရဲ့ Method ကိုတော့ Public Access ပေးထားပြီး Override လုပ်တဲ့ Method မှာတော့ ပိုပြီးကျဉ်းတဲ့ Default တို့ Protected တို့ကို သတ်မှတ်လို့မရပါဘူး။

အဲ့ဒါဟာ ဘာကြောင့်လဲဆိုရင် Is A Relationship ကြောင့် ဖြစ်ပါတယ်။ Super Class မှာတော့ အသုံးပြုနိုင်တယ်လို့သတ်မှတ်ထားပြီး သူ့နေရာမှာလာပြီး Reference လုပ်ခံရမည့် Sub Class မှာ သုံးလို့မရဘူးလို့ သတ်မှတ်ထားတာဟာ မဖြစ်နိုင်တဲ့ အတွက် Compile Time မှာ Error ဖြစ်မှာ ဖြစ်ပါတယ်။

ဒါ့ကြောင့် Access Modifier တွေကို ရေးသားမယ်ဆိုရင် Super Class ထဲမှာ ရေးထားတာနဲ့ ထပ်တူ ဒါမှမဟုတ် ပိုပြီး ဘောင်ကျယ်တဲ့ Access Modifier ကို အသုံးပြုရပါမယ်။ Super Class ထဲမှာ Protected လို့ရေးထားတာကို Override လုပ်ထားတဲ့ Sub Class ထဲမှာ Public လို့ရေးထားနိုင်ပါတယ်။

Return Type တွေဟာလဲ Super Class မှာ ရေးထားတဲ့ Return Type နဲ့ သူ့ရဲ့ Sub Type တွေပဲဖြစ်ရပါမယ်

Java SE 5 မတိုင်ခင် အထိကတော့ Override လုပ်မယ်ဆိုရင် Super Class မှာ ရေးထားတဲ့ Return Type နဲ့ Sub Class ထဲမှာ ရေးထားတဲ့ Return Type တွေဟာ တူညီနေဖို့လိုအပ်ပါတယ်။ ဒါပေမဲ့ Java SE 5 မှာ Covarient Return ဆိုတာကို ထည့်သွင်းလာပြီး Override မှာ Sub Class တွေရဲ့ Return Type ဟာ Super Class ရဲ့ Type ဒါမှမဟုတ် Sub Type တွေကိုလဲ Return လုပ်ခွင့်ရလာပါတယ်။

Sub Class တွေရဲ့ Object ကို Super Class ရဲ့ Reference တွေကနေ အသုံးပြုမယ်ဆိုရင် Super Class ရဲ့ Method ကို Invoke လုပ်မှာ ဖြစ်ပါတယ်။ အဲ့ဒီတော့ Super Class ရဲ့ Return Type ကို ပြန်ရမယ်ဆိုပြီး Invoke လုပ်တဲ့ သူက စောင့်နေမှာ ဖြစ်ပါတယ်။ တကယ် Runtime ကျတော့ Object Type ဖြစ်တဲ့ Sub Class ကနေ အလုပ်လုပ်မှာ ဖြစ်ပြီး Super Class က Return ပြန်တဲ့ Type ကို မပြန်မိရင် Invoke လုပ်တဲ့ သူမှာ ပြဿနာရှိသွားနိုင်ပါတယ်။ IS A Relationship ရှိတဲ့ Type တွေကို Return ပြန်ရင်တော့ သူလဲ Return Type အနေနဲ့ အသုံးပြုလို့ရတာ ဖြစ်တဲ့ အတွက် ပြဿနာရှိမှာ မဟုတ်ပါဘူး။

ဆိုကြပါစို့ ဈေးဝယ်တဲ့ Method တစ်ခုရဲ့ Return Type ဟာ ငါး ဖြစ်နေပြီး Sub Class ကနေ Return ပြန်တာက ကြက်သားဖြစ်နေရင် အဆင်မပြေပါဘူး။ ငါးကြင်းတို့ ငါးမြင်းတို့ကို Return ပြန်ရင်တော့ ငါးဖြစ်တဲ့ အတွက် ပြဿနာရှိမှာ မဟုတ်ပါဘူး။

Exception တွေနဲ့ ပတ်သက်ပြီး

Method တွေဟာ Exception တွေကို Throw လုပ်လို့ရပါတယ်။ Exception တွေကို Throw လုပ်တဲ့နေရာမှာ Unchecked Exception တွေဖြစ်တဲ့ RuntimeException တွေဆိုရင် ပြဿနာမရှိပါဘူး။ Super Class ရဲ့ Method ထဲမှာ Exception ကို Throw မလုပ်ထားရင်တောင်မှ Sub Class ထဲက Throw လုပ်တဲ့ Exception ဟာ Unchecked Exception ဖြစ်နေမယ်ဆိုရင် ပြဿနာမရှိပါဘူး။

ဘာလို့လဲဆိုတော့ Unchecked Exception တွေဟာ မဖြစ်မနေ Handle လုပ်ရမည့် Exception တွေမဟုတ်ပါဘူး။ ဒါ့ကြောင့် Sub Class ကနေ Throw လုပ်တဲ့ Exception ဟာ Unchecked Exception တွေဆိုရင်တော့ Invoke လုပ်တဲ့ ဘက်ကိုဘာမှ အထွေအထူးပြုပြင်စရာမလိုအပ်ပါဘူး။ ဒါ့ကြောင့် Unchecked Excpetion တွေဆိုရင်တော့ Sub Class ထဲကနေ ကြိုက်တာကို Throw လုပ်နိုင်ပါတယ်။

ဒါပေမဲ့ Checked Exception တွေဆိုရင်တော့ မတူပါဘူး။ Super Class ထဲမှာ Throw မလုပ်ထားတဲ့ Exception ကို Sub Class ထဲကနေ Throw လုပ်လို့မရပါဘူး။ နောက်ပြီး Super Class ထဲက Throw လုပ်ထားတဲ့ Checked Exception နဲ့ အမျိုးအစားမတူတဲ့ Exception တွေကိုလဲ Sub Class ထဲက နေ Throw လုပ်လို့မရပါဘူး။

Super Class ထဲမှာ Checked Exception ကို Throw လုပ်ထားတယ်။ Sub Class ထဲကနေ Exception ကို Throw မလုပ်ဘူး ဒါကရပါတယ်။ ပြီးတော့ Super Class ထဲက Throw လုပ်ထားတဲ့ Exception နဲ့ ထပ်တူ ဒါမှမဟုတ် Sub Exception ကို Throw လုပ်မယ်ဆိုရင်လဲရပါတယ်။ Super Class ထဲက Throw လုပ်ထားတဲ့ Exception ထက်ကြီးတဲ့ Exception တွေပြီးတော့ အမျိုးအစားမတူတဲ့ Exception တွေကို Throw လုပ်လို့မရပါဘူး။

Override နှင့် Overload

ဒီနေရာမှာ တဆက်ထဲ ဆက်ပြီး ပြောချင်တာက Overload ဖြစ်ပါတယ်။ Overload ဆိုတာက OOP နဲ့တော့ မဆိုင်ပါဘူး။ Override နဲ့ နာမည်ချင်းဆင်တဲ့အတွက် အမှတ်မှားတတ်ပါတယ်။ ဒါ့ကြောင့် ဘယ်လိုကွာခြားတယ်ဆိုတာကို လေ့လာသွားကြပါမယ်။

Overload

Overload ဆိုတာက နာမည်တူပြီး Method Signature မတူတဲ့ Method တွေကို Class တစ်ခုထဲမှာ ရေးသားချင်းကို Overload လုပ်တယ်လို့ခေါ်ပါတယ်။ နာမည်တူပြီး Argument မတူတဲ့ Method တွေကို ရေးသားခြင်းအားဖြင့် တစ်နေနေရာရာမှာ တစ်ခြား Method တစ်ခုကို အသုံးပြုဖူးခဲ့ရင့် နောက် Method တစ်ခုလဲ ဒီလို အလုပ်လုပ်မှာပါလားဆိုပြီး ခန့်မှန်းနိုင်မှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် Java API တွေရဲ့ Class တွေထဲမှာ Overload လုပ်ပြီး ရေးသားထားတဲ့ Method တွေကို အများအပြားတွေ့နိုင်မှာ ဖြစ်ပါတယ်။

```
public class NoteManager {

      public viod takeNoteByPaper(String message) {
            // codes
      }

      public void takeNoteByPhone(String message) {
            // codes
      }
}
```

အထက်ပါ အတိုင်း ရေးသားထားတဲ့ Method တွေဆိုရင် နာမည် တစ်မျိုးစီဖြစ်နေတဲ့ အတွက် Overload မဟုတ်ပါ။

```
public class NoteManager {

      public viod takeNote(String message, Paper paper) {
            // codes
      }

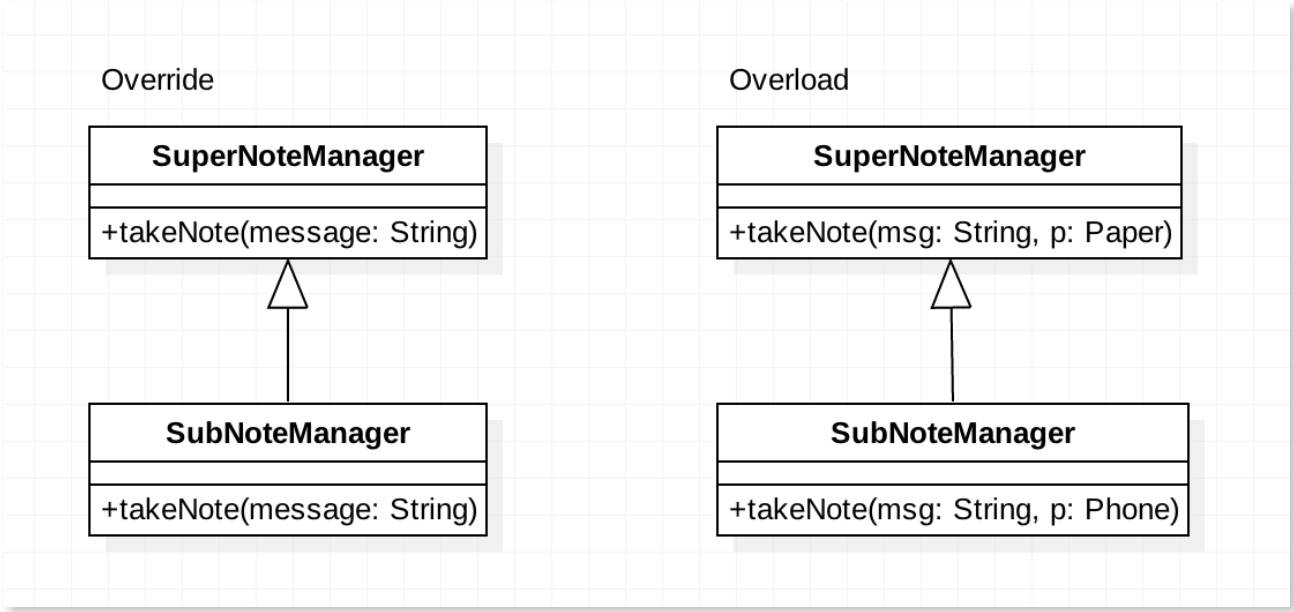
      public void takeNote(String message, Phone phone) {
            // codes
      }
}
```

အထက်ပါအတိုင်း နာမည်တူပြီး Argument မတူတဲ့ Method တွေကို ရေးလိုက်ပြီဆိုရင် Overload ဖြစ်သွားပါမယ်။ Overload ကို Instance Method တွေသာမက Static Method တွေမှာလဲရေးသား အသုံးပြုနိုင်ပါတယ်။ ပြီးတော့ Inheritance Hierarchy ထဲမှာလဲရေးသားလို့ရနိုင်ပါတယ်။ Method Name တူပြီး Argument မတူတဲ့ Method တွေကို ရေးသားထားခြင်းအားဖြင့် Method တစ်ခုခုကို အသုံးပြုဘူးရင် နောက် Method တွေလဲ ဒီလို အလုပ်လုပ်မှာပါလားဆိုပြီး ခန့်မှန်းလွယ်မှာ ဖြစ်ပါတယ်။

Override Vs Overload

Override နဲ့ Overload ဆိုတာဟာ နာမည်ချင်းလဲ ဆင်တူပါတယ်။ ပြီးတော့ Method တွေအပေါ်မှာ ဖြစ်ပေါ်တာ ချင်းလဲ တူညီတဲ့ အတွက် အမှတ်မှားလေ့ရှိပါတယ်။ ဒီနေရာမှာ Override နဲ့ Overload ကို ဘယ်လိုခွဲခြားရမယ်ဆိုတာကို ဖေါ်ပြသွားပါမယ်။

အရင်ဆုံး Method Signature ဆိုတာဘာလဲဆိုတာကို လေ့လာကြရအောင်။ Method Signature ဆိုတာက Compiler နဲ့ Runtime မှာ Method တစ်ခုဖြစ်တယ်ဆိုတာကို သတ်မှတ်နေတဲ့ အရာတစ်ခုဖြစ်ပါတယ်။ Method Signature ကို သတ်မှတ်တဲ့ နေရာမှာ Method Name နဲ့ Arguments တွေကို အသုံးပြုပါတယ်။ ဒါ့ကြောင့် Method Name နဲ့ Arguments တွေ တူတဲ့ Method တွေကို Method Signature တူတယ်လို့သတ်မှတ်ပါတယ်။ Method Argument တူတယ်ဆိုတာက Argument Type တွေတူတယ်ကို ပြောတာပါ။ Argument Name တွေကို မဆိုလိုပါဘူး။



Override ဆိုတာက Super Class ထဲမှာရှိတဲ့ Method နဲ့ Signature ချင်းတူတဲ့ Method တစ်ခုကို Sub Class ထဲမှာ ရေးသားထားတာကို ဆိုလိုပါတယ်။ တကယ်ဆိုရင် Method Signature တူတဲ့ Method တွေကို Class တစ်ခုထဲမှာ ရေးလို့မရပါဘူး။ ဒါပေမဲ့ Inheritance Hirerachy Tree ထဲမှာ ဆိုရင်တော့ ရေးလို့ရပါတယ်။ Method Signature ချင်းတူတဲ့ အတွက် Override လုပ်ထားတဲ့ Method တွေကို Object ဆီကနေ ကြည့်မယ်ဆိုရင် တစ်ခုထဲသော Method လို့မြင်ပါလိမ့်မယ်။

Overload ဆိုတာက Class တစ်ခု ဒါမှမဟုတ် Object တစ်ခုမှာ Method Name တူပြီး Argument မတူတဲ့ Method တွေကို ရေးသားထားခြင်းကို ဆိုလိုပါတယ်။ Method Signature ချင်း မတူတဲ့ အတွက် Overload Method တွေဟာ နာမည်သာတူတဲ့ သီးခြား Method တွေ ဖြစ်ကြပါတယ်။ Overload ဆိုတာက Class တစ်ခုထဲမှာလဲ ရေးသားနိုင်သလို Inheritance လုပ်လို့ရလာတဲ့ Method တွေကြားမှာလဲ ဖြစ်နိုင်ပါတယ်။ Compiler နဲ့ Runtime ဘက်က ကြည့်မယ်ဆိုရင် Overload Method တွေကို မတူတဲ့ Method တွေလို့ မြင်ပါလိမ့်မယ်။

ဒါ့ကြောင့် Override Method တွေဟာ Same Method တွေဖြစ်ကြပေမဲ့ Overload Method တွေဟာ Different Method တွေဖြစ်ကြပါတယ်။ တဖန် Override Method တွေဟာ Inheritance ရှိတဲ့ Instance Method တွေကြားမှာသာ ဖြစ်ပွါးလေ့ရှိပေမဲ့၊ Overload ကတော့ Instance Method ကြားမှာကော Static Method တွေကြားမှာပါ ဖြစ်ပွါးလေ့ရှိပါတယ်။

အသေးစိတ်ကွာခြားချက်တွေကိုတော့ အောက်ပါ ဇယားမှာ လေ့လာနိုင်ပါတယ်။

အကြောင်းအရာ	Override	Overload
Method Name	တူညီရပါမယ်	တူညီရပါမယ်
Method Arguments	ထပ်တူညီမှ ရပါမယ်	လုံးဝ တူလို့မရပါဘူး
Return Type	Super Class ထဲက Return Type နဲ့ ထပ်တူဒါမှမဟုတ် Sub Type သာ ဖြစ်ရပါမယ်	မည်သည့် Return Type မျိုးကို မဆို အသုံးပြုနိုင်ပါတယ်
Access Modifier	Super Class ထဲက Access Modifier နဲ့ ထပ်တူ ဒါမှမဟုတ် ပိုပြီး ကျယ်ဝန်းတဲ့ Access Modifier ကို အသုံးပြုရပါမယ်	မည်သည့် Access Modifier မျိုးကို မဆို အသုံးပြုနိုင်ပါတယ်
Checked Exceptions	Super Class ထဲက Checked Exception နဲ့ ထပ်တူဒါမှမဟုတ် Sub Type သာ ဖြစ်ရပါမယ်	မည်သည့် Exception မျိုးကို မဆို အသုံးပြုနိုင်ပါတယ်
Inheritance Relation	Inheritance လုပ်ထားတဲ့ Class တွေ အကြားမှာသာ ဖြစ်ပေါ်ပါမယ်	Inheritance လုပ်ထားထား မထားထား ဖြစ်ပွါးလေ့ရှိပါတယ်
Method Type	Instance Method အမျိုးအစားမှာပဲ ဖြစ်ပေါ်ပါတယ်	Instance Method ကော Static Method မှာပါ ဖြစ်နိုင်ပါတယ်
Method Place	Super Class ထဲက Method နဲ့ Sub Class ထဲက Method တို့ အကြားမှာ ဖြစ်ပေါ်ပါတယ်	Class တစ်ခုထဲမှာ ရှိတဲ့ Method တွေ အကြားမှာလဲဖြစ်ပေါ်နိုင်သလို Super Class နဲ့ Sub Class ထဲမှာရှိတဲ့ Class တွေကြားမှာလဲ ဖြစ်ပေါ်နိုင်ပါတယ်

Polymorphism ရဲ့ အကျိုးများ

ဒီနေရာမှာတော့ Polymophism ရဲ့ အကျိုးများဆိုပေမဲ့ Polymorphism ရဲ့ အကျိုးတွေကို လေ့လာသွားမှာ ဖြစ်ပါတယ်။ ဒါတွေကို မလေ့လာခင် အရင်ဆုံး Polymorphism ကို ဘယ်လိုနေရာတွေမှာ ဖြစ်တာလဲ ဆိုတာကို ပြန်စဉ်းစားကြည့်ရအောင်။

ပထမ တစ်နေရာကတော့ Class တစ်ခုက အခြား Class တစ်ခုကို Extends လုပ်တဲ့အခါ ဒါမှမဟုတ် Interface တွေကို Implement လုပ်တဲ့ အခါမှာလဲ Object တစ်ခုကို ပုံစံအမျိုးမျိုးနဲ့ အသုံးပြုနိုင်တဲ့ အတွက် Polymorphism ကို ဖြစ်ပေါ်စေပါတယ်။ Super Class နဲ့ Sub Class ရဲ့ အကြားမှာ၊ ပြီးတော့ Interface နဲ့ Implement လုပ်ထားတဲ့ Class တွေအကြားမှာ IS A Relation ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

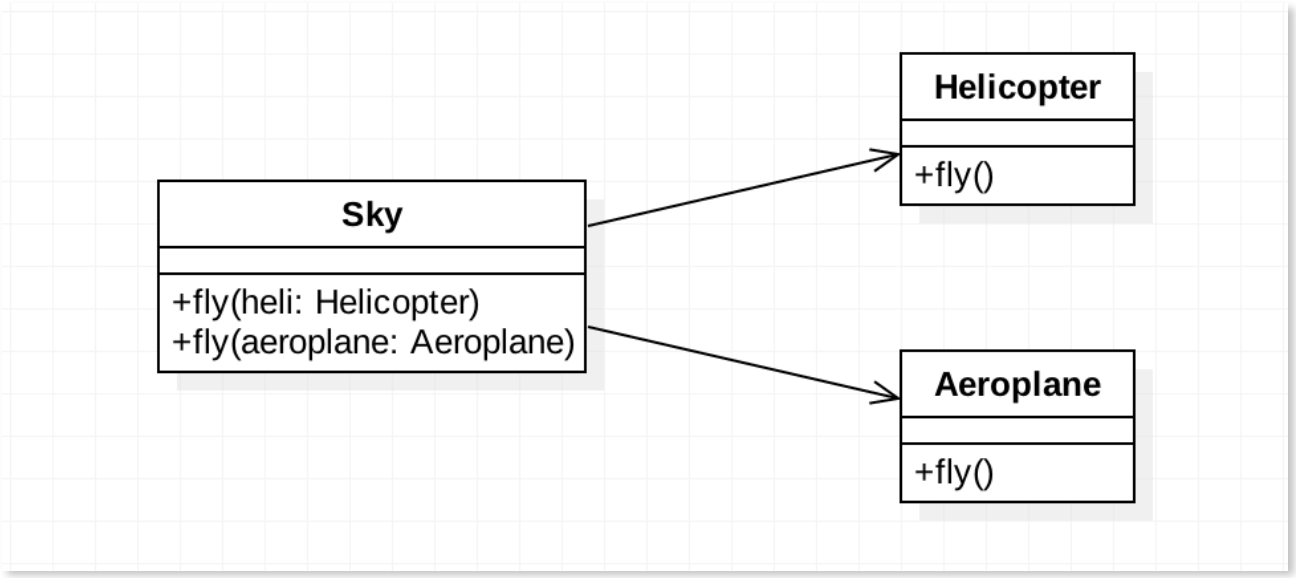
ဒုတိယ တစ်နေရာကတော့ Method Override လုပ်တဲ့အခါမှာ အမျိုးအစားတူပြီး ပုံစံမတူတဲ့ Object တွေကို ရေးသားလာနိုင်ပြီး Polymorphism ကို ဖြစ်စေနိုင်ပါတယ်။ ပြီးတော့ Method Override ဆိုတာကလဲ IS A Relationship ရှိတဲ့ နေရာတွေကျမှသာ ဖြစ်ပေါ်နိုင်ခြင်း ဖြစ်ပါတယ်။ ဒီအချက်တွေကို ကြည့်မယ်ဆိုရင် Polymorphism ဆိုတာ IS A Relationship ရှိတဲ့ နေရာတွေမှာ ဖြစ်ပေါ်စေတယ်ဆိုတာကို သိနိုင်ပါတယ်။

Inheritance ကြောင့် Code တွေကို Reuse လုပ်နိုင်သလို အုပ်စုတူတဲ့ Object တွေ အဖြစ်ရေးနိုင်တယ်လို့ ပြောခဲ့ပါတယ်။ တကယ်တော့ အုပ်စုတူတဲ့ Object တွေကို ရေးသားခြင်းဆိုတာက Polymorphic ဖြစ်တဲ့ ကုဒ်တွေကို ရေးသားခြင်းပါပဲ။ Polymophic ဖြစ်တဲ့ ကုဒ်တွေကို ဘယ်လိုသုံးမလဲ။ သုံးတဲ့ အတွက် ဘယ်လိုအကျိုးရှိလဲ ဆိုတာကို လေ့လာကြည့်ကြရအောင်။ Polymorphic Code တွေကို အခြေခံအားဖြင့် ၃ နေရာမှာ အသုံးပြုနိုင်ပါတယ်။

- Method Argument တွေအနေနဲ့ အသုံးပြုနိုင်ပါတယ်။
- Method Return တွေ အနေနဲ့ အသုံးပြုနိုင်ပါတယ်။
- Instance Variable တွေနေရာမှာ အသုံးပြုနိုင်ပါတယ်။

Method Argument အနေနဲ့ အသုံးပြုမယ်ဆိုရင်

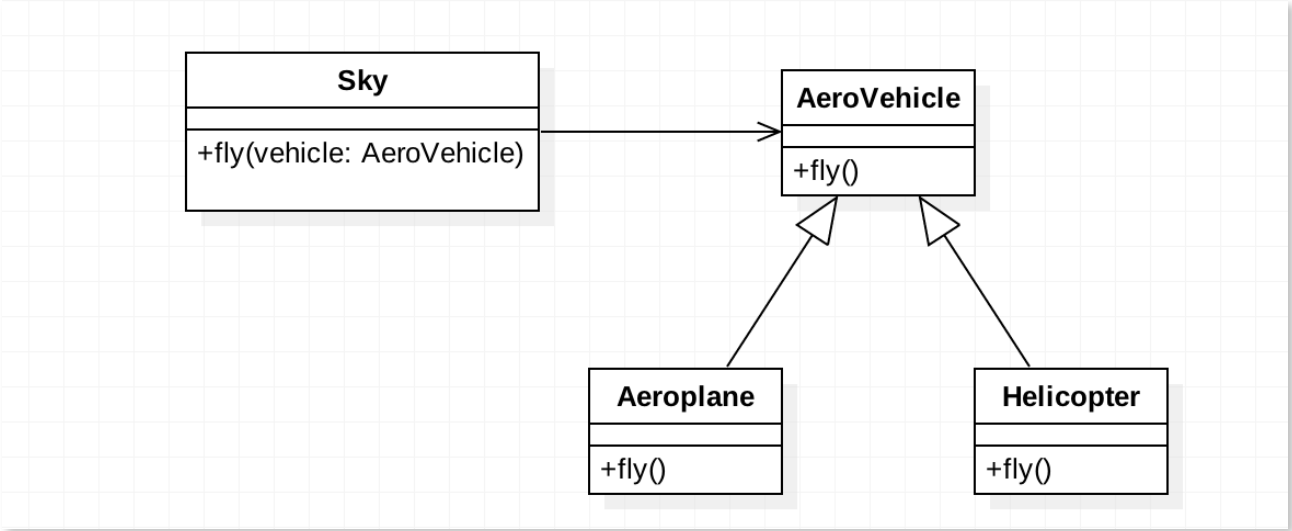
ကျွန်တော်တို့ Game တစ်ခုကို ရေးနေကြတယ်ဆိုကြပါစို့။ လေယဉ်ပျံတွေ ဟယ်လီကောပတာတွေ ပါမယ်။ ကောင်းကင်လဲရှိမယ်။ အဲ့ဒီကောင်းကင်ထဲကို လေယဉ်ပျံတွေ ဟယ်လီကောပတာ တွေကို ပျံခိုင်းတဲ့ Method ကို ရေးနေကြတယ်ဆိုကြပါစို့။ အောက်ပါ အတိုင်း Class Design ကို ရေးမိပါမယ်။



အထက်ပါအတိုင်း Helicopter ကို ပျံခိုင်းဖို့ Method တစ်ခု Airplane ကို ပျံခိုင်းဖို့ Method တစ်ခု ရေးရပါမယ်။

ဒီအထိတော့ပြဿနာမဟုတ်သေးပါဘူး။ ဒါပေမဲ့ ဒီနေရာမှာ Drone တွေကိုလဲ Game ထဲမှာ သုံးချင်လာပြီ ဆိုကြပါစို့။ ကျွန်တော်တို့တွေ Drone အတွက် Method တစ်ခုကို Sky ထဲမှာ လာရေးရပါတော့မယ်။ ဒီလိုဆိုရင် Sky ကို အသုံးပြုနေတဲ့ Class တွေ အတွက် ပြင်ဖို့လိုချင်လိုလာနိုင်ပါတယ်။ မပြင်ရရင်တောင် Sky ကို ပြင်လိုက်တဲ့ အတွက် ဘေးထွက်ဆိုးကျိုးတွေမရှိဘူးဆိုတာကို သက်သေပြဖို့အတွက် Test တွေကို ပြန်လုပ်ဖို့လိုအပ်လာပါလိမ့်မယ်။

ဒါဆိုရင် Sky ရဲ့ fly method ကို Polymorphic ကိုလက်ခံတဲ့ Method ဖြစ်အောင် ပြောင်းကြည့်ပါမယ်။



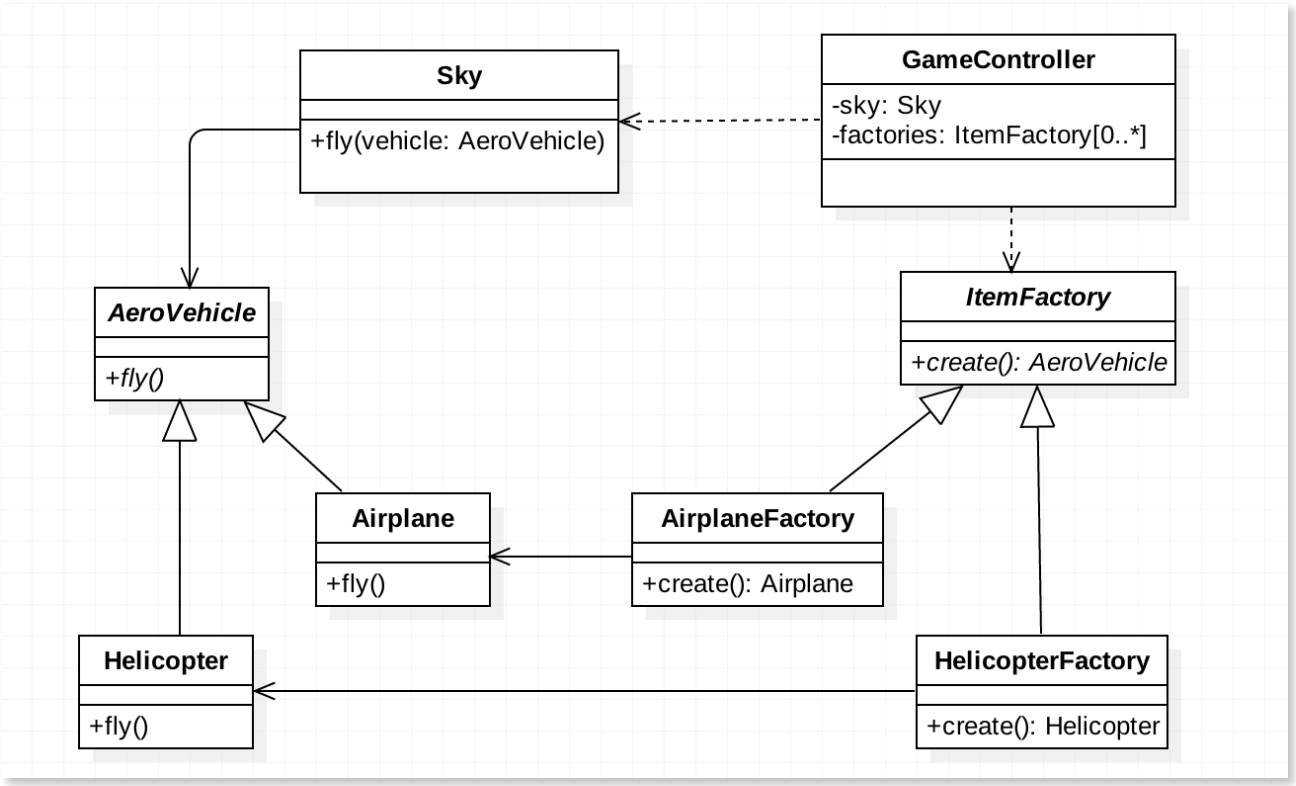
Fly Method ကိုပိုင်တဲ့ AeroVehicle ကို တည်ဆောက်ပြီး Airplane နဲ့ Helicopter တို့ကို Extends လုပ်ခိုင်းပါတယ်။ အဲ့ဒီအတွက် Airplane နဲ့ Helicopter တွေဟာ AeroVehicle တွေ ဖြစ်သွားကြပါတယ်။ ပြီးတဲ့နောက်မှာ Sky Class ရဲ့ fly method ကိုလဲ AeroVehicle ကို Argument အဖြစ်ယူခိုင်းလိုက်ပါတယ်။ အဲ့ဒီအတွက် fly method တစ်ခုထဲနဲ့ Airplane ကော Helicopter ကိုပါ အသုံးပြုလို့ရလာနိုင်ပါတယ်။

ဒါ့ကြောင့် Drone ကို ထပ်ပြီး ထည့်ချင်လာတယ်ဆိုရင်လဲ AeroVehicle ကို Extends လုပ်ပြီး Drone Class ကို ရေးသားလိုက်ရုံပါပဲ။ Drone ဟာလဲ AeroVehicle ဖြစ်တဲ့အတွက် Sky ရဲ့ fly Method ကိုဆက်ပြီး အသုံးပြုရုံပါပဲ။ Sky ကို ပြင်စရာမလိုအပ်ပါဘူ။ အဲ့ဒီအတွက် Sky ကို အသုံးပြုတဲ့ Class တွေမှာလဲ အကျိုးသက်ရောက်မှုတွေ ရှိမှာ မဟုတ်ပါဘူး။

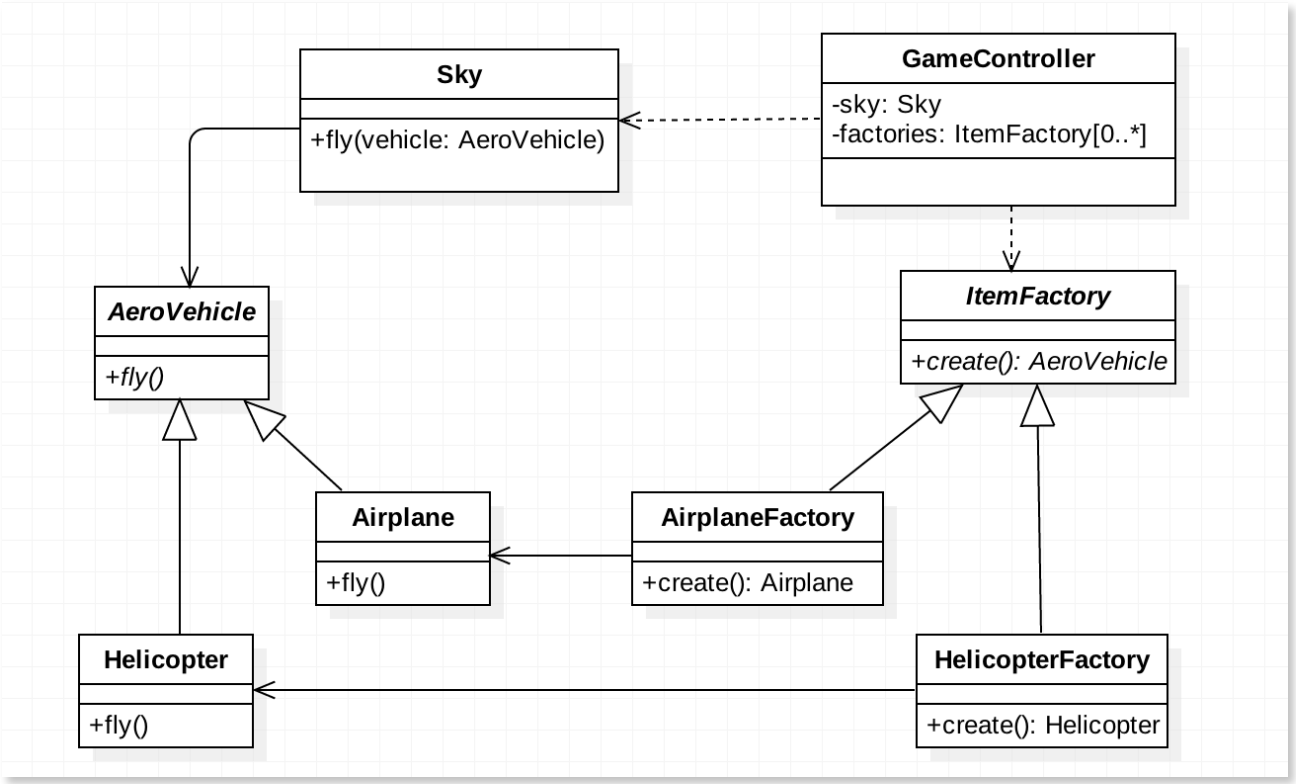
ဒါ့ကြောင့် POLYMORPHIC CODE တွေကို METHOD ARGUMENT တွေ အနေနဲ့ အသုံးပြုနိုင်ပြီး၊ APPLICATION အတွက် FEATURE အသစ်တွေ ထည့်ချင်လာပြီဆိုရင်လဲ အလွယ်တကူ ဆောင်ရွက်နိုင်မှာ ဖြစ်ပါတယ်။ အလွယ်ပြောရမယ်ဆိုရင် POLYMORPHIC CODE တွေကို အသုံးပြုခြင်းအားဖြင့် APPLICATION တွေကို ENHANCE လုပ်ရတာလွယ်ကူစေပါတယ်။

Method Return Type တွေအနေနဲ့ အသုံးပြုရင် Instance Variable တွေ အနေနဲ့ အသုံးပြုမယ်ဆိုရင်

အရင်ဆုံး Method Return Type တွေ Instance Variable တွေနေရာမှာ Polymorphic Code တွေကို မသုံးပဲ ရေးကြည့်ရင် ဘယ်လို ဖြစ်မယ်ဆိုတာကို ကြည့်ကြည့်ကြရအောင်။
ရှေ့မှာ အသုံးပြုခဲ့တဲ့ နမူနာကိုပဲ ပြန်ပြီး အသုံးပြုပါမယ်။ AirplaneFactory တစ်ခုနဲ့ HelicopterFactory Class နှစ်ခုရှိတာကို သတိပြုမိမှာပါ။ သူတို့တွေက အသီးသီး Airplane နဲ့ Helicopter တွေကို Create လုပ်ပေးနိုင်ပါတယ်။ သူတို့ကို GameController ထဲမှာ Instance Variable အနေနဲ့ အသုံးပြုထားပါတယ်။ ပြီးတော့ GameController ကနေ Sky ကိုလဲ Instance Variable အနေနဲ့ အသုံးပြုထားပါတယ်။



GameController ထဲမှာ play() ဆိုတဲ့ Method ပါဝင်ပြီး အဲ့ဒီအထဲမှာ AirplaneFactory ထဲကနေ Airplane ကို Create လုပ်ပြီး Sky ထဲမှာ ပျံခိုင်းမယ်၊ ပြီးတော့ HelicopterFactory ထဲကနေ Helicopter ကို Create လုပ်ပြီး Sky ထဲမှာ ပျံခိုင်းပါမယ်။



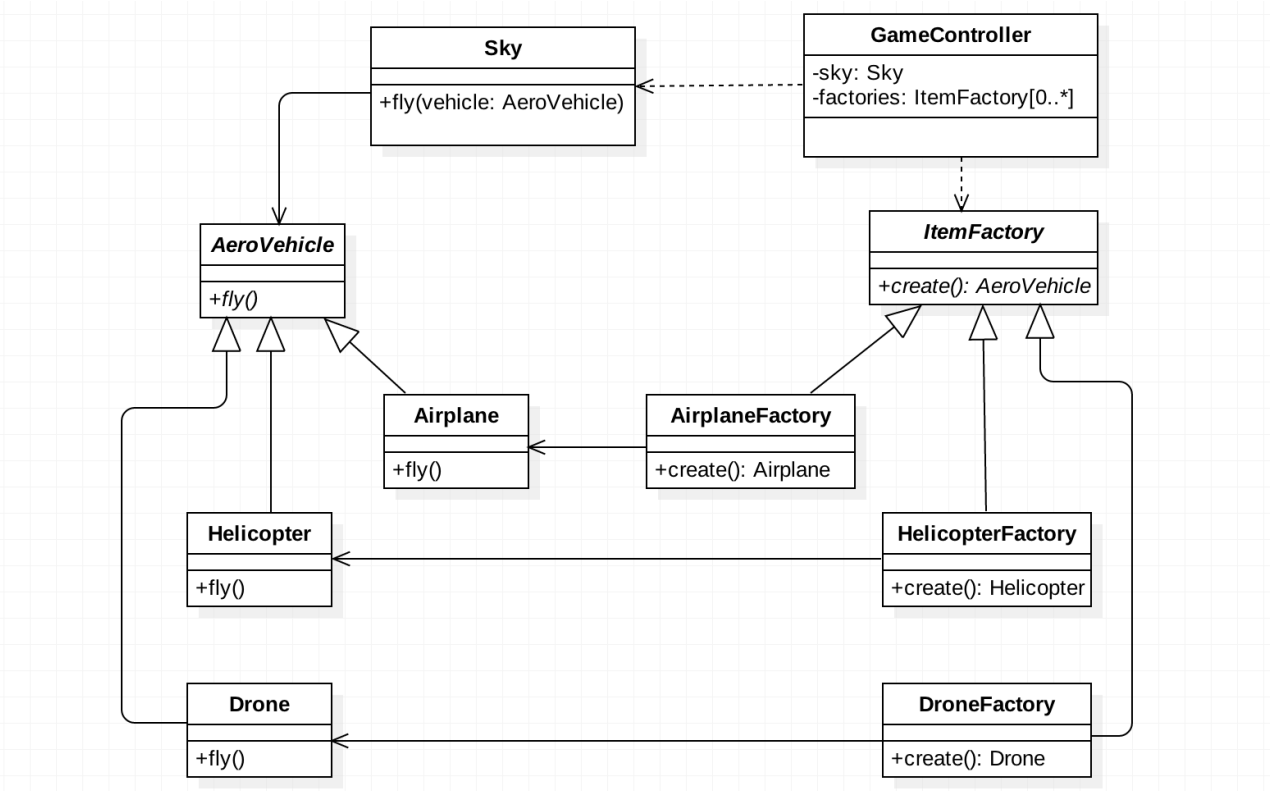
ဒီနေရာမျိုးမှာလဲ Item အသစ်ကို ဖြည့်စွက်လိုပါက Item တစ်ခု ItemFactory တစ်ခုနဲ့ Controller ကို ပြင်ဖို့လိုနေမှာပါပဲ။ Class တွေရဲ့ Design ကို အထက်ပါအတိုင်း အတိုင်းပြင်ကြည့်ပါမယ်။

ItemFactory ဆိုတဲ့ Abstract Class တစ်ခုကို ထပ်ပြီးတိုးပါတယ်။ ပြီးတော့ ItemFactory ထဲမှာ AeroVehicle ကို Return ပြန်တဲ့ Abstract Method တစ်ခုပါဝင်ပါတယ်။ AeroVehicle ဆိုတာက Polymorphic Type ကို အသုံးပြုထားတဲ့ Type တစ်ခုဖြစ်ပါတယ်။ အဲ့ဒီအတွက် create Method ကနေ Airplane ကိုလဲ Return ပြန်လို့ ရသလို Helicopter ကိုလဲ Return ပြန်လို့ရမှာပါ။

ပြီးတော့မှ AirplaneFactory နဲ့ HelicopterFactory Class နှစ်ခုကို ItemFactory ကို Inheritance လုပ်ခိုင်းပါတယ်။ အဲ့ဒီအတွက် create() Method ကို Override လုပ်ဖို့လိုအပ်လာပါတယ်။ အသီးသီး Override လုပ်ပြီး Airplane နဲ့ Helicopter ကို Return ပြန်ပေးပါတယ်။ Airplane ကော Helicopter ပါ AeroVehicle ကို Extends လုပ်ထားတဲ့အတွက် AeroVehicle အဖြစ် Return ပြန်လို့ရပါတယ်။

ဤနည်းအားဖြင့် ItemFactory ရဲ့ create Method မှာ Polymorphic Type ဖြစ်တဲ့ AeroVehicle ကို Return ပြန်ခြင်းအားဖြင့် AeroVehicle ရဲ့ Sub Type တွေမှန်သမျှကို Return ပြန်လို့ရပါတယ်။ Drone ကိုပါ ပေးသုံးချင်တယ်ဆိုရင် Drone ကို AeroVehicle ကို Extends လုပ်၊ ပြီးတော့ DroneFactory ကို ItemFactory ကို Extend လုပ်ပြီး create Method ကို Drone Object ကို Return ပြန်ပေးလိုက်လို့ရပါတယ်။

ပြီးတော့ GameController ဘက်မှာ ယခင် Factory တစ်ခုချင်းစီကို Instance Member အဖြစ်ယူနေတဲ့ နေရာကို ItemFactory ရဲ့ Collection ကို Dependency အဖြစ်ယူလိုက်ရုံပါပဲ။ ဒါဆိုရင် ItemFactory ကို Extendss လုပ်ထားတဲ့ Factory မှန်သမျှကို Collection ထဲမှာ သိမ်းထားလို့ရပါတယ်။ Game Level အလိုက် ItemFactory တွေကို ထပ်ထပ်ပြီး ပေါင်းထည့်သွားမယ်ဆိုရင် Game Item အမျိုးမျိုးကို ထပ်တိုးပြီး အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ဒီနေရာမှာ အသုံးပြုသွားတဲ့ ItemFactory ဆိုတာကလဲ Polymorphic Type ဖြစ်ပါတယ်။



Polymorphic Type တွေကို Instance Member အဖြစ် အသုံးပြုခြင်းအားဖြင့် ပြုပြင်ပြောင်းလဲရလွယ်ကူတဲ့ Application တွေကို ရေးသားလာနိုင်တယ်ဆိုတာကို သိနိုင်ပါတယ်။

Polymorphism ကို အသုံးပြုခြင်းအားဖြင့် Requirement ပြောင်းသွားတဲ့အခါမှာလဲ အလွယ်တကူ ပြောင်းလဲနိုင်ပြီး၊ Behavior အသစ်တွေကို ထပ်မံဖြည့်စွက်လိုတဲ့ အခါမှာလဲ အလွယ်တကူ ဆောင်ရွက်နိုင်တယ်ဆိုတာကို သိရှိနိုင်ပါတယ်။

Java 5 Features



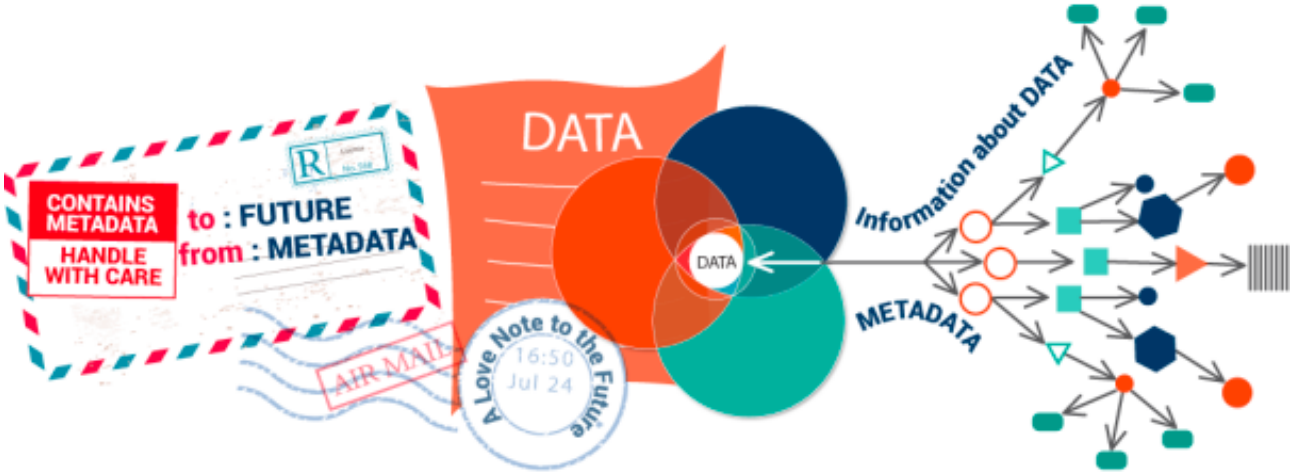
Java ဟာ ၁၉၉၆ ခုနှစ်ကစပြီး Release လုပ်ခဲ့ပြီး အကြိမ်ကြိမ် Version Update လုပ်လာခဲ့ပါတယ်။ ဒီလို Version တွေကို ပြောင်းလာတဲ့အခါမှာ အခြေခံအားဖြင့် Bug Fix တွေ၊ Performance Improvement တွေ၊ Api အသစ်တွေနဲ့ Language ပိုင်းဆိုင်ရာ ပြောင်းလဲမှုတွေကို ထည့်သွင်းလာခဲ့ကြပါတယ်။

Java ရဲ့သမိုင်းမှာ Language ပိုင်းဆိုင်ရာပြောင်းလဲမှုကို စတင်ခဲ့တဲ့ Version လို့ပြောလို့ရနိုင်ပါတယ်။ အရင်တုန်းက မရှိသေးတဲ့ ရေးသားပုံရေးသားနည်းတွေ စတင်ပါဝင်လာခဲ့ပါတယ်။ ပါဝင်လာတဲ့အကြောင်းအရာတွေကတော့ Annotations, Enumerations နဲ့ Generics တို့ပဲ ဖြစ်ကြပါတယ်။

Java 5 ရဲ့ Annotations ကြောင့် Java Framework တွေမှာ XML တွေကို ရေးသားစရာမလိုတော့ပဲ Annotation ကို အသုံးပြုပြီး အလွယ်တကူရေးသားလာနိုင်ခဲ့ပါတယ်။ Generics ကြောင့် Java မှာ Type ကိုလဲ Abstraction ဖြစ်အောင် ရေးသားလာနိုင်ခဲ့ပါတယ်။ ဒါ့ကြောင့် Collections Framework မှာ Type Safe ဖြစ်အောင် ရေးသားလာနိုင်ခဲ့ပါတယ်။

Annotation

Java SE 5 အရောက်မှာ အသစ်ဖြည့်စွက်ခဲ့သော Interface အမျိုးအစားတစ်ခု ဖြစ်ပါတယ်။ Source Code တွေထဲမှာ Meta Data တွေကို ဖေါ်ပြလိုတဲ့ အခါမှာ အသုံးပြုနိုင်ပါတယ်။ ဒီနေရာမှာ Meta Data ဆိုတာဘာလဲ ဆိုတာကို အရင်ဆုံးရှင်းပြပါမယ်။



Application ထဲကနေ အသုံးပြုနိုင်တဲ့ အချက်အလက်တွေကို Data လို့ခေါ်ဆိုပြီး အဲ့ဒဲ Data တွေဟာ ဘာဖြစ်တယ်၊ ဘယ်လို ထူးခြားချက်ရှိတယ်ဆိုတာတွေကို ဖေါ်ပြနိုင်တာက Meta Data ဖြစ်ပါတယ်။

ဆိုကြပါစို့ ကျွန်တော်တို့ ရေးတဲ့ Project ထဲမှာ Class တွေကို ရေးတဲ့အခါမှာ တစ်ချို့တွေကို Data Base ထဲက Table တွေကို ကိုယ်စားပြုတဲ့ Class အနေနဲ့ ရေးချင် ရေးပါလိမ့်မယ်။ အဲ့ဒီအခါမျိုးဆိုရင် အဲ့ဒဲ Class တွေကို Database ထဲက Table တွေကို ကိုယ်စားပြုတဲ့ Class ဖြစ်ကြောင်း၊ Table Name က ဘယ်လို ဖြစ်ကြောင်း၊ Member Properties တွေကိုလဲ ဘယ် Colum ကို ကိုယ်စားပြုကြောင်း၊ ပြီးတော့ Primary Key ဖြစ်တဲ့ Column ဖြစ်ကြောင်းတို့ကို ဖေါ်ပြလိုတဲ့အခါမှာ Annotation ကိုအသုံးပြုပြီး ဖေါ်ပြနိုင်ပါတယ်။

Annotation မရှိခင်တုန်းကဆိုရင် Meta Data တွေကို ရေးသား အသုံးပြုလိုတဲ့ အခါမှာ XML Mapping File တွေကို အသုံးပြုခဲ့ကြရပါတယ်။ ဒါ့ကြောင့် Application တစ်ခုကို ရေးချင်ပြီဆိုရင် Class တွေကိုလဲ ရေးသားဖို့လိုအပ်သလို XML Configuration File တွေလဲ ပွစာကြဲနေအောင်ရေးဖို့ လိုအပ်ခဲ့ပါတယ်။ Annotation ကို အသုံးပြုပြီး Meta Data တွေကို Source Code တွေထဲမှာ ရေးသားလို့ရနိုင်လာတဲ့ အတွက် XML တွေကို ရေးသားစရာမလိုအပ်တော့ပဲ Java Code တွေနဲ့တင် Data တွေကိုကော Meta Data တွေကိုပါ ရေးသားလာနိုင်ခဲ့ပါတယ်။ ဒါ့ကြောင့် API တွေ Framework တွေကို အလွယ်တကူ အသုံးပြုလာနိုင်ခဲ့ပါတယ်။

ကျွန်တော်တို့ Annotation ကို လေ့လာတဲ့ နေရာမှာ အပိုင်း ၂ ပိုင်းနဲ့ လေ့လာနိုင်မှာ ဖြစ်ပါတယ်။ ပထမပိုင်းကတော့ ရေးသားထားတဲ့ Annotation တွေကို ဘယ်လို အသုံးပြုမလဲ ဆိုတဲ့ အပိုင်းဖြစ်ပြီး၊ ဒုတိယ အပိုင်းကတော့ Annotation တွေကို ဘယ်လို ရေးသားပြီး ဘယ်လို Service တွေကို ရေးသားမလဲ ဆိုတဲ့ အပိုင်းပဲ ဖြစ်ပါတယ် အမှန်အတိုင်း ပြောရမယ်ဆိုရင် Java ကို စတင်လေ့လာနေသူတစ်ယောက် အတွက်ဆိုရင်တော့ ပထမအပိုင်းဖြစ်တဲ့ Annotation တစ်ခုကို ဘယ်လို အသုံးပြုမလဲဆိုတာကို ကောင်းကောင်းနားလည်တယ်ဆိုရင် လုံလောက်ပါတယ်။ ဒုတိယအပိုင်းကတော့ Framework ရေးသားသူတွေပြီးတော့ API ရေးသားသူတွေနဲ့ ဆိုင်တဲ့ အပိုင်းပါ။

နောက်ကွယ်မှာ ဘယ်လိုအလုပ်လုပ်နေတယ် ဆိုတာကို နားလည်ရုံရည်ရွယ်ချက်နဲ့ လေ့လာသွားတာ သင့်တော်မယ်လို့ ထင်ပါတယ်။

Using Annotations

ကျွန်တော်တို့ ဒီနေရာမှာတော့ အသင့် ရေးသားထားတဲ့ Annotation တွေကို ဘယ်လို အသုံးပြုသွားမလဲ ဆိုတာကို လေ့လာသွားကြပါမယ်။ ဒါဆိုရင် အသင့် အသုံးပြုနိုင်တဲ့ Annotation တွေကို ဘယ်သူတွေက ရေးပေးထားမှာလဲ ဆိုတာကနေ လေ့လာသွားကြရအောင်။

Java SE 5 အရောက်မှာ Annotation တွေကို ရေးသား အသုံးပြုလာနိုင်ခဲ့ပါတယ်။ Java SE ရဲ့ API တွေမှာလဲ Predefine Annotation တွေလဲ ပါဝင်လာသလို JPA တို့လို Hibernated တို့လို Spring Framework တို့လို Third Party Framework တွေမှာလဲ Annotation တွေနဲ့ Annotation Processor တို့ကို အသုံးပြုပြီး သူတို့ရဲ့ Framework တွေကို အသုံးပြုရတာလွယ်ကူအောင် ပြင်ဆင်လာခဲ့ပါတယ်။

ဒီနေရာမှာတော့ Java SE ရဲ့ API ထဲမှာ Default အနေနဲ့ အသုံးပြုနိုင်တဲ့ Annotation တွေကို ဘယ်လို အသုံးပြုရေးသားရမယ်ဆိုတာကို လေ့လာသွားကြရအောင်။ လက်ရှိအနေအထားအရ Default အနေနဲ့ အသုံးပြုနိုင်တဲ့ Annotation တွေကတော့ အောက်ပါအတိုင်း ဖြစ်ကြပါတယ်။

@Deprecated

အရင်တုန်းကတော့ အသုံးပြုနေခဲ့ပြီး ရှေ့လျောက် အသုံးမပြုစေလိုတော့တဲ့ Element တွေမှာ @Deprecated Annotation ကို ရေးသားအသုံးပြုသင့်ပါတယ်။ @Deprecated Annotation ကို Type, Field, Method, Package, Constructor, Parameter နဲ့ Local Variable တွေမှာ ရေးသား အသုံးပြုနိုင်ပါတယ်။ တကယ်လို့ Compile လုပ်စဉ်မှာ @Deprecated Annotation ကို ရေးသားထားတဲ့ Element ကို အသုံးပြုထားမယ်ဆိုပါက Compile Time မှာ Warning ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

```
public class Student {

	@Deprecated
	public void greet() {
		System.out.println("Hello");
	}
}
```

အထက်ပါအတိုင်း Student Class ရဲ့ greet() method မှာ @Deprecated Annotation ကို ရေးသားထားပါတယ်။

```
3 public class AnnotationDemo {
4
5     public static void main(String[] args) {
6         Student student = new Student();
7         student.greet();
8     }
9 }
```

အထက်ပါအတိုင်း @Deprecated Annotation ကို ရေးသားထားတဲ့ greet() method ကို အသုံးပြုတဲ့နေရာမှာ Warning ပြပေးနိုင်တယ်ဆိုတာကို ဖော်ပြပေးနိုင်ပါတယ်။ ရှေ့လျှောက် အသုံးမပြုစေလိုတဲ့ နေရာတွေမှာ @Deprecated Annotation ကို အသုံးပြုပြီး ရေးသားနိုင်ပါတယ်။

@Override

အမှုမဲ့ အမှတ်မဲ့ အသုံးအများဆုံး Annotation ဟာ ဘာလဲလို့ မေးရင် @Override Annotation ဖြစ်ပါမယ်။ ယနေ့ခေတ် IDE ကို အသုံးပြုပြီး ကုဒ်ရေးနေကြတဲ့အခါမှာ ဒီ Annotation ဟာ Class တွေကို Create လုပ်တဲ့ အခါမျိုးမှာ ပါလာတတ်ပါတယ်။

ဆိုကြပါစို့ Eclipse IDE ကို အသုံးပြုပြီး Servlet တွေရေးတဲ့အခါ၊ Servlet Listener တွေ Servlet Filter တွေရေးတဲ့အခါမျိုးမှာ Code Template ကနေ Generate လုပ်လာတဲ့ ကုဒ်တွေထဲမှာ @Override ဟာ ပါလာတတ်ပါတယ်။ ပြီးတော့ Interface ကို Implement လုပ်တဲ့အခါမျိုး၊ ဒါမှ မဟုတ် Abstract Class ကို Extends လုပ်တဲ့အခါမျိုးမှာ ဒီအတိုင်း ရေးသားထားပါက IDE မှာ Error တက်နေပါမယ်။ အဲ့ဒီနေရာမှာ Hint အတိုင်း Implement လုပ်ရသေးတဲ့ Abstract Method တွေကို Create လုပ်မယ်ဆိုတာကို ရွေးပြီး Code တွေကို Generate လုပ်တဲ့အခါမျိုးတွေမှာ @Override Annotation တွေဟာ ပါလာတတ်ပါတယ်။

ကဲ ဒီနေရာမှာ @Override Annotation ကို ရေးထားရင် Super Class ထဲမှာပါတဲ့ Method ကို Override လုပ်ထားတယ်ဆိုတာကို ပြောပြပေးနိုင်တယ်လို့ အလွယ်ပြောလို့ရနိုင်ပါတယ်။ မဟုတ်လို့လားဆိုတော့ ဟုတ်ပါတယ်။ ဒါပေမဲ့ ဒီလို တွေးကြည့်ရအောင် @Override Annotation ကို မရေးထားရင် Override မဖြစ်တော့လို့လား။ စမ်းကြည့်ရင် @Override Annotation ကို မရေးထားရင်လဲ Override ဖြစ်နေတယ်ဆိုတာကို တွေ့ရပါမယ်။

ကိုင်း ဒါဖြင့် ဘာလို့ @Override Annotation ကို ရေးဖို့လိုအပ်ရတာလဲ။ မရေးတာနဲ့ ရေးတာနဲ့ ဘာများကွာသွားလို့လဲ ဆိုတာကို ဒီနေရာမှာ လေ့လာသွားကြရအောင်။

အရင်ဆုံး @Override မပါပဲ Override ဖြစ်မဖြစ် စမ်းကြည့်ကြရအောင်။ Override နဲ့ ပတ်သက်ပြီး OOP ရဲ့ Inheritance အကြောင်းမှာ ဖော်ပြပြီး ဖြစ်ပါတယ်။ တကယ်လို့ ဒီနေရာမှာ Override ဆိုတာဘာလဲမသိသေးရင် အဲ့ဒီအခန်းကို အရင်ဖတ်စေလိုပါတယ်။

```
class Super {
	public void hello() {
		System.out.println("Super Hello");
	}
}

class Sub extends Super {
	public void hello() {
		System.out.println("Sub Hello");
	}
}
```

အထက်ပါအတိုင်း Super Class ထဲမှာ hello() method ကို ရေးသားထားပါတယ်။ ပြီးတော့ အဲ့ဒီ Method မှာ Super Hello ဆိုတာကို System Console မှာ ဖော်ပြထားခိုင်းပါတယ်။ ပြီးတော့ Sub Class ကနေ Super Class ကို Entends လုပ်ထားတယ်။ အဲ့ဒီအတွက် Super Class ထဲမှာ ရေးထားတဲ့ hello() method ဟာ Inheritance ကြောင့် Sub Class ကနေ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

```
public class OverrideDemo {

	public static void main(String[] args) {
		new Sub().hello();
	}
}
```

ဒီလိုမျိုး Super Class ကနေ Inheritance လုပ်လို့ရလာတဲ့ Method တွေကို Sub Class ထဲကနေ Signature တူတဲ့ Method တစ်ခုကို ရေးသားခြင်းကို Override လုပ်တယ်လို့ ခေါ်ပါတယ်။ တကယ်လို့ Sub Class ကနေ Object ဆောက်ပြီး hello() method ကို အလုပ်လုပ်ခိုင်းမယ်ဆိုရင် Inheritance လုပ်လာတဲ့ Behavior ကို မသုံးပဲ Sub Class ထဲမှာ ရှိတဲ့ Method ကို အသုံးပြုသွားခြင်းကို Override လုပ်တယ်လို့ ခေါ်ပါတယ်။

```
class Sub extends Super {

	@Override
	public void hello() {
		System.out.println("Sub Hello");
	}
}
```

အထက်ပါအတိုင်း Sub Object ကို ဆောက်ပြီး hello() method ကို Invoke လုပ်ကြည့်ပါတယ်။ တကယ်တမ်းအဖြေမှာတော့ Sub Hello ဖြစ်ပါတယ်။ ဒါဟာ ဘာကိုပြောတာလဲဆိုရင် @Override Annotation ကို မရေးထားလဲ Override ဖြစ်သွားတယ်ဆိုတာကို ပြောပြတာ ဖြစ်ပါတယ်။

```
3  class Super {
4      private void hello() {
5          System.out.println("Super Hello");
6      }
7  }
8
9  class Sub extends Super {
10
11     @Override
12     public void hello() {
13         System.out.println("Sub Hello");
14     }
15 }
```

ဒီနေရာမှာ Sub Class ရဲ့ hello() Method ရှေ့မှာ @Override Annotation ကို အောက်ပါအတိုင်း ရေးသားကြည့်ပြီး OverrideDemo ကို Run ကြည့်ပါမယ်။
Annotation မရေးခင်ကအတိုင်းပဲ အဖြေထွက်တာကို တွေ့ရပါတယ်။ ဒါဟာ ဘာကိုပြောတာလဲ ဆိုရင် @Override ကို ရေးထားထားမထားထား Override ဟာ ဖြစ်ပေါ်တယ်ဆိုတာပါပဲ။ ကဲဒါဖြင့် @Override ကို ဘာကြောင့် ရေးကြတာလဲ။
အထက်ပါအတိုင်း Super Class ထဲက hello() method ရဲ့ Access Modifier ကို private လို့ပြောင်းလိုက်ပါတယ်။ ဒီနေရာမှာ @Override Annotation ကို Sub Class ထဲမှာ ရေးသားထားရင် Compile Time မှာ Error

တက်တာကို တွေ့ရပါတယ်။ ဘာလို့လဲဆိုတော့ Super ထဲမှာပါတဲ့ hello() method ဟာ private ဖြစ်သွားတဲ့အတွက် Override မဖြစ်တော့ပါဘူး။ ဒါ့ကြောင့် @Override Annotation ကို အသုံးပြုထားရင် Override မဖြစ်တဲ့နေရာတွေမှာ ဆိုရင် Compile Time မှာ Error ထုတ်ပေးနိုင်တယ်ဆိုတာကို သိနိုင်ပါတယ်။

@SupressWarnings

Compile Time မှာ Warning အနေနဲ့ ထွက်နေတဲ့ အရာတွေကို Warning မထုတ်စေလိုတဲ့ အခါမျိုးမှာ ဒီ Annotation ကို ရေးသား အသုံးပြုနိုင်ပါတယ်။ ဆိုကြပါစို့ Serializable Interface ကို Implement လုပ်ထားပြီး Serial Version UID ကို မရေးထားခဲ့တဲ့ အခါမျိုးတွေ အသုံးမပြုထားတဲ့ Private Variable တွေနဲ့ Private Method တွေ အစရှိတဲ့ နေရာတွေမှာ Compile Time Warning တွေပြနေလေ့ရှိပါတယ်။

ရှေ့မှာ ဖော်ပြခဲ့တဲ့ Super Class ထဲမှာလဲ hello() method ကို private လုပ်လိုက်တဲ့အခါမှာ အဲ့ဒီ hello() method ကို အသုံးပြုနေတဲ့ နေရာမရှိတဲ့ အတွက် Compile Time မှာ Warning ထွက်နေတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။ အဲ့ဒီလို နေရာမျိုးမှာ @SuppressWarnings Annotation ကို အသုံးပြုပြီး Warning ကို မထွက်အောင် ရေးသားနိုင်ပါတယ်။

```
2
3  class Super {
4
5      @SuppressWarnings("unused")
6      private void hello() {
7          System.out.println("Super Hello");
8      }
9  }
10
```

အထက်ပါအတိုင်း hello() method ရှေ့မှာ @SuppressWarnings(“unused”) လို့ရေးသားလိုက်တာနဲ့ Warning ကို မပြတော့တာကို တွေ့ရမှာ ဖြစ်ပါတယ်။

```
@SuppressWarnings("serial")
public class Student implements Serializable {

}
```

အထက်ပါ Student Class ဟာ Serializable Interface ကို Implement လုပ်ထားတဲ့ အတွက် ပုံမှန်ဆိုရင် serialVersionUID ကို သတ်မှတ်ရေးသားထားဖို့ လိုအပ်ပါတယ်။ serialVersionUID အကြောင်းနဲ့ ပတ်သက်ပြီး Java IO အခန်းမှာ အသေးစိတ် ဖော်ပြပါမယ်။ ဒီနေရာမှာမတော့ Serializable Interface ကို Implement လုပ်ထားတဲ့ Class တွေမှာ serialVersionUID ကို မရေးထားရင် Compile Time မှာ Warning ထွက်နေမယ်လို့ပဲ မှတ်ထားပေးပါ။ ဒီလို Warning တွေမှာလဲ @SuppressWarnings(“serial”) လို့ရေးသားထားပြီး Warning မထွက်အောင် လုပ်နိုင်ပါတယ်။

အခြားသော Predefine Annotation အနေနဲ့ Java SE 7 အရောက်မှာ @SaveVerarges Annotation ကို အသစ် ဖြည့်စွက်လာခဲ့ပြီး Java SE 8 အရောက်မှာလဲ @FunctionalInterface တို့ကို ဖြည့်စွက်လာခဲ့ပါတယ်။ @SaveVerarges ကိုတော့ Generics အကြောင်းမပြောပဲ ပြောလို့ မဖြစ်တဲ့ အတွက် Verarges အကြောင်းနဲ့ Generics အကြောင်းကို ဖော်ပြတဲ့ အခါကျမှ ဖြည့်စွက် ဖော်ပြသွားပါမယ်။

အဲ့ဒီလိုပဲ @FunctionalInterface Annotation အကြောင်းကိုလဲ Java SE 8 ရဲ့ Lambda Expression အကြောင်းကို ဖော်ပြတဲ့ အခါကျမှ ပူးတွဲဖော်ပြသွားပါမယ်။

Writing Annotation

ဒီနေရာကနေစပြီး Annotation တစ်ခုကို ဘယ်လို ရေးသားပြီး ဘယ်လို အသုံးပြုနိုင်တယ်ဆိုတာကို လေ့လာသွားကြရအောင်။ Annotation တစ်ခုကို ရေးသားမယ်ဆိုရင် @interface ဆိုတဲ့ Keyword ကို အသုံးပြုပြီး ရေးသားရမှာ ဖြစ်ပါတယ်။ Annotation ဆိုတာက တကယ်တော့ Interface အမျိုးအစားတစ်ခုဖြစ်ပါတယ်။ အနဲဆုံးတော့ Annotation တစ်ခုဖြစ်အောင် @interface ဆိုတဲ့ Keyword ရယ်၊ Table ဆိုတဲ့ Identifier ရယ်၊ {} လို့ရေးသားထားတဲ့ Body သာ လိုအပ်ပါတယ်။

```
public @interface Table {

}
```

Identifier နေရာမှာကတော့ မိမိနှစ်သက်ရာ နာမည်ကို ပေးပြီး ရေးသားလို့ရပါတယ်။ Interface Body ထဲမှာကတော့ Abstract Method တွေကို ရေးသားလို့ရပါတယ်။ Annotation ထဲမှာ ရေးသားနိုင်တဲ့ Method တွေအကြောင်းကိုတော့ နောက်မှ ဆက်ပြီးဖော်ပြသွားပါမယ်။

Meta Annotations

အထက်ပါ အတိုင်းရေးသားထားရုံနဲ့ Annotation တော့ဖြစ်သွားပါတယ်။ ဒါပေမဲ့ တကယ်တမ်းတော့ အသုံးပြုလို့ မရသေးပါဘူး။ တကယ်တမ်း အသုံးပြုနိုင်အောင် ရေးသားမယ်ဆိုရင် ရေးသားထားတဲ့ Annotation ကို ဘယ်လို နေရာတွေမှာ ရေးသားလို့ရနိုင်မလဲ ပြီးတော့ ဘယ်နေရာမျိုးတွေမှာ ဒီ Annotation ကို အသုံးပြုလို့ရနိုင်မလဲ ဆိုတာတွေကို သတ်မှတ်ပေးဖို့လိုအပ်ပါတယ်။

အဆိုပါ သတ်မှတ်ချက်တွေကိုလဲ Annotation တွေကို အသုံးပြုပြီးရေးသားရမှာ ဖြစ်ပါတယ်။ ဒီလို Annotation တွေကို ရေးတဲ့နေရာမှာ အသုံးပြုနိုင်တဲ့ Annotation တွေကို Meta Annotation လို့ခေါ်ပါတယ်။
အသုံးပြုနိုင်တဲ့ Meta Annotation တွေကတော့ အောက်ပါအတိုင်းဖြစ်ပါတယ်။

@Retention

ရေးသားထားတဲ့ Annotation ဟာ ဘယ်နေရာတွေမှာ အသုံးပြုမှာလဲ ဆိုတာကို သတ်မှတ်ပေးနိုင်တဲ့ Meta Annotation တစ်ခုဖြစ်ပါတယ်။ @Retention Annotation မပါဲ့ Annotation တစ်ခုကို ရေးသားလို့ မရနိုင်ပါဘူး။ value() ဆိုတဲ့ Method တစ်ခုပါဝင်တဲ့အတွက် အဲ့ဒီ Annotation ကို ရေးသားတဲ့အခါမှာ RetentionPolicy Enumeration တစ်ခုကို value အနေနဲ့ ရေးသားပေးရမှာ ဖြစ်ပါတယ်။

```
package java.lang.annotation;

@Documented
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.ANNOTATION_TYPE)
public @interface Retention {
    RetentionPolicy value();
}
```

RetentionPolicy အနေနဲ့ရေးသားနိုင်တဲ့ တန်ဖိုးတွေကတော့ အောက်ပါအတိုင်းဖြစ်ပါတယ်။

RetentionPolicy	Description
SOURCE	Compiler ကနေ Compile Time မှာ အသုံးပြုစေလိုတဲ့ Annotation တွေကို ရေးသားလိုတဲ့အခါမှာ သတ်မှတ် အသုံးပြုနိုင်ပါတယ်
CLASS	Compiler ကနေ Compile လုပ်တဲ့အခါမှာ အသုံးပြုပြီး ရလာတဲ့ Class ထဲမှာ ထည့်သွင်းပေးလိုက်ပြီး JVM ကနေ Runtime မှာ အသုံးမပြုလိုတဲ့ Annotation တွေကို ရေးသားလိုတဲ့အခါမှာ CLASS ကို အသုံးပြုနိုင်ပါတယ်
RUNTIME	JVM ကနေ Runtime မှာ အသုံးပြုလိုတဲ့ Annotation တွေကို ရေးသားလိုတဲ့နေရာမှာ အသုံးပြုနိုင်ပါတယ်

@Target

Target Annotation ဟာလဲ Annotation တစ်ခုကို ရေးသားတဲ့နေရာမှာ မဖြစ်မနေရေးသားပေးရမည့် Meta Annotation တစ်ခုဖြစ်ပြီး လက်ရှိရေးသားထားတဲ့ Annotation ကို ဘယ်နေရာမှာ ရေးသားနိုင်သလဲ ဆိုတာကို သတ်မှတ်ပေးနိုင်ပါတယ်။

```
package java.lang.annotation;

@Documented
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.ANNOTATION_TYPE)
public @interface Target {
    ElementType[] value();
}
```

Target Annotation ထဲမှာ ElementType Enum Array ကို Return ပြန်တဲ့ value() ဆိုတဲ့ Method တစ်ခုကို ရေးသားထားပါတယ်။ ဒါ့ကြောင့် Target ကို ရေးသားတဲ့နေရာမှာ ElementType ကို တစ်ခုထက်မက သတ်မှတ် ရေးသားဖို့လိုအပ်ပါတယ်။သတ်မှတ် အသုံးပြုနိုင်တဲ့ ElementType တွေကတော့ အောက်ပါအတိုင်းဖြစ်ပါတယ်။

Element Type	Description
TYPE	Class, Interface နဲ့ Enum တို့ရဲ့ Type Declaration တွေမှာ အသုံးပြုနိုင်ပါတယ်
FIELD	Instance Variable တွေမှာ ရေးသားနိုင်ပါတယ်
METHOD	Method တွေရဲ့ရှေ့မှာ ရေးသား အသုံးပြုနိုင်ပါတယ်
PARAMETER	Method Parameter တွေနဲ့ Constructor Parameter တွေမှာ ရေးသားအသုံးပြုနိုင်ပါတယ်
CONSTRUCTOR	Constructor တွေကိုရေးသားတဲ့နေရာမှာ ရေးသားနိုင်ပါတယ်
LOCAL_VARIABLE	Local Variable တွေကို Declare လုပ်တဲ့နေရာမှာ ရေးသားအသုံးပြုနိုင်ပါတယ်
ANNOTATION_TYPE	Annotation Type တွေကို ရေးသားတဲ့နေရာမှာ အသုံးပြုနိုင်ပါတယ်
PACKAGE	Package တွေကို Declare လုပ်တဲ့နေရာမှာ အသုံးပြုနိုင်ပါတယ်

Element Type	Description
TYPE_PARAMETER	Java SE 8 ကျမှ အသစ်ပါဝင်လာသော Element ဖြစ်ပါတယ် Generic Type Parameter တွေကို ရေသားတဲ့နေရာမှာ အသုံးပြုနိုင်ပါတယ်
TYPE_USE	Java SE 8 ကျမှ အသစ်ပါဝင်လာသော Element ဖြစ်ပါတယ် Type တွေကို အသုံးပြုတဲ့နေရာမှာ ရေးသားနိုင်ပါတယ်

Optional Meta Annotations

အရှေ့မှာ ဖေါ်ပြထားသော @Retention နဲ့ @Target Annotation တွေကတော့ Annotation တစ်ခုကို ရေးသားတဲ့နေရာမှာ မဖြစ်မနေ ရေးသားရန်လိုအပ်တဲ့ Annotation တွေ ဖြစ်ကြပါတယ်။ တဖန် Meta Annotation တွေထဲမှာ လိုအပ်လာရင် ရေးသားနိုင်တဲ့ Annotation တွေလဲရှိကြပါသေးတယ်။

@Docmentated

@Documentated Annotation တွေကတော့ ရေးသားထားတဲ့ Annotation ကို Java Doc အနေနဲ့ ဖေါ်ပြဖို့ လိုအပ်ကြောင်း သတ်မှတ်ပေးနိုင်တဲ့ Annotation တစ်ခုဖြစ်ပါတယ်။ Javadoc Tools ကို အသုံးပြုပြီး Java Doc တွေကို Generate လုပ်တဲ့ နေရာမှာ အသုံးပြုမှာ ဖြစ်ပါတယ်။ Default အတိုင်းဆိုပါက Annotation တွေကို Java Doc ထုတ်တဲ့နေရာမှာ အသုံးမပြုပါဘူး။

@Inherited

@Inherited Annotation ကို Annotation တစ်ခုမှာ ရေးသားထားနိုင်ပါတယ်။ အဲ့ဒီလိုရေးသားထားတဲ့ Annotation ကို Super Class ထဲမှာ ရေးသားထားမယ်ဆိုရင် အဲ့ဒီ Super Class တွေမှာ အဲ့ဒီ Annotation ကို မရေးသားထားဘူးဆိုရင်တောင်မှ အဲ့ဒီ Interface ကို အဲ့ဒီ Sub Class တွေမှာ ရရှိနေမှာ ဖြစ်ပါတယ်။

မှတ်ချက် ။ ။ Annotation ကို ရေးသားတဲ့ အခန်းရောက်မှ @Inherited Annotation ရဲ့ အသုံးပြုပုံကို လက်တွေ့ရေးသားကြည့်ပါမယ်။

@Repeatable

Java SE 8 အရောက်မှ ဖြည့်စွက်လာခဲ့တဲ့ Meta Annotation အမျိုးအစားတစ်ခုဖြစ်ပါတယ်။ Annotation တစ်ခုကို ရေးသားတဲ့အခါမှာ @Repeatable Annotation ကို ရေသားထားခဲ့မယ်ဆိုရင် အဲ့ဒီ Annotation ကို တစ်နေရာထဲမှာ အကြိမ်ကြိမ် ရေးသား အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

မှတ်ချက် ။ ။ Annotation ကို ရေးသားတဲ့ အခန်းရောက်မှ @Repeatable Annotation ရဲ့ အသုံးပြုပုံကို လက်တွေ့ရေးသားကြည့်ပါမယ်။

Using Meta Annotations

Meta Annotation အကြောင်းကိုလေ့လာပြီးတဲ့ နောက်မှာ Meta Annotation တွေကို အသုံးပြုပြီး Annotation တစ်ခုကို ဘယ်လို ရေးသားမယ်ဆိုတာကို လေ့လာသွားကြပါမယ်။

String Type Variable တွေမှာ ရေးသားနိုင်တဲ့ Annotation တစ်ခုကို ရေးသားကြည့်ပါမယ်။ တကယ်လို့ အဲ့ဒီ Annotation ကို တတ်ဆင်ရေးသားထားတဲ့ Variable တွေရဲ့ တန်ဖိုးဟာ Null ဖြစ်နေရင် Blank String တန်ဖိုး တစ်ခုကို အစားထိုးပေးနိုင်တဲ့ Annotation တစ်ခုကို ရေးသားသွားပါမယ်။

```
@Retention(RUNTIME)
@Target(FIELD)
public @interface NullWithBlank {

}
```

အထက်ပါအတိုင်း NullWithBlank Annotation ကို ရေးသားထားပါတယ်။ ဒီ Annotation ကို Runtime မှာ အသုံးပြုလိုတဲ့အတွက် @Retention(RUNTIME) လို သတ်မှတ်ရေးသားထားပါတယ်။ တဖန် ဒီ Annotation ကို Instance Variable တွေမှာ ရေးသားလိုတဲ့ အတွက် @Target(FIELD) လို့ ရေးသားထားပါတယ်။

အထက်ပါအတိုင်း ရေးသားထားခြင်းအားဖြင့် @NullWithBlank Annotation ကို Instance Variable တွေမှာ ရေးသားနိုင်ပြီး Runtime မှာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

```
public class Student {

      @NullWithBlank
      private String name;

      public String getName() {
            return name;
      }

      public void setName(String name) {
            this.name = name;
      }
}
```

အထက်ပါ Student Class ကတော့ အရှေ့မှာ ရေးသားထားခဲ့တဲ့ Annotation ကို အသုံးပြုမည့် Class ဖြစ်ပါတယ်။ ဒီ Class ရဲ့ name Variable နေရာမှာ @NullWithBlank Annotation ကို တပ်ဆင်ရေးသားထားပါတယ်။

@NullWithBlank Annotation ရဲ့ Target မှာ FIELD လို့ ရေးသားထားခဲ့တဲ့ အတွက်ကြောင့်သာ Instance Variable မှာ ရေးသားနိုင်တာ ဖြစ်ပါတယ်။ တစ်ခြားနေရာတွေမှာတော့ ရေးသားလို့ ရနိုင်မှာ မဟုတ်ပါဘူး။

ကဲဆက်လက်ပြီး ရေးသားထားခဲ့တဲ့ Annotation ကို ပြန်ပြီး အသုံးပြုတဲ့ အပိုင်းကို လေ့လာသွားကြရအောင်။ အရင်ဆုံး Annotation ကို မသုံးပဲ ရေးကြည့်ပါမယ်။

```
public class AnnotationDemo {

	public static void main(String[] args) {
		showStudentName(new Student());
	}

	static void showStudentName(Student student) {
		System.out.println(student.getName());
	}
}
```

အထက်ပါ Program ကို Run ကြည့်မယ်ဆိုရင် Console ပေါ်မှာ null ဆိုတာကို တွေ့ရပါမယ်။ ဘာလို့လဲ ဆိုတော့ showStudent method ထဲကို Parameter အနေနဲ့ new Student() ဆိုပြီး ထည့်ပေးလိုက်တဲ့ အတွက် Student Object ရဲ့ name ဟာ တန်ဖိုး မသတ်မှတ်ထားနိုင်ခဲ့ပါဘူး။ ဒါ့ကြောင့် null ဆိုပြီး ပြနေတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။

ဆက်လက်ပြီး Annotation ကို အသုံးပြုတဲ့ ပုံစံဖြစ်အောင် ပြင်ပြီးရေးသားကြည့်ပါမယ်။

```
static void showStudentName(Student student) throws Exception {

	for(Field f : student.getClass().getDeclaredFields()) {

		NullWithBlank anno =
			f.getDeclaredAnnotation(NullWithBlank.class);

		if(null != anno) {
			f.setAccessible(true);
			f.set(student, "");
		}
	}

	System.out.println(student.getName());
}
```

အထက်ပါအတိုင်း showStudent method ထဲမှာ student object ရဲ့ class ကနေ getDeclaredFields() method ကို အသုံးပြုပြီး Student Class ထဲမှာ ရေးသားထားတဲ့ Field တွေအားလုံးကို for statement နဲ့ Loop ပတ်ပါတယ်။

ပြီးတော့မှ Field ကနေ အဲ့ဒီ Field မှာ NullWithBlank Annotation ကို ရေးသားထားသလားဆိုတာကို f.getDeclaredAnnotation(NullWithBlank.class) method နဲ့ရှာ ဖွေပါတယ်။ တကယ်လို့ @NullWithBlank Annotation ကို ရေးသားထားတယ်ဆိုရင် အဲ့ဒီ Field ရဲ့ တန်ဖိုးကို “” နဲ့ set လုပ်ပေးပါတယ်။

ဒါ့ကြောင့် အထက်ပါ Class ကို ပြန်ပြီး Run ကြည့်မယ်ဆိုရင် null ထွက်မလာတော့တာကို တွေ့ရပါမယ်။ ဒီလိုမျိုး @NullWithBlank Annotation ကို Runtime မှာ အသုံးပြုနိုင်တာကလဲ Annotation ကို ရေးသားတဲ့အခါမှာ @Retention(RUNTIME) လို့ သတ်မှတ်ရေးသားထားခဲ့တဲ့ အတွက် ဖြစ်ပါတယ်။

Annotation with Parameter

Annotation ကို ရေးသားတဲ့ နေရာမှာ Annotation ကနေ Parameter တွေကို လက်ခံယူလို့ရနိုင်ပါတယ်။ Annotation ကနေ Parameter တွေကို လက်ခံလိုပါက Annotation ရဲ့ Body ထဲမှာ Abstract Method တွေကို ရေးသားပြီးရယူနိုင်ပါတယ်။

Annotation ထဲမှာ Abstract Method တွေကို ရေးသားတဲ့နေရာမှာ ပုံစံတစ်မျိုးထဲသာ ရေးသားနိုင်ပါတယ်။ ရေးသားတဲ့ Method ဟာ Return Type တစ်ခုကို ရေးသားထားဖို့လိုအပ်သလို Argument ကို တော့ ရေးသားလို့ မရနိုင်ပါဘူး။ Return Type အနေနဲ့ကတော့ Parameter အနေနဲ့ လက်ခံလိုတဲ့ Type ကို ရေးသားရမှာ ဖြစ်ပါတယ်။ Return Type အနေနဲ့ကတော့ နှစ်သက်ရာ Type ကို ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။ Java Basic Data Type တွေဖြစ်တဲ့ Primitive Type, String, Array နဲ့ Enum တို့ကိုလဲ ရေးသားအသုံးပြုနိုင်သလို၊ Customer Java Data Type ကိုလဲ ရေးသားနိုင်ပါတယ်။ အခြားသော Annotation Type တွေကိုလဲ ရေးသား အသုံးပြုနိုင်ပါတယ်။

ပြီးတော့ လိုချင်တဲ့ Parameter အရေအတွက် အလိုက် Abstract Method တွေကို ရေးသားပေးရမှာ ဖြစ်ပါတယ်။ ကျွန်တော်တို့ ရှေ့မှာရေးသားခဲ့တဲ့ @NullWithBlank Annotation ထဲမှာ Parameter တစ်ခုကို လက်ခံရေးသားကြည့်ကြရအောင်။

```
@Retention(RUNTIME)
@Target(FIELD)
public @interface NullWithBlank {
      String name();
}
```

အထက်ပါအတိုင်း @NullWithBlank Annotation ရဲ့ Body အတွင်းမှာ name() ဆိုတဲ့ Abstract Method တစ်ခုကို ရေးသားထားနိုင်ပါတယ်။ ဤသို့ရေးသားထားခြင်းအားဖြင့် @NullWithBlank Annotation ကို ရေးသားအသုံးပြုတဲ့နေရာမှာ name ဆိုတဲ့ တန်ဖိုးကို မရေးသားလို့ မရတော့ပါဘူး။

```
public class Student {

      @NullWithBlank(name = "No name")
      private String name;

      public String getName() {
            return name;
      }

      public void setName(String name) {
            this.name = name;
      }
}
```

အထက်ပါအတိုင်း @NullWithBlank Annotation ကို ရေးသားတဲ့အခါမှာ name ရဲ့ တန်ဖိုးကို သတ်မှတ် ရေးသားပေးရမှာ ဖြစ်တယ်။ ဆက်လက်ပြီး ရေးသားထားတဲ့ Parameter ကို ပြန်ပြီး အသုံးပြုတဲ့ အပိုင်းကို ရေးသားကြည့်ကြရအောင်။

အထက်ပါအတိုင်း f.set(student, anno.name()) ဆိုပြီး ရေးသားပေးနိုင်ပါတယ်။ @NullWithBlank Annotation ထဲမှာ name() ဆိုတဲ့ Abstract Method ကို ရေးသားထားတာ ဖြစ်တဲ့အတွက် anno.name() ဆိုတဲ့ Method ကို အသုံးပြုနိုင်မှာ ဖြစ်တယ်။

အထက်ပါ AnnotationDemo Class ကို Run ကြည့်တဲ့ အခါမှာ Student Class ထဲမှာ ရေးသားထားတဲ့ No Name ဆိုတာကို တွေ့ရှိရမှာ ဖြစ်ပါတယ်။

ဒီနေရာမှာ တစ်ခုသတိပြုစရာရှိတာက Annotation ထဲမှာ ရေးသားထားတဲ့ Abstract Method တွေအားလုံးကို Annotation ကို ရေးသားတဲ့ နေရာမှာ သတ်မှတ်ပေးဖို့လိုတယ်ဆိုတဲ့ အချက်ဖြစ်ပါတယ်။ တကယ်လို့ Method Name ကို value လို့ ပေးထားခဲ့ရင် Parameter ကို သတ်မှတ်တဲ့အခါမှာ Parameter Name ကို မရေးပဲ ထားလို့ ရပါတယ်။

Default Value for Annotation Methods

Annotation တွေထဲမှာ Method ကို ရေးသားတဲ့နေရာမှာ default keyword ကို အသုံးပြုပြီး Default Value ကို သတ်မှတ်ထားလို့ရပါတယ်။

```
@Retention(RUNTIME)
@Target(FIELD)
public @interface NullWithBlank {
      String name() default “Default Name”;
}
```

အထက်ပါအတိုင်း Default Value ကို သတ်မှတ်ထားပါက တကယ်လို့ name ရဲ့ တန်ဖိုးကို Annotation ကို အသုံးပြုတဲ့ အခါမှာ သတ်မှတ်မပေးခဲ့ရင် Default တန်ဖိုးကို အသုံးပြုသွားမှာ ဖြစ်ပါတယ်။ name ရဲ့ တန်ဖိုးကို သတ်မှတ်ထားခဲ့ရင်တော့ သတ်မှတ်ထားတဲ့ တန်ဖိုးကို အသုံးပြုသွားမှာပါ။

Enum

ကျွန်တော်တို့ ပရိုဂရမ်တွေ ရေးသားတဲ့ အခါမှာ နှစ်တစ်နှစ်ရဲ့ လတွေတို့ တပတ်မှာရှိတဲ့ ရက်တွေလို မပြောင်းလဲတဲ့ တန်ဖိုးတွေကို အသုံးပြုလိုတဲ့ အခါတွေရှိတတ်ပါတယ်။ နှစ်တစ်နှစ်ရဲ့ လတွေအနေနဲ့လဲ ဇန္နဝါရီကနေ ဒီဇင်ဘာ ပြောင်းလဲမှာ မဟုတ်သလို၊ ရက်အနေနဲ့လဲ တနင်္လာနေ့ကနေ တနင်္ဂနွေနေ့ အထိ ပြောင်းလဲမှာ မဟုတ်ပါဘူး။ ဒီလိုတန်ဖိုးတွေကို Constant တွေလို့ ခေါ်ဆိုလေ့ရှိပါတယ်။ Enum ဆိုတာက Constant တွေကို ဖေါ်ပြနိုင်တဲ့ Static Class အမျိုးအစားတွေ ဖြစ်ပါတယ်။



ကျွန်တော်တို့ Enum တွေကို လေ့လာတဲ့အခါမှာ Enum မပေါ်ခင်တုန်းက ဘယ်လိုရေးခဲ့ကြသလဲ၊ ပြီးတော့ Enum ပေါ်လာတော့ ဘယ်လိုပြောင်းသွားလဲဆိုတာရယ် Enum နဲ့ ဘာတွေလုပ်လို့ရလဲ ဆိုတဲ့ အချက်တွေနဲ့ လေ့လာသွားကြရအောင်။

Before Enum

Enum တွေမပေါ်ခင်က Constant တွေကိုရေးသားလိုတဲ့အခါမှာ Interface ထဲမှာ Static Final Variable တွေနဲ့ ရေးသားခဲ့ကြပါတယ်။ ဆိုကြပါစို့ ကျွန်တော်တို့ ရက်သတပတ်မှာပါတဲ့ ရက်တွေကို ကျွန်တော်တို့ ရေးချင်တယ်ဆိုရင် ဒီလို ရေးသားခဲ့ကြရပါတယ်။

```
public interface DaysOfWeek {

      static final int MON = 1;
      static final int TUE = 2;
      static final int WED = 3;
      static final int THU = 4;
      static final int FRI = 5;
      static final int SAT = 6;
      static final int SUN = 7;
}
```

အထက်ပါအတိုင်း Interface တစ်ခုကို ရေးပြီးတော့ ပါမည့် Item တွေကို Static Final Variable တွေအနေနဲ့ တန်ဖိုးတစ်ခုခုကို အသေသတ်မှတ်ပြီး ရေးသားခဲ့ကြရပါတယ်။

အဲ့ဒီလို Static Final Variable တွေကို ရေးသားပြီး Constant တွေအနေနဲ့ ရေးသား အသုံးပြုခဲ့ကြရပါတယ်။ အဲ့ဒီ Variable တွေဟာ Static Final ဖြစ်တဲ့ အတွက် ပြောင်းလို့ မရနိုင်ပါဘူး။ ဒါပေမဲ့ အဲ့ဒီ Constant တွေကို အခြားသော Variable တွေဆီကို Assign လုပ်လိုက်ရင်တော့ သတ်မှတ်ထားတဲ့ Type ရဲ့ Variable တွေသာ ဖြစ်သွားပါမယ်။

```
public class DaysOfWeekDemo {

      public static void main(String[] args) {

            int monday = DaysOfWeek.MON;

            monday = 11;

            System.out.println(monday);
      }
}
```

အထက်ပါအတိုင်း monday variable မှာ DaysOfWeek.MON ဆိုပြီး Assign လုပ်လို့ရပါတယ်။ DaysOfWeek.MON ရဲ့ Type ဟာလဲ int ဖြစ်သလို monday ရဲ့ Type ဟာ int ဖြစ်တဲ့ အတွက် ဖြစ်ပါတယ်။ ဒီနေရာအထိ ဘာမှ ပြဿနာ မရှိသေးပါဘူး။ Database ထဲမှာ int အနေနဲ့ သိမ်းမယ်။ ပြီးရင် View မှာ ပြန်ပြမယ်။ ပြဿနာ မရှိပါဘူး။

ဒါပေမဲ့ monday variable ဟာ int type ဖြစ်တဲ့အတွက် ရက် ခုနှစ်ရက် အပြင် အခြားသော int တန်ဖိုးတွေကို အစားထိုးလို့ရပါတယ်။ ဒါ့ကြောင့် monday = 11 ဆိုပြီးရေးလို့ရနေပါတယ်။ ရက် ခုနှစ်ရက်ရဲ့ တန်ဖိုးအပြင် အခြားသော တန်ဖိုးတွေကိုလဲ အစားထိုးလို့ရနေပါတယ်။

ဆိုလိုတာက DaysOfWeek.MON ရဲ့ တန်ဖိုးကို ပြောင်းလို့မရပေမဲ့ monday ရဲ့ တန်ဖိုးကတော့ ပြောင်းလို့ရနေပါတယ်။ monday ကို Days Of Week အနေနဲ့ အသုံးပြုလို့ပေမဲ့ မရနိုင်ပါဘူး။ သူဟာ int type ဖြစ်တဲ့ အတွက် int type variable တစ်ခုက လုပ်လို့ရသမျှ လုပ်လို့ရနေမှာဖြစ်တယ်။ monday ++ လို increment တွေ monday — လို decrement တွေလဲ ရေးလို့ရနေပါမယ်။ ဒါတွေက ရက် ခုနှစ်ရက်ရဲ့ တန်ဖိုးအတွင်းမှာရှိနေရင် ပြဿနာမရှိဘူး။ ၁ နဲ့ ၇ ထက်လွန်သွားရင်လဲ သိနိုင်မှာ မဟုတ်ပါဘူး။

တဖန် System.out.println(monday) လို့ရေးရင်လဲ Monday ဆိုပြီးထွက်မလာပဲ သူ့ရဲ့ တန်ဖိုးဖြစ်တဲ့ int တန်ဖိုးသာ ထွက်လာမှာ ဖြစ်ပါတယ်။ အဲ့ဒီလို ပြဿနာမျိုးကို ဖြေရှင်းဖို့အတွက် enum Type ဆိုတာကို Constants တွေကို ရေးသားဖို့ Java SE 5 အရောက်မှာ ထည့်သွင်းလာခဲ့ရခြင်းဖြစ်ပါတယ်။

ဆက်လက်ပြီး Enum ကို ဘယ်လိုရေးရတယ်၊ Enum ကို အသုံးပြုတဲ့ အတွက် အထက်ပါ ပြဿနာတွေကို ဘယ်လို ဖြေရှင်းနိုင်ခဲ့တယ် ဆိုတာကို ဆက်ပြီးလေ့လာသွားကြရအောင်။

After Enum

Java SE 5 အရောက်မှာ Enum ဆိုတဲ့ Static Class အမျိုးအစားတစ်ခုကို ထည့်သွင်းခဲ့ကြပါတယ်။ Enum နဲ့ ရေးသားထားတဲ့ Data Type တွေကို Enum Type လို့ခေါ်ဆိုကြပြီး Constant တွေကို ရေးသားတဲ့ နေရာမှာ အသုံးပြုခဲ့ကြပါတယ်။

Enum Type Class ကို ရေးသားဖို့ဆိုရင် enum ဆိုတဲ့ Keyword ကို အသုံးပြုရပြီး Body ထဲမှာတော့ Enum Type ထဲမှာ အသုံးပြုနိုင်တဲ့ Element တွေကို ရေးသားနိုင်ပါတယ်။

```
public enum DaysOfWeek {
      MON, TUE, WED, THU, FRI, SAT, SUN
}
```

Enum Type ကို ရေးသားတဲ့ နေရာမှာ အထက်ပါအတိုင်း Source File ရဲ့ Top Level မှာရေးသား သီးခြားရေးသားနိုင်သလို Class ရဲ့ Body အတွင်းမှာလဲ Nested Member အနေနဲ့ ရေးသားနိုင်ပါတယ်။ ဆက်လက်ပြီး Enum Type Data တွေကို ဘယ်လို အသုံးပြုရမလဲ ဆိုတာကို လေ့လာသွားကြရအောင်။

```
public class DaysOfWeekDemo {

      public static void main(String[] args) {
            DaysOfWeek monday = DaysOfWeek.MON;
            System.out.println(monday);
      }
}
```

Enum အနေနဲ့ Constant တွေကို ရေးသားထားမယ်ဆိုရင်အဲ့ဒီ Constant ကို Enum Type အနေနဲ့ ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။ အဲ့ဒါကြောင့် monday ဆိုတဲ့ Variable ကို DaysOfWeek လို့သတ်မှတ်လိုက်တဲ့အတွက် monday နေရာကို DaysOfWeek ထဲမှာ သတ်မှတ်ထားတဲ့ တန်ဖိုးတွေအပြင် အခြားသော တန်ဖိုးတွေကို သတ်မှတ်လို့ မရနိုင်တော့ပါဘူး။

Enum Type Variable နေရာကို အဲ့ဒီ Enum ထဲမှာ ကြိုတင်သတ်မှတ်ထားတဲ့ Constant Element တွေအပြင် အခြားသော တန်ဖိုးတွေကို သတ်မှတ်လို့ မရနိုင်တော့ပါဘူး။ ပြီးတော့ System.out.println(monday) လို့ ရေးလိုက်ရင်လဲ Enum ရဲ့ Name ကို ပြပေးနိုင်မှာ ဖြစ်ပါတယ်။

ဒါ့ကြောင့် Enum ကို အသုံးပြုပြီး Constant တွေကိုမှန်ကန်စွာရေးသားလာနိုင်တယ်ဆိုတာကို သိရှိနိုင်ပါတယ်။

What can we do with Enum?

ဆက်လက်ပြီး Enum ကို အသုံးပြုပြီး ဘာတွေလုပ်လို့ရတယ်ဆိုတာကို လေ့လာသွားကြရအောင်။ Enum ကို ရေးသားရတဲ့ အဓိက ရည်ရွယ်ချက်ကတော့ Constant တွေကို ရေးသားဖို့ဖြစ်ပါတယ်။ Enum နဲ့ လုပ်လို့ရတာတွေကတော့ အောက်ပါအတိုင်းဖြစ်ပါတယ်။

- Enum Type တိုင်းမှာ အသုံးပြုနိုင်တဲ့ Method တွေကိုလဲ ပြင်ထားပါတယ်
- Enum Constant တွေဟာလဲ Static Object တွေ ဖြစ်ကြတဲ့ အတွက် Object ကနေ Inheritance လုပ်လာတဲ့ Behavior တွေကို မကြိုက်လို့ရှိရင် Override လုပ်ပြီးရေးနိုင်ပါတယ်
- Enum Constant တွေမှာ State တွေ Behavior တွေကို ရေးသားမယ်ဆိုရင်လဲ ရပါတယ်။
- Abstraction ကိုအသုံးပြုပြီး Abstract Behavior တွေကိုလဲ Enum တွေမှာ ရေးသားလို့ရနိုင်ပါတယ်။
- Interface တွေကိုလဲ Implement လုပ်နိုင်ပါတယ်။
- တဖန် JVM Memory ပေါ်မှာ တစ်ခုထဲရှိတယ်ဆိုတဲ့ ထူးခြားချက်ကို အသုံးပြုပြီး Enum တွေကို Singleton Object အနေနဲ့ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

```
	static void showStudentName(Student student) throws Exception {

		for(Field f : student.getClass().getDeclaredFields()) {

			NullWithBlank anno =
				f.getDeclaredAnnotation(NullWithBlank.class);

			if(null != anno) {
				f.setAccessible(true);
				f.set(student, anno.name());
			}
		}

		System.out.println(student.getName());
	}
```

Default Behavior

Enum တွေအားလုံးအတွက် အလွယ်တကူအသုံးပြုနိုင်အောင် Method တွေကို ပြင်ဆင်ထားပါတယ်။ ပြင်ဆင်ထားတဲ့ Method တွေထဲမှာ Static Method တွေလဲပါသလို Instance Method တွေလဲပါပါတယ်။ ပြီးတော့ Language Feature တွေထဲမှာ အလိုအလျောက် အသုံးပြုနေနိုင်တာတွေလဲရှိပါတယ်။ ဒီနေရာမှာတော့ Enum ဖြစ်တဲ့အတွက် ရနေနိုင်တဲ့ Default Behavior တွေကို လေ့လာသွားကြပါမယ်။

values()

Enum Type တစ်ခုအတွင်းမှာ Declare လုပ်ထားတဲ့ Constant Object တွေအားလုံးလို ရယူလိုတဲ့ အခါမျိုးမှာ အသုံးပြုနိုင်ပါတယ်။ ရေသားထားတဲ့ Constant Object အားလုံးကို ရေးသားထားတဲ့ အစီအစဉ်အတိုင်း Array Object တစ်ခုအနေနဲ့ ရရှိနိုင်မှာဖြစ်ပါတယ်။

```
	public enum Status {

		On, Off;

		public static void main(String[] args) {
			Status [] array = Status.values();
			for(Status s : array) {
				System.out.println(s);
			}
		}
	}
```

အထက်ပါနမူနာထဲမှာလို Status.values() Method ကို Invoke လုပ်လိုက်တဲ့အခါမှာ Status Enum Type ထဲမှာ Declare လုပ်ထားတဲ့ Constant Object တွေကို Array ပုံစံနဲ့ ပြန်ရမှာဖြစ်ပါတယ်။ ဒီ Method ကတော့ အသုံးပြုမည့် Enum Type ထဲမှာ ဘယ်လို Constant တွေရှိတယ်ဆိုတာကို သိချင်တဲ့အခါမျိုးမှာ အသုံးပြုနိုင်ပါတယ်။

valueOf(String name)

ရေးသားထားတဲ့ Enum Type ရဲ့ Constant Name ကနေ Enum Constant ကို ရယူလိုတဲ့အခါမျိုးမှာ အသုံးပြုနိုင်တဲ့ Method ဖြစ်ပါတယ်။

```
public enum Status {

	On, Off;

	public static void main(String[] args) {
		Status status = Status.valueOf("On");
		System.out.println(status);
	}
}
```

Parameter ဖြစ်တဲ့ String Type နဲ့ နာမည်တူတဲ့ Enum Constant ကို ရှာဖွေပေးမှာ ဖြစ်ပြီးတကယ်လို့ မတွေ့ခဲ့ပါက IllegalArgumentException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

name() and ordinal() methods

Enum Constant Object တွေမှာ အသုံးပြုနိုင်တဲ့ Instance Method တွေဖြစ်ကြပါတယ်။ name() method ကတော့ Enum Constant ကိုရေးသားထားတဲ့ နာမည်ကို ပြန်ပေးနိုင်တဲ့ Method ဖြစ်ပြီး ordinal() method ကတော့ Enum Constant ကို ရေးသားထားတဲ့ နေရာကို ဖော်ပြတဲ့ Index တန်ဖိုးကို သိရှိနိုင်တဲ့ Method ဖြစ်ပါတယ်။ Enum တွေရဲ့ Index တွေကို ဖော်ပြတဲ့နေရာမှာ Array တွေလိုပဲ အရင်ဆုံး Index ဟာ သုညကနေ စပါတယ်။

```
public enum Status {

	On, Off;

	public static void main(String[] args) {

		for(Status s : values()) {
			System.out.printf("Ordinal : %d, Name : %s%n",
					s.ordinal(), s.name());
		}
	}
}
```

အထက်ပါ Program ကို Run ကြည့်တဲ့အခါမှာ အောက်ပါအတိုင်း အဖြေထွက်တာကို တွေ့ရပါမယ်။

```
Ordinal : 0, Name : On
Ordinal : 1, Name : Off
```

Enum Constant On ရဲ့ Ordinal ဟာ 0 ဖြစ်ပြီး Off ရဲ့ Ordinal ဟာ 1 ဖြစ်တယ်ဆိုတာကို ဖေါ်ပြနိုင်တာကို တွေ့ရပါတယ်။

Sorting Enum

Enum Type တွေဟာ Serializable Interface နဲ့ Comparable<E> Interface တို့ကို Implement လုပ်ထားပါတယ်။ ဒါ့ကြောင့် Enum Constance တွေကို Serialize လုပ်နိုင်သလို တစ်ခုနဲ့ တစ်ခု Compare လုပ်နိုင်မှာ ဖြစ်ပါတယ်။ အဲ့ဒီလိုပဲ Arrays ဒါမှမဟုတ် Collections Utility Class တွေကို အသုံးပြုပြီး Sort လုပ်နိုင်မှာ ဖြစ်ပါတယ်။

Serializable နဲ့ ပတ်သက်ပြီး Essential Java API ဖြစ်တဲ့ Java IO အခန်းရောက်တဲ့အခါမှာ ဖေါ်ပြသွားမှာ ဖြစ်ပါတယ်။ ဒီနေရာမှာတော့ Comparable<T> နဲ့ Sorting တွေအကြောင်းကို လေ့လာသွားကြရအောင်။

Enum Type တွေဟာ Comparable<T> Interface ကို Implement လုပ်ထားပါတယ်။ Class တစ်ခုဟာ Interface တစ်ခုကို Implement လုပ်ထားပြီဆိုတာနဲ့ Implement လုပ်ထားတဲ့ Interface ထဲမှာ ပါတဲ့ Abstract Method တွေကို Implement လုပ်ပြီးရေးသားပေးရတော့မှာ ဖြစ်ပါတယ်။ ဆိုလိုတာက Interface ဟာ Abstraction ဖြစ်တဲ့ အတွက် ဘာလုပ်လို့ရတယ်ပဲ သတ်မှတ်ပေးထားပါတယ်။ ဘယ်လို လုပ်ရမယ်ဆိုတာကိုတော့ အဲ့ဒီ Interface ကို Implement လုပ်ထားတဲ့ Class တွေမှာ ရေးသားပေးရမှာပါ။

ဒါဖြင့် Comparable Interface က ဘာလုပ်လို့ရတဲ့ Interface လဲ ဆိုတာကို Comparable Interface ထဲမှာပါတဲ့ Abstract Method ကနေလေ့လာကြည့်ကြရအောင်။

```
public interface Comparable<T> {
    public int compareTo(T o);
}
```

အထက်ပါအတိုင်း Comparable Interface ထဲမှာ compareTo(T o) Method တစ်ခုကို ရေးသားထားပါတယ်။ ဆိုလိုတာက Comparable Interface ကို Implement လုပ်သူတိုင်း ဒီ Method ကို ဘယ်လို Compare လုပ်မယ် ဆိုတာကို ရေးသားပေးရမယ် လို့ သတ်မှတ်ထားတာ ဖြစ်ပါတယ်။

ဆက်လက်ပြီး compareTo() method ဟာ ဘာလုပ်တဲ့ Method လဲဆိုတာကို လေ့လာကြည့်ကြရအောင်။ Argument အနေနဲ့ (T o) လို့ယူထားပါတယ်။ Enum Type နဲ့တူတဲ့ အခြားသော Object တစ်ခုကို Parameter အနေနဲ့ လက်ခံမယ်လို့ ရေးထားတာ ဖြစ်ပါတယ်။ Return Type အနေနဲ့ int type တန်ဖိုးတစ်ခုကို ရေးထားပါတယ်။ ဘာ အဓိပ္ပါယ်လဲ။

Enum Type ကနေ compareTo() Method ကို Implement လုပ်ထားတဲ့ ကုဒ်တွေကနေ လေ့လာကြည့်ကြရအောင်။

```
public final int compareTo(E o) {
    Enum<?> other = (Enum<?>)o;
    Enum<E> self = this;
    if (self.getClass() != other.getClass() && // optimization
        self.getDeclaringClass() != other.getDeclaringClass())
        throw new ClassCastException();
    return self.ordinal - other.ordinal;
}
```

Enum Class ထဲမှာ ရေးသားထားတဲ့ compareTo() method ဟာ အထက်ပါအတိုင်းဖြစ်ပါတယ်။ အရင်ဆုံး other နဲ့ Self ကို Declare လုပ်ပါတယ်။ self နဲ့ other မတူဘူးပြီးတော့ self နဲ့ other တို့ရဲ့ Class တွေမတူဘူးဆိုရင် ClassCastException ကို Throw လုပ်ပါတယ်။ ဒါဟာ Compare မလုပ်ခင် သူတို့ရဲ့ Type တူမတူစစ်ဆေးနေတာပဲဖြစ်ပါတယ်။ မတူခဲ့ရင်တော့ ClassCastException ကို ဖြစ်ပေါ်စေမယ်လို့ သိနိုင်ပါတယ်။

တကယ်တမ်း အလုပ်လုပ်နေတာကတော့ အောက်ဆုံး Return Statement ပဲဖြစ်ပါတယ်။ Self ရဲ့ Ordinal ထဲက Other ရဲ့ Ordinal ကို နှုတ်ပြီးရတဲ့ ရလဒ်ကို Return ပြန်တယ်ဆိုတာကို တွေ့ရပါတယ်။ ဆိုလိုတာက Enum တွေကို ဘယ်သူကကြီးပြီး ဘယ်သူက ငယ်တယ်ဆိုတာကို သူတို့တွေရဲ့ Ordinal (Index Number) နဲ့နှုင်းယှဉ်နေတယ်ဆိုတာကို တွေ့ရပါတယ်။ ပြီးတော့လဲ အဲ့ဒီ Method ကို final လို့ ရေးသားထားတဲ့အတွက် ဒီ Method ကို Override လုပ်ပြီးရေးလို့ မရနိုင်ဘူးဆိုတာကို သိနိုင်ပါတယ်။

အထက်ပါကုဒ်ကို ကြည့်ပြီး Enum နှစ်ခု ဘယ်သူကြီးလဲ ဘယ်သူငယ်လဲ ဆိုတာကို နှုင်းယှဉ်တဲ့အခါမှာတနည်းပြောရမယ်ဆိုင် Enum ချင်းတူနေရင်၊ Ordinal ချင်းတူပါမယ်။ အဲ့ဒီအတွက် ရလဒ်ဟာ သုညဖြစ်ပါမယ်။ ဒါ့ကြောင့် compareTo() method ရဲ့ ရလဒ်ဟာ သုညဖြစ်နေရင်အဲ့ဒီ Enum နှစ်ခုဟာတူနေလို့ပဲ ဖြစ်ပါမယ်။

ပြီးတော့ Other Enum ရဲ့ တန်ဖိုးက ကြီးနေရင် အနှုတ် (Minus) တန်ဖိုးရပါမယ်။ ဒါ့ကြောင့် ရလဒ်ဟာ အနှုတ်တန်ဖိုးဖြစ်နေရင် ကိုယ်ကငယ်နေပြီး တစ်ခြား Enum Constant ကကြီးနေပါမယ်။ ပြီးတော့ ရလဒ်ဟာ သုညထက်ကြီးနေရင်လဲ ကိုယ်က ကြီးတယ်ဆိုတာကို သိနိုင်ပါတယ်။

Sorting Algorithm တွေမှာ Element နှစ်ခုဟာ ဘယ်သူကကြီးပြီး ဘယ်သူက ငယ်တယ်ဆိုတာကို ဝေခွဲလို့မရရင် Sort လုပ်လို့ မရနိုင်ပါဘူး။ ဒါကို ကြည့်ချင်းအားဖြင့်ကွန်တော်တို့ အောက်ပါအကြောင်းအရာတွေကို သိရှိနိုင်ပါတယ်။

- Enum တွေဟာ Sort လုပ်လို့ရပါတယ်။
- Sort လုပ်တဲ့အခါမှာ သူတို့ရဲ့ Ordinal အစီအစဉ်အတိုင်း Sort လုပ်သွားပါတယ်
- Enum တွေရဲ့ compareTo() method ဟာ final method ဖြစ်တဲ့ အတွက် Override လုပ်လို့မရနိုင်ပါဘူး
- ကွန်တော်တို့ နှစ်သက်တဲ့ ပုံစံအတိုင်း Sort လုပ်ချင်ရင်တော့ Comparator Object ကို အသုံးပြုရပါမယ်

Sort လုပ်ခြင်း Comparator ကို အသုံးပြုခြင်းနဲ့ ပတ်သက်ပြီး Collections Framework အကြောင်းရောက်တဲ့အခါမှာ ထပ်မံဖေါ်ပြသွားပါမယ်။

Equality Operator

Enum Constant တွေဟာ Constant တွေဖြစ်တဲ့အတွက် တန်ဖိုးတူမတူဆိုတာကို Object ရဲ့ equals() Method သာမက Equality Operator ကို အသုံးပြုပြီးရေးသားနိုင်ပါတယ်။

```
public enum Status {

      On, Off;

      public static void main(String[] args) {

            Status status = Status.On;

            System.out.println(status == On);  // true
            System.out.println(status == Off); // false
            System.out.println(status != On);  // false
            System.out.println(status != Off); // true
      }
}
```

အထက်ပါအတိုင်း main method ထဲမှာ status variable ရဲ့ တန်ဖိုးအနေနဲ့ Status.On လို့ ရေးသားထားပါတယ်။ ဆိုလိုတာက status ရဲ့ တန်ဖိုးဟာ On ဖြစ်ပါတယ်။

ဖော်ပြပါအတိုင်း status နဲ့ အခြားသော Constant တန်ဖိုးများကို Equality Operator ကို အသုံးပြုပြီး စစ်ဆေးသွားနိုင်ပါတယ်။ status == On ရဲ့ တန်ဖိုးဟာ true ဖြစ်မှာ ဖြစ်ပြီး status == Off ရဲ့ တန်ဖိုးဟာ false ဖြစ်ပါမယ်။ အလားတူပဲ status != On ရဲ့ တန်ဖိုးဟာ false ဖြစ်မှာ ဖြစ်ပြီး status != Off ရဲ့ တန်ဖိုးဟာလဲ true ဖြစ်မှာ ဖြစ်ပါတယ်။ ဖော်ပြပါ status variable ကနေ Reference လုပ်နေတဲ့ တန်ဖိုးဟာ On ဖြစ်တဲ့ အတွက် On နဲ့တူတယ်ပြောလာရင် true ဖြစ်မှာ ဖြစ်ပြီး Off နဲ့တူတယ်ဆိုရင် false ဖြစ်ပါမယ်။ ဒီလိုပဲ On နဲ့ မတူဘူးပြောတဲ့အခါမှာ false ဖြစ်မှာ ဖြစ်ပြီး Off နဲ့ မတူဘူးဆိုရင် true ဖြစ်မှာ ဖြစ်ပါတယ်။

Enum Object တွေတူမတူဆိုတာကို အခြားသော Object တွေအတိုင်း equals method ကို အသုံးပြုပြီး စမ်းသပ်နိုင်မှာဖြစ်ပြီး Operator နဲ့ စစ်ဆေးတဲ့ ရလဒ်နဲ့ အတူတူပဲ ဖြစ်ပါမယ်။

Switch Parameter

Enum တွေဟာ Constant တွေဖြစ်တဲ့အတွက် switch statement ရဲ့ Argument တွေအနေနဲ့လဲ အသုံးပြုနိုင်ပါတယ်။ Java SE 5 မတိုင်ခင်က Enum ကိုလဲ အသုံးပြုလို့ မရသေးပါဘူး။ အဲဒီအတွက် Switch Statement ရဲ့ Parameter အနေနဲ့ Primitive Type တွေဖြစ်ကြတဲ့ Character, Byte, Short, Integer တွေကိုပဲ အသုံးပြုနိုင်ခဲ့ပတယ်။

Java SE 5 အရောက်မှာ Enum ကို Java Language မှာ အသုံးပြုလာနိုင်ပြီး Enum Type တွေကို Switch Statement ရဲ့ Parameter အနေနဲ့ အသုံးပြုလာနိုင်ပါတယ်။ Java SE 7 အရောက်မှာတော့ Project Coin နဲ့ String Type ကိုလဲ Switch Statement ရဲ့ Parameter အနေနဲ့ အသုံးပြုလာနိုင်ပါတယ်။

```
public class DaysOfWeekDemo {

      public enum DaysOfWeek {
            MON, TUE, WED, THU, FRI, SAT, SUN
      }

      public void showWorkOrOff(DaysOfWeek day) {
            switch (day) {
            case MON:
            case TUE:
            case WED:
            case THU:
            case FRI:
                  System.out.println(day + " is work day.");
                  break;
            case SAT:
            case SUN:
                  System.out.println(day + " is off day.");
                  break;
            default:
                  break;
            }
      }
}
```

အထက်ပါအတိုင်း Enum Type Variable ဖြစ်တဲ့ day ကို switch ရဲ့ Argument အနေနဲ့ရယူနိုင်ပြီး Case Statement တွေမှာလဲ Enum Constant တွေကို ရေးသား အသုံးပြုနိုင်ပါတယ်။

အထက်ပါ Method ရဲ့ day ဟာ MON ကနေ FRI အထိဆိုရင် Work Day လို့ဖော်ပြနိုင်မှာ ဖြစ်ပြီး SAT နဲ့ SUN ဆိုပါက Off Day လို့ ဖော်ပြပေးနိုင်မှာ ဖြစ်ပါတယ်။

Object's Behaviors

Enum Constant တွေဟာ ကြိုတင်ရေးသားထားရတဲ့ Constant Object တွေဖြစ်ကြပါတယ်။ အဲ့ဒါကြောင့် Enum Constatnt တိုင်းမှာ java.lang.Object ရဲ့ Method တွေကိုလဲ အသုံးပြုလို့ရပါတယ်။ ပြီးတော့ Enum Class ထဲမှာ အခြို့ Method တွေကို Override လုပ်ထားပါတယ်။

equals() method

```
    public final boolean equals(Object other) {
        return this==other;
    }
```

Enum Class ထဲမှာ equals method ကို Override လုပ်ပြီးရေးသားထားပါတယ်။ အတွင်းမှာ equality Operator ကို အသုံးပြုပြီး ရလာတဲ့ ရလဒ်ကို Return ပြန်ထားပါတယ်။ ပြီးတော့ အဲ့ဒီ final modifier ကို အသုံးပြုထားတဲ့အတွက် Enum Class တွေရဲ့ equals method ကို Override လုပ်ပြီး ရေးလို့ရမှာ မဟုတ်တော့ပါဘူး။

clone() method

```
    protected final Object clone() throws CloneNotSupportedException {
        throw new CloneNotSupportedException();
    }
```

Enum Class ထဲမှာ clone method ကိုလဲ Override လုပ်ပြီး ရေးသားထားခဲ့ပါတယ်။ Access Modifier အနေနဲ့ protected Modifier ကို အသုံးပြုထားတဲ့ အတွက် Sub Class တွေကနေသာ အသုံးပြုနိုင်ပြီး ပြင်ပကနေတော့ သုံးလို့ရမှာ မဟုတ်ပါဘူး။ တဖန် Method Body အတွင်းမှာတော့ CloneNoSupportedException ကို throw လုပ်ထားပါတယ်။ အဲ့ဒီအတွက် Enum တွေမှာ Clone Operation ကို Support မလုပ်ဘူးလို့သိနိုင်ပါတယ်။ နောက်ဆုံး final modifier ကို ရေးသားထားတဲ့ အတွက် ဘယ် Enum Type မှာမှ clone method ကို Override လုပ်လို့မရတော့ပါဘူး။ ဒါကြောင့် ဘယ် Enum Type မျိုးမှာမဆို Clone Operation ကို ရေးလို့ရမှာ မဟုတ်တောမပါဘူး။

toString() method

```
    public String toString() {
        return name;
    }
```

Object ရဲ့ toString() method ကိုလဲ Enum Class က Override လုပ်ပြီးရေးသားထားပါတယ်။ Method Body ထဲမှာ name ကို Return ပြန်နေတဲ့ အတွက် Enum တွေရဲ့ toString method ကို Invoke လုပ်ရင် Enum Constant ကိုရေးသားထားတဲ့ Name ကို ပြန်ရမှာ ဖြစ်ပါတယ်။

ပြီးတော့ toString() method ကို final လို့ မရေးထားတဲ့ အတွက် Enum Class တွေထဲကနေ Override လုပ်ပြီး ရေးမယ်ဆိုရင်ရနိုင်ပါတယ်။ ဆက်လက်ပြီး Override လုပ်ကြည့်ကြရအောင်။

Enum Type တွေကနေ Override လုပ်ရင် ပုံစံ ၂ မျိုးနဲ့ ရေးသားလို့ရနိုင်ပါတယ်။ ပထမတစ်မျိုးကတော့ Enum Class Body အောက်မှာ ရေးသားတဲ့ ပုံစံပါ။

```
public enum DaysOfWeek {
      MON, TUE, WED, THU, FRI, SAT, SUN;

      @Override
      public String toString() {
            return String.format("%d : %s", ordinal(), name());
      }
}
```

Class Body အောက်မှာ Override လုပ်ထားတဲ့ Method တွေကို Enum Constant တိုင်းမှာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

```
public static void main(String[] args) {

      for(DaysOfWeek d : DaysOfWeek.values()) {
            System.out.println(d);
      }
}
```

အထက်ပါအတိုင်း Enum Constant တွေကို System.out.println(d) လို့ရေးသားပြီး Run ကြည့်မယ်ဆိုရင် Constant တိုင်းဟာ Override လုပ်ထားတဲ့ toString() Method ကို အသုံးပြုသွားတယ်ဆိုတာကို တွေ့ရပါမယ်။

```
0 : MON
1 : TUE
2 : WED
3 : THU
4 : FRI
5 : SAT
6 : SUN
```

Method တွေကို Override လုပ်တဲ့အခါမှာ Enum Constant တစ်ခုချင်းဆီမှာလဲ ရေးသားလို့ရနိုင်ပါတယ်။

```
public enum DaysOfWeek {
      MON {
            public String toString() {
                  return "Monday";
            }
      }, TUE, WED, THU, FRI, SAT, SUN;

      @Override
      public String toString() {
            return String.format("%d : %s", ordinal(), name());
      }
}
```

အထက်ပါအတိုင်း DaysOfWeek ရဲ့ MON Constant ရဲ့ Body ထဲကနေ toString() Method ကို Override လုပ်လို့ရပါတယ်။ MON ရဲ့ toString() Method ကို အသုံးပြုရင် သူ့ Body ထဲမှာ ရေးထားတဲ့ Method ကိုပဲ အသုံးပြုသွားတာကို တွေ့ရပါမယ်။အရှေ့မှာရေးခဲ့တဲ့ Main Method ကို Run ကြည့်တဲ့အခါမှာ MON Constant တခုပဲ Constant Body ထဲက Override လုပ်ခဲ့တဲ့ toString() Method ကို အသုံးပြုသွားတယ်ဆိုတာကို တွေ့ရပါလိမ့်မယ်။

```
Monday
1 : TUE
2 : WED
3 : THU
4 : FRI
5 : SAT
6 : SUN
```

Writing Object Members

Enum Type တွေမှာလဲ Object တွေရဲ့ Member တွေဖြစ်ကြတဲ့ Instance Variable တွေ Instant Method တွေ ကို ရေးသားအသုံးပြုနိုင်ပါတယ်။

```
public enum DaysOfWeek {

      MON ("Monday") , TUE ("Tuesday"), WED ("Wednesday"),
      THU ("Thursday"), FRI ("Friday"), SAT ("Saturday"),
      SUN ("Sunday");

      private String viewName;

      private DaysOfWeek(String viewName) {
            this.viewName = viewName;
      }

      public String getViewName() {
            return viewName;
      }
}
```

Enum Class တွေမှာ Object တွေရဲ့ State တွေဖြစ်တဲ့ Instance Variable တွေကို ရေးသားလို့ရနိုင်ပါတယ်။ ရေးသားတဲ့ နေရာမှာ နှစ်သက်ရာ Access Modifier ကို ရေးသားလို့ရနိုင်ပါတယ်။ Constructor ကို လဲရေးသားလို့ ရပါတယ်။ ဒါပေမဲ့ Access Modifier အနေနဲ့တော့ private ဒါမှမဟုတ် Default ကိုပဲ အသုံးပြုလို့ရမှာ ဖြစ်ပါ တယ်။

ဒါကြောင့် Enum Constant တွေကို Enum ရဲ့ Body ထဲမှာပဲ ရေးသားလို့ရမှာ ဖြစ်ပါတယ်။ Enum Class အပြင် ကနေ Enum Constant တွေရဲ့ Object တွေကို တည်ဆောက်လို့ ရနိုင်မှာ မဟုတ်ပါဘူး။

ပြီးတော့ Enum Class တွေမှာ Object ရဲ့ Behavior ကို သတ်မှတ်ပေးနိုင်တဲ့ Instance Method တွေကို ရေးသားနိုင်မှာဖြစ်ပြီး Enum Constant တိုင်းကနေ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Inheritance

Enum Class တွေဟာ အခြားသော Class တွေကို Inheritance လုပ်လို့မရသလို အခြား Class တွေကနေလဲ Enum Class တွေကို Inheritance လုပ်လို့မရပါဘူး။ အဲ့ဒီလိုပဲ Enum အချင်းချင်းလဲ Inheritance လုပ်လို့မရနိုင် ပါဘူး။

Enum Type မှာရတဲ့ Inheritance ဆိုလို့ Default Inheritance တွေပဲရှိပါတယ်။ အဲ့ဒါတွေကတော့ java.lang.Enum<E extends Enum<E>> class နဲ့ java.lang.Object Class တို့ပဲ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် Enum Type တွေမှာ မိမိကိုယ်တိုင် နှစ်သက်ရာ Inheritance ကို သုံးလို့ရမှာ မဟုတ်ပါဘူး။

Abstraction

Java Class တွေမှာ Abstraction ကို Abstract Class တွေနဲ့ ရေးသားလို့ရနိုင်သလို Interface တွေနဲ့လဲ ရေးသားလို့ ရနိုင်ပါတယ်။ ဒါပေမဲ့ Enum Class တွေဟာ အခြား Class တွေကို Inherantance လုပ်လို့ မရနိုင်တဲ့ အတွက် Abstract Class တွေကို သုံးလို့ မရနိုင်ပါဘူး။ အဲ့ဒီအစား Enum Class တွေထဲမှာ Abstract Method တွေကို ရေးလို့ရနိုင်ပါတယ်။

ဒါ့ကြောင့် Java မှာ Abstract Method တွေကို Abstract Class တွေ၊ Interface တွေနဲ့ Enum Class တွေမှာ ရေးသားလို့ရနိုင်တယ်လို့ ဆိုနိုင်ပါတယ်။

Abstract Method in Enum Class

```
public enum Status {
      Request{
            @Override
            public boolean isEndOperation() {
                  return false;
            }

      }, Approved {
            @Override
            public boolean isEndOperation() {
                  return true;
            }
      }, Cancel {
            @Override
            public boolean isEndOperation() {
                  return true;
            }
      };

      public abstract boolean isEndOperation();
}
```

အထက်ပါအတိုင်း Status Enum Class အတွင်းမှာ Request, Approved နဲ့ Cancel ဆိုတဲ့ Constant တွေကို ပိုင်ဆိုင်ပါတယ်။ ပြီတော့ အဲ့ဒီ Constant တိုင်းကနေ မတူညီတဲ့ Behavior တွေကို ရရှိစေလိုတယ်ဆိုရင် အထက်ပါအတိုင်း Abstract Method ဖြစ်တဲ့ isEndOperation() Method ကို ရေးသားထားနိုင်ပါတယ်။

Enum Class အတွင်းမှာ Abstract Method ကို ရေးသားလိုက်ပြီဆိုတာနဲ့ Enum Constant တွေတိုင်းကနေ Override လုပ်ပြီးရေးသားပေးရမှာ ဖြစ်ပါတယ်။ ဒီလို ရေးသားခြင်းအားဖြင့် Status Constant တိုင်းကနေ Operation ပြီးသွားပြီလား မပြီးသေးဘူးလားဆိုတာကို သိနိုင်တဲ့ isEndOperation() Method ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

ဒါ့ကြောင့် Enum Constant တစ်ခုချင်းဆီကနေ မတူညီတဲ့ Behavior မျိုးကို လုပ်ဆောင်စေလိုတဲ့ အခါမျိုးမှာ Enum Class ထဲမှာ Abstract Method တွေကို ရေးသား အသုံးပြုစေနိုင်ပါတယ်။

Interface Implementation

Enum Type တွေဟာ Abstract Class တွေကို Extends မလုပ်နိုင်ပေမဲ့ Interafce တွေကိုတော့ Implement လုပ်နိုင်ပါတယ်။

```
public interface FileGenerator {
      File generate();
}

public enum Settings implements FileGenerator{

      ViewNameSetting ("views.properties"),
      TitleSetting ("titles.properties");

      private String fileName;

      private Settings(String fileName) {
            this.fileName = fileName;
      }

      @Override
      public File generate() {
            return new File(fileName);
      }
}
```

အထက်ပါနမူနာထဲမှာ FileGenerator Interface တစ်ခုကို ရေးသားထားပြီး File ကို Return ပြန်ပေးနိုင်တဲ့ generate() method တစ်ခုကို ရေးသားထားပါတယ်။ ပြီးတော့ Setting Enum ကနေ FileGenerator ကို Implement လုပ်ထားပါတယ်။ အဲဒီအတွက် Enum Class Body ထဲမှာ generate() method ကို Override လုပ်ပြီးရေးသားထားပါတယ်။

Override လုပ်တဲ့နေရာမှာ ရှေ့မှာ ဖေါ်ပြခဲ့သလို Constant တစ်ခုချင်းဆီကနေလဲ လိုအပ်သလို Override လုပ်နိုင်မှာ ဖြစ်ပါတယ်။ ကဲ ဒီလို Interface ကို Implement လုပ်တဲ့အတွက် ဘာဖြစ်သွားမလဲ ဆိုတာကို ဆက်ပြီး လေ့လာကြည့်ကြရအောင်။

Settings Enum ဟာ FileGenerator Interface ကို Implement လုပ်ထားတဲ့အတွက် Setting ထဲက Constant တွေကို FileGenerator Object အနေနဲ့ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ တဖန် FileGenerator Interface ကို Implement လုပ်ထားတဲ့ အခြား Class တွေရဲ့ Object တွေနဲ့ အတူတူ Setting Enum Constant တွေကိုလဲ FileGenerator အနေနဲ့ အသုံးပြုလို့ရနိုင်မှာ ဖြစ်ပါတယ်။ ဆိုလိုတာက Interface ကို အသုံးပြုခြင်းအားဖြင့် Enum Constant တွေကို Polymorphic ဖြစ်အောင် ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

```
public class TestFileGenerator implements FileGenerator {
      @Override
      public File generate() {
            return new File("test.txt");
      }
}

public void generateFile(FileGenerator generator) {
      try {
            generator.generate().createNewFile();
      } catch (IOException e) {
            e.printStackTrace();
      }
}

public void test() {
      Settings viewName = Settings.ViewNameSetting;
      Settings title = Settings.TitleSetting;

      TestFileGenerator test = new TestFileGenerator();

      generateFile(viewName);
      generateFile(title);
      generateFile(test);
}
```

အထက်ပါနမူနာထဲမှာ TestFileGenerator Class ဟာ FileGenerator Interface ကို Implement လုပ်ထားပါတယ်။ ပြီးတော့ generateFile(FileGenerator) Method ကတော့ FileGenerator Interface Object တစ်ခုကို Argument အနနေနဲ့ လက်ခံထားပါတယ်။

Settings Enum ကိုကော TestFileGenerator ကိုပါ FileGenerator Interface ကို Implement လုပ်ထားတဲ့အတွက် test() Method ထဲမှာ generateFile() Method ထဲကို Setting Enum တွေကိုလဲ Parameter အနေနဲ့ အသုံးပြုနိုင်သလို TestFileGenerator Object ကိုလဲ အသုံးပြုလို့ရသွားပါတယ်။

ဒီနေရာမှာ လေ့လာနိုင်တာကတော့ Interface တွေကို အသုံးပြုပြီး Enum တွေနဲ့ အခြားသော Object တွေကို တူညီစွာ အသုံးပြုစေနိုင်မယ် ဆိုတဲ့ အချက်ဖြစ်ပါတယ်။

Singleton (Only One) Object

ရှေ့မှာ ဖေါ်ပြထားခဲ့သလို Enum Type တွေမှာ Public Constructor မပါတဲ့ အတွက် Enum Object တွေကို အပြင်ကနေတည်ဆောက်လို့မရပါဘူး။ Enum Constant တွေကို Enum Type ကို ရေးသားကထဲက ကြိုပြီး ရေးသားထားရပါတယ်။ ဒါ့ကြောင့် Enum Constant တွေဟာ JVM Memory ပေါ်မှာ တစ်ခုထဲသော Object တွေ ဖြစ်ကြပါတယ်။

ဒီလို Enum ရဲ့ ထူးခြားချက်ကို အသုံးပြုပြီး JVM Memory ပေါ်မှာ တစ်ခုထဲသာရှိနေစေချင်တဲ့ Singleton Object တွေနေရာမှာ Enum Constant တွေကို အသုံးပြုနိုင်ပါတယ်။

```
public enum BookStore {

      instance;

      private List<String> books;

      private BookStore() {
            books = new ArrayList<>();
      }

      public void add(String book) {
            books.add(book);
      }

      public List<String> getAllBooks() {
            return new ArrayList<>(books);
      }
}
```

အထက်ပါအတိုင်း BookStore Enum Object ကို ရေးသားထားပြီး instance constant တစ်ခုကို ရေးသားထားရင် instance Object ကို အခြားဘယ်နေရာကမှ Object ထပ်ဆောက်လို့ရမှာ မဟုတ်တော့ပါဘူး။ ဒါ့ကြောင့် JVM Memory ထဲမှာ တစ်ခုထဲသာရှိတဲ့ Singleton Object တွေ ဖြစ်လာမှာ ဖြစ်ပါတယ်။

Generics

Generics ဟာ Java SE 5 အရောက်မှာ အသုံးပြုလာနိုင်ခဲ့တဲ့ Programming ရေးသားပုံ ရေးသားနည်းအသစ် ဖြစ်ပါတယ်။ Generics Programming Concept ကို Java မှာ ရေးသားနိုင်အောင် ထည့်သွင်းခဲ့တာလဲ ဖြစ်ပါတယ်။



ဆိုကြပါစို့။ ကျွန်တော်တို့ Cup တစ်ခုအတွက် Class တစ်ခုရေးမယ်။ ပြီးတော့ အဲ့ဒီအထဲမှာ Coffee တွေကို အသုံးပြုမယ်ဆိုပြီး ရေးသားထားခဲ့ရင် အဲ့ဒီ Cup ဟာ Coffee Cup ပဲ ဖြစ်သွားပါမယ်။ Tea Cup တစ်ခုကို လိုချင်ရင် နောက်ထပ် Cup Class တစ်ခုကို ရေးသားရမှာ ဖြစ်ပါတယ်။

Coffee Cup ပဲ ဖြစ်ဖြစ် Tea Cup ပဲဖြစ်ဖြစ် Cup ဖြစ်တဲ့အတွက် လုပ်လို့ရတဲ့ အလုပ်ကတော့ အတူတူပဲ ဖြစ်ပါမယ်။ အသုံးပြုနေတဲ့ Data မတူတာနဲ့ Class တွေကို ရေးနေရပါတယ်။ ဒီလိုနေရာမျိုးမှာ အသုံးပြုနိုင်တဲ့ Data Type ကို Abstraction ဖြစ်အောင် ရေးထားနိုင်မယ်ဆိုရင် အသုံးပြုနေတဲ့ Data Type မတူလို့ဆိုပြီး နောက်ထပ် Class တွေကို ထပ်ပြီးရေးစရာလိုမှာ မဟုတ်တော့ပါဘူး။ အခုလို ပြဿနာတွေကို ဖြေရှင်းနိုင်အောင်ဆိုပြီး Java SE 5 အရောက်မှာ Class တစ်ခုမှာ အသုံးပြုနေတဲ့ Data Type တွေကို Abstraction ဖြစ်အောင် လုပ်ပေးနိုင်တဲ့ Generics ဆိုတဲ့ Programming Concept ကို Java ထဲကို ထည့်သွင်းလာခဲ့ကြပါတယ်။

Before Generics

Container
-data: String
+Container(data: String) +getData(): String +setData(data: String)

နမူနာအနေနဲ့ စဉ်းစားကြည့်ကြရအောင်။ ကျွန်တော်တို့ Container ဆိုတဲ့ Object တစ်ခုရေးနေတယ်။ အဲ့ဒီ Container ထဲမှာ ကျွန်တော်တို့ String တန်ဖိုးတစ်ခုကို သိမ်းထားနိုင်တယ်။

String Argument တစ်ခုပါတဲ့ Constructor တစ်ခုရှိပါမယ်။ ပြီးတော့ Data ကို ပြန်ပေးနိုင်တဲ့ getData ဆိုတဲ့ Method တစ်ခုရှိပါမယ်။ ပြီးတော့မှ အဲ့ဒီ data ကို ပြောင်းပေးနိုင်တဲ့ setData(String) Method တစ်ခုပါဝင်ပါမယ်

Container Class ကို လက်တွေ့ရေးကြည့်ကြရအောင်။

```
public class Container {

	private String data;

	public Container(String data) {
		super();
		this.data = data;
	}

	public void setData(String data) {
		this.data = data;
	}

	public String getData() {
		return data;
	}
}
```

ဒီလို Class မျိုးကို ရေးသားပြီး၊ မကြာခင်မှာပဲ Integer တွေကို သိမ်းထားတဲ့ Container လိုမျိုးလိုချင်တယ်တို့၊ မဟုတ်သေးဘူး Date Class တွေကို သိမ်းပေးထားနိုင်တဲ့ Container လဲလိုချင်တယ်တို့ ဖြစ်လာနိုင်ပါတယ်။ ဒီနေရာမှာ ကျွန်တော်တို့ရွေးစရာလမ်း နှစ်ခုရှိပါမယ်။ တစ်ခုကတော့ လိုချင်တဲ့ Data Type တွေအလိုက် Container Class တွေကို ရေးပေးခြင်းဖြစ်ပါတယ်။ ဟင် ဒါလဲမနိပ်သေးဘူး။ တူညီတဲ့ ကုဒ်တွေကို Class တွေအများကြီးမှာ လိုက်ရေးနေရတယ်။

```
public class Container {

	private Object data;

	public Container(Object data) {
		super();
		this.data = data;
	}

	public void setData(Object data) {
		this.data = data;
	}

	public Object getData() {
		return data;
	}
}
```

ဒီနေရာမှာ တွေးနိုင်တာကတော့ ဘယ် Type မျိုးမဆို အသုံးပြုနိုင်တဲ့ Object ကို သုံးလိုက်ရင်ပြီးတာပဲလို့။

ဟုတ်တယ်။ Object ဟာ Java Class တွေအားလုံးရဲ့ Super Class ဖြစ်နေတယ်။ Class တွေကို ရေးထားထား မရေးထားထား Java Class တွေအားလုံးဟာ java.lang.Object ရဲ့ Sub Class တွေ ဖြစ်ကြတယ်။ ဒါ့ကြောင့် Data Type နေရာမှာ Object လို့သုံးလိုက်ရင် အားလုံးအစဉ်ပြေသွားပြီ။ ထင်ပါလိမ့်မယ်။

Container ထဲကို သွင်းတဲ့အချိန်တွေမှာတော့ အစဉ်ပြေပါလိမ့်မယ်။ Instance Member ဖြစ်တဲ့ Data ရဲ့ Type ကို Object လို့ယူထားတဲ့ အတွက် ဘယ်လို့ Java Object မျိုးမဆိုလာပြီး အစားထိုးလို့ရပါတယ်။ ဒီလိုပဲ Constructor ရဲ့ Argument နေရာမှာလဲ Object လို့ရေးထားတော့ ဘယ်လို Object မျိုးမဆို အသုံးပြုလို့ရပါမယ်။ Setter Method ဖြစ်တဲ့ setData(Object) method ရဲ့ Argument ဟာလဲ Object ဖြစ်တဲ့အတွက် ဘယ်လို Object မျိုးမဆိုလာပြီး ဒီ Method နဲ့ အသုံးပြုလို့ရပါမယ်။ Happy! ဖြစ်သွားပါပြီ။ Class တွေအများကြီးရေးနေရတာတွေကို Object ကို အသုံးပြုပြီး ဖြေရှင်းလိုက်နိုင်ပြီ။ ထင်ကောင်းထင်ပါလိမ့်မယ်။

ဒါပေမဲ့ ပြဿနာက Container ကနေ သိမ်းထားတဲ့ Data ကို အပြင်ကို ပြန်ထုတ်ပေးတဲ့ getData() Method မှာ ဖြစ်ပါတယ်။ Member ဖြစ်တဲ့ data ရဲ့ Type ဟာ Object ဖြစ်တဲ့ အတွက် Return ပြန်တဲ့ Type ဟာလဲ Object ပဲ ဖြစ်နေမှာပါ။ တကယ်သုံးကြည့်တဲ့အခါကျမှ ဒီနေရာက ပြဿနာ ဖြစ်လာပါတယ်။

```
public class ContainerDemo {

	public static void main(String[] args) {
		Container c1 = new Container("String Data");
		Container c2 = new Container(1000);
		Container c3 = new Container(new Date());

		showContents(c1, c2, c3);
	}

	static void showContents(Container ... containers) {

		for(Container c : containers) {
			String data = (String) c.getData();
			System.out.printf("Data is %s%n", data);
		}
	}
}
```

အထက်ပါနမူနာထဲက showContents() Method မှာ Container Class ရဲ့ Verarges Type ကို Argument အနေနဲ့ လက်ခံယူထားပါတယ်။ အဲ့ဒီအတွက် main() method ထဲမှာ showContents(c1, c2, c3) ဆိုပြီး Container Object တွေကို Parameter အနေနဲ့ ပေးလို့ရနေပါတယ်။ ဘာလို့လဲဆိုတော့ c1, c2 နဲ့ c3 တွေဟာ Container Type Object တွေ ဖြစ်တဲ့ အတွက်ဖြစ်ပါတယ်။

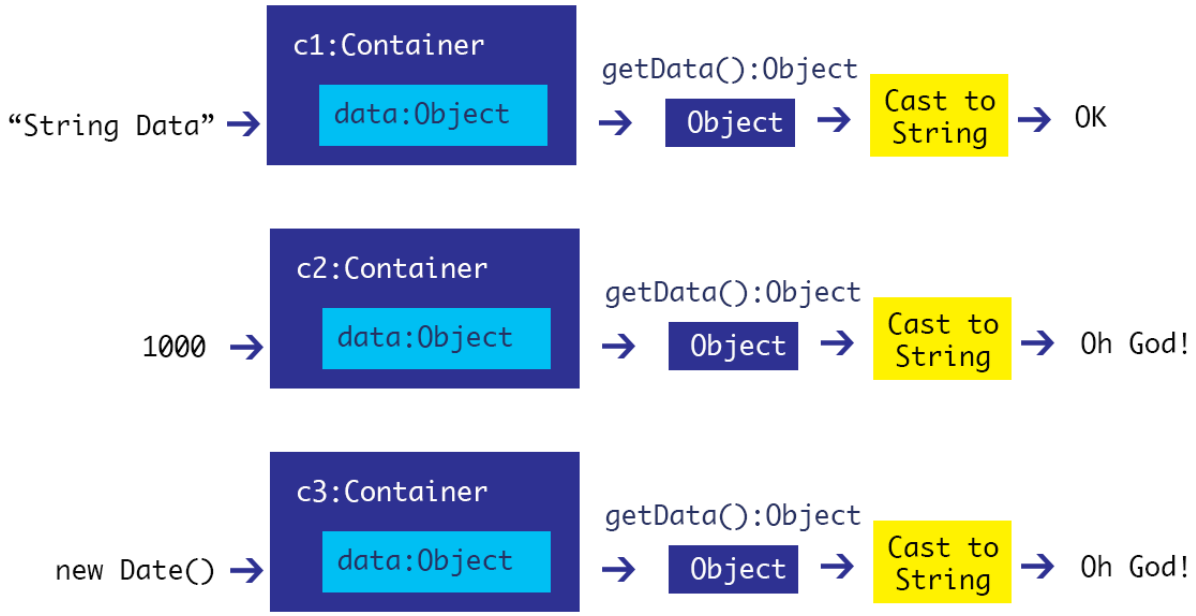
တစ်ခုသတိထားစေချင်တာက အဲ့ဒီ Object တွေကို တည်ဆောက်တဲ့ နေရာမှာ Constructor Argument အနေနဲ့ အသီးသီး String, Integer နဲ့ Date Object တွေကို ပေးထားတယ်ဆိုတဲ့ အချက်ပါပဲ။ ဒီလိုပေးလို့ရတာလဲ Container Class ထဲက Constructor ကို Container(Object data) လို့ရေးသားထားတဲ့ အတွက်ဖြစ်ပါတယ်။ Constructor ရဲ့ Argument မှာ Object ကို အသုံးပြုထားတဲ့ အတွက် ဘယ်လို Type မျိုးမဆို Container Class နဲ့ အသုံးပြုလို့ရနေတာပဲ ဖြစ်ပါတယ်။

Type အားလုံးကို သုံးလို့ရနေတာ မကောင်းဘူးလားလို့ တွေးစရာရှိပါမယ်။ အဖြေထုတ်မလောသေးပဲ ရေးထားတဲ့ ကုဒ်တွေကို ဆက်ပြီး လေ့လာကြည့်ကြအောင်။

အထက်ပါနမူနာထဲက showContents() method ထဲက ကုဒ်တွေကို ကြည့်ကြရအောင်။ Argument ဖြစ်တဲ့ containers ထဲက Container တွေကို for loop နဲ့ Iterate လုပ်ထားပါတယ်။ Container Object တစ်ခုချင်းစီကို for loop ထဲမှာ c ဆိုတဲ့ reference variable နဲ့ Reference လုပ်နိုင်ပါတယ်။

အဲ့ဒီတော့ c ဟာ Container Object ဖြစ်ပါတယ်။ c object ရဲ့ getData() method ကို သုံးတဲ့အခါမှာ ပြဿနာက စတော့တာပါပဲ။ Container Class ရဲ့ Member ဖြစ်တဲ့ data ရဲ့ Type ဟာ Object ဖြစ်တဲ့ အတွက် getData() method ရဲ့ Return Type ဟာလဲ Object Type ဖြစ်နေပါတယ်။ Container Class ရဲ့ Member ဖြစ်တဲ့ data ရဲ့ Type ဟာ Object ဖြစ်နေတဲ့ အတွက် ဘယ်လို Type မျိုးမဆို Container နဲ့ အသုံးပြုနိုင်တယ်။ ဒါပေမဲ့ data ရဲ့ Type ဟာ Object ဖြစ်နေတဲ့ အတွက် getData() ရဲ့ Return Type ဟာလဲ Object ပဲ ဖြစ်နေမှာပါ။

Container Object ရဲ့ getData() method ကို အသုံးပြုတဲ့ အခါမှာ ရလာတဲ့ ရလဒ်ဟာ Object ဖြစ်နေတဲ့ အတွက် တကယ်တမ်း ဘယ်လို Type ဖြစ်တယ်ဆိုတာကို ဒီနေရာမှာ မသိနိုင်တော့ပါဘူး။ ကိုယ်အသုံးပြုလိုတဲ့ Type ဖြစ်အောင်ဆိုရင် ပြန်ပြီး Cast လုပ်ပေးနေရမှာ ဖြစ်ပါတယ်။ Type Casting ဟာ တကယ်ဆိုရင် အရမ်းကို အန္တရာယ်ရှိတဲ့ အလုပ်တစ်ခုဖြစ်ပါတယ်။



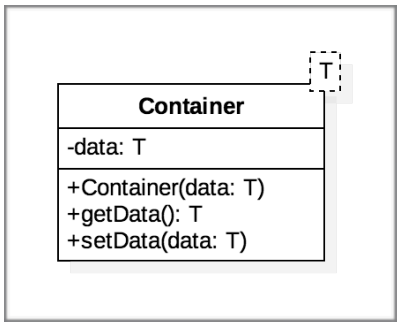
နောက်ကွယ်မှာရှိတဲ့ Object ရဲ့ Type နဲ့ Casting လုပ်လိုတဲ့ Type တို့ မတူဘူးဆိုရင် Runtime ကျမှ Exception ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။ အထက်ပါနမူနာမှာတော့ String အနေနဲ့ အသုံးပြုလိုတဲ့ အတွက် getData() ရဲ့ ရလဒ်ကို (String)getData() လို့ရေးသားပြီး String Type အနေနဲ့ Cast လုပ်နေရပါတယ်။ ဒီနေရာမှာ ပြဿနာ စတော့တာပါပဲ။ c1 မှာတော့ String ကို data အနေနဲ့ Set လုပ်ထားတဲ့ အတွက် Class Cast လုပ်တဲ့ အခါမှာ ပြဿနာမတက်ပေမဲ့ c2 နဲ့ c3 မှာတော့ အသီးသီး Integer နဲ့ Date Type တို့ကို အသုံးပြုထားပါတယ်။ ဒါ့ကြောင့် for loop ရဲ့ ဒုတိယအပတ်မှာ ClassCastException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

အိုကေ၊ ဒီနေရာမှာ အရှေ့မှာ မေးခဲ့တဲ့ မေးခွန်းကို ပြန်ပြီး ဖြေကြည့်ကြရအောင်။ Type အားလုံးကို သုံးလို့ရတိုင်း ကောင်းလားဆိုတာ။ အဖြေက သိမ်းထားတုန်းကတော့ Object ဖြစ်တဲ့ အတွက် အကုန်သိမ်းလို့ရတယ်။ ပြန်ထုတ်တဲ့အခါကျမှ Object အနေနဲ့ပဲ ထုတ်လို့ရတဲ့အတွက် လိုချင်တဲ့ Type ဖြစ်အောင် Cast လုပ်တဲ့ နေရာမှာ ပြဿနာတက်နိုင်တယ်။ အဲ့ဒီတော့ မကောင်းဘူး ဆိုတဲ့ အဖြေထွက်ပါတယ်။ ဒါဟာ Java 5 မတိုင်ခင်က Collections Framework တွေမှာ ဖြစ်နေတဲ့ ပြဿနာဖြစ်ပါတယ်။ ဒီလို ပြဿနာတွေကို ဖြေရှင်းဖို့အတွက် Java SE 5 မှာ Generics ဆိုတဲ့ Programming Concept ကို Java ထဲကို ထည့်ခေါ်လာခဲ့ရတာ ဖြစ်ပါတယ်။ ဆက်လက်ပြီး Generics နဲ့ ဒီလိုပြဿနာတွေကို ဖြေရှင်းခဲ့လဲ ဆိုတာကို လေ့လာသွားကြရအောင်။

After Generics

Java ဟာ အထက်ပါ ပြဿနာတွေကို Generics ဆိုတဲ့ Programming Concept ကိုအသုံးပြုပြီး ဖြေရှင်းခဲ့ပါတယ်။ Generics ဆိုတာဟာလဲ OOP တို့ Functional Programming တို့လို Programming Paradigm တစ်မျိုးဖြစ်ပါတယ်။ Programming Module တွေမှာအသုံးပြုနေတဲ့ Data Type တွေကို Abstraction ဖြစ်အောင် ရေးသားခြင်းအားဖြင့် လုပ်ဆောင်မှုတူတဲ့ Module တွေကို အကြိမ်ကြိမ် မရေးရအောင် အကူအညီပေးနိုင်တဲ့ Programming Concept တစ်မျိုးဖြစ်ပါတယ်။

Generics ရဲ့ Concept ကို စာနဲ့ ရှင်းပြနေတာထက် အထက်ပါ ပြဿနာမျိုးကို Generics နဲ့ ဘယ်လို ဖြေရှင်းသွားတယ်ဆိုတာကို ကြည့်လိုက်တာက အဆပေါင်းများစွာပိုပြီးနားလည်လွယ်ပါလိမ့်မယ်။



Generics ကို အသုံးပြုမယ်ဆိုရင် Class တွေ Method တွေမှာ အသုံးပြုနိုင်တဲ့ Type တွေကို Type Parameter အနေနဲ့ ရေးသား အသုံးပြုနိုင်ပါမယ်။

Class Diagram ထဲမှာလို Container ဆိုတဲ့ Class မှာ အသုံးပြုမည့် Type ကို T ဆိုပြီး type parameter အနေနဲ့ သတ်မှတ်ထားလို့ရပါတယ်။

အဲ့ဒီလို ရေးသားထားတဲ့ Type Parameter T ကို Container Class ရဲ့ Scope ထဲမှာ ရေးသား အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် Member ဖြစ်တဲ့ instance variable data ရဲ့ Type အနေနဲ့လဲ T ကို အသုံးပြုနိုင်ပါတယ်။

ဒီလိုပဲ Constructor Argument နေရာမှာလဲ T ကို အသုံးပြုနိုင်တယ်။ Method တွေရဲ့ Argument တွေမှာကော Return Type တွေမှာ T လို့ရေးသားနိုင်ပါတယ်။ Class ကို ရေးသားတဲ့ အချိန်မှာတော့ ဘာ Type ကို သုံးမှာလဲ ဆိုတာကိုတော့ မသိသေးဘူး။ လောလောဆယ် သဘောတရားအရသာရှိတဲ့ T ဆိုပြီး ရေးသားထားမယ်ပေါ့။

တကယ်တမ်း Object ဆောက်တဲ့ အခါကျမှ T ရဲ့ နေရာမှာ ဘာသုံးမလဲ ဆိုတာကို သတ်မှတ်ပေးရမှာပါ။ Object ဆောက်တဲ့ အခါမှာ သတ်မှတ်ပေးလိုက်တဲ့ Type Parameter ရဲ့တန်ဖိုးကို အဲ့ဒီ Object က အသုံးပြုသွားမှာ ဖြစ်ပါတယ်။ ကုဒ်တွေနဲ့ ရေးကြည့်ကြရအောင်။

```
public class Container<T> {

	private T data;

	public Container(T data) {
		super();
		this.data = data;
	}

	public void setData(T data) {
		this.data = data;
	}

	public T getData() {
		return data;
	}
}
```

Generics ကို အသုံးပြုမယ်ဆိုရင် Type Parameter ကို အသုံးပြုနိုင်ပါတယ်။ Generics ကို Class တွေ၊ Interface တွေနဲ့ Method တွေမှာ ရေးသားနိုင်ပါတယ်။ Generics ကို အသုံးပြုပြီးရေးသားထားတဲ့ Type တွေကို Generics Type လို့ခေါ်ပါတယ်။ Class တွေနဲ့ Interface တွေမှာ Generics ကို အသုံးပြုထားရင် Generics Type လို့ခေါ်နိုင်မှာ ဖြစ်ပါတယ်။ အဲ့ဒီလိုပဲ Method တွေမှာ Generics ကို အသုံးပြုပြီး ရေးသားထားရင် Generics Method တွေလို့ခေါ်နိုင်ပါတယ်။

Generics အကြောင်းကို အသေးစိတ်မသွားသေးခင်၊ Generics ကို အသုံးပြုတဲ့ အတွက် အရှေ့မှာ ဖေါ်ပြခဲ့တဲ့ ပြဿနာကို ဘယ်လို ဖြေရှင်းနိုင်ခဲ့တယ်ဆိုတာကနေ လေ့လာသွားကြရအောင်။ ပြီးတော့မှ Generics နဲ့ ဘာလုပ်လို့ရနိုင်ပြီး နောက်ကွယ်မှာ ဘယ်လို အလုပ်လုပ်နေလဲဆိုတာကို ဆက်ပြီး လေ့လာသွားကြရအောင်။

```
public class Container<T> {

}
```

Container Class ကို ရေးသားတဲ့ နေရာမှာ Generics ကို အသုံးပြုပြီး Type Parameter ကို ရေးသားခဲ့ပါတယ်။ Type Parameter ဆိုတာက Class တွေ Interface တွေကို Declare လုပ်တဲ့အခါမှာ ရေးသားရမှာ ဖြစ်ပါတယ်။ <T> လို့ရေးသားထားတာက Type Parameter ဖြစ်ပါတယ်။ ဒေါင့်ကွင်းထဲမှာ ရေးသားထားတဲ့ T က Type Parameter ဖြစ်ပါတယ်။

ဒီနေရာမှာ ရေးသားထားတဲ့ T ဟာ Container Class မှာ အသုံးပြုနိုင်တဲ့ Data Type ရဲ့ Abstraction ဖြစ်ပါတယ်။ ဒီလိုမျိုး Generices ကို အသုံးပြုပြီး Class တွေမှာ Type Parameter ကို အသုံးပြုနိုင်တဲ့အတွက် Type Abstraction ကို ဖြစ်ပေါ်စေပါတယ်။ ရေးသားထားတဲ့ Abstract Type ကို Class တစ်ခုလုံးမှာ ရေးသား အသုံးပြုနိုင်ပါတယ်။

```
        private T data;

        public Container(T data) {
              super();
              this.data = data;
        }
```

ဒါ့ကြောင့် Type Abstraction ဖြစ်တဲ့ T ကို Container Class ရဲ့ Instance member ဖြစ်တဲ့ data ရဲ့ Type ဖြစ်တဲ့ နေရာမှာလဲ အသုံးပြုနိုင်သလို Constructor Argument မှာလဲ အသုံးပြုနိုင်ပါတယ်း

```
        public void setData(T data) {
              this.data = data;
        }

        public T getData() {
              return data;
        }
```

ဒီလိုပဲ အထက်ပါ Setter Method ရဲ့ Argument တွေရဲ့ Type အနေနဲ့ကော getter method ရဲ့ Return Type မှာပါ Type Parameter ကို အသုံးပြုလို့ရနိုင်ပါတယ်။ တကယ်တန်းပြောရရင် Class တစ်ခုမှာType Parameter ကို ရေးသားထားရင် အဲ့ဒီ Type ကို အဲ့ဒီ Class တစ်ခုလုံးမှာ ရေးသား အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

ဒီလိုမျိုး Type Parameter ကို အသုံးပြုလိုရလို့ ဘာဖြစ်သွားလဲ။ ဒါက တကယ်အသုံးပြုကြည့်တဲ့အခါကျမှ ပိုပြီး သဘောပေါက်မှာ ဖြစ်ပါတယ်။ ဆက်ပြီး ရေးကြည့်ကြရအောင်။ အရင်ဆုံး Generics Type တွေကို Object ဆောက်တဲ့ နေရာကနေ ကြည့်ကြရအောင်။

```
Container<String> c1 = new Container<>("String Data");
Container<Integer> c2 = new Container<>(1000);
Container<Date> c3 = new Container<>(new Date())
```

အထက်ပါအတိုင်း Generics Type တွေကို Object ဆောက်တဲ့ အခါမှာ Type Parameter နေရာမှာ ဘာ Type ကို သုံးမလဲဆိုတာကို သတ်မှတ်ပေးရပါတယ်။ အဲ့ဒီလို သတ်မှတ်ပေးလိုက်တဲ့အခါမှာ Type Parameter ဖြစ်တဲ့ T ဆိုတဲ့ နေရာတွေအားလုံးကို Object ဆောက်တဲ့ အခါမှာ သတ်မှတ်ပေးလိုက်တဲ့ Type က အစားထိုးသွားမှာ ဖြစ်ပါတယ်။

အထက်ပါနမူနာထဲကအတိုင်းဆိုရင် c1 Object အတွက် Type Parameter နေရာမှာ String ကို ရေးသားလိုက်တဲ့ အတွက် c1 Object အတွက် T လို့ရေးထားတဲ့ နေရာတိုင်းဟာ String ဖြစ်သွားမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် Constructor ရဲ့ Argument နေရာမှာ String ဖြစ်တဲ့ “String Data” ကို အသုံးပြုလို့ရမှာ ဖြစ်ပါတယ်။

အဲ့ဒီလိုပဲ Method Argument ဖြစ်တဲ့ နေရာမှာလဲ Generics Type ကို သတ်မှတ် အသုံးပြုနိုင်ပါတယ်။

```
static void showContents(Container<String> ... containers) {

     for(Container<String> c : containers) {
          String data = c.getData();
          System.out.printf("Data is %s%n", data);
     }
}
```

အထက်ပါ နမူနာထဲက showContents() method ရဲ့ Argument နေရာမှာ Container<String> … containers လို့ ရေးသားလိုက်တဲ့အတွက် String တွေကိုသာ အသုံးပြုတဲ့ Container Object တွေကြီး အသုံးပြုလို့ရသွားပါမယ်။ ပြီးတော့ For Loop ထဲမှာရှိတဲ့ Container Object ကလဲ String တွေကိုသာ အသုံးပြုနိုင်တဲ့ Container Obeject ဖြစ်သွားတဲ့ အတွက် getObject() Method ရဲ့ Return Type ဟာလဲ String ကလွဲပြီး တစ်ခြားဖြစ်စရာ အကြောင်းမရှိတော့ပါဘူး။ ဒါ့ကြောင့် ဒီနေရာမှာ String အနေနဲ့ အသုံးပြုနိုင်ဖို့ Type Casting လုပ်စရာမလိုတော့ ပါဘူး။ ဒီနေရာက Generics ကို Java ထဲကို ထည့်ခေါ်လာခဲ့ရတဲ့ အဓိက အချက်ဖြစ်ပါတယ်။

```
7    public static void main(String[] args) {
8        Container<String> c1 = new Container<>("String Data");
9        Container<Integer> c2 = new Container<>(1000);
10       Container<Date> c3 = new Container<>(new Date());
11
12       showContents(c1, c2, c3);
13   }
```

ဒီလိုပဲ String Type ကိုသာ အသုံးပြုတဲ့ showContents() method မှာ Integer ကို အသုံးပြုတဲ့ c2 တို့ Date ကို အသုံးပြုတဲ့ c3 တို့ကို အသုံးပြုလို့ မရနိုင်တော့ပါဘူး။ Generics ဟာ Type Parameter ကို အသုံးပြုပြီး Object တစ်ခုချင်းစီမှာ အသုံးပြုနိုင်တဲ့ Data Type ကိုသတ်မှတ်ပေးနိုင်ပါတယ်။ Object ဆောက်တဲ့ အချိန်မှာ ဘာ Type ကို သုံးမှာလဲ ဆိုတာကို သတ်မှတ်ပေးနိုင်ပြီး Type Parameter နေရာကို အဲ့ဒီ Type က အစားထိုးသွားပါတယ်။

ဒီနေရာမှာ Generics ကို အသုံးပြုတဲ့ အတွက် ပြောင်းသွားတဲ့ အချက် ၄ ချက်ရှိပါတယ်။

- Generics Type Variable ကို Declare လုပ်တဲ့အခါမှာ သတ်မှတ်ထားတဲ့ Type အတိုင်း Type Parameter ကို သတ်မှတ်သွားတဲ့ အတွက် Setter တွေနဲ့ Constractor တွေမှာ သတ်မှတ်ခဲ့တဲ့ Type အပြင် အခြား Type တွေကို သုံးလို့မရတော့ပါဘူး။ Container<String> c1 လို့ရေးလိုက်တဲ့ အတွက် T ရှိသမျှနေရာတွေဟာ String ဖြစ်သွားပါမယ်။

  ဒါ့ကြောင့် Constructor ဟာ Container(String) ဖြစ်သွားပြီး String ကလွဲရင် အခြား Type တွေကို သုံးလို့ မရတော့ပါဘူး။ ဒီလိုပဲ Setter Method ဖြစ်တဲ့ setData(String) ရဲ့ Argument ဟာလဲ String ဖြစ်သွားတဲ့ အတွက် String ကလွဲရင် အခြား Type တွေကို Set လုပ်လို့မရတော့ပါဘူး။

- ဒုတိယအချက်ကတော့ Getter Method တွေဖြစ်တဲ့ getData() method ရဲ့ Return Type ဟာလဲ String ဖြစ်သွားပါတယ်။ အဲ့ဒီအတွက် ရေးထားတဲ့ Variable ရဲ့ Return Type ဟာ သတ်မှတ်ပြီးသားဖြစ်သွားတဲ့ အတွက် Type Casting တွေရေးစရာမလိုတော့ပါဘူး

- တတိယအချက်က Generics Type တွေကို Argument အနေနဲ့ အသုံးပြုနေတဲ့ Method တွေ၊ Constructor တွေမှာလဲ သတ်မှတ်ထားတဲ့ Type ကလွဲရင် အခြား Type တွေကို အသုံးပြုတဲ့ Object တွေကို အသုံးပြုလို့ရတော့ မှာမဟုတ်ဘူး။

  အထက်ပါနမူနာထဲကလို showContents(Container<String>) လို့ အသုံးပြုနိုင်တဲ့ Type ကို Container<String> လို့ရေးလိုက်တဲ့အတွက် String တွေကို အသုံးပြုနေတဲ့ Container တွေကလွဲပြီး အခြား Container တွေကို အသုံးပြုလို့ မရတော့ပါဘူး။ ဒါ့ကြောင့် Integer ကို အသုံးပြုနေတဲ့ c2 Object နဲ့ Date ကို အသုံးပြုတဲ့ c3 Object တို့ကို showContents(Container<String>) method နဲ့ အသုံးပြုလို့မရနိုင်တော့ပါဘူး။

- နောက်ဆုံးအချက်ကတော့ Generics ကို အသုံးပြုရတဲ့ အဓိက ရည်ရွယ်ချက်ဖြစ်ပါတယ်။ Generics ကို အသုံးပြုလို့ရတဲ့ အတွက် Type Abstraction ကို ဖြစ်ပေါ်စေပါတယ်။ အထက်ပါ နမူနာထဲမှာလို့ Container Class လို တစ်ခါရေးလိုက်ရုံနဲ့ အမျိုးမျိုးသော Data Type တွေနဲ့ တွဲပြီး အသုံးပြုသွားနိုင်မှာ ဖြစ်ပါတယ်။ ပြီးတော့လဲ Type Safe ဖြစ်တဲ့ ပုံစံနဲ့ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

ကျွန်တော်တို့ ဆက်လက်ပြီး Generics ကို ဘယ်လို ရေးလို့ရမလဲ၊ ပြီးတော့ Generics ကို အသုံးပြုတဲ့ အတွက် နောက်ကွယ်မှာ ဘယ်လို အလုပ်လုပ်နေလဲ ဆိုတာကို ဆက်ပြီး လေ့လာသွားကြရအောင်။

Writing Generics

ဒီနေရာမှာတော့ Generics ကို ဘယ်လို ရေးရမလဲ၊ ရေးသားထားတဲ့ Generics ကို ဘယ်လို အသုံးချမလဲဆိုတာနဲ့၊ Generics ကို ဘယ်လိုနေရာတွေမှာ ရေးသားနိုင်သလဲ ဆိုတဲ့ အချက်တွေကို လေ့လာသွားကြပါမယ်။

Generics မှာ အဓိက အရေးပါတာကတော့ Type Parameter ပါပဲ။ Type Parameter ဆိုတာဘာလဲ ပြီးတော့ Type Parameter ကို ဘယ်လိုရေးရလဲ၊ ပြီးတော့ Type Parameter ကို ဘယ်လို Customize လုပ်လို့ရလဲ ဆိုတာကို လေ့လာတာဟာ Generics ကို လေ့လာတာနဲ့ အတူတူပါပဲ။

ပြီးတော့မှာ Generics ကို ဖြစ်အောင် Java Ecosystem ရဲ့နောက်ကွယ်မှာ ဘာတွေလုပ်ပေးနေရလဲ ဆိုတာကို သိသွားရင် Generics တွေကို မမှားပဲ ရေးသားနိုင်မှာ ဖြစ်သလို၊ Generics တွေကိုလဲ မှန်မှန်ကန်ကန် အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Type Parameter

Type Parameter ကို <> ထဲမှာ ရေးသားရတာမှန်ပေမဲ့ Class တွေ Interface တွေမှာ ရေးသားပုံ ရေးသားနည်းနဲ့ Method တွေမှာ ရေးသားပုံရေးသားနည်းက မတူပါဘူး။ အရင်ဆုံး Class တွေ Interface တွေမှာ ရေးသားပုံကနေ စပြီး လေ့လာသွားကြရအောင်။

Type Parameter in Interface

```
public interface Repository<T> {
    void insert(T t);
    T findById(Object id);
    List<T> findAll();
}
```

အထက်ပါအတိုင်း Interface ကို ရေးသားတဲ့အခါမှာ Identifier ရဲ့ Name ရဲ့နောက်မှာ <> အတွင်းမှာ အသုံးပြုလိုတဲ့ Type Parameter ကို ရေးသားရမှာ ဖြစ်ပါတယ်။ T က Type Parameter ပါ။ ရေးသားထားတဲ့ Type Parameter T ကို Method Arguments တွေနဲ့ Return Type တွေမှာ အသုံးပြုနိုင်ပါတယ်။ Type Parameter ကို တစ်ချို့သော OOP Programming Language တွေမှာ Type Template လို့လဲခေါ်ဆိုလေ့ရှိပါတယ်။ UML Diagram တွေမှာ Type Parameter တွေကို Type Template ဆိုပြီးရေးသားအသုံးပြုကြတာကို တွေ့ရပါတယ်။

တကယ်လို့ Reference Type Variable ကို Declare လုပ်တဲ့အခါမှ အသုံးပြုမည့် Type ကို Type Parameter နေရာမှာ သတ်မှတ်ရေးသားပေးရမှာ ဖြစ်ပါတယ်။

အထက်ပါအတိုင်း Reference Variable ကို Declare လုပ်တဲ့အချိန်မှာ Customer Type ကို Type Parameter မှာ သတ်မှတ်ရေးသားလိုက်တဲ့အတွက် repo.insert() method မှာလဲ Customer Object ကို အသုံးပြုသွားနိုင်ပါတယ်။ Customer Object ကလွဲရင်လဲ အခြား Type တွေကို အသုံးပြုလို့ရမှာ မဟုတ်တော့ပါဘူး။

```
    public static void main(String [] args) {

        Repository<Customer> repo = ....;

        Customer c = new Customer();

        // set other members

        repo.insert(c);

        Customer c1 = repo.findById(1);

        List<Customer> list = repo.findAll();

    }
```

အဲ့ဒီလိုပဲ repo.findById() method ရဲ့ Return Type ဟာလဲ Customer ဖြစ်သွားတဲ့ အတွက် Type Cast လုပ်စရာမလိုတော့ပါဘူး။ အလားတူ repo.findAll() method ရဲ့ Return Type ဟာလဲ List<Customer> ဖြစ်သွားတာကို တွေ့ရပါတယ်။ ဒါ့ကြောင့် Generics ကို အသုံးပြုချင်းအားဖြင့် Argument တွေကို ကန့်သန့်နိုင်သလို၊ Return Type တွေကိုလဲ Type Cast လုပ်စရာမလိုတော့တာကို တွေ့ရပါတယ်။

Type Parameter In A Class

Class တွေမှာ Generics ကို အသုံးပြုတဲ့ နေရာမှာ တစ်ခုထဲသာ Class ထဲမှာ Generics Type အနေနဲ့ ရေးသားအသုံးပြုတာလဲရှိနိုင်သလို Generics Type Class တွေ Generics Type Interface တွေကို Extends ဒါမှမဟုတ် Implement လုပ်ပြီး ရေးတဲ့ အခါလဲရှိပါလိမ့်မယ်။

Writing Generics Type Class

Concrete Class တွေနဲ့ Abstract Class တွေမှာ Generics ကို အသုံးပြုလိုတဲ့အခါမှာ Interface မှာ ရေးသားခဲ့တုန်းကလိုပဲ Class Name ရဲ့ နောက်မှာ <T> လို့ရေးသားပြီး Type Parameter ကို အသုံးပြုနိုင်ပါတယ်။ အသုံးပြုလိုတဲ့အခါ Reference Variable ကို Declare လုပ်တဲ့အခါကျမှ Type ကို သတ်မှတ်ရေးသားပေးရမှာ ဖြစ်ပါတယ်။

အဲ့ဒီအချိန်မှာ ရေးသားပေးလိုက်တဲ့ Type က တောက်လျောက် Reference Variable ကနေ အသုံးပြုသွားနိုင်မှာ ဖြစ်ပါတယ်။ ဥပမာအားဖြင့် CustomerRepo<Customer> လို့ရေးသားလိုက်ရင် အဲ့ဒီ Class ထဲမှာ ရေးသားထားတဲ့ Type ဖြစ်တဲ့ Customer Type က Type Parameter ဖြစ်တဲ့ T ရှိတဲ့ နေရာတွေကို အစားထိုးသွားမှာ ဖြစ်ပါတယ်။

Constructor Argument မှာရဲ့တဲ့ Type ကော Method Argument နေရာမှာရှိတဲ့ Type တွေပါ Customer ဖြစ်သွားတဲ့အတွက် Customer ကလဲပြီး အဲ့ဒီ Object နဲ့ အသုံးပြုလို့ရမှာ မဟုတ်တော့ပါဘူး။ ဒီလိုပဲ Method Return Type မှာ သုံးထားတဲ့ Type ဟာလဲ Customer ဖြစ်သွားတဲ့ အတွက် တစ်ခြား Type တွေအနေနဲ့ Cast လုပ်စရာမလိုတော့ပါဘူး။

```
public class CustomerRepo<T> {

      private List<T> list;

      public CustomerRepo(T ... data) {
            // codes
      }

      public T findById(int id) {
            return null;
      }

      public void add(T t) {
            // codes
      }
}
```

Implementing Generics Type Interface

Class တစ်ခုက Generics Type Abstract Class ကို Extends လုပ်တဲ့အခါ၊ ဒါမှမဟုတ် Generics Type Interface တစ်ခုကို Implement လုပ်တဲ့အခါမျိုးမှာ ကျွန်တော်တို့အနေနဲ့ အနေအထား နှစ်မျိုးကို ရွေးချယ်ရေးသားနိုင်ပါတယ်။ ဒီနေရာမှာတော့ Generics Type ကို Implement လုပ်တဲ့အခါမျိုးမှာ ဘယ်လို လုပ်မလဲ ဆိုတာကို နမူနာအနေနဲ့ ရေးသားသွားပါမယ်။

ရှေ့မှာ ဖော်ပြခဲ့တဲ့ Repository Interface ကိုပဲ နမူနာအနေနဲ့ အသုံးပြုသွားပါမယ်။

```
public interface Repository<T> {
    void insert(T t);
    T findById(Object id);
    List<T> findAll();
}
```

Generics Type Interface ကို Class တစ်ခုက Implement လုပ်တော့မယ်ဆိုရင် ရွေးချယ်ရမှာက လက်ရှိရေးသားမည့် Class မှာ Generics ကို ဆက်ထားမလား ဒါမှမဟုတ် ဒီနေရာမှာ Type ကို သတ်မှတ်သွားမလားဆိုတာပါပဲ။ ပထမဆုံး Generics ကို ဆက်ပြီးသုံးမယ်ဆိုရင် ဘယ်လို ရေးမယ်ဆိုတာကို လေ့လာကြည့်ကြရအောင်။ လက်ရှိရေးသားတဲ့ Class ကို Generics အနေနဲ့ ဆက်ပြီး သုံးလိုတယ်ဆိုရင် Class Name နောက်မှာ Type Parameter ကို ယူပြီး Interface ဖြစ်တဲ့ နေရာမှာလဲ အဲ့ဒီ Type ကိုပဲ ရေးသားပြီး Class တစ်ခုလုံးမှာ Type Parameter ကို အသုံးပြုနိုင်ပါတယ်။ Reference Variable ကို ရေးတဲ့အခါကျမှ Type ကို သတ်မှတ်ပေးရမှာပါ။

နောက်တစ်ခုရွေးချယ်စရာကတော့ လက်ရှိ Class ကို ရေးသားတဲ့ အခါမှာ Type ကို သတ်မှတ်လို့ရသွားပြီလား ဆိုတဲ့ အနေအထားပဲ ဖြစ်ပါတယ်။ ဒါဆိုရင်တော့ လက်ရှိရေးသားတဲ့ Class ကို Generics အနေနဲ့ ရေးသားစရာမလိုတော့ပါဘူး။

```
public class CustomerRepository implements Repository<Customer>{

	@Override
	public void insert(Customer t) {
		// TODO Auto-generated method stub
	}

	@Override
	public Customer findById(Object id) {
		// TODO Auto-generated method stub
		return null;
	}

	@Override
	public List<Customer> findAll() {
		// TODO Auto-generated method stub
		return null;
	}
}
```

အထက်ပါနမူနာထဲမှာ CustomerRepository Class ဟာ Generics Type ဖြစ်တဲ့ Repository ကို Implement လုပ်ထားပါတယ်။ ဒီနေရာမှာ CustomerRepository ဟာ Customer Class ကို သုံးမယ်ဆိုတာကို သတ်မှတ်ပြီးသား ဖြစ်နေတဲ့ အတွက် Repository<Customer> လို ရေးသားလိုက်ပါတယ်။ ဒါ့ကြောင့် Repository Interface ရဲ့ T ရှိသမျှနေရာတွေအားလုံးကို Customer က အစားထိုးသွားတာကို တွေ့ရပါတယ်။

```
public static void main(String[] args) {
	CustomerRepository cRepo = new CustomerRepository();
	Repository<Customer> repo = cRepo;
}
```

ဒီလိုရေးသားလိုက်တဲ့အတွက် CustomerRepository Object ကို CustomerRepository အနေနဲ့လဲ အသုံးပြုနိုင်သလို Repository<Customer> အနေနဲ့လဲ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

နောက်တစ်မျိုးရှိနိုင်တာက Generics Type Interface လုပ်တဲ့ Class မှာ Type ကို မသတ်မှတ်သေးနိုင်တဲ့ အနေအထားမျိုး ဖြစ်ပါတယ်။

```
public class BaseRepository<T> implements Repository<T> {

	// Override methods
}
```

ဒီလိုအခါမျိုးတွေမှာ ဆိုရင်တော့ Class Declaration ဖြစ်တဲ့ BaseRepository မှာ Type Parameter ဖြစ်တဲ့ <T> ကို ရေးသားပြီး Interface မှာလဲ T Type ကို ဆက်ပြီး အသုံးပြုနိုင်ပါတယ်။

Type Parameter in a Method

Generics တွေကို Class တွေလို Interface တွေလို Type အနေနဲ့ ရေးသားနိုင်တဲ့ နေရာတွေမှာလဲ ရေးသားနိုင်သလို Method တွေမှာလဲ ရေးသားနိုင်ပါတယ်။ Type တွေမှာ Generics တွေကို သုံးထားရင် Generics Type တွေလို့ခေါ်ဆိုပြီး Method တွေမှာ Generics တွေကို အသုံးပြုထားရင် Generics Method တွေလို့ ခေါ်ဆိုလေ့ရှိပါတယ်။ Static Method တွေကော Instance Method တွေမှာပါ Generics ကို အသုံးပြုနိုင်ပါတယ်။

```
static<T> T hello(T t) {
      T local = t;
      // other codes
      return local;
}
```

Static Method တွေမှ Generics ကို အသုံးပြုမယ်ဆိုရင် static modifier ရဲ့ အနောက် Return Type ရဲ့ အရှေ့မှာ Type Parameter ကို ရေးသားရပါမယ်။ Method တစ်ခုမှာ Type Parameter ကို သတ်မှတ်လိုက်ပြီဆိုတာနဲ့ အဲ့ဒီ Type Parameter ကို Methd ရဲ့ Return Type မှာလဲအသုံးပြုနိုင်သလို Methd ရဲ့ Argument နေရာမှာလဲ အသုံးပြုနိုင်ပါမယ်။ အဲ့ဒီအပြင် Method ထဲမှာရှိတဲ့ Local Variable တွေရဲ့ Type အဖြစ်လဲ အသုံးပြုနိုင်ပါတယ်။

```
public static void main(String[] args) {
      String str  = hello("Hello Java");
      System.out.println(str);
}
```

အထက်ပါအတိုင်း hello method ကို Invoke လုပ်တဲ့ အခါမှာ အသုံးပြုတဲ့ Type အတိုင်းကျန်တဲ့ Type တွေလဲ သတ်မှတ်သွားတာဖြစ်ပါတယ်။ ဆက်လက်ပြီး Instance Method တွေမှာလဲ Generics ကို အသုံးပြုကြည့်ကြပါမယ်။

```
public class Demo {

      <T> T hello(T t) {
            T local = t;
            return local;
      }

      public static void main(String[] args) {
            String str  = new Demo().hello("Hello Java");
            System.out.println(str);
      }
}
```

Instance Method တွေမှာဆိုရင်တော့ Return Type ရဲ့ ရှေ့မှာ Type Parameter ကို ရေးသားနိုင်ပါတယ်။ Instance Method တွေဖြစ်တဲ့ အတွက် Object တွေကနေ အဲ့ဒီ Method ကို ခေါ်ယူနိုင်မှာဖြစ်ပါတယ်။ ကျန်တာတွေကတော့ Static Method တွေနဲ့ အတူတူပဲ ဖြစ်ပါတယ်။

Multiple Type Parameters

Generics Type တွေ Generics Method တွေကို ရေးသားတဲ့ နေရာမှာ Type Parameter ကို တစ်ခုထက်မက သတ်မှတ် အသုံးပြုလို့ရနိုင်ပါတယ်။ အဲ့ဒီလိုအခါမျိုးမှာ Type Parameter တစ်ခုနဲ့ တစ်ခုကြားမှာ Comma (,) ကို ခံပြီး ရေးသားပေးရမှာ ဖြစ်ပါတယ်။ နမူနာအနေနဲ့ Type A ကို ယူပြီး Type B ကို ပြောင်းပေးနိုင်တဲ့ Converter Interface ကို ရေးသားကြည့်ပါမယ်။

```
interface Converter<A,B> {
      B convert(A a);
}

class IntToStringConverter implements
            Converter<Integer, String>() {

      @Override
      public String convert(Integer a) {
            return a.toString();
      }
}
```

အထက်ပါနမူနာထဲမှာတော့ Converter Interface မှာ <A, B> လို့ရေးသားထားတဲ့ အတွက် Type Parameter အနေနဲ့ A နဲ့ B ကို အသုံးပြုမယ်လို့ အဓိပ္ပါယ်ရပါတယ်။ တဖန် convert() method ရဲ့ Return Type နေရာမှာ B ကို အသုံးပြုထားပြီး Argument နေရာမှာ A ကို အသုံးပြုထားပါတယ်။

ဆက်လက်ပြီး IntToStringConverter Class ကနေ Converter Interface ကို Implement လုပ်တဲ့ နေရာမှာ <Integer, String> ဆိုပြီး Type တွေကို သတ်မှတ်ပေးထားပါတယ်။ ဒါ့ကြောင့် A နေရာတွေက Integer ဖြစ်သွားပြီး B နေရာတွေက String ဖြစ်သွားပါတယ်။

တစ်ခုသတိထားဖို့လိုအပ်တာက Type Parameter တွေနေရာမှာ Primitive Type တွေကို ရေးသား အသုံးပြုလို့ မရနိုင်ပါဘူး။ ဒါ့ကြောင့် Primitive Type ဖြစ်တဲ့ int ကို မသုံးပဲ Wrapper Type ဖြစ်တဲ့ Integer ကို အသုံးပြုထားတာ ဖြစ်ပါတယ်။

```
public class ConvertDemo {

      public static void main(String[] args) {
            Converter<Integer, String> c =
                                    new IntToStringConverter();
            String str = c.convert(20);
      }
}
```

ဒါ့ကြောင့် Converter<Integer, String> Type ဖြစ်တဲ့ c Reference Variable ကနေ IntToStringConverter Object ကို Reference လုပ်နေနိုင်တာ ဖြစ်ပါတယ်။

Bounded Type in Writing Generics

Generics အကြောင်းကို လေ့လာတဲ့ နေရာမှာ Generics တွေကို ရေးသားတဲ့ အပိုင်းနဲ့ Generics တွေကို အသုံးပြုပြီး ရေးသားထားတဲ့ API တွေကို အသုံးပြုတဲ့ အပိုင်းဆိုပြီး ရှိပါတယ်။ Java ကို စတင်လေ့လာသူတွေ အနေနဲ့ Generics တွေကို ဘယ်လို ရေးမယ်ဆိုတာထက် ဘယ်လိုသုံးမယ်ဆိုတာကို သိရင်လုံလောက်ပါတယ်။ ကျွန်တော် ရှေ့မှာ ရေးသားခဲ့တဲ့ အပိုင်းတွေက Generics တွေကို ဘယ်လိုရေးမလဲ ဆိုတဲ့ အပိုင်းဖြစ်ပါတယ်။

ကျွန်တော်တို့ Generics တွေမှာ အသုံးပြုတဲ့ Type Parameter တွေကို ဘယ်လိုရေးမယ်ဆိုတာကို သိသွားပြီးတဲ့ နောက်မှာ Type Parameter တွေမှာ အစားထိုးအသုံးပြုနိုင်တဲ့ Type တွေကို ဘယ်လို ကန့်သတ်မယ်ဆိုတဲ့ အပိုင်းကို ဆက်ပြီး လေ့လာသွားကြရအောင်။

Generics Type တွေ Generics Method တွေကို ရေးသားတဲ့ နေရာမှာ Type Parameter တွေကို အသုံးပြုနိုင်ပါတယ်။ ဒါပေမဲ့ ဘယ်လို Type တွေပဲ အသုံးပြုနိုင်တယ်ဆိုတာကို သတ်မှတ်မထားခဲ့ပါဘူး။ ဒါ့ကြောင့် ကျွန်တော်တို့ ရေးထားတဲ့ Generics Type တွေမှာ ကြိုက်တဲ့ Type ကို အသုံးပြုပြီးရေးသားနေနိုင်မှာ ဖြစ်ပါတယ်။

ဆိုကြပါစို့ ကျွန်တော်တို့က Database Operation တွေကို Generics အနေနဲ့ ရေးသားချင်တယ်။ ဘာလို့လဲဆိုတော့ Database Operation တွေဟာ အခြေခံအားဖြင့် လုပ်ဆောင်မှုတွေတူညီပြီး အသုံးပြုနေတဲ့ Data Type တွေပဲ မတူကြတာပါ။ Database အသုံးပြုပုံဆိုလို့ကတော့ CRUD ဆိုတဲ့ Create, Reference, Update နဲ့ Delete တွေပဲမျှားပါတယ်။ မတူတာကတော့ Table တွေပဲ ဖြစ်ပါမယ်။ အဲ့ဒါတွေကို အသုံးပြုဖို့ Interface တစ်ခုကို ရေးချင်တယ်ဆိုရင် ဒီလို ရေးရမယ်ထင်ပါတယ်။

```
public interface Repository<T> {
      void insert(T t);
      T findById(Object id);
      List<T> getAll();
      int update(T t);
      int delete(T t);
}
```

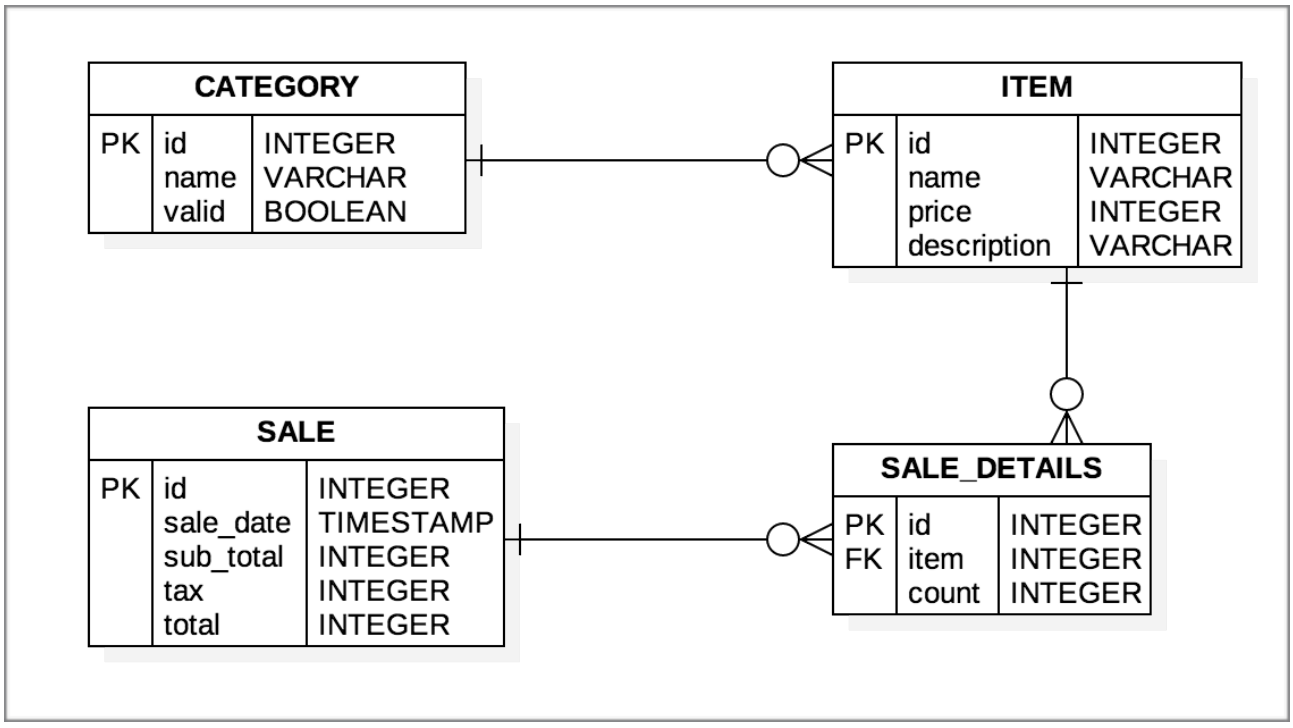
ဟုတ်ပြီ။ အထက်ပါအတိုင်း Repository ဆိုတာကတော့ Database ထဲက Table တွေရဲ့ CURD Operation ကို ဆောင်ရွက်ပေးနိုင်တဲ့ Interface ကို သတ်မှတ်ပေးထားတာပဲ ဖြစ်ပါတယ်။ ဒီနေရာမှာ Type Parameter ကို T လို့ သတ်မှတ်ထားတဲ့ အတွက် ကျွန်တော်တို့တွေက Repository ကို Type အားလုံးနဲ့ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

ဒါပေမဲ့ဗျာ။ သေသေခြာခြာစဉ်းစားကြည့်ရင် ဒီ Repository တွေနဲ့ အသုံးပြုစေချင်တာက Database ထဲက Table တွေကို ကိုယ်စားပြုတဲ့ Java Class တွေပဲ အသုံးပြုစေချင်တာပါ။ Repository နဲ့ Date တွေ၊ String တွေ Interger တွေကို မသုံးစေချင်ပါဘူး။ ဒီလိုနေရာမျိုးမှာ Bounded Type ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

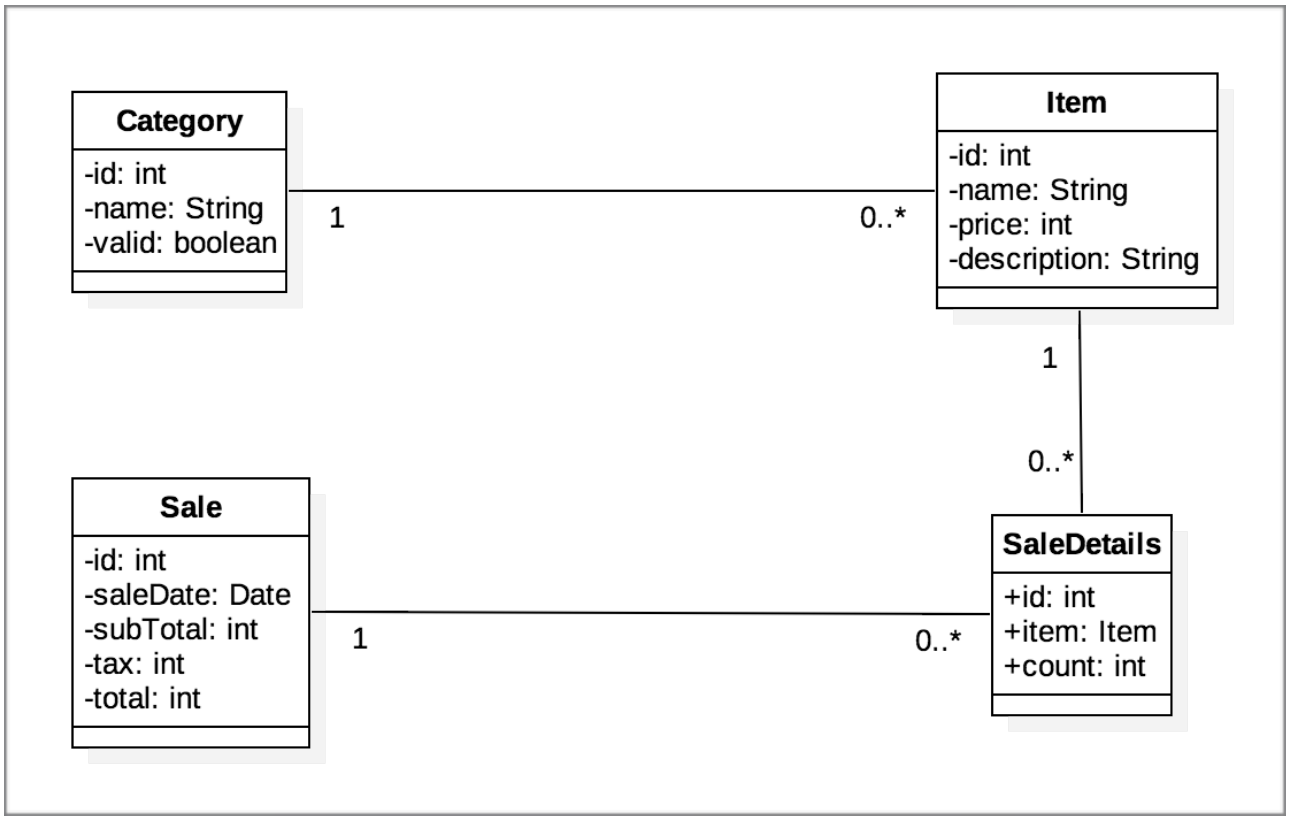
Bounded Type ဆိုတာကတော့ Type Parameter တွေကို ဘယ်အထိသုံးနိုင်တယ်ဆိုတာကို သတ်မှတ်ပေးနိုင်တဲ့ ရေးသားပုံရေးသားနည်းပဲဖြစ်ပါတယ်။ Type Parameter တွေနေရာမှာ အမြင့်ဆုံးဘယ်အထိ အသုံးပြုနိုင်တယ်၊ ပြီးတော့ အနိမ့်ဆုံးဒီလောက်ကမှ စပြီး အသုံးပြုနိုင်လိမ့်မယ်ဆိုတာကို သတ်မှတ်ပေးနိုင်တဲ့ ရေးသားပုံရေးသားနည်းတွေ ဖြစ်ပါတယ်။ Type Parameter တွေရဲ့ နေရာမှာ အမြင့်ဆုံး ဘယ်လောက်အထိရေးသား အသုံးပြုနိုင်တယ်ဆိုတာကို သတ်မှတ်ပေးနိုင်တာကတော့ Upper Bounded ဖြစ်ပြီး ဒီလောက်မှမရှိရင်တော့ အသုံးပြုလို့မရပါဘူးဆိုတာကို သတ်မှတ်ပေးနိုင်တာကတော့ Lower Bounded ဖြစဒ်ပါတယ်။

Generics တွေကို ရေးသားတဲ့နေရာမှာတော့ Upper Bounded တစ်မျိုးထဲကိုသာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ကျွန်တော်တို့ ဆက်လက်ပြီး Upper Bounded တွေကို လက်ရှိရေးသားထားတဲ့ Interface မှာ အသုံးပြုလို့ရမယ်၊ ပြီးတော့ အသုံးပြုတဲ့ အတွက် ဘယ်လို အကျိုးရှိသွားတယ်ဆိုတာကို လေ့လာသွားကြရအောင်။

ကျွန်တော်တို့ POS အရောင်း System တစ်ခုကို ရေးသားနေကြတယ်ဆိုကြပါစို့။ ကျွန်တော်တို့ အသုံးပြုမည့် Database System ရဲ့ ERD က အောက်ပါအတိုင်းဖြစ်နေပါတယ်။



ဒါ့ကြောင့် Repository Interface ကနေ အသုံးပြုချင်တာကလဲ အဲ့ဒီ Data တွေပဲ ဖြစ်ပါတယ်။ ဒါဆိုရင် ကျွန်တော်တို့ ဘယ်လို လုပ်ကြမလဲ။ ဘယ်လို Class တွေကို ရေးကြမလဲ ဆိုတာကို Class Diagram နဲ့ လေ့လာကြည့်ကြရအောင်။

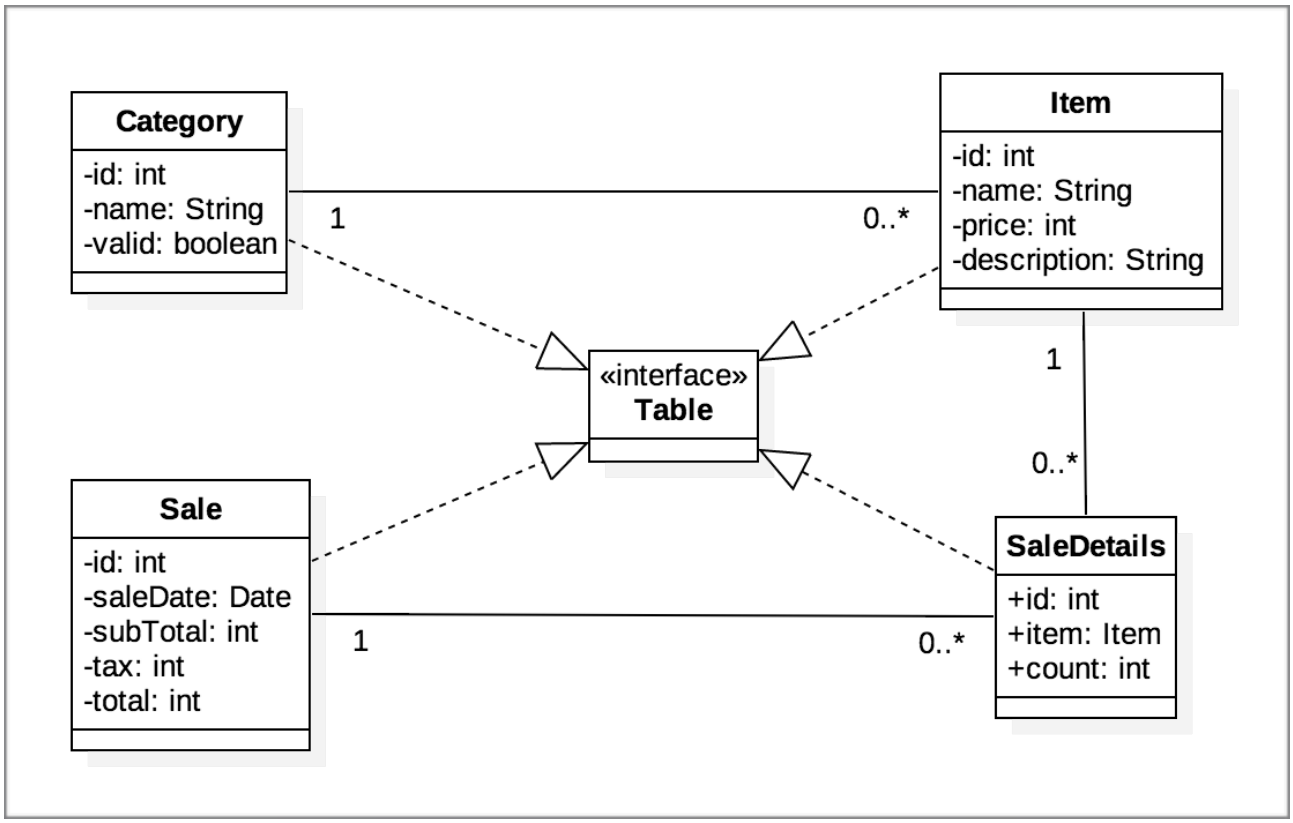


အထက်ပါအတိုင်း Table တွေကို ကိုယ်စားပြုတဲ့ Class တွေကို ရေးသားမှာ ဖြစ်ပါတယ်။

ဒီနေရာမှာ ပြန်ပြီး စဉ်းစားကြည့်ကြရအောင်။ ကျွန်တော်တို့ ရေးခဲ့တဲ့ Repository ဆိုတာက Database ထဲက Table ထဲက Data တွေကို CRUD လုပ်ချင်ရုံသက်သက်ပဲ။ Table မဟုတ်တဲ့ Data တွေနဲ့ လာသုံးနေရင် အခက်တွေ့ပါမယ်။ ဒါဆိုရင် ကျွန်တော်တို့ ဘယ်လို လုပ်ကြမလဲ။

ကျွန်တော်တို့ Database ထဲက Table တွေကို ကိုယ်စားပြုတဲ့ Class ကို သတ်မှတ်နိုင်ရင် ပြီးတာပါပဲ။ အဲ့ဒီ Type နဲ့ အဲ့ဒီ Type ရဲ့ Sub Class တွေသာ အသုံးပြုနိုင်တဲ့ Generics Type ကို ကျွန်တော်တို့ Bounded Type ကို အသုံးပြုပြီး သတ်မှတ်နိုင်ပါတယ်း

ဒါကရှင်းပါတယ်။ ကျွန်တော်တို့ OOP အခန်းမှာကထဲက လေ့လာပြီးသားပါ။ Class တွေကို အုပ်စုဖွဲ့ပြီး အသုံးပြုဖို့ အတွက် Abstraction ကို အသုံးပြုလို့ ရနိုင်ပါတယ်။ Abstraction ကို သုံးရင် IS A Relationship ကို ဖြစ်ပေါ်စေပြီး Class တွေကို တူရာစုပြီး အသုံးပြုနိုင်မှာဖြစ်ပါတယ်။ Abstraction ကိုဖြစ်ဖို့ Abstract Class တွေနဲ့လဲ ရေးသားနိုင်သလို Interface ကို အသုံးပြုပြီးလဲ ရေးသားနိုင်ပါတယ်။ Abstract Class ကို သုံးသင့်သလား Interface ကို သုံးသင့်သလားဆိုတာကတော့ OOP ရဲ့ Abstraction အခန်းမှာ အသေးစိတ် ဖေါ်ပြပြီး ဖြစ်ပါတယ်။ ဒီနေရာမှတော့ Interface ကိုပဲ Marker Interface အနေနဲ့ အသုံးပြုသွားပါမယ်။



ဒီနေရာမှာ Table ဆိုတဲ့ Interface ကို ဘာ Method မှမပါပဲရေးသားပါမယ်။ ပြီးတော့မှ Category, Item, Sale နဲ့ SaleDetails Class တွေကနေ Implement လုပ်ပါမယ်။ Class တစ်ခုဟာ Interface ကို Implement လုပ်လိုက်ခြင်းအားဖြင့် Class နဲ့ Interface ကြားမှာ IS - A Relationship ကို ဖြစ်ပေါ်စေပါတယ်။

Category က Table ကို Implement လုပ်လိုက်ခြင်းအားဖြင့် Category ကို Table အနေနဲ့ အသုံးပြုသွားနိုင်ပါတယ်။ ကျွန်တော်တို့ ရေးသားထားတဲ့ Repository က Database Operation တွေကို လုပ်နိုင်တဲ့ Class တွေကိုပဲ အသုံးပြုချင်တယ်။ Table ဆိုတဲ့ Interface ကလဲ Database ထဲက Table တွေကိုပဲ ကိုယ်စားပြုပါတယ်။

ဒီလိုအနေအထားမျိုးမှာလဲ Interface ကို အသုံးပြုနိုင်တယ်။ Serializable လို Marker Interface လိုမျိုး Table Interface ကို Implement လုပ်ထားတဲ့ Class မှန်သမျှကို Database ထဲက Table ကို ကိုယ်စားပြုနိုင်တဲ့ Class တွေဖြစ်ကြောင်း သတ်မှတ်ပေးနိုင်ပါတယ်။ ဘယ် Class တွေဟာ Table တွေကို ကိုယ်စားပြုနိုင်တယ်ဆိုတာကို သတ်မှတ်ပြီးရင် Repository ကို ဒီ Class တွေကိုသာသုံးနိုင်တဲ့ Interface ဖြစ်အောင် ရေးပေးရုံပါပဲ။

```
import java.util.*;

interface Table {}

public interface Repository<T extends Table> {
      void insert(T t);
      T findById(Object id);
      List<T> getAll();
      int update(T t);
      int delete(T t);
}
```

အထက်ပါ နမူနာထဲက Repository ဟာ Bounded Type ကို အသုံးပြုထားပါတယ်။ ရှေ့မှာ ဖေါ်ပြထားသလိုပါပဲ၊ Generics Type ကို ရေးသားတဲ့ နေရာမှာ အမျှားဆုံး ဘယ် Type အထိအသုံးပြုနိုင်တယ်ဆိုတဲ့ Upper Bounded ကို သာ အသုံးပြုနိုင်တာပဲ ဖြစ်ပါတယ်။ အဲ့ဒါကို Type ကို Declare လုပ်တဲ့ နေရာမှာ extends keyword ကို အသုံးပြုပြီး အမြင့်ဆုံး အသုံးပြုနိုင်တဲ့ Type ကို ရေးသားပေးရမှာ ဖြစ်ပါတယ်။

Type Parameter ကို ရေးသားတဲ့ နေရာမှာ T extends Table လို့ ရေးသားထားတဲ့ အတွက် Repository ကို အသုံးပြုရေးသားတဲ့ နေရာမှာ Table နဲ့ IS - A Relationship ရှိတဲ့ Type တွေကိုပဲ အသုံးပြုလို့ရတော့မှာ ဖြစ်ပါတယ်။

အထက်ပါအတိုင်း Upper Bounded ကို ရေးသားလိုက်တဲ့ အတွက် Table Interface ကို Implement လုပ်ထားတဲ့ Category, Item, Sale နဲ့ SaleDetails Class တွေကိုသာ Repository နဲ့တွဲဖက် အသုံးပြုလို့ရတော့မှာ ဖြစ်ပါတယ်။ Table မဟုတ်တဲ့ အခြား Class တွေကို ရေးသားမယ်ဆိုရင်တော့ Compile Time မှာတင် Error ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။ ဒီလိုနည်းနဲ့ Upper Bounded ကို အသုံးပြုပြီး အမြင့်ဆုံးအသုံးပြုနိုင်တဲ့ Type ကို သတ်မှတ်ပေးနိုင်မှာ ဖြစ်ပါတယ်။

Using Generics Type Objects

Generics နဲ့ပတ်သက်ပြီး Generics Type တွေကို ဘယ်လိုရေးရမယ်ဆိုတာကို ရှေ့မှာ လေ့လာခဲ့ကြပါတယ်။ ဆက်လက်ပြီး ရေးသားထားတဲ့ Generics Type တွေကို ဘယ်လို အသုံးပြုမယ်ဆိုတာကို ဆက်လက်ပြီးလေ့လာသွားကြည့်ကြရအောင်။

```
class Container<T> {
      T data;

      public void setData(T t) {
            this.data = t;
      }

      public T getData() {
            return data;
      }
}
```

အထက်ပါအတိုင်း Container Class ကိုရေးသားထားတဲ့အခါမျိုးမှာ Container Object ကို ရေးသားတော့မယ်ဆိုရင် အသုံးပြုမည့် Type ကို သတ်မှတ်ရေးသားပေးရပါတော့မယ်။

```
public static void main(String[] args) {
      // Before Java SE 7
      Container<String> c1 = new Container<String>();

      // After Java SE 7
      Container<String> c2 = new Container<>();
}
```

Reference Variable ကို Declare လုပ်တဲ့နေရာမှာ Type ကို သတ်မှတ်ပေးရမှာ ဖြစ်ပါတယ်။ Object ဆောက်တဲ့နေရာမှာ ဆိုရင်တော့ Java SE 7 မတိုင်ခင်တုန်းကတော့ Reference Variable မှာ သတ်မှတ်ခဲ့တဲ့ Type ကို ပြန်ပြီး ရေးသားနေရပါတယ်။ ဒါပေမဲ့ Java SE 7 ကိုရောက်တဲ့အခါမှာ Diamond ဆိုတဲ့ ရေးသားပုံအသစ်ပေါ်ပေါက်လာပြီး Constructor ကို ခေါ်တဲ့နေရာမှာ Type ကို ရေးစရာမလိုပဲ <> လို့ ရေးသားနိုင်လာပါတယ်။

Wildcards

Generics Type Variable တွေကိုရေးသားတဲ့ နေရာမှာ Type တွေကို သတ်မှတ်လို့ မရနိုင်သေးဘူးဆိုရင် Wildcard Keyword ဖြစ်တဲ့ ? ကို အသုံးပြုနိုင်ပါတယ်။ Object ဆောက်တာကတော့ အသုံးပြုမည့် Type ကို ပေးပြီး တည်ဆောက်လို့ရနိုင်သလို Java 7 ရဲ့ Diamond ပုံစံနဲ့လဲ တည်ဆောက်လို့ ရနိုင်ပါတယ်။

```
public static void main(String[] args) {
      // Using Wildcard
      Container<?> c1 = new Container<String>();

      // Java 7 Diamond Style
      Container<?> c2 = new Container<>();
}
```

နောက်ပြီး Wildcard နဲ့ ရေးသားထားတဲ့ Generics Type Reference တွေဟာ Type ကိုမသိတဲ့ အတွက် Set လုပ်တဲ့နေရာမှာနဲ့ Get လုပ်တဲ့နေရာမှာ ဘာ Type ဖြစ်နေတယ်ဆိုတာကိုလဲ ခွဲခြားသိနိုင်မှာ မဟုတ်ပါဘူး။

```
public static void main(String[] args) {

      Container<?> c = new Container<>();
      // compile error
      c.setData("Hello Java");          → Error

      // compile error
      String s = c.getData();           → Error
}
```

Bounded in Wildcards

Wildcard ကိုရေးသားတဲ့နေရာမှာလဲ Bounded Type တွေကို ရေးသားအသုံးပြုနိုင်ပါတယ်။ Wildcard တွေရဲ့ နောက်မှာ extends ကို ရေးပြီး Upper Bounded ကို ရေးသားနိုင်သလို super ကို ရေးပြီး Lower Bounded တွေကို ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

```
class A {}
class B extends A {}
class C extends B {}

class Container<T> {
    T data;

    public void setData(T data) {
        this.data = data;
    }

    public T getData() {
        return data;
    }
}
```

အထက်ပါအတိုင်း Class A ကို Class B က Extends လုပ်ထားပြီး Class B ကို Class C က Extends လုပ်ထားပါတယ်။ ပြီးတော့ Generics ကို အသုံးပြုထားတဲ့ Container Class ရှိပါတယ်။

Upper Bounded Wildcard

Generics Type တွေကို Declare လုပ်တဲ့ နေရာမှာ Wildcard ကို အသုံးပြုနိုင်ပြီး extends keyword ကို အသုံးပြုပြီး Upper Bound တွေကို သတ်မှတ်နိုင်ပါတယ်။

```
static void useUpper(Container<? extends B> container) {
    B b = container.getData();
}

public static void main(String[] args) {

    Container<B> bContainer = new Container<>();
    bContainer.setData(new B());
    useUpper(bContainer);

    Container<C> cContainer = new Container<>();
    cContainer.setData(new C());
    useUpper(cContainer);
}
```

အထက်ပါနမူနာထဲမှာ Container<? extends B> လို့ရေးသားထားတဲ့ အတွက် Wildcard မှာ Upper Bound ကို အသုံးပြုထားပါတယ်။ ဒီလိုရေးသားထားတဲ့ အတွက် useUpper() method ရဲ့ Parameter နေရာမှာ B ဒါမှမဟုတ် B ရဲ့ Sub Class တွေကို အသုံးပြုထားတဲ့ Container တွေကို အသုံးပြုနိုင်တာကို တွေ့ရပါတယ်။

ဒီလိုပဲ useUpper() method ထဲမှာ container ကနေတဆင့် getData() method ကို ပြန်ခေါ်သုံးတဲ့အခါမှာလဲ Type B ကို ပြန်လည်ရရှိမှာ ဖြစ်ပါတယ်။ တဖန် B ကို အစားထိုးပြီး အသုံးပြုနိုင်တဲ့ A အနေနဲ့လဲ လက်ခံ ရယူနိုင်မှာ ဖြစ်ပါတယ်။ Wildcard Upper Bounded ကို အသုံးပြုတဲ့နေရာမှာ မရတာတွေလဲတယ်။

ပထမအချက်ကတော့ Upper Bounded Wildcard ကို အသုံးပြုထားတဲ့ Reference ကနေ Type တွေကို Return အနေနဲ့ ရယူတဲ့ အခါမှာ Bounded လုပ်ထားတဲ့ Type ကိုသာ ရယူနိုင်မှာ ဖြစ်ပါတယ်။ Sub Type တွေကိုတော့ Return အနေနဲ့ လက်ခံနိုင်မှာ မဟုတ်ပါဘူး။ container.getData() ရဲ့ တန်ဖိုးကို C မှာ အစားထိုးလို့မရပါဘူး။ B နဲ့ A အနေနဲ့ပဲလက်ခံနိုင်မှာ ဖြစ်ပါတယ်။

ဒုတိယ အချက်ကတော့ Upper Bounded Wildcard ကို အသုံးပြုထားတဲ့ Reference ကနေ တန်ဖိုးတွေကို Set လုပ်ပေးလို့ မရနိုင်ပါဘူး။ contaiener.setData() method ကနေ ဘယ်လို Type မျိုးကိုမှ Set လုပ်နိုင်မှာ မဟုတ်ပါဘူး။ Upper Bounded Wildcard ရဲ့ Reference တွေဟာ တန်ဖိုးတွေကို ထုတ်ယူတဲ့ နေရာမှာပဲ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Lower Bounded Wildcard

Lower Bounded တွေကို ရေးသားလိုတဲ့ အခါမျိုးမှာ super Keyword ကို အသုံးပြုရေးသားနိုင်ပါတယ်။

```
    static void useLower(Container<? super B> container) {
        container.setData(new B());
    }

    public static void main(String[] args) {

        Container<A> aContainer = new Container<>();
        Container<B> bContainer = new Container<>();

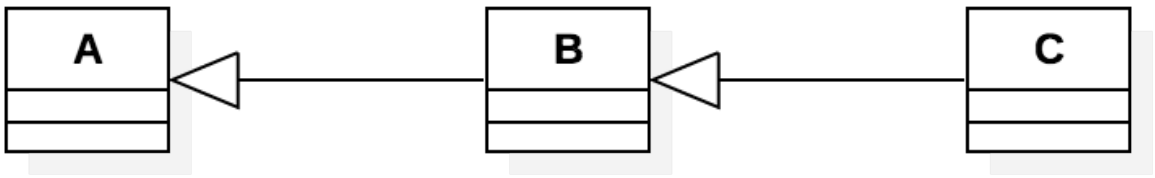
        useLower(aContainer);
        useLower(bContainer);
    }
```

အထက်ပါ နမူနာထဲက useLower() method ရဲ့ Parameter မှာ Container<? super B> လို့ရေးသားထားတဲ့ အတွက် Lower Bounded ကို အသုံးပြုထားတာပါ။ အဲ့ဒီအတွက် useLower() method ကို အသုံးပြုတဲ့ နေရာမှာ B Type ကို အသုံးပြုထားတဲ့ Container နဲ့ B ရဲ့ Super Type တွေကို အသုံးပြုထားတဲ့ Container တွေကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ B ရဲ့ Sub Type ဖြစ်တဲ့ A ရဲ့ Container တွေကို အသုံးပြုလို့ ရတော့မှာ မဟုတ်ပါဘူး။

တဖန် Lower Bounded Wildcard ကို အသုံးပြုထားတဲ့ Reference တွေကနေလဲ Input တွေကိုပဲ လက်ခံနိုင်မှာ ဖြစ်ပြီး Output တွေကိုတော့ သတ်မှတ်ပေးနိုင်မှာ မဟုတ်တော့ပါဘူး။ Set လုပ်တဲ့နေရာမှာလဲ B Type အနေနဲ့ အသုံးပြုနိုင်တဲ့ Type တွေကိုသာ Set လုပ်ပေးနိုင်မှာ ဖြစ်ပါတယ်။ အဲ့ဒီအတွက် B နဲ့ C Type တွေကိုတော့ Set လုပ်နိုင်မှာ ဖြစ်ပြီး B ရဲ့ Super ဖြစ်တဲ့ A ကိုတော့ Set လုပ်လို့ မရနိုင်ပါဘူး။ Lower Bounded Wildcard ကို အသုံးပြုထားတဲ့ Reference တွေကနေ Getter တွေကနေ Type ကိုသတ်မှတ်ပေးလို့ မရတော့တဲ့ အတွက် လက်ခံမယ်ဆိုရင်တော့ Object အနေနဲ့ပဲ လက်ခံနိုင်မှာ ဖြစ်ပါတယ်။

Sub Typing of Generics

OOP အခန်းမှာ Inheritance တွေ Abstraction တွေကို လေ့လာခဲ့စဉ်က IS - A Relationship ကြောင့် Sub Class တွေကို Super Class အနေနဲ့ အသုံးပြုနိုင်တယ်ဆိုတာကို တွေ့ရပါတယ်။



အထက်ဖော်ပြပါ ပုံအတိုင်း B က A ကို Extends လုပ်ပြီး C က A ကို Extends လုပ်ထားတဲ့ အတွက် B နဲ့ C ကို A အနေနဲ့ အသုံးပြုနိုင်သလို C ကိုလဲ B အနေနဲ့ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

```
A a1 = new A();
A a2 = new B();
A a3 = new C();
B b1 = new B();
B b2 = new C();
```



ဒါပေမဲ့ Generics Type တွေကို အသုံးပြုတဲ့အခါမှာတော့ Type တွေရဲ့ IS - A Relationship ဟာ Generics Type တွေရဲ့ IS - A Relationship ဖြစ်မလာပါဘူး။ B ကို အသုံးပြုတဲ့ Container ဟာ A ကို အသုံးပြုတဲ့ Container ရဲ့ Sub Type ဖြစ်မလာပါဘူး။

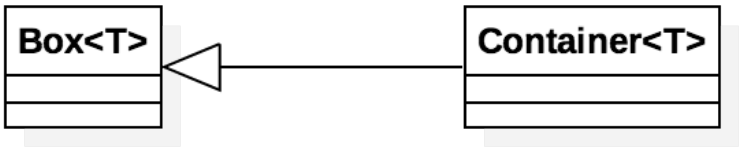
```
Container<A> a = new Container<>();
Container<B> b = new Container<>();

a = b;
```



Container<A> နဲ့ Container<B> အကြားမှာ IS - A Relationship ကို မဖြစ်ပေါ်စေပါဘူး။ ဒါ့ကြောင့် a ရဲ့နေရာမှာ b ကို အစားထိုးပြီး ရေးသားလို့ မရနိုင်ပါဘူး။ Variable a ဟာ A ကို အသုံးပြုနိုင်တဲ့ Container ဖြစ်ပြီး Variable b ဟာ B ကို အသုံးပြုလို့ရတဲ့ Container သာ ဖြစ်ပါမယ်။

ဒါပေမဲ့ Generics Type တွေအချင်းချင်းကြားမှာတော့ IS - A Relationship ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ဆိုကြပါစို့ Generics Type ဖြစ်တဲ့ Box ဆိုတဲ့ Class ကို ရေးသားထားမယ်။ ပြီးရင် Generics Type ကို အသုံးပြုတဲ့ Container Class ကို ရေးသားထားပါမယ်။ အဲဒီနေရာမှာ Container<T> ကနေ Box<T> ကို Extends လုပ်လိုက်မယ်ဆိုရင် Container နဲ့ Box အကြားမှာ Is - A Relationship ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။



ဒီလိုအနေအထားမျိုးမှာ A Type တွေကို အသုံးပြုရေးသားထားတဲ့ Box တွေနေရာကို A Type တွေကို အသုံးပြုပြီး ရေးသားထားတဲ့ Container တွေက အစားထိုးပြီး အသုံးပြုလာနိုင်ပါမယ်။

```
Box<A> box = new Box<>();
Container<A> container = new Container<>();

box = container;
```



ဒါပေမဲ့ A Type တွေကို အသုံးပြုထားတဲ့ Box ရဲ့နေရာမှာ B Type ကို အသုံးပြုထားတဲ့ Container ကို တော့ အစားထိုးလို့ ရနိုင်မှာ မဟုတ်ပါဘူး။

```
Box<A> box = new Box<>();
Container<B> container = new Container<>();

box = container;
```



ဒီလိုနေရာမျိုးမှာ Wildcards တွေကို အသုံးပြုပြီး ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

```
Container<?> con1 = new Container<A>();
Container<? extends B> con2 = new Container<C>();
Container<? super B> con3 = new Container<A>();
```

Wildcard ကို အသုံးပြုထားတဲ့ Container နေရာမှာ မည်သည့် Type မျိုးကို အသုံးပြုထားတဲ့ Container Object ကို အစားထိုးအသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ တဖန် Upper Bounded Wildcard ကို အသုံးပြုထားတဲ့ Container ရဲ့ နေရာမှာ Bounded လုပ်ထားတဲ့ Type နဲ့ သူ့ရဲ့ Sub Type တွေကို အသုံးပြုထားတဲ့ Container တွေကို အစားထိုး အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ဒီလိုပဲ Lower Bounded Wildcard ကို အသုံးပြုပြီး ရေးသားထားတဲ့ Container ရဲ့ နေရာမှာ Bounded Type နဲ့ သူ့ရဲ့ Super Type တွေကို အသုံးပြုထားတဲ့ Container တွေကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

How Generics Work!

Generics ကို အသုံးပြုခြင်းအားဖြင့် မိမိရေးသားထားတဲ့ Class တစ်ခုကို အမျိုးမျိုးသော Data တွေကို အသုံးပြုနိုင်တဲ့ Object တွေကို တည်ဆောက်နိုင်တယ်ဆိုတာကို သိမြင်နိုင်ပါတယ်။ ပြီးတော့လဲ Generics ကို အသုံးပြုတဲ့အတွက် Method တွေနဲ့ Conctructor ရဲ့ Argument တွေရဲ့ Type ကို သတ်မှတ်နိုင်ခြင်းအားဖြင့် Method တွေရဲ့ Return Type ကို သတ်မှတ်ပေးနိုင်ပြီး Type Safe ဖြစ်တဲ့ ပုံစံနဲ့ မိမိရဲ့ Program တွေကို ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

ဒီနေရာမှာတော့ Generics ဟာ နောက်ကွယ်မှာ ဘယ်လို အလုပ်လုပ်နေတယ်ဆိုတာကို လေ့လာကြည့်ကြရအောင်။

```
public class Container<T> {
    T data;

    public void setData(T data) {
        this.data = data;
    }

    public T getData() {
        return data;
    }
}
```

အထက်ပါ Code တွေကို Compile လုပ်လို့ရလာတဲ့ Class တွေကို Decompile လုပ်ကြည့်တဲ့အခါမှာ အောက်ပါအတိုင်း တွေ့ရှိရမှာ ဖြစ်ပါတယ်။

```
public class Container {

    Object data;

    public Container(){}

    public void setData(Object obj){
        data = obj;
    }

    public Object getData(){
        return data;
    }
}
```

Container Class မှာရေးသားထားတဲ့ Type Parameter တွေဟာ Compile လုပ်လို့ရလာတဲ့ Class တွေမှာ ပါမလာတော့သလို Type Parameter တွေရဲ့ နေရာမှာလဲ Object တွေ အစားထိုးဝင်ရောက်သွားတာကို တွေ့ရပါတယ်။ ဒါကို ကြည့်ခြင်းအားဖြင့် Generics ဟာ Compile Time မှာ စစ်ဆေးပေးနေတယ်ဆိုတာကို သိမြင်နိုင်ပါတယ်။ Compile လုပ်လိုက်တဲ့ အခါမှာတော့ နဂိုမူလ Type တွေအဖြစ်ပြန်ရောက်သွားတာကို တွေ့ရပါတယ်။

ဒီလိုမျိုး Generics ရဲ့ Type Parameter တွေကို Raw Type တွေအဖြစ်ပြောင်းလဲပေးနိုင်တဲ့ Process ကို Type Erasure လို့ခေါ်ဆိုပါတယ်။ ဆက်လက်ပြီး Bounded Type ကို အသုံးပြုပြီး ရေးသားထားတဲ့ Generics Type တွေမှာကော Type Erasure က ဘယ်လို အလုပ်လုပ်ပေးသွားတယ်ဆိုတာကို လေ့လာသွားကြရအောင်။

```
public class Container<T extends Number> {
    T data;

    public void setData(T data) {
        this.data = data;
    }

    public T getData() {
        return data;
    }
}
```

အထက်ပါနမူနာမှာတော့ Container Class ရဲ့ Type Parameter ကို <T extends Number> လို့ရေးသားထားတဲ့ အတွက် Number နဲ့ Number ရဲ့ Sub Class တွေကို Type Parameter တွေနေရာမှာ အစားထိုးအသုံးပြုလို့ရမှာ ဖြစ်ပါတယ်။

```
public class Container {
    Number data;

    public Container() {}

    public void setData(Number number) {
        data = number;
    }

    public Number getData() {
        return data;
    }
}
```

Reverse Engineering နဲ့ Compile လုပ်လို့ရလာတဲ့ Class တွေကို Decompile လုပ်ကြည့်တဲ့ အခါမှာတော့ Type Parameter တွေဟာ Bounded လုပ်ထားတဲ့ Type ဖြစ်တဲ့ Number တွေအဖြစ်ပြောင်းလဲသွားတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။

ဒါကို ကြည့်ခြင်းအားဖြင့် Type Erasure ဟာ Type Parameter တွေကို Compile လုပ်တဲ့အခါမှာ Upper Bounded နဲ့ ရေးသားထားရင် Bounded လုပ်ထားတဲ့ Type အဖြစ်ပြောင်းပေးပြီး Bounded မလုပ်ထားဘူးဆိုပါက အမြင့်ဆုံး Type ဖြစ်တဲ့ Object အဖြစ်ပြောင်းပေးတယ်ဆိုတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။

Type Erasure in Generics Methods

Generics Type တွေမှာ Type Erasure က ဘယ်လိုအလုပ်လုပ်တယ်ဆိုတာက အရှေ့မှာလေ့လာပြီးခဲ့ပါပြီ။ ဆက်လက်ပြီး Generics Method တွေမှာကော Type Erasure က ဘယ်လို အလုပ်လုပ်တယ်ဆိုတာကို လေ့လာသွားကြရအောင်။

```
public class GenericsMethodDemo {

      static<T> void use(T t) {

      }

      static<T extends Number> void use(T t) {

      }
}
```

အထက်ပါနမူနာမှာတော့ Generics Method တစ်ခုနဲ့ Bounded Type ကို အသုံးပြုတဲ့ Generics Method တစ်ခုတို့ကို ရေးသားထားပါတယ်။ Reverse Engineering ကို အသုံးပြုပြီး Type Erasure က ဘယ်လို အလုပ်လုပ်သွားတယ်ဆိုတာကို လေ့လာသွားကြရအောင်။

```
public class GenericsMethodDemo {

    public GenericsMethodDemo() {
    }

    static void use(Object obj) {
    }

    static void use(Number number) {
    }
}
```

Generics Method မှာလဲ Type Erasure ဟာ Generics Type တွေတုန်းကလိုပဲ Bounded လုပ်ထားတဲ့ Type Parameter တွေကို Bounded Type အဖြစ်ပြောင်းပေးပြီး၊ Bounded မလုပ်ထားရင်တော့ Object အဖြစ်ပြောင်းပေးတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။

Bridge Method

Generics Type မဟုတ်တဲ့ Type တွေကနေ Generics Type တွေကို Extends လုပ်တဲ့အခါမျိုးမှာ Java Compiler ကနေ Type Erasure Process နှင့်အတူဖြည့်စွက်ပေးရတဲ့ Synthetic Method တွေရှိပါတယ်။ အဲ့ဒီ Method မျိုးကို Bridge Method လို့ခေါ်ပါတယ်။

```
public interface Box<T> {
      void add(T t);
      T get();
}

class IntegerBox implements Box<Integer> {

      private Integer data;

      @Override
      public void add(Integer i) {
            this.data = i;
      }

      @Override
      public Integer get() {
            return data;
      }
}
```

အထက်ပါနမူနာထဲမှာ Generics Type ဖြစ်တဲ့ Box<T> Interface ကို Generics Type မဟုတ်တဲ့ IntegerBox ကနေ Implement လုပ်ထားပါတယ်။ Type Erasure Process ကြောင့်ရလာတဲ့ Box Interface ကို လေ့လာကြည့်ကြရအောင်။

```
public interface Box {

    public abstract void add(Object obj);

    public abstract Object get();
}
```

Type Parameter ဖြစ်တဲ့ T နေရာကို Object တွေက အစားထိုးသွားတာကို တွေ့ရပါမယ်။ add() method ရဲ့ Parameter ဟာ Object ဖြစ်သွားသလို get() method ရဲ့ Return Type ဟာလဲ Object ဖြစ်သွားပါတယ်။

Box Interface နဲ့ Sub Class ဖြစ်တဲ့ IntegerBox Class တို့ရဲ့ Method တွေကို Override ဖြစ်သလား မဖြစ်ဘူးလားဆိုတဲ့ အမြင်နဲ့ ကြည့်ကြရအောင်။ မှန်ကန်တဲ့ Override Rule မှာ Super Type ထဲမှာပါတဲ့ Method နဲ့ Sub Type ထဲမှာ ပါတဲ့ Method တွေရဲ့ Method Signature (Method Name & Parameter List) ဟာ အတိအကျတူရမယ်လို့ပါရှိပါတယ်။ ဒါပေမဲ့ Box Interface ရဲ့ add() method ရဲ့ Parameter Type ဟာ Object ဖြစ်ပြီး IntegerBox မှာတော့ Parameter Type ဟာ Integer ဖြစ်နေပါတယ်။ Parameter Type တွေမတူတော့တဲ့အတွက် Override Rule ကို ဖောက်ဖျက်သွားပါပြီ။

ဒီလိုပြဿနာမျိုးကို ဖြေရှင်းဖို့အတွက် Java Compiler ကနေ Type Erasure Process ကို လုပ်ဆောင်နေချိန်အတွင်းမှာ Synthetic Method ဖြစ်တဲ့ Bridge Method တွေကို ဖြည့်စွက်ပေးရတဲ့ အခါလဲ ရှိတတ်ပါတယ်။ Bridge Method ဆိုတာ ဘယ်လိုဟာလဲ ဆိုတာကို IntegerBox Class ကို Decompile လုပ်ထားတဲ့ ကုဒ်တွေက နေလေ့လာကြည့်ကြရအောင်။

```
class IntegerBox implements Box {

    private Integer data;

    IntegerBox() {}

    public void add(Integer integer) {
        data = integer;
    }

    public Integer get() {
        return data;
    }

    public volatile Object get() {
        return get();
    }

    public volatile void add(Object obj) {
        add((Integer)obj);
    }
}
```

Bridge Method

Object ကို return ပြန်တဲ့ get() method နဲ့ Object ကို Parameter အနေနဲ့ လက်ခံထားတဲ့ add() method တွေဟာ Compiler ကနေ ဖြည့်စွက်ပေးတဲ့ Bridge Method တွေ ဖြစ်ကြပါတယ်။

Bridge Method ဖြစ်တဲ့ add() Method ကတော့ Super Type ထဲက Method နဲ့ Override လုပ်တဲ့ Method တို့ရဲ့ Method Signature ကိုတူအောင် ဖြည့်ပေးရတဲ့ bridge method ဖြစ်ပါတယ်။ get() method ကတော့ Java 5 မှာ စတင်ပါဝင်လာခဲ့တဲ့ Covariant Return ကြောင့်ဖြစ်ပါတယ်။ Java 5 မတိုင်ခင်က Override Rule မှာ Super Class မှာ Return ပြန်ထားတဲ့ အတိုင်း Override လုပ်တဲ့ Method ကနေ Return မပြန်လို့ မရပါဘူး။ ဒီလိုမှ မဟုတ်ရင် Override မဖြစ်မြောက်ပါဘူး။

Java SE 5 အရောက်မှာ Covariant Return ဆိုတာကို ဖြည့်စွက်လာပြီး Super Type ထဲမှာ Return ပြန်တဲ့ Type ရဲ့ Sub Type တွေကိုလဲ Override Method တွေကနေ Return ပြန်လို့ရလာပါတယ်။ ဒီလို ဖြစ်ရပ်မျိုးကို Covariant Return လို့ခေါ်ပါတယ်။ အထက်ဖော်ပြပါ နမူနာထဲမှာပါတဲ့ Object ကို Return ပြန်နေတဲ့ get() Method ဟာ Covariant Return ကို ဖြစ်မြောက်စေဖို့ Compiler ကနေ ဖြည့်စွက်ပေးထားတဲ့ Bridge Method တစ်ခု ဖြစ်ပါတယ်။

Side Effect of Type Erasure

ကျွန်တော်တို့ Generics တွေကို အသုံးပြုတဲ့နေရာမှာ Type Erasure ကို အသုံးပြုပြီး Generics Type တွေကို နဂိုမူလပုံစံ (Raw Type) တွေအဖြစ်ပြောင်းလဲ ပေးနိုင်တယ်ဆိုတာကို လေ့လာခဲ့ပါတယ်။ Generics Type ဖြစ်တဲ့ Box<T> ကို Raw Type ဖြစ်တဲ့ Box အဖြစ် Compile Time မှာ ပြောင်းလဲပေးခြင်းကို ဆိုလိုပါတယ်။ Type Erasure တွေကြောင့် ကျွန်တော်တို့တွေ မျှော်လင့်မထားတဲ့ အခြေအနေတွေကို ရောက်ရှိသွားနိုင်ပါသေးတယ်။

```
class Box<T> {
      T data;

      Box(T t) {
            data = t;
      }
      void setData(T t) {
            data = t;
      }
}

class IntBox extends Box<Integer> {
      IntBox(Integer i) {
            super(i);
      }
      void setData(Integer integer) {
            data = integer;
      }
}
```

အထက်ပါနမူနာထဲမှာတော့ Generics Type ဖြစ်တဲ့ Box<T> ကို IntBox က Extends လုပ်ထားပါတယ်။ ရလာတဲ့ IntBox Class ကို Decompile လုပ်ကြည့်တဲ့အခါ အောက်ပါအတိုင်း ကုဒ်တွေကို Generate လုပ်သွားပါတယ်။

```
class IntBox extends Box {
    IntBox(Integer integer) {
        super(integer);
    }
    void setData(Integer integer) {
        data = integer;
    }
    volatile void setData(Object obj) {
        setData((Integer)obj);
    }
}
```

Generate လုပ်လာတဲ့ ကုဒ်ထဲမှာ Object ကို Parameter အနေနဲ့ လက်ခံတဲ့ setData() method တစ်ခုကို Bridge အနေနဲ့ Generate လုပ်သွားတာကို တွေ့ရပါတယ်။

ဆက်လက်ပြီး Type Erasure ကြောင့် ဘယ်လို Side Effect တွေ ဖြစ်နိုင်တယ်ဆိုတာကို အောက်ပါ ကုဒ်တွေက နေ တဆင့် လေ့လာကြည့်ကြရအောင်။

```
public class EffectDemo {
     public static void main(String[] args) {
          IntBox box = new IntBox(10);
          changeData(box);
          Integer i = box.data;
     }

     static void changeData(Box box) {
          box.setData("Hello");
     }
}
```

အထက်ပါ changeData() method ဟာ Box<T> ရဲ့ Raw Type ကို Parameter အနေနဲ့ ရယူထားပါတယ်။ ဒီလို ရေးသားထားရင် Compiler ကနေ Raw Types တွေကို သုံးထားပါတယ်ဆိုပြီး Warning ကို ထုတ်ပေးပါမယ်။ အလားတူပဲ box.setData() Method ကို ရေးသားတဲ့နေရာမှာလဲ Compiler ကနေ Check မလုပ်နိုင်တော့ဘူးဆိုပြီး Unchecked Called Warning ကို ထုတ်ပေးပါမယ်။ ဒါပေမဲ့ Compile Time Exception တော့မဖြစ်သွားပါဘူး။

main() method ထဲမှာတော့ IntBox Type box ကို Object ဆောက်ထားပါတယ်။ ပြီးတော့ changeData() method ရဲ့ Parameter အနေနဲ့ IntBox Type Variable ကို ပေးလိုက်ပါတယ်။ ဒါပေမဲ့ changeData() method ထဲက သိတာကတော့ box ရဲ့ Type ဟာ Box ပဲဖြစ်ပါတယ်။ Box ဟာ Raw Type ဖြစ်တဲ့ အတွက် setData() ရဲ့ Parameter ဟာ Object ဖြစ်ပါမယ်။

```
     static void changeData(Box box) {
          box.setData("Hello");
     }
```

ဒါ့ကြောင့် အထက်ပါအတိုင်း changeData() method ထဲမှာ box.setData(“Hello”) ဆိုပြီး String တစ်ခုကို အသုံးပြုသွားနိုင်တာ ဖြစ်ပါတယ်။ ဒီနေရာမှာ သတိပြုဖို့လိုတာက changeData() method ရဲ့ box ကနေ Reference လုပ်နေတာက IntBox Object ဖြစ်နေတယ်ဆိုတာပါပဲ။ IntBox ကို ရေးသားခဲ့တုန်းက setData() method ရဲ့ Argument ဟာ Integer ဖြစ်ပေမဲ့၊ Type Erasure ကနေ Generate လုပ်ပေးလိုက်တဲ့ Bridge Method ကြောင့် Object ကိုလဲ လက်ခံလို့ရသွားပါတယ်။

```
    volatile void setData(Object obj) {
        setData((Integer)obj);
    }
```

ဒီ Bridge Method ထဲမှာ ပြဿနာက စတော့တာပါပဲ။ ဒီ Method ထဲမှာ (Integer)obj ဆိုပြီး Object ကို Integer အနေနဲ့ Type Cast လုပ်နေပါတယ်။ Parameter အနေနဲ့ ရရှိလာတဲ့ “Hello” ကို Integer အဖြစ် Type Cast လုပ်ဖို့ကြိုးစားတဲ့ အခါမှာ ClassCastException ကိုဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

Generics မှာ လုပ်လို့မရသော အရာများ

Generics မှာ ရေးသားလို့မရနိုင်တဲ့ ကန့်သတ်ချက်တွေရှိကြပါတယ်။ အဲ့ဒါတွေကတော့ အောက်ပါအတိုင်း ဖြစ်ကြပါတယ်။

- ✓ Primitive တွေကို Type Parameter အနေနဲ့ ရေးသားလို့ မရနိုင်ပါဘူး
- ✓ Type Parameter တွေကို new keyword နဲ့ Object ဆောက်လို့ မရနိုင်ပါဘူး
- ✓ Static Variable တွေမှာ Type Parameter ကို အသုံးပြုလို့ မရပါဘူး
- ✓ Generics Type တွေမှာ Cast နဲ့ instanceof ကိုသုံးလို့မရပါဘူး
- ✓ Parameterized Type တွေရဲ့ Array တွေကို Create လုပ်လို့မရနိုင်ပါဘူး
- ✓ Exception တွေအနေနဲ့ Generics Type တွေကိုသုံးလို့မရနိုင်ပါဘူး
- ✓ Generics Type တွေကို Argument အနေနဲ့ ရေးပြီး Overload ရေးလို့ မရနိုင်ပါဘူး

Primitive တွေကို Type Parameter အနေနဲ့ ရေးသားလို့ မရနိုင်

အောက်ပါအတိုင်း Generics Type ဖြစ်တဲ့ Box<T> ကို ရေးသားထားပါတယ်။

```
class Box<T> {
      T data;
      Box(T t) {
            data = t;
      }
}
```

BoxDemo Class ရဲ့ Main Method ထဲမှာ Primitive Type ဖြစ်တဲ့ int ကို Parameter အနေနဲ့ ထားပြီး Compile လုပ်ကြည့်ပါမယ်။
အောက်ပါအတိုင်း Compile Error ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

```
public class BoxDemo {
      public static void main(String[] args) {
            Box<int> b = new Box<>(10);
      }
}
```

Generics Type တွေရဲ့ Type Parameter နေရာမှာ Reference Type ကိုသာ လက်ခံတဲ့ အတွက် Primitive ဖြစ်

```
BoxDemo.java:10: error: unexpected type
            Box<int> b = new Box<>(10);
                ^
  required: reference
  found:    int
1 error
```

တဲ့ int ကို လက်မခံတာပဲ ဖြစ်ပါတယ်။

Type Parameter တွေကို Object ဆောက်လို့ မရနိုင်ပါဘူး

Java မှာ Type တွေအနေနဲ့ Class တွေ၊ Interface တွေ၊ Abstract Class တွေနဲ့ Enum တွေကို ရေးသားလို့ရနိုင်ပါတယ်။ ဒီ Type တွေထဲမှာမှ Object ဆောက်လို့ရတာကတော့ Concrete Class တွေသာ ဖြစ်ပါတယ်။ Abstract Class တွေနဲ့ Interface တွေကတော့ ဒီအတိုင်း new ဆိုပြီး Constructor ကိုခေါ်ပြီး Object ဆောက်လို့ မရနိုင်ပါဘူး။ Anonymous Class တွေကနေတော့ ဆောက်လို့ရတာပေါ့ဗျာ။ ဒါပေမဲ့ အဲ့ဒီ Class ကနေတိုက်ရိုက် ဆောက်လို့မရဘူးဆိုတာကို ဆိုလိုတာပါ။ Enum Type တွေက ကြိုပြီး တည်ဆောက်ထားရတဲ့ Static Class အမျိုးအစားတွေဖြစ်တဲ့ အတွက် Runtime ကျမှ Object တွေကို တည်ဆောက်လို့ မရနိုင်ပါဘူး။

ဒီလိုပဲ Generics Type တွေမှာ အသုံးပြုနေတဲ့ Type Parameter တွေကနေ new Keyword ကို အသုံးပြုပြီး Object ကို မတည်ဆောက်နိုင်ပါဘူး။ ဆိုကြပါစို့၊ Object တွေကို ထုတ်ပေးနိုင်တဲ့ Generics Method တစ်ခုကို ရေးကြပြီဆိုပါစို့။

```
public static <T> T getNewBox() {
     return new T();
}
```

အထက်ပါအတိုင်း T Object ကို လိုချင်တဲ့ အတွက် new T() ဆိုပြီး ရေးမိမယ်ထင်ပါတယ်။ ဒါပေမဲ့ မရနိုင်ပါဘူး။ Compile လုပ်ကြည့်တဲ့ အခါမှာ အောက်ပါအတိုင်း Compile Error ကို ဖြစ်ပေါ်စေပါတယ်။

```
BoxFactory.java:4: error: unexpected type
          return new T();
                     ^
  required: class
  found:    type parameter T
  where T is a type-variable:
    T extends Object declared in method <T>getNewBox()
1 error
```

လိုချင်တဲ့ Type က Class ဖြစ်ပြီး တကယ်သုံးထားတာက Type Parameter T ဖြစ်တယ်ဆိုပြီး Compile Time Error ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။ ဟုတ်ပါတယ် Java မှာက ဘာ Class မှန်းမသိရင် Object ဆောက်လို့ မရနိုင်ပါဘူး။ Type Parameter ဖြစ်တဲ့ T နေရာမှာ ဘယ်လို Class ကရောက်လာမယ်ဆိုတာကို မခန့်မှန်းနိုင်တဲ့ အတွက် Type Parameter ရဲ့ Construtor ကို ခေါ်ပြီး Object ဆောက်လို့ မရနိုင်ပါဘူး။

```
public static <T> T getNewBox(Class<T> type)
          throws Exception {
     return type.newInstance();
}
```

ဒါပေမဲ့ အထက်ပါအတိုင်း Class<T> ကို သိမယ်ဆိုရင်တော့ အဲ့ဒီ Class ရဲ့ newInstance() method ကိုအသုံးပြုပြီး Object ဆောက်လို့ရမှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ အဲ့ဒီ Type ကလဲ Object ဆောက်လို့ရတဲ့ Type ဖြစ်နေဖို့လိုအပ်သလို Argument မပါတဲ့ Default Constructor တစ်ခုရှိနေဖို့လဲ လိုအပ်ပါတယ်။ Abstract Class တွေ၊ Interface တွေနဲ့ Enum တွေဆိုရင်တော့ ဒီပုံစံနဲ့လဲ Object ကို တည်ဆောက်လို့ရမှာမဟုတ်ပါဘူး။ Concrete Class ဖြစ်ပေမဲ့ Default Constructor မရှိပြန်ရင်လဲ ဒီပုံစံနဲ့ Object ဆောက်လို့ မရနိုင်ပါဘူး။

Static Variable တွေမှာ Type Parameter ကို သုံးလို့မရဘူး

Generics Type တွေထဲမှာရေးသားထားတဲ့ Type Parameter တွေဟာ Generics Type တွေရဲ့ Object တွေမှာသာ အသုံးပြုနိုင်ပါတယ်။ ဒါ့ကြောင့် Type Parameter တွေကို Static Variable တွေရဲ့ Type တွေအနေနဲ့ ရေးသားလို့ မရနိုင်ပါဘူး။

```
public class StaticTest<T> {
     static T testData;
}
```

အထက်ပါအတိုင်း Generics Type Class တစ်ခုကို ရေးသားပြီး Type Parameter ဖြစ်တဲ့ T ကို Static Variable ရဲ့ Type အဖြစ် အသုံးပြုကြည့်ပါမယ်။ အထက်ပါ ကုဒ်ကို Compile လုပ်ကြည့်တဲ့ အခါမှာ ကွန်ပိုင်းလုပ်လို့ ရမှာ မဟုတ်ပါဘူး။

```
StaticTest.java:2: error: non-static type variable T cannot
be referenced from a static context
     static T testData;
            ^
1 error
```

Type Parameter T ကို Static Context တစ်ခုကနေ Reference လုပ်လလို့မရဘူးဆိုပြီး Compile Error ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။ တကယ်တမ်းကျတော့ Static Variable တွေဟာ Class Loader ကနေ Class ကို JVM Memory ပေါ်ကို ဆွဲတင်လိုက်ကထဲက သတ်မှတ် ထားဖို့လိုအပ်ပါတယ်။ ပြီးတော့ Static Variable တွေဟာ JVM Memory ပေါ်မှာ တစ်ခုထဲသော Variable တွေဖြစ်နေပါတယ်။ ဒါ့ကြောင့် Type Parameter ကို Type အမျိုးမျိုးနဲ့ လာသတ်မှတ်နေရင် Static Variable ကို ဘယ် Type နဲ့ သတ်မှတ်ရမလဲ ဆိုတာကို ဝေခွဲလို့ ရမှာ မဟုတ်တော့ပါဘူး။

Generics Type တွေမှာ Cast နဲ့ instanceof ကို သုံးလို့မရနိုင်ဘူး

```
static<T> void checkType(Box<T> box) {
     if(box instanceof Box<String>) {
          System.out.println("String Box");
     }
}
```

အထက်ပါကုဒ်ကို ရေးသားထားရင် Compile Time မှာ Error တက်မှာ ဖြစ်ပါတယ်။ တကယ်တမ်းကျတော့ Generics ဆိုတာဟာ Compile Time မှာသာ စစ်ဆေးပေးနိုင်တဲ့ အရာဖြစ်ပါတယ်။ ဒါ့ကြောင့် Box<String> နဲ့ Box<Integer> တွေဟာ Compile လုပ်ပြီးတဲ့ အခါမှာ အတူတူပဲ ဖြစ်သွားကြပါမယ်။

```
public static void main(String[] args) {
     Box<String> b1 = new Box<>();
     Box<Integer> b2 = new Box<>();
}
```

အထက်ပါနမူနာထဲမှာ Box<String> နဲ့ Box<Integer> တို့ကို ရေးသားထားပါတယ်။ Compile လုပ်ပြီးရလာတဲ့ Class ကို Decompile လုပ်ကြည့်တဲ့ အခါမှာ အဲ့ဒီ Type တွေဟာ အတူတူပဲ ဖြစ်နေတာကို တွေ့ရပါတယ်။

```
public static void main(String args[]) {
    Box box = new Box();
    Box box1 = new Box();
}
```

Generics Type တွေကို Compile လုပ်တဲ့ အခါမှာ Type Erasure ကြောင့် Generics Type တွေဟာ Raw Type တွေဖြစ်သွားကြပါတယ်။ Compile လုပ်ပြီးတဲ့ အခါမှာရလာတဲ့ Raw Type ဟာ တစ်ခုထဲဖြစ်တဲ့ အတွက် ခွဲခြားလို့ မရနိုင်တော့ပါဘူး။ ဒါ့ကြောင့် Generis Type တွေမှာ Casting ကိုလဲ ရေးသားလို့ မရနိုင်တာ ဖြစ်ပါတယ်။

```
static void checkType(Box<?> box) {
     if(box instanceof Box<?>) {
          System.out.println("Object Box");
     }
}
```

ဒါပေမဲ့ အရှေ့မှာဖေါ်ပြခဲ့တဲ့ checkType() Method ကို အထက်ပါအတိုင်းပြင်ပြီးရေးမယ်ဆိုရင်တော့ Compile လုပ်လို့ရပါလိမ့်မယ်။ Type Parameter နေရာမှာ Wildcard ကိုရေးသားလိုက်တဲ့ အတွက် ဘယ် Type မျိုးမဆို အသုံးပြုလို့ ရသွားပါမယ်။ Raw Type ကို သုံးတဲ့ ပုံစံဖြစ်သွားတာပေါ့။

```
public class BoxDemo {

     public static void main(String[] args) {
          Box<String> b1 = new Box<>();
          Box<Integer> b2 = new Box<>();

          checkType(b1);
          checkType(b2);
     }

     static void checkType(Box<?> box) {
          if(box instanceof Box<?>) {
               System.out.println("Object Box");
          }
     }
}
```

အထက်ပါ အတိုင်း ရေးသားပြီး Run ကြည့်မယ်ဆိုရင် Box<String> ကော Box<Integer> ပါ တူညီတဲ့ ရလဒ်ကို ရမှာ ဖြစ်ဒါပါတယ်။ အထက်မှာ ဖေါ်ပြထားတဲ့ အတိုင်း Generics Type တွေကို Compile လုပ်ပြီးတဲ့ အခါမှာ အားလုံးဟာ Object ကို အသုံးပြုတဲ့ Raw Type တွေ ဖြစ်သွားမှာ ဖြစ်တဲ့ အတွက် အားလုံးရဲ့ Type ဟာ အတူတူပဲလို့ သတ်မှတ်သွားတာ ဖြစ်ပါတယ်။ ရသာရတာပါ တကယ်တမ်းကျတော့ ဘာမှ အဓိပ္ပါယ်မရှိပါဘူး။

ကဲဆက်လက်ပြီး Generics Type တွေရဲ့ Casting အကြောင်းကိုလေ့လာသွားကြရအောင်။ တကယ်တမ်းကျတော့ Generics တွေရဲ့ Sub Typing တွေအကြောင်းမှာ ဖေါ်ပြပြီး ဖြစ်ပါတယ်။

```
class Box<T> {}

class Container<T> extends Box<T> {}

public class BoxDemo {

      public static void main(String[] args) {

            Box<Number> b1 = new Box<>();
            Box<Integer> b2 = (Box<Integer>)b1; // compile error

            Box<Integer> b3 = new Container<>();
            Container<Integer> b4 = (Container<Integer>)b3; // OK
      }
}
```

Generics Type တွေရဲ့ Sub Type တွေအကြောင်းကို လေ့လာခဲ့တုန်းက Type Parameter ကြားမှာ ရှိတဲ့ IS- A Relationship ဟာ Generics Type တွေကြားကို သယ်မသွားနိုင်ဘူးဆိုတာကို လေ့လာခဲ့ပါတယ်။ ဆိုကြပါစို့ Integer ဟာ Number ရဲ့ Sub Type ဖြစ်ပေမဲ့ Box<Integer> ဟာ Box<Number> ရဲ့ Sub Type ဖြစ်မလာပါဘူး။ ဒါ့ကြောင့် Is - A Relationshp မရှိတဲ့ နေရာမှာ Type Casting ကို အသုံးပြုနေတဲ့ အတွက် Compile Time Error ကို ဖြစ်ပေါ်စေတာ ဖြစ်ပါတယ်။

ဒီလိုပဲ Generics Type အချင်းချင်း Inheritance ရှိတဲ့ နေရာတွေမှာ ဆိုရင်တော့ Type Casting ကို အသုံးပြုလို့ရမှာ ဖြစ်ပါတယ်။ ဒါကြောင့် Container<Integer> b4 = (Container<Integer>)b3; ဆိုပြီး Casting ကို ရေးသားလို့ ရနေတာ ဖြစ်ပါတယ်။

Exception တွေမှာ Generics Type တွေကို သုံးလို့ မရနိုင်ဘူး

```
public class GenericsException<T> extends Exception {

}
```

အထက်ပါ Class ကို Compile လုပ်ကြည့်တဲ့ အခါမှာ Generics Type တွေဟာ Throwable ကို Extends လုပ်လို့မရဘူးဆိုပြီး Compile Error ကို ဖြစ်စေပါတယ်။ Throwable ကို Extends လုပ်လို့မရဘူးဆိုတော့မှ Generics Class တွေဟာ Exception အနေနဲ့ ရေးသားလို့ရမှာ မဟုတ်တော့ပါဘူး။

```
GenericsException.java:1: error: a generic class may not
extend java.lang.Throwable
public class GenericsException<T> extends Exception {
                                          ^
1 error
```

ဘယ်နည်းနဲ့မှ Generics တွေဟာ Exception ဖြစ်မလာနိုင်ဘူးဆိုမှတော့ Generics Type ရေးသားထားတဲ့ Type Parameter တွေကို Catch လဲလုပ်လို့ရမှာ မဟုတ်သလို Method တွေရဲ့ Throw နေရာမှာလဲ ရေးသားလို့ရမှာ မဟုတ်ပါဘူး။

Generics Type တွေကို Argument အနေနဲ့ ရေးပြီး Overload ရေးလို့ မရနိုင်ပါဘူး

```
import java.util.*;

public class OverrideDemo {

	public void showList(List<String> list) {
		// Codes
	}

	public void showList(List<Date> list) {
		// Codes
	}
}
```

Overload ဆိုတာက Class တစ်ခုအတွင်းမှာ Method Name တူပြီး Argument List မတူတဲ့ Method တွေကို ထပ်ခါ ထပ်ခါရေးသားခြင်းကို ဆိုလိုပါတယ်။ အထက်ပါ ကုဒ်တွေထဲမှာ showList() method နှစ်ခုကို ရေးသားထားပြီး Generics Type ဖြစ်တဲ့ List<Stirng> နဲ့ List<Date> တို့ကိုရေးသားထားပါတယ်။ Parameter List မတူဘူးလို့ မြင်ရပါမယ်။ ဒါပေမဲ့ Compile လုပ်ပြီးရလာတဲ့ Raw Type တွေဟာ အတူတူဖြစ်တဲ့ အတွက် ဒုတိယ Method ကို Compile လုပ်တဲ့ အခါမှာ Compile Error ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

```
OverrideDemo.java:9: error: name clash: showList(List<Date>)
and showList(List<String>) have the same erasure
	public void showList(List<Date> list) {
	            ^
1 error
```

Essential Java API

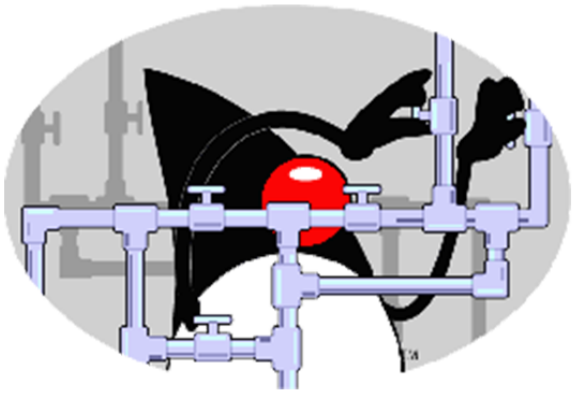


API ဆိုတာက Application Programming Interface ဆိုတဲ့ စကားလုံးရဲ့ အတိုကောက် အခေါ်အဝေါ်ဖြစ်ပါတယ်။ ကျွန်တော်တို့ Java Programming ကို အသုံးပြုပြီး Application တွေကို ရေးသားတဲ့ နေရာမှာ Program တွေထဲကနေ အလွယ်တကူအသုံးပြုနိုင်အောင် ကြိုတင်ပြင်ဆင်ပေးထားတဲ့ Class Library တွေ ဖြစ်ကြပါတယ်။ ဆိုကြပါစို့ ကျွန်တော်တို့ Application ထဲကနေ File တွေကို Read Write လုပ်ချင်တဲ့ အခါရှိမယ်။ ဒီလိုပဲ Data တွေကို အစုလိုက် အသုံးပြုတာတို့၊ သိမ်းထားလိုတဲ့ Data တွေကို ဘယ်လို Data Structure တွေကို အသုံးပြုမှာလဲတို့ သုံးလိုတဲ့ အခါလဲရှိပါမယ်။ ဒီလိုပဲ နေ့ရက် အချိန်တွေကို အသုံးပြုမယ်၊ ပြီးတော့ နေ့ရက်တွေရဲ့ ကွာခြားချိန်တွေကို တွက်ချက်လိုတဲ့ အခါတွေလဲရှိပါမယ်။ ဒီလိုပဲ Internet ကို ဖြတ်ပြီး Cloud တစ်နေရာရာက Web Service နေ ပံ့ပိုးပေးထားတဲ့ Service တွေကို အသုံးပြုလိုတဲ့ အခါမျိုးလဲရှိပါမယ်။

အဲ့ဒီလို လိုအပ်ချက်တွေမှာ ကိုယ့်ဘာသာကိုယ် Low Level အထိ ဝင်ရေးစရာမလိုအောင် Java SE မှာ Standard API တွေကို ပြင်ဆင်ပေးထားပါတယ်။ Java SE 8 မှာတင် JRE ထဲမှာ Class ပေါင်း ၄၂၄၀ ခု ရေးသားပြင်ဆင်ပေးထားပါတယ်။ တစ်ချို့ Class တွေကတော့ API တွေရဲ့ အတွင်းပိုင်းမှာသာ အသုံးပြုတဲ့ Class တွေ ဖြစ်ကြပြီး၊ တစ်ချို့ကျတော့လဲ Framework ရေးသားသူတွေ အသုံးပြုနိုင်တဲ့ Class တွေဖြစ်ကြပြီး၊ ကျွန်တော်တို့လို Application Developer တွေမှာ အသုံးပြုသင့်တဲ့ API တွေဆိုပြီးရှိနိုင်ပါတယ်။

ကျွန်တော်တို့ ဒီအခန်းထဲမှာတော့ Java Application Developer တစ်ယောက်အနေနဲ့ မသိမဖြစ်သိထားသင့်တဲ့ API တွေအကြောင်းကို စုစည်းဖော်ပြသွားမှာ ဖြစ်ပါတယ်။

Exceptions Handling



လူဆိုတာ အမှားနဲ့ မကင်းကြပါဘူး။ မမှားသောရွေ့နေ မသေသော ဆေးသမားဆိုပြီးလဲ ပြောလေ့ရှိကြပါတယ်။ ဒီလိုပဲ ကွန်တော်တို့လို Program ကို ရေးသားတဲ့ Developer တွေဟာလဲ ကုဒ်တွေရေးတဲ့နေရာမှာ အမှားတွေကို ရေးမိနေတဲ့ အခါမျိုးရှိတတ်ပါတယ်။

တစ်ခါတစ်လေတော့ ရေးတုန်းကတော့ မမှားဘူး။ သုံးကြည့်လိုက်တဲ့ အခါမျိုးမှာ ကွန်တော်တို့ အစတုန်းက ထင်မထားတဲ့ အသုံးပြုပုံမျိုးနဲ့ကြုံတဲ့အခါမှာ အမှားတွေ ဖြစ်သွားတတ်ကြပါတယ်။ လူတွေဆိုရင်တော့ ခိုင်းတာမှားနေရင် မဟုတ်သေးဘူး။ ဘယ်လိုလုပ်ရမှာလဲ အစရှိသဖြင့် Complain တက်တာတို့ မလုပ်ပဲနေတာတို့ ဆုံးဖြတ်ပြီး လုပ်သွားနိုင်ပါတယ်။ ဒါပေမဲ့ Computer အနေနဲ့တော့ ရေးသားထားတဲ့ Program အတိုင်း အလုပ်လုပ်သွားမှာ ဖြစ်ပါတယ်။ တစ်ချို့သော Program တွေမှာ ဆိုရင် မိမိရေးသားထားတဲ့ Application ထဲမှာသာမက ပတ်ဝန်းကျင်ဖြစ်တဲ့ OS ကိုတောင် ပျက်စီးစေနိုင်တဲ့ အထိ ဖြစ်သွားတတ်ပါတယ်။

User Input မှားတာကြောင့်ပဲဖြစ်ဖြစ်၊ System ရဲ့ ချို့ယွင်းမှုကြောင့်ပဲ ဖြစ်ဖြစ် အလုပ်လုပ်နေစဉ်မှာ မှားယွင်းမှုတွေကို ဖြစ်ပေါ်စေပါတယ်။ ဘယ်လိုကြောင့်ပဲမှားမှားဒီလိုအနေအထားမျိုးမှာ Application အနေနဲ့ ဘယ်လို ဆက်လုပ်မလဲ ဆိုတာကို ဆောင်ရွက်ပေးနိုင်ဖို့လိုအပ်ပါတယ်။ ဒါမှသာ Application က ဆက်အလုပ်မလုပ်ပဲ ရပ်သွားတာတို့၊ နောက်ထပ် လုပ်ဆောင်မှုတွေကို ဆက်မလုပ်နိုင်တာတို့ User ကို Data အမှားတွေ ပြမိတာတို့ မဖြစ်အောင် ဆောင်ရွက်ပေးနိုင်မှာ ဖြစ်ပါတယ်။

Java မှာတော့ Exception ဆိုတဲ့ Mechanism ကို အသုံးပြုပြီး Error အဖြစ်များတဲ့ နေရာတွေကို ကြိုတင်ပြင်ဆင်ထားနိုင်အောင်နဲ့၊ Error တွေဖြစ်လာတဲ့အခါမှာလဲ Handle လုပ်နိုင်အောင် ဆောင်ရွက်ထားပါတယ်။ Application တစ်ခုရဲ့ တန်ဖိုးဟာ ဘယ်လောက်အထိမှန်ကန်မှုကိုအာမခံနိုင်သလဲ၊ ဘယ်လောက်အထိ အသုံးပြုတဲ့သူကို အစဉ်ပြေပြေနဲ့ ဆောင်ရွက်နိုင်အောင် စီစဉ်ထားနိုင်သလဲ ဆိုတဲ့ အပေါ်မှာ မူတည်နေပါတယ်။ ဒါ့ကြောင့် မိမိရဲ့ Application ကို မှန်ကန်မှုရှိအောင် ဘယ်လိုလုပ်မလဲနဲ့ အမှားတွေနဲ့ ကြုံတွေ့တဲ့နေရာမှာ ဘယ်လောက်အထိ အသုံးပြုသူအတွက် နားလည်လွယ်အောင် ဆောင်ရွက်ပေးရမလဲ ဆိုတာကို Application ကို စတင်ရေးသားကထဲက ကြိုတင်ပြင်ဆင်ထားဖို့လိုအပ်ပါတယ်။

ဒါ့ကြောင့် Essential Java API တွေအကြောင်းရေးတော့မယ်ဆိုတာနဲ့ Exception အကြောင်းကို ရှေ့ဆုံးကနေ ရေးသားထားတာဖြစ်ပါတယ်။

What is Exception?

Method Invocation တွေကို လုပ်ဆောင်နေစဉ် မျှော်လင့်မထားတဲ့ အမှားတစ်ခုခုနဲ့ တွေ့ကြုံတဲ့အခါ Java Runtime ဟာ ဘာဆက်လုပ်ရမှန်း ဝေခွဲမရတော့ပဲ ကျန်တာတွေကို ဆက်မလုပ်တော့ပဲ Event Object တစ်မျိုးကို Create လုပ်ကာ Runtime ကို ပြန်ပေးလိုက်ပါတယ်။

အဲ့ဒီ Event Object တွေဟာ Exception တွေဖြစ်ကြပါတယ်။ Exception Object ထဲမှာ ဖြစ်ပေါ်စေတဲ့ Error နဲ့ ပတ်သက်တဲ့ အချက်အလက်တွေ၊ အဲ့ဒီ Exception ရဲ့ အမျိုးအစားနဲ့ အဲ့ဒီ Exception ကို ဖြစ်ပေါ်နေစဉ်ရှိနေတဲ့ Program ရဲ့ State တွေကို ဖေါ်ပြပေးနိုင်တဲ့ အချက်အလက်တွေကို ဖေါ်ပြထားပါတယ်။

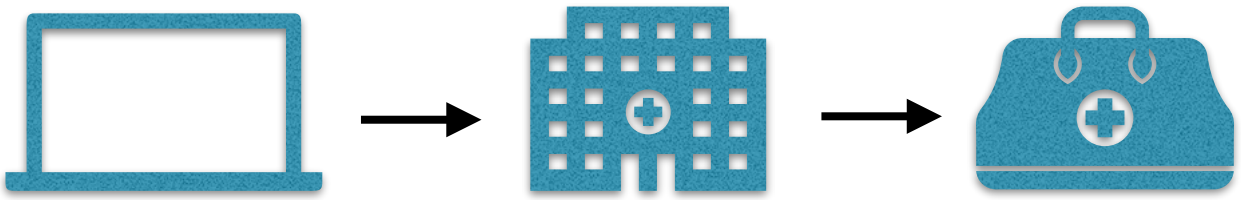
ကျွန်တော်တို့ ရေးသားထားတဲ့ Application တွေထဲမှာ Exception ဖြစ်လာပြီဆိုရင် အဲ့ဒီ Exception Object ကို ကြည့်ခြင်းအားဖြင့်၊ ဘာကြောင့် ဘယ်နေရာမှာ ဘယ်လိုအမှားမျိုးကို ဖြစ်စေတယ်ဆိုတာကို သိရှိစေနိုင်မှာ ဖြစ်ပါတယ်။ ကျွန်တော်တို့ Java ကို စတင်လေ့လာပြီး Java Program တွေကို ရေးသားတဲ့ နေရာမှာ မကြာခဏဆိုသလို Exception တွေနဲ့ တွေ့ကြရမှာ ဖြစ်ပါတယ်။ ရေးပြီး Run လိုက်တဲ့ Program မှာ Exception ဖြစ်သွားပြီဆိုရင် စာတွေအများကြီးထွက်လာတတ်ပါတယ်။ ရုတ်တရက် အဲ့ဒီလိုအနေအထားနဲ့ ကြုံတွေ့တဲ့ အခါမှာ ကျွန်တော်တို့ တွေဟာ ပြာလောင်သွားပြီး ဘာလုပ်လို့ လုပ်ရမှန်းမသိပဲ ဖြစ်သွားတတ်ပါတယ်။

အထူးသဖြင့် Programming ကို စတင်လေ့လာသူတွေဆိုရင် အဲ့ဒီစာတွေတွေ့ရင် ဘာလုပ်ရမှန်းမသိဖြစ်သွားတဲ့ အတွက် IDE နဲ့ Application ကို Run နေတဲ့ အခါမျိုးမှာတောင် Console ကို ပိတ်ထားတတ်လေ့ရှိပါတယ်။ တကယ်တမ်းကျတော့ အဲ့ဒါဟာ မှားယွင်းမှုတစ်ခုဖြစ်ပါတယ်။ အဲ့ဒီဘာမှန်းမသိတဲ့ စာတွေထဲမှာ ဘယ်နေရာမှာ ဘာကြောင့် ဒီအမှားကို မှားရတယ်ဆိုတဲ့ အချက်အလက်တွေကို ဖေါ်ပြထားပါတယ်။ ဘာကြောင့်မှားတယ်ဆိုတာကို သိမှ ဘယ်လို ဖြေရှင်းရမယ်ဆိုတာကို သိရှိနိုင်မှာဖြစ်ပါတယ်။ အဲ့ဒါကြောင့် ကျွန်တော် အထူးမှာချင်တာက Exception Message တွေကို ဖတ်ဖို့ မပျင်းပါနဲ့ ဆိုတာပါပဲ။

ဆက်လက်ပြီး Java Runtime မှာ ဘယ်လို Exception ကို ဖြစ်ပွါးစေပြီး၊ အဲ့ဒီ Exception ကို Runtime မှာ ဘယ်လို Handle လုပ်သွားတယ်ဆိုတာကို လေ့လာသွားကြရအောင်။

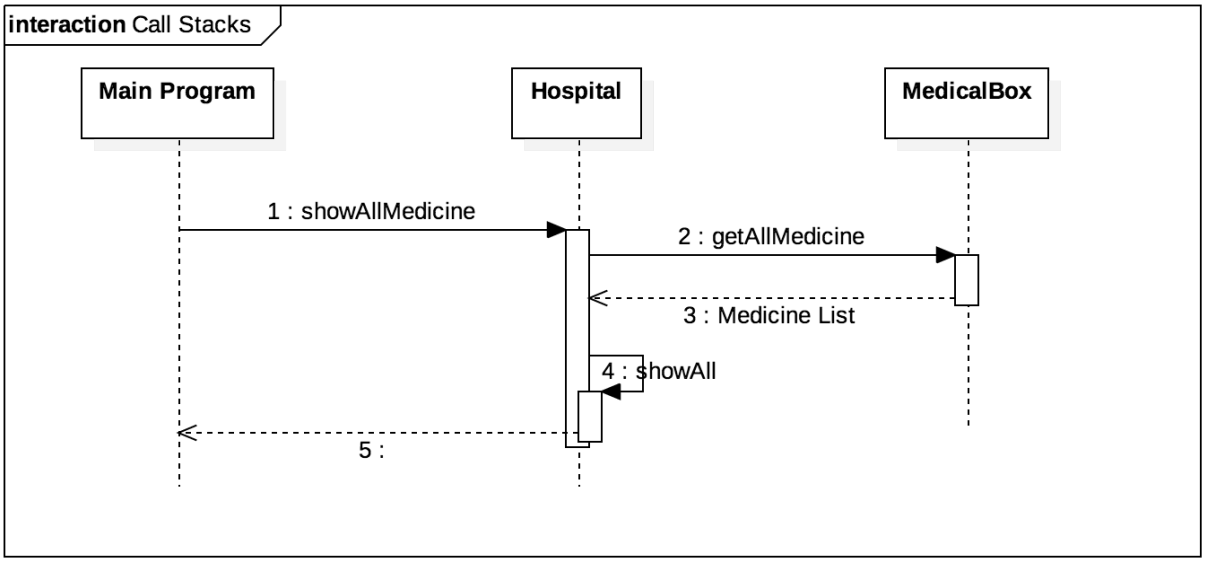
Call Stacks

Program တွေကို လုပ်ဆောင်နေစဉ် Error ဖြစ်ပေါ်လာပြီး Exception Object ကို Create လုပ်ပြီး Runtime ကို ပြန်ပေးတာကို Exception ကို Throw လုပ်တယ်လို့ ခေါ်ဆိုပါတယ်။



Main Program Hospital MedicalBox

Java Application တွေကို Run မယ်ဆိုရင် အမြဲတမ်း Main Method ကနေ စတင် အလုပ်လုပ်ခိုင်းရမှာ ဖြစ်ပါတယ်။ Hospital မှာ ရှိတဲ့ ဆေးတွေကို ပြန်ပြတဲ့ Application ကို ရေးနေကြတယ်ဆိုကြပါစို့။ Hospital Object မှာ showAllMedicine() ဆိုတဲ့ Method ကို ရေးသားထားပါမယ်။ Hospital ထဲမှာ MedicalBox ဆိုတဲ့ Object ကို ပိုင်ပြီး အဲ့ဒီ Object ထဲမှာ Medicine တွေကို သိမ်းထားပါမယ်။ အဲ့ဒီ MedicalBox Object မှာတော့ getAllMedicine() ဆိုတဲ့ Method ကို ပိုင်ဆိုင်ပြီး၊ Hospital ရဲ့ showAllMedicine() ကို လုပ်ခိုင်းတဲ့ အခါမှာ Hospital က သူပိုင်တဲ့ MedicalBox ရဲ့ getAllMedicine() ကို အသုံးပြုပြီး ရလာတဲ့ Medicine တွေကို ပြန်ပြပေးမှာ ဖြစ်ပါတယ်။



အထက်ပါအတိုင်း Main Program ရဲ့ main() method ထဲကနေ Hospital Object ရဲ့ showAllMedicine() ကို လှမ်းခေါ်ပါမယ်။ အဲ့ဒီအခါမှာ Hospital ရဲ့ showAllMedicine() Method ထဲကနေ MedicalBox Object ရဲ့ getAllMedicine() ကို ဆက်ခေါ်ပါမယ်။ MedicalBox ရဲ့ getAllMedicine() Method ရဲ့ Return တန်ဖိုးဖြစ်တဲ့ Medicine List ကို အသုံးပြုပြီး Hospital Object က သူ့ရဲ့ ကိုယ်ပိုင် Method ဖြစ်တဲ့ showAll() Method ကို ပြန်ခေါ်ပါလိမ့်မယ်။ အဲ့ဒီ လုပ်ဆောင်မှုပြီးသွားတဲ့ အခါကျမှ Main Method ကို Control ကိုပြန်ပေးမှာ ဖြစ်ပါတယ်။ ဒီလိုမျိုး Main Method ကနေ စပြီး အခြားသော Method တွေကို အဆင့်ဆင့် Invoke လုပ်သွားတာကို Call Stack လို့ခေါ်ဆိုပါတယ်။

အဲ့ဒီ Call Stack အတွင်း တစ်နေနေရာမှာ Exception ကို ဖြစ်ပေါ်စေခဲ့ရင် Java Runtime ဟာ ရှေ့ကိုဆက် မလုပ်တော့ပဲ၊ Call Stack တစ်လျှောက် ဖြစ်ပေါ်လာတဲ့ Exception ကို Handle လုပ်နိုင်မည့် Exception Handler ကို လိုက်ရှာပါတယ်။ Handler ကို တွေ့ခဲ့ရင်တော့ အဲ့ဒီ Handler ကို Handle လုပ်ခိုင်းပါတယ်။ ဒီလိုမျိုး သင့်တော်တဲ့ Exception handler ကိုတွေ့ပြီး Handler ကနေ Exception ကို လက်လွဲယူလိုက်တာကိုတော့ Exception ကို Catch လုပ်တယ်လို့ခေါ်ပါတယ်။

မတွေ့ခဲ့ဘူးဆိုပါက Call Stack အတိုင်း Main Method ကို ရောက်တဲ့အထိ တဆင့်ချင်း Handler ကို ဆက်ရှာသွားပါတယ်။ Call Stack ရဲ့ အစဖြစ်တဲ့ Main Method ကိုရောက်တဲ့အထိ Exception Handler ကို မတွေ့ခဲ့ဘူးဆိုပါက ဖြစ်ပေါ်လာတဲ့ Exception အကြောင်းကို Console ပေါ်မှာ ဖေါ်ပြပြီး System ကို Error Status နဲ့ ပြီးဆုံးစေမှာ ဖြစ်ပါတယ်။

အထက်ပါ နမူနာကို လက်တွေ့ ကုဒ်တွေနဲ့ ရေးသားပြီး စမ်းသပ်ကြည့်ပါမယ်။ အောက်ဖေါ်ပြပါကုဒ်ကတော့ MedicalBox Class ဖြစ်ပါတယ်။

```
class MedicalBox {
	private List<String> medicines;

	public List<String> getAllMedicine() {
		return new ArrayList<>(medicines);
	}
}
```

ဒီနေရာမှာ Exception ကို ဖြစ်ပေါ်စေအောင် getAllMedicine() Method ထဲမှာ Null တန်ဖိုးရဲ့တဲ့ medicines ကို ArrayList Constructor ရဲ့ parameter အနေနဲ့ အသုံးပြုထားပါတယ်။ ဒါ့ကြောင့် ဒီနေရာကိုရောက်တဲ့အခါမှာ null တန်ဖိုးရှိတဲ့ medicines ကို သွားသုံးမိတဲ့ အတွက် NullPointerException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

```
class Hospital {

	private MedicalBox box;

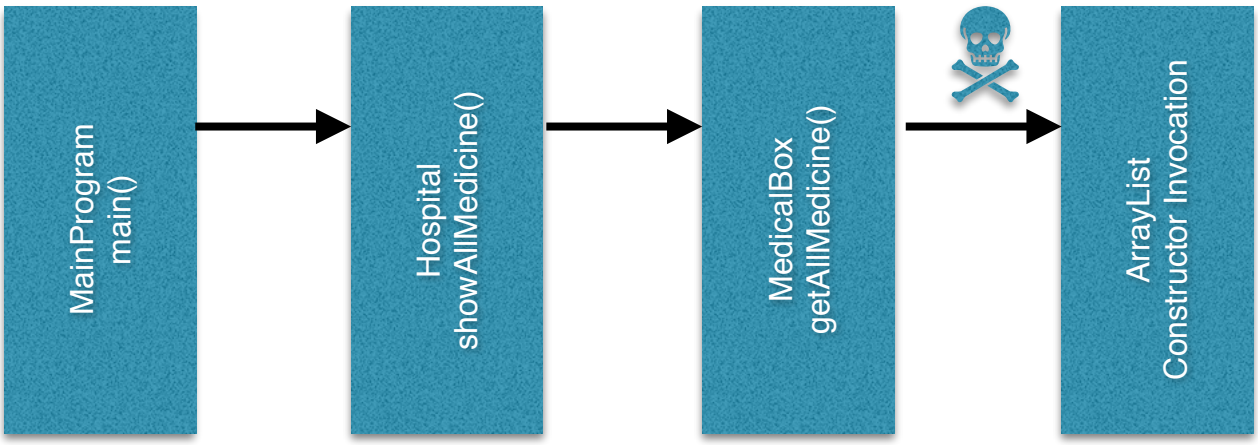
	Hospital() {
		box = new MedicalBox();
	}

	public void showAllMedicine() {
		List<String> medicines = box.getAllMedicine();
		for(String str : medicines) {
			System.out.println(str);
		}
	}
}
```

အထက်ပါ Class ကတော့ MedicalBox ကို အသုံးပြုမည့် Hospital Class ဖြစ်ပါတယ်။ Hospital ရဲ့ showAllMedicine() method ထဲကနေ MedicalBox ရဲ့ getAllMedicine() ကို ပြန်ပြီး Invoke လုပ်နေပါတယ်။

```
public class MainProgram {
	public static void main(String[] args) {
		Hospital hospital = new Hospital();
		hospital.showAllMedicine();
	}
}
```

အထက်ပါ Class ကတော့ Program ကို စတင်စေမည့် MainProgram Class ဖြစ်ပါတယ်။ main() method က နေစပြီး Hospital ရဲ့ showAllMedicine() ကို ခေါ်တယ်။ အဲ့ဒီ Method ထဲကနေ ဆက်လက်ပြီး MedicalBox ရဲ့ getAllMedicine() method ကို ခေါ်ပါမယ်။ အဲ့ဒီ getAllMedicine() method ထဲမှာ ArrayList ရဲ့ Constructor ကိုသုံးပြု Object ဆောက်တဲ့နေရာမှာ Exception ကို ဖြစ်ပေါ် စေမှာ ဖြစ်ပါတယ်။



အထက်ပါ Program ကို Run ကြည့်ပါမယ်။

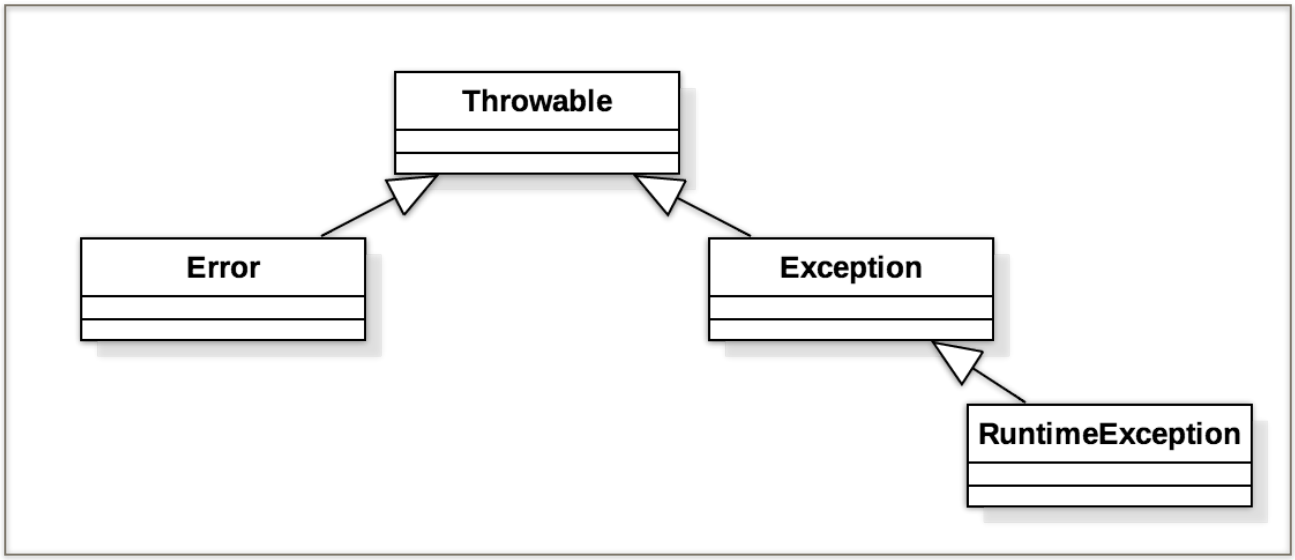
```
Exception in thread "main" java.lang.NullPointerException
     at java.util.ArrayList.<init>(ArrayList.java:177)
     at MedicalBox.getAllMedicine(MainProgram.java:24)
     at Hospital.showAllMedicine(MainProgram.java:12)
     at MainProgram.main(MainProgram.java:31)
```

Call Stack တလျောက်မှာ ဘာ Exception Handler ကို မရေးသားထားခဲ့တဲ့ အတွက် နောက်ဆုံး Main Thread ကို ပြီးဆုံးစေပြီး Exception ကို ဘယ်လို ဖြစ်ခဲ့ကြောင်းကို ဖေါ်ပြပေးနေတာပဲ ဖြစ်ပါတယ်။ Message ကို ကြည့်ခြင်းအားဖြင့် ArrayList ကို Initialize လုပ်တဲ့ နေရာမှာ NullPointerException ကို ဖြစ်ပေါ်စေခဲ့ကြောင်းကို သိရနိုင်ပါတယ်။ Exception ဖြစ်ပေါ်ပုံအဆင့်ဆင့်ကို Program ကို စတင်ခဲ့တဲ့ နေရာအထိ တဆင့်ခြင်း Line Number နှင့်တကွ ဖေါ်ပြပေးနေတာကို တွေ့ရပါတယ်။

Types of Exception

Exception တွေကို အခြေခံအားဖြင့် Checked Exception နဲ့ Unchecked Exception ဆိုပြီးခွဲခြားနိုင်ပါတယ်။ Checked Exception တွေကတော့ ကုဒ်ရေးသားတဲ့နရာမှာ အမှားများတဲ့အတွက် ကြိုတင်စစ်ဆေးပြီး ဘယ်လို Handle လုပ်မယ်ဆိုတာကို ဆုံးဖြတ်ဖို့လိုအပ်တဲ့ Exception အမျိုးအစားတွေဖြစ်ကြပါတယ်။ Unchecked Exception တွေကတော့ အမြဲတော့မှားတာမဟုတ်ဘူး၊ ပတ်ဝန်းကျင်ကြောင့်ဒါမှမဟုတ် User Operation ကြောင့်မှားတဲ့ အမှားတွေဆိုရင်တော့ Unchecked Exception တွေကို အသုံးပြုလေ့ရှိကြပါတယ်။

တဖန် Exception တွေကို Inheritance Hierarchy အရလဲ ခွဲခြားသတ်မှတ်နိုင်ပါတယ်။



✓ Exception
Checked Exception အမျိုးအစားဖြစ်ပြီး Exception Class နဲ့ RuntimeException အမျိုးအစားမဟုတ်တဲ့ Exception ရဲ့ Sub Class တွေဟာ Exception တွေဖြစ်ကြပါတယ်။ FileNotFoundException, IOException နဲ့ SQLException တို့ဟာ ထင်ရှားတဲ့ Checked Exception အမျိုးအစားတွေ ဖြစ်ကြပါတယ်။

✓ RuntimeException
Unchecked Exception အမျိုးအစားတစ်မျိုးဖြစ်ပြီး RuntimeException Class နဲ့ RuntimeException ရဲ့ Sub Class တွေအားလုံးဟာ Runtime Exception အမျိုးအစားတွေ ဖြစ်ကြပါတယ်။

NullPointerException, ArrayIndexOutOfBoundsException နဲ့ ArithmeticException တို့ဟာအသိများတဲ့ Runtime Exception တို့ဖြစ်ကြပါတယ်။ Runtime Exception တွေဟာ Unchecked Exception အမျိုးအစားထဲမှာပါဝင်ပါတယ်။

✓ Error
Unchecked Exception အမျိုးအစားတစ်မျိုးဖြစ်ပြီး Application တွေကနေ Catch လုပ်ပြီး Handle မလုပ်သင့်တဲ့ အမှားမျိုးတွေကို Error အနေနဲ့ ရေးသား အသုံးပြုလေ့ရှိကြပါတယ်။ Error Class နဲ့ Error အောက်က Sub Class တွေအားလုံးဟာ Error အမျိုးအစားတွေ ဖြစ်ကြပါတယ်။ NoClassDefFoundError, OutOfMemoryError နဲ့ StackOverflowError တို့ဟာ ထင်ရှားတဲ့ Error တွေ ဖြစ်ကြပါတယ်။ Error တွေကလဲ Unchecked Exception အမျိုးအစားဖြစ်ပါတယ်။

Handling Exceptions

ကျွန်တော်တို့ ရှေ့မှာ Java Runtime ကနေ Method Invocation တွေရဲ့ Statement တွေကို လုပ်ဆောင်နေစဉ် အကြောင်းကြောင်းကြောင့် ဆက်လုပ်လို့မရတဲ့ အနေအထားနဲ့ ကြုံတွေ့ရတတ်ပါတယ်။ အဲ့ဒီအခါမျိုးမှာ Method Invocation တွေကို ဆက်မလုပ်တော့ပဲ Event Object တစ်ခုကို Create လုပ်ပြီး Runtime ကို ပြန်ပေးလိုက်ပါတယ်။ ဒါကို Exception ကို Throw လုပ်တယ်လို့ခေါ်လိုက်ပါတယ်။

Runtime ကနေ ဖြစ်ပေါ်လာတဲ့ Exception Object ကို အမျိုးအစားအလိုက် Handle လုပ်နိုင်မည့် Handler တွေကို Call Stack တောက်လျောက်လိုက်ရှာပြီး တွေ့တဲ့ အခါမှာ အဲ့ဒီ Handler ကို Exception Object ကို ပေးပြီး Handle လုပ်ခိုင်းပါတယ်။ ဒီလို အဖြစ်အပျက်ကို Exception ကို Catch လုပ်တယ်လို့ ခေါ်ပြန်တယ်။

တဖန် Checked Exception တွေကို Throw လုပ်ထားတဲ့ Method တွေ Constructor တွေကို အသုံးပြုမိတဲ့အခါမျိုးမှာလဲ Exception တွေကို Handle လုပ်ဖို့ပြင်ဆင်ထားဖို့လိုအပ်ပါတယ်။ ဒီနေရာမှာ Catch လုပ်ပြီး Handle လုပ်မလား လတ်တလော Handle မလုပ်ပဲ Throw လုပ်မလား ဆိုတာကို ရွေးချယ်ရတော့မှာ ဖြစ်ပါတယ်။

Catching Exception

Exception တွေကို Catch လုပ်ဖို့အတွက် Statement တွေကို ပြင်ဆင်ထားပါတယ်။ အသုံးပြုနိုင်တဲ့ Statement တွေကတော့ TRY - CATCH Statement, TRY - CATCH - FINALLY Statement, TRY - MULTI CATCH Statement နဲ့ TRY - WITH - RESOURCE Statement တို့ ဖြစ်ကြပါတယ်။

TRY - CATCH

Try - Catch Statement ဟာ Java မှာ Exception ကို Handle လုပ်ဖို့အတွက် ရေးသားရသော အခြေခံ ရေးသားပုံ Statement တစ်ခုဖြစ်ပါတယ်။ Try - Catch Statement တစ်ခုမှာ အခြေခံအားဖြင့် Try Clause နဲ့ Catch Clause တို့ပါဝင်ကြပါတယ်။ တစ်ခုသတိပြုဖို့လိုတာက Try Catch Statement ကို ဘယ်လို ရေးသားရမယ်ဆိုတဲ့ အပိုင်းနဲ့ Try Catch Statement ကို ဘယ်နေရာမှာရေးသားသင့်တယ်ဆိုတာက တစ်ပိုင်းစီဖြစ်ပါတယ်။ ဒီနေရာမှာတော့ ဘယ်လိုရေးသားရမယ်ဆိုတာကိုပဲ အဓိထားပြီးလေ့လာသွားကြရအောင်။

```
try {
      // code that can happen exceptions
} catch (Exception e) {
      // code that handle exception
}
```

Try Clause ကိုတော့ try {} ဆိုပြီးရေးသားရမှာ ဖြစ်ပါတယ်။ try ရဲ့ နောက်က Code Block ထဲမှာတော့ Exception ကို ဖြစ်စေနိုင်တဲ့ ကုဒ်တွေကို ရေးသားရမှာ ဖြစ်ပါတယ်။

Catch Clause ကတော့ Exception Handler အပိုင်းဖြစ်ပါတယ်။ Catch Clause ကို ရေးသားတဲ့နေရာမှာ catch(Exception) {} ကို ရေးသားရမှာ ဖြစ်ပါတယ်။ catch ရဲ့နောက်က () အထဲမှာတော့ Handle လုပ်နိုင်တဲ့ Exception ကို Argument အနေနဲ့ ရေးသားရမှာ ဖြစ်ပါတယ်။

Try Catch Statement ထဲမှာ Try ကိုတော့ တစ်ကြိမ်သာရေးသားနိုင်ပေမဲ့ Catch Clause ကိုတော့ အကြိမ်ကြိမ် ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

```
try {
      // code that can happen exceptions
} catch (NullPointerException e) {
      // code that handle NullPointerException
} catch (Exception e) {
      // code that handle Exception
}
```

အထက်ပါအတိုင်း Try - Catch Statement မှာ NullPointerException ကို Handle လုပ်မည့် Exception Handler နဲ့ Exception ကို Handle လုပ်မည့် Handler တို့ကို ရေးသားထားနိုင်ပါတယ်။ ဒီလိုရေးသားတဲ့နေရာမှာ Exception Object ရဲ့ Inheritance Hierarchy ကို သတိပြုဖို့တော့ လိုအပ်ပါတယ်။ Inheritance Hierarchy ထဲမှာ အမြင့်ဆုံး Type ကို အောက်ဆုံးမှာရေးသားထားဖို့လိုအပ်ပါတယ်။

TRY - CATCH - FINALLY

Try - Catch Clause တွေနဲ့ အတူ Finally Clause ကို တွဲဖက်ရေးသားလို့ရပါတယ်။ Finally Clause ဟာ အဲ့ဒီ Statement ကိုလုပ်ဆောင်ပြီး နောက်ဆုံးမှာ သေခြာပေါက် လုပ်ဆောင်စေလိုတဲ့ ကုဒ်တွေကိုရေးသားလိုတဲ့ နေရာမှာ အသုံးပြုနိုင်ပါတယ်။ Exception ဖြစ်ဖြစ် မဖြစ်ဖြစ် Finally Clause ထဲကကုဒ်တွေကိုတော့ နောက်ဆုံးမှာ လုပ်ဆောင်ပေးမှာ ဖြစ်ပါတယ်။ တကယ်လို့ Try Block ထဲမှာ Return ပြန်ထားမယ်ဆိုရင်တောင်မှ နောက်ဆုံးမှာ Finally Clause ထဲက ကုဒ်တွေကို လုပ်ဆောင် စေမှာ ဖြစ်ပါတယ်။

```
try {
      // code that can happen exceptions
} catch (NullPointerException e) {
      // code that handle NullPointerException
} catch (Exception e) {
      // code that handle Exception
} finally {
      // နောက်ဆုံးမှာ သေခြာပေါက်လုပ်ဆောင်စေလိုတဲ့ ကုဒ်တွေကို ရေးသားရပါမယ်
}
```

Finally Clause ကိုရေးသားတဲ့နေရာမှာ Catch တွေရဲ့ နောက်ဆုံးမှာ ရေးသားရမှာ ဖြစ်ပါတယ်။ Finally Clause ရဲ့နောက်မှာ Catch Clause တွေကို ရေးသားလို့ မရနိုင်ပါဘူး။

TRY - MULTI CATCH

Java SE 7 အရောက်မှာ Exception Handling ရေးသားပုံ Statement နှစ်ခု အသစ်ပါဝင်လာခဲ့ပါတယ်။ အခဲ့ဒီထဲကတစ်ခုကတော့ Try With Multi Catch နဲ့ Try With Resource Statement တို့ပဲ ဖြစ်ကြပါတယ်။

Try With Multi Catch ဆိုတာကတော့ အခြေခံအားဖြင့် Try - Catch Statement နဲ့ အတူတူပဲ ဖြစ်ပါတယ်။ ထူးခြားတာကတော့ Catch Block တွေမှာ Exception Type တွေကို တစ်ခုထက်မက ရေးသားလာနိုင်တာပဲ ဖြစ်ပါတယ်။ ဒီနေရာမှာလဲ Exception Type တွေရဲ့ Class Hirarechy ကိုလဲ သတိပြုဖို့လိုအပ်ပါတယ်။ Exception Type တွေကို ရေးသားတဲ့ နေရာမှာ Super Type တွေကို ရှေ့မှာ ရေးသားလို့ မရနိုင်ပါဘူး။

```
try {
      // code that can happen exceptions
} catch(NullPointerException | IOException e) {
      // Exception Handling Codes
}
```

အထက်ပါအတိုင်း Catch Block ထဲမှာ ရေးသားထားခြင်းအားဖြင့် လက်ရှိ Handler မှာ NullPointerException နဲ့ IOException တို့ကို Handle လုပ်နိုင်မှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် Multi Catch Statement ကို Exception Handler တစ်ခုနဲ့ အမျိုးအစားမတူတဲ့ Exception တွေကို Handle လုပ်လိုတဲ့ အခါမျိုးမှာ ရေးသားအသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

TRY - WITH - RESOURCE

Try - With - Resource Statement ဟာ AutoClosable Interface ကို Implement လုပ်ထားတဲ့ Object တွေကို အသုံးပြုတဲ့ အခါမှာ ရေးသားနိုင်တဲ့ Statement အမျိုးအစားဖြစ်ပါတယ်။

```
try (Connection conn = this.getConnection();
      Statement stmt = conn.createStatement()){

      // Database Operation

} catch (Exception e) {
      // codes for handeling

}
```

AutoClosable Interface ကို Implement လုပ်ထားတဲ့ Object တွေကို try နောက်က () ထဲမှာ ထည့်သွင်းရေးသားထားရပါမယ်။ ထိုသို့ရေးသားထားခြင်းအားဖြင့် မိမိကိုယ်တိုင် Resource တွေကို Close လုပ်စရာမလိုအပ်တော့ပါဘူး။ Statement ပြီးဆုံးတဲ့ အခါမှာ အလိုအလျောက် Close လုပ်ပေးနိုင်မှာ ဖြစ်ပါတယ်။ Close လုပ်စေလိုတဲ့ Resource Object တွေ တစ်ခုထက်မက ရှိတဲ့အခါမျိုးမှာ အထက်ပါအတိုင်း Seme Column (;) ကိုအသုံးပြုပြီး ရေးသားနိုင်ပါမယ်။

Throwing Exceptions

Checked Exception တွေကို Throw လုပ်ထားတဲ့ Method တွေ၊ Constructor တွေကို အသုံးပြုမိပြီး လတ်တလော မဖြေရှင်းနိုင်သေးတဲ့ အခါ၊ မိမိရေးသားထားတဲ့ Application တွေရဲ့ မှန်ကန်မှုကို ထိမ်းသိမ်းလိုတဲ့ အတွက် နောက်က Code တွေကို ဆက်မလုပ်စေလိုတဲ့အခါမျိုးမှာ Application တွေထဲကနေလဲ Exception တွေကို Throw လုပ်နိုင်ပါတယ်။

Exception တွေကို Throw လုပ်တဲ့အခါမှာ Checked Exception တွေကို Throw လုပ်တာနဲ့ Unchecked Exception တွေကို Throw လုပ်တာက မတူညီပါဘူး။

Throwing Unchecked Exception

မိမိရေးသားထားတဲ့ Application ကို မှန်ကန်မှုရှိစေရန် ပြင်ပါမှ Data များကို Set လုပ်တဲ့နေရာမျိုးမှာ စစ်ဆေးပြီး မှားယွင်းနေပါက Unchecked Exception တွေကို Throw လုပ်ပြီး ကာကွယ်နိုင်ပါတယ်။

```
public class Person {

      private String name;
      private int age;

      public Person(String name, int age) {

            if(age <= 0) {
                  throw new IllegalArgumentException(
                        “Age must be greater than 0");
            }

            this.name = name;
            this.age = age;
      }
}
```

အထက်ပါအတိုင်း Person Class ထဲမှာ name နဲ့ age ကို ယူတဲ့ Constructor တစ်ခုကို ရေးသားထားပါတယ်။ ဒီနေရာမှာ ရေးသားထားတဲ့ Person Class ရဲ့ Data တွေကို မှန်ကန်မှုရှိအောင် စစ်ဆေးဖို့လိုအပ်ပါတယ်။ အသက်က Minus တန်ဖိုးရှိတဲ့ Person တို့ဆိုတာ အဓိပ္ပါယ်မရှိပါဘူး။

ဒါ့ကြောင့် Constructor ထဲမှာ age <= 0 ဆိုရင်လဲ IllegalArgumentException ကို Throw လုပ်ထားပါတယ်။ IllegalArgumentException ဟာ Runtime Exception အမျိုးအစားဖြစ်တဲ့ အတွက် Unchecked Exception အမျိုးအစားဖြစ်ပါတယ်။ ဒါ့ကြောင့် Throw လုပ်ထားပေမဲ့ အဲ့ဒီ Constructor Body ထဲမှာ Handle လုပ်ထားစရာမလိုအပ်ပါဘူး။

ဒါ့ကြောင့်ဒီနေရာမှာ Try Catch တို့ Method Declaration တွေမှာ Throw တွေကို ရေးထားစရာမလိုအပ်ပါဘူး။ အလားတူပဲ Error တွေဟာလဲ Unchecked Exception အမျိုးအစားတွေ ဖြစ်ကြတဲ့ အတွက် Handle လုပ်စရာမလိုအပ်ပါဘူး။

Throwing Checked Exception

Method ဒါမှမဟုတ် Constructor တစ်ခုခုထဲမှာ Checked Exception တစ်ခုကို Throw လုပ်မယ်ဆိုရင်တော့ Unchecked Exception တွေနဲ့ မတူတော့ပါဘူး။ Checked Exception တွေဟာ အများများတဲ့နေရာမှာ Programmer တွေကို ကြိုတင်ပြင်ဆင်စေချင်တဲ့ အခါမှာ အသုံးပြုကြရတဲ့ Exception အမျိုးအစားဖြစ်တဲ့ အတွက် ရေးသားထားတဲ့ Method ထဲမှာ Handle လုပ်ပေးဖို့လိုအပ်ပါတယ်။

ရှေ့မှာရေးသားထားတဲ့ IllegalArgumentException နေရာမှာ Exception ကို ပြောင်းပြီးရေးလိုက်ပါမယ်။ Person Class ထဲမှာ Checked Exception ကို Throw လုပ်ထားတာ ဖြစ်ပါတယ်။

```
public class Person {

      private String name;
      private int age;

      public Person(String name, int age) {

            if(age <= 0) {
                  throw new Exception(
                        "Age must be greater than 0");
            }

            this.name = name;
            this.age = age;
      }
}
```

အထက်ပါ Class ကို Compile လုပ်တဲ့နေရာမှာ Checked Exception ကို Throw လုပ်ထားတဲ့ အတွက် catch ဒါမှမဟုတ် throws declaration ကို ရေးသားထားဖို့လိုအပ်တယ်ဆိုပြီး Compiler မှာ စစ်ဆေးပေးနေတာကို တွေ့ရပါတယ်။

```
Person.java:9: error: unreported exception Exception; must
be caught or declared to be thrown
                  throw new Exception("Name must not be
null!");
                  ^
1 error
```

Checked Exception တွေကို Throw လုပ်တော့မယ်ဆိုရင်ရွေးချယ်စရာ Option နှစ်ခုရှိလာပါတယ်။ Catch လုပ်မလား Throw လုပ်မလား ဆိုတာပါပဲ။ ဘာကိုကြည့်ပြီး Catch လုပ်မလား Throw လုပ်မလားဆိုတာကို ရွေးချယ်ကြရမှာလဲ ဆိုတာကို ဆက်ပြီး လေ့လာသွားကြရအောင်။

Catch လုပ်မလား Throw လုပ်မလားဆိုတာကတော့ မိမိရေးသားနေတဲ့ Method ရဲ့ အခြေအနေအပါအဝင်မှာပဲ မူတည်ပြီး ဆုံးဖြတ်ရမှာ ဖြစ်ပါတယ်။ တစ်ချို့ Exception တွေကတော့ ရေးနေတဲ့ Method ထဲမှာပဲ Handle လုပ်လို့ရတဲ့ အခါမျိုးလဲရှိနိုင်သလို တစ်ချို့ကျတော့လဲ လက်ရှိ Method ထဲမှာတော့ Handle လုပ်လို့မရဘူး။ ရေးထားတဲ့ Method ကို အသုံးပြုတဲ့ နေရာမှာမှ Handle လုပ်လို့ရတာမျိုးရှိတတ်ပါတယ်။ လက်ရှိ Method ထဲမှာပဲ Handle

လုပ်နိုင်တဲ့ အခါမျိုးမှာဆိုရင် Catch လုပ်ပြီး ဖြေရှင်းသင့်ပေမဲ့ လက်ရှိ Method ထဲမှာ Handle မလုပ်နိုင်တဲ့ အခါမျိုးမှာဆိုရင်တော့ Declaration ကနေ Throw လုပ်ပေးတာ သင့်တော်ပါတယ်။

ရှေ့မှာ Exception တွေကို ဘယ်လို Catch လုပ်မယ်ဆိုတာကို ဖော်ပြပြီး ဖြစ်တဲ့ အတွက် ဒီနေရာမှာတော့ ဘယ်လို Throw Declaration ကို ရေးသားရမယ်ဆိုတာကိုပဲ ဖော်ပြသွားပါမယ်။

```
public class Person {

      private String name;
      private int age;

      public Person(String name, int age) throws Exception {

            if(age <= 0) {
                  throw new Exception(
                        "Age must be greater than 0");
            }

            this.name = name;
            this.age = age;
      }
}
```

Throw Declaration ကို ရေးသားမယ်ဆိုရင် Constructor ဒါမှမဟုတ် Method Declaration တွေရဲ့ နောက်မှာ throws ဆိုပြီး Throw လုပ်လိုတဲ့ Exception တွေကို ရေးသားရမှာ ဖြစ်ပါတယ်။ Exception တစ်ခုထက်မက Throw လုပ်ချင်တယ်ဆိုရင် Comma (,) ကို ခံပြီးရေးသားပေးရမှာ ဖြစ်ပါတယ်။

```
public void readAndSaveToDB()
      throws IOException, SQLException {
      // other codes
}
```

တကယ်လို့ IOException နဲ့ SQLException ကို Method ထဲကနေ Throw လုပ်ချင်တယ်ဆိုရင် အထက်ပါအတိုင်း throws IOException, SQLException ဆိုပြီး ရေးသားပေးရမှာ ဖြစ်ပါတယ်။

Throw Declaration တွေမှာ Unchecked Exception တွေကို ရေးသားထားရင်လဲ ရပါတယ်။ ဒါပေမဲ့ မရေးထားရင်လဲ ပြဿနာမရှိတဲ့ အတွက် ရေးထားထား မရေးထားထား အတူတူပဲ ဖြစ်ပါမယ်။

One more Option

Checked Exception တွေကို Throw လုပ်ထားတဲ့ Method တွေ Constructor တွေကို အသုံးပြုမိတဲ့ အခါမျိုးမှာ၊ Catch လုပ်ပြီး ဖြေရှင်းမလား၊ Throw Declaration ကို ရေးမလားလို့ ရှေ့မှာ ဖေါ်ပြခဲ့ပါတယ်။ ဒါပေမဲ့ အခြားရွေးချယ်စရာ လမ်းတစ်ခုရှိနေပါသေးတယ်။ ဒါကတော့ Catch လုပ်ပြီး အခြားသော Exception တစ်ခုအနေနဲ့ ပြန်ပြီး Throw လုပ်ပစ်တာပဲ ဖြစ်ပါတယ်။

ဒါကလဲ Checked Exception တွေကို ဒီအတိုင်း Throw လုပ်ပစ်ရင် ဖြေရှင်းနိုင်တဲ့ အစွမ်းမရှိပဲ Catch လုပ်ပြီး ဖျောက်ပစ်မှာစိုးတဲ့ အတွက် ကြားထဲမှာ Catch လုပ်စရာမလိုအပ်တဲ့ Runtime Exception အနေနဲ့ ပြန်ပြီး Throw လုပ်ပစ်ချင်တဲ့ အခါမျိုးမှာ အသုံးပြုနိုင်ပါတယ်။ ဒါက API တွေကို ရေးသားတဲ့နေရာမှာတော့ အသုံးမပြုသင့်ပါဘူး။ Application တွေကို ရေးသားတဲ့ နေရာမှာတော့ အသုံးပြုသင့်ပါတယ်။

```
    public void insert(Customer c) {

        try(Connection conn == DriverManager.getConnection();
                Statement stmt = conn.createStatement()) {

            // Business Codes

        } catch (SQLException e) {
            throw new RuntimeException("Database Error", e);
        }
    }
```

အထက်ပါနမူနာက JDBC API ကို အသုံးပြုနေတဲ့ Insert Method လေးတစ်ခုဖြစ်ပါတယ်။ DriverManager နဲ့ Connection Object ကို ရယူပြီဆိုတာနဲ့ SQLException ကို ဖြစ်ပေါ်စေပါတယ်။ SQLException ဟာ Checked Exception ဖြစ်တဲ့ အတွက် Handle လုပ်ဖို့လိုပါတယ်။ ဒီနေရာမှာ SQLException ကို Catch လုပ်ပြီး Handle မလုပ်ပဲ နောက်ထပ် RuntimeException တစ်ခုအနေနဲ့ Throw လုပ်လိုက်ပါတယ်။

RuntimeException ဖြစ်တဲ့ အတွက် insert method မှာ throw declaration ကို ရေးသားစရာမလိုတော့ပါဘူး။ ပြီးတော့လဲ insert method ကို အသုံးပြုနေတဲ့ Call Site မှာလဲ မလိုအပ်ပဲ Handle လုပ်စရာမလိုအပ်ပါဘူး။

တကယ်တမ်းကျတော့ Exception တွေဟာ ဖြစ်ပေါ်လာရင် မှန်မှန်ကန်ကန် Handle လုပ်နိုင်ဖို့လိုအပ်ပါတယ်။ Handle လုပ်တယ်ဆိုတာက လက်ရှိလုပ်နေတဲ့ အချက်အလက်တွေမှန်ကန်စေဖို့အတွက် တစ်ပိုင်းတစ်စ သွင်းထားတဲ့ Data တွေကို ပြန်ပြီး Rollbback လုပ်တာတို့၊ ဘာကြောင့် Exception ဖြစ်တယ်ဆိုတာကို User ကို ပြန်ပြီး အကြောင်းကြားနိုင်ဖို့လိုအပ်ပါတယ်။

အဲ့ဒါတွေကို မလုပ်နိုင်ပဲ၊ တစ်နေနေရာကနေ Catch လုပ်ပြီး ဖျောက်ပစ်လိုက်တယ်ဆိုတာက ကြောက်စရာအကောင်းဆုံး အနေအထားတစ်ခုဖြစ်ပါတယ်။ ဒါ့ကြောင့် Application တွေကို ရေးသားတဲ့နေရာမှာ အသုံးပြုတဲ့နေရာတိုင်း မဖြစ်မနေ Handle လုပ်ဖို့လိုတဲ့ Checked Exception တွေထက်စာရင်၊ လိုအပ်မှသာ Handle လုပ်ဖို့လိုတဲ့ Unchecked Exception တွေကို Throw လုပ်တာများပါတယ်။

Custom Exception

ရှေ့မှာလေ့လာခဲ့ကြသလို Exception တွေကို Developer တွေက Create လုပ်ပြီး Throw လုပ်နိုင်ပါတယ်။ ဒီလို Throw လုပ်တဲ့နေရာမှာ ရှိပြီးသား Exception တစ်ခုကို Throw လုပ်လိုက်ရင်လဲရသလို မိမိကိုယ်ပိုင် Custom Exception တစ်ခုကို ရေးသားပြီး Throw လုပ်ရင်လဲ ရနိုင်ပါတယ်။

Platform မှာပါတဲ့ Exception တွေနဲ့ မိမိရဲ့ Application ထဲကနေ ဖြစ်တဲ့ Exception တွေကိုခွဲခြားလိုတဲ့အခါ၊ ဒါမှမဟုတ် Exception ထဲမှာ အခြားသော State တွေကို ပိုင်ဆိုင်ထားစေလိုတဲ့ အခါမျိုးမှာ Custom Exception တွေကို ရေးသားအသုံးပြုနိုင်ပါတယ်။

Writing Custom Exception

Custom Exception တစ်ခုကို ရေးသားမယ်ဆိုရင် Throwable Class အမျိုးအစား၊ နှစ်သက်ရာ Exception တစ်ခုကို Extends လုပ်ပြီးရေးသားနိုင်ပါတယ်။ ရှေ့မှာဖေါ်ပြခဲ့သလိုပဲ Checked Exception မဟုတ်တဲ့ Unchecked Exception ဖြစ်တဲ့ RuntimeException အမျိုးအစားတစ်ခုခုကို Extends လုပ်ပြီးရေးသားကြတာများပါတယ်။

```
public class HotelAppException extends RuntimeException {

}
```

HotelAppException ဟာ RuntimeException ကို Extends လုပ်ထားတဲ့အတွက် Unchecked Exception အမျိုးအစား Exception တစ်ခုဖြစ်လာပါတယ်။ ဒါ့ကြောင့် မလိုအပ်ပဲ Handle လုပ်စရာမလိုအပ်ပါဘူး။

```
public void insert(Customer c) {

      try(Connection conn == DriverManager.getConnection();
                  Statement stmt = conn.createStatement()) {

            // Data Validation
            if(null == c.getName()) {
                  throw new HotelAppException(
                        "Customer Name must not be null!");
            }

            // Business Codes

      } catch (SQLException e) {
            throw new HotelAppException("Database Error", e);
      }
}
```

Custom Exception ဖြစ်တဲ့ HotelAppException ကို Throw လုပ်ထားတဲ့အတွက် Handle လုပ်တဲ့နေရာမှာ မိမိ Application ကနေ Throw လုပ်တဲ့ Exception နဲ့ Platform ကနေ Throw လုပ်တဲ့ Exception တွေကို ခွဲခြားပြီး Handle လုပ်နိုင်မှာ ဖြစ်ပါတယ်။

Throwing Custom Exception

Custom Exception တွေကို Throw လုပ်တယ်ဆိုတဲ့နေရာမှာ အနေအထားနှစ်မျိုးရှိနိုင်ပါတယ်။ တစ်ခုကတော့ အခြား Exception တွေကို Catch လုပ်ပြီး Throw လုပ်တာဖြစ်ပြီး၊ နောက်တစ်ခုကတော့ မိမိရဲ့ Application Data တွေကို မှန်ကန်စေရန် စစ်ဆေးပြီး မသင့်တော်တဲ့နေရာမျိုးမှာ Throw လုပ်တာမျိုးဖြစ်ပါတယ်။ အနေအတားအပေါ်မှာမူတည်ပြီး Exception Object ထဲမှာ ထည့်ပေးသင့်တဲ့ Information တွေဟာ မတူနိုင်ပါဘူး။

အခြေခံအားဖြင့် Exception တစ်ခုထဲမှာ String message နဲ့ Throwable cause တိုကို သတ်မှတ်ပေးနိုင်ပါတယ်။ String message ကတော့ ဘာကြောင့်ဘယ်လိုဖြစ်တယ်ဆိုတာကို ဖေါ်ပြတဲ့နေရာမှာ အသုံးပြုသင့်ပြီး Exception Object ကနေ getMessage() Method ကို အသုံးပြုပြီး ပြန်လည်ရယူနိုင်ပါတယ်။ Exception ကို Create လုပ်တဲ့နေရာမှာ Message ကို သတ်မှတ်ပေးထားခဲ့ရင်၊ Exception ကို Handle လုပ်တဲ့နေရာမှာ ပြန်ပြီး အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ Throwable cause ကတော့ လက်ရှိ Exception ကို ဖြစ်စေတဲ့ မူရင်း Exception Object ကို သတ်မှတ်ပေးတဲ့နေရာမှာ အသုံးပြုနိုင်ပါတယ်။

```
    public void insert(Customer c) {

        try(Connection conn == DriverManager.getConnection();
                Statement stmt = conn.createStatement()) {
            // Business Codes

        } catch (SQLException e) {
            throw new HotelAppException("Database Error", e);
        }
    }
```

အခြား Exception တစ်ခုကို Catch လုပ်ပြီး Custom Exception ကို Throw လုပ်တဲ့ အနေအထားမျိုးမှာဆိုရင်၊ ဒီ Exception ကို ဖြစ်စေတဲ့ Root Cause Exception ကို သတ်မှတ်ပေးသင့်ပါတယ်။ ဒါမှသာ Handle လုပ်တဲ့နေရာမှာ ဒီ Exception ကို ဖြစ်ပေါ်စေတဲ့ အကြောင်းအရင်းအပြည့်အစုံကို သိနိုင်မှာ ဖြစ်ပါတယ်။ အထက်ပါအတိုင်း HotelAppException ကို တည်ဆောက်တဲ့နေရာမှာ SQLException Object ဖြစ်တဲ့ e ကိုထည့်ပေးလိုက်တာဟာ Root Cause ကို သတ်မှတ်ပေးလိုက်တာပဲ ဖြစ်ပါတယ်။ အဲ့ဒီအတွက် HotelAppException ကို ဖြစ်စေတာဟာ SQLException ဆိုတာကို သိစေနိုင်မှာ ဖြစ်ပါတယ်။

```
    // Data Validation
    if(null == c.getName()) {
        throw new HotelAppException(
            "Customer Name must not be null!");
    }
```

Customer ကို Insert လုပ်တဲ့နေရာမှာ Name မပါဘူးဆိုတာဟာ မရှိသင့်ပါဘူး။ အဲ့ဒီလို မပြည့်စုံတဲ့ Information တွေကို Database ထဲကို Insert မလုပ်အောင် Exception တွေကို Throw လုပ်နိုင်ပါတယ်။ Handle လုပ်တဲ့နေရာကနေ ဘာကြောင့် Exception ကို ဖြစ်စေတယ်ဆိုတာကို သိရှိနိုင်အောင် Message တွေနဲ့ Exception ကို Create လုပ်နိုင်ပါတယ်။ ဒါမှသာ User ဆီကို ဘာကြောင့် Insert လုပ်လို့မရဘူးဆိုတာကို ဖေါ်ပြပေးနိုင်မှာ ဖြစ်ပါတယ်။

Additional States in Custom Exception

Custom Exception တွေကို ရေးသားတဲ့နေရာမှာ Throwable Class မှာပါတဲ့ State တွေကိုလဲ အသုံးပြုနိုင်သလို၊ လိုအပ်ပါက မိမိကိုယ်ပိုင် State တွေကိုလဲ ရေးသား အသုံးပြုနိုင်ပါတယ်။ Exception ကို Handle လုပ်တဲ့နေရာမှာ Message တွေကို Alert Box နဲ့ပြမှာလား၊ Message Bar မှာပြမှာလား အစရှိတဲ့ Information တွေကို ထည့်သွင်းအသုံးပြုလိုတဲ့အခါမျိုးမှာ ကိုယ်ပိုင် State တွေကို ရေးသားအသုံးပြုနိုင်ပါတယ်။

```
public class HotelAppException extends RuntimeException {

    private boolean needToAlert;

    public HotelAppException(String message, boolean needToAlert) {
        super(message);
        this.needToAlert = needToAlert;
    }

    public boolean isNeedToAlert() {
        return needToAlert;
    }
}
```

အထက်ပါအတိုင်း needToAlert ဆိုတဲ့ boolean variable တစ်ခုကို State အနေနဲ့ထားပြီး၊ Exception Object ကို ဆောက်တဲ့အခါမှာ သတ်မှတ်ခိုင်းမှာ ဖြစ်ပါတယ်။ Exception ကို Handle လုပ်တဲ့အခါကျမှ ဒီ State ကိုကြည့်ပြီး Alert နဲ့ပြမလား ဆိုတာကို ဆုံးဖြတ်နိုင်မှာ ဖြစ်ပါတယ်။

Benefit of Exception

Exception ကို အသုံးပြုပြီး ရရှိနိုင်တဲ့ အကျိုးတွေကတော့ အောက်ပါအတိုင်းဖြစ်ပါတယ်။

- ✓ Exception Handling Code တွေနဲ့ Business Logic တွေကို ခွဲထုတ်ပေးနိုင်
  Try - Catch Statement တွေကို အသုံးပြုခြင်းအားဖြင့် Business Logic နဲ့ Error Handling Code တွေကို သီးခြားစီရှိအောင် ခွဲထုတ်ပေးနိုင်ပါတယ်။

- ✓ Error တွေကို ဖြစ်စေတဲ့ မူလနေရာကိုရောက်အောင် ပြန်ပို့ပေးနိုင်ခြင်း
  Exception ဖြစ်ပြီဆိုတာနဲ့ Call Stack အလိုက် ဖြေရှင်းနိုင်မည့် Handler တွေကို ရှာဖွေသွားပြီးမတွေ့ရှိခဲ့ပါက စတင်ခေါ်ဆိုခဲ့တဲ့ နေရာအထိရောက်အောင် ပြန်ပို့ပေးနိုင်ပါတယ်

- ✓ Error အမျိုးအစားအားခွဲခြားသိရှိနိုင်
  Exception တွေကို Inheritance Hirarechy အလိုက်ခွဲခြားထားနိုင်ခြင်း၊ မိမိကိုယ်ပိုင် Custom Exception တို့ကို ရေးသားထားနိုင်ခြင်းတို့ကြောင့် ဘာကြောင့်ဒီ Exception ကို ဖြစ်စေတယ်ဆိုတာကို အလွယ်တကူသိရှိစေနိုင်ပါတယ်

Method Overriding

OOP ရဲ့ Inheritance အကြောင်းကို လေ့လာခဲ့တုန်းက Super Class ကနေ Inheritance လုပ်လာတဲ့ Method တွေကို Sub Class ကနေ Override လုပ်နိုင်တယ်ဆိုတာကို လေ့လာခဲ့ကြပါတယ်။ Exception ကို Throw လုပ်ထားတဲ့ Method တစ်ခုကို Override လုပ်တော့မယ်ဆိုရင် သတိပြုစရာတစ်ခုရှိပါတယ်။ ဒီနေရာမှာလဲ Checked Exception နဲ့ Unchecked Exception တို့ရဲ့ အခြေအနေတွေဟာ မတူကြပါဘူး။

အခြေခံအားဖြင့် Unchecked Exception တွေဟာ Compiler ကနေ Check လုပ်စရာမလိုတဲ့ အတွက် Throw လုပ်ထားရင်လဲ Call Site မှာ အထွေအထူး Handle လုပ်စရာမလိုအပ်ပါဘူး။ ဒါ့ကြောင့် Unchecked Exception တွေကို Super Class မှာပဲဖြစ်ဖြစ် Override လုပ်မည့် Sub Class မှာပဲဖြစ်ဖြစ် Throw လုပ်လို့ရပါတယ်။ အဲ့ဒီအတွက် Override Rule ကို ချိုးဖောက်သွားတယ်ဆိုတာ မရှိနိုင်ပါဘူး။

ဒါပေမဲ့ Checked Exception တွေကတော့ မတူပါဘူး။ Checked Exception တွေကို Throw လုပ်ထားတဲ့ Method တွေကို အသုံးပြုမယ်ဆိုရင် Call Site မှာ Handle လုပ်ထားတယ် မလုပ်ထားဘူးဆိုတာကို Compiler ကနေ စစ်ဆေးနေပါတယ်။ ပြီးတော့ Inheritance ဖြစ်ပြီဆိုကထဲက IS - A Relationship ကို ဖြစ်ပေါ်စေပြီး Sub Type တွေကို Super Type အနေနဲ့ အသုံးပြုနိုင်ရပါမယ်။ ဒါကြောင့်မို့လို့ Override လုပ်မည့် Method တွေဟာ Super Type မှာ Throw လုပ်မထားတဲ့ Checked Exception တွေကို Throw လုပ်လို့ မရနိုင်ပါဘူး။

ဘာတွေလုပ်လို့ရပြီး ဘာတွေလုပ်လို့မရဘူးဆိုတာကိုတော့ အောက်မှာ ဖော်ပြထားပါတယ်။

Unchecked Exception

- ✓ Unchecked Exception တွေဟာ Compiler ကနေ စစ်ဆေးစရာမလိုတဲ့ Exception တွေ ဖြစ်ကြတဲ့ အတွက် Override မှာ အကျိုးသက်ရောက်မှုမရှိပါဘူး
- ✓ Super Class ရဲ့ Method ထဲမှာ Unchecked Exception ကို Throw လုပ်ထားတယ်၊ Override လုပ်မည့် Method မှာ Throw မလုပ်လဲရပါတယ်။
- ✓ Super Class ရဲ့ Method ထဲမှာ Unchecked Exception ကို Throw လုပ်ထားတယ်၊ Override လုပ်မည့် Method မှာ အခြား Unchecked Exception တစ်ခုကို Throw ချင်ရင်လဲရပါတယ်။
- ✓ Super Class ရဲ့ Method မှာ ဘာ Exception ကိုမှ Throw မလုပ်ထားတဲ့ အခါမျိုးမှာလဲ Override လုပ်မည့် Method ထဲကနေ Unchecked Exception ကို Throw လုပ်လို့လဲရမှာ ဖြစ်ပါတယ်။

Checked Exception

- ✓ Super Class ရဲ့ Method ထဲက Throw လုပ်မထားပဲ Override လုပ်မည့် Method ထဲကနေ Checked Exception တစ်ခုကို Throw လုပ်လို့ မရနိုင်ပါဘူး
- ✓ Super Class ရဲ့ Method ထဲမှာ Checked Exception တစ်ခုကို Throw လုပ်ထားပေမဲ့ Override လုပ်မည့် Method မှာ Exception ကို Throw လုပ်မထားရင်လဲ ရပါတယ်
- ✓ Super Class ရဲ့ Method ထဲက Throw လုပ်ထားတဲ့ Checked Exception နဲ့ ထပ်တူ ဒါမှမဟုတ် Sub Type မျိုးကို Override လုပ်တဲ့ Method ကနေ Throw လုပ်ပိုင်ခွင့်ရှိပါတယ်
- ✓ Super Class ရဲ့ Method ထဲက Throw လုပ်ထားတဲ့ Checked Exception ရဲ့ Super Type ဒါမှမဟုတ် မဆိုင်တဲ့ အခြား Checked Exception တစ်မျိုးမျိုးကို Override လုပ်မည့် Method ထဲကနေ Throw လုပ်လို့ မရနိုင်ပါဘူး

Using Strings



ကျွန်တော်တို့ Java Programming ကိုလေ့လာတဲ့ အခါမှာ အသုံးအများဆုံး Data Type လို့ပြောမယ်ဆိုရင် String ပဲ ဖြစ်ပါတယ်။ Java ဆိုတာဘာမှန်းမသိခင် စစပြီးရေးတဲ့ Hello World ကို ရေးမယ်ဆိုရင်တောင် String ဆိုတာကို ဘာမှန်းမသိပဲ ရေးသားခဲ့ရမှာ သေချာပါတယ်။ ဒီအခန်းမှာတော့ ကျွန်တော်တို့ အသုံးအများဆုံး Data Type ဖြစ်တဲ့ String အကြောင်းကို လေ့လာသွားကြပါမယ်။ String ဆိုတာဘာလဲ String တွေရဲ့ ထူးခြားချက်တွေက ဘယ်လိုလဲ၊ နောက်ကွယ်မှာ String တွေက ဘယ်လို အလုပ်လုပ်နေကြသလဲ ဆိုတာနဲ့ String တွေကို Compose လုပ်မယ်ဆိုရင် ဘယ်လို ရေးသားကြမလဲ ဆိုတာကို ဒီအခန်းမှာ လေ့လာသွားကြပါမယ်။

About String

String ဟာ အသုံးအများဆုံး Data Type ဖြစ်ပြီး၊ Object တစ်မျိုးဖြစ်ပါတယ်။ Java မှာက Charactor တစ်လုံးစီကို ကိုယ်စားပြုဖို့အတွက် Primitive data type ဖြစ်တဲ့ char ကို အသုံးပြုနိုင်ပါတယ်။ String ဆိုတာကတော့ charactor array ကို ကိုယ်စားပြုနိုင်တဲ့ Object ဖြစ်ပြီး၊ ထည့်သွင်းထားတဲ့ charactor တစ်လုံးစီရဲ့ အစီအစဉ်ကို သိမ်းထားပေးနိုင်ပါတယ်။

အသုံးအများဆုံး Data Type ဖြစ်တဲ့ အလျောက် String တွေမှာ အခြားသော Object တွေနဲ့ မတူတဲ့ အချက်တွေပါ ထည့်သွင်းရေးသားထားခဲ့ကြပါတယ်။ ကျွန်တော်တို့ ဒီနေရာမှာ String ဟာ ဘယ်လို သဘာဝရှိတယ်ဆိုတာကို လေ့လာသွားကြရအောင်။

String has own Literals like Primitives

Java Source file တွေထဲမှာ ဘာတွေရေးလို့ရလဲ။ Source Code တွေ၊ Comment တွေ အပြင်ကို Value တွေကို တိုက်ရိုက်ရေးသားလို့ရပါတယ်။

```
int intData = 0;
char charData = 'c';
Object objData = null;
```

အထက်ဖော်ပြပါ Assignment Statement တွေထဲမှာ 0 တို့ 'c' တို့ null တို့ဟာ ကုဒ်တွေမဟုတ်ပါဘူး။ Assignmrny တွေမှာ အစားထိုးမည့် Value တွေဖြစ်ပါတယ်။ ဤကဲ့သို့ Source Code တွေထဲမှာ Value တွေကို ရေးသားထားတာတွေကို Literals လို့ ခေါ်ကြပါတယ်။

ရှေ့မှာ Java Basic Data Type တွေအကြောင်းကိုလေ့လာခဲ့စဉ်တုန်းက Primitive Data Type တွေအကြောင်းကို လေ့လာခဲ့ကြပါတယ်။ Primitive Data Type တွေမှာ သူတို့ရဲ့ ကိုယ်ပိုင် Literals တွေကို ပိုင်ဆိုင်ကြပြီးတော့ Refernce Data Type တွေမှာတော့ Literals အနေနဲ့ null တစ်ခုပဲ အသုံးပြုလို့ရပါတယ်။ Strring တွေကတော့ ခြွင်းချက်လို့ ရေးသားဖေါ်ပြခဲ့ပါတယ်။

ဟုတ်ပါတယ် String တွေဟာ Reference Type တွေဖြစ်ပေမဲ့ Primitive Type တွေလိုပဲ ကိုယ်ပိုင် literals တွေကို ပိုင်ဆိုင်ပါတယ်။ ကျွန်တော်တို့ String Type Variable တစ်ခုကို အသုံးပြုပြီး Assignment Statement တွေကို ရေးသားတဲ့ နေရာမှာ Source Code တွေထဲမှာ Value ကို တိုက်ရိုက်ရေးသားနိုင်ပါတယ်။

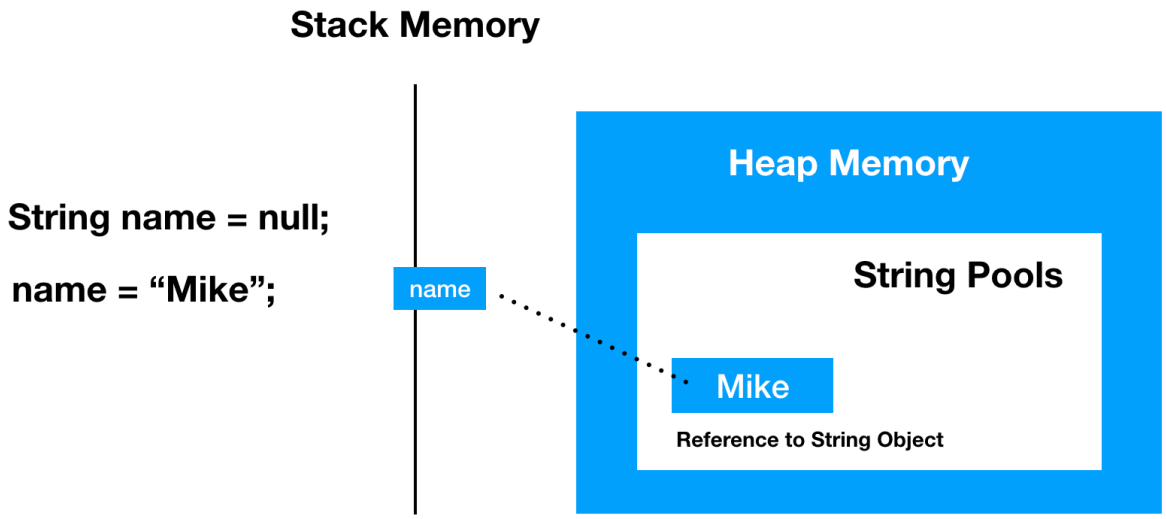
```
String name = null;
name = “Mike”;
```

String Literals တွေကို ရေးသားတဲ့နေရာမှာ Double Quotation (“) ကို အသုံးပြုပြီး ရေးသားနိုင်ပါတယ်။ အထက်ပါ Statement ထဲမှာ name ဟာ String Type Variable ဖြစ်ပြီး “Mike” ဟာ name နေရာမှာ အစားထိုးမည့် တန်ဖိုးဖြစ်ပါတယ်။

ဒီနေရာမှာ အထင်မမှားစေလိုတာက String တွေနေရာမှာ Value က တိုက်ရိုက်အစားထိုးလို့ရနေတာ မဟုတ်ပါဘူး။ Strring တွေဟာ Object တွေပဲ ဖြစ်ပါတယ်။ ဒီလို Literal နဲ့ ရေးသားထားတဲ့ String တွေဟာ ဘယ်လို အလုပ်လုပ်သလဲ ဆိုတာကို ဆက်ပြီး လေ့လာသွားကြရအောင်။

String take String Pools

String တွေဟာ အသုံးအများဆုံး Data Type ဖြစ်တဲ့ အတွက် Java ကို ရေးသားသူတွေက ရေးသားရလွယ်ကူစေရန် Literal တွေကို ပြင်ဆင်ပေးထားခဲ့ပါတယ်။ အဲ့ဒါအပြင် String တိုင်းကို Object တစ်ခုဆောက်နေရင် Memory အပေါ်မှာ နေရာအများကြီး ဖြစ်ကုန်ပါမယ်။ အဲ့ဒီအတွက် Literal နဲ့ ရေးသားထားတဲ့ String တွေကို Heap Memory အပေါ်မှာ String Pools ဆိုတဲ့ နေရာတစ်နေရာမှာ သိမ်းထားစေပါတယ်။



တကယ်လို့ ကျွန်တော်တို့ Program ထဲမှာ Literal နဲ့ ရေးသားထားတဲ့ String Value တစ်ခုကို တွေ့ပြီဆိုတာနဲ့ String Pools ထဲမှာ အဲ့ဒီတန်ဖိုးနဲ့ String Object ရှိလားဆိုတာကို သွားရှာပါတယ်။ ရှိနေရင်တော့ အဲ့ဒီ String Object ရဲ့ Referrence ကို String Variable နေရာမှာ အသုံးပြုပါတယ်။ မရှိဘူးဆိုရင်တော့ String Pools ထဲမှာ Literal Value ရှိတဲ့ String Object ကိုတည်ဆောက်ပြီး String Variable နေရာမှာ Reference ကို အစားထိုးပါတယ်။

ဒီနေရာမှာ Reference လို့ပြောတာက အဲ့ဒီ String Object တည်ရှိတဲ့ Memory Address ကို ပြောတာ ဖြစ်ပါတယ်။ Reference နဲ့ပတ်သက်ပြီး Reference Data Type တွေအကြောင်းမှာ ဖေါ်ပြခဲ့ပြီး ဖြစ်ပါတယ်။ String Pools ကို အသုံးပြုခြင်းအားဖြင့် တန်ဖိုးတူ String တွေကို JVM ရဲ့ Memory ပေါ်မှာ အများကြီး နေရာပေးထားစရာမလိုအပ်တော့ပါဘူး။

String Variable တွေကို ရေးသားအသုံးပြုတဲ့နေရာမှာ Object ကို ဘယ်လို တည်ဆောက်ထားသလဲ ဆိုတဲ့ အနေအထားအပေါ်မှာမူတည်ပြီး ကွဲပြားလေ့ရှိကြပါတယ်။

```
public static void main(String[] args) {

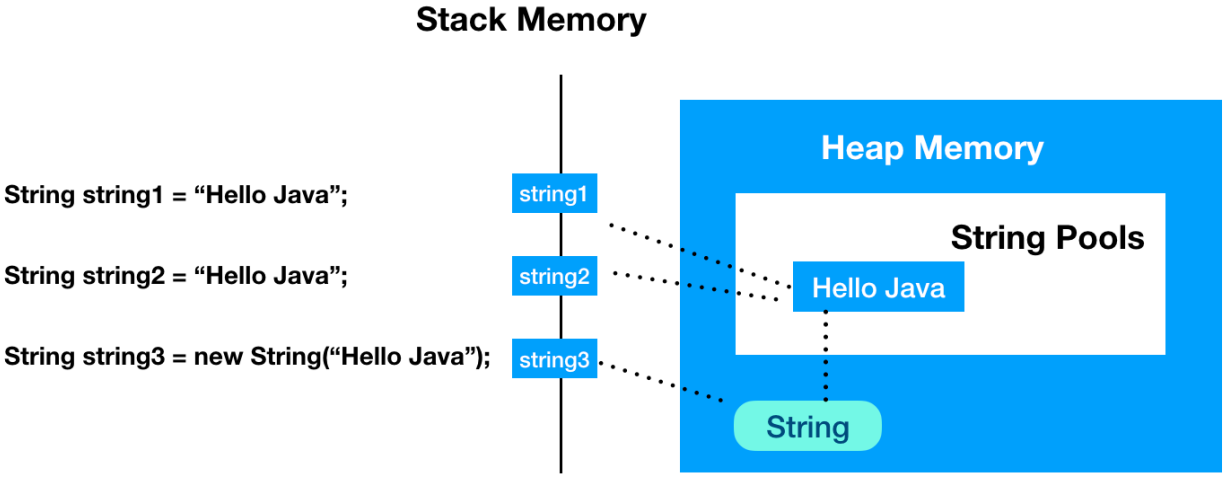
	String string1 = "Hello Java";
	String string2 = "Hello Java";
	String string3 = new String("Hello Java");

	// true
	System.out.println(string1 == string2);

	// false
	System.out.println(string1 == string3);

	// true
	System.out.println(string1.equals(string3));
}
```

အထက်ပါ ကုဒ်နမူနာထဲမှာ “Hello Java” ဆိုတဲ့ တန်ဖိုးရှိတဲ့ String Variable သုံးခုကို ရေးသားထားပါတယ်။ string1 နဲ့ string2 ကိုတော့ Literals နဲ့ ရေးသားထားပြီး string3 ကိုတော့ String Constructor ကို အသုံးပြုပြီး ရေးသားထားပါတယ်။



အထက်ဖေါ်ပြပါပုံထဲမှာလို string1 နဲ့ string2 ဟာ String Pools ထဲမှာရှိတဲ့ “Hello Java” ကို လှမ်းပြီး Reference လုပ်နေတဲ့ အတွက် string1 == string2 ရဲ့ တန်ဖိုးဟာ true ဖြစ်ပါမယ်။ ဒါပေမဲ့ string3 ဟာ String Constructor ကို အသုံးပြုပြီး တည်ဆောက်ထားတဲ့ အတွက် Heap Memory ပေါ်မှာ Object တစ်ခုကို တည်ဆောက်သွားပါတယ်။ ဒါ့ကြောင့် string1 == string3 ရဲ့ တန်ဖိုးဟာ false ဖြစ်နေမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် String တွေရဲ့ Equality ကို စမ်းစစ်ချင်ရင် equals() method ကို အသုံးပြုသင့်ပါတယ်။

Strings are Immutable

ရှေ့မှာ ဖော်ပြခဲ့သလို String တွေကို String Pools ထဲမှာ သိမ်းထားပြီး အဲ့ဒီ Object ကို အခြားသော Variable တွေကနေ တပြိုင်နက်ထဲ Reference လုပ်နေနိုင်ပါတယ်။ အဲ့ဒီအတွက် String Object တစ်ခုရဲ့ တန်ဖိုးတွေကို ပြောင်းလဲလို့ရနေရင် အဲ့ဒီ Object ကို Reference လုပ်နေတဲ့ Variabe တွေမှာ မသိလိုက်ပဲ တန်ဖိုးတွေပြောင်းကုန်တာတွေ ဖြစ်လာနိုင်ပါတယ်။

အဲ့ဒါကြောင့် String Object တွေကို တစ်ခါတည်ဆောက်ပြီးရင် တန်ဖိုးကို ပြောင်းလို့မရအောင် စီစဉ်ထားခဲ့ကြပါတယ်။ ဒီလိုမျိုး Object တစ်ခုကို တည်ဆောက်ပြီးတဲ့ အခါမှာ တန်ဖိုးပြောင်းလို့ မရနိုင်ခြင်းကို Immutable ဖြစ်တယ်လို့ခေါ်ဆိုလေ့ရှိပါတယ်။

```
public static void main(String[] args) {

	String hello = "Hello ";
	String java = "Java";

	// Code 1 - Print "Hello Java"
	System.out.println(hello.concat(java));

	// Code 2 - Print "Hello "
	System.out.println(hello);

	// Code 3 - Print "Java"
	System.out.println(java);
}
```

အထက်ပါကုဒ်နမူနာထဲမှာ hello နဲ့ java ဆိုတဲ့ String Type Variable တွေကို ရေးသားထားပါတယ်။ Code 1 ဆိုတဲ့ နေရာမှာ hello.concat(java) ဆိုတဲ့ Method Invocation ကို Print လုပ်ခိုင်းနေပါတယ်။ concat(String) method ဟာ String Object ရဲ့ Member Method ဖြစ်ပြီး String နှစ်ခုကို ပေါင်းတဲ့ နေရာမှာ အသုံးပြုနိုင်ပါတယ်။ Console ပေါ်မှာ “Hello Java” ဆိုပြီး Print လုပ်သွားတာကို တွေ့ရပါမယ်။

ပြီးတဲ့နောက်မှာ Code 2 နဲ့ Code 3 တို့မှာ hello နဲ့ java တို့ကို Print လုပ်ခိုင်းနေပါတယ်။ hello နဲ့ java ကို concat လုပ်ထားပေမဲ့ ဒီနေရာမှာတော့ သူတို့တွေဟာ နဂိုတန်ဖိုးအတိုင်းရှိနေတာကို တွေ့ရပါမယ်။ ဘာလို့လဲ ဆိုတော့ hello.concat(java) လို့ခေါ်လိုက်ပေမဲ့ String တွေဟာ Immutable ဖြစ်တဲ့ အတွက် သူတို့တွေရဲ့ တန်ဖိုးတွေဟာ ပြောင်းမသွားပဲ “Hello Java” ဆိုတဲ့ နောက်ထပ် String Object တစ်ခုကို တည်ဆောက်ပေးလိုက်တဲ့ အတွက် ဖြစ်ပါတယ်။

String တွေဟာ Immutable Object တွေဖြစ်တဲ့ အတွက် String ရဲ့ ဘယ်လို Method မျိုးကို သုံးသုံး String Variable တွေရဲ့ တန်ဖိုးဟာ ပြောင်းလဲသွားမှာ မဟုတ်ပါဘူး။

String can use Arithmetic Operator

String တွေအချင်းချင်း ဆက်ပြီး နောက်ထပ် String တစ်ခုကို ရယူလိုတဲ့ အခါမျိုးမှာ Arithmetic Operator ဖြစ်တဲ့ + ကို အသုံးပြုပြီး ရေးသားနိုင်ပါတယ်။ ဒီလို + Operator ကို အသုံးပြုရေးသားတဲ့ နေရာမှာလဲ String တွေအချင်းချင်းအကြားမှာ ရေးသားနိုင်သလို၊ String တွေနဲ့ အခြားသော Number Type တွေကြားမှာလဲ ရေးသားနိုင်ပါတယ်။ String နဲ့ အခြားသော Number တွေအကြားမှာ + ကိုအသုံးပြုပြီး Expression တွေကို ရေးသားတဲ့ နေရာမှာ အခြားသော Number တွေဟာ String လိုပဲ အလုပ်လုပ်ကြပါတယ်။ ဒါပေမဲ့ Operator Precedence အပေါ်မူတည်ပြီး ရလာတဲ့ ရလဒ်တွေကိုတော့ သတိပြုသင့်ပါတယ်။

```
	public static void main(String[] args) {

		// Code 1 -> "Hello Java"
		String str1 = "Hello " + "Java";
		System.out.println(str1);

		// Code 2 -> "101"
		String str2 = "10" + 1;
		System.out.println(str2);

		// Code 3
		String str3 = "10" + 1 + 1;
		System.out.println(str3);

		// Code 4 -> "105"
		String str4 = "10" + 1 * 5;
		System.out.println(str4);
	}
```

အထက်ပါနမူနာထဲမှာ String တွေကို + Operator အသုံးပြုပြီး ပေါင်းဆက်တာကို ရေးသားထားပါတယ်။ Code 1 ကို ကြည့်ပါ။ “Hello “ နဲ့ “Java” ကို ပေါင်းလိုက်တဲ့ အခါမှာ String နှစ်ခုပေါင်းတာ ဖြစ်တဲ့ အတွက် “Hello Java” ဆိုပြီးထွက်ပါလိမ့်မယ်။

Code 2 ကတော့ “10” ဆိုတဲ့ String နဲ့ 1 ဆိုတဲ့ int ကိုပေါင်းထားတာပါ။ ဒီနေရာမှာ 1 ဟာ int ဖြစ်ပေမဲ့ + Operator ရဲ့ တစ်ဘက်ဘက်က String ဖြစ်နေတဲ့ အတွက် 1 ကိုလဲ String အနေနဲ့ ပြောင်းပြီး အလုပ်လုပ်စေပါတယ်။ အဲ့ဒီ အတွက် ရလဒ်ဟာ “101” ဆိုတဲ့ String တစ်ခုဖြစ်လာပါတယ်။

Code 1 နဲ့ Code 2 တို့မှာလို Operator တစ်ခုနဲ့ Variable နှစ်ခုပါတဲ့ Simple Expression တွေမှာဆိုရင်တော့ ပြဿနာမရှိပါဘူး။ တစ်ဘက်ဘက်မှာ String ပါနေရင် ကျန်တဲ့ Number ကို String အနေနဲ့ ပြောင်းသွားတာကို မှတ်ရလွယ်ပါတယ်။ ဒါပေမဲ့ Expression တစ်ခုမှာ Operator တစ်ခုထက်မက အသုံးပြုလာပြီဆိုရင်တော့ Operator Precedence ကို သတိပြုဖို့လိုအပ်ပါတယ်။

Arithmetic Operator တွေကို တစ်ခုထက်မက အသုံးပြုပြီး Expression တွေကို ရေးသားတဲ့ နေရာမှာ Operator တွေရဲ့ Precedence အပေါ်မူတည်ပြီး Evaluate လုပ်တဲ့ အစီအစဉ်တွေက မတူညီကြပါဘူး။ အပေါင်း (+) နဲ့ အနှုတ် (-) ဟာ Precedence တူညီပြီး အမြှောက် (*) နဲ့ အစား (/) ဟာ တူညီကြပါတယ်။ ပါဝင်တဲ့ Operator တွေဟာ Precedent ချင်းတူနေရင် ဘယ်ဘက် Expression ကနေ စပြီး Evaluate လုပ်ကြပါတယ်။ Code 3 မှာ “10” + 1 + 1 လို့ရေးထားတဲ့ Expression မှာ ဘယ်ဘက်က Operator ကို အရင် Evaluate လုပ်တဲ့ အခါမှာ “101” + 1 ဆိုပြီး အဖြေရပါတယ်။ အဲ့ဒီအတွက် ရလဒ်ဟာ “1011” ဖြစ်သွားပါတယ်။

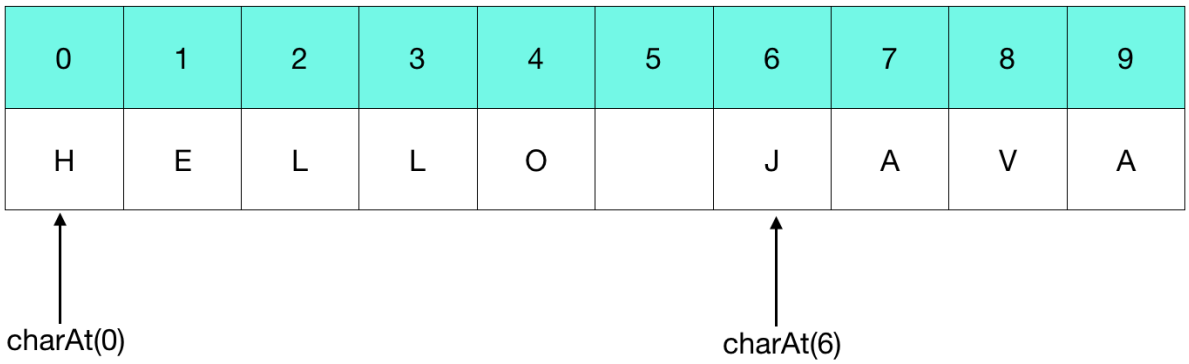
အပေါင်းအနှုတ်တွေထက် အမြှောက်အစား Operator တွေဟာ Precedence မြင့်မားကြပါတယ်။ Expression တစ်ခုမှာ အပေါင်းအနှုတ်တွေနဲ့ အမြောက်အစား Operator တွေရောနှောပြီးပါလာရင် Precedence မြင့်တဲ့ Expression ကို အရင် Evaluate လုပ်ပါတယ်။ ပြီးမှ ကျန်တဲ့ Expression ကို Evaluate လုပ်မှာ ဖြစ်ပါတယ်။ Code 4 မှာ “10” + 1 * 5 လို့ ရေးသားထားတဲ့ အတွက် 1 * 5 ကို အရင် Evaluate လုပ်သွားမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် ပထမအကြိမ် Evaluate လုပ်လိုက်တဲ့အခါ “10” + 5 လို့ ဖြစ်လာပြီး အဲ့ဒီ Expression ကို Evaluate လုပ်တဲ့ အခါမှာ ရလဒ်ဟာ “105” ဖြစ်သွားပါတယ်။

Manipulating Strings

String တွေကို အသုံးပြုမယ်ဆိုရင် အခြေခံအားဖြင့် အောက်ပါ လုပ်ဆောင်မှုတွေကို လုပ်ဆောင်နိုင်မှာ ဖြစ်ပါတယ်။

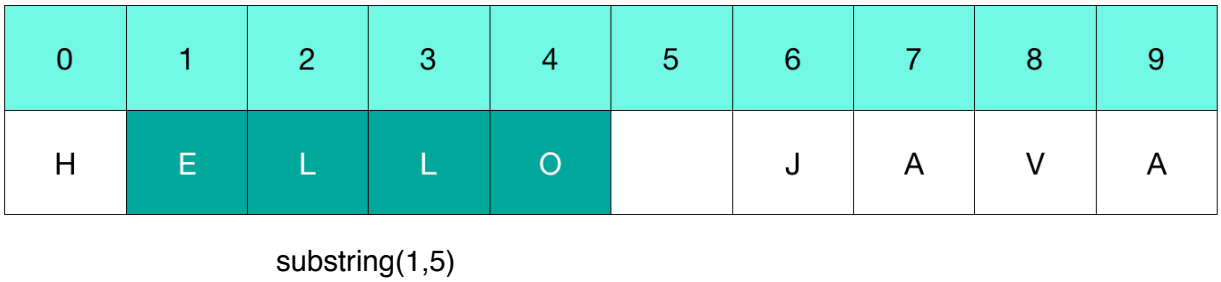
- Character တွေကို Index Number တွေနဲ့ ရှာဖွေနိုင်ပါတယ်
- String တစ်ခုကနေ အခြားသော String တွေအနေနဲ့ (Sub String) ဖြတ်တောက်နိုင်ပါတယ်
- String ထဲမှာပါဝင်တဲ့ Character တွေရဲ့ နေရာကို ရှာဖွေနိုင်ပါတယ်
- String ထဲမှာရှိတဲ့ စာလုံးတွေကို အခြားစာလုံးတွေနဲ့ အစားထိုးနိုင်ပါတယ်
- String တွေအချင်းချင်း Compare လုပ်နိုင်ပါတယ်

Character with Index



String တွေဟာ Character တွေကို Array အနေနဲ့ သိမ်းဆည်းထားပေးနိုင်တဲ့ Object တွေဖြစ်တဲ့ အတွက် သိမ်းထားတဲ့ Character တွေကို Index Number တွေနဲ့ ပြန်လည် ထုတ်ယူနိုင်ပါတယ်။ အသုံးပြုရမဲ့ method ကတော့ charAt(int) method ဖြစ်ပါတယ်။ အထက်ပါပုံထဲမှာလို charAt(0) ဆိုရင် Index Number 0 မှာရှိတဲ့ 'H' ကို ပြန်လည်ရရှိမှာ ဖြစ်ပြီး charAt(6) ဆိုရင်တော့ 'J' ကို ရရှိစေမှာ ဖြစဒ်ပါတယ်။

Sub String to Another Strings



String Object တွေကို ဖြတ်တောက်လိုတဲ့အခါမျိုးမှာ substring(int) နဲ့ substring(int, int) Method တို့ကို အသုံးပြုနိုင်ပါတယ်။ Integer Argument တစ်ခုပါတဲ့ subString() method ဟာ Argument ရဲ့ နေရာကနေ စပြီး String ကို အဆုံးအထိ ဖြတ်တောက်ပေးမှာ ဖြစ်ပါတယ်။ Argument နှစ်ခုပါတာကတော့ From Index နဲ့ To Index တွေဖြစ်ကြပြီး From ကနေ To ရဲ့ အရှေ့နေရာအထိ ဖြတ်တောက်ပေးမှာ ဖြစ်ပါတယ်။ substring(1, 5) လို့ ရေးသားထားရင် 1 နေရာကနေ 5 ရဲ့ အရှေ့အထိဖြတ်တောက်ပေးမှာ ဖြစ်တဲ့ အတွက် "ELLO" ကို ရရှိမှာ ဖြစ်ပါတယ်။

Searching Character

String Object ထဲမှာပါတဲ့ Character တွေကို ရှာဖွေတဲ့နေရာမှာ အောက်ပါ Method တွေကို အသုံးပြုနိုင်ပါတယ်။

Return Type	Method
int	indexOf(int ch)
int	lastIndexOf(int ch)
int	indexOf(int ch, int fromIndex)
int	lastIndexOf(int ch, int fromIndex)
int	indexOf(String str)
int	lastIndexOf(String str)
int	indexOf(String str, int fromIndex)
int	lastIndexOf(String str, int fromIndex)
boolean	contains(CharSequence seq)

String ဟာ Character တွေကို Array အနေနဲ့ သိမ်းပေးထားနိုင်တဲ့ Object ဖြစ်တဲ့အတွက် ရှာဖွေလိုတဲ့ character တစ်လုံးရဲ့ Array ထဲမှာရှိတဲ့ Index Number ကို ရှာဖွေနိုင်ပါတယ်။ ရှာလိုတဲ့ Character ဟာ String ထဲမှာ တစ်လုံးထက်မက ပါနေရင်တော့ ရှေ့ဆုံးကနေ စရှာမလား၊ နောက်ဆုံးကနေရှာမလား၊ စဉ်းစားဖို့လိုပါတယ်။ တဖန် From Index ကို အသုံးပြုပြီး ရှာချင်တဲ့ နေရာကနေ စရှာနိုင်အောင် Method တွေကိုလဲ ပြင်ပေးထားပါတယ်။ ရှာဖွဲ့တဲ့နေရာမှာ String ကိုလဲရှာဖွေနိုင်မှာ ဖြစ်ပါတယ်။

```
public class StringDemo {

	public static void main(String[] args) {
		String extension = getExtension("StringDemo.java");
		System.out.println(extension);
	}

	public static String getExtension(String fileName) {

		if(fileName.contains(".")) {
			int indexOfLastDot = fileName.lastIndexOf('.');
			return fileName.substring(indexOfLastDot + 1);
		}

		return null;
	}
}
```

အထက်ပါနမူနာထဲက getExtension(String) Method ဟာ File Name တွေကနေ Extension Type ကို ရှာပေးနိုင်တဲ့ Method ဖြစ်ပါတယ်။ contains(“.”) ကို အသုံးပြုပြီး Dot ရှိလားဆိုတာကို စစ်ပါတယ်။ တွေ့ခဲ့ရင် lastIndexOf(‘.’) ကို အသုံးပြုပြီး Dot ရဲ့ နောက်ဆုံးနေရာကို ရှာပြီး substring(indexOfLastDot + 1) ကို အသုံးပြုပြီး Dot နောက်က Extension Type ကို ဖြတ်ယူပြီး Return ပြန်ပေးနေပါတယ်။

Replacing Characters

String Object ထဲမှာပါတဲ့ Character တွေ၊ ဒါမှမဟုတ်ရင် String အစိတ်အပိုင်းတွေကို အခြား တန်ဖိုးတွေနဲ့ အစားထိုးပြီး နောက်ထပ် String Object တစ်ခုကို တည်ဆောက်ပေးနိုင်တဲ့ Method တွေကတော့ အောက်ပါ အတိုင်း ဖြစ်ကြပါတယ်။

Return Type	method
String	replace(char oldChar, char newChar)
String	replace(CharSequence target, CharSequence replacement)
String	replaceAll(String regex, String replacement)
String	replaceFirst(String regex, String replacement)

Character နဲ့လဲ Replace လုပ်နိုင်သလို၊ CharSequence အလိုက်လဲ Replace လုပ်နိုင်ပါတယ်။ Regular Expression ကို အသုံးပြုပြီး Pattern အလိုက် Match ဖြစ်တဲ့ String တွေကိုလဲ Replace လုပ်နိုင်မှာ ဖြစ်ပါ တယ်။

```
    public static void main(String[] args) {

        String original = "Java Programming";

        // Code 1 -> Jzvz Progrzmming
        String replace1 = original.replace('a', 'z');
        System.out.println(replace1);

        // Code 2 -> Jzvz Progrzmming
        String replace2 = original.replace("a", "z");
        System.out.println(replace2);

        // Code 3 -> Jzvz Progrzmming
        String replace3 = original.replaceAll("a", "z");
        System.out.println(replace3);

        // Code 4 -> Jzva Programming
        String replace4 = original.replaceFirst("a", "z");
        System.out.println(replace4);
    }
```

Code 1 ကတော့ Character ကို အသုံးပြုပြီး Replace လုပ်တာဖြစ်ပြီး၊ Code 2 ကတော့ Character Sequence ကို အသုံးပြုပြီး Replace လုပ်တာ ဖြစ်ပါတယ်။ Code 1 ကော Code 2 ပါ Replace လုပ်ချင်တဲ့ စာလုံးနဲ့ တူသမျှ စာလုံးတွေအားလုံးကို Replace လုပ်နိုင်တာကို တွေ့ရပါတယ်။

Code 3 နဲ့ Code 4 ကတော့ Regular Expression ကို အသုံးပြုပြီး Pattern Match ဖြစ်တဲ့ String တွေကို Replace လုပ်တာဖြစ်ပါတယ်။ Code 3 မှာ replaceAll() Method ကို အသုံးပြုထားတဲ့ အတွက် Match ဖြစ်သမျှ String တွေအားလုံးကို Replace လုပ်သွားပြီး၊ Code 4 မှာတော့ replaceFirst() Function ကို အသုံးပြုထားတဲ့ အတွက် ပထမဆုံး Match ဖြစ်တဲ့ String ကိုသာ Replace လုပ်သွားတယ်ဆိုတာကို တွေ့နိုင်ပါတယ်။

Comparing Strings

Compare လုပ်တယ်ဆိုတာဟာ ဘယ်သူကြီးသလဲ ဘယ်သူငယ်သလဲဆိုတာကို နှိုင်းယှဉ်တာဖြစ်ပါတယ်။ Compare လုပ်နိုင်မှသာ ငယ်စဉ်ကြီးလိုက်၊ ကြီးစဉ်ငယ်လိုက် အစရှိတဲ့ Sorting လုပ်တာတွေကို ဆောင်ရွက်နိုင်မှာ ဖြစ်ပါတယ်။ Java မှာက Collection တွေနဲ့ Array တွေထဲက Element တွေကို Sort လုပ်နိုင်ဖို့ Collections Framework ထဲမှာ Utility Class တွေကို ပြင်ပေးထားပါတယ်။ Utility Class တွေကို အသုံးပြုပြီး Sort လုပ်နိုင်ဖို့ အတွက် Comparable Interface ကို Implement လုပ်ထားဖို့လိုအပ်ပါတယ်။

String ဟာလဲ Comparable Interface ကို Implement လုပ်ထားတဲ့ အတွက် compareTo() Method ကို ရေးသားထားပြီး ဖြစ်ပါတယ်။ ဒါ့ကြောင့် String တွေကိုလဲ Collection Framework ရဲ့ Utility Class တွေကို အသုံးပြုပြီး Natural Ordering အလိုက် Sort လုပ်လို့ရပါတယ်။ Sort လုပ်တာနဲ့ ပတ်သက်ပြီး Collection Framework အကြောင်းကို ရေးသားတဲ့အခါမှာ အသေးစိတ် ဖေါ်ပြသွားပါမယ်။

String မှာက String ဖြစ်သည်အလျောက် Compare လုပ်တာနဲ့ ပတ်သက်ပြီး အသုံးများနိုင်တဲ့ Method တွေကိုလဲ ထပ်မံပြင်ဆင်ထားပါသေးတယ်။

Return Type	Method
boolean	endsWith(String suffix)
boolean	startsWith(String prefix)
boolean	startsWith(String suffix, int offset)
int	compareTo(String anotherString)
int	compareToIgnoreCase(String anotherString)
boolean	equals(Object object)
boolean	equalsIgnoreCase(String anotherString)
boolean	reginMatchs(int toOffset, String other, int offset,int len)
boolean	reginMatchs(boolean ignorecase, int toOffset, String other, int offset, int len)

String တစ်ခုဟာ သတ်မှတ်ထားတဲ့ String တစ်ခုနဲ့ စသလား ဆုံးသလားဆိုတာကို သိရှိနိုင်ဖို့ startsWith() နဲ့ endsWidth() Method တွေကို အသုံးပြုနိုင်ပါတယ်။

```
      public static void main(String[] args) {
            String original = "Java Programming”;

            // Code 1 -> true
            boolean result1 = original.startsWith("Java");
            System.out.println(result1);

            // Code 2 -> true
            boolean result2 = original.startsWith("Pro", 5);
            System.out.println(result2);

            // Code 3 -> true
            boolean result3 = original.endsWith("ming");
            System.out.println(result3);
      }
```

String နှစ်ခုကို Compare လုပ်တဲ့နေရာမှာ Case ကိုပါ Ignore လုပ်ပြီး Compare လုပ်နိုင်အောင် compareToIgnoreCase() Method ကိုလဲ ပြင်ဆင်ပေးထားပါသေးတယ်။

```
	public static void main(String[] args) {

		// Code 1 -> -32
		int result1 = "Java".compareTo("java");
		System.out.println(result1);

		// Code 2 -> 0
		int result2 = "Java".compareToIgnoreCase("java");
		System.out.println(result2);
	}
```

Compare လုပ်တဲ့ Method တွေရဲ့ Return Type ဟာ Integer တွေဖြစ်ကြပါတယ်။ သုညဆိုရင် String နှစ်ခုဟာ တူတယ်လို့ အဓိပ္ပါယ်ရပြီး၊ အပေါင်းကိန်းဆိုရင်တော့ ဘယ်ဘက်က String ဟာ ကြီးတယ်လို့ အဓိပ္ပါယ်ရပါတယ်။ Return တန်ဖိုး အနုတ်ကိန်းဆိုပါက ဘယ်ဘက်က String က ငယ်တယ်လို့ အဓိပ္ပါယ်ရပါတယ်။

အထက်ပါ ကုဒ်ထဲက Code 1 မှာ “Java”.compareTo(“java”) လို့ Compare လုပ်နေပါတယ်။ “J” ဟာ “j” ထက် ငယ်တဲ့ အတွက် ရလဒ်ဟာ -32 အနုတ်ကိန်းဖြစ်ပါတယ်။ တဖန် Code 2 မှာတော့ ရှေ့မှာပါတဲ့ String နှစ်ခုကိုပဲ compareToIgnoreCase() Method ကို အသုံးပြုပြီး Compare လုပ်ပါတယ်။ Case တွေကို ထည့်မတွက်ပဲ Compare လုပ်တဲ့ အတွက် “Java” နဲ့ “java” ဟာ တူပါတယ်လို့ အဖြေရပါတယ်။

တဖန် String နှစ်ခုကို တူမတူ စစ်ဆေးနိုင်ဖို့အတွက် equals() နဲ့ equalsIgnoreCase() Method တို့ကို ပြင်ပေးထားပါတယ်။ အတိအကျတူမတူကို သိချင်ရင်တော့ equals() method ကို အသုံးပြုသင့်ပြီး၊ စာလုံး အကြီးအသေးကို ထည့်မတွက်ပဲ တူမတူစစ်ချင်ရင်တော့ equalsIgnoreCase() method ကို အသုံးပြုသင့်ပါတယ်။

```
	public static void main(String[] args) {

		// Code 1 -> false
		boolean result1 = "Java".equals("java");
		System.out.println(result1);

		// Code 2 -> true
		boolean result2 = "Java".equalsIgnoreCase("java");
		System.out.println(result2);

		// Code 3 -> true
		boolean result3 = "JAVA".equalsIgnoreCase("java");
		System.out.println(result3);
	}
```

အထက်ပါ ကုဒ်နမူနာထဲက Code 1 မှာ “Java”.equals(“java”) လို့ရေးထားတဲ့ နေရာမှာ J စာလုံးနဲ့ j စာလုံးဟာ မတူတဲ့ အတွက် ရလဒ်ဟာ false ဖြစ်နေပါတယ်။ Case 2 နဲ့ Case 3 ကတော့ equalsIgnoreCase() method ကို အသုံးပြုထားတဲ့ အတွက် စာလုံးအကြီးအသေးကွဲနေလဲ ပါဝင်တဲ့ Letter တွေတူနေတဲ့ အတွက် ရလဒ်ဟာ true တွေပဲ ဖြစ်နေတာကို တွေ့ရပါတယ်။

String မှာက String နှစ်ခုရဲ့ အစိတ်အပိုင်းတွေ တူမတူကို စစ်ဆေးနိုင်ဖို့ regionMatch() Method တွေကို ပြင်ပေးထားပါသေးတယ်။

Argument	Type	Definition
toOffset	int	String Object ရဲ့ Match လုပ်ချင်တဲ့ Region ရဲ့ အစ Index
other	String	Match လုပ်လိုတဲ့ အခြား String Object
offset	int	အခြား String Object ရဲ့ Match လုပ်ချင်တဲ့ Region ရဲ့ အစ Index
len	int	Region Match လုပ်လိုတဲ့ စာလုံးအရေအတွက်
ignorecase	boolean	Case ကို Ignore လုပ်မလုပ်ဆုံးဖြတ်ရန်တန်ဖိုး

Region Match ကို အသုံးပြုဖို့အတွက် regionMatch() Method နှစ်ခုကို Overload လုပ်ပြီး ရေးသားထားပါတယ်။ Case ကို Ignore လုပ်ပြီး Match လုပ်မယ်ဆိုရင်တော့ ignorecase Argument ပါတဲ့ method ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

```
    public static void main(String[] args) {

        String self = "Hello JAVA Programming";
        String other = "Java Programmer";

        // Code 1 -> true
        boolean result1 = self.regionMatches(11, other, 5, 5);
        System.out.println(result1);

        // Code 2 -> false
        boolean result2 = self.regionMatches(6, other, 0, 4);
        System.out.println(result2);

        // Code 3 -> true
        boolean result3 = self.regionMatches(true, 6, other, 0, 4);
        System.out.println(result3);
    }
```

Code 1 နဲ့ Code 2 တို့ကတော့ ignorecase Argument မပါတဲ့ regionMatches() method ကို အသုံးပြုထားကြတာပါ။ Code 1 မှာ self ရဲ့ 11 လုံးမြောက် စာလုံးဟာ P ဖြစ်ပြီး other ရဲ့ ၅ လုံးမြောက် စာလုံးဟာလဲ P ဖြစ်ပါတယ်။ အဲ့ဒီအစစာလုံးတွေကနေ ၅ လုံးအထိတိုက်စစ်လိုက်တဲ့အခါမှာ နှစ်ခုစလုံး Progr ဖြစ်နေတဲ့ အတွက် ရလဒ်ဟာ true ဖြစ်ပါတယ်။

Case 2 မှာကျတော့ self ရဲ့ ၆ လုံးမြောက် စာလုံးဟာ J ဖြစ်ပြီး ၄ လုံးအထိယူရင် JAVA ဖြစ်ပါမယ်။ otherr ရဲ့ 0 ကနေ စာလုံး ၄ လုံးယူရင် Java ဖြစ်နေပါတယ်။ Case မတူတဲ့ အတွက် regionMatch() Method ရဲ့ ရလဒ်ဟာ false ဖြစ်ပါမယ်။ Case 3 ကတော့ Case 2 နဲ့ ကျန်တဲ့ တန်ဖိုးတွေကတော့ အတူတူပဲ။ caseignore Argument ကို true လို့ပေးထားတဲ့ အတွက် JAVA နဲ့ Java ကို Case ကို Ignore လုပ်ပြီး Match လုပ်ပါမယ်။ ဒါ့ကြောင့် ရလဒ်ဟာ ture ဖြစ်သွားတာ ဖြစ်ပါတယ်။

Other Useful Methods

String တွေကို အသုံးပြုတဲ့ နေရာမှာ အခြားသော အသုံးများသော Method တွေလဲရှိကြပါသေးတယ်။

Return Type	Method
String []	split(String regex)
String []	split(String regex, int limit)
CharSequence	subSequence(int beginIndex, int endIndex)
String	trim()
String	toLowerCase()
String	toUpperCase()

String Object တစ်ခုကနေ သတ်မှတ်ထားတဲ့ နောက် String တစ်ခုနဲ့ Split လုပ်လိုတဲ့ အခါမှာ split() Method ကို အသုံးပြုနိုင်ပါတယ်။ System တစ်ခုကနေ အခြား System တစ်ခုကို Data တွေပေးပို့တဲ့ အခါမျိုးတွေမှာ Comma Sperated File တွေ Tabb Sperrated File တွေကို အသုံးပြုလေ့ရှိပါတယ်။ ထို File တွေထဲက Data တွေကို ပြန်ပြီး ထုတ်ယူတဲ့ အခါမျိုးမှာ split() Method ကို အသုံးပြုနိုင်ပါတယ်။ String ရဲ့ split() method မှာ Overload လုပ်ထားတဲ့ Method နှစ်ခုရှိပါတယ်။ ပထမတစ်ခုကတော့ String တစ်ခုကို Argument အနေနဲ့ ယူထားတဲ့ Method ဖြစ်ပြီး အဲ့ဒီ Argument နဲ့ Split လုပ်မှာ ဖြစ်ပါတယ်။ အောက်ပါနမူနာထဲကလို csvData ကို “,” နဲ့ Split လုပ်မယ်ဆိုရင် [“Yangon”, “Mandalay”, “Taungyi”,”Pathein”] Array ကို ပြန်လည်ရရှိမှာ ဖြစ်ပါတယ်။

```
	public static void main(String[] args) {

		String csvData = "Yangon,Mandalay,Taungyi,Pathien";

		String [] array = csvData.split(",", 2);

		for(String data : array) {
			System.out.println(data);
		}
	}
```

ဒုတိယ Method ကတော့ String တစ်ခုနဲ့ Integer တစ်ခုကို Argument အနေနဲ့ ယူထားတဲ့ Method ပါ။ String Argument ကတော့ Split လုပ်မည့် String ဖြစ်ပြီး ဒုတိယ Integer Argument ကတော့ မူလ String Object ကို ဘယ်နှစ်ပိုင်းပိုင်းမှာလဲ ဆိုတာကို သတ်မှတ်ပေးနိုင်တဲ့ Parameter ဖြစ်ပါတယ်။ အထက်ပါ နမူနာထဲမှာလို csvData.split(“,”, 2) လို့ရေးထားရင် [“Yangon”, “Mandalay,Taungyi,Pathein”] ဆိုတဲ့ String နှစ်ခုပါတဲ့ Array Object ကို ရရှိမှာ ဖြစ်ပါတယ်။

String Object ရဲ့ ရှေ့နောက်မှာရှိတဲ့ Space တွေကို trim() method ကို အသုံးပြုနိုင်ပြီး၊ String ထဲမှာရှိတဲ့ စာလုံးတွေအားလုံးကို Lower Case ကို ပြောင်းချင်ရင် toLowerCase() ၊ Upper Case ကို ပြောင်းချင်ရင် toUpperCase() Method တို့ကို အသုံးပြုနိုင်ပါတယ်။

တစ်ခုမှတ်ထားဖို့လိုတာက String ဟာ Immutabble Object ဖြစ်တဲ့ အတွက် ဘယ်လို Method ကိုပဲ သုံးသုံး နောက်ထပ် String Object အသစ်တစ်ခုကို Create လုပ်ပေးမှာ ဖြစ်ပြီး မူလ Object ရဲ့ တန်ဖိုးဟာ ပြောင်းသွားမှာ မဟုတ်ပါဘူး။

Composing String

String တွေကို ပေါင်းချင်ရင် + Operator ဒါမှမဟုတ် concat() Method ကို အသုံးပြုနိုင်တယ်လို့ ရှေ့မှာ လေ့လာခဲ့ကြပါတယ်။ ဒါပေမဲ့ မိမိရေးသားနေတဲ့ Application ထဲမှာ အကြိမ်ကြိမ် String တွေကို Compose လုပ်နေရမယ်ဆိုရင်တော့ Memory Usage နဲ့ ပတ်သက်ပြီး သတိထားဖို့လိုအပ်ပါတယ်။

အောက်က ကုဒ်နမူနာလေးနဲ့ ဘယ်လို သတိထားသင့်သလဲဆိုတာကို လေ့လာသွားကြရအောင်။

```
static String add(String s1, String s2, String s3, String s4) {
      return s1 + s2 + s3 + s4;
}

public static void main(String[] args) {

      // Code 1 -> “Hello Java World Champion"
      String result1 = add("Hello ", "Java ", "World ", "Champion");

      // Code 2 -> “ABCD”
      String result2 = add("A", "B", "C", "D");
}
```

နမူနာထဲမှာ String 4 ခုကိုယူပြီး ပေါင်းပေးနေတဲ့ add() Method ရှိပြီး၊ Main Method ထဲကနေ add() Method ကို နှစ်ကြိမ်ခေါ်နေတာကို တွေ့ရပါမယ်။ ဘာမှ ပြဿနာမရှိသလိုမြင်ရပါမယ်။ ဒါပေမဲ့ + တွေကို အသုံးပြုပြီး String တွေကို ပေါင်းတဲ့ အခါမှာ ဘယ်လို အလုပ်လုပ်တယ်ဆိုတာနဲ့ နောက်ကွယ်မှာ String Pools ရှိနေတယ်ဆိုတဲ့ အချက်ကို ခေါင်းထဲမှာ ထည့်ပြီး လုပ်ဆောင်ပုံတွေကို တဆင့်ချင်းလေ့လာကြည့်ကြရအောင်။

Main Method ထဲကနေ Code 1 Statement ကို Evaluate လုပ်တဲ့ အခါမှာ “Hello “, “Java “, “World” နဲ့ “Champion” ဆိုတဲ့ String Object လေးခုကို String Pools ထဲမှာ သွားဆောက်ပါလိမ့်မယ်။ ပြီးတော့ add() Method ကို Invoke လုပ်တဲ့အခါမှာ s1 + s2 + s3 + s4 လို့ရေးထားပါတယ်။ ကျွန်တော်တို့ စိတ်ထဲမှာတော့ အားလုံးကို ပေါင်းပြီး “Hello Java World Champion” ဆိုပြီး အဖြေရမယ်လို့ တွက်ပါမယ်။

ဒါပေမဲ့ Computer က အလုပ်လုပ်သွားမှာက အရင်ဆုံး s1 နဲ့ s2 ကို ပေါင်းပါတယ်။ အဲ့ဒီတော့ “Hello Java “ ဆိုတဲ့ String Object တစ်ခုကို String Pools ထဲ ထပ်တိုးပါမယ်။ ပြီးမှ ရလာတဲ့ ရလဒ်နဲ့ s3 ကို ဆက်ပေါင်းတော့ “Hello Java World “ String Object အသစ်တစ်ခုထပ်ရလာပြီး String Pools ထဲကို ထပ်ပြီး ထည့်ပါတယ်။ ပြီးမှ s4 နဲ့ပေါင်းပြီး “Hello Java World Champion” ဆိုတဲ့ String တစ်ခုကို String Pools ထဲကို ထပ်ထည့်ပါမယ်။

+ Operator ကို အသုံးပြုပြီး String တွေကို Compose လုပ်မယ်ဆိုရင် ဘယ်ကမှ Reference လုပ်မနေတဲ့ String Object တွေကို String Pools ထဲကို ရောက်ရှိသွားစေမှာ ဖြစ်ပါတယ်။ ပြီးတော့ String Pools ထဲက String တွေဟာ Application မပြီးဆုံးမချင်း ဒီအတိုင်းပဲတည်ရှိနေမှာ ဖြစ်တဲ့ အတွက် မလိုလားအပ်ပဲ Memory ပေါ်မှာ နေရာတွေကို ယူသွားစေနိုင်ပါတယ်။ ဒါ့ကြောင့် မိမိရဲ့ Appliation ထဲမှာ အကြိမ်ကြိမ် String တွေကို ပေါင်းနေရတဲ့ Operation တွေပါနေရင် + တို့ concat() တို့ကို အသုံးမပြုသင့်ပါဘူး။

String တွေကို Compose လုပ်တဲ့နေရာမှာ အသုံးပြုဖို့အတွက် Java မှာ Class နှစ်မျိုးကို ပြင်ပေးထားပါတယ်။ အဲ့ဒါတွေကတော့ StringBuffer နဲ့ StringBuilder တို့ပဲ ဖြစ်ပါတယ်။ သူတို့တွေဟာ Constructor တွေလဲအတူတူ ဖြစ်သလို ပါတဲ့ Method တွေလဲ အတူတူပဲ ဖြစ်ပါတယ်။ အဲ့ဒီအတွက် လုပ်လို့ရတာတွေလဲ အတူတူပဲ ဖြစ်ပါတယ်။ မတူတာကတော့ StrringBuffer ဟာ Thread Safe ဖြစ်ပြီး StringBuilder ဟာ Thread Safe မဖြစ်ဘူးဆိုတာပါပဲ။ Multi Thread ပတ်ဝန်းကျင်မှာဆိုရင် StringBuffer ကို အသုံးပြုသင့်ပြီး၊ Multi Thread ပတ်ဝန်းကျင်မဟုတ်ဘူးဆိုရင်တော့ StringBuilder ကို အသုံးပြုသင့်ပါတယ်။

Return Type	Methods
void	setLength(int length)
void	ensureCapacity(int capacity)
StringBuilder	append(boolean b) append(char c) append(char[] str) append(char[] str, int offset, int len) append(double d) append(float f) append(int i) append(long lng) append(Object obj) append(String s)
StringBuilder	delete(int start, int end)
StringBuilder	deleteCharAt(int index)
StringBuilder	insert(int offset, boolean b) insert(int offset, char c) insert(int offset, char[] str) insert(int index, char[] str, int offset, int len) insert(int offset, double d) insert(int offset, float f) insert(int offset, int i) insert(int offset, long lng) insert(int offset, Object obj) insert(int offset, String s)
StringBuilder	replace(int start, int end, String s)
StringBuilder	reverse()
String	toString()

Append Method

အသုံးအများဆုံး Method ဖြစ်ပြီး Argument ထဲက တန်ဖိုးတွေကို StringBuilder ထဲက နောက်ဆုံးနေရာကနေ စပြီး ထည့်သွင်းပေးနိုင်ပါတယ်။ Primitive Data Type တွေနဲ့ Char Array တွေအပြင် String တွေကိုလဲ Append လုပ်နိုင်အောင် Method Overload လုပ်ပြီးရေးသားထားပါတယ်။ Return Type ဟာ StringBuilder မိမိကိုယ်တိုင်ပဲ ဖြစ်တဲ့အတွက် Dot (.) ကို အသုံးပြုပြီး Method Invocation Chain ပုံစံနဲ့ ရေးသားနိုင်ပါတယ်။

```
            StringBuilder b1 = new StringBuilder();
            b1.append("Hello ");
            b1.append("Java");

            StringBuilder b2 = new StringBuilder();
            b2.append("Hello ").append("Java");
```

Insert Method

String တွေကို Build လုပ်နေရင်း၊ Index နေရာကို သတ်မှတ်ပြီး Insert လုပ်လိုတဲ့ အခါမှာ အသုံးပြုနိုင်ပါတယ်။ Append Method တွေလိုပဲ Insert Method တွေမှာ Method Overload ကိုအသုံးပြုပြီး ရေးသားထားတဲ့ အတွက် Primitive Data Type တွေ၊ Char Array တွေနဲ့ String တွေကို Insert လုပ်နိုင်ပါတယ်။

```
StringBuilder b1 = new StringBuilder();
b1.append("Hello").append("Java");

b1.insert(5, " ").insert(6, "Happy ");

// "Hello Happy Java"
System.out.println(b1.toString());
```

အထက်ပါနမူနာမှာ b1.insert(5, " ") လို့ရေးသားထားတဲ့ အတွက် StringBuilder Object ရဲ့ Index Number 5 နေရာမှာ " " Space ကို Insert လုပ်ပါတယ်။ ပြီးတဲ့နောက်မှာ insert(6, "Happy ") လို့ခေါ်ပြီး Insert လုပ်ပြန်တယ်။ ဒီအချိန်မှာ StringBuilder ထဲမှာ "Hello Happy Java" ဆိုတဲ့ String ကို Compose လုပ်ပြီး ဖြစ်နေပါပြီ။

Index Number ကို သတ်မှတ်ပြီး Insert လုပ်တဲ့နေရာမှာ သတိပြုစရာအချက်ရှိပါတယ်။ Index Number မရှိတဲ့ နေရာကို သတ်မှတ်ပြီး Insert လုပ်မိရင်တော့ StringIndexOutOfBoundsException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

Delete Methods

StringBuilder တွေမှာ Index Number ကို သတ်မှတ်ပြီး အဲ့ဒီ Index မှာရှိတဲ့ Character တွေ Index From နဲ့ To ကို သတ်မှတ်ပြီး အဲ့ဒီကြားထဲက Character တွေကို Delete လုပ်နိုင်အောင် Method တွေကို ပြင်ဆင်ပေးထားပါတယ်။ Index Number ကို အသုံးပြုနေတာဖြစ်တဲ့ အတွက် မရှိတဲ့ Index Number ကို သတ်မှတ်မိမယ်ဆိုရင် StringIndexOutOfBoundsException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

Replace Methods

```
StringBuilder b1 = new StringBuilder();
b1.append("Java ").append(9).append(" is interesting.");

b1.replace(5, 6 , "11");

// "Java 11 is interesting"
System.out.println(b1.toString());
```

StringBuilder ထဲက From နဲ့ To Index ကို သတ်မှတ်ပြီး အခြားသော String တန်ဖိုးတစ်ခုနဲ့အစားထိုးလိုတဲ့အခခါမှာ replace() Method ကို အသုံးပြုနိုင်ပါတယ်။ ထို့အပြင် StringBuilder Object နဲ့ Compose လုပ်ထားတဲ့ String ကိုတော့ toString() Method ကို အသုံးပြုပြီး ရယူနိုင်ပါတယ်။ Composed လုပ်ထားတဲ့ String တွေကို ပြောင်းပြန်ပြန်လိုတဲ့ အခါမျိုးမှာ reverse() Method ကို အသုံးပြုနိုင်ပါတယ်။

User Input အပေါ်မှာမူတည်ပြီး Query တွေကိုတည်ဆောက်တဲ့ Method မျိုးတွေမှာ StringBuilder တွေကို အသုံးပြုသင့်ပါတယ်။

Formatting Strings

Application တွေကို ရေးသားတဲ့ နေရာမှာ String တွေကို Input အနေနဲ့ လက်ခံပြီး Data အနေနဲ့ သိမ်းထားရုံသာမက၊ သိမ်းထားတဲ့ အမျိုးမျိုးသော Data တွေနဲ့ User ကိုနားလည်လွယ်အောင် ပြန်ပြရတဲ့ အခါလဲရှိပါတယ်။ Hello World Program လေးကို စရေးစဉ်ကထဲက Console ကနေ User Input ကိုလက်ခံပြီး Hello ဆိုပြီး Console မှာ ပြန်ပြခဲ့ကြဘူးပါလိမ့်မယ်။

```
      public static void main(String[] args) {

            // Create Console
            Console console = System.console();

            // Read Data from Console
            String name = console.readLine("Please enter your name : ");

            // Write Data to Console
            console.printf("Hello %s%n", name);
      }
```

Hello World Program လေးကိုရေးတယ်ဆိုရင်တောင်မှ မြင်သာအောင် String တွေကို ပြင်ဆင်ပြီး ပြသဖို့လိုအပ်ပါတယ်။ Java မှာ String တွေကို Format ချနိုင်ဖို့အတွက် နည်းလမ်းအမျိုးမျိုးပြင်ဆင်ပေးထားပါတယ်။

အရင်ဆုံးဘယ်လိုနေရာမျိုးတွေမှာ String တွေကို Format ချပြီး ပြသဖို့လိုသလဲ ဆိုတာကို စဉ်းစားကြည့်ကြရအောင်။ ရှေ့မှာဖေါ်ပြခဲ့သလို Console မှာ Print လုပ်တဲ့အခါမျိုးတွေ၊ Print Writer တွေနဲ့ Output တွေမှာ String တွေကို Print လုပ်တဲ့ အခါတွေ၊ User Interface တနေနေရာမှာ User ကို နားလည်အောင် ပြပေးတဲ့အခါတွေ၊ Log File တွေ Error Message တွေမှာ Output လုပ်တဲ့ အခါမျိုးတွေမှာ String တွေကို Format လုပ်ဖို့လိုအပ်ပါလိမ့်မယ်။

Java ပတ်ဝန်းကျင်မှာ String တွေကို Format ချဖို့အတွက် အောက်ပါနည်းလမ်းတွေနဲ့ ပြင်ဆင်ပေးထားပါတယ်။

- PrintWriter, PrintOutputStream နဲ့ Console တွေမှာ format() နဲ့ printf() Method တွေကို ပြင်ပေးထားပါတယ်။
- String Class မှာ format() method ကို ပြင်ဆင်ပေးထားပါတယ်။
- TextFormat Class မှာလဲ Message တွေကို Format ချနိုင်ဖို့ ပြင်ဆင်ပေးထားပါတယ်။
- DateFormat နဲ့ NumberFormat Class တွေနဲ့လဲ Date တွေနဲ့ Number တွေကို Format ချနိုင်အောင် ပြင်ဆင်ပေးထားပါတယ်။

Date Format နဲ့ Number Format တွေနဲ့ ပတ်သက်ပြီး သက်ဆိုင်ရာ အခန်းတွေမှာ ဖေါ်ပြသွားမှာ ဖြစ်တဲ့အတွက် ဒီအခန်းမှာတော့ String တွေကို Format ချတာနဲ့ပတ်သက်တာတွေကိုပဲ လေ့လာသွားကြပါမယ်။ String တွေကို Format ချတာနဲ့ ပတ်သက်ပြီး Output တွေကို Format ချတဲ့ အပိုင်း၊ String တွေကို Format ချတဲ့ အပိုင်းနဲ့ Message တွေကို Format ချတဲ့ အပိုင်းဆိုပြီး အပိုင်း ၃ ပိုင်းခွဲပြီး လေ့လာသွားကြပါမယ်။

Output တွေကို Format ချတဲ့ အပိုင်းနဲ့ String တွေကို Format ချတဲ့ အပိုင်းကတော့ အခြေခံအားဖြင့် Format ရေးသားပုံချင်း ဆင်တူပြီး Message တွေကို Format ချတဲ့ အပိုင်းကတော့ ရှေ့က နှစ်ပိုင်းနဲ့ မတူပါဘူး။

Formatting Outputs

Output String တွေကို Format လုပ်ဖို့အတွက် PrintStream, PrintWriter နဲ့ Console Class တွေမှာ Method တွေကိုပြင်ပေးထားပါတယ်။ System.out ဆိုတာကလဲ System Class ထဲမှာပါတဲ့ Static Variable တစ်ခုဖြစ်ပြီး Type ဟာ PrintStream ဖြစ်ပါတယ်။ ဒါ့ကြောင့် System.out.printf() နဲ့ System.out.format() Method တွေကို အသုံးပြုလို့ရနေတာ ဖြစ်ပါတယ်။

Return Type	Methods
void	format(String format, Object … args)
void	printf(String format, Object … args)
void	printf(Locale l, String format, Object … args)

Arguments

Arguments	Description
format	String String Format Specifier တွေကို အသုံးပြုပြီး Format ချမည့်ပုံစံကို သတ်မှတ်ပေးထားသော String Object
args	Object … Format String မှာပါတဲ့ String Format Specifier တွေနေရာမှာ အစားထိုးမည့် Data Object Varargs
l	Locale Format String မှာပါတဲ့ နိုင်ငံဆိုင်ရာသတ်မှတ်ချက်ကို သတ်မှတ်ပေးနိုင်သော Locale Object

Internationalization အရ Output String တွေကို Format ချချင်တယ်ဆိုရင်ဖြင့် Locale ကို သတ်မှတ်ပေးနိုင်သော printf(Locale l, String format, Object … args) Method ကို အသုံးပြုနိုင်ပြီး၊ Locale ကို သတ်မှတ်စရာမလိုဘူးဆိုရင်တော့ format(String format, Object … args) နဲ့ printf(String format, Object … args) တို့ကို အသုံးပြုနိုင်ပါတယ်။

```
      // String format
      String format = "My name is %s and I am %d' years old.%n";

      // "My name is Mike and I am 20' years old."
      console.format(format, "Mike", 20);

      // "My name is Mike and I am 20' years old."
      console.printf(format, "Mike", 20);
```

String format ထဲမှာပါတဲ့ “%s” , “%d” နဲ့ “%n” တို့ဟာ Format Specifier တွေဖြစ်ကြပြီး Method Arguments တွေဖြစ်ကြတဲ့ “Mike” နဲ့ 20 တို့ဟာ Format Specifier တွေမှာ အစားထိုးမည့် Argument တွေဖြစ်ကြပါတယ်။ Specifier တွေကို % နဲ့စရေးပြီး %s ဆိုရင်တော့ String ကလာပြီး အစားထိုးနိုင်တယ်လို့ အဓိပ္ပါယ်ရပြီး String ဖြစ်တဲ့ “Mike” က အစားထိုးပါမယ်။ %d ဆိုရင်တော့ Digit တွေကို အစားထိုးနိုင်ပြီး ဒီနေရာမှာ 20 က အစားထိုးပါ

တယ်။ တဖန် %n ကတော့ Line Sperator ဖြစ်ပြီး အောက်ကို စာကြောင်းတစ်ကြောင်း ဆင်းမယ်လို့ အဓိပ္ပါယ်ရပါတယ်။ ဒါ့ကြောင့် Method နှစ်ခုလုံးရဲ့ Output တွေဟာ "My name is Mike and I am 20' years old." ဖြစ်နေတာ ဖြစ်ပါတယ်။ အထက်ပါ Method နှစ်ခုလုံးဟာ Argument List ကော အလုပ်လုပ်ပုံပါ အတူတူပဲ ဖြစ်တဲ့အတွက် နှစ်သက်ရာ Method ကို ရွေးချယ် အသုံးပြုနိုင်ပါတယ်။

Formatting String

Format လုပ်ထားတဲ့ String တွေကို ရယူဖို့အတွက် String Class မှာ Static Method တွေပြင်ပေးထားပါတယ်။

Return Type	Methods
String	static format(String format, Object … args)
String	static format(Locale l, String format, Object … args)

Arguments တွေကတော့ OutputStream တွေမှာပါတဲ့ Argument တွေနဲ့ အတူတူပါပဲ။ Locale ကို သတ်မှတ်နိုင်တဲ့ Method နဲ့ သတ်မှတ်စရာမလိုတဲ့ Method တွေကို Overload လုပ်ပြီး ရေးသားပေးထားပါတယ်။

```
// String format
String format = "My name is %s and I am %d' years old.";

// "My name is Mike and I am 20' years old."
String formattedStr = String.format(format, "Mike", 20);

System.out.println(formattedStr);
```

String Format ရေးသားပုံ၊ Argument တွေနဲ့ Formatted String ကို တည်ဆောက်ပုံတွေကတော့ PrintStream တွေမှာပါတဲ့ Method တွေနဲ့ အတူတူပဲ ဖြစ်ပါတယ်။ ဒါပေမဲ့ String ရဲ့ format() method မှာက Formatted String ကို Return ပြန်ပေးနိုင်ပါတယ်။ Format ချထားပြီးသား String တွေကို ရယူလိုတယ်ဆိုရင်တော့ ဒီ Method ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ဆက်လက်ပြီး Format String တွေကို ဘယ်လိုရေးရသလဲ ဆိုတာကို လေ့လာသွားကြရအောင်။

String Format Syntax

ရှေ့မှာလေ့လာခဲ့တဲ့ printf() နဲ့ format() Method တွေမှာ Format String တွေကို ရေးသားအသုံးပြုခဲ့ကြပါတယ်။ ဒီနေရာမှာတော့ Format String တွေကို ဘယ်လိုရေးသားတယ်ဆိုတဲ့ အပိုင်းကို လေ့လာသွားပါမယ်။

```
Calendar c = Calendar.getInstance();
String today = String.format("Today is %1$tm %1$te %1$tY", c);
```

Format Method မှာပါတဲ့ ပထမဆုံး Parameter ဟာ format string ဖြစ်ပါတယ်။ Format String ကို Format ချချင်တဲ့ String ကြားမှာ “%1$™” , “%1$te” , “%1$tY” အစရှိတဲ့ Embedded Format Specifier တွေနဲ့ ထည့်သွင်းရေးသားထားပါတယ်။ String တစ်ခုကို Format ချတဲ့ နေရာမှာ Format String ထဲမှာပါတဲ့ Embedded Format Specifier တွေနေရာမှာ Argument တွေနဲ့ အစားထိုးတည်ဆောက်ပေးသွားမှာ ဖြစ်ပါတယ်။

Format Specifier တွေကို ရေးသားမယ်ဆိုရင် Argument တွေကို သတ်မှတ်ပြီး သုံးမလား မသုံးဘူးလားဆိုတဲ့ အပေါ်မှာမူတည်ပြီး နှစ်မျိုးခွဲခြားနိုင်ပါတယ်။ တဖန် Argument တွေကို သတ်မှတ်ပြီး သုံးမယ်ဆိုရင်လဲ Date အမျိုးအစားတွေကို အသုံးပြုတဲ့ ပုံစံနဲ့ အခြားသော Char, String နဲ့ Numeric Data Type တွေကို သုံးမည့်ပုံစံဆိုပြီး နှစ်မျိုးခွဲခြား ရေးသားနိုင်ပါတယ်။ ဒါ့ကြောင့် Format Specifier တွေကို ပုံစံသုံးမျိုး ရေးသားနိုင်ပါတယ်။

Argument	Type	Format Specifier
Yes	General, Char, Numeric Type	%[argument_index$][flags][width][.precision]conversion
Yes	Date & Calendar	%[argument_index$][flags][width]conversion
No	All Types	%[flags][width]conversion

Format Specifier တွေကို ရေးသားတဲ့ နေရာမှာ % နဲ့ conversion တွေဟာ မပါမဖြစ်ပါရပါမယ်။ ဆိုလိုတာက % နဲ့ conversion မပါပဲနဲ့ Format Specifier အနေနဲ့ မသုံးနိုင်ပါဘူး။ ကျန်တဲ့ [ ] နဲ့ပိတ်ပြီးရေးသားထားတဲ့ အရာတွေကတော့ လိုအပ်ပါမှ ရေးသားရမည့် အစိတ်အပိုင်းတွေဖြစ်ကြပါတယ်။

Argument Index

Format Specifier နေရာမှာ အသုံးပြုမည့် Argument List ထဲက Argument ရဲ့ နေရာကို သတ်မှတ်လိုတဲ့ နေရာမှာ အသုံးပြုနိုင်ပြီး Digit တွေနဲ့ $ ကိုတွဲဖက်ရေးသားရမှာ ဖြစ်ပါတယ်။ Digit တွေကတော့ အသုံးပြုမည့် Argument ရဲ့ Index Number ကို ရည်ညွှန်းပါတယ်။ တဖန် < ကို အသုံးပြုပြီး ရှေ့မှာ အသုံးပြုခဲ့တဲ့ Argument Index ကိုပြန်ပြီး အသုံးပြုနိုင်ပါတယ်။

```
// Code 1 -> “A B C D"
String str1 = String.format("%s %s %s %s", "A", "B", "C", "D");

// Code 2 -> “A B A A"
String str2 = String.format("%s %s %1$s %1$s", "A", "B", "C", "D");

// Code 3 -> “A A B C"
String str3 = String.format("%s %1$s %s %s", "A", "B", "C", "D");

// Code 4 -> “A B B C"
String str4 = String.format("%s %s %<s %s", "A", "B", "C", "D");
```

Code 1 မှာကတော့ Argument Index တွေ ရေးမထားပါဘူး။ အဲ့ဒီအတွက် %s တွေနေရာမှာ အစဉ်အလိုက် Argument တွေ အစားထိုးသွားပြီး “A B C D” လို့ Format ချပေးမှာ ဖြစ်ပါတယ်။

Code 2 မှာတော့ နောက်ဆုံးက Format Specifier နှစ်ခုကို %1$s လို့ရေးသားထားပါတယ်။ 1$ ဟာ Argument Index 1 ကို ရည်ညွှန်းပါတယ်။ ဒါကြောင့် အဲ့ဒီနေရာတွေကို A က အစားထိုးသွားပြီး “A B A A” ကိုရရှိပါတယ်။

Code 3 မှာတော့ ဒုတိယ Format Specifier နေရာမှာ 1$ ကို ရေးသားထားတဲ့အတွက် Argument 1 ကို အသုံးပြုပြီး “A A B C” လို့ထွက်ပါတယ်။

Code 4 မှာတော့ တတိယ Format Specifier နေရာမှာ < ကို ရေးသားထားတဲ့ အတွက် အရှေ့က Argument Index ဖြစ်တဲ့ 2 ကို အသုံးပြုသွားပါတယ်။ ဒါ့ကြောင့် “A B B C” ဆိုပြီး Format ချပေးမှာ ဖြစ်ပါတယ်။

Conversions

Format Specifier ကိုရေးသားတဲ့ နေရာမှာ မဖြစ်မနေ ရေးသားရမည့် အစိတ်အပိုင်းဖြစ်ပြီး၊ ဘယ်လို Data Type မျိုးအနေနဲ့ Format လုပ်တဲ့နေရာမှာ အသုံးပြုမယ်ဆိုတာကို သတ်မှတ်ပေးနိုင်ပါတယ်။ အသုံးပြုနိုင်တဲ့ Conversion Category တွေကတော့ General, Character, Numeric, Date / Time, Percent နဲ့ Line Separator တို့ပဲ ဖြစ်ပါတယ်။

General, Character, Percent and Line Separator

Conversion	Category	Description
b, B	General	Argument ဟာ null ဖြစ်နေရင်တော့ false လို့ပဲ ဖေါ်ပြမှာ ဖြစ်ပါတယ် Boolean Type ဆိုရင်တော့ Boolean တန်ဖိုးကို အသုံးပြုပါမယ် အခြားတန်ဖိုးတစ်ခုခုဆိုရင်တော့ TRUE လို့ပြန်ပြပေးနိုင်မှာ ဖြစ်ပါတယ်
h, H	General	Argument ဟာ null ဖြစ်နေရင်တော့ null လို့ပဲ ဖေါ်ပြမှာ ဖြစ်ပါတယ် မဟုတ်ရင်တော့ Integer.toHexString(arg.hashCode()) ကို အသုံးပြုပါမယ်
s, S	General	Argument ဟာ null ဖြစ်နေရင်တော့ null လို့ပဲ ဖေါ်ပြမှာ ဖြစ်ပါတယ် Formattable Interface ကို Implement လုပ်ထားတဲ့ Object တွေဆိုရင်တော့ formatTo() Method ကို Invoke လုပ်ပြီး သုံးပါမယ် မဟုတ်ရင်တော့ toString() ကို အသုံးပြုပါမယ်
c, C	Character	Unicode Character တွေကို အသုံးပြုပါမယ်
%	Percent	ရာခိုင်နှုန်း Char ကို ဖေါ်ပြတဲ့နေရာမှာ အသုံးပြုပါတယ်
n	Line Separator	Line Separator Char တစ်လုံးကို ဖြည့်စွက်ပေးမှာဖြစ်ပြီး အသုံးပြုနေတဲ့ OS အလိုက် မှန်ကန်စွာ ဖြည့်စွက်ပေးနိုင်မှာ ဖြစ်ပါတယ်

```
// "false TRUE false TRUE"
String str1 = String.format("%b %B %b %B", null, true, false, "D");

// "null 1 42 84567C67"
String str2 = String.format("%h %H %h %H", null, 1, "B", new Date());
```

အထက်ပါနမူနာကတော့ Boolean နဲ့ Hash String ကို အသုံးပြုတဲ့ နမူနာဖြစ်ပါတယ်။ Boolean တွေမှာက Argument တန်ဖိုးက null ဖြစ်နေရင် false လို့ဖေါ်ပြပြီး၊ Hash String မှာကတော့ null ဖြစ်နေရင် null လို့ပဲ ဖေါ်ပြမှာ ဖြစ်ပါတယ်။

```
// "10%"
String result = String.format("%d%%", 10);
```

Percent Character '%' ဟာ Format Specifier တွေကို ရေးသားတဲ့ နေရာမှာ သုံးနေတဲ့ အတွက် Format String တွေကြားထဲမှာ ဒီအတိုင်းရေးသားထားလို့ မရနိုင်ပါဘူး။ ဥပမာအားဖြင့် 10% လို့ Format မှာပြချင်ရင် "%d%" လို့ရေးသားလို့မရနိုင်ပါဘူး။ ဒုတိယ % ကိုတွေ့ရင့် နောက်မှာ Conversion မရှိလို့ဆိုပြီး UnknownFormatConversionException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။ အထက်ပါ နမူနာမှာ %d ဟာ Digit ကို ဖေါ်ပြဖို့ Format Specifier တစ်ခုဖြစ်ပြီး %% ဟာ % ကို ဖေါ်ပြဖို့အတွက် အသုံးပြုရတဲ့ Format Specifier တစ်ခုဖြစ်ပါတယ်။

Numeric Conversions

Conversion	Category	Description
d	Integral	Decimal Integer အနေနဲ့ Format လုပ်ပေးမှာ ဖြစ်ပါတယ်
o	Integral	Octal Integer အနေနဲ့ Format လုပ်ပေးမှာ ဖြစ်ပါတယ်
x, X	Integral	Hexadecimal Integer အနေနဲ့ Format လုပ်ပေးမှာ ဖြစ်ပါတယ်
e, E	Floating Number	Floating Number တွေကို Scientific Notation နဲ့ Format လုပ်ပေးမှာ ဖြစ်ပါတယ်
f	Floating Number	Floating Number တွေကို Decimal Number အနေနဲ့ Format လုပ်ပေးမှာ ဖြစ်ပါတယ်
g, G	Floating Number	Precision နဲ့ Rounding Number အပေါ်မူတည်ပြီး Floating Number တွေကို Scientific Notation နဲ့ Format လုပ်ပေးနိုင်မှာ ဖြစ်ပါတယ်
a, A	Floating Number	Floating Number တွေကို Hexadecimal Number အနေနဲ့ Format လုပ်ပေးမှာ ဖြစ်ပါတယ်

Integral Number တွေကို Format အတွင်း အသုံးပြုမယ်ဆိုရင်တော့ အောက်ပါ အတိုင်းရေးသားရမှာ ဖြစ်ပါတယ်။ Hexadecimal Number တွေမှာ x ကို အသုံးပြုရင် lower case နဲ့ ဖော်ပြနိုင်မှာ ဖြစ်ပြီး X ကို သုံးထားရင်တော့ Upper Case နဲ့ ဖော်ပြပေးနိုင်မှာ ဖြစ်ပါတယ်။

```
      // Decimal : 10
      String result1 = String.format("Decimal : %d", 10);

      // Octal : 12
      String result2 = String.format("Octal : %o", 10);

      // Hexa : a
      String result3 = String.format("Hexa : %x", 10);

      // Hexa : A
      String result4 = String.format("Hexa : %X", 10);
```

Floating Number တွေကို Format မှာ ရေးသားချင်ရင်တော့ အောက်ပါအတိုင်း ရေးသားရမှာ ဖြစ်ပါတယ်။ Conversion e ကို အသုံးပြုရင် Lower Case နဲ့ ဖော်ပြပြီး E ကို သုံးထားရင်တော့ Upper Case နဲ့ ဖော်ပြတာကို တွေ့ရပါတယ်။

```
      // Floating : 3.141593e+00
      String result1 = String.format("Floating : %e", Math.PI);

      // Floating : 3.141593E+00
      String result2 = String.format("Floating : %E", Math.PI);

      // Floating : 3.141593
      String result3 = String.format("Floating : %f", Math.PI);
```

Conversion G နဲ့ ပတ်သက်ပြီး Precision အကြောင်းနှင့်အတူ ဖော်ပြသွားပါမယ်။

Date / Time Conversions

Conversion	Category	Description
tH	Time	နာရီတွေကို 00 - 23 အထိ
tI	Time	နာရီတွေကို 01 - 12 အထိ
tk	Time	နာရီတွေကို 0 - 23 အထိ
tl	Time	နာရီတွေကို 1 - 12 အထိ
tM	Time	မိနစ်ကို 00 - 59 အထိ
tS	Time	စက္ကန့်ကို 00 - 60 အထိ (60 ကို Leap Seconds မှာသာ အသုံးပြု)
tL	Time	မိလီစက္ကန့်ကို 000 - 999 အထိ
tN	Time	နာနိုစက္ကန့်ကို 000000000 - 999999999 အထိ
tP	Time	နံနက်ပိုင်း၊ မွန်းလွဲပိုင်းကို am, pm ဖြင့်ပြပေးနိုင်
tz	Time	Numeric Time Zone
tZ	Time	String Time Zone Name
ts	Time	Epoch Time (1970/1/1 00:00:00 UTC) ကနေ စက္ကန့်အရေအတွက်
tQ	Time	Epoch Time (1970/1/1 00:00:00 UTC) ကနေ မိလီစက္ကန့်အရေအတွက်
tB	Date	လအမည် အပြည့်အစုံ "January", "February"
tb	Date	လ အတိုကောက်အမည် "Jan", "Feb"
th	Date	လ အတိုကောက်အမည် "Jan", "Feb"
tA	Date	ရက်အမည် "Sunday", “Monday"
ta	Date	ရက်အမည် အတိုကောက် "Sun", “Mon"
tC	Date	၄ လုံးပါနှစ်ကို ၁၀၀ နှင့်စားပြီး ပြမည် (00 - 99)
tY	Date	၄ လုံးပါနှစ် (2018, 0095)
ty	Date	၂ လုံးပါ နှစ် (00 - 99)
tj	Date	တစ်နှစ်စာမှ ရက်စွဲ (001 - 366)
tm	Date	လအား ကိန်း ၂ လုံးဖြင့်ပြ (01 - 13)
td	Date	လတစ်လ၏ ရက်စွဲအား ကိန်း ၂ လုံးဖြင့်ပြ (01 - 31)
te	Date	လတစ်လ၏ ရက်စွဲ (1 - 31)
tR	Date Time	၂၄ နာရီဖြင့် နာရီမိနစ်အားဖေါ်ပြ (%tH:%tM)
tT	Date Time	၂၄ နာရီဖြင့် နာရီမိနစ်စက္ကန့်အားဖေါ်ပြ (%tH:%tM:%tS)
tr	Date Time	၁၂နာရီဖြင့် နာရီမိနစ်စက္ကန့်အားဖေါ်ပြ (%tI:%tM:%tS %Tp)
tD	Date Time	ရက်စွဲအား (%ty/%tm/%td) ဖြင့်ဖေါ်ပြ
tF	Date Time	ရက်စွဲအား ISO 8601 Format (%ty-%tm-%td) ဖြင့်ဖေါ်ပြ
tc	Date Time	နေ့ရက်အချိန်အား (%ta %tb %td %tT %tZ %tY) ဖြင့်ဖေါ်ပြ

```
Date today = new Date();

// I wrote this at August 29, 2018.
System.out.format("I wrote this at %tB %<td, %<tY.%n", today);

// The day is Wednesday.
System.out.format("The day is %tA.%n", today);

// Time is at 05:07:48 PM.
System.out.format("Time is at %tr.%n", today);
```

အထက်ပါအတိုင်း Date / Time တွေကို String Format အတွင်း ထည့်သွင်းရေးသားနိုင်ပါတယ်။ Date Time Format နဲ့ပတ်သက်ပြီး Date & Calendar အခန်းနဲ့ Date & Time API အခန်းတို့မှာ ဆက်လက်ဖော်ပြသွားပါဦးမည်။

Width

- ✓ Width ဖြင့် Output မှာဖော်ပြမည့် အနဲဆုံး Character Count ကို သတ်မှတ်ပေးနိုင်ပါတယ်
- ✓ Width တွေကို General, Numeric နဲ့ Date / Time Conversion တွေမှာ ရေးသား အသုံးပြုနိုင်ပါတယ်
- ✓ Line Separator Conversions မှာ Width ကို သတ်မှတ်လို့မရနိုင်ပါဘူး၊ ရေးသားမိရင် IllegalFormatWidthException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်
- ✓ Percent Conversion မှာ ရေးသားမိရင်တော့ Exception တော့မဖြစ်ပါဘူး၊ ဒါပေမဲ့ Width ပါဝင်ခြင်းရှိမှာ မဟုတ်ပါဘူး

```
// Code 1 -> "      Mike"
System.out.format("%10s%n", "Mike");

// Code 2 -> "       100"
System.out.format("%10d%n", 100);

// Code 3 -> "0000000100"
System.out.format("%010d%n", 100);

// Code 4 -> "  3.141593"
System.out.format("%10f%n", Math.PI);
```

Code 1 ထဲမှာ %10s လို့ရေးထားတာဟာ Format Specifier တစ်ခုဖြစ်ပြီး 10 ဟာ Width ဖြစ်ပါတယ်။ String Argument တစ်ခုကို Character 10 လုံးစာ Format ချမယ်လို့ အဓိပ္ပါယ်ရပါတယ်။ တကယ်လို့ Argument String ရဲ့ Length ဟာ ၁၀ လုံးမပြည့်ခဲ့ရင် ဘယ်ဘက်ခြမ်းမှာ Space တွေကို ဖြည့်ပြီး Format ချသွားမှာ ဖြစ်ပါတယ်။ တကယ်လို့ Argument String ဟာ 10 လုံးကျော်ခဲ့မယ်ဆိုရင် ရှိသလောက် အရေအတွက်ကို ပြန်ပြပေးမှာ ဖြစ်ပါတယ်။

Code 2 ကတော့ Integer ကို Format ချနေတာပါ။ %10d လို့ရေးထားတဲ့ အတွက် ၁၀ လုံးမပြည့်တဲ့ နေရာတွေကို Space နဲ့ ဘယ်ဘက်မှာ ဖြည့်ပြီး Format ချသွားပါတယ်။

Code 3 မှာတော့ %010d ဖြစ်တဲ့ အတွက် ၁၀ မပြည့်တဲ့ နေရာတွေကို 0 အစားထိုးဖြည့်ပြီး Format ချပေးနိုင်ပါတယ်။ အလားတူပဲ Floating Number တွေကိုလဲ Width သတ်မှတ်ပြီး Format ချလို့ရပါတယ်။

Precision

Format Specifier တွေကို ရေးသားတဲ့ နေရာမှာ Precision တွေကို ရေးသားအသုံးပြုနိုင်ပါတယ်။ Precision တွေကို ".precision" ကို ရေးသားရမှာ ဖြစ်ပြီး Conversion တွေရဲ့ ရှေ့မှာ ရေးသားရမှာ ဖြစ်ပါတယ်။ Precision တွေကို General Conversion တွေနဲ့ Floating Number Conversion တွေမှာသာ ရေးသားအသုံးပြုနိုင်မှာဖြစ်ပါတယ်။ တစ်ခြား Conversion တွေနဲ့တွဲပြီး ရေးသားမိပါက IllegalFormatPrecisionException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

General Conversion တွေနဲ့ Precision ကို ရေးသားထားရင် အများဆုံးဖော်ပြနိုင်တဲ့ Character Count ကို သတ်မှတ်ပေးနိုင်ပါတယ်။

```
// “Mike Is Te"
System.out.format("%.10s%n", "Mike Is Test Data");
```

အထက်ပါအတိုင်း "%10.10s" လို့ရေးသားထားတဲ့ နေရာမှာ ".10" ဟာ Precision ဖြစ်ပါတယ်။ အများဆုံး စာလုံး ၁၀ လုံးစာကိုပဲ ဖော်ပြပေးမှာ ဖြစ်ပါတယ်။ Argument ဟာ "Mike is Test Data" ဖြစ်ပေမဲ့ Precision ကို 10 လို့ သတ်မှတ်ထားတဲ့ အတွက် ရလဒ်အနေနဲ့ "Mike is Te" အထိပဲ ဖော်ပြပေးနိုင်တာဖြစ်ပါတယ်။

Floating Number Conversion တွေနဲ့ Precision ကို ရေးသားရင်တော့ အဓိပ္ပါယ်တစ်မျိုးရပါတယ်။ "e", "E" နဲ့ "f" မှာဆိုရင် ဒဿမကိန်းရဲ့ အများဆုံးဖော်ပြနိုင်တဲ့ အရေအတွက်ကို သတ်မှတ်ပေးမှာ ဖြစ်ပါတယ်။

```
// “3.142"
System.out.format("%.3f%n", Math.PI);
```

Conversion "g" နဲ့ "G" တွေမှာဆိုရင် Round လုပ်ပြီးတဲ့အခါမှာ ရှိနေတဲ့ Digit အရေအတွက် စုစုပေါင်းကို သတ်မှတ်ပေးနိုင်ပါတယ်။

```
// “3.14"
System.out.format("%.3g%n", Math.PI);
```

Conversion "a" နဲ့ "A" တွေမှာဆိုရင်တော့ Precision တွေဟာ အလုပ်လုပ်မှာ မဟုတ်ပါဘူး။

Flags

Format Specifier တွေကို ရေးသားတဲ့ နေရာမှာ Default Format တွေကို Customize လုပ်ဖို့အတွက် Flag တွေကို အသုံးပြုနိုင်ပါတယ်။ Flag တွေဟာ Conversion တွေရဲ့ အထောက်အကူတွေဖြစ်တဲ့ အတွက် Conversion အမျိုးအစားအပေါ်မူတည်ပြီး ရေးသားလို့ ရနိုင်တယ် မရနိုင်ဘူးဆိုတာရှိပါတယ်။ String Format တွေမှာ အသုံးပြုနိုင်တဲ့ Flag တွကတော့ Minus ('-'), Hash ('#'), Plus ('+') , Space (' ') , Comma (',') နဲ့ Left Parenthesis '(' တို့ပဲဖြစ်ပါတယ်။

```
// Code 1 -> "     Hello"
System.out.format("%10s%n", "Hello");

// Code 2 -> "Hello     "
System.out.format("%-10s%n", "Hello");
```

Minus (‘-‘) Flag တွေကို String Format ချတဲ့နေရာမှာ Padding Side ကို သတ်မှတ်တဲ့ နေရာမှာ အသုံးပြုပါတယ်။ ပြီးတော့ Width တွေကို ရေးသားထားမှသာ Minus Flag ကို ရေးသားလို့ရမှာ ဖြစ်ပါတယ်။ Width မပါပဲ Minus Flag ကို ရေးသားမိရင် MissingFormatWidthException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

Width တွေကို သတ်မှတ်ထားတဲ့ နေရာမှာ တကယ်လို့ Argument ရဲ့ တန်ဖိုးသာ သတ်မှတ်ထားတဲ့ Width တန်ဖိုးထက် ငယ်နေခဲ့မယ်ဆိုရင် Code 1 ထဲက အတိုင်း မလောက်တဲ့ အရေအတွက်အလိုက် ဘယ်ဘက်မှာ Space တွေကို ဖြည့်ပြီး ညာဘက်ကို ကပ်ပြီး ညီအောင် စီပေးမှာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ Code 2 မှာလို Minus Flag ကို အသုံးပြုလိုက်တဲ့ အခါမှာ (“%-10s”) ဘယ်ဘက်ကို ကပ်ပြီး ညီအောင်စီပေးမှာဖြစ်ပါတယ်။ မလောက်တဲ့ နေရာတွေကိုတော့ Space တွေကို အသုံးပြုပြီး သတ်မှတ်ထားတဲ့ Width ဖြစ်အောင် စီပေးပါမယ်။

ကိန်းဂဏန်းတွေကို Format ချတဲ့နေရာမှာ Right Justified ဖြစ်တာ (ညာဘက်မှာ စီတာ) ဟာ ကြည့်ကောင်းပေမဲ့ String လိုနေရာမျိုးမှာတော့ Left Justified (ဘယ်ဘက်မှာစီတာ) ဟာပိုပြီး သင့်တော်ပါတယ်။

```
// "1750"
System.out.format("%o%n", 1000);

// "01750"
System.out.format("%#o%n", 1000);

// "3E8"
System.out.format("%X%n", 1000);

// "0x3E8"
System.out.format("%#x%n", 1000);

// "0X3E8"
System.out.format("%#X%n", 1000);
```

Hash (‘#’) Flag ကိုတော့ ‘o’, ‘x’ နဲ့ ‘X’ Conversions တို့နဲ့ တွဲဖက်အသုံးပြုနိုင်ပြီး အခြားသော ပုံစံတစ်မျိုးနဲ့ ပြပေးနိုင်ပါတယ်။ Octal Decimal တွေဆိုရင်တော့ ရလဒ်ရဲ့ ရှေ့မှာ 0 ကို ဖြည့်ပြီး ဖေါ်ပြပေးမှာ ဖြစ်ပြီး Hexadecimal တွေမှာဆိုရင်တော့ ရှေ့မှာ ‘0x’ ဒါမှမဟုတ် ‘0X’ ကို ဖြည့်စွက်ပြီးဖေါ်ပြပေးမှာ ဖြစ်ပါတယ်။

```
// "-11"
System.out.format("%+d%n", new BigInteger("-11"));

// "+13"
System.out.format("%+o%n", new BigInteger("11"));

// "+B"
System.out.format("%+X%n", new BigInteger("11"));

// "+11"
System.out.format("%+d%n", 11);

// "-3.141593"
System.out.format(“%+f%n", 0 - Math.PI);
```

Plus (‘+’) Flag ကို Number တွေရဲ့ အပေါင်းအနုတ် သင်္ကေတ တွေကို Format ထဲမှာဖေါ်ပြပေးလိုတဲ့ အခါမျိုးမှာ အသုံးပြုနိုင်ပါတယ်။ အပေါင်းကိန်းတွေဆိုရင် Plus Sign ကို Format ထဲမှာဖေါ်ပြပြီး၊ အနုတ်ကိန်းကို Minus Sign နဲ့ ဖေါ်ပြပေးနိုင်ပါမယ်။

Plus Flag ကို BigInteger တွေနဲ့ ‘d’, ‘o’, ‘x’, ‘X’ Conversions တွေနဲ့ တွဲဖက် အသုံးပြုတဲ့ နေရာမျိုးမှာ အသုံးပြုနိုင်ပါတယ်။ ပြီတော့ byte, short, int, long အစရှိတဲ့ Integral Data Type တွေနဲ့ သက်ဆိုင်ရာ Wrapper Type တွေနဲ့ ‘d’ conversions တွေကိုတွဲဖက်ရေးသားတဲ့ နေရာမှာ အသုံးပြုနိုင်ပါတယ်။ တဖန် Floating Numberr တွေကို Format ချတဲ့ နေရာမှာ အသုံးပြုနိုင်ပါတယ်။

အခြားသောနေရာတွေမှာ Plus Flag ကို အသုံးပြုမိရင်တော့ FormatFlagsConversionMismatchException ကိုဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

```
// " 100"
System.out.format("% d%n", 100);

// "-3.141593"
System.out.format("% f%n", 0 - Math.PI);
```

Space (‘ ‘) Flag တွေကိုတော့ ‘d’ နဲ့ ‘f’ Conversions တို့နဲ့ တွဲဖက်အသုံးပြုနိုင်ပြီး အပေါင်းကိန်းတွေနေရာမှာ အရှေ့မှာ Space တစ်လုံးကိုခံပြီး Format လုပ်ပေးနိုင်ပါတယ်။

```
// "18,985"
System.out.format("%,d%n", 18985);

// "181,715.098000"
System.out.format("%,f%n", 181715.098);
```

Comma (‘,’) Flag ကိုတော့ ‘d’ conversion နဲ့ Floating Number Conversion တွေမှာ ရေးသားနိုင်ပါတယ်။ အသုံးပြုနေတဲ့ Locale အလိုက် ကိန်းတွေကို ခွဲပြီးရေးသားပေးနိုင်ပါတယ်။

```
// "100"
System.out.format("%(d%n", 100);
// "(100)"
System.out.format("%(d%n", -100);

// "10.019"
System.out.format("%(.3f%n", 10.019);
// "(10.019)"
System.out.format("%(.3f%n", -10.019);
```

Left Parenthesis ‘(‘ Flag ကိုလဲ BigInteger တွေနဲ့ ‘d’, ‘o’, ‘x’, ‘X’ Conversions တွေနဲ့ တွဲဖက် အသုံးပြုတဲ့ နေရာနဲ့၊ byte, short, int, long အစရှိတဲ့ Integral Data Type တွေနဲ့ သက်ဆိုင်ရာ Wrapper Type တွေနဲ့ ‘d’ conversions တွေကိုတွဲဖက်ရေးသားတဲ့ နေရာမှာ အသုံးပြုနိုင်ပါတယ်။ တဖန် Floating Numberr တွေကို Format ချတဲ့ နေရာမှာလဲ အသုံးပြုနိုင်ပါတယ်။ အဆိုပါ Conversions တွေမှာ Left Parenthesis ‘(‘ Flag ကို ရေးသားထားရင် အနုတ်ကိန်းတွေနေရာမှာ () နဲ့ပိတ်ပြီးဖေါ်ပြပေးမှာ ဖြစ်ပါတယ်။

Formatting Message

Application တွေကို ရေးသားတဲ့ နေရာမှာ End User တွေဆီကို Message တွေကို ဖေါ်ပြရမည့် အခါမျိုးတွေနဲ့ ကြုံတွေ့ရတတ်ပါတယ်။ အဲ့ဒီလိုအခါမျိုးမှာ String Message တွေကို အသေဖေါ်ပြလို့ရတဲ့ အခါမျိုးလဲ ရှိသလို၊ Program ထဲမှာပါတဲ့ Data တွေနဲ့ Message တွေကိုတည်ဆောက်ပြီးပြရတဲ့ အခါလဲရှိပါတယ်။

“Hello Min Lwin, you have to enter phone number!” ဆိုတဲ့ Message ကိုဖေါ်ပြမယ်ဆိုရင် “Min Lwin” ဆိုတဲ့ နေရာကို အသေရေးထားခဲ့ရင် အခြား User တွေအတွက်ဆိုရင် နောက်ထပ် Message ကို အသစ်တည်ဆောက်နေရမှာ ဖြစ်ပါတယ်။ ပြီးတော့ “phone number” ဆိုတဲ့ နေရာကို အသေရေးထားခဲ့ရင် “email address” မပါခဲ့လို့ Message ပြန်ပြမယ်ဆိုရင်တောင် နောက်ထပ် Message တွေကို ရေးသားနေရပါမယ်။

Name နဲ့ Input Fields တွေကို Parameter အနေနဲ့ ရယူနိုင်တဲ့ Template တစ်ခုကို ရေးသားထားခဲ့ရင် Message တွေကို အမျိုးမျိုး Format ချပြီး အသုံးပြုနိုင်မှာဖြစ်ပါတယ်။ Java Language မှာ Message တွေကို Format ချနိုင်ဖို့အတွက် MessageFormat ဆိုတဲ့ Class ကိုပြင်ဆင်ပေးထားပါတယ်။

```
String template = "Hello {0}, You have to enter {1}!";

// Hello Min Lwin, You have to enter Phone Number!
String m1 = MessageFormat.format(template, "Min Lwin", "Phone Number");
System.out.println(m1);

// Hello Pa Pa, You have to enter Email Address!
String m2 = MessageFormat.format(template, "Pa Pa", "Email Address");
System.out.println(m2);
```

MessageFormat Class မှာ သတ်မှတ်ထားတဲ့ Message Format Pattern နဲ့ အသုံးပြုမည့် Parameter Set တို့ကို အသုံးပြုပြီး အလွယ်တကူ Format လုပ်ထားတဲ့ Message တွေကို ရေးသားဖို့အတွက် format() ဆိုတဲ့ Static Method တစ်ခုကို ပြင်ပေးထားပါတယ်။

Return Type	Methods
String	static format(String pattern, Object … args)

Argument အနေနဲ့ Format Pattern ကို ရေးသားထားတဲ့ String တစ်ခုနဲ့ Object Varargs Type တစ်ခုတို့ကို အသုံးပြုပါတယ်။ Format Pattern ကတော့ String Object ဖြစ်ပြီး String ကြားထဲမှာ Format Element တွေကို ညှပ်ပြီးရေးသားထားပါတယ်။ အထက်နမူနာထဲမှာပါတဲ့ {0} နဲ့ {1} တို့ဟာ Format Template တွေဖြစ်ပါတယ်။ ဒုတိယ Argument ကတော့ Verarges Type ဖြစ်တဲ့ အတွက် Format Element တွေမှာ အစားထိုးရေးသာမည့် Object တွေကို အသုံးပြုလိုသလောက် Comma ခံပြီးရေးသားရမှာ ဖြစ်ပါတယ်။

အထက်နမူနာထဲက “Min Lwin”, “Phone Number” ဆိုတာကတော့ Arguments တွေပဲ ဖြစ်ပါတယ်။ Format ချပြီးတဲ့ နောက်မှာ Format Element {0} နေရာကို ပထမ Argument ကအစားထိုးသွားပြီး {1} နေရာကို ဒုတိယ Argument က အစားထိုးသွားတဲ့ အတွက် “Hello Min Lwin, You have to enter Phone Number!” ဆိုတဲ့ String Message ကို တည်ဆောက်ပေးနိုင်တာပဲ ဖြစ်ပါတယ်။

Message Format ကို အသုံးပြုခြင်းအားဖြင့် သတ်မှတ်ထားတဲ့ Message Pattern တစ်ခုနဲ့ Argument အမျိုးမျိုးကို အသုံးပြုပြီး မတူညီတဲ့ Message တွေကို အလွယ်တကူတည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။

Format Elements

Format Element တွေကို Message Format Pattern တွေထဲမှာ ရောပြီးရေးသားနိုင်ပါတယ်။ Format Element တွေကို Argument Index, Format Type နဲ့ Format Style တို့နဲ့ ဖွဲ့စည်းတည်ဆောက်ထားပါတယ်။ Format Element တွေကို အောက်ပါ ပုံစံ ၃ မျိုးနဲ့ ရေးသားနိုင်ပါတယ်။

1. {Argument Index}
2. {Argument Index, Format Type}
3. {Argument Index, Format Type, Format Style}

Argument Index

```
Object [] arguments = {"A", "B", "C"};

// Code 1 -> A, B, C
String m1 = MessageFormat.format("{0}, {1}, {2}", arguments);

// Code 2 -> C, B, A
String m2 = MessageFormat.format("{2}, {1}, {0}", arguments);

// Code 3 -> A, A, A
String m3 = MessageFormat.format("{0}, {0}, {0}", arguments);

// Code 4 -> A, B, {3}
String m4 = MessageFormat.format("{0}, {1}, {3}", arguments);
```

Argument Index တွေဟာ Format Element တွေမှာ မဖြစ်မနေပါဝင်ရမည့် အစိတ်အပိုင်းဖြစ်ပြီး၊ ရှေ့ဆုံးမှာ ရေးသားရမှာ ဖြစ်ပါတယ်။ Argument Index တွေဟာ Arguments တွေထဲက အသုံးပြုမည့် Index Number ကို သတ်မှတ်ပေးနိုင်ပြီး Array တွေ List တွေလိုပဲ သုညကနေစတင်ရမှာ ဖြစ်ပါတယ်။

Message တွေကို Format ချတဲ့နေရာမှာ Format Element ရဲ့ Argument Index တွေနဲ့ Arguments Array ထဲက Object ကိုယူပြီး Message တွေကို တည်ဆောက်ပေးတာပဲ ဖြစ်ပါတယ်။ Format Index တွေအတိုင်း Argument တွေထဲမှာ တန်ဖိုးကို ရှာမတွေ့ရင်တော့ Code 4 ထဲမှာလို Format မချတော့ပဲ Format Pattern ထဲမှာ ပါတဲ့ အတိုင်းပဲ ပြန်ဖေါ်ပြပါတယ်။

Format Type

```
String pattern = "I am learning Java {0} at {1, date}, {1,time}";
Object [] arguments = {11, new Date()};

// I am learning Java 11 at Sep 3, 2018, 9:17:45 PM
String m1 = MessageFormat.format(pattern, arguments);
System.out.println(m1);
```

Format Type တွေဟာ Format Element တွေရဲ့ ဒုတိယနေရာမှာ ရေးသားရပြီး အသုံးပြုမည့် Argument ရဲ့ Type ကို သတ်မှတ်ပေးနိုင်ပါတယ်။ number, date, time နဲ့ choice တို့ထဲက တစ်ခုခုကို ရွေးချယ် အသုံးပြုနိုင်ပါတယ်။

Format Style

Format Style တွေကို Format Type တွေနဲ့ တွဲဖက်ရေးသားရမှာ ဖြစ်ပြီး၊ Format Type အလိုက် ရေးသားနိုင်တဲ့ Format Style တွေဟာ မတူညီကြပါဘူး။ တကယ်တော့ Format Element တွေမှာ Format Type တွေပါလာရင် Format Style အလိုက် သက်ဆိုင်ရာ Sub Format Object တွေကို Create လုပ်ပြီး Format Element ရဲ့ တန်ဖိုးကို Format လုပ်ပေးမှာ ဖြစ်ပါတယ်။ တည်ဆောက်သွားမည့် Sub Format Object တွေကတော့ အောက်ပါအတိုင်းဖြစ်ပါတယ်။

Format Type	Format Style	Sub Format Created
(none)	(none)	null
number	(none)	NumberFormat.getInstance(getLocale())
	integer	NumberFormat.getIntegerInstance(getLocale())
	currency	NumberFormat.getCurrencyInstance(getLocale())
	percent	NumberFormat.getPercentInstance(getLocale())
	[sub pattern]	new DecimalFormat(pattern, DecimalFormatSymbols.getInstance(getLocale()))
date	(none)	DateFormat.getDateInstance(DateFormat.DEFAULT, getLocale())
	short	DateFormat.getDateInstance(DateFormat.SHORT, getLocale())
	medium	DateFormat.getDateInstance(DateFormat.DEFAULT, getLocale())
	long	DateFormat.getDateInstance(DateFormat.LONG, getLocale())
	full	DateFormat.getDateInstance(DateFormat.FULL, getLocale())
	[sub pattern]	new SimpleDateFormat(pattern, getLocale())
time	(none)	DateFormat.getTimeInstance(DateFormat.DEFAULT, getLocale())
	short	DateFormat.getTimeInstance(DateFormat.SHORT, getLocale())
	medium	DateFormat.getTimeInstance(DateFormat.DEFAULT, getLocale())
	long	DateFormat.getTimeInstance(DateFormat.LONG, getLocale())
	full	DateFormat.getTimeInstance(DateFormat.FULL, getLocale())
	[sub pattern]	new SimpleDateFormat(pattern, getLocale())
choice	[sub pattern]	new ChoiceFormat(pattern)

FORMATTING NUMBERS

```
// integer -> "1,000"
String m1 = MessageFormat.format("{0,number,integer}", 1000.09);

// currency -> "MMK100,000.00"
String m2 = MessageFormat.format("{0,number,currency}", 100000);

// percent -> "79%"
String m3 = MessageFormat.format("{0,number,percent}", 0.79);

// Custom Format -> "10,790,000.80"
String m4 = MessageFormat.format("{0,number,#,##0.00}", 10790000.798);
```

Number Format Type တွေနဲ့ Integer Format Style ကို အသုံးပြုပြီး Floating Number တွေကိုလဲ Integer အနေနဲ့ ဖေါ်ပြပေးနိုင်မှာ ဖြစ်ပါတယ်။ Number Separated Character တွေကိုတော့ လိုအပ်သလို အသုံးပြုနေတဲ့ Locale အလိုက် သတ်မှတ်ပေးသွားမှာ ဖြစ်ပါတယ်။

Currency Format Style ကို အသုံးပြုမယ်ဆိုရင် သက်ဆိုင်ရာ Locale အလိုက် Currency တွေနဲ့ တွဲဖက်ဖေါ်ပြပေးနိုင်ပါမယ်။ အထက်ပါ အတိုင်း "MMK100,000.00" ဆိုပြီးဖေါ်ပြပေးသွားနိုင်ပါတယ်။ Percent Format Style ကို အသုံးပြုမယ်ဆိုရင် Argument တန်ဖိုးကို 100 နဲ့ မြှောက်ပြီး % ပုံစံနဲ့ ဖေါ်ပြပေးသွားမှာ ဖြစ်ပါတယ်။ မိမိကိုယ်ပိုင် Format ကို ရေးသားမယ်ဆိုရင်တော့ Decimal Format ရေးသားပုံအတိုင်းရေးသားရမှာ ဖြစ်ပါတယ်။

FORMATTING DATE & TIME

```
// short -> "9/3/18"
String m1 = MessageFormat.format("{0,date,short}", date);

// medium -> "Sep 3, 2018"
String m2 = MessageFormat.format("{0,date,medium}", date);

// time,short -> "10:52 PM"
String m3 = MessageFormat.format("{0,time,short}", date);

// time,long -> "10:52:11 PM MMT"
String m4 = MessageFormat.format("{0,time,long}", date);

// full -> "2018-09-03 22:28"
String m5 = MessageFormat.format("{0,date,yyyy-MM-dd HH:mm}", date);
```

Format Type date နဲ့ time တွေမှာ short, medium, long နဲ့ full တို့ကို အသုံးပြုနိုင်ပါတယ်။ အသုံးပြုနေတဲ့ Locale အလိုက် သတ်မှတ်ထားတဲ့ Format Style တွေနဲ့ အသီးသီးပေါ်ပြပေးနိုင်ပါတယ်။ Simple Date Format ရဲ့ Style ကို အသုံးပြုပြီး နှစ်သက်ရာ Custom Format တွေကို ရေးသားအသုံးပြုနိုင်ပါတယ်။ Date Format နဲ့ ပတ်သက်ပြီး Date & Calendar အခန်းမှာ အသေးစိတ်ဖေါ်ပြထားပါတယ်။

Choice Format နဲ့ပတ်သက်ပြီး MessageFormat Object ရဲ့နောက်မှာ ဖေါ်ပြသွားပါမယ်။

MessageFormat Object

Message တွေကို Format ချတဲ့ နေရာမှာ Database ထဲက Select လုပ်လာတဲ့ Data တွေနဲ့ String တွေအများကြီးကို Format ချဖို့လိုတဲ့ အခါလဲ ရှိတတ်ပါတယ်။ အဲ့ဒီ Message တွေကို အကြိမ်ကြိမ် Format ချဖို့လိုတဲ့ အခါမျိုးမှာ Static Method ကို အသုံးပြုတာထက် MessageFormat Object ကို တည်ဆောက်ပြီး အသုံးပြုတာပိုပြီး သင့်တော်ပါတယ်။

```
public static String format(String pattern, Object ... arguments) {
    MessageFormat temp = new MessageFormat(pattern);
    return temp.format(arguments);
}
```

အထက်ပါကုဒ်တွေက MessageFormat Class ရဲ့ format() method ကို ရေးသားထားတာ ဖြစ်ပါတယ်။ Method Body ထဲမှာ MessageFormat Object ကို တည်ဆောက်ပြီး Format လုပ်ခိုင်းနေတာကို တွေ့ရပါတယ်။ temp Object ဟာ format() method ပြီးဆုံးတဲ့အခါမှာ ဆက်အသုံးမပြုတော့ပါဘူး။

ဒါကြောင့် အကြိမ်ကြိမ် Message တွေကို Format လုပ်ပြီးသုံးတဲ့ နေရာမျိုးမှာဆိုရင် format() method ကို သာ အသုံးပြုနေခဲ့ရင် MessageFormat Object ကို အကြိမ်ကြိမ် Create လုပ်နေမိမှာ ဖြစ်ပါတယ်။ Garbage Collector က မသုံးတော့တဲ့ Object တွေကို ဖျက်ပေးတာမှန်ပေမဲ့ ချက်ချင်းကြီးဖျက်ပေးတာ မဟုတ်ပါဘူး။ ဒါ့အပြင် Object တွေကို မလိုပဲအကြိမ်ကြိမ် Create လုပ်တယ်ဆိုတာကလဲ Performance ကို အထိုက်အလျောက် ထိခိုက်စေနိုင်ပါတယ်။

```
public static void main(String[] args) {

      String pattern = "{1} is Roll Number {0} and take {2} Course.";

      Object [][] array = {
                  {1, "Mike", "Java Basic"},
                  {2, "Jeny", "Java EE"},
                  {3, "Julia", "Spring Framework"}
      };

      MessageFormat format = new MessageFormat(pattern) ;

      for(Object [] arguments : array) {
            String message = format.format(arguments);
            System.out.println(message);
      }
}
```

Pattern တစ်ခုနဲ့ Data တစ်ခုထက်မက Message တွေကို Format လုပ်ချင်တယ်ဆိုရင် အထက်ပါအတိုင်း MessageFormat Object ကို တည်ဆောက်ပြီး အသုံးပြုနိုင်ပါတယ်။ MessageFormat Object ကို တည်ဆောက်တဲ့ နေရာမှာ Message Format Pattern String ကို အသုံးပြုပြီး တည်ဆောက်နိုင်ပါတယ်။

MessageFormat Object ရဲ့ format() ကို အသုံးပြုပြီး အကြိမ်ကြိမ် Message တွေကို တည်ဆောက်နိုင်ပါတယ်။ ဒီ Looping ပြီးတဲ့ အထိ MessageFormat Object တစ်ခုထဲကိုသာ အသုံးပြုသွားမှာ ဖြစ်ပါတယ်။

Choice Format

Message တွေကို Format ချတဲ့ နေရာမှာ အနေအထားပေါ်မူတည်ပြီး ခွဲခြားပြလိုတဲ့ အခါမျှားလဲ ရှိတတ်ပါတယ်။ ဘယ်လိုလဲ ဆိုတာကို မြင်သာအောင် နမူနာကုဒ်လေးတွေနဲ့ လေ့လာကြည့်ကြရအောင်။

```
      public static void main(String[] args) {

            String pattern = "{0} has {1} Apples";

            Object [][] array = {
                        {"Mike", 0},
                        {"Jeny", 1},
                        {"Julia", 2},
                        {"Tom", 10}
            };

            MessageFormat format = new MessageFormat(pattern) ;

            for(Object [] arguments : array) {
                  String message = format.format(arguments);
                  System.out.println(message);
            }
      }
```

အထက်ပါ ကုဒ်နမူနာကို Run ကြည့်မယ်ဆိုရင် အောက်ပါအတိုင်း အဖြေထွက်ပါမယ်။

```
Mike has 0 Apples
Jeny has 1 Apples
Julia has 2 Apples
Tom has 10 Apples
```

ရလဒ်ကို ကြည့်ရင် အဆင်မပြေဘူးဆိုတာကို တွေ့ရပါမယ်။ “Mike has 0 Apples” ဆိုတာက ကျွန်တော်တို့လို Programmer တွေကကြည့်မယ်ဆိုရင်တော့ နားလည်ပေးလိုက်လို့ရပေမဲ့ သာမန် အသုံးပြုသူတွေအတွက်ဆိုရင်တော့ “Mike has not any Apple” လို့ပြပေးတာက ပိုပြီးသင့်တော်ပါမယ်။ အလားတူပဲ “Jeny has 1 Apples” ဆိုတာကလဲ မသင့်တော်ပါဘူး။ “Jeny has an Apple” ဆိုတာက ပိုပြီးသဘာဝကျပါမယ်။

ဒီလိုအဆင်မပြေမှုတွေမှာ ChoiseFormat ကို အသုံးပြုပြီး ဖြေရှင်းနိုင်ပါတယ်။

```
      double [] limits = {0, 1, 2};
      String [] formats = {"not any Apple", "an Apple", "{1, number} Apples"};
      ChoiceFormat choices = new ChoiceFormat(limits, formats);
```

ChoiceFormat Object ကို တည်ဆောက်မယ်ဆိုရင် double array တစ်ခုနဲ့ String array တစ်ခုကို အသုံးပြုပြီး ရေးသားထားဖို့လိုအပ်ပါတယ်။ Array နှစ်ခုစလုံး Langth တွေတူနေဖို့လိုအပ်ပါတယ်။ Format လုပ်မည့် Argument ရဲ့ တန်ဖိုးဟာ 0 ဖြစ်နေတယ်ဆိုရင် Limits Array ရဲ့ Index ဟာလဲ 0 ဖြစ်ပါတယ်။ အဲ့ဒီ Index Number နဲ့ ကိုက်ညီတဲ့ formats Arrray ထဲက String ကို Format Pattern အနေနဲ့ သုံးသွားမှာပဲ ဖြစ်ပါတယ်။ အဲ့ဒါကြောင့် “not any Apple” ကို ဖေါ်ပြပေးပါမယ်။

```
	public static void main(String[] args) {

		Object [][] array = {
			{"Mike", 0},
			{"Jeny", 1},
			{"Julia", 2},
			{"Tom", 10}
		};

		// ChoiceFormat ကိုတည်ဆောက်ပါတယ်
		double [] limits = {0, 1, 2};
		String [] formats = {"not any Apple", "an Apple",
				"{1, number} Apples"};
		ChoiceFormat choices = new ChoiceFormat(limits, formats);

		// MessageFormat ကိုတည်ဆောက်ပါတယ်
		MessageFormat format = new MessageFormat("{0} has {1}.") ;

		// Argument Index 1 မှာ ChoiceFormat ကိုသုံးမယ်လို့ သတ်မှတ်
		format.setFormatByArgumentIndex(1, choices);

		for(Object [] arguments : array) {
			String message = format.format(arguments);
			System.out.println(message);
		}
	}
```

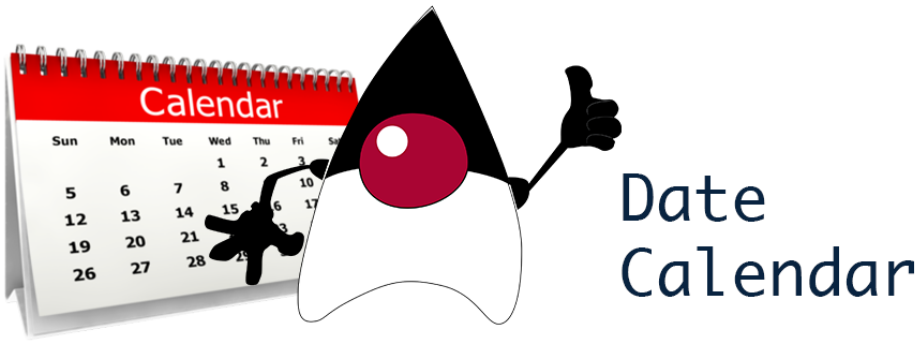
အထက်ပါအတိုင်း ChoiceFormat Object ကို တည်ဆောက်ပြီးတဲ့ နောက်မှာ MessageFormat Object ရဲ့ Index Number 1 နေရာမှာ ChoiceFormat ကို အသုံးပြုမယ်လို့ ရေးသားပေးနိုင်ပါတယ်။ ဒါဆိုရင် Argument တန်ဖိုးအလိုက် Message တွေကို ရွေးချယ် Format လုပ်ပေးနိုင်မှာ ဖြစ်ပါတယ်။

အထက်ပါ Program ကို Run ကြည့်ပြီးရလာတဲ့ Output ကတော့ အောက်ပါအတိုင်းဖြစ်ပါတယ်။

```
Mike has not any Apple.
Jeny has an Apple.
Julia has 2 Apples.
Tom has 10 Apples.
```

ကြည့်မယ်ဆိုရင် Argument တန်ဖိုးအလိုက် Format တွေကို ရွေးချယ် အသုံးပြုနိုင်တယ်ဆိုတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။

Dates & Calendar



java.utils.Date ဟာ အချိန်တွေကို ကိုယ်စားပြုဖို့ ရေးသားထားတဲ့ Java Class တစ်မျိုးဖြစ်ပါတယ်။ Java Date ဟာ Nano Seconds အထိ ဖော်ပြနိုင်ပါတယ်။ java.utils.Date ဟာ Java ဘာသာရပ်စတင်ပေါ်ပေါက်စဉ်ကတဲက ပါဝင်တဲ့ API တစ်မျိုးဖြစ်ပြီး၊ သူ့စီမှာ ရက်၊ လ၊ နှစ်ရက် နာရီ မိနစ် အစရှိတဲ့ ယူနစ်တွေကို ပြောင်းပေးနိုင်တဲ့ Method တွေပါဝင်ပေမဲ့လဲ Thread Save မဖြစ်တာက တကြောင်း၊ Internationalization မဖြစ်တာကတစ်ကြောင်း အကြောင်း အမျိုးမျိုးကြောင့် လက်ရှိ အချိန်မှာ အသုံးမပြုသင့်တော့ တာကြောင့် တားမြစ်ထားပါတယ်။ Date Class ဟာ တန်ဖိုးကို သိမ်းဆည်းရုံသာ အသုံးပြုပြီး၊ တွက်ချက်မှုတွေ၊ Interpret လုပ်တာတွေကိုတော့ Calendar Class မှ တဆင့် ပြုလုပ်ရန် စီစဉ်ထားပါတယ်။

API ကို မလေ့လာခင် ကျွန်တော်တို့ ရေးမည့် Program ထဲမှာ Date ကို ဘယ်လိုသုံးချင်တာလဲ ဆိုတာကို စပြီး လေ့လာကြည့်ကြရအောင်။

- Date Object တွေကို တည်ဆောက်မယ်
- တည်ဆောက်ထားတဲ့ Date Object ကနေ Year, Month, Date, Day, Hours, Minutes , Seconds အစရှိတဲ့ လိုအပ်တဲ့ တန်ဖိုးတွေကို ပြောင်ယူမယ်
- Date တွေကို တွက်ချက်မယ်
- Date တွေကို လိုအပ်တဲ့ပုံစံနဲ့ ပြောင်းပြမယ်

Creation Of Date

Date Object ကို ဘယ်လို တည်ဆောက်မလဲ။ ဒါဟာ အရေးကြီးတဲ့ အချက်ဖြစ်ပါတယ်။ ကျွန်တော်တို့ Date Object ကို တည်ဆောက်တဲ့ နေရာမှာ Current Time နဲ့ ယူပြီး ဆောက်ချင်တာရှိသလို၊ အချိန် သတ်မှတ်ပြီး ဆောက်ချင်တဲ့အခါလဲ ရှိပါမယ်။ အရင်ဆုံး Current Time ကို သုံးပြီး ဆောက်တဲ့နည်းကို ကြည့်ပါမယ်။

```
// Current Date
// By Default Constructor
Date date1 = new Date();

// By Calendar
Calendar cal = Calendar.getInstance();
Date date2 = cal.getTime();
```

အထက်ပါအတိုင်း Java Date Object ကို Current Time ကိုသုံးပြီး တည်ဆောက်လိုလျှင် နည်းလမ်းနှစ်မျိုးရှိပါသည်။

အလွယ်ဆုံးကတော့ java.utils.Date ရဲ့ Default Constructor ကို သုံးပြီး Instantiate လုပ်တဲ့နည်းပါပဲ။ လက်ရှိ Runtime ရဲ့ Current Time ကို သုံးပြီး Date Object ကို တည်ဆောက်ပေးပါလိမ့်မယ်။

ဒုတိယနည်းကတော့ Calendar Object ကို သုံးတဲ့ နည်းပါပဲ။ အထက်မှာ ဖေါ်ပြထားတဲ့ အတိုင်း Calendar Object ကို Calendar.getInstance() method နဲ့တည်ဆောက်ပါတယ်။ ပြီးရင် getTime() ကနေ Date Object ကို ရယူနိုင်ပါတယ်။ အဲ့ဒီလို ခေါ်လိုက်ရင် Current Time နဲ့ Date Object ကို ပြန်ရမှာ ဖြစ်ပါတယ်။

Constructors of Date Object

Constructor Summary

Constructors

Constructor and Description
`Date()` Allocates a Date object and initializes it so that it represents the time at which it was allocated, measured to the nearest millisecond.
`Date(int year, int month, int date)` **Deprecated.** As of JDK version 1.1, replaced by Calendar.set(year + 1900, month, date) or GregorianCalendar(year + 1900, month, date).
`Date(int year, int month, int date, int hrs, int min)` **Deprecated.** As of JDK version 1.1, replaced by Calendar.set(year + 1900, month, date, hrs, min) or GregorianCalendar(year + 1900, month, date, hrs, min).
`Date(int year, int month, int date, int hrs, int min, int sec)` **Deprecated.** As of JDK version 1.1, replaced by Calendar.set(year + 1900, month, date, hrs, min, sec) or GregorianCalendar(year + 1900, month, date, hrs, min, sec).
`Date(long date)` Allocates a Date object and initializes it to represent the specified number of milliseconds since the standard base time known as "the epoch", namely January 1, 1970, 00:00:00 GMT.
`Date(String s)` **Deprecated.** As of JDK version 1.1, replaced by DateFormat.parse(String s).

အထက်ဖေါ်ပြပါအတိုင်း java.util.Date မှာ JDK 1.1 ကတည်းက Constructor တွေ အတော်များများပါသော်လည်း၊ လက်ရှိမှာတော့ အတော်များများ Deprecated ဖြစ်သွားပြီး ၂ မျိုးသာ သုံးလို့ရပါတော့မယ်။ တစ်မျိုးကတော့ အထက်မှာ အသုံးပြုခဲ့တဲ့ Current Time ကိုရယူနိုင်တဲ့ Default Constructor ဖြစ်ပါတယ်။

နောက်တစ်မျိုးကတော့ long တန်ဖိုးတစ်ခုကို Parameter အဖြစ်ရယူတဲ့ Constructor ဖြစ်ပါတယ်။ အဲ့ဒီ long Parameter ဟာ standard base time (epoch) ကနေ ကွာဟမှုကို milliseconds နဲ့ တွက်ယူထားတဲ့ တန်ဖိုး ဖြစ်ရပါမယ်။ GMT 1970-01-01 00:00:00 ကနေ တွက်ရမှာ ဖြစ်ပါတယ်။

ဒါ့ကြောင့် အချိန် သတ်မှတ်ပြီး Date Object ကို ဆောက်ချင်တယ်ဆိုရင်တော့ သုံးနိုင်ပါမယ်။ ဒါပေမဲ့ epoch ကနေ အချိန်ဘယ်လောက် ကွာလဲ ဆိုတာကို milliseconds နဲ့ တွက်ပြီးပြဖို့ဆိုတာ လွယ်တဲ့ ကိစ္စတော့ မဟုတ်ပါဘူး။

ဆက်ပြီး အချိန်သတ်မှတ်ပြီး Date Object တွေကို ဘယ်လို တွက်မလဲ ဆိုတာကို ကြည့်ရအောင်။

```
// Target Date
// By Constructor
Date date1 = new Date(1473874200000L);

// By Calendar
Calendar cal = Calendar.getInstance();
cal.set(2018,07,11);
Date date2 = cal.getTime();

// By Formatting String
DateFormat df = new SimpleDateFormat("yyyy/MM/dd");
Date date3 = df.parse("2018/07/11");
```

အချိန်သတ်မှတ်ပြီး Date Object ကို ဆောက်တဲ့နေရာမှာ အထက်ဖော်ပြပါအတိုင်း နည်း ၃ မျိုးနဲ့ ဆောက်လုပ်နိုင်မှာ ဖြစ်ပါတယ်။

ပထမဆုံးနည်းကတော့ Date Constructor မှာ epoch time ကို ပေးတဲ့ နည်းပါပဲ။ ဒါပေမဲ့ဗျာ Epoch Time ကို Milliseconds နဲ့ တွက်ထုတ်ရမယ်ဆိုတာက လက်တွေ့မဆန်ပါဘူး။

ဒုတိယနည်းကတော့ Calendar Object ကို သုံးတဲ့နည်းပါ။ Calendar Object ရဲ့ သက်ဆိုင်ရာ Fields တွေကို လိုသလို Update လုပ်ပြီးမှ Date Object ကို ရယူတဲ့နည်းပါ။ သူလဲပဲ Calendar ကို ဆောက်ပြီးတော့ တန်ဖိုးတွေကို ပြောင်းနေရတဲ့အတွက် Step တွေ များပါတယ်။ ပြီးတော့ လတန်ဖိုးဟာ 0 ကနေ စတာကို သတိထားဖို့လိုပါတယ်။ ၁ လပိုင်းဆိုရင် တန်ဖိုးက 0 ဖြစ်ပြီး ၂ လပိုင်းဆိုရင် တန်ဖိုးက 1 လို့ပေးရမှာ ဖြစ်ပါတယ်။

နောက်ဆုံးနည်းကတော့ DateFormat Object ကို သုံးပြီး parse လုပ်တဲ့နည်းပါ။ DateFromat Object ကို SimpleDateFormat Class ကနေ တည်ဆောက်ပါမယ်။ Constructor Arguments အနေနဲ့ yyyy-MM-dd ကို ပေးထားပြီး အဲ့ဒီ Format နဲ့ ရေးထားတဲ့ String Object ကနေ Date ကို ရယူနိုင်မှာ ဖြစ်ပါတယ်။

Format ချထားတဲ့ String ကို သုံးမှာ ဖြစ်တဲ့အတွက် နားလည်ရလွယ်ကူတဲ့အတွက် အသုံးများတဲ့ နည်းဖြစ်ပါတယ်။ တစ်ခုသတိထားရမှာက parse method ကို သုံးတဲ့နေရာမှာ format မမှန်တဲ့ String ကို ပေးမိရင် ParseException ကို ဖြစ်ပွားစေနိုင်တဲ့ အတွက် အဆိုပါ Exception ကို Handle လုပ်ပေးဖို့လို အပ်ပါတယ်။

DateFormat နဲ့ပတ်သက်ပြီး နောက်ပိုင်းမှာ ဆက်လက်ဖော်ပြသွားပါမယ်။

Interpretation of Calendar Values

Date Object မှာ Calendar Field ရဲ့ တန်ဖိုးများကို ရယူဖို့ Method တွေလဲ ရှိတော့ ရှိပါတယ်။ ဒါပေမဲ့ Internationalization နဲ့ မကိုက်တဲ့အတွက် အသုံးမပြုတော့ရန် Deprecate လုပ်ထားပါတယ်။ Calendar Object ကနေ တဆင့် Interpret လုပ်ယူရမှာ ဖြစ်ပါတယ်။

Calendar Class မှာ Calendar Fields တွေကို Get / Set လုပ်ဖို့အတွက် Method တွေကို ပြင်ထားပါတယ်။

Method Name	Return Type	Arguments
set	void	int field int amount
get	int	int field

Calendar Object ရဲ့ သက်ဆိုင်ရာ Calendar Field တန်ဖိုးတွေကို ပြုပြင်နိုင်ဖို့အတွက် set method ကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။ set method မှာ Parameter ၂ ခုကို ရယူနေပြီး ပထမ Parameter ဟာ Calendar Field ကို ကိုယ်စားပြုပြီး၊ ဒုတိယ Parameter ဟာ အဲ့ဒီ Field ရဲ့ တန်ဖိုးကို ကိုယ်စားပြုပါတယ်။

Calendar Class မှာ Calendar Field တွေကို ကိုယ်စားပြုနိုင်ဖို့ Static Variable များကို ပြင်ဆင်ထားပြီး အဲ့ဒီ Fields တွေကို အသုံးပြုနိုင်ပါတယ်။

တဖန် get method မှာလဲ int parameter တစ်ခုကို ရယူနေပြီး မိမိက ရယူလိုသော Calendar Field ကို ပေးရမှာ ဖြစ်ပါတယ်။ အဲ့ဒီနေရာမှာလဲ Calendar ရဲ့ Static Variable များကို အသုံးပြုနိုင်ပါတယ်။

အသုံးပြုနိုင်တဲ့ Static Variable တွေကတော့ အောက်ပါအတိုင်းဖြစ်ပါတယ်။

Static Variable	Definition
YEAR	နှစ်ကို ကိုယ်စားပြုပါတယ်
MONTH	လ ကို ကိုယ်စားပြုပါတယ်
DAY_OF_MONTH	လတစ်လရဲ့ ရက်ကို ကိုယ်စားပြုပါတယ်
DAY_OF_WEEK	အပါတ်ရဲ့ ရက်ကို ကိုယ်စားပြုပါတယ်
DAY_OF_WEEK_IN_MONTH	လရဲ့ အပါတ် အတွင်းမှာရှိတဲ့ ရက်ကို ကိုယ်စားပြုပါတယ်
DAY_OF_YEAR	နှစ်တစ်နှစ်အတွင်းမှာရှိတဲ့ ရက်ကို ကိုယ်စားပြုပါတယ်
WEEK_OF_YEAR	နှစ် တစ်နှစ်အတွင်းမှာရှိတဲ့ အပါတ်ကို ကိုယ်စားပြုပါတယ်
WEEK_OF_MONTH	လတစ်လအတွင်းမှာရှိတဲ့ အပါတ်ကို ကိုယ်စားပြုပါတယ်
HOUR_OF_DAY	နေ့တစ်နေ့ရဲ့ နာရီကို ကိုယ်စားပြုပါတယ်
HOUR	နာရီကို ကိုယ်စားပြုပါတယ်
MINUTE	မိနစ်ကို ကိုယ်စားပြုပါတယ်
SECOND	စက္ကန့်ကို ကိုယ်စားပြုပါတယ်
MILLISECOND	မီလီ စက္ကန့်ကို ကိုယ်စားပြုပါတယ်

Setting Calendar Values

```
        public static void main(String[] args) {

            // set with fields
            Calendar c1 = Calendar.getInstance();
            c1.set(Calendar.YEAR, 2018);
            c1.set(Calendar.MONTH, 7);
            c1.set(Calendar.DAY_OF_MONTH, 20);

            // set fields
            Calendar c2 = Calendar.getInstance();
            c1.set(2018, 7, 20);
        }
```

အထက်ပါ အတိုင်း သက်ဆိုင်ရာ Calendar Field များရဲ့ တန်ဖိုးကို တစ်ခုချင်းဆီသတ်မှတ်ပေးနိုင်ပါတယ်။ တဖန် အောက်ပါ အတိုင်း Calender Field အစုလိုက် တန်ဖိုးကို သတ်မှတ်ပေးနိုင်တဲ့ Method တွေလဲ ရှိပါတယ်။

void	**set**(int field, int value) Sets the given calendar field to the given value.
void	**set**(int year, int month, int date) Sets the values for the calendar fields YEAR, MONTH, and DAY_OF_MONTH.
void	**set**(int year, int month, int date, int hourOfDay, int minute) Sets the values for the calendar fields YEAR, MONTH, DAY_OF_MONTH, HOUR_OF_DAY, and MINUTE.
void	**set**(int year, int month, int date, int hourOfDay, int minute, int second) Sets the values for the fields YEAR, MONTH, DAY_OF_MONTH, HOUR, MINUTE, and SECOND.

Calendar Object ရဲ့ နှစ်၊ လ၊ ရက်တို့ကို သတ်မှတ်လိုပါက int parameter ၃ ခုပါတဲ့ Method ကို သုံးမယ်ဆိုလဲရပါတယ်။ သတိပြုရမှာက လတန်ဖိုးဟာ သုညကနေ စတယ်ဆိုတာပါပဲ။ int တန်ဖိုးကို တိုက်ရိုက်မပေးပဲ Calendar Class ရဲ့ Month အတွက် ရေးသားထားတဲ့ Static Variable တွေကို သုံးမယ်ဆိုလဲ ရပါတယ်။ Calendar Class မှာ တစ်နှစ်တာ အတွက် လတွေကို Static Variable တွေ အနေနဲ့ ကြိုတင် ရေးသားပေးထားပါတယ်။

Getting Calendar Value

```
        public static void main(String[] args) {

            Calendar c = Calendar.getInstance();
            int year = c.get(Calendar.YEAR);
            int month = c.get(Calendar.MONTH);
            int day = c.get(Calendar.DAY_OF_MONTH);
        }
```

Leniency of Calendar

Calendar Object ရဲ့ တန်ဖိုးတွေကို တွက်ချက်ရာတွင် Leniency Mode ကို ကြည့်ပြီးတွက်ချက်လေ့ရှိပါတယ်။ Calendar Object ရဲ့ isLenient method နဲ့ Calendar Class ရဲ့ Leniency Mode ကို သိရှိနိုင်ပြီး setLenient method ဖြင့်လဲ Leniency Mode ကို သတ်မှတ်ပေးနိုင်ပါတယ်။

Calendar Object တစ်ခုရဲ့ Default Leniency တန်ဖိုးဟာ true ဖြစ်ပါတယ်။

Leniency တန်ဖိုးဟာ true ဖြစ်နေရင် Calendar Field ရဲ့တန်ဖိုးက သတ်မှတ်ထားတဲ့ တန်ဖိုးထက်ကျော်လွန်သွားပါက လက်ရှိတန်ဖိုးထက်ကြီးတဲ့ Field တွေကို ပါဆက်ပြီး ပြောင်းလဲပေးမှာ ဖြစ်ပါတယ်။

ဆိုလိုတာက လက်ရှိ Calendar ရဲ့တန်ဖိုးက 2016/09/15 ဖြစ်တယ်ဆိုပါစို့။ c.set(Calendar.DAY_OF_MONT, 45) လို့ရေးလိုက်ရင် တန်ဖိုးဟာ 2016/10/30 လို့ပြောင်းသွားပါမယ်။

```
        public static void main(String[] args) {

            Calendar c = Calendar.getInstance();

            System.out.println(c.isLenient());

            c.set(2016, Calendar.SEPTEMBER, 15);

            c.set(Calendar.DAY_OF_MONTH, 45);

            System.out.println(c.getTime());
        }
```

အထက်ပါ Program ကို Run ကြည့်မယ်ဆိုရင် အောက်ပါ အတိုင်းတွေ့ရပါလိမ့်မယ်။

```
    true
    Sat Oct 15 13:14:36 MMT 2016
```

Calendar Object c ရဲ့ lenient တန်ဖိုးဟာ သတ်မှတ်ထားခြင်းမရှိတဲ့ အတွက် Default အတိုင်း true ဖြစ်နေတာကို တွေ့ရပါတယ်။ အဲဒီအတွက် DAY_OF_MONTH ကို တစ်လရှိတာထက် ပိုပြီး 45 လို့ သတ်မှတ်လိုက်တဲ့အခါ လက်ရှိလကို ကျော်ပြီး ၁၀ လပိုင်း ၁၅ ရက်လို့ သတ်မှတ်ပေးသွားတာ ဖြစ်ပါတယ်။

ကျွန်တော်တို့ Leniency ကို False ဆိုပြီး ပြောင်းကြည့်မယ်ဆိုရင် Calendar Object ရဲ့ Leniency တန်ဖိုးဟာ Non-Lenient Mode ကို ပြောင်းသွားပါမယ်။ ဘယ်လို ဖြစ်မယ်ဆိုတာကို လက်တွေ့ ရေးကြည့်ကြရအောင်။

```
        public static void main(String[] args) {

            Calendar c = Calendar.getInstance();

            c.setLenient(false);
            System.out.println(c.isLenient());

            c.set(2016, Calendar.SEPTEMBER, 15);

            c.set(Calendar.DAY_OF_MONTH, 45);

            System.out.println(c.getTime());
        }
```

အထက်ပါအတိုင်း c.setLenient(false) ဆိုပြီး ပြောင်းလိုက်ပါတယ်။ Lenient Mode ဟာ Non-Lenient Mode ကို ပြောင်းသွားပါမယ်။ ဒီလိုဆိုရင် DAY_OF_MONTH Field မှာ 45 လို့ သတ်မှတ်လို့ ရမှာ မဟုတ်တော့ပါဘူး။ ၂၀၁၆ ခု ၁၀ ပိုင်းမှာ ရက် ၃၀ သာရှိတဲ့ အတွက် IllegalArgumentException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

```
false
Exception in thread "main" java.lang.IllegalArgumentException:
DAY_OF_MONTH
at java.util.GregorianCalendar.computeTime(GregorianCalendar.java:2648)
at java.util.Calendar.updateTime(Calendar.java:3393)
at java.util.Calendar.getTimeInMillis(Calendar.java:1782)
at java.util.Calendar.getTime(Calendar.java:1755)
at com.jdc.dummy.GenericsDemo.main(GenericsDemo.java:18)
```

Leniency ရဲ့သဘောတရားဟာ Calendar Field တွေကို Set လုပ်ရာမှာသာမက ရက်စွဲ အချိန်တွေကို တွက်ချက်တဲ့နေရာမှာလဲ အသုံးပြုပါတယ်။ ဆက်လက်ပြီး အချိန်တွေကို ဘယ်လို တွက်ချက်တယ်ဆိုတာကို ကြည့်ကြည့်ရအောင်။

Calculating Date & Time

အချိန်တွေကို အသုံးပြုရာမှာ သူ့ရဲ့တန်ဖိုးတွေကို တွက်ချက်ဖို့လဲ လိုပါမယ်။ Calendar Field တွေကို ပေါင်းဖို့ အတွက် Method နှစ်ခုရှိပါတယ်။

Method	Arguments	Description
add	int field int amount	အချိန်ရက်နှစ်လတို့အား အပေါင်းအနှုတ် ပြုလုပ်လိုပါက အသုံးပြုနိုင်
roll	int field int amount	လက်ရှိ Field ထက်ကြီးသော Field အား မပြောင်းလည်းစေပဲ အပေါင်း အနှုတ် ပြုလုပ်

အထက်ပါ နမူနာထဲမှာ Calendar Object c1 ရဲ့ တန်ဖိုးကို 2016/09/15 လို့သတ်မှတ်ပါတယ်။ ပြီးရင် ရက် ၃၀ ကို add method ကို သုံးပြီး ပေါင်းစေပါတယ်။ Add Method ကို သုံးပြီးပေါင်းတာ ဖြစ်တဲ့အတွက် ရက်ကို ပေါင်းပြီး လကို ပါ ဆက်ပြီးတိုးပေးနိုင်မှာ ဖြစ်ပါတယ်။ အဲဒီ အတွက် ရလဒ်ဟာ 2016/10/15 ဖြစ်သွားပါမယ်။

```
        public static void main(String[] args) throws ParseException {

            DateFormat df = new SimpleDateFormat("yyyyMMdd");
            Date d = df.parse("20160915");

            Calendar c = Calendar.getInstance();
            c.setTime(d);

            c.add(Calendar.DAY_OF_MONTH, 30);
            System.out.println(c.getTime());

            c.setTime(d);
            c.roll(Calendar.DAY_OF_MONTH, 30);
            System.out.println(c.getTime());
        }
```

တဖန် roll ကို သုံးပြီး ပေါင်းရင်တော့ လက်ရှိ Field ရဲ့ တန်ဖိုးအပြင်ကို ပြောင်းလဲ ပေးမှာ မဟုတ်ပါဘူး။ အဲဒီ အတွက် ရလဒ်ဟာ 2016/09/15 ပဲ ဖြစ်နေမှာ ဖြစ်ပါတယ်။ Run ကြည့်ထားတဲ့ ရလဒ်ကတော့ အောက်ပါ အတိုင်း ဖြစ်ပါတယ်။

```
Sat Oct 15 00:00:00 MMT 2016
Thu Sep 15 00:00:00 MMT 2016
```

Formatting Date

Date Object ကို View မှာ ပြန်ပြီး ပြသလိုတဲ့အခါမှာ User က နားလည်လွယ်တဲ့ ပုံစံနဲ့ ပြန်ပြီး ပြဖို့လိုအပ်ပါတယ်။ တဖန် Localization အရ မည်သို့ဖေါ်ပြရမယ်ဆိုတဲ့ သတ်မှတ်ချက်တွေလဲရှိတတ်ပါတယ်။ User Input ကတော့ String တွေဖြစ်နေရင်လဲ String ကနေ Date Object ကို ပြောင်းဖို့လိုအပ်ပါမယ်။ အဲ့လိုနေရာမျိုးမှာ Date တွေကို ဘယ်လို Format နဲ့ပြမယ်ဆိုတာကို ရေးသားတတ်ဖို့လိုအပ်ပါတယ်။Date Object အား String အဖြစ် Format လုပ်လိုသည့်အခါတွင်၎င်း၊ Format ချထားသော String မှတဆင့် Date Object အား Parse လုပ်လိုသည့်အခါတွင်၎င်း အသုံးပြုနိုင်ပါတယ်။ DateFormat သည် Abstract Class ဖြစ်ပြီး၊ Concrete Class မှာ SimpleDateFormat Class ဖြစ်ပါတယ်။

Date တွေကို Format လုပ်လိုသည့် အခါ format(date date) Method ကို အသုံးပြုနိုင်ပြီး သတ်မှတ်ထားတဲ့ Format နဲ့ Format လုပ်ထားတဲ့ String Object ကို ပြန်ရမှာ ဖြစ်ပါတယ်။ Format အတိုင်းရေးထားတဲ့ String Object ကနေ Date Object ကို Parse လုပ်လိုတဲ့ အခါ parse(String source) method ကို အသုံးပြုနိုင်ပါတယ်။ String တန်ဖိုးနဲ့ တည်ဆောက်ထားတဲ့ Date Object ကို ရရှိမှာ ဖြစ်ပါတယ်။ parse method ကို အသုံးပြုတဲ့ အခါမှာ ParseException ကို ဖြစ်ပေါ်စေနိုင်တဲ့အတွက် Handle လုပ်ပေးဖို့ လိုအပ်ပါတယ်။

Date And Time Patterns

Letter	Date or Time Component	Presentation	Examples
G	Era designator	Text	AD
y	Year	Year	1996; 96
Y	Week year	Year	2009; 09
M	Month in year	Month	July; Jul; 07
w	Week in year	Number	27
W	Week in month	Number	2
D	Day in year	Number	189
d	Day in month	Number	10
F	Day of week in month	Number	2
E	Day name in week	Text	Tuesday; Tue
u	Day number of week	Number	1
a	Am/pm marker	Text	PM
H	Hour in day (0-23)	Number	0
k	Hour in day (1-24)	Number	24
K	Hour in am/pm (0-11)	Number	0
h	Hour in am/pm (1-12)	Number	12
m	Minute in hour	Number	30
s	Second in minute	Number	55
S	Millisecond	Number	978
z	Time zone	General time zone	Pacific Standard Time; PST; GMT-08:00
Z	Time zone	RFC 822 time zone	-0800
X	Time zone	ISO 8601 time zone	-08; -0800; -08:00

Examples

Date and Time Pattern	Result
"yyyy.MM.dd G 'at' HH:mm:ss z"	2001.07.04 AD at 12:08:56 PDT
"EEE, MMM d, ''yy"	Wed, Jul 4, '01
"h:mm a"	12:08 PM
"hh 'o''clock' a, zzzz"	12 o'clock PM, Pacific Daylight Time
"K:mm a, z"	0:08 PM, PDT
"yyyyy.MMMMM.dd GGG hh:mm aaa"	02001.July.04 AD 12:08 PM
"EEE, d MMM yyyy HH:mm:ss Z"	Wed, 4 Jul 2001 12:08:56 -0700
"yyMMddHHmmssZ"	010704120856-0700
"yyyy-MM-dd'T'HH:mm:ss.SSSZ"	2001-07-04T12:08:56.235-0700
"yyyy-MM-dd'T'HH:mm:ss.SSSXXX"	2001-07-04T12:08:56.235-07:00
"YYYY-'W'ww-u"	2001-W27-3

Date Object ကို Format ချထားတဲ့ String အဖြစ်ပြောင်းလိုရင် သတ်မှတ်လိုတဲ့ Format နဲ့ DateFormat ကို အရင်ဆောက်ရပါမယ်။ ပြီးပါမှ Format လုပ်လိုတဲ့ Date Object ကို သုံးပြီး format ပြောင်းရုံဖြစ်ပါမယ်။

```
        public static void main(String[] args) throws ParseException {

            DateFormat df = new SimpleDateFormat("yyyy-MM-dd");

            // Formatting Date
            String formatedDate = df.format(new Date());
            System.out.println(formatedDate);

            // Parsing Date
            Date date = df.parse(formatedDate);
            System.out.println(date);

        }
```

တဖန် Format လုပ်ထားတဲ့ String ကနေ Date Object ကို ပြန်လိုချင်ရင် parse method ကို အသုံးပြုနိုင်ပါတယ်။

အမှန်အတိုင်းပြောရမယ်ဆိုရင် Date တစ်ခုနဲ့ တစ်ခုဘယ်လိုကွာတယ်ဆိုတာ ကို တွက်ချက်တတ်ဖို့လဲလိုပါလိမ့်မယ်။ လက်ရှိ Date နဲ့ Calendar မှာ အဲ့ဒါတွေကို တွက်ချက်ဖို့ Method တွေ အသင့်ပေးမထားတာဘူး။ အဲ့ဒီအတွက် Date Object တွေရဲ့ Time ကို Millisecond နဲ့ ပြောင်းယူပြီး ကွာခြားချက်ကို ရှာ၊ ပြီးရင် နာရီ၊ ရက်တွေကို ပြန်ပြီးတွက်ထုတ်ရမှာ ဖြစ်ပါတယ်။

Java SE 8 မတိုင်မီကတော့ Third Party API တွေဖြစ်တဲ့ Joda Time API တို့ကို အသုံးပြုခဲ့ကြရပါတယ်။ Java SE 8 ရောက်တဲ့ အခါမှာတော့ Date & Time API အသစ်ကို ဖြည့်စွက်လာခဲ့ပြီး Duration တွေကို တွက်ချက်နိုင်လာခဲ့ပါတယ်။

Date & Times API



Java SE 8 အရောက်မှာ အသစ်ဖြည့်စွက်လာခဲ့တဲ့ နေ့ရက် အချိန်တွေကို အသုံးပြုနိုင်တဲ့ API တစ်မျိုးဖြစ်ပါတယ်။ တကယ်ဆိုရင် Java မှာ နေ့ရက်အချိန်တွေကို အသုံးပြုဖို့အတွက် Date နဲ့ Calendar တို့ကို အသုံးပြုခဲ့ကြပါတယ်။ ဒါပေမဲ့ Java SE 8 အရောက်မှာ API အသစ်တစ်ခုကို ဖြည့်စွက်လာခဲ့ကြပါတယ်။

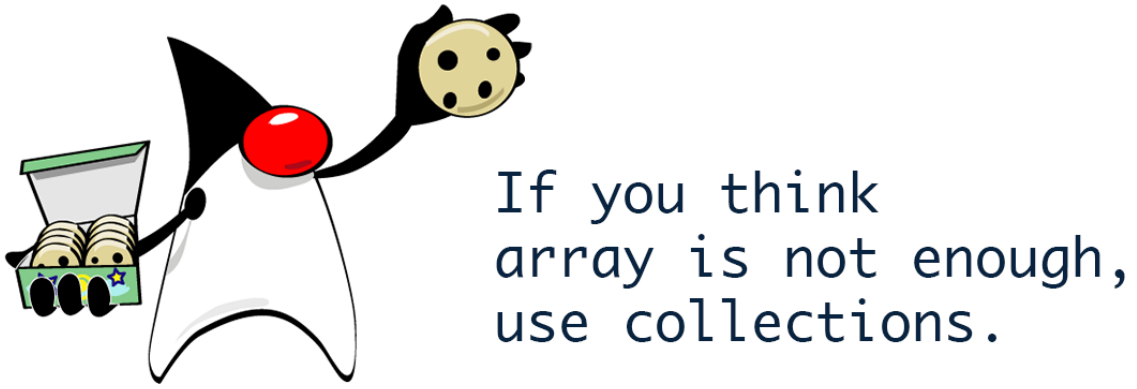
ဒီအခန်းမှာတော့ ဘာလို့ Date တွေ Time တွေကို အသုံးပြုဖို့ API အသစ်တစ်ခုလိုအပ်ခဲ့ရတာလဲ။ ပြီးတော့ API အသစ်မှာ Date Time တွေကို အသုံးပြုဖို့ ဘယ်လို Class တွေ Interface တွေကို အသုံးပြုရမလဲ။ ဘယ်လို နေရာမျိုးတွေမှာ ဒီ API နဲ့ ဘယ်လို အသုံးပြုနိုင်သလဲ ဆိုတာတွေကို လေ့လာသွားကြပါမယ်။

Why Java need new Date API?

Types of Dates

Usages of Date and Times

Collections Framework

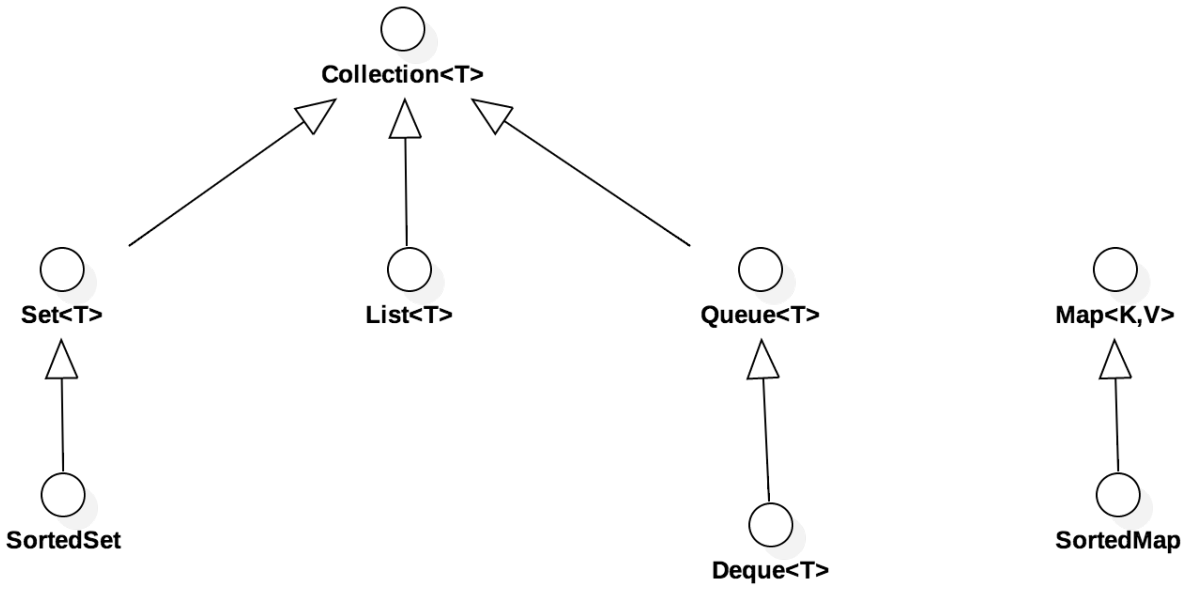


ကျွန်တော်တို့ Java Program တွေကို ရေးသားတဲ့ နေရာမှာ Data တွေကို တစုတစည်းထဲ အသုံးပြုလိုတဲ့ အခါမှာ Array တွေကို အသုံးပြုခဲ့ကြပါတယ်။ Array တွေဟာ အသုံးပြုမည့် Data Type အလိုက် ထည့်သွင်းထားတဲ့ အစီအစဉ်ကို ထိမ်းပေးထားနိုင်တဲ့ Object တစ်မျိုးဖြစ်ပါတယ်။

ဒါပေမဲ့ Array တွေကို ရေးသားအသုံးပြုကြတဲ့ နေရာမှာ Array Object ကို တည်ဆောက်တော့မယ်ဆိုရင် အသုံးပြုမည့် အရေအတွက်ကို အရင်ဆုံးသတ်မှတ်ပေးခဲ့ရပါတယ်။ နောက်ပြီး Array Object တွေဟာ တစ်ခါတည်ဆောက်ပြီးပြီဆိုရင် အသုံးပြုလိုတဲ့ အခန်းအရေအတွက်ကို ထပ်ပြီးတိုးလို့မရတော့ပါဘူး။ နောက်ပြီး Array တွေဟာ သိမ်းထားတဲ့ Data တွေရဲ့ Index အတိုင်းသာ Data တွေကို Access လုပ်လို့ရမှာဖြစ်ပါတယ်။ အခြားသော Data Structure တွေကို Support မလုပ်ပေးနိုင်ပါဘူး။

ဒါ့ကြောင့် လက်တွေ့ Java Application တွေကို ရေးသားတဲ့ နေရာမှာ Array တွေဟာ သုံးရခက်ခဲတဲ့ အတွက် Arrays တွေအစား Collections တွေကို အသုံးပြုကြတာများပါတယ်။ ကျွန်တော်တို့ ဒီအခန်းမှာတော့ Collections တွေကို ပံ့ပိုးပေးတဲ့ Collections Framework အကြောင်းကို လေ့လာသွားကြပါမယ်။

Collection Interface



Collection တွေကို ကိုယ်စားပြုဖို့အတွက် Interface တွေကိုပြင်ဆင်ပေးထားပါတယ်။ Collection တွေအားလုံးရဲ့ Base Interface မှာ Collection Interface ဖြစ်ပြီး Generics Type Interface အမျိုးအစားဖြစ်ပါတယ်။ တဖန် အသုံးပြုလိုတဲ့ Data Structure အလိုက် Sub Interface တွေကို ပြင်ဆင်ပေးထားပါတယ်။ အသုံးပြုနိုင်တဲ့ Interface တွေကတော့ Set, List နဲ့ Queue တို့ပဲဖြစ်ကြပါတယ်။

Map တွေဟာ Key Value အနေနဲ့ တွဲပြီး Data တွေကို သိမ်းပေးထားနိုင်တဲ့ Collection Interface ရဲ့ Sub Interface တော့မဟုတ်ပါဘူး။ ဒါပေမဲ့ Map တွေမှာပါတဲ့ Key, Value အတွဲကို Entry ဆိုတဲ့ Object နဲ့ သိမ်းပေးထားပြီး၊ အဲ့ဒီ Entry တွေကို Set အနေနဲ့ သိမ်းပေးထားပါတယ်။

Methods of Collection Interface

Method	Return Type	Description
size()	int	Collection ထဲမှာပါတဲ့ Element တွေရဲ့ Size ကို သိရှိနိုင်
isEmpty()	boolean	Element တွေရှိမရှိကို သိရှိနိုင်
contains(Object)	boolean	Collection ထဲမှာ Argument ထဲက Element ရှိမရှိကို သိရှိနိုင်
iterator()	Iterator<E>	Iterator Object ကိုပြန်လည်ရရှိနိုင်
toArray()	Object []	Element တွေကို Object [] ပုံစံနဲ့ ပြန်လည်ရရှိနိုင်
toArray(T[])<T>	T []	Element တွေကို Array ပုံစံနဲ့ ပြန်လည်ရရှိနိုင်
containsAll(Collection<?>)	boolean	အခြား Collection ထဲမှာပါတဲ့ Element တွေအားလုံး လက်ရှိ Collection ထဲမှာပါမပါ သိရှိနိုင်ပါတယ်
retainAll(Collection<?>)	boolean	လက်ရှိ Collection ထဲမှာလဲပါပြီး အခြား Collection ထဲမှာပါတဲ့ Element တွေနဲ့ လက်ရှိ Collection ကို ပြန်လည်တည်ဆောက်လိုတဲ့ အခါမှာအသုံးပြုနိုင်ပါတယ်
add(E)	boolean	Parameter Element ကို Collection ထဲကို ဖြည့်စွက်နိုင်
remove(Object)	boolean	Parameter Element ကို Collection ထဲကနေ ထုတ်ပစ်နိုင်
addAll(Collection<? extends E>)	boolean	Collection ထဲကို အခြား Collection တွေကို ဖြည့်စွက်နိုင်
removeAll(Collection<?>)	boolean	အခြား Collection ထဲမှာပါ Element တွေအားလုံးကို ထုတ်ပစ်နိုင်
removeIf(Predicate<? super E>)	boolean	Predicate ထဲမှာပါတဲ့ Condition နဲ့ တူတဲ့ Element တွေကို ဖျက်ထုတ်ပစ်နိုင်
clear()	boolean	Element တွေအားလုံးကို ဖျက်ထုတ်ပစ်နိုင်
spliterator()	Spliterator<E>	Spliterator Object ကို ပြန်လည်ရရှိနိုင်
stream()	Stream<E>	Sequential Stream Object ကို ပြန်လည်ရရှိနိုင်
parallelStream()	Stream<E>	Parallel Stream Object ကို ပြန်လည်ရရှိနိုင်

အထက်ပါ Method တွေကတော့ Collection Interface ထဲမှာပါတဲ့ Method တွေဖြစ်ကြပြီး၊ Collection ရဲ့ Sub Interface တွေဖြစ်ကြတဲ့ Set, List နဲ့ Queue တို့မှာလဲ အထက်ပါ Method တွေကို အသုံးပြုနိုင်ပါတယ်။ Set, List နဲ့ Queue တို့ဟာ Data Structure တွေကို ကိုယ်စားပြုရေးသားထားတဲ့ Interface တွေဖြစ်ကြတဲ့ အတွက် Collection Interface ထဲမှာပါတဲ့ အခြေခံ Metdhod တွေအပြင် Data Structure အလိုက် အသုံးပြုလိုတဲ့ Method တွေကို ထပ်မံဖြည့်စွက်ထားကြပါတယ်။

Set Interface

Set Interface ဟာ Collection Interface ရဲ့ Sub Interface တစ်ခုဖြစ်ပြီး အထက်တန်း သခ်ၤာမှာ ပါတဲ့ အစု (Set) ဆိုတဲ့ Data Structure ကို အခြေခံပြီးရေးသားထားပါတယ်။ အစု ဆိုတဲ့ အတိုင်း Set တစ်ခုထဲမှာ Element တွေဟာ Duplicate ဖြစ်တာကို လက်မခံပါဘူး။ ပြီးတော့ ပါဝင်တဲ့ Element တွေရဲ့ ဘယ် Element ပြီးရင် ဘယ် Element လာမယ် ဆိုတဲ့ သိမ်းထားတဲ့ အစီအစဉ်ကိုလဲ မှတ်မထားပေးနိုင်ပါဘူး။ ဒါ့ကြောင့် Set တွေမှာ Index Number ကို အသုံးပြုတဲ့ Method တွေလဲ မပါသလို ရှေ့ဆုံး နောက်ဆုံးအစရှိတဲ့ Element တွေကို ခေါ်ယူတဲ့ Method တွေလဲ ပါမလာပါဘူး။

Set တွေမှာတော့ Collection Interface ထဲမှာပါတဲ့ Method တွေကိုသာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ Set ဟာ Interface တစ်ခုဖြစ်တဲ့ အတွက် ဒီအတိုင်းတော့ Object တစ်ခုကို တည်ဆောက်လို့မရနိုင်ပါဘူး။ Set Interface ကို Implement လုပ်ထားတဲ့ Class တွေကနေ တဆင့်သာ တည်ဆောက်လို့ရပါမယ်။ Collections Framework ထဲမှာ Set Object တွေကို အလွယ်တကူတည်ဆောက်နိုင်ဖို့အတွက် General Purpose Implementation Class တွေကို ရေးသားပေးထားပါတယ်။

Class	Data Structure	ထူးခြားချက်များ
HashSet	Hash Table	Performance အကောင်းဆုံး ပုံစံဖြစ်ပါတယ်
TreeSet	Balance Tree	Natural Ordering အလိုက် Sort လုပ်ပေးထားနိုင်ပါတယ်။ ဒါပေမဲ့ Compare လုပ်လို့မရတဲ့ Object တွေနဲ့တော့ အသုံးပြုလို့ မရနိုင်ပါဘူး
LinkedHashSet	Link List & Hash Tablee	Link List Data Structure ကို အသုံးပြုထားတဲ့ အတွက် ထည့်သွင်းထားတဲ့ အစီအစဉ်ကို သိမ်းပေးထားနိုင်ပါတယ်

အရင်ဆုံး General Purpose Implementation Class တွေရဲ့ မတူညီမှုတွေကို ကုဒ်လေးတွေနဲ့ လေ့လာကြည့်ကြရအောင်။

```
    public static void main(String[] args) {

        Set<Integer> hash = new HashSet<>();
        Set<Integer> tree = new TreeSet<>();
        Set<Integer> link = new LinkedHashSet<>();

        for(int i = 10; i > 0; i --) {
            hash.add(i);
            tree.add(i);
            link.add(i);
        }

        System.out.println(hash);
        System.out.println(tree);
        System.out.println(link);
    }
```

အထက်ပါနမူနာထဲမှာ Set Object တွေကို အသီးသီး သက်ဆိုင်ရာ Implementation Class တွေနဲ့ တည်ဆောက်ထားပါတယ်။ ပြီးတော့ for Loop ကို 10 ကနေ 0 အထိ Desending Order နဲ့ Loop ပတ်စေပြီး Set Object တွေ

ထဲကို တန်ဖိုးတွေ ဖြည့်စွက်ထားပါတယ်။ ပြီးတဲ့နောက်မှာ Set Object အားလုံးကို Console ပေါ်မှာ ဖော်ပြစေပါတယ်။

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

ပထမစာကြောင်းက HashSet Object ကို ပြထားတာဖြစ်ပါတယ်။ ကျွန်တော်တို့ ထည့်သွင်းခဲ့တာက ၁၀ ကနေ သုညအထိ ပြောင်းပြန်အစီအစဉ်နဲ့ ထည့်သွင်းခဲ့တာဖြစ်ပါတယ်။ ဒါပေမဲ့ ရလဒ်အရကြည့်ရင် 1 ကနေ 10 အထိ စီထားသလိုရှိပါတယ်။ ဒါက Value နဲ့ စီထားတာ မဟုတ်ပါဘူး။ ပြီးတော့ HashMap တွေဟာ Ordering တွေ အစီအစဉ်တွေကို အာမခံမပေးထားနိုင်တဲ့ အတွက် ဒီလိုရလဒ်မျိုးထွက်ပေမဲ့ အသုံးမပြုသင့်ပါဘူး။

ဒုတိယ စာကြောင်းကတော့ TreeMap ကို အသုံးပြုထားတာပါ။ ဒီနေရာမှာလဲ Integer တွေရဲ့ Natural Ordering အတိုင်း ငယ်စဉ်ကြီးလိုက်ပြန်ပြီး စီပေးထားနိုင်တာကို တွေ့ရပါတယ်။ Sort လုပ်တယ်ဆိုတာက Element တစ်ခုနဲ့ တစ်ခုအကြား ငယ်သလားကြီးသလားကို နှိုင်းယှဉ်နိုင်မှသာ Sort လုပ်လို့ရမှာ ဖြစ်ပါတယ်။ Compare လုပ်လို့ မရတဲ့ Object တွေကို TreeSet ထဲမှာ ထည့်သုံးရင် ဘယ်လို ဖြစ်မယ်ဆိုတာကို နောက်မှ လေ့လာသွားကြပါမယ်။ ဒါကြောင့် သဘာဝအတိုင်း ငယ်စဉ်ကြီးလိုက် Sort လုပ်ပြီးသား Data တွေကို ရယူချင်တယ်ဆိုရင်တော့ TreeSet Object ကို အသုံးပြုသင့်ပါတယ်။ ဒါပေမဲ့ Set ထဲမှာပါတဲ့ Element တန်ဖိုးတွေပြောင်းတိုင်း Sort လုပ်နေရတဲ့ အတွက် Element တန်ဖိုးတွေကို ခဏခဏပြောင်းဖို့လိုတဲ့ နေရာတွေမှာ TreeSet ကို အသုံးပြုရင် Performance ပိုင်းဆိုင်ရာမှာ အားနဲသွားတတ်ပါတယ်။

တတိယ စာကြောင်းကတော့ LinkedHashMap ကို အသုံးပြုထားတာ ဖြစ်ပါတယ်။ ဒါကြောင့် ရလဒ်ဟာ Add လုပ်ထားတဲ့ အစီအစဉ်အတိုင်းပြန်ပြနိုင်တာပဲ ဖြစ်ပါတယ်။

ကျွန်တော်တို့ ဆက်လက်ပြီး Compare လုပ်လို့ မရတဲ့ Object တွေကို TreeSet တွေထဲမှာ ရေးသားအသုံးပြုရင် ဘယ်လိုဖြစ်မယ်ဆိုတာကို လေ့လာသွားကြရအောင်။ အောက်ပါ User Class ဟာ ကျွန်တော်တို့ TreeSet ထဲမှာ ရေးသားအသုံးပြုသွားမည့် Type ဖြစ်ပါတယ်။

```
public class User {

	private String name;

	public User(String name) {
		this.name = name;
	}

	@Override
	public String toString() {
		return name;
	}

	// getter & setter
}
```

User Class ကို ဒီအတိုင်းရေးထားမယ်ဆိုရင် Object တွေအကြားမှာ ဘယ်သူကြီးပြီး ဘယ်သူငယ်တယ်ဆိုတာကို နှိုင်းယှဉ်လို့ မရနိုင်ပါဘူး။ Compare လုပ်လို့ မရဘူးဆိုကထဲက Sort လုပ်လို့မရနိုင်ပါဘူး။ ဒီလို Compare လုပ်လို့ မရတဲ့ Object တွေကို TreeSet ထဲကို ထည့်ရင် ဘယ်လိုဖြစ်မယ်ဆိုတာကို လေ့လာကြည့်ကြရအောင်။

```
	public static void main(String[] args) {

		Set<User> set = new TreeSet<>();
		set.add(new User("Thidar"));
		set.add(new User("Aung Aung"));

		System.out.println(set);
	}
```

အထက်ပါအတိုင်း Main Method ထဲမှာ User Type ကို အသုံးပြုနိုင်တဲ့ TreeSet Object တစ်ခုကို တည်ဆောက်ပြီး User Object တွေကို ထည့်သွင်းကြည့်ပါတယ်။ အထက်ပါ Code တွေဟာ Compile Time မှာ လုံးဝ Error မဖြစ်ပါဘူး။ Compile လုပ်လို့ရပါတယ်။ ဒါပေမဲ့ Run ကြည့်တဲ့ အခါမှာတော့ အောက်ပါအတိုင်း RuntimeException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

```
Exception in thread "main" java.lang.ClassCastException:
com.jdc.test.FormatDemo$User cannot be cast to java.lang.Comparable
at java.util.TreeMap.compare(TreeMap.java:1294)
at java.util.TreeMap.put(TreeMap.java:538)
at java.util.TreeSet.add(TreeSet.java:255)
at com.jdc.test.FormatDemo.main(FormatDemo.java:25)
```

TreeSet တွေဟာ Element တွေကို Sort လုပ်နိုင်ဖို့အတွက် နောက်ကွယ်မှာ TreeMap တစ်ခုကို အသုံးပြုနေပါတယ်။ အဲ့ဒါကြောင့် TreeSet ရဲ့ add() Method ထဲမှာ TreeMap ရဲ့ put() Method ကို ဆက်ပြီးခေါ်ပါတယ်။ TreeMap ရဲ့ put() method ကနေ compare() Method ကို ဆက်ခေါ်ပြီး အဲ့ဒီ Method ထဲမှာ User Object ကို Comparable Type အဖြစ်ပြောင်းလိုတဲ့ အတွက် Cast လုပ်တဲ့နေရာမှာ User Class ဟာ Comparable Interface ကို Implement လုပ်မထားတဲ့ အတွက် ClassCastException ကို ဖြစ်ပေါ်စေတာပဲ ဖြစ်ပါတယ်။

ဒါ့ကြောင့် ကျွန်တော်တို့ TreeSet မှာ အသုံးပြုလိုတဲ့ Data Type တွေဟာ Comparable Interface ကို Implement လုပ်ထားဖို့လိုအပ်တယ်ဆိုတာကို သိနေဖို့လိုအပ်ပါတယ်။ Java မှာပါတဲ့ Basic Data Type တွေဖြစ်ကြတဲ့ Primitive Wrapper တွေနဲ့ String, Date, Calendar အစရှိတဲ့ Type တွေဟာ ရေးသားထားစဉ်ကထဲက Comparable Interface ကို Implement လုပ်ပြီးသားဖြစ်တဲ့ အတွက် TreeSet ထဲမှာ ဒီအတိုင်း အသုံးပြုနိုင်ပေမဲ့ မိမိကိုယ်တိုင်ရေးသားထားတဲ့ Class တွေဆိုရင်တော့ Comparable Interface ကို Implement လုပ်ထားမှသာ TreeSet ထဲမှာ ရေးသား အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

```
	public class User implements Comparable<User>{
		private String name;

		@Override
		public int compareTo(User o) {
			return name.compareTo(o.name);
		}

		// constructor, getter, setter & toString() method
	}
```

အထက်ပါအတိုင်း User Class ကို Comparable Interface ကို Implement လုပ်လိုက်ပါမယ်။ အဲ့ဒါကြောင့် compareTo() Method ကို Override လုပ်ပြီးရေးပေးဖို့ လိုအပ်လာပါတယ်။

ဒါ့ကြောင့် User Class ရဲ့ compareTo() Method မှာ User o ကို Argument အနေနဲ့ ယူထားပြီး ဒါကတော့ လက်ရှိ Object နဲ့ လာပြီး Compare လုပ်မည့် အခြား User Object ဖြစ်ပါတယ်။ Compare Method တွေရဲ့ Return Type ဟာ Integer ဖြစ်ပြီး တူနေရင် 0 ကို Return ပြန်ပြီး ကိုယ်က ကြီးရင် အပေါင်းတန်ဖိုး Integer ကို Return ပြန်ပြီး ကိုယ်ကငယ်နေရင် အနုတ်တန်ဖိုး Return ပြန်ရမှာဖြစ်ပါတယ်။ နမူနာထဲမှာတော့ Current Object ရဲ့ name နဲ့ o.name ကို Compare လုပ်လို့ရတဲ့ တန်ဖိုးကို Return ပြန်စေပါတယ်။

ဒါ့ကြောင့် User Object ချင်း Compare လုပ်လာရင် name နဲ့ Compare လုပ်လို့ရတဲ့ Result ကို ရရှိမှာ ဖြစ်ပါတယ်။ အထက်ပါအတိုင်း User Class ကို ပြင်ဆင်ရေးသားပြီးတဲ့ နောက်မှာ ရှေ့မှာရေးသားခဲ့တဲ့ Main Method ကို ပြန်ပြီး Run ကြည့်တဲ့ အခါ အောက်ပါအတိုင်း အဖြေကို ရရှိမှာ ဖြစ်ပါတယ်။

```
[Aung Aung, Thidar]
```

Comparable Interface ကို Implement လုပ်လိုက်တဲ့ အတွက် ClassCastException မဖြစ်တော့ပါဘူး။ ပြီးတော့လဲ Aung Aung ကို Thidar ရဲ့ နောက်မှာ Add လုပ်ပေမဲ့၊ TreeSet တွေဟာ Sort လုပ်ပေးနိုင်တဲ့ အတွက် Aung Aung ဟာ ရှေ့မှာ ရောက်နေတာ ဖြစ်ပါတယ်။

List Interface

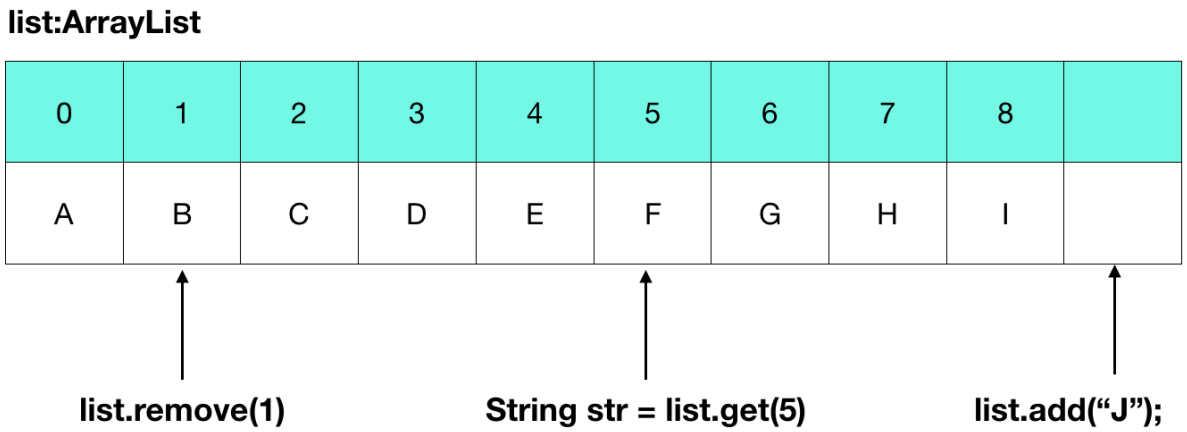
List တွေဟာ Set တွေလိုပဲ Collection Interface ရဲ့ Sub Interface တစ်ခုဖြစ်ပါတယ်။ ဒါ့ကြောင့် Collection ထဲမှာပါတဲ့ Method တွေကိုလဲ List တွေမှာ အသုံးပြုနိုင်ပါတယ်။ တဖန် List တွေဟာ Unlimitted Array တွေကို ကိုယ်စားပြုတဲ့ အတွက် Element တွေကို Index တွေနဲ့ သိမ်းပေးထားနိုင်ပြီး Index တွေကို အသုံးပြုပြီး ရေးသားနိုင်တဲ့ Method တွေပိုပါလာပါတယ်။ List တွေမှာ ပိုပါလာတဲ့ Method တွေကတော့အောက်ပါအတိုင်းဖြစ်ပါတယ်။

Method	Return Type	Description
get (int index)	E	Index မှာရှိတဲ့ Element ကိုပြန်လည်ရယူနိုင်
set (int index, E element)	void	Index ကို သတ်မှတ်ပြီး Element ကို Set လုပ်နိုင်
add (int index, E element)	void	Index ကို သတ်မှတ်ပြီး Element ကို Add လုပ်နိုင်
remove(int index)	E	Index ကို သတ်မှတ်ပြီး Element ကို Remove လုပ်နိုင်
addAll(int index, Collection<? extends E> c)	boolean	Index ကို သတ်မှတ်ပြီး Collection ထဲမှာရှိတဲ့ Element တွေအားလုံးကို ဖြည့်စွက်နိုင်
indexOf(Object o)	Int	ရှေ့ဆုံးမှာရှိတဲ့ Element ရဲ့ Index ကို ပြန်လည်ရယူနိုင်
lastIndexOf(Object o)	Int	နောက်ဆုံးမှာရှိတဲ့ Element ရဲ့ Index ကို ပြန်လည်ရယူနိုင်
listIterator()	ListIterator<E>	List ထဲမှာရှိတဲ့ Element တွေအားလုံးကို ListIterator Object အနေနဲ့ ပြန်လည်ရယူနိုင်
listIterator(int index)	ListIterator<E>	Index Number ကနေစပြီး နောက်ဆုံးနေရာအထိ List ထဲမှာရှိတဲ့ Element တွေကို ListIterator Object အနေနဲ့ ပြန်လည်ရယူနိုင်
subList(int from, int to)	List<E>	အစနဲ့ အဆုံး Index တွေကိုပေးပြီး အခြား List Object တစ်ခုကို ပြန်လည်ရယူနိုင်

Index Number တွေကို အသုံးပြုပြီး List ထဲက Element တွေကို Access လုပ်တဲ့ အခါမှာ မရှိတဲ့ Index တွေကို သွားသုံးမိရင် IndexOutOfBoundsException ကို ဖြစ်ပေါ်စေမှာဖြစ်တာကို သတိပြုစေလိုပါတယ်။

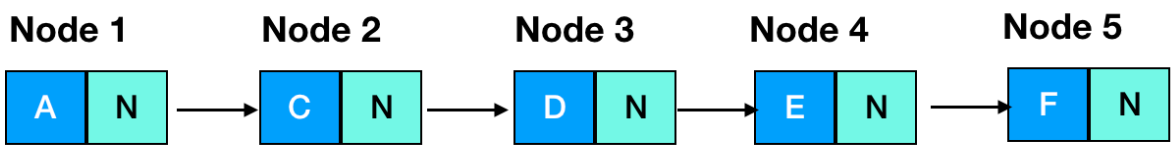
Class	Data Structure	ထူးခြားချက်များ
ArrayList	Unlimited Array	Index နဲ့ရှာဖွေတာ အသစ်ဖြည့်စွက်တာကောင်းမွန်ပါတယ် ကြားဖြတ်ပြီး ဖြည့်စွက်တာ၊ ကြားဖြတ်ပြီး ဖျက်ထုတ်တာ တာတော့ မကောင်းပါဘူး
LinkedList	Link List	Memory ပေါ်မှာ နေရာယူတာများပါတယ် ကြားဖြတ်ပြီး ဖြည့်စွက်တာ၊ ကြားဖြတ်ပြီး ဖျက်ထုတ်တာ တာတော့ ကောင်းမွန်ပါတယ်

List Interface Object ကိုတည်ဆောက်ဖို့အတွက် ArrayList နဲ့ LinkedList Class တို့ကို အသုံးပြုနိုင်ပါတယ်။ ArrayList ကတော့ Unlimited Array Data Structure ကို အသုံးပြုထားပါတယ်။ Unlimited Array တွေဟာ Array အမျိုးအစားဖြစ်တဲ့ အတွက် Element တွေကို ထည့်သွင်းထားတဲ့ Index Number တွေနဲ့သိမ်းပေးထားနိုင်ပါတယ်။ Array တွေကတော့ အသုံးပြုမယ်ဆိုရင် အသုံးပြုမည့် Element အရေအတွက်ကို ကြိုတင် သတ်မှတ်ထားဖို့ လိုအပ်ပေမဲ့ Unlimited Array တွေကတော့ ကြိုက်သလောက် Element တွေကို ဖြည့်စွက်နိုင်ပါတယ်။



ArrayList တွေဟာ Unlimited Array ဖြစ်တဲ့ အတွက် Index ကို အသုံးပြုပြီး Element တွေကို ရှာဖွေတဲ့ နေရာမှာ အလွန်ကောင်းမွန်ပါတယ်။ အထက်ပါပုံထဲကလို list.add("J") လို့ အသစ်ဖြည့်စွက်မယ်ဆိုရင် နောက်ဆုံးနေရာမှာ အခန်းတစ်ခန်းတိုးပြီး အဲ့ဒီနေရာမှာ "J" ကို ဖြည့်စွက်ရုံပါပဲ။ ဒီလိုပဲ Element တွေကို Index နဲ့ သိမ်းထားတာ ဖြစ်တဲ့ အတွက် list.get(5) လို့ ပြန်ရှာမယ်ဆိုရင်လဲ အလွယ်တကူရှာဖွေနိုင်မှာ ဖြစ်ပါတယ်။

ဒါပေမဲ့ list.remove(1) လို့ Index 1 မှာရှိတဲ့ Element ကို Delete လုပ်မယ်ဆိုရင်တော့ Index 2 ကနေ အဆုံးအထိ Index Number တွေကို ပြန်ပြောင်းပြီးလိုတဲ့ နေရာမျိုးတွေမှာ Performance ကို ကျဆင်းစေနိုင်ပါတယ်။ ဒါကြောင့် ArrayList Object တွေကို ကြားဖြတ်ပြီး ဖျက်ထုတ်တာတို့၊ ကြားဖြတ်ပြီး ဖြည့်စွက်ဖို့လိုအပ်တဲ့ Operation တွေမှာ သိပ်ပြီး မသုံးသင့်ပါဘူး။



LinkedList Class တွေကတော့ Link List Data Structure ကို အသုံးပြုပါတယ်။ Link List တွေမှာ Element တွေကို Node တွေနဲ့သိမ်းဆည်းထားကြပါတယ်။ Node တစ်ခုမှာ Element Value နဲ့ Next Node ဟာ ဘယ် Node လဲဆိုတာကို သတ်မှတ်ပေးနိုင်တဲ့ တန်ဖိုးတစ်ခုတို့နဲ့ ဖွဲ့စည်းထားပါတယ်။ ဒါကြောင့် LinkedList တွေဟာ

Array List တွေနဲ့ နှိုင်းယှဉ်မယ်ဆိုရင် Memory ပေါ်မှာ နေရာယူပိုယူလေ့ရှိပါတယ်။ ဒါပေမဲ့ ကြားဖြတ်ပြီး ဖြတ်တောက်တဲ့နေရာတွေ၊ ကြားဖြတ်ပြီး ဖြည့်စွက်တဲ့ Operation တွေမှာတော့ Performance ကောင်းမွန်ပါတယ်။ ဆိုကြပါစို့ Node 2 ကိုဖျက်ထုတ်ပစ်ချင်တယ်ဆိုရင် Node 1 ရဲ့ Next Element နေရာမှာ Node 3 ကို ပြောင်းချိတ်ပေးလိုက်ရုံပါပဲ။ အခြား Node တွေကို ဘာမှလုပ်စရာမလိုအပ်ပါဘူး။ ဒီလိုပဲ Node 2 နဲ့ Node 3 အကြားမှာ Node တစ်ခုကို ဖြည့်စွက်ချင်တယ်ဆိုရင်လဲ Node 2 ရဲ့ Next နေရာမှာ Node အသစ်ကို သတ်မှတ်ပြီး၊ Node အသစ်ရဲ့ Next နေရာမှာ Node 3 ကို သတ်မှတ်လိုက်ရုံပါပဲ။ ဒါကြောင့် List ရဲ့ Element တွေကို ခဏခဏ ကြားဖြတ်ပြီး ဖြည့်စွက်တာတို့ ဖျက်ပစ်တာတို့ များတဲ့ Operation တွေမှာ LinkedList ကို အသုံးပြုသင့်ပါတယ်။

```
// create list object
List<Integer> list = new ArrayList<>();

// [10]
list.add(10);

// [10, 4]
list.add(4);

// [10, 5, 4]
list.add(1, 5);

// [10, 2, 4]
list.set(1, 2);

// [2, 4]
list.remove(0);
```

List Object တစ်ခုကိုတည်ဆောက်တော့မယ်ဆိုရင် မိမိရဲ့ Application ရဲ့ လိုအပ်ချက်အလိုက် ArrayList နဲ့ LinkedList တို့မှတဆင့်တည်ဆောက်နိုင်ပါတယ်။

List Object တွေမှာ add() method နှစ်မျိုးပါရှိပါတယ်။ ပထမ add() method ကတော့ Element ကိုယူပြီး List ထဲကို ထည့်ပေးနိုင်တဲ့ Method ဖြစ်ပါတယ်။ အထက်ဖော်ပြပါအတိုင်း list.add(10) လို့ရေးလိုက်ပြီးတဲ့ နောက်မှာ List ထဲက နောက်ဆုံး Index နေရာကနေ တစ်ခုတိုးပြီး Element ကို ဖြည့်စွက်ပေးမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် add(E) Method ကို အသုံးပြုရင် ဖြည့်စွက်လိုက်တဲ့ Element ဟာ နောက်ဆုံးနေရာမှာ ရောက်ရှိသွားမှာ ဖြစ်ပါတယ်။ နောက် Method တစ်ခုကတော့ add(int, E) ပါ။ Argument နှစ်ခုပါရှိပြီး၊ ပထမ Argument ကတော့ Add လုပ်မည့် Index Number ဖြစ်ပြီး နောက် Argument ကတော့ ဖြည့်စွက်မည့် Element ဖြစ်ပါတယ်။ အသစ်ဖြည့်စွက်လိုက်တဲ့ Element က Index နေရာမှာ ရောက်ရှိသွားပြီး၊ အရင် Index မှာ ရှိနေတဲ့ Element ကတော့ နောက်ကို ရွှေ့သွားမှာ ဖြစ်ပါတယ်။ ဒီလိုပဲ နောက်မှာရှိတဲ့ Element တွေလဲ နောက်ကို Index တစ်ခုဆီဆုတ်သွားမှာ ဖြစ်ပါတယ်။ ဘယ်လိုပဲ ဖြစ်ဖြစ် add() method ကို အသုံးပြုရင် Element တစ်ခုတော့ တိုးသွားမှာ ဖြစ်ပါတယ်။

set(int, E) Method ကတော့ Index Number မှာရှိနေတဲ့ Element နဲ့ Argument မှာပါလာတဲ့ Element နဲ့ လဲပေးနိုင်တဲ့ Method တစ်ခုဖြစ်ပါတယ်။ ဒါကြောင့် နဂို List ထဲမှာ [10, 5, 4] ရှိနေတဲ့ နေရာမှာ list.set(1, 2) ကို Evaluate လုပ်ပြီးတဲ့ အခါမှာ [10, 2, 4] လို့ Index 1 နေရာကို တန်ဖိုး 2 က အစားထိုးသွားတာကို တွေ့ရပါမယ်။ တဖန် Index Number ကိုပေးပြီးလဲ remove လုပ်လို့ရပါသေးတယ်။ list.remove(0) ကို Evaluate လုပ်လိုက်ရင်တော့ Index 1 မှာရှိတဲ့ 10 ကို ဖျက်ထုတ်ပစ်ပြီး [2, 4] ပဲ List ထဲမှာ ကျန်ပါတော့မယ်။

တဖန် List ထဲကို အခြားသော Collection တွေဖြည့်စွက်နိုင်အောင် Method တွေကိုလဲ ပြင်ဆင်ပေးထားပါတယ်။

```
List<String> list = new ArrayList<>();

list.add("Hello");

Set<String> set = new HashSet<>();
set.add("Java");

list.addAll(set);

// [Hello, Java]
System.out.println(list);

list.addAll(1, list);

// [Hello, Hello, Java, Java]
System.out.println(list);
```

Utility Classes for Collection

Java IO

Java IO ဆိုတာဟာ Java Program ထဲကနေ Inputs Outputs တွေကို လုပ်နိုင်အောင် ကြိုရေးပေးထားတဲ့ API ပဲ ဖြစ်တယ်။ ဒါဆိုရင် IO ကို သိအောင် ဘာတွေလေ့လာထားဖို့လိုမှာလဲ။ File System ကိုနာလည်ထာဖို့တော့ လိုပါလိမ့်မယ်။ ပြီးမှ API ကို သုံးပြီး Input Output ကို ဘယ်လိုလုပ်မယ်ဆိုတာရယ် ဘယ်လို အရာမျိုးတွေကို Input လုပ်လို့ရမယ်ဆိုတာရယ်၊ အသုံးပြုနိုင်တဲ့ Interface Object တွေကို သိရှိထားရပါမယ်။

Java SE ရဲ့ java.io package ရဲ့ အောက်မှာ IO အတွက် API Class တွေ Interfaces တွေကို စုစည်းထားပါတယ်။ ကျွန်တော်တို့ Java IO အခန်းနဲ့ အောက်ပါ အကြောင်းအရာများကို လေ့လာသွားပါမယ်။

FILE OBJECT

File System ထဲမှာ ရှိတဲ့ File တွေ Directory တွေရဲ့ Path Name တွေကို ကိုယ်စားပြုရန်အတွက် java.io.File Class ကို ပြင်ဆင်ထားပါတယ်။ File Object ကနေ File Directory တို့ရဲ့ Properties တွေကို သိရှိနိုင်သလို၊ File Directory တွေကို တည်ဆောက်နိုင်ပါတယ်။ တဖန် Directory ထဲမှာရှိတဲ့ File တွေကို ရှာဖွေတာတွေ၊ File တွေကို နာမည်ပြောင်းတာတွေ၊ တနေရာက တနေရာကို ပြောင်ရွှေ့တာတွေ၊ ပြီးတော့ ဖျက်ပစ်တာတွေ လုပ်နိုင်ပါတယ်။

I/O STREAMS

File ထဲမှာရှိတဲ့ Data တွေကို Read / Write လုပ်ဖို့အတွက်ပြင်ဆင်ထားတဲ့ Object တွေဖြစ်ပါတယ်။ Read / Write လုပ်မည့် Data Type အလိုက် အသုံးပြုရမည့် I/O Stream အမျိုးအစားတွေဟာ မတူညီပါဘူး။ ဘယ်လို Data အမျိုးအစားဆိုရင် ဘယ် Stream ကို သုံးမလဲ ဆိုတာတွေကို အသေအချာ ခွဲခြားသိနေဖို့လိုအပ်ပါတယ်။ နမူနာတွေနဲ့ I/O Stream တွေကို သဘောပေါက်အောင် လေ့လာသွားကြပါမယ်။

SERIALIZATION AND DESERIALIZATION

JVM ရဲ့ Memory ပေါ်မှာရှိတဲ့ Object တွေကို တခြားတနေရာရာမှာ သွားပြီး သိမ်းတာကို Serialize လုပ်တယ်လို့ခေါ်ပါတယ်။တဖန် Serialized လုပ်ထားတဲ့ Object တွေကို JVM ရဲ့ Memory ပေါ်ကို ပြန်ပြီး ခေါ်ယူ အသုံးပြုတာကို Deserialize လုပ်တယ်လို့ခေါ်ပြီး Deserialize လုပ်တာကို ဘယ်လို လုပ်တာလဲ။ ပြီးတော့ သတိထားစရာ ဘာတွေရှိလဲဆိုတာကို လေ့လာသွားပါမယ်။

java.io.File

Java IO API ဟာ JDK version 1.0 ကထဲကပါဝင်လာခဲ့တဲ့ Input Output ကို လုပ်ဆောင်ဖို့အတွက် API တစ်ခု ဖြစ်ပါတယ်။ File System ထဲမှာ ရှိတဲ့ File တွေ Directory တွေရဲ့ Path ကို ကိုယ်စားပြုဖို့အတွက် java.io.File Class ကို ပြင်ဆင်ထားပါတယ်။ File Class ဟာ Serializable နဲ့ Comparable<File> ကို Implement လုပ်ထားတဲ့ Class တစ်ခုဖြစ်ပါတယ်။

How To Create File Object

အရင်ဦးဆုံး File Object တစ်ခုကို ဘယ်လို တည်ဆောက်မလဲ ဆိုတာကနေစပြီး လေ့လာကြည့်ကြရအောင်။ File Object ကို တည်ဆောက်ဖို့ အတွက် Constructors တွေကို ပြင်ဆင်ထားပါတယ်။ File Object ကို တည်ဆောက်တဲ့ နေရာမှာ လက်ရှိရှိပြီးသား File Directory တွေကိုကိုယ်စားပြုတဲ့ Object တွေကိုလဲ တည်ဆောက်နိုင်သလို၊ လက်တွေ့မှာ မရှိသော်လည်း Virtually တည်ဆောက်ထားလို့ရပါတယ်။ File Object ကို တည်ဆောက်ပြီး လိုအပ်လာတဲ့အခါကျမှ Create လုပ်လို့လဲ ရနိုင်ပါတယ်။ ဒါဆို File Class ထဲမှာပါတဲ့ Constructors တွေကို လေ့လာကြည့်ကြရအောင်။

Constructors	Descriptions
File(File parent, String child)	Parent ဖြစ်တဲ့ File Object တစ်ခုနဲ့ Child Path Name တို့ဖြင့် File Object ကို တည်ဆောက်နိုင်တဲ့ Constructor ဖြစ်ပါတယ်
File(String pathname)	File Directory ရဲ့ Path String ကနေ File Object ကို တည်ဆောက်နိုင်တဲ့ Constructor ဖြစ်ပါတယ်။ Path အနေနဲ့ Absolute Path ကော Relative Path ကို ပါ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်
File(String parent, String child)	File Directory တို့ရှိနေတဲ့ Directory Path နဲ့ File Name တို့ကနေ File Object ကို တည်ဆောက်နိုင်တဲ့ Constructor ဖြစ်ပါတယ်။
File(URI uri)	File Directory တို့ကိုဖေါ်ပြနိုင်တဲ့ URI Object ကနေတဆင့် File Object ကို တည်ဆောက်နိုင်တဲ့ Costructor ဖြစ်ပါတယ်။

File Class ရဲ့ Constructor တွေကို လေ့လာကြည့်ရင် File တစ်ခုကို တည်ဆောက်ဖို့အတွက်က အဲ့ဒီ File ရဲ့ Path ကို ဖေါ်ပြပေးနိုင်ဖို့လိုတာကို တွေ့ရပါမယ်။ အဲ့ဒီ Path ကို ဖေါ်ပြတဲ့နေရာမှာ Absoulte Path နဲ့လည်း ဖေါ်ပြနိုင်သလို Relative Path နဲ့ လည်းဖေါ်ပြနိုင်မှာ ဖြစ်ပါတယ်။

```
    // from file path
    File file1 = new File("folder1");

    // from parent file object and file path
    File file2 = new File(file1, "file2.txt");

    // from parent file path and child file path
    File file3 = new File("folder1", "file2.txt");

    // from URI
    URI uri = new URI("c:\\tmp\\names.txt");
    File file = new File(uri);
```

Create & Delete Files

File Object ကနေ File Directory တွေကို တည်ဆောက်နိုင်ဖို့၊ ပြီးတော့ ရှိပြီးသား File တွေကို ဖျက်နိုင်ဖို့အတွက် အောက်ပါ Methods တွေကို အသုံးပြုနိုင်ပါတယ်။

Methods	Return Type and Descriptions
createNewFile()	boolean File Object မှာ သတ်မှတ်ထားခဲ့တဲ့ Abstract Path Name ဖြင့် Empty New File တစ်ခုကို တည်ဆောက်ပေးပါလိမ့်မယ်။ တကယ်လို့ Path အတိုင်း File ဟာရှိနေမယ်ဆိုရင် ဆောက်ပေးနိုင်မှာ မဟုတ်ပါဘူး။ File ကို တည်ဆောက်နိုင်ခဲ့ရင် true ကို Return လုပ်မှာ ဖြစ်ပြီး မဆောက်နိုင်ခဲ့ပါက false ကို Return ပြန်မှာ ဖြစ်ပါတယ်။ IOException ကို ဖြစ်ပေါ်စေနိုင်တဲ့အတွက် Handle လုပ်ပေးဖို့လိုအပ်ပါတယ်။
mkdir()	boolean File Object မှာ သတ်မှတ်ထားတဲ့ Abstract Path Name ဖြင့် Directory ကို တည်ဆောက်ပေးမှာ ဖြစ်ပါတယ်။ တည်ဆောက်နိုင်ခဲ့ရင် true ကို Return ပြန်မှာ ဖြစ်ပြီး မတည်ဆောက်နိုင်ခဲ့ပါက false ကို Return ပြန်မှာ ဖြစ်ပါတယ်
mkdirs()	boolean File Object မှာ သတ်မှတ်ထားတဲ့ Abstract Path Name ဖြင့် Directory ကို တည်ဆောက်ပေးမှာ ဖြစ်ပါတယ်။ Child Directories တွေပါလာရင်တော့ ဒီ Method ကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။ တည်ဆောက်နိုင်ခဲ့ရင် true ကို Return ပြန်မှာ ဖြစ်ပြီး မတည်ဆောက်နိုင်ခဲ့ပါက false ကို Return ပြန်မှာ ဖြစ်ပါတယ်
delete()	boolean File Object ဖြင့် ကိုယ်စားပြုသော File Directory ကို delete လုပ်လိုတဲ့အခါမှာ အသုံးပြုနိုင်ပါတယ်။ အောင်မြင်စွာ ပြီးဆုံးပါက true ကို Return ပြန်မှာ ဖြစ်ပြီး မဖျက်နိုင်ခဲ့ပါက false ကို Return ပြန်မှာ ဖြစ်ပါတယ်။
deleteOnExit()	void File Object ကို JVM ကနေ Application ကို Run တာပြီးတဲ့ အချိန်မှာ Delete လုပ်ပေးမှာ ဖြစ်ပါတယ်။ Return Type တော့မရှိပါဘူး။

```
File file = new File("Text.txt");
boolean result1 = file.createNewFile();

System.out.println(result1 ? "Success" : "Fails");

File dir1 = new File("outer");
File dir2 = new File(dir1, “inner");

dir1.mkdirs();

boolean result2 = dir1.delete();
System.out.println(result2 ? "Success" : "Fails");
```

အထက်ပါ နမူနာထဲမှာ outer ဆိုတဲ့ Directory ထဲမှာ inner ဆိုတဲ့ Directory ရှိနေတဲ့ အတွက် mkdirs method ကို အသုံးပြုနေတာဖြစ်ပါတယ်။

Methods Related to Path Info

Path လမ်းကြောင်းနဲ့ပတ်သက်ပြီး အောက်ပါ Method များကို အသုံးပြုနိုင်ပါတယ်။

Methods	Return Type and Descriptions
getAbsoluteFile()	File Absolute Path ပုံစံနဲ့ တည်ဆောက်ထားတဲ့ File Object ကို ရရှိမှာ ဖြစ်ပါတယ်။
getAbsolutePath()	String File Object ရဲ့ Absolute Path ကို ရရှိမှာ ဖြစ်ပါတယ်
getCanonicalFile()	File Canonical ပုံစံဖြင့်တည်ဆောက်ထားတဲ့ File Object ကို ရရှိမှာ ဖြစ်ပါတယ်
getCanonicalPath()	String Absolute and Unique ဖြစ်တဲ့ Canonical File Path ကို ရရှိမှာ ဖြစ်ပါတယ်
getName()	String File Name ကို ရရှိမှာ ဖြစ်ပါတယ်။
getParent()	String Parent Directory ရဲ့ Name ကို ရရှိမှာ ဖြစ်ပါတယ်။ တကယ်လို့ သတ်မှတ်ထားခြင်း မရှိပါက null ကို ပြန်ရမှာ ဖြစ်ပါတယ်။
getParentFile()	File Parent Directory ရဲ့ File Object ကို ပြန်ရမှာဖြစ်ပါတယ်။ သတ်မှတ်ထားခြင်းမရှိပါက null ကို ပြန်ရမှာ ဖြစ်ပါတယ်။
getPath()	String Absolute Path Name ကို Return အနေနဲ့ ပြန်လည် ရရှိမှာ ဖြစ်ပါတယ်။
isAbsolute()	boolean File Object ကို သတ်မှတ်ထားတဲ့ File Name ဟာ Absolute Path ဟုတ်မဟုတ်ကို စစ်ဆေးနိုင်တဲ့ Method ဖြစ်ပါတယ်။
renameTo(File dest)	boolean လက်ရှိ File Object ကို Parameter ထဲက File အဖြစ်ပြောင်းလဲနိုင်တဲ့ Method တစ်ခု ဖြစ်ပါတယ်။ File အမည် ပြောင်းတဲ့ နေရာမှာလဲ အသုံးဝင်သလို File တစ်ခုကနေ တစ်ခုကို ပြောင်းရွှေ့တဲ့နေရာမှာလဲ အသုံးဝင်ပါတယ်။

Move File

```
File f1 = new File("file1.txt");
f1.createNewFile();

File f2 = new File("dir");
f2.mkdir();

File f3 = new File(f2, "file2.txt");
f1.renameTo(f3);
```

Methods related to Meta Data

Methods	Return Type and Descriptions
isDirectory()	boolean Directory တစ်ခု ဟုတ်မဟုတ်ကို သိရှိစေနိုင်သော Method ဖြစ်ပါတယ်
isFile()	boolean File တစ်ခု ဟုတ်မဟုတ်ကို သိရှိစေနိုင်သော Method ဖြစ်ပါတယ်
isHidden()	boolean Hidden File တစ်ခု ဟုတ်မဟုတ်ကို သိရှိစေနိုင်သော Method ဖြစ်ပါတယ်
lastModified()	long နောက်ဆုံး Modified လုပ်သွားသော အချိန်ကို သိရှိစေနိုင်သော Method ဖြစ်ပါတယ်
getTotalSpace()	long File Name နဲ့သတ်မှတ်ထားတဲ့ Partition ရဲ့ ပမာဏကို သိရှိစေနိုင်သော Method ဖြစ်ပါတယ်
getFreeSpace()	long File Name နဲ့သတ်မှတ်ထားတဲ့ Partition ထဲမှာရှိတဲ့ Free Space ကို သိရှိစေနိုင်သော Method ဖြစ်ပါတယ်
getUsableSpace()	long File Name နဲ့သတ်မှတ်ထားတဲ့ Partition ထဲမှာရှိတဲ့ အသုံးပြုနိုင်သော ပမာဏကို သိရှိစေနိုင်သော Method ဖြစ်ပါတယ်
setLastModified(long time)	boolean နောက်ဆုံး Modified လုပ်တဲ့အချိန်ကို သတ်မှတ်ပေးနိုင်သော Method ဖြစ်ပါတယ်
setExcutable(boolean executable)	boolean Program တစ်ခုအနေနဲ့ Run လို့ရမရကို သတ်မှတ်ပေးနိုင်သော Method ဖြစ်ပါတယ်
setExcutable(boolean executable, boolean ownerOnly)	boolean Program တစ်ခုအနေနဲ့ Run လို့ရမရကို File Owner တစ်ယောက်ထဲ လုပ်လို့ရတာလား ဆိုတဲ့ အချက်ကိုပါ သတ်မှတ်ပေးနိုင်သော Method ဖြစ်ပါတယ်
setReadable(boolean readable)	boolean File ကို Read လုပ်လို့ရမရကိုသတ်မှတ်ပေးနိုင်သော Methodဖြစ်ပါတယ်
setReadable(boolean readable, boolean ownerOnly)	boolean File ကို Read လုပ်လို့ရမရကို File Owner တစ်ယောက်ထဲ လုပ်လို့ရတာလား ဆိုတဲ့ အချက်ကိုပါ သတ်မှတ်ပေးနိုင်သော Method ဖြစ်ပါတယ်
setReadOnly()	boolean Read Only File အဖြစ်သတ်မှတ်ပေးနိုင်သော Method ဖြစ်ပါတယ်။
setWritable(boolean writable)	boolean Write လုပ်လို့ရမရကို သတ်မှတ်ပေးနိုင်သော Method ဖြစ်ပါတယ်။
setWritable(boolean writable, boolean ownerOnly)	boolean Write လုပ်လို့ရမရကို File Owner တစ်ယောက်ထဲ လုပ်လို့ရတာလား ဆိုတဲ့ အချက်ကိုပါ သတ်မှတ်ပေးနိုင်သော Method ဖြစ်ပါတယ်

Methods for Directory

အောက်ပါ Method တွေကတော့ File Object ဟာ Directory တစ်ခုဖြစ်မှသာ အသုံးပြုနိုင်မည့် Method တွေ ဖြစ်ပါတယ်။

Methods	Return Type and Descriptions
list()	String[] လက်ရှိ Directory အောက်မှာရှိတဲ့ File တွေ Directory တွေရဲ့ အမည်ကို String [] အနေနဲ့ ပြန်လည် ရရှိမှာ ဖြစ်ပါတယ်။
list(FileNameFilter filter)	String[] လက်ရှိ Directory အောက်မှာရှိတဲ့ File တွေရဲ့ Directory တွေရဲ့ အမည်ကို FileNameFilter Interface Object ကို သုံးပြီး Filter လုပ်ပြီး ပြန်လည်ရရှိနိုင်မှာ ဖြစ်ပါတယ်
listFiles()	File[] လက်ရှိ Directory အောက်မှာရှိတဲ့ File တွေ Directory တွေကို File[] အနေနဲ့ ပြန်လည် ရရှိမှာ ဖြစ်ပါတယ်။
listFiles(FileNameFilter filter)	File[] လက်ရှိ Directory အောက်မှာရှိတဲ့ File တွေ Directory တွေကို FileNameFilter Interface Object ကို သုံးပြီး Filter လုပ်ပြီး ပြန်လည်ရရှိနိုင်မှာ ဖြစ်ပါတယ်
listFiles(FileFilter filter)	File[] လက်ရှိ Directory အောက်မှာရှိတဲ့ File တွေ Directory တွေကို FileFilter Interface Object ကို သုံးပြီး Filter လုပ်ပြီး ပြန်လည်ရရှိနိုင်မှာ ဖြစ်ပါတယ်

Example

ဒီတစ်ခေါက်တော့ Directory ထဲမှာရှိတဲ့ File တွေကို နာမည်တစ်ခုနဲ့ ရှာပြီး ဖျက်တဲ့ နမူနာတစ်ခု ရေးကြည့်ပါမယ်။ Maven ကို သုံးပြီး Library တွေကို Download လုပ်တဲ့အခါမှာ Connection မကောင်းရင် LastUpdate ဆိုတဲ့ နာမည်နဲ့ File တွေကို ချန်ထားပြီး ရပ်သွားတတ်ပါတယ်။ အဲဒီ File တွေကို ရှာပြီး Delete လုပ်ပေးနိုင်တဲ့ နမူနာ တစ်ခုကို ရေးကြည့်ပါမယ်။

```
public class MavenDelete {

    public static void main(String[] args) {
        String userHome = System.getProperty("user.home");
        File maven = new File(userHome, ".m2");
        File mavenRepo = new File(maven, "repository");
        deleteFiles(mavenRepo);
    }

    static void deleteFiles(File file) {
        if(file.isDirectory()) {
            File [] files = file.listFiles(f -> f.isDirectory() || f.getName().endsWith("lastUpdate"));
            for(File f : files) {
                deleteFiles(f);
            }
        } else {
            file.delete();
        }
    }
}
```

အထက်ပါ နမူနာထဲမှာတော့ mavenRepo file ကို deleteFiles method ကို သုံးပြီး ဖျက်ခိုင်းပါတယ်။ တကယ်လို့ file Object ဟာ directory ဖြစ်ရင် အဲ့ဒီ Directory ထဲက Directory တွေနဲ့ lastUpdate နဲ့ ဆုံးရင် ရယူလာအောင် FileFilter နေရာကို Lambda Expression နဲ့ Filter လုပ်ထားပါတယ်။ရလာတဲ့ File တွေကို Loop ပတ်ပြီး deleteFile ကို ပြန်ခေါ်ခိုင်းပါတယ်။ တကယ်လို့ Directory ဆိုရင် အောက်မှာ ရှိတဲ့ File တွေကို ရှာပြီး deleteFile ကို ပြန်ခေါ်ပါလိမ့်မယ်။ Directory မဟုတ်ပဲ File တွေဖြစ်နေလို့ကတော့ lastUpdate နဲ့ ဆုံးနေမှာ ဖြစ်တဲ့အတွက် delete လုပ်သွားပါမယ်။

Switch to Other Types

Methods	Return Type and Descriptions
toPath()	Path Java NiO2 ရဲ့ Path Object အဖြစ်ပြောင်းပေးနိုင်တဲ့ Method ဖြစ်ပါတယ်
toStirng()	String File Object ရဲ့ Path Name ကို ရရှိနိုင်မှာ ဖြစ်ပါတယ်။ getPath method နှင့် ရလဒ်ခြင်း အတူတူဖြစ်ပါတယ်
toURI()	URI File Path Name ကို ကိုယ်စားပြုသော URI Object ကို ပြန်လည်ရရှိမှာ ဖြစ်ပါတယ်

Conclusion

File Object ဟာ Java IO ကို အသုံးပြုရာမှာ မရှိမဖြစ်လိုအပ်သော Object တစ်ခုဖြစ်ပါတယ်။ လက်ရှိ File တွေ Directory တွေရဲ့ Abstract Path Name ကို ကိုယ်စားပြုတဲ့ Object တစ်ခုဖြစ်ပါတယ်။ File Object တစ်ခုကို ဘယ်လို Create လုပ်မယ်။ File တွေ Directory တွေကို ဘယ်လို တည်ဆောက်မယ်၊ ပြီးရင် Delete လုပ်မယ်။ Directory ထဲမှာရှိတဲ့ File တွေကို ဘယ်လို ရှာဖွေမယ်ဆိုတာကို သိရှိထားအောင် လေ့လာထားဖို့ လိုအပ်ပါမယ်။

IO Streams

အရင်တစ်ခေါက် File အကြောင်းရေးသားခဲ့စဉ်က File တွေ Directory တွေကို Create လုပ်နိုင်တယ်။ ပြီးတော့လဲ Delete လုပ်နိုင်တယ်။ အမည်ပြောင်းတာတို့လဲ လုပ်နိုင်ခဲ့ပါတယ်။ ဒါပေမဲ့ အဲဒီ File တွေထဲက Data တွေကို Read လုပ်တာတို့ Write လုပ်တာတို့ကို မလုပ်နိုင်ခဲ့ပါဘူး။

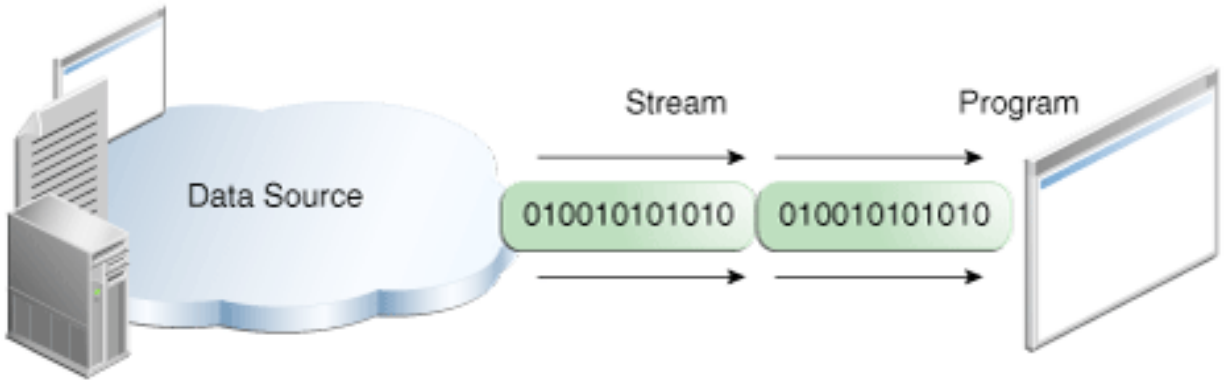
File ထဲက Data တွေကို Input Output လုပ်နိုင်ဖို့အတွက် I/O Stream တွေကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။ Java Language မှာ I/O Stream တွေကို အသုံးပြုမည့် Data Type အပေါ်မူတည်ပြီး အမျိုးမျိုးပြင်ဆင်ထားပါတယ်။ ပြီးတော့ I/O Stream တွေကို ရေးသားတဲ့နေရာမှာလဲ Design Pattern တွေကို အသုံးပြုရေးသားထားပါတယ်။ အရင်ဆုံး I/O Stream တွေကို လေ့လာတဲ့နေရာမှာ အသုံးပြုနိုင်တဲ့ I/O Stream တွေနဲ့ အသုံးပြုပုံတွေကို လေ့လာပြီး၊ နောက်ပိုင်းမှ I/O Stream တွေမှာ အသုံးပြုထားတဲ့ Design Patterns တွေကို လေ့လာသွားပါမယ်။

What are streams?

Stream တွေဟာ Java Program ထဲမှာအသုံးပြုမည့် Input Source တွေနဲ့ Output Destination တွေကို ကိုယ်စားပြုတဲ့ Interface Object တွေဖြစ်ကြပါတယ်။ Source တွေကော Destination တွေပါ ပုံစံအမျိုးမျိုးရှိနိုင်ကြပြီး Input Output လုပ်ကြမည့် Data တွေလဲအမျိုးမျိုးဖြစ်နိုင်ပါတယ်။

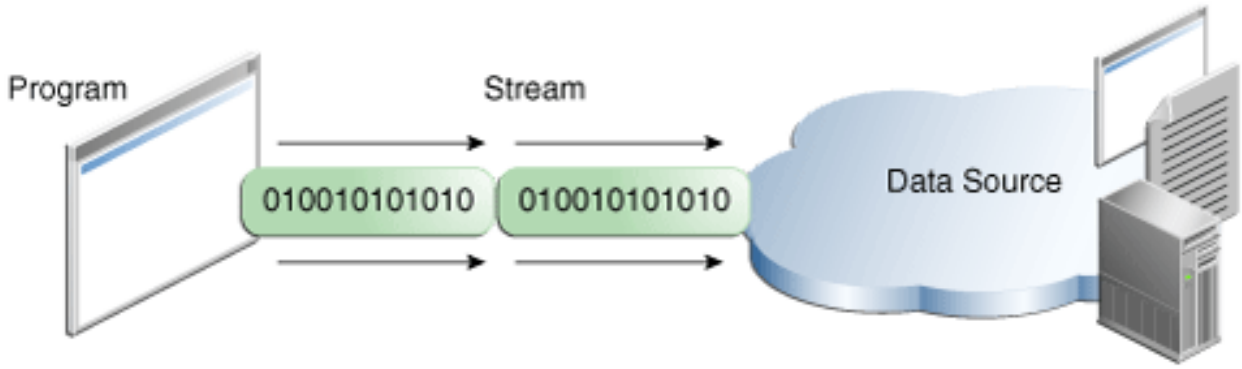
Input Source လိုပြောတဲ့နေရာမှာ File တစ်ခုလဲဖြစ်နိုင်သလို System Console ကလာတဲ့ Input လဲ ဖြစ်နေနိုင်ပါတယ်။ ဒါမှမဟုတ် အခြားသော Program တစ်ခုခုလဲ ဖြစ်နိုင်ပါတယ်။ ဒီလိုပဲ Output Destination လို့ပြောတဲ့နေရာမှာလဲ File တစ်ခု၊ System Console ဒါမှမဟုတ် အခြားသော Program တစ်ခုခုလဲ ဖြစ်နေနိုင်ပါတယ်။ ဒါပေမဲ့ အခြေခံအားဖြင့်တော့ I/O Stream တွေကို Input Stream တွေနဲ့ Output Streams တွေဆိုပြီး သတ်မှတ်ထားပါတယ်။

Input Streams



Java Program ထဲကအသုံးပြုရန် Data များအား Source တစ်နေရာရာမှ ဖတ်ယူနိုင်ဖို့အတွက် Input Stream များကို အသုံးပြုရပါတယ်။ အသုံးပြုလိုတဲ့ Data Type အပေါ်မူတည်ပြီး Streams အမျိုးမျိုးကိုလဲ ပြင်ဆင်ထားပါတယ်။ အသုံးပြုနိုင်တဲ့ Data Type တွေကတော့ bytes, primitive type data, localize character များနဲ့ Object တွေပဲ ဖြစ်ကြပါတယ်။

Output Streams



Java Program ထဲကနေ Destination တစ်ခုဆီကို Data များကို Output လုပ်ချင်တဲ့အခါ Output Stream တွေကို အသုံးပြုနိုင်ပါတယ်။ Destination အနေနဲ့ File တစ်ခုခုဖြစ်နိုင်သလို၊ System Console လဲ ဖြစ်နိုင်သလို အခြားသော Program တစ်ခုခုလဲ ဖြစ်နိုင်ပါတယ်။ ပြီးတော့ byte အနေနဲ့သော်၎င်း၊ primitive data type အနေနဲ့သော်၎င်း၊ localize character များအနေနဲ့သော်၎င်း၊ Object များအနေနဲ့သော်၎င်း Outputလုပ်နိုင်ပါတယ်။

Input Stream တုန်းကလိုပဲ အသုံးပြုလိုတဲ့ Data Type တွေအလိုက် Stream တွေကို ရွေးချယ် အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Data Types and Streams

Java IO ရဲ့ Stream Class တွေအားလုံးဟာ InputStream နဲ့ OutputStream Class တွေကနေ ဆင်းသက်လာကြတဲ့ Class တွေဖြစ်ပါတယ်။ Input Output အနေနဲ့ အသုံးပြုလိုတဲ့ Data Type အလိုက် Stream တွေကို ခွဲခြားအသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Data Type	Input Streams	Output Stream
Byte	FileInputStream	FileOutputStream
Unicode Characters	FileReader	FileWriter
Line Oriented IO	BufferedReader	PrintWriter
Primitive Data	DataInputStream	DataOutputStream
Object	ObjectInputStream	ObjectOutputStream

Byte Streams

File တစ်ခုထဲက Data တွေကို Binary Data တွေဖြစ်တဲ့ 8-bit bytes ပုံစံနဲ့ Read Write လုပ်လိုတဲ့အခါ Byte Streams တွေကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။ Byte Stream တွေအားလုံးဟာ InputStream နဲ့ OutputStream Class တွေရဲ့ Sub Class တွေဖြစ်ကြပြီး အသုံးများတာကတော့ FileInputStream နဲ့ FileOutputStream Class တို့ပဲ ဖြစ်ကြပါတယ်။

FileInputStream ကတော့ File တစ်ခုကနေ Byte တွေကို Read လုပ်ချင်တဲ့အခါမှာ အသုံးပြုနိုင်တဲ့ Stream တစ်ခုဖြစ်ပြီး၊ FileInputStream Object ကို တည်ဆောက်ရန် Constructor ၃ မျိုးပြင်ဆင်ထားပါတယ်။

Constructor	Description
FileInputStream(File file)	Input လုပ်ရမည့် File ကို ကိုယ်စားပြုတဲ့ File Object ကို Argument အနေနဲ့ အသုံးပြုရပါမယ်
FileInputStream(FileDescription fdObj)	Input လုပ်မည့် File ကို ဖေါ်ပြပေးနိုင်တဲ့ FileDescriptor Object ကို Argument အနေနဲ့ အသုံးပြုရပါမယ်
FileInputStream(String name)	Input လုပ်မည့် File ရဲ့ အမည် String Object ကို Argument အနေနဲ့ အသုံးပြုရပါမယ်

FileInputStream Object ကနေ byte တွေကို 1 byte ချင်း Read လုပ်လိုတဲ့အခါ read() ဆိုတဲ့ Method ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

FileOutputStream ကတော့ File တစ်ခုဆီကို Byte တွေ အနေနဲ့ Output လုပ်ပေးနိုင်တဲ့ Stream Object တစ်ခု ဖြစ်ပါတယ်။ FileOutputStream Object ကို တည်ဆောက်ဖို့အတွက် ပြင်ဆင်ထားတဲ့ Constructor တွေကတော့ အောက်ပါ အတိုင်းဖြစ်ကြပါတယ်။

Constructor	Description
FileOutputStream(File file)	Output လုပ်ရမည့် File ကို ကိုယ်စားပြုတဲ့ File Object ကို Argument အနေနဲ့ အသုံးပြုရပါမယ်
FileOutputStream(File file, boolean append)	Output လုပ်ရမည့် File ကို ကိုယ်စားပြုတဲ့ File Object နဲ့ boolean တန်ဖိုးတစ်ခုကို Argument အနေနဲ့ ပေးရပါမယ်။ boolean ရဲ့ တန်ဖိုးဟာ true ဖြစ်မယ်ဆိုရင် လက်ရှိရှိပြီးသား File မှာ Output လုပ်လာတဲ့ Data တွေကို ဆက်ပြီးထည့်ပေးနိုင်မှာ ဖြစ်ပါတယ်
FileOutputStream(FileDescription fdObj)	Output လုပ်မည့် File ကို ဖေါ်ပြပေးနိုင်တဲ့ FileDescriptor Object ကို Argument အနေနဲ့ အသုံးပြုရပါမယ်
FileOutputStream(String name)	Output လုပ်မည့် File ရဲ့ အမည် String Object ကို Argument အနေနဲ့ အသုံးပြုရပါမယ်
FileOutputStream(String name, boolean append)	Output လုပ်မည့် File ရဲ့ အမည်ကို String Object နဲ့ boolean တန်ဖိုးတစ်ခုကို Argument အနေနဲ့ ပေးရပါမယ်။ boolean ရဲ့ တန်ဖိုးဟာ true ဖြစ်မယ်ဆိုရင် လက်ရှိရှိပြီးသား File မှာ Output လုပ်လာတဲ့ Data တွေကို ဆက်ပြီးထည့်ပေးနိုင်မှာ ဖြစ်ပါတယ်

Output လုပ်လိုတဲ့ Byte Data တွေကို ရေးသားနိုင်ဖို့အတွက် write(int b), write(byte [] b) နဲ့ write(byte [] b, int off, int len) method တို့ကို ပြင်ဆင်ထားပါတယ်။ 1 byte ချင်း Write လုပ်ချင်ရင်တော့ write(int b) ကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။

Stream Object တွေဟာ Resource တွေကို အသုံးပြုနေတဲ့ Object အမျိုးအစား ဖြစ်တဲ့အတွက် အသုံးပြုပြီးတဲ့ အခါမှာ ပြန်ပြီး ပိတ်ပေးဖို့လိုအပ်တဲ့အတွက် close() ဆိုတဲ့ Method ကိုလဲ ပြင်ဆင်ပေးထားပါတယ်။ ဒါပေမဲ့ Java SE 7 ရဲ့ ရေးသားပုံအသစ်ဖြစ်တဲ့ try-with-resource ရေးသားပုံနဲ့ ရေးသားမယ်ဆိုရင်တော့ အလိုအလျောက် ပိတ်ပေးနိုင်မှာ ဖြစ်ပါတယ်။

```java
import java.io.*;

public class CopyByte {

    public static void main(String[] args) {

        try(FileInputStream in = new FileInputStream("from.txt");
            FileOutputStream out = new FileOutputStream("to.txt")) {

            int i = 0;

            while((i = in.read()) != -1) {
                out.write(i);
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
    }

}
```

အထက်ပါနမူနာကတော့ from.txt ဆိုတဲ့ File ထဲမှာရှိတဲ့ Data တွေကို 1 byte ချင်း read လုပ်ပြီး to.txt ဆိုတဲ့ဖိုင်ထဲကို 1 byte ချင်း write လုပ်နေတဲ့ နမူနာဖြစ်ပါတယ်။

FileInputStream နဲ့ FileOutputStream Object တွေကို try with resource နဲ့ ရေးသားထားတဲ့အတွက် နောက်ဆုံးမှာ အလိုအလျောက်ပိတ်ပေးသွားမှာ ဖြစ်တယ်။ အဲ့ဒီအတွက် တကူးတက close လုပ်ပေးစရာမလိုတော့ ပါဘူး။ တဖန် while clause ထဲမှာ i = in.read() ဆိုပြီး FileInputStream ကနေ 1 byte ချင်း Read လုပ်နေပါတယ်။ နောက်ဆုံး Read လုပ်စရာမရှိတော့တဲ့အခါ -1 ကို Return ပြန်မှာ ဖြစ်ပြီး While loop ထဲကနေထွက်သွားမှာ ဖြစ်ပါတယ်။

While Loop ထဲမှာတော့ Read လုပ်လာတဲ့ တန်ဖိုးကို 1 byte ချင်း FileOutputStream ကို သုံးပြီး write လုပ်ပေးနေပါတယ်။ ဤနည်းအားဖြင့် from.txt ဖိုင်ထဲက Data တွေကို to.txt ဖိုင်ထဲကို 1 byte ချင်း Write လုပ်ပေးသွားနိုင်မှာ ဖြစ်ပါတယ်။

Character Streams

Text File တစ်ခုအတွင်းရှိ Data များကို Character ပုံစံနဲ့ Read / Write လုပ်ချင်ရင် Character Streams များကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။ Character Streams တွေဟာ InputStreamReader နဲ့ OutputStreamWriter တို့ရဲ့ Sub Class တွေ ဖြစ်ကြပြီး FileReader နဲ့ FileWriter တို့ဟာ အသုံးများပါတယ်။

FileReader နဲ့ FileWriter တို့ဟာ Default Encoding နဲ့ Character တွေကို အလွယ်တကူ Read / Write လုပ်နိုင်အောင်ပြင်ဆင်ထားတဲ့ Class တွေဖြစ်ကြပြီး၊ Character Set တို့ကို ကိုယ်တိုင် သတ်မှတ်ပြီး Read / Write လုပ်ချင်ရင်တော့ InputStreamReader နဲ့ OutputStreamWriter တို့ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

FileReader ကို Character များကို Read လုပ်တဲ့နေရာမှာ အသုံးပြုပြီး ရေးသားနိုင်တဲ့ Constructor များကတော့ အောက်ပါ အတိုင်းဖြစ်ကြပါတယ်။

Constructor	Description
FileReader(File file)	Input လုပ်ရမည့် File ကို ကိုယ်စားပြုတဲ့ File Object ကို Argument အနေနဲ့ အသုံးပြုရပါမယ်
FileReader(FileDescription fdObj)	Input လုပ်မည့် File ကို ဖော်ပြပေးနိုင်တဲ့ FileDescriptor Object ကို Argument အနေနဲ့ အသုံးပြုရပါမယ်
FileReader(String name)	Input လုပ်မည့် File ရဲ့ အမည် String Object ကို Argument အနေနဲ့ အသုံးပြုရပါမယ်

FileWriter ကို Character များကို Write လုပ်တဲ့နေရာမှာ အသုံးပြုပြီး ရေးသားနိုင်တဲ့ Constructor များကတော့ အောက်ပါ အတိုင်းဖြစ်ကြပါတယ်။

Constructor	Description
FileWriter(File file)	Output လုပ်ရမည့် File ကို ကိုယ်စားပြုတဲ့ File Object ကို Argument အနေနဲ့ အသုံးပြုရပါမယ်
FileWriter(File file, boolean append)	Output လုပ်ရမည့် File ကို ကိုယ်စားပြုတဲ့ File Object နဲ့ boolean တန်ဖိုးတစ်ခုကို Argument အနေနဲ့ ပေးရပါမယ်။ boolean ရဲ့ တန်ဖိုးဟာ true ဖြစ်မယ်ဆိုရင် လက်ရှိရှိပြီးသား File မှာ Output လုပ်လာတဲ့ Data တွေကို ဆက်ပြီးထည့်ပေးနိုင်မှာ ဖြစ်ပါတယ်
FileWriter(FileDescription fdObj)	Output လုပ်မည့် File ကို ဖော်ပြပေးနိုင်တဲ့ FileDescriptor Object ကို Argument အနေနဲ့ အသုံးပြုရပါမယ်
FileWriter(String name)	Output လုပ်မည့် File ရဲ့ အမည် String Object ကို Argument အနေနဲ့ အသုံးပြုရပါမယ်
FileWriter(String name, boolean append)	Output လုပ်မည့် File ရဲ့ အမည်ကို String Object နဲ့ boolean တန်ဖိုးတစ်ခုကို Argument အနေနဲ့ ပေးရပါမယ်။ boolean ရဲ့ တန်ဖိုးဟာ true ဖြစ်မယ်ဆိုရင် လက်ရှိရှိပြီးသား File မှာ Output လုပ်လာတဲ့ Data တွေကို ဆက်ပြီးထည့်ပေးနိုင်မှာ ဖြစ်ပါတယ်

FileReader နဲ့ FileWriter တို့ရဲ့ အသုံးပြုပုံဟာ အခြေခံအားဖြင့် FileInputStream နဲ့ FileOutputStream တို့နဲ့ အတူတူပဲ ဖြစ်ပါတယ်။

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class CopyChar {

    public static void main(String[] args) {

        try(FileReader in = new FileReader("from.txt");
                FileWriter out = new FileWriter("out.txt")) {

            int c = 0;

            while((c = in.read()) != -1) {
                out.write(c);
            }

        } catch (IOException e) {
            e.printStackTrace();
        }

    }
}
```

FileReader ကို အသုံးပြုပြီး Text File အတွင်းမှ Character များကို Read လုပ်ရာမှာ read() method ကို အသုံးပြုနိုင်မှာဖြစ်ပါတယ်။ Return Type မှာ int ဖြစ်ပြီး Character တစ်လုံးကို ကိုယ်စားပြုမှာ ဖြစ်ပါတယ်။ နောက်ဆုံး Read လုပ်စရာမရှိတော့တဲ့အခါ -1 ကို Return ပြန်မှာဖြစ်ပါတယ်။

Character စာလုံးများကို Write လုပ်လိုတဲ့အခါ FileWriter Object ရဲ့ write(int char) method ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ Write Method ကို အသုံးပြုပြီး Character တစ်လုံးချင်းစီကို Output လုပ်ပေးနိုင်မှာ ဖြစ်ပါတယ်။

အထက်ပါနမူနာထဲမှာတော့ from.txt file ထဲမှ Character များကို while statement ဖြင့် တစ်လုံးစီဖတ်ပြီး out.txt file ထဲကို Character တစ်လုံးစီ Output လုပ်ပေးနိုင်မှာ ဖြစ်ပါတယ်။

Lined Oriented IO

Character I/O ကို အသုံးပြုတဲ့အခါမှာ Character တစ်လုံးခြင်း Read / Write လုပ်တာထက် ဒီထက်ပိုပြီး ကြီးမားတဲ့ Unit နဲ့ Read / Write လုပ်လေ့ရှိပါတယ်။ အသုံးများတာက စာကြောင်း တစ်ကြောင်းစီကို Read / Write လုပ်တဲ့ပုံစံပဲ ဖြစ်ပါတယ်။ Line တစ်ခုချင်းစီကို Read / Write လုပ်ဖို့အတွက် BufferedReader, BufferedWriter နဲ့ PrintWriter တို့ကို အသုံးပြုနိုင်ပါတယ်။

BufferedReader

BufferedReader တွေဟာ Text File တွေကနေ Character တွေကို သတ်မှတ်ထားတဲ့ Buffered Size အလိုက် Character Array အဖြစ်သော်၎င်း၊ စာကြောင်းတစ်ကြောင်းစီလိုက်သော်၎င်း Read လုပ်နိုင်အောင်ပြင်ဆင်ထားတဲ့ Class တွေ ဖြစ်ပါတယ်။

BufferedReader Class ကို Object ဆောက်နိုင်တဲ့ Constructor တွေကတော့ အောက်ပါ အတိုင်းဖြစ်ပါတယ်။

Constructor	Description
BufferedReader(Reader in)	Reader Object တစ်ခုကို Argument အနေနဲ့ပေးရပါမယ်။ Buffer Size ကိုတော့ Default Size အတိုင်း အသုံးပြုသွားမှာ ဖြစ်ပါတယ်။
BufferedReader(Reader in, int size)	Reader Object တစ်ခုနဲ့ Buffer Size ကို Argument အနေနဲ့ ပေးပြီး BufferedReader Object တစ်ခုကို တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။ Buffer Size ကို သတ်မှတ်ပြီး အသုံးပြုလိုပါက ဤ Constructor ကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။

```
public class LineReader {

    public static void main(String[] args) {

        try(BufferedReader in = new BufferedReader(new FileReader("line.txt"))) {

            String line = null;

            while((line = in.readLine()) != null) {
                System.out.println(line);
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
    }

}
```

BufferedReader ကနေ စာကြောင်း တစ်ကြောင်းစီ Read လုပ်လိုပါက readLine() method ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ အထက်ပါ နမူနာမှာတော့ in.readLine() ကို အသုံးပြုပြီး line variable မှာ အစားထိုးနေပါတယ်။ ပြီးတော့ While Statement မှာ line ဟာ null မဖြစ်မခြင်း loop ပတ်နေမှာ ဖြစ်ပြီး စာကြောင်းတွေ ဖတ်စရာကုန်သွားရင် null ကို Return လုပ်မှာ ဖြစ်တဲ့အတွက် looping ထဲကနေ ထွက်သွားပါလိမ့်မယ်။

ဒါ့ကြောင့် line.txt ထဲက စာကြောင်းတွေကို အစ စာကြောင်းကနေ တစ်ကြောင်းချင်း ကုန်သွားသည် အထိ Read လုပ်သွားနိုင်မှာ ဖြစ်ပါတယ်။

PrintWriter

Java Program ထဲကနေ Character တွေကို String အနေနဲ့ Write လုပ်လိုတဲ့အခါမှာ PrintWriter ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ PrintWriter ကို အသုံးပြုပြီး String တွေလို ဒီအတိုင်း Write လုပ်နိုင်သလို၊ Format ချပြီးလဲ Write လုပ်နိုင်ပါတယ်။ ထို့အပြင် Locale အလိုက်သတ်မှတ်ပြီးလဲ Format လုပ်နိုင်မှာ ဖြစ်ပါတယ်။ PrintWriter Object ကို တည်ဆောက်နိုင်တဲ့ Constructor တွေကတော့ အောက်ပါ အတိုင်းဖြစ်ကြပါတယ်။

Constructor	Description
PrintWriter(File file)	Output File ကို သတ်မှတ်ပြီး PrintWriter Object ကို တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။ Auto Flush ကိုတော့ လုပ်ပေးနိုင်မှာ မဟုတ်ပါဘူး
PrintWriter(File file, String csn)	Output File နဲ့ Character Set ကို သတ်မှတ်ပြီး PrintWriter Object ကို တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။ Auto Flush ကိုတော့ လုပ်ပေးနိုင်မှာ မဟုတ်ပါဘူး
PrintWriter(OutputStream out)	OutputStream ကို သတ်မှတ်ပြီး PrintWriter Object ကို တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။ Auto Flush ကိုတော့ လုပ်ပေးနိုင်မှာ မဟုတ်ပါဘူး
PrintWriter(OutputStream out, boolean autoFlush)	OutputStream နဲ့ Auto Flush Mode ကို သတ်မှတ်ပြီး PrintWriter Object ကို တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။
PrintWriter(String fileName)	File Name ကို သတ်မှတ်ပြီး PrintWriter Object ကို တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။ Auto Flush ကိုတော့ လုပ်ပေးနိုင်မှာ မဟုတ်ပါဘူး
PrintWriter(String fileName, String csn)	File Name နဲ့ Character Set ကို သတ်မှတ်ပြီး PrintWriter Object ကို တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။ Auto Flush ကိုတော့ လုပ်ပေးနိုင်မှာ မဟုတ်ပါဘူး
PrintWriter(Writer out)	Writer ကို သတ်မှတ်ပြီး PrintWriter Object ကို တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။ Auto Flush ကိုတော့ လုပ်ပေးနိုင်မှာ မဟုတ်ပါဘူး
PrintWriter(Writer out, boolean autoFlush)	Writer နဲ့ Auto Flush Mode ကို သတ်မှတ်ပြီး PrintWriter Object ကို တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။

```
public class LineWriter {

    private static final String [] ARRAY = {
            "Hello Java",
            "Hello Java IO",
            "Learn java Programming."
    };

    public static void main(String[] args) {

        try(PrintWriter out = new PrintWriter(new FileWriter("line.txt"))) {

            for (String line : ARRAY) {
                out.println(line);
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

String တွေကို Write လုပ်ဖို့အတွက် print(String str) ဒါမှမဟုတ် write(String str) ကို အသုံးပြုနိုင်ပါတယ်။ ဒါအပြင် Line တွေကို Write လုပ်ဖို့အတွက် println(String str) ကို အသုံးပြုနိုင်ပါတယ်။ Format လုပ်ချင်ရင်တော့ printf(String format, String … param) ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Data Streams

Primitive Data နဲ့ String တန်ဖိုး တွေကို Read / Write လုပ်ချင်တဲ့အခါ DataInputStream နဲ့ DataOutputStream တို့ကို အသုံးပြုနိုင်ပါတယ်။

Constructor	Description
DataOutputStream(OutputStream out)	OutputStream Object တစ်ခုကို အသုံးပြုပြီး DataOutputStream ကို တည်ဆောက်ရန် ပြင်ဆင်ထားတဲ့ Constructor ဖြစ်ပါတယ်။
DataInputStream(InputStream in)	InputStream Object တစ်ခုကို အသုံးပြုပြီး DataInputStream ကို တည်ဆောက်ရန် ပြင်ဆင်ထားတဲ့ Constructor ဖြစ်ပါတယ်။

အသုံးပြုလိုတဲ့ Data Type အလိုက် Method တွေကိုလဲ ခွဲခြားပြင်ဆင်ထားပါတယ်။

Data Type	DataInputStream	DataOutputStream
boolean	readBoolean()	writeBoolean(boolean v)
char	readChar()	writeChar(int v)
byte	readByte()	writeByte(int v)
short	readShort()	writeShort(int v)
int	readInt()	writeInt(int v)
long	readLong()	writeLong(long v)
float	readFloat()	writeFloat(float v)
double	readDouble()	writeDouble(doble v)
String	readUTF()	writeUTF(String str)

မိမိအသုံးပြုလိုတဲ့ Data Type အလိုက် သက်ဆိုင်ရာ Method ကို အသုံးပြုပြီး ရေးသားရမှာ ဖြစ်ပါတယ်။

Writing As Data

```
public class DataOutput {

    private static final String name = "Aung Aung";
    private static final int age = 20;
    private static final boolean isMan = true;

    public static void main(String[] args) {

        System.out.println("Starting Output");

        try(DataOutputStream out = new DataOutputStream(new FileOutputStream("data.dat"))) {

            out.writeUTF(name);
            out.writeInt(age);
            out.writeBoolean(isMan);

            System.out.println("End Output");
        } catch (IOException e) {
            System.out.println("Output Error");
        }

    }

}
```

အထက်ပါနမူနာမှာတော့ name variable ဟာ String ဖြစ်တဲ့အတွက် writeUTF ကို အသုံးပြုပြီး၊ age ဟာ int ဖြစ်တဲ့အတွက် writeInt ကို အသုံးပြုပါတယ်။ တဖန် isMan ဆိုတဲ့ Variable ဟာ boolean type ဖြစ်တဲ့အတွက် writeBoolean ကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။

Reading Data

```
public class DataInput {

    public static void main(String[] args) {

        try(DataInputStream in = new DataInputStream(
                new FileInputStream("data.dat"))) {

            System.out.println("Name : " + in.readUTF());
            System.out.println("Age  : " + in.readInt());
            System.out.println("Male : " + in.readBoolean());

        } catch (IOException e) {
            e.printStackTrace();
        }

    }
}
```

အထက်ပါ နမူနာကတော့ DataOutput ဖြင့် Write လုပ်ထားတဲ့ data.dat file ထဲက Data တွေကို ပြန်ပြီး Read လုပ်နေတာဖြစ်ပါတယ်။ String Data တွေကို readUTF ကိုအသုံးပြုပြီး၊ int ကိုတော့ readInt ကို အသုံးပြုပါတယ်။ တဖန် boolean ကိုတော့ readBoolean method ကို အသုံးပြုပါတယ်။

Serialization

အရှေ့တွင်ဖေါ်ပြခဲ့သလို Java ဘာသာရပ်တွင် Byte, Character, Primitive Data များနှင့် String များကို Read / Write လုပ်နိုင်တဲ့အပြင် Object တွေကိုလဲ Object အတိုင်း Read / Write လုပ်နိုင်ပါတယ်။

Java ဘာသာရပ်မှာ JVM Memory အပေါ်မှာရှိတဲ့ Object တွေကို အခြား တစ်နေနေရာကို ပို့ပေးလိုက်ခြင်းကို Serialize လုပ်တယ်လို့ခေါ်ပါတယ်။ တဖန် တစ်ခြားတစ်နေနေရာကနေလာတဲ့ Serialized လုပ်ထားတဲ့ Data တွေကနေ JVM Memory အပေါ်ကို ပြန်ပြီး Object အဖြစ်ပြန်ပြောင်းတာကိုတော့ Deserialize လုပ်တယ်လို့ခေါ်ပါတယ်။ Deseiralize လုပ်တယ်ဆိုတာကလဲ Serialize လုပ်ထားတဲ့ Data များကိုသာ ပြုလုပ်နိုင်မှာ ဖြစ်ပါတယ်။

နမူနာအားဖြင့် Program ထဲမှာ သုံးနေတဲ့ Java Object တွေကို File တစ်ခုခုမှာ သွားသိမ်းတာတို့၊ Internet ကို ဖြတ်ပြီး တစ်ခြားတစ်နေနေရာကိုပို့ပေးတာတို့ဟာ Serialize လုပ်တာဖြစ်ကြပါတယ်။ Server အပေါ်မှာရှိတဲ့ Object တွေကို Session ပေါ်မှာ သုံးနေတယ်ဆိုတာကလဲ Serialize လုပ်နေတာပဲ ဖြစ်ပါတယ်။

ဒါဖြင့် ဘယ်လို Object တွေကို Serialize လုပ်လို့ရတာလဲ။ Primitive Data Type တွေ၊ Serializable Interface ကို Implement လုပ်ထားတဲ့ Object တွေနဲ့ Serializable Object တွေရဲ့ Collection တွေ ဆိုရင် Serialize လုပ်လို့ရမှာ ဖြစ်ပါတယ်။

Object တစ်ခုကို Serialize လုပ်တဲ့အခါမှာ Object ရဲ့ State တွေဖြစ်ကြတဲ့ Instance Variable တွေကိုသာ Serialize လုပ်သွားမှာ ဖြစ်ပါတယ်။ ဒါ့ကြောင့် Serialize လုပ်မည့် Object ရဲ့ Instance Variable တွေဟာလဲ Serializable ဖြစ်ဖို့လိုအပ်ပါတယ်။ တကယ်လို့ Serialize မလုပ်ချင်တဲ့ Instance Variable တွေဆိုရင် transient modifier ကို ရေးသားပြီး ဖေါ်ပြနိုင်ပါတယ်။ Instance Variable တွေရှေ့မှာ transient ကို ရေးသားထားရင် အဲ့ဒီ Field ကို Serialize လုပ်မှာ မဟုတ်တော့ပါဘူး။

```
public class Employee implements Serializable{

    private static final long serialVersionUID = 1L;

    private int id;
    private String name;
    private String phone;

    // getters & setters

}
```

Serial Version UID

Serializable Object တစ်ခုကို Serialize လုပ်ပြီး ပြန်ပြီး Deserialize လုပ်တဲ့အခါ Serial Version UID ကို အသုံးပြုပြီး Class ရဲ့ Version မှန်မမှန်ကို ပြန်ပြီး စစ်ဆေးလေ့ရှိပါတယ်။ Serial Version UID ကို ရေးသားရာမှာ static final long serialVersionUID ဆိုတဲ့ပုံစံနဲ့ ရေးသားပေးရပါတယ်။ Access Modifier ကိုတော့ နှစ်သက်ရာ ရေးသားအသုံးပြုနိုင်ပါတယ်။

Serialize လုပ်တဲ့အခါမှာ Serial Version UID ဟာ Static Variable ဖြစ်ပေမဲ့ ခြွင်းချက်အနေနဲ့ Serialize အလုပ်ခံရမှာ ဖြစ်ပါတယ်။ Deserialize လုပ်တဲ့အခါမှာ လက်ရှိ Class နဲ့ Deserialize လုပ်မည့် Object ရဲ့ Serial Version UID ကို တိုက်စစ်ပြီး မှန်ကန်မှသာ Serialize လုပ်ပေးမှာ ဖြစ်ပါတယ်။ တကယ်လို့ Version နှစ်ခုမတူညီခဲ့ပါက InvalidClassException ကို ဖြစ်ပေါ်စေမှာ ဖြစ်ပါတယ်။

တကယ်လို့ Serializable Class တစ်ခုမှာ Serial Version UID ကို မရေးထားဖူးဆိုရင် Serialize လုပ်တဲ့အခါမှာ JVM ကနေ နှစ်သက်ရာ ID တစ်ခုကို Generate လုပ်ပေးမှာ ဖြစ်ပါတယ်။

ဒါပေမဲ့ Serial Version UID Calculation ဟာ Compiler Implementation အပေါ်မူတည်ပြီး ပြောင်းလဲတတ်တဲ့အတွက် Serial Version UID ကို Class အတွင်းမှာ မဖြစ်မနေရေးသားသင့်တယ်လို့ ညွှန်းဆိုထားပါတယ်။

Serialization

Java Program ထဲကနေ Memory အပေါ်မှာရှိတဲ့ Serializable Object တွေနဲ့ Serializable Object ရဲ့ Collection တွေကို Serialize လုပ်လိုတဲ့အခါမှာ ObjectOutputStream ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ObjectOutputStream ကို အသုံးပြုပြီး Primitive Data တွေနဲ့ Serializable Object Graphs တွေကို Serialize လုပ်ပေးနိုင်မှာ ဖြစ်ပါတယ်။

Object တွေကို Serialize လုပ်တဲ့နေရာမှာလဲ File တွေမှာလဲ Serialize လုပ်နိုင်သလို Network ကို အသုံးပြုတဲ့ Socket Stream တွေနဲ့လဲ တွဲဖက် အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ ObjectOutputStream ရဲ့ Constructor တွေကတော့ အောက်ပါအတိုင်း ဖြစ်ကြပါတယ်။

Constructor	Description
ObjectOutputStream()	ObjectOutputStream ရဲ့ Sub Class တွေအတွက် ပြင်ဆင်ထားပေးတဲ့ Constructor ဖြစ်ပါတယ်။
ObjectOutputStream(OutputStream out)	OutputStream Object တစ်ခုကို အသုံးပြုပြီး ObjectOuputStream ကို တည်ဆောက်ရန် ပြင်ဆင်ထားတဲ့ Constructor ဖြစ်ပါတယ်။

```
public class ObjectWriteTest {

    public static void main(String[] args) {
        Employee e = new Employee();
        e.setId(1);
        e.setName("Aung Aung");
        e.setPhone("09788986677");

        try(ObjectOutputStream out = new ObjectOutputStream(
                                        new FileOutputStream("employee.obj"))) {
            out.writeObject(e);
        } catch (IOException e1) {
            e1.printStackTrace();
        }
    }
}
```

အထက်ပါနမူနာအတိုင်း JVM Memory အပေါ်မှာရှိတဲ့ Object တွေကို Serialize လုပ်လိုပါက writeObject(Object obj) method ကို အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Deserialization

ObjectOutputStream ကို အသုံးပြုပြီး Serialize လုပ်ထားတဲ့ Primitive Data တွေနဲ့ Object တွေကို Deserialize လုပ်လိုတဲ့အခါ ObjectInputStream ကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။

Constructor	Description
ObjectInputStream()	ObjectInputStream ရဲ့ Sub Class တွေအတွက် ပြင်ဆင်ထားပေးတဲ့ Constructor ဖြစ်ပါတယ်။
ObjectInputStream(InputStream in)	OutputStream Object တစ်ခုကို အသုံးပြုပြီး ObjectOuputStream ကို တည်ဆောက်ရန် ပြင်ဆင်ထားတဲ့ Constructor ဖြစ်ပါတယ်။

File ထဲမှာရှိတဲ့ Data တွေကို Deserialize လုပ်တဲ့အခါမျိုးမှာ ObjectInputStream ဟာ FileInputStream ကို အသုံးပြုပါတယ်။ ဒါကြောင့် File ထဲက Data တွေကို Read လုပ်လိုတဲ့အခါမျိုးမှာ InputStream Parameter နေရာမှာ FileInputStream Object ကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။

တဖန် Remote System တစ်ခုကနေ Data တွေကို ရယူတဲ့အခါမျိုးမှာတော့ Socket Streams ကို အသုံးပြုပြီး Data တွေကို Deserialize လုပ်နိုင်ပါတယ်။

```
public class ObjectReadTest {

    public static void main(String[] args) {

        try(ObjectInputStream in = new ObjectInputStream(
                                new FileInputStream("employee.obj"))) {

            Employee e = (Employee) in.readObject();

            System.out.println(e.getName());

        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

ယခင် နမူနာဖြင့် Serialize လုပ်ထားတဲ့ Employee Object ကို Deserialize လုပ်လိုတဲ့အခါ အထက်ပါအတိုင်း ObjectInputStream ကို အသုံးပြုပြီး ရေးသားနိုင်ပါတယ်။ အထက်ပါအတိုင်း in.readObject() method ရဲ့ Return Type ဟာ Object ဖြစ်နေတဲ့အတွက် Employee အဖြစ်ပြန်ပြီး Cast လုပ်ပေးဖို့လိုအပ်ပါတယ်။

NiO2

Java 8 New Features

Lambda Expression

Lambda Expression ဆိုတာဘာလဲ

ဘာ့ကြောင့် **Lambda Expression** ကို လိုအပ်ခဲ့ရတာလဲ

Lambda Expression ကို ဘယ်လို ရေးသားရသလဲ

Lambda Expression ကြောင့် ဘာတွေ ကောင်းသွားလဲ

ဘယ်လိုနေရာတွေမှာ **Lambda Expression** ကို အသုံးပြုသင့်သလဲ

Java ဟာ **Functional** လား **OOP** လား

Method References

Changes in Interfaces

Streams API

Java 9 New Features

JShell

Modularity In Java

Changes in Streams API

HTTP 2 Client